
Глава 4. Синтаксис SQL

В этой главе описывается синтаксис языка SQL. Тем самым закладывается фундамент для следующих глав, где будет подробно рассмотрено, как с помощью команд SQL описывать и изменять данные.

Мы советуем прочитать эту главу и тем, кто уже знаком SQL, так как в ней описываются несколько правил и концепций, которые реализованы в разных базах данных SQL по-разному или относятся только к Postgres Pro.

4.1. Лексическая структура

SQL-программа состоит из последовательности *команд*. Команда, в свою очередь, представляет собой последовательность *компонентов*, оканчивающуюся точкой с запятой («;»). Конец входного потока также считается концом команды. Какие именно компоненты допустимы для конкретной команды, зависит от её синтаксиса.

Компонентом команды может быть *ключевое слово*, *идентификатор*, *идентификатор в кавычках*, *строка* (или константа) или специальный символ. Компоненты обычно разделяются пробельными символами (пробел, табуляция, перевод строки), но это не требуется, если нет неоднозначности (например, когда спецсимвол оказывается рядом с компонентом другого типа).

Например, следующий текст является правильной (синтаксически) SQL-программой:

```
SELECT * FROM MY_TABLE;  
UPDATE MY_TABLE SET A = 5;  
INSERT INTO MY_TABLE VALUES (3, 'hi there');
```

Это последовательность трёх команд, по одной в строке (хотя их можно было разместить и в одну строку или наоборот, разделить команды на несколько строк).

Кроме этого, SQL-программы могут содержать *комментарии*. Они не являются компонентами команд, а по сути равносильны пробельным символам.

Синтаксис SQL не очень строго определяет, какие компоненты идентифицируют команды, а какие — их операнды или параметры. Первые несколько компонентов обычно содержат имя команды, так что в данном примере мы можем говорить о командах «SELECT», «UPDATE» и «INSERT». Но например, команда UPDATE требует, чтобы также в определённом положении всегда стоял компонент SET, а INSERT в приведённом виде требует наличия компонента VALUES. Точные синтаксические правила для каждой команды описаны в [Части VI](#).

4.1.1. Идентификаторы и ключевые слова

Показанные выше команды содержали компоненты SELECT, UPDATE и VALUES, которые являются примерами *ключевых слов*, то есть слов, имеющих фиксированное значение в языке SQL. Компоненты MY_TABLE и A являются примерами *идентификаторов*. Они идентифицируют имена таблиц, столбцов или других объектов баз данных, в зависимости от того, где они используются. Поэтому иногда их называют просто «именами». Ключевые слова и идентификаторы имеют одинаковую лексическую структуру, то есть, не зная языка, нельзя определить, является ли некоторый компонент ключевым словом или идентификатором. Полный список ключевых слов приведён в [Приложении С](#).

Идентификаторы и ключевые слова SQL должны начинаться с буквы (a-z, хотя допускаются также не латинские буквы и буквы с диакритическими знаками) или подчёркивания (_). Последующими символами в идентификаторе или ключевом слове могут быть буквы, цифры (0-9), знаки доллара (\$) или подчёркивания. Заметьте, что строго следуя букве стандарта SQL, знаки доллара нельзя использовать в идентификаторах, так что их использование вредит переносимости приложений. В стандарте SQL гарантированно не будет ключевых слов с цифрами и начинающихся или заканчивающихся подчёркиванием, так что идентификаторы такого вида защищены от возможных конфликтов с будущими расширениями стандарта.

Система выделяет для идентификатора не более NAMEDATALEN-1 байт, а более длинные имена усекаются. По умолчанию NAMEDATALEN равно 64, так что максимальная длина идентификатора равна 63 байтам.

Ключевые слова и идентификаторы без кавычек воспринимаются системой без учёта регистра. Таким образом:

```
UPDATE MY_TABLE SET A = 5;
```

равносильно записи:

```
uPDaTE my_Table SeT a = 5;
```

Часто используется неформальное соглашение записывать ключевые слова заглавными буквами, а имена строчными, например:

```
UPDATE my_table SET a = 5;
```

Есть и другой тип идентификаторов: *отделённые идентификаторы* или *идентификаторы в кавычках*. Они образуются при заключении обычного набора символов в двойные кавычки ("). Такие идентификаторы всегда будут считаться идентификаторами, но не ключевыми словами. Так "select" можно использовать для обозначения столбца или таблицы «select», тогда как select без кавычек будет воспринят как ключевое слово и приведёт к ошибке разбора команды в месте, где ожидается имя таблицы или столбца. Тот же пример можно переписать с идентификаторами в кавычках следующим образом:

```
UPDATE "my_table" SET "a" = 5;
```

Идентификаторы в кавычках могут содержать любые символы, за исключением символа с кодом 0. (Чтобы включить в такой идентификатор кавычки, продублируйте их.) Это позволяет создавать таблицы и столбцы с именами, которые иначе были бы невозможны, например, с пробелами или амперсандами. Ограничение длины при этом сохраняется.

Идентификатор, заключённый в кавычки, становится зависимым от регистра, тогда как идентификаторы без кавычек всегда переводятся в нижний регистр. Например, идентификаторы FOO, foo и "foo" считаются одинаковыми в Postgres Pro, но "F00" и "FOO" отличны друг от друга и от предыдущих трёх. (Приведение имён без кавычек к нижнему регистру, как это делает Postgres Pro, несовместимо со стандартом SQL, который говорит о том, что имена должны приводиться к верхнему регистру. То есть, согласно стандарту foo должно быть эквивалентно "FOO", а не "foo". Поэтому при создании переносимых приложений рекомендуется либо всегда заключать определённое имя в кавычки, либо не заключать никогда.)

Ещё один вариант идентификаторов в кавычках позволяет использовать символы Unicode по их кодам. Такой идентификатор начинается с U& (строчная или заглавная U и амперсанд), а затем сразу без пробелов идёт двойная кавычка, например U&"foo". (Заметьте, что при этом возникает неоднозначность с оператором &. Чтобы её избежать, окружайте этот оператор пробелами.) Затем в кавычках можно записывать символы Unicode двумя способами: обратная косая черта, а за ней код символа из четырёх шестнадцатеричных цифр, либо обратная косая черта, знак плюс, а затем код из шести шестнадцатеричных цифр. Например, идентификатор "data" можно записать так:

```
U&"d\0061t\+000061"
```

В следующем менее тривиальном примере закодировано русское слово «слон», записанное кириллицей:

```
U&"\0441\043B\043E\043D"
```

Если вы хотите использовать не обратную косую черту, а другой спецсимвол, его можно указать, добавив UESCAPE после строки, например:

```
U&"d!0061t!+000061" UESCAPE '!'
```

В качестве спецсимвола можно выбрать любой символ, кроме шестнадцатеричной цифры, знака плюс, апострофа, кавычки или пробельного символа. Заметьте, что спецсимвол заключается не в двойные кавычки, а в апострофы, после UESCAPE.

Чтобы сделать спецсимволом знак апострофа, напишите его дважды.

Записывать суррогатные пары UTF-16 и таким образом составлять символы с кодами больше чем U+FFFF можно либо в четырёх-, либо в шестизначной форме, хотя наличие шестизначной формы технически делает это ненужным. (Суррогатные пары не сохраняются непосредственно, а объединяются в один символ, который затем кодируется в UTF-8.)

Когда кодировка сервера — не UTF-8, символ с кодом, указанным этой спецпоследовательностью, преобразуется в фактическую кодировку сервера; если такое преобразование невозможно, выдаётся ошибка.

4.1.2. Константы

В Postgres Pro есть три типа констант *подразумеваемых типов*: строки, битовые строки и числа. Константы можно также записывать, указывая типы явно, что позволяет представить их более точно и обработать более эффективно. Эти варианты рассматриваются в следующих подразделах.

4.1.2.1. Строковые константы

Строковая константа в SQL — это обычная последовательность символов, заключённая в апострофы ('), например: 'Это строка'. Чтобы включить апостроф в строку, напишите в ней два апострофа рядом, например: 'Жанна д' 'Арк'. Заметьте, это *не* то же самое, что двойная кавычка (").

Две строковые константы, разделённые пробельными символами и *минимум одним переводом строки*, объединяются в одну и обрабатываются, как если бы строка была записана в одной константе. Например:

```
SELECT 'foo'
'bar';
```

эквивалентно:

```
SELECT 'foobar';
```

но эта запись:

```
SELECT 'foo'      'bar';
```

считается синтаксической ошибкой. (Это несколько странное поведение определено в стандарте SQL, Postgres Pro просто следует ему.)

4.1.2.2. Строковые константы со спецпоследовательностями в стиле C

Postgres Pro также принимает «спецпоследовательности», что является расширением стандарта SQL. Строка со спецпоследовательностями начинается с буквы E (заглавной или строчной), стоящей непосредственно перед апострофом, например: E'foo'. (Когда константа со спецпоследовательностью разбивается на несколько строк, букву E нужно поставить только перед первым открывающим апострофом.) Внутри таких строк символ обратной косой черты (\) начинает C-подобные *спецпоследовательности*, в которых сочетание обратной косой черты со следующим символом(ами) даёт определённое байтовое значение, как показано в [Таблице 4.1](#).

Таблица 4.1. Спецпоследовательности

Спецпоследовательность	Интерпретация
\b	символ «забой»
\f	подача формы
\n	новая строка
\r	возврат каретки
\t	табуляция
\o, \oo, \ooo (o = 0-7)	восьмеричное значение байта
\xh, \xhh (h = 0-9, A-F)	шестнадцатеричное значение байта

Спецпоследовательность	Интерпретация
<code>\uxxxx</code> , <code>\Uxxxxxxxx</code> ($x = 0-9, A-F$)	16- или 32-битный шестнадцатеричный код символа Unicode

Любой другой символ, идущий после обратной косой черты, воспринимается буквально. Таким образом, чтобы включить в строку обратную косую черту, нужно написать две косых черты (`\\`). Так же можно включить в строку апостроф, написав `\'`, в дополнение к обычному способу `'`.

Вы должны позаботиться, чтобы байтовые последовательности, которые вы создаёте таким образом, особенно в восьмеричной и шестнадцатеричной записи, образовывали допустимые символы в серверной кодировке. Также может быть полезно использовать спецпоследовательности Unicode или альтернативную запись, описанную в [Подразделе 4.1.2.3](#); в этом случае сервер будет проверять, возможно ли преобразовать указанный символ.

Внимание

Если параметр конфигурации `standard_conforming_strings` имеет значение `off`, Postgres Pro распознаёт обратную косую черту как спецсимвол и в обычных строках, и в строках со спецпоследовательностями. Однако в версии PostgreSQL 9.1 по умолчанию принято значение `on`, и в этом случае обратная косая черта распознаётся только в спецстроках. Это поведение больше соответствует стандарту, хотя может нарушить работу приложений, рассчитанных на предыдущий режим, когда обратная косая черта распознавалась везде. В качестве временного решения вы можете изменить этот параметр на `off`, но лучше уйти от такой практики. Если вам нужно, чтобы обратная косая черта представляла специальный символ, дайте строковую константу с `E`.

В дополнение к `standard_conforming_strings` поведением обратной косой черты в строковых константах управляют параметры `escape_string_warning` и `backslash_quote`.

Строковая константа не может включать символ с кодом 0.

4.1.2.3. Строковые константы со спецпоследовательностями Unicode

Postgres Pro также поддерживает ещё один вариант спецпоследовательностей, позволяющий включать в строки символы Unicode по их кодам. Строковая константа со спецпоследовательностями Unicode начинается с `U&` (строчная или заглавная U и амперсанд), а затем сразу без пробелов идёт апостроф, например `U&'foo'`. (Заметьте, что при этом возникает неоднозначность с оператором `&`. Чтобы её избежать, окружайте этот оператор пробелами.) Затем в апострофах можно записывать символы Unicode двумя способами: обратная косая черта, а за ней код символа из четырёх шестнадцатеричных цифр, либо обратная косая черта, знак плюс, а затем код из шести шестнадцатеричных цифр. Например, строку `'data'` можно записать так:

```
U&'d\0061t\+000061'
```

В следующем менее тривиальном примере закодировано русское слово «слон», записанное кириллицей:

```
U&' \0441\043B\043E\043D'
```

Если вы хотите использовать не обратную косую черту, а другой спецсимвол, его можно указать, добавив `UESCAPE` после строки, например:

```
U&'d!0061t!+000061' UESCAPE '!'
```

В качестве спецсимвола можно выбрать любой символ, кроме шестнадцатеричной цифры, знака плюс, апострофа, кавычки или пробельного символа.

Чтобы включить спецсимвол в строку буквально, напишите его дважды.

Записывать суррогатные пары UTF-16 и таким образом составлять символы с кодами больше чем `U+FFFF` можно либо в четырёх-, либо в шестизначной форме, хотя наличие шестизначной формы

технически делает это ненужным. (Суррогатные пары не сохраняются непосредственно, а объединяются в один символ, который затем кодируется в UTF-8.)

Когда кодировка сервера — не UTF-8, символ с кодом, указанным этой спецпоследовательностью, преобразуется в фактическую кодировку сервера; если такое преобразование невозможно, выдаётся ошибка.

Также заметьте, что спецпоследовательности Unicode в строковых константах работают, только когда параметр конфигурации [standard_conforming_strings](#) равен `on`. Это объясняется тем, что иначе клиентские программы, проверяющие SQL-операторы, можно будет ввести в заблуждение и эксплуатировать это как уязвимость, например, для SQL-инъекций. Если этот параметр имеет значение `off`, эти спецпоследовательности будут вызывать ошибку.

4.1.2.4. Строковые константы, заключённые в доллары

Хотя стандартный синтаксис для строковых констант обычно достаточно удобен, он может плохо читаться, когда строка содержит много апострофов, так как каждый такой символ приходится дублировать. Чтобы и в таких случаях запросы оставались читаемыми, Postgres Pro предлагает ещё один способ записи строковых констант — «заключение строк в доллары». Строковая константа, заключённая в доллары, начинается со знака доллара (\$), необязательного «тега» из нескольких символов и ещё одного знака доллара, затем содержит обычную последовательность символов, составляющую строку, и оканчивается знаком доллара, тем же тегом и замыкающим знаком доллара. Например, строку «Жанна д'Арк» можно записать в долларах двумя способами:

```
$$Жанна д'Арк$$
$SomeTag$Жанна д'Арк$SomeTag$
```

Заметьте, что внутри такой строки апострофы не нужно записывать особым образом. На самом деле, в строке, заключённой в доллары, все символы можно записывать в чистом виде: содержимое строки всегда записывается буквально. Ни обратная косая черта, ни даже знак доллара не являются спецсимволами, если только они не образуют последовательность, соответствующую открывающему тегу.

Строковые константы в долларах можно вкладывать друг в друга, выбирая на разных уровнях вложенности разные теги. Чаще всего это используется при написании определений функций. Например:

```
$function$
BEGIN
    RETURN ($1 ~ $q$[\t\r\n\v\\]$q$);
END;
$function$
```

Здесь последовательность `$q$[\t\r\n\v\\]$q$` представляет в долларах текстовую строку `[\t\r\n\v\\]`, которая будет обработана, когда Postgres Pro будет выполнять эту функцию. Но так как эта последовательность не соответствует внешнему тегу в долларах (`$function$`), с точки зрения внешней строки это просто обычные символы внутри константы.

Тег строки в долларах, если он присутствует, должен соответствовать правилам, определённым для идентификаторов без кавычек, и к тому же не должен содержать знак доллара. Теги регистрозависимы, так что `$tag$string content$tag$` — правильная строка, а `$TAG$string content$tag$` — нет.

Строка в долларах, следующая за ключевым словом или идентификатором, должна отделяться от него пробельными символами, иначе доллар будет считаться продолжением предыдущего идентификатора.

Заключение строк в доллары не является частью стандарта SQL, но часто это более удобный способ записывать сложные строки, чем стандартный вариант с апострофами. Он особенно полезен, когда нужно представить строковую константу внутри другой строки, что часто требуется в определениях процедурных функций. Ограничившись только апострофами, каждую обратную косую черту в приведённом примере пришлось бы записывать четырьмя такими символами, которые бы

затем уменьшились до двух при разборе внешней строки, и наконец до одного при обработке внутренней строки во время выполнения функции.

4.1.2.5. Битовые строковые константы

Битовые строковые константы похожи на обычные с дополнительной буквой *b* (заглавной или строчной), добавленной непосредственно перед открывающим апострофом (без промежуточных пробелов), например: `b'1001'`. В битовых строковых константах допускаются лишь символы 0 и 1.

Битовые константы могут быть записаны и по-другому, в шестнадцатеричном виде, с начальной буквой *x* (заглавной или строчной), например: `x'1FF'`. Такая запись эквивалентна двоичной, только четыре двоичных цифры заменяются одной шестнадцатеричной.

Обе формы записи допускают перенос строк так же, как и обычные строковые константы. Однако заключать в доллары битовые строки нельзя.

4.1.2.6. Числовые константы

Числовые константы могут быть заданы в следующем общем виде:

```
цифры
цифры. [цифры] [e [+ -] цифры]
[цифры] . цифры [e [+ -] цифры]
цифрые [+ -] цифры
```

где *цифры* — это одна или несколько десятичных цифр (0..9). До или после десятичной точки (при её наличии) должна быть минимум одна цифра. Как минимум одна цифра должна следовать за обозначением экспоненты (*e*), если оно присутствует. В числовой константе не может быть пробелов или других символов. Заметьте, что любой знак минус или плюс в начале строки не считается частью числа; это оператор, применённый к константе.

Несколько примеров допустимых числовых констант:

```
42
3.5
4.
.001
5e2
1.925e-3
```

Числовая константа, не содержащая точки и экспоненты, изначально рассматривается как константа типа `integer`, если её значение уместается в 32-битный тип `integer`; затем как константа типа `bigint`, если её значение уместается в 64-битный `bigint`; в противном случае она принимает тип `numeric`. Константы, содержащие десятичные точки и/или экспоненты, всегда считаются константами типа `numeric`.

Изначально назначенный тип данных числовой константы это только отправная точка для алгоритмов определения типа. В большинстве случаев константа будет автоматически приведена к наиболее подходящему типу для данного контекста. При необходимости вы можете принудительно интерпретировать числовое значение как значение определённого типа, приведя его тип к нужному. Например, вы можете сделать, чтобы числовое значение рассматривалось как имеющее тип `real (float4)`, написав:

```
REAL '1.23' -- строковый стиль
1.23::REAL  -- стиль Postgres Pro (исторический)
```

На самом деле это только частные случаи синтаксиса приведения типов, который будет рассматриваться далее.

4.1.2.7. Константы других типов

Константу *обычного* типа можно ввести одним из следующих способов:

```
type 'string'
```

```
'string'::type
CAST ( 'string' AS type )
```

Текст строковой константы передаётся процедуре преобразования ввода для типа, обозначенного здесь *type*. Результатом становится константа указанного типа. Явное приведение типа можно опустить, если нужный тип константы определяется однозначно (например, когда она присваивается непосредственно столбцу таблицы), так как в этом случае приведение происходит автоматически.

Строковую константу можно записать, используя как обычный синтаксис SQL, так и формат с долларами.

Также можно записать приведение типов, используя синтаксис функций:

```
typename ( 'string' )
```

но это работает не для всех имён типов; подробнее об этом написано в [Подразделе 4.2.9](#).

Конструкцию `::`, `CAST()` и синтаксис вызова функции можно также использовать для преобразования типов обычных выражений во время выполнения, как описано в [Подразделе 4.2.9](#). Во избежание синтаксической неопределённости, запись *тип* 'строка' можно использовать только для указания типа простой текстовой константы. Ещё одно ограничение записи *тип* 'строка': она не работает для массивов; для таких констант следует использовать `::` или `CAST()`.

Синтаксис `CAST()` соответствует SQL, а запись *тип* 'string' является обобщением стандарта: в SQL такой синтаксис поддерживает только некоторые типы данных, но Postgres Pro позволяет использовать его для всех. Синтаксис `::` имеет исторические корни в Postgres Pro, как и запись в виде вызова функции.

4.1.3. Операторы

Имя оператора образует последовательность не более чем `NAMEDATALEN-1` (по умолчанию 63) символов из следующего списка:

```
+ - * / < > = ~ ! @ # % ^ & | ` ?
```

Однако для имён операторов есть ещё несколько ограничений:

- Сочетания символов `--` и `/*` не могут присутствовать в имени оператора, так как они будут обозначать начало комментария.
- Многосимвольное имя оператора не может заканчиваться знаком `+` или `-`, если только оно не содержит также один из этих символов:

```
~ ! @ # % ^ & | ` ?
```

Например, `@-` — допустимое имя оператора, а `*-` — нет. Благодаря этому ограничению, Postgres Pro может разбирать корректные SQL-запросы без пробелов между компонентами.

Записывая нестандартные SQL-операторы, обычно нужно отделять имена соседних операторов пробелами для однозначности. Например, если вы определили префиксный оператор с именем `@`, вы не можете написать `x*@y`, а должны написать `x* @y`, чтобы Postgres Pro однозначно прочитал это как два оператора, а не один.

4.1.4. Специальные знаки

Некоторые не алфавитно-цифровые символы имеют специальное значение, но при этом не являются операторами. Подробнее их использование будет рассмотрено при описании соответствующего элемента синтаксиса. Здесь они упоминаются только для сведения и обобщения их предназначения.

- Знак доллара (`$`), предвещающий число, используется для представления позиционного параметра в теле определения функции или подготовленного оператора. В других контекстах знак доллара может быть частью идентификатора или строковой константы, заключённой в доллары.

- Круглые скобки (`()`) имеют обычное значение и применяются для группировки выражений и повышения приоритета операций. В некоторых случаях скобки — это необходимая часть синтаксиса определённых SQL-команд.
- Квадратные скобки (`[]`) применяются для выделения элементов массива. Подробнее массивы рассматриваются в [Разделе 8.15](#).
- Запятые (`,`) используются в некоторых синтаксических конструкциях для разделения элементов списка.
- Точка с запятой (`;`) завершает команду SQL. Она не может находиться нигде внутри команды, за исключением строковых констант или идентификаторов в кавычках.
- Двоеточие (`:`) применяется для выборки «срезов» массивов (см. [Раздел 8.15](#).) В некоторых диалектах SQL (например, в Embedded SQL) двоеточие может быть префиксом в имени переменной.
- Звёздочка (`*`) используется в некоторых контекстах как обозначение всех полей строки или составного значения. Она также имеет специальное значение, когда используется как аргумент некоторых агрегатных функций, а именно функций, которым не нужны явные параметры.
- Точка (`.`) используется в числовых константах, а также для отделения имён схемы, таблицы и столбца.

4.1.5. Комментарии

Комментарий — это последовательность символов, которая начинается с двух минусов и продолжается до конца строки, например:

```
-- Это стандартный комментарий SQL
```

Кроме этого, блочные комментарии можно записывать в стиле C:

```
/* многострочный комментарий
 * с вложенностью: /* вложенный блок комментария */
 */
```

где комментарий начинается с `/*` и продолжается до соответствующего вхождения `*/`. Блочные комментарии можно вкладывать друг в друга, как разрешено по стандарту SQL (но не разрешено в C), так что вы можете комментировать большие блоки кода, которые при этом уже могут содержать блоки комментариев.

Комментарий удаляется из входного потока в начале синтаксического анализа и фактически заменяется пробелом.

4.1.6. Приоритеты операторов

В [Таблице 4.2](#) показаны приоритеты и очерёдность операторов, действующие в Postgres Pro. Большинство операторов имеют одинаковый приоритет и вычисляются слева направо. Приоритет и очерёдность операторов жёстко фиксированы в синтаксическом анализаторе. Если вы хотите, чтобы выражение с несколькими операторами разбиралось не в том порядке, который диктуют эти приоритеты, добавьте скобки.

Таблица 4.2. Приоритет операторов (от большего к меньшему)

Оператор/элемент	Очерёдность	Описание
<code>.</code>	слева-направо	разделитель имён таблицы и столбца
<code>::</code>	слева-направо	приведение типов в стиле Postgres Pro
<code>[]</code>	слева-направо	выбор элемента массива
<code>+ -</code>	справа-налево	унарный плюс, унарный минус
<code>COLLATE</code>	слева-направо	выбор правила сортировки

Оператор/элемент	Очерёдность	Описание
AT	слева-направо	AT TIME ZONE
^	слева-направо	возведение в степень
* / %	слева-направо	умножение, деление, остаток от деления
+ -	слева-направо	сложение, вычитание
(любой другой оператор)	слева-направо	все другие встроенные и пользовательские операторы
BETWEEN IN LIKE ILIKE SIMILAR		проверка диапазона, проверка членства, сравнение строк
< > = <= >= <>		операторы сравнения
IS ISNULL NOTNULL		IS TRUE, IS FALSE, IS NULL, IS DISTINCT FROM и т. д.
NOT	справа-налево	логическое отрицание
AND	слева-направо	логическая конъюнкция
OR	слева-направо	логическая дизъюнкция

Заметьте, что правила приоритета операторов также применяются к операторам, определённым пользователем с теми же именами, что и вышеперечисленные встроенные операторы. Например, если вы определите оператор «+» для некоторого нестандартного типа данных, он будет иметь тот же приоритет, что и встроенный оператор «+», независимо от того, что он у вас делает.

Когда в конструкции `OPERATOR` используется имя оператора со схемой, например так:

```
SELECT 3 OPERATOR(pg_catalog.+) 4;
```

тогда `OPERATOR` имеет приоритет по умолчанию, соответствующий в [Таблице 4.2](#) строке «любой другой оператор». Это не зависит от того, какие именно операторы находятся в конструкции `OPERATOR()`.

Примечание

В PostgreSQL до версии 9.5 действовали немного другие правила приоритета операторов. В частности, операторы `<=`, `>=` и `<>` обрабатывались по общему правилу; проверки `IS` имели более высокий приоритет; а `NOT BETWEEN` и связанные конструкции работали несогласованно — в некоторых случаях приоритетнее оказывался оператор `NOT`, а не `BETWEEN`. Эти правила были изменены для лучшего соответствия стандарту SQL и для уменьшения путаницы из-за несогласованной обработки логически равнозначных конструкций. В большинстве случаев эти изменения никак не проявятся, либо могут привести к ошибкам типа «нет такого оператора», которые можно разрешить, добавив скобки. Однако возможны особые случаи, когда запрос будет разобран без ошибки, но его поведение может измениться.

4.2. Выражения значения

Выражения значения применяются в самых разных контекстах, например в списке результатов команды `SELECT`, в значениях столбцов в `INSERT` или `UPDATE` или в условиях поиска во многих командах. Результат такого выражения иногда называют *скаляром*, чтобы отличить его от результата табличного выражения (который представляет собой таблицу). А сами выражения значения часто называют *скалярными* (или просто *выражениями*). Синтаксис таких выражений позволяет вычислять значения из примитивных частей, используя арифметические, логические и другие операции.

Выражениями значения являются:

- Константа или непосредственное значение
- Ссылка на столбец
- Ссылка на позиционный параметр в теле определения функции или подготовленного оператора
- Выражение с индексом
- Выражение выбора поля
- Применение оператора
- Вызов функции
- Агрегатное выражение
- Вызов оконной функции
- Приведение типов
- Применение правил сортировки
- Скалярный подзапрос
- Конструктор массива
- Конструктор табличной строки
- Кроме того, выражением значения являются скобки (предназначенные для группировки подвыражений и переопределения приоритета)

В дополнение к этому списку есть ещё несколько конструкций, которые можно классифицировать как выражения, хотя они не соответствуют общим синтаксическим правилам. Они обычно имеют вид функции или оператора и будут рассмотрены в соответствующем разделе [Главы 9](#). Пример такой конструкции — предложение `IS NULL`.

Мы уже обсудили константы в [Подразделе 4.1.2](#). В следующих разделах рассматриваются остальные варианты.

4.2.1. Ссылки на столбцы

Ссылку на столбец можно записать в форме:

`отношение.имя_столбца`

Здесь *отношение* — имя таблицы (возможно, полное, с именем схемы) или её псевдоним, определённый в предложении `FROM`. Это имя и разделяющую точку можно опустить, если имя столбца уникально среди всех таблиц, задействованных в текущем запросе. (См. также [Главу 7](#).)

4.2.2. Позиционные параметры

Ссылка на позиционный параметр применяется для обращения к значению, переданному в SQL-оператор извне. Параметры используются в определениях SQL-функций и подготовленных операторов. Некоторые клиентские библиотеки также поддерживают передачу значений данных отдельно от самой SQL-команды, и в этом случае параметры позволяют ссылаться на такие значения. Ссылка на параметр записывается в следующей форме:

`$число`

Например, рассмотрим следующее определение функции `dept`:

```
CREATE FUNCTION dept(text) RETURNS dept
AS $$ SELECT * FROM dept WHERE name = $1 $$
LANGUAGE SQL;
```

Здесь `$1` всегда будет ссылаться на значение первого аргумента функции.

4.2.3. Индексы элементов

Если в выражении вы имеете дело с массивом, то можно извлечь определённый его элемент, написав:

```
выражение[индекс]
```

или несколько соседних элементов («срез массива»):

```
выражение[нижний_индекс:верхний_индекс]
```

(Здесь квадратные скобки [] должны присутствовать буквально.) Каждый *индекс* сам по себе является выражением, результат которого округляется к ближайшему целому.

В общем случае *выражение* массива должно заключаться в круглые скобки, но их можно опустить, когда выражение с индексом — это просто ссылка на столбец или позиционный параметр. Кроме того, можно соединить несколько индексов, если исходный массив многомерный. Например:

```
моя_таблица.столбец_массив[4]
моя_таблица.столбец_массив_2d[17][34]
$1[10:42]
(функция_массив(a,b))[42]
```

В последней строке круглые скобки необходимы. Подробнее массивы рассматриваются в [Разделе 8.15](#).

4.2.4. Выбор поля

Если результат выражения — значение составного типа (строка таблицы), тогда определённое поле этой строки можно извлечь, написав:

```
выражение.имя_поля
```

В общем случае *выражение* такого типа должно заключаться в круглые скобки, но их можно опустить, когда это ссылка на таблицу или позиционный параметр. Например:

```
моя_таблица.столбец
$1.столбец
(функция_кортеж(a,b)).стол3
```

(Таким образом, полная ссылка на столбец — это просто частный случай выбора поля.) Важный особый случай здесь — извлечение поля из столбца составного типа:

```
(составной_столбец).поле
(моя_таблица.составной_столбец).поле
```

Здесь скобки нужны, чтобы показать, что *составной_столбец* — это имя столбца, а не таблицы, и что *моя_таблица* — имя таблицы, а не схемы.

Вы можете запросить все поля составного значения, написав `.*`:

```
(составной_столбец).*
```

Эта запись действует по-разному в зависимости от контекста; подробнее об этом говорится в [Подразделе 8.16.5](#).

4.2.5. Применение оператора

Существуют два возможных синтаксиса применения операторов:

```
выражение оператор выражение (бинарный инфиксный оператор)
оператор выражение (унарный префиксный оператор)
```

где *оператор* соответствует синтаксическим правилам, описанным в [Подразделе 4.1.3](#), либо это одно из ключевых слов AND, OR и NOT, либо полное имя оператора в форме:

```
OPERATOR(схема.имя_оператора)
```

Существование конкретных операторов и их тип (унарный или бинарный) зависит от того, как и какие операторы определены системой и пользователем. Встроенные операторы описаны в [Главе 9](#).

4.2.6. Вызовы функций

Вызов функции записывается просто как имя функции (возможно, дополненное именем схемы) и список аргументов в скобках:

```
имя_функции ([выражение [, выражение ... ]])
```

Например, так вычисляется квадратный корень из 2:

```
sqrt(2)
```

Список встроенных функций приведён в [Главе 9](#). Пользователь также может определить и другие функции.

Выполняя запросы в базе данных, где одни пользователи могут не доверять другим, в записи вызовов функций соблюдайте меры предосторожности, описанные в [Разделе 10.3](#).

Аргументам могут быть присвоены необязательные имена. Подробнее об этом см. [Раздел 4.3](#).

Примечание

Функцию, принимающую один аргумент составного типа, можно также вызывать, используя синтаксис выбора поля, и наоборот, выбор поля можно записать в функциональном стиле. То есть записи `col(table)` и `table.col` равносильны и взаимозаменяемы. Это поведение не оговорено стандартом SQL, но реализовано в Postgres Pro, так как это позволяет использовать функции для эмуляции «вычисляемых полей». Подробнее это описано в [Подразделе 8.16.5](#).

4.2.7. Агрегатные выражения

Агрегатное выражение представляет собой применение агрегатной функции к строкам, выбранным запросом. Агрегатная функция сводит множество входных значений к одному выходному, например, сумма или среднее. Агрегатное выражение может записываться следующим образом:

```
агрегатная_функция (выражение [ , ... ] [ предложение_order_by ] ) [ FILTER
( WHERE условие_фильтра ) ]
агрегатная_функция (ALL выражение [ , ... ] [ предложение_order_by ] ) [ FILTER
( WHERE условие_фильтра ) ]
агрегатная_функция (DISTINCT выражение [ , ... ] [ предложение_order_by ] ) [ FILTER
( WHERE условие_фильтра ) ]
агрегатная_функция ( * ) [ FILTER ( WHERE условие_фильтра ) ]
агрегатная_функция ( [ выражение [ , ... ] ] ) WITHIN GROUP ( предложение_order_by )
[ FILTER ( WHERE условие_фильтра ) ]
```

Здесь *агрегатная_функция* — имя ранее определённой агрегатной функции (возможно, дополненное именем схемы), *выражение* — любое выражение значения, не содержащее в себе агрегатного выражения или вызова оконной функции. Необязательные предложения *предложение_order_by* и *условие_фильтра* описываются ниже.

В первой форме агрегатного выражения агрегатная функция вызывается для каждой строки. Вторая форма эквивалентна первой, так как указание `ALL` подразумевается по умолчанию. В третьей форме агрегатная функция вызывается для всех различных значений выражения (или набора различных значений, для нескольких выражений), выделенных во входных данных. В четвёртой форме агрегатная функция вызывается для каждой строки, так как никакого конкретного значения не указано (обычно это имеет смысл только для функции `count(*)`). В последней форме используются *сортирующие* агрегатные функции, которые будут описаны ниже.

Большинство агрегатных функций игнорируют значения `NULL`, так что строки, для которых выражения выдают одно или несколько значений `NULL`, отбрасываются. Это можно считать истинным для всех встроенных операторов, если явно не говорится об обратном.

Например, `count (*)` подсчитает общее количество строк, а `count (f1)` только количество строк, в которых `f1` не `NULL` (так как `count` игнорирует `NULL`), а `count (distinct f1)` подсчитает число различных и отличных от `NULL` значений столбца `f1`.

Обычно строки данных передаются агрегатной функции в неопределённом порядке и во многих случаях это не имеет значения, например функция `min` выдаёт один и тот же результат независимо от порядка поступающих данных. Однако некоторые агрегатные функции (такие как `array_agg` и `string_agg`) выдают результаты, зависящие от порядка данных. Для таких агрегатных функций можно добавить *предложение_order_by* и задать нужный порядок. Это *предложение_order_by* имеет тот же синтаксис, что и предложение `ORDER BY` на уровне запроса, как описано в [Разделе 7.5](#), за исключением того, что его выражения должны быть просто выражениями, а не именами результирующих столбцов или числами. Например:

```
SELECT array_agg(a ORDER BY b DESC) FROM table;
```

Заметьте, что при использовании агрегатных функций с несколькими аргументами, предложение `ORDER BY` идёт после всех аргументов. Например, надо писать так:

```
SELECT string_agg(a, ',' ORDER BY a) FROM table;
```

а не так:

```
SELECT string_agg(a ORDER BY a, ',') FROM table; -- неправильно
```

Последний вариант синтаксически допустим, но он представляет собой вызов агрегатной функции одного аргумента с двумя ключами `ORDER BY` (при этом второй не имеет смысла, так как это константа).

Если *предложение_order_by* дополнено указанием `DISTINCT`, тогда все выражения `ORDER BY` должны соответствовать обычным аргументам агрегатной функции; то есть вы не можете сортировать строки по выражению, не включённому в список `DISTINCT`.

Примечание

Возможность указывать и `DISTINCT`, и `ORDER BY` в агрегатной функции — это расширение Postgres Pro.

При добавлении `ORDER BY` в обычный список аргументов агрегатной функции, описанном до этого, выполняется сортировка входных строк для универсальных и статистических агрегатных функций, для которых сортировка необязательна. Но есть подмножество агрегатных функций, *сортирующие агрегатные функции*, для которых *предложение_order* является *обязательным*, обычно потому, что вычисление этой функции имеет смысл только при определённой сортировке входных строк. Типичными примерами сортирующих агрегатных функций являются вычисления ранга и процентиля. Для сортирующей агрегатной функции *предложение_order_by* записывается внутри `WITHIN GROUP (...)`, что иллюстрирует последний пример, приведённый выше. Выражения в *предложении_order_by* вычисляются однократно для каждой входной строки как аргументы обычной агрегатной функции, сортируются в соответствии с требованием *предложения_order_by* и поступают в агрегатную функцию как входящие аргументы. (Если же *предложение_order_by* находится не в `WITHIN GROUP`, оно не передаётся как аргумент(ы) агрегатной функции.) Выражения-аргументы, предшествующие `WITHIN GROUP`, (если они есть), называются *непосредственными аргументами*, а выражения, указанные в *предложении_order_by* — *агрегируемыми аргументами*. В отличие от аргументов обычной агрегатной функции, непосредственные аргументы вычисляются однократно для каждого вызова функции, а не для каждой строки. Это значит, что они могут содержать переменные, только если эти переменные сгруппированы в `GROUP BY`; это суть то же ограничение, что действовало бы, будь эти непосредственные аргументы вне агрегатного выражения. Непосредственные аргументы обычно используются, например, для указания значения процентиля, которое имеет смысл, только если это конкретное число для всего расчёта агрегатной функции. Список непосредственных аргументов может быть пуст; в этом случае запишите просто `()`, но не `(*)`. (На самом деле Postgres Pro примет обе записи, но только первая соответствует стандарту SQL.)

Пример вызова сортирующей агрегатной функции:

```
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY income) FROM households;
percentile_cont
-----
50489
```

она получает 50-й процентиль, или медиану, значения столбца `income` из таблицы `households`. В данном случае 0.5 — это непосредственный аргумент; если бы дробь процентиля менялась от строки к строке, это не имело бы смысла.

Если добавлено предложение `FILTER`, агрегатной функции подаются только те входные строки, для которых `условие_фильтра` вычисляется как истинное; другие строки отбрасываются. Например:

```
SELECT
    count(*) AS unfiltered,
    count(*) FILTER (WHERE i < 5) AS filtered
FROM generate_series(1,10) AS s(i);
unfiltered | filtered
-----+-----
10 | 4
(1 row)
```

Предопределённые агрегатные функции описаны в [Разделе 9.21](#). Пользователь также может определить другие агрегатные функции.

Агрегатное выражение может фигурировать только в списке результатов или в предложении `HAVING` команды `SELECT`. Во всех остальных предложениях, например `WHERE`, они запрещены, так как эти предложения логически вычисляются до того, как формируются результаты агрегатных функций.

Когда агрегатное выражение используется в подзапросе (см. [Подраздел 4.2.11](#) и [Раздел 9.23](#)), оно обычно вычисляется для всех строк подзапроса. Но если в аргументах (или в `условии_filter`) агрегатной функции есть только переменные внешнего уровня, агрегатная функция относится к ближайшему внешнему уровню и вычисляется для всех строк соответствующего запроса. Такое агрегатное выражение в целом является внешней ссылкой для своего подзапроса и на каждом вычислении считается константой. При этом допустимое положение агрегатной функции ограничивается списком результатов и предложением `HAVING` на том уровне запросов, где она находится.

4.2.8. Вызовы оконных функций

Вызов оконной функции представляет собой применение функции, подобной агрегатной, к некоторому набору строк, выбранному запросом. В отличие от вызовов не оконных агрегатных функций, при этом не происходит группировка выбранных строк в одну — каждая строка остаётся отдельной в результате запроса. Однако оконная функция имеет доступ ко всем строкам, вошедшим в группу текущей строки согласно указанию группировки (списку `PARTITION BY`) в вызове оконной функции. Вызов оконной функции может иметь следующие формы:

```
имя_функции ([выражение [, выражение ... ]]) [ FILTER ( WHERE предложение_фильтра ) ]
OVER имя_окна
имя_функции ([выражение [, выражение ... ]]) [ FILTER ( WHERE предложение_фильтра ) ]
OVER ( определение_окна )
имя_функции ( * ) [ FILTER ( WHERE предложение_фильтра ) ] OVER имя_окна
имя_функции ( * ) [ FILTER ( WHERE предложение_фильтра ) ] OVER ( определение_окна )
```

Здесь `определение_окна` записывается в виде

```
[ имя_существующего_окна ]
[ PARTITION BY выражение [, ...] ]
[ ORDER BY выражение [ ASC | DESC | USING оператор ] [ NULLS { FIRST | LAST } ]
[, ...] ]
```


[*определение_рамки*]

Необязательное *определение_рамки* может иметь вид:

{ RANGE | ROWS | GROUPS } *начало_рамки* [*исключение_рамки*]

{ RANGE | ROWS | GROUPS } BETWEEN *начало_рамки* AND *конец_рамки* [*исключение_рамки*]

Здесь *начало_рамки* и *конец_рамки* задаются одним из следующих способов:

UNBOUNDED PRECEDING

смещение PRECEDING

CURRENT ROW

смещение FOLLOWING

UNBOUNDED FOLLOWING

и *исключение_рамки* может быть следующим:

EXCLUDE CURRENT ROW

EXCLUDE GROUP

EXCLUDE TIES

EXCLUDE NO OTHERS

Здесь *выражение* — это любое выражение значения, не содержащее вызовов оконных функций.

имя_окна — ссылка на именованное окно, определённое предложением WINDOW в данном запросе. Также возможно написать в скобках полное *определение_окна*, используя тот же синтаксис определения именованного окна в предложении WINDOW; подробнее это описано в справке по [SELECT](#). Стоит отметить, что запись OVER *имя_окна* не полностью равнозначна OVER (*имя_окна* ...); последний вариант подразумевает копирование и изменение определения окна и не будет допустимым, если определение этого окна включает определение рамки.

Указание PARTITION BY группирует строки запроса в *разделы*, которые затем обрабатываются оконной функцией независимо друг от друга. PARTITION BY работает подобно предложению GROUP BY на уровне запроса, за исключением того, что его аргументы всегда просто выражения, а не имена выходных столбцов или числа. Без PARTITION BY все строки, выдаваемые запросом, рассматриваются как один раздел. Указание ORDER BY определяет порядок, в котором оконная функция обрабатывает строки раздела. Оно так же подобно предложению ORDER BY на уровне запроса и так же не принимает имена выходных столбцов или числа. Без ORDER BY строки обрабатываются в неопределённом порядке.

определение_рамки задаёт набор строк, образующих *рамку окна*, которая представляет собой подмножество строк текущего раздела и используется для оконных функций, работающих с рамкой, а не со всем разделом. Подмножество строк в рамке может меняться в зависимости от того, какая строка является текущей. Рамку можно задать в режимах RANGE, ROWS или GROUPS; в каждом случае она начинается с положения *начало_рамки* и заканчивается положением *конец_рамки*. Если *конец_рамки* не задаётся явно, подразумевается CURRENT ROW (текущая строка).

Если *начало_рамки* задано как UNBOUNDED PRECEDING, рамка начинается с первой строки раздела, а если *конец_рамки* определён как UNBOUNDED FOLLOWING, рамка заканчивается последней строкой раздела.

В режиме RANGE или GROUPS *начало_рамки*, заданное как CURRENT ROW, определяет в качестве начала первую *родственную* строку (строку, которая при сортировке согласно указанному для окна предложению ORDER BY считается равной текущей), тогда как *конец_рамки*, заданный как CURRENT ROW, определяет концом рамки последнюю родственную строку. В режиме ROWS вариант CURRENT ROW просто обозначает текущую строку.

В вариантах определения рамки *смещение* PRECEDING и *смещение* FOLLOWING в качестве *смещения* должно задаваться выражение, не содержащее какие-либо переменные и вызовы агрегатных или оконных функций. Что именно будет означать *смещение*, определяется в зависимости от режима рамки:

- В режиме `ROWS` *смещение* должно задаваться отличным от `NULL` неотрицательным целым числом, и это число определяет сдвиг, с которым начало рамки позиционируется перед текущей строкой, а конец — после текущей строки.
- В режиме `GROUPS` *смещение* также должно задаваться отличным от `NULL` неотрицательным целым числом, и это число определяет сдвиг (по количеству групп родственных строк), с которым начало рамки позиционируется перед группой строк, родственных текущей, а конец — после этой группы. Группу родственных строк образуют строки, которые считаются равными согласно `ORDER BY`. (Для использования режима `GROUPS` определение окна должно содержать предложение `ORDER BY`.)
- В режиме `RANGE` для использования этих указаний предложение `ORDER BY` должно содержать ровно один столбец. В этом случае *смещение* задаёт максимальную разницу между значением этого столбца в текущей строке и значением его же в предшествующих или последующих строках рамки. Тип данных выражения *смещение* зависит от типа данных упорядочивающего столбца. Для числовых столбцов это обычно тот же числовой тип, а для столбцов с типом `дата/время` — тип `interval`. Например, если упорядочивающий столбец имеет тип `date` или `timestamp`, возможна такая запись: `RANGE BETWEEN '1 day' PRECEDING AND '10 days' FOLLOWING`. Значение *смещение* при этом может так же быть отличным от `NULL` и неотрицательным, хотя что считать «неотрицательным», будет зависеть от типа данных.

В любом случае расстояние до конца рамки ограничивается расстоянием до конца раздела, так что для строк, которые находятся у конца раздела, рамка может содержать меньше строк, чем для других.

Заметьте, что в режимах `ROWS` и `GROUPS` указания `0 PRECEDING` и `0 FOLLOWING` равнозначны указанию `CURRENT ROW`. Обычно это справедливо и для режима `RANGE`, в случае подходящего для типа данных определения значения «нуля».

Дополнение *исключение_рамки* позволяет исключить из рамки строки, которые окружают текущую строку, даже если они должны быть включены согласно указаниям, определяющим начало и конец рамки. `EXCLUDE CURRENT ROW` исключает из рамки текущую строку. `EXCLUDE GROUP` исключает из рамки текущую строку и родственные ей согласно порядку сортировки. `EXCLUDE TIES` исключает из рамки все родственные строки для текущей, но не собственно текущую строку. `EXCLUDE NO OTHERS` просто явно выражает поведение по умолчанию — не исключает ни текущую строку, ни родственные ей.

По умолчанию рамка определяется как `RANGE UNBOUNDED PRECEDING`, что равносильно расширенному определению `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. С указанием `ORDER BY` это означает, что рамка будет включать все строки от начала раздела до последней строки, родственной текущей (для `ORDER BY`). Без `ORDER BY` это означает, что в рамку включаются все строки раздела, так как все они считаются родственными текущей.

Действуют также следующие ограничения: в качестве *начала_рамки* нельзя задать `UNBOUNDED FOLLOWING`, в качестве *конца_рамки* не допускается `UNBOUNDED PRECEDING` и *конец_рамки* не может идти в показанном выше списке указаний *начало_рамки* **AND** *конец_рамки* перед *началом_рамки*. В частности, синтаксис `RANGE BETWEEN CURRENT ROW AND смещение PRECEDING` не допускается. Но при этом, например, определение `ROWS BETWEEN 7 PRECEDING AND 8 PRECEDING` допустимо, хотя оно и не выберет никакие строки.

Если добавлено предложение `FILTER`, оконной функции подаются только те входные строки, для которых *условие_фильтра* вычисляется как истинное; другие строки отбрасываются. Предложение `FILTER` допускается только для агрегирующих оконных функций.

Встроенные оконные функции описаны в [Таблице 9.67](#), но пользователь может расширить этот набор, создавая собственные функции. Кроме того, в качестве оконных функций можно использовать любые встроенные или пользовательские универсальные, а также статистические агрегатные функции. (Сортирующие и гипотезирующие агрегатные функции в настоящее время использовать в качестве оконных нельзя.)

Запись со звёздочкой (*) применяется при вызове не имеющих параметров агрегатных функций в качестве оконных, например `count (*) OVER (PARTITION BY x ORDER BY y)`. Звёздочка (*) обычно не применяется для исключительно оконных функций. Такие функции не допускают использования `DISTINCT` и `ORDER BY` в списке аргументов функции.

Вызовы оконных функций разрешены в запросах только в списке `SELECT` и в предложении `ORDER BY`.

Дополнительно об оконных функциях можно узнать в [Разделе 3.5](#), [Разделе 9.22](#) и [Подразделе 7.2.5](#).

4.2.9. Приведения типов

Приведение типа определяет преобразование данных из одного типа в другой. Postgres Pro воспринимает две равносильные записи приведения типов:

```
CAST ( выражение AS тип )
выражение :: тип
```

Запись с `CAST` соответствует стандарту SQL, тогда как вариант с `::` — историческое наследие Postgres Pro.

Когда приведению подвергается значение выражения известного типа, происходит преобразование типа во время выполнения. Это приведение будет успешным, только если определён подходящий оператор преобразования типов. Обратите внимание на небольшое отличие от приведения констант, описанного в [Подразделе 4.1.2.7](#). Приведение строки в чистом виде представляет собой начальное присваивание строковой константы и оно будет успешным для любого типа (конечно, если строка содержит значение, приемлемое для данного типа данных).

Явное приведение типа можно опустить, если возможно однозначно определить, какой тип должно иметь выражение (например, когда оно присваивается столбцу таблицы); в таких случаях система автоматически преобразует тип. Однако автоматическое преобразование выполняется только для приведений с пометкой «допускается неявное применение» в системных каталогах. Все остальные приведения должны записываться явно. Это ограничение позволяет избежать сюрпризов с неявным преобразованием.

Также можно записать приведение типа как вызов функции:

```
имя_типа ( выражение )
```

Однако это будет работать только для типов, имена которых являются также допустимыми именами функций. Например, `double precision` так использовать нельзя, а `float8` (альтернативное название того же типа) — можно. Кроме того, имена типов `interval`, `time` и `timestamp` из-за синтаксического конфликта можно использовать в такой записи только в кавычках. Таким образом, запись приведения типа в виде вызова функции провоцирует несоответствия и, возможно, лучше будет её не применять.

Примечание

Приведение типа, представленное в виде вызова функции, на самом деле соответствует внутреннему механизму. Даже при использовании двух стандартных типов записи внутри происходит вызов зарегистрированной функции, выполняющей преобразование. По соглашению именем такой функции преобразования является имя выходного типа, и таким образом запись «в виде вызова функции» есть не что иное, как прямой вызов нижележащей функции преобразования. При создании переносимого приложения на это поведение, конечно, не следует рассчитывать. Подробнее это описано в справке [CREATE CAST](#).

4.2.10. Применение правил сортировки

Предложение `COLLATE` переопределяет правило сортировки выражения. Оно добавляется после выражения:

выражение `COLLATE правило_сортировки`

где `правило_сортировки` — идентификатор правила, возможно дополненный именем схемы. Предложение `COLLATE` связывает выражение сильнее, чем операторы, так что при необходимости следует использовать скобки.

Если правило сортировки не определено явно, система либо выбирает его по столбцам, которые используются в выражении, либо, если таких столбцов нет, переключается на установленное для базы данных правило сортировки по умолчанию.

Предложение `COLLATE` имеет два распространённых применения: переопределение порядка сортировки в предложении `ORDER BY`, например:

```
SELECT a, b, c FROM tbl WHERE ... ORDER BY a COLLATE "C";
```

и переопределение правил сортировки при вызове функций или операторов, возвращающих языкозависимые результаты, например:

```
SELECT * FROM tbl WHERE a > 'foo' COLLATE "C";
```

Заметьте, что в последнем случае предложение `COLLATE` добавлено к аргументу оператора, на действие которого мы хотим повлиять. При этом не имеет значения, к какому именно аргументу оператора или функции добавляется `COLLATE`, так как правило сортировки, применяемое к оператору или функции, выбирается при рассмотрении всех аргументов, а явное предложение `COLLATE` переопределяет правила сортировки для всех других аргументов. (Однако добавление разных предложений `COLLATE` к нескольким аргументам будет ошибкой. Подробнее об этом см. [Раздел 22.2.](#)) Таким образом, эта команда выдаст тот же результат:

```
SELECT * FROM tbl WHERE a COLLATE "C" > 'foo';
```

Но это будет ошибкой:

```
SELECT * FROM tbl WHERE (a > 'foo') COLLATE "C";
```

здесь правило сортировки нельзя применить к результату оператора `>`, который имеет несравнимый тип данных `boolean`.

4.2.11. Скалярные подзапросы

Скалярный подзапрос — это обычный запрос `SELECT` в скобках, который возвращает ровно одну строку и один столбец. (Написание запросов освещается в [Главе 7.](#)) После выполнения запроса `SELECT` его единственный результат используется в окружающем его выражении. В качестве скалярного подзапроса нельзя использовать запросы, возвращающие более одной строки или столбца. (Но если в результате выполнения подзапрос не вернёт строк, скалярный результат считается равным `NULL`.) В подзапросе можно ссылаться на переменные из окружающего запроса; в процессе одного вычисления подзапроса они будут считаться константами. Другие выражения с подзапросами описаны в [Разделе 9.23.](#)

Например, следующий запрос находит число жителей в самом населённом городе в каждом штате:

```
SELECT name, (SELECT max(pop) FROM cities WHERE cities.state = states.name)
FROM states;
```

4.2.12. Конструкторы массивов

Конструктор массива — это выражение, которое создаёт массив, определяя значения его элементов. Конструктор простого массива состоит из ключевого слова `ARRAY`, открывающей квадратной скобки `[`, списка выражений (разделённых запятыми), задающих значения элементов массива, и закрывающей квадратной скобки `]`. Например:

```
SELECT ARRAY[1, 2, 3+4];
      array
-----
{1, 2, 7}
```

```
(1 row)
```

По умолчанию типом элементов массива считается общий тип для всех выражений, определённый по правилам, действующим и для конструкций `UNION` и `CASE` (см. [Раздел 10.5](#)). Вы можете переопределить его явно, приведя конструктор массива к требуемому типу, например:

```
SELECT ARRAY[1,2,22.7]::integer[];
      array
-----
 {1,2,23}
(1 row)
```

Это равносильно тому, что привести к нужному типу каждое выражение по отдельности. Подробнее приведение типов описано в [Подразделе 4.2.9](#).

Многомерные массивы можно образовывать, вкладывая конструкторы массивов. При этом во внутренних конструкторах слово `ARRAY` можно опускать. Например, результат работы этих конструкторов одинаков:

```
SELECT ARRAY[ARRAY[1,2], ARRAY[3,4]];
      array
-----
 {{1,2},{3,4}}
(1 row)
```

```
SELECT ARRAY[[1,2],[3,4]];
      array
-----
 {{1,2},{3,4}}
(1 row)
```

Многомерные массивы должны быть прямоугольными, и поэтому внутренние конструкторы одного уровня должны создавать вложенные массивы одинаковой размерности. Любое приведение типа, применённое к внешнему конструктору `ARRAY`, автоматически распространяется на все внутренние.

Элементы многомерного массива можно создавать не только вложенными конструкторами `ARRAY`, но и другими способами, позволяющими получить массивы нужного типа. Например:

```
CREATE TABLE arr(f1 int[], f2 int[]);

INSERT INTO arr VALUES (ARRAY[1,2],[3,4], ARRAY[5,6],[7,8]);

SELECT ARRAY[f1, f2, '{{9,10},{11,12}}'::int[]] FROM arr;
      array
-----
 {{1,2},{3,4}},{5,6},{7,8}},{9,10},{11,12}}
(1 row)
```

Вы можете создать и пустой массив, но так как массив не может быть не типизированным, вы должны явно привести пустой массив к нужному типу. Например:

```
SELECT ARRAY[]::integer[];
      array
-----
 {}
(1 row)
```

Также возможно создать массив из результатов подзапроса. В этом случае конструктор массива записывается так же с ключевым словом `ARRAY`, за которым в круглых скобках следует подзапрос. Например:

```
SELECT ARRAY(SELECT oid FROM pg_proc WHERE proname LIKE 'bytea%');
```

array

```
-----
{2011,1954,1948,1952,1951,1244,1950,2005,1949,1953,2006,31,2412}
(1 row)
```

```
SELECT ARRAY(SELECT ARRAY[i, i*2] FROM generate_series(1,5) AS a(i));
          array
```

```
-----
{{1,2},{2,4},{3,6},{4,8},{5,10}}
(1 row)
```

Такой подзапрос должен возвращать один столбец. Если этот столбец имеет тип, отличный от массива, результирующий одномерный массив будет включать элементы для каждой строки-результата подзапроса и типом элемента будет тип столбца результата. Если же тип столбца — массив, будет создан массив того же типа, но большей размерности; в любом случае во всех строках подзапроса должны выдаваться массивы одинаковой размерности, чтобы можно было получить прямоугольный результат.

Индексы массива, созданного конструктором `ARRAY`, всегда начинаются с одного. Подробнее о массивах вы узнаете в [Разделе 8.15](#).

4.2.13. Конструкторы табличных строк

Конструктор табличной строки — это выражение, создающее строку или кортеж (или составное значение) из значений его аргументов-полей. Конструктор строки состоит из ключевого слова `ROW`, открывающей круглой скобки, нескольких выражений (разделённых запятыми), определяющих значения полей, и закрывающей скобки. При этом выражения могут отсутствовать, то есть конструктор имеет вид `ROW()`. Например:

```
SELECT ROW(1,2.5,'this is a test');
```

Если в списке более одного выражения, ключевое слово `ROW` можно опустить.

Конструктор строки поддерживает запись *составное_значение.**, при этом данное значение будет развёрнуто в список элементов, так же, как в записи *.** на верхнем уровне списка `SELECT` (см. [Подраздел 8.16.5](#)). Например, если таблица `t` содержит столбцы `f1` и `f2`, эти записи равнозначны:

```
SELECT ROW(t.*, 42) FROM t;
SELECT ROW(t.f1, t.f2, 42) FROM t;
```

Примечание

До версии PostgreSQL 8.2 запись *.** не разворачивалась в конструкторах строк, так что выражение `ROW(t.*, 42)` создавало составное значение из двух полей, в котором первое поле так же было составным. Новое поведение обычно более полезно. Если вам нужно получить прежнее поведение, чтобы одно значение строки было вложено в другое, напишите внутреннее значение без *.**, например: `ROW(t, 42)`.

По умолчанию значение, созданное выражением `ROW`, имеет тип анонимной записи. Если необходимо, его можно привести к именованному составному типу — либо к типу строки таблицы, либо составному типу, созданному оператором `CREATE TYPE AS`. Явное приведение может потребоваться для достижения однозначности. Например:

```
CREATE TABLE mytable(f1 int, f2 float, f3 text);

CREATE FUNCTION getf1(mytable) RETURNS int AS 'SELECT $1.f1' LANGUAGE SQL;

-- Приведение не требуется, так как существует только одна getf1()
SELECT getf1(ROW(1,2.5,'this is a test'));
getf1
```

```

-----
      1
(1 row)

CREATE TYPE myrowtype AS (f1 int, f2 text, f3 numeric);

CREATE FUNCTION getf1(myrowtype) RETURNS int AS 'SELECT $1.f1' LANGUAGE SQL;

-- Теперь приведение необходимо для однозначного выбора функции:
SELECT getf1(ROW(1,2.5,'this is a test'));
ОШИБКА: функция getf1(record) не уникальна

SELECT getf1(ROW(1,2.5,'this is a test')::mytable);
  getf1
-----
      1
(1 row)

SELECT getf1(CAST(ROW(11,'this is a test',2.5) AS myrowtype));
  getf1
-----
     11
(1 row)

```

Используя конструктор строк (кортежей), можно создавать составное значение для сохранения в столбце составного типа или для передачи функции, принимающей составной параметр. Также вы можете сравнить два составных значения или проверить их с помощью `IS NULL` или `IS NOT NULL`, например:

```

SELECT ROW(1,2.5,'this is a test') = ROW(1, 3, 'not the same');

-- выбрать все строки, содержащие только NULL
SELECT ROW(table.*) IS NULL FROM table;

```

Подробнее см. [Раздел 9.24](#). Конструкторы строк также могут использоваться в сочетании с подзапросами, как описано в [Разделе 9.23](#).

4.2.14. Правила вычисления выражений

Порядок вычисления подвыражений не определён. В частности, аргументы оператора или функции не обязательно вычисляются слева направо или в любом другом фиксированном порядке.

Более того, если результат выражения можно получить, вычисляя только некоторые его части, тогда другие подвыражения не будут вычисляться вовсе. Например, если написать:

```
SELECT true OR somefunc();
```

тогда функция `somefunc()` не будет вызываться (возможно). То же самое справедливо для записи:

```
SELECT somefunc() OR true;
```

Заметьте, что это отличается от «оптимизации» вычисления логических операторов слева направо, реализованной в некоторых языках программирования.

Как следствие, в сложных выражениях не стоит использовать функции с побочными эффектами. Особенно опасно рассчитывать на порядок вычисления или побочные эффекты в предложениях `WHERE` и `HAVING`, так как эти предложения тщательно оптимизируются при построении плана выполнения. Логические выражения (сочетания `AND/OR/NOT`) в этих предложениях могут быть видоизменены любым способом, допустимым законами Булевой алгебры.

Когда порядок вычисления важен, его можно зафиксировать с помощью конструкции `CASE` (см. [Раздел 9.18](#)). Например, такой способ избежать деления на ноль в предложении `WHERE` ненадёжен:


```
SELECT ... WHERE x > 0 AND y/x > 1.5;
```

Безопасный вариант:

```
SELECT ... WHERE CASE WHEN x > 0 THEN y/x > 1.5 ELSE false END;
```

Применяемая так конструкция CASE защищает выражение от оптимизации, поэтому использовать её нужно только при необходимости. (В данном случае было бы лучше решить проблему, переписав условие как $y > 1.5 \cdot x$.)

Однако CASE не всегда спасает в подобных случаях. Показанный выше приём плох тем, что не предотвращает раннее вычисление константных подвыражений. Как описано в [Разделе 37.7](#), функции и операторы, помеченные как IMMUTABLE, могут вычисляться при планировании, а не выполнении запроса. Поэтому в примере

```
SELECT CASE WHEN x > 0 THEN x ELSE 1/0 END FROM tab;
```

, скорее всего, произойдёт деление на ноль из-за того, что планировщик попытается упростить константное подвыражение, даже если во всех строках в таблице $x > 0$, а значит во время выполнения ветвь ELSE никогда не будет выполняться.

Хотя этот конкретный пример может показаться надуманным, похожие ситуации, в которых неявно появляются константы, могут возникать и в запросах внутри функций, так как значения аргументов функции и локальных переменных при планировании могут быть заменены константами. Поэтому, например, в функциях PL/pgSQL гораздо безопаснее для защиты от рискованных вычислений использовать конструкцию IF-THEN-ELSE, чем выражение CASE.

Ещё один подобный недостаток этого подхода в том, что CASE не может предотвратить вычисление заключённого в нём агрегатного выражения, так как агрегатные выражения вычисляются перед всеми остальными в списке SELECT или предложении HAVING. Например, в следующем запросе может возникнуть ошибка деления на ноль, несмотря на то, что он вроде бы защищён от неё:

```
SELECT CASE WHEN min(employees) > 0
            THEN avg(expenses / employees)
            END
FROM departments;
```

Агрегатные функции min() и avg() вычисляются независимо по всем входным строкам, так что если в какой-то строке поле employees окажется равным нулю, деление на ноль произойдёт раньше, чем станет возможным проверить результат функции min(). Поэтому, чтобы проблемные входные строки изначально не попали в агрегатную функцию, следует воспользоваться предложениями WHERE или FILTER.

4.3. Вызов функций

Postgres Pro позволяет вызывать функции с именованными параметрами, используя запись с *позиционной* или *именной* передачей аргументов. Именная передача особенно полезна для функций со множеством параметров, так как она делает связь параметров и аргументов более явной и надёжной. В позиционной записи значения аргументов функции указываются в том же порядке, в каком они описаны в определении функции. При именной передаче аргументы сопоставляются с параметрами функции по именам и указывать их можно в любом порядке. Для каждого варианта вызова также учитывайте влияние типов аргументов функций, описанное в [Разделе 10.3](#).

При записи любым способом параметры, для которых в определении функции заданы значения по умолчанию, можно вовсе не указывать. Но это особенно полезно при именной передаче, так как опустить можно любой набор параметров, тогда как при позиционной параметры можно опускать только последовательно, справа налево.

Postgres Pro также поддерживает *смешанную* передачу, когда параметры передаются и по именам, и по позиции. В этом случае позиционные параметры должны идти перед параметрами, передаваемыми по именам.

Мы рассмотрим все три варианта записи на примере следующей функции:


```
CREATE FUNCTION concat_lower_or_upper(a text, b text,
    uppercase boolean DEFAULT false)
RETURNS text
AS
$$
    SELECT CASE
        WHEN $3 THEN UPPER($1 || ' ' || $2)
        ELSE LOWER($1 || ' ' || $2)
    END;
$$
LANGUAGE SQL IMMUTABLE STRICT;
```

Функция `concat_lower_or_upper` имеет два обязательных параметра: `a` и `b`. Кроме того, есть один необязательный параметр `uppercase`, который по умолчанию имеет значение `false`. Аргументы `a` и `b` будут сложены вместе и переведены в верхний или нижний регистр, в зависимости от параметра `uppercase`. Остальные тонкости реализации функции сейчас не важны (подробнее о них рассказано в [Главе 37](#)).

4.3.1. Позиционная передача

Позиционная передача — это традиционный механизм передачи аргументов функции в PostgreSQL. Пример такой записи:

```
SELECT concat_lower_or_upper('Hello', 'World', true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

Все аргументы указаны в заданном порядке. Результат возвращён в верхнем регистре, так как параметр `uppercase` имеет значение `true`. Ещё один пример:

```
SELECT concat_lower_or_upper('Hello', 'World');
concat_lower_or_upper
-----
hello world
(1 row)
```

Здесь параметр `uppercase` опущен, и поэтому он принимает значение по умолчанию (`false`), и результат переводится в нижний регистр. В позиционной записи любые аргументы с определённым значением по умолчанию можно опускать справа налево.

4.3.2. Именная передача

При именной передаче для аргумента добавляется имя, которое отделяется от выражения значения знаками `=>`. Например:

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World');
concat_lower_or_upper
-----
hello world
(1 row)
```

Здесь аргумент `uppercase` был так же опущен, так что он неявно получил значение `false`. Преимуществом такой записи является возможность записывать аргументы в любом порядке, например:

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World', uppercase => true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

```
SELECT concat_lower_or_upper(a => 'Hello', uppercase => true, b => 'World');
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

Для обратной совместимости поддерживается и старый синтаксис с «:=»:

```
SELECT concat_lower_or_upper(a := 'Hello', uppercase := true, b := 'World');
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

4.3.3. Смешанная передача

При смешанной передаче параметры передаются и по именам, и по позиции. Однако как уже было сказано, именованные аргументы не могут стоять перед позиционными. Например:

```
SELECT concat_lower_or_upper('Hello', 'World', uppercase => true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

В данном запросе аргументы `a` и `b` передаются по позиции, а `uppercase` — по имени. Единственное обоснование такого вызова здесь — он стал чуть более читаемым. Однако для более сложных функций с множеством аргументов, часть из которых имеют значения по умолчанию, именная или смешанная передача позволяют записать вызов эффективнее и уменьшить вероятность ошибок.

Примечание

Именная и смешанная передача в настоящий момент не может использоваться при вызове агрегатной функции (но они допускаются, если агрегатная функция используется в качестве оконной).

Глава 5. Определение данных

Эта глава рассказывает, как создавать структуры базы данных, в которых будут храниться данные. В реляционной базе данных данные хранятся в таблицах, так что большая часть этой главы будет посвящена созданию и изменению таблиц, а также средствам управления данными в них. Затем мы обсудим, как таблицы можно объединять в схемы и как ограничивать доступ к ним. Наконец, мы кратко рассмотрим другие возможности, связанные с хранением данных, в частности наследование, секционирование таблиц, представления, функции и триггеры.

5.1. Основы таблиц

Таблица в реляционной базе данных похожа на таблицу на бумаге: она так же состоит из строк и столбцов. Число и порядок столбцов фиксированы, а каждый столбец имеет имя. Число строк переменное — оно отражает текущее количество находящихся в ней данных. SQL не даёт никаких гарантий относительно порядка строк таблицы. При чтении таблицы строки выводятся в произвольном порядке, если только явно не требуется сортировка. Подробнее это рассматривается в [Главе 7](#). Более того, SQL не назначает строкам уникальные идентификаторы, так что можно иметь в таблице несколько полностью идентичных строк. Это вытекает из математической модели, которую реализует SQL, но обычно такое дублирование нежелательно. Позже в этой главе мы увидим, как его избежать.

Каждому столбцу сопоставлен тип данных. Тип данных ограничивает набор допустимых значений, которые можно присвоить столбцу, и определяет смысловое значение данных для вычислений. Например, в столбец числового типа нельзя записать обычные текстовые строки, но зато его данные можно использовать в математических вычислениях. И наоборот, если столбец имеет тип текстовой строки, для него допустимы практически любые данные, но он непригоден для математических действий (хотя другие операции, например конкатенация строк, возможны).

В Postgres Pro есть внушительный набор встроенных типов данных, удовлетворяющий большинство приложений. Пользователи также могут определять собственные типы данных. Большинство встроенных типов данных имеют понятные имена и семантику, так что мы отложим их подробное рассмотрение до [Главы 8](#). Наиболее часто применяются следующие типы данных: `integer` для целых чисел, `numeric` для чисел, которые могут быть дробными, `text` для текстовых строк, `date` для дат, `time` для времени и `timestamp` для значений, включающих дату и время.

Для создания таблицы используется команда `CREATE TABLE`. В этой команде вы должны указать как минимум имя новой таблицы и имена и типы данных каждого столбца. Например:

```
CREATE TABLE my_first_table (  
    first_column text,  
    second_column integer  
);
```

Так вы создадите таблицу `my_first_table` с двумя столбцами. Первый столбец называется `first_column` и имеет тип данных `text`; второй столбец называется `second_column` и имеет тип `integer`. Имена таблицы и столбцов соответствуют синтаксису идентификаторов, описанному в [Подразделе 4.1.1](#). Имена типов также являются идентификаторами, хотя есть некоторые исключения. Заметьте, что список столбцов заключается в скобки, а его элементы разделяются запятыми.

Конечно, предыдущий пример ненатурален. Обычно в именах таблиц и столбцов отражается, какие данные они будут содержать. Поэтому давайте взглянем на более реалистичный пример:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric  
);
```

(Тип `numeric` может хранить дробные числа, в которых обычно выражаются денежные суммы.)

Подсказка

Когда вы создаёте много взаимосвязанных таблиц, имеет смысл заранее выбрать единый шаблон именования таблиц и столбцов. Например, решить, будут ли в именах таблиц использоваться существительные во множественном или в единственном числе (есть соображения в пользу каждого варианта).

Число столбцов в таблице не может быть бесконечным. Это число ограничивается максимумом в пределах от 250 до 1600, в зависимости от типов столбцов. Однако создавать таблицы с таким большим числом столбцов обычно не требуется, а если такая потребность возникает, это скорее признак сомнительного дизайна.

Если таблица вам больше не нужна, вы можете удалить её, выполнив команду `DROP TABLE`. Например:

```
DROP TABLE my_first_table;
DROP TABLE products;
```

Попытка удаления несуществующей таблицы считается ошибкой. Тем не менее в SQL-скриптах часто применяют безусловное удаление таблиц перед созданием, игнорируя все сообщения об ошибках, так что они выполняют свою задачу независимо от того, существовали таблицы или нет. (Если вы хотите избежать таких ошибок, можно использовать вариант `DROP TABLE IF EXISTS`, но это не будет соответствовать стандарту SQL.)

Как изменить существующую таблицу, будет рассмотрено в этой главе позже, в [Разделе 5.6](#).

Имея средства, которые мы обсудили, вы уже можете создавать полностью функциональные таблицы. В продолжении этой главы рассматриваются дополнительные возможности, призванные обеспечить целостность данных, безопасность и удобство. Если вам не терпится наполнить свои таблицы данными, вы можете вернуться к этой главе позже, а сейчас перейти к [Главе 6](#).

5.2. Значения по умолчанию

Столбцу можно назначить значение по умолчанию. Когда добавляется новая строка и каким-то её столбцам не присваиваются значения, эти столбцы принимают значения по умолчанию. Также команда управления данными может явно указать, что столбцу должно быть присвоено значение по умолчанию, не зная его. (Подробнее команды управления данными описаны в [Главе 6](#).)

Если значение по умолчанию не объявлено явно, им считается значение `NULL`. Обычно это имеет смысл, так как можно считать, что `NULL` представляет неизвестные данные.

В определении таблицы значения по умолчанию указываются после типа данных столбца. Например:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric DEFAULT 9.99
);
```

Значение по умолчанию может быть выражением, которое в этом случае вычисляется в момент присваивания значения по умолчанию (а не когда создаётся таблица). Например, столбцу `timestamp` в качестве значения по умолчанию часто присваивается `CURRENT_TIMESTAMP`, чтобы в момент добавления строки в нём оказалось текущее время. Ещё один распространённый пример — генерация «последовательных номеров» для всех строк. В Postgres Pro это обычно делается примерно так:

```
CREATE TABLE products (
```

```
product_no integer DEFAULT nextval('products_product_no_seq'),
...
);
```

здесь функция `nextval()` выбирает очередное значение из *последовательности* (см. [Раздел 9.17](#)). Это употребление настолько распространено, что для него есть специальная короткая запись:

```
CREATE TABLE products (
    product_no SERIAL,
    ...
);
```

`SERIAL` обсуждается позже в [Подразделе 8.1.4](#).

5.3. Генерируемые столбцы

Генерируемый столбец является столбцом особого рода, который всегда вычисляется из других. Таким образом, для столбцов он является тем же, чем представление для таблицы. Есть два типа генерируемых столбцов: хранимые и виртуальные. Хранимый генерируемый столбец вычисляется при записи (добавлении или изменении) и занимает место в таблице так же, как и обычный столбец. Виртуальный генерируемый столбец не занимает места и вычисляется при чтении. Поэтому можно сказать, что виртуальный генерируемый столбец похож на представление, а хранимый генерируемый столбец — на материализованное представление (за исключением того, что он всегда обновляется автоматически). В настоящее время в Postgres Pro реализованы только хранимые генерируемые столбцы.

Чтобы создать генерируемый столбец, воспользуйтесь предложением `GENERATED ALWAYS AS` команды `CREATE TABLE`, например:

```
CREATE TABLE people (
    ...,
    height_cm numeric,
    height_in numeric GENERATED ALWAYS AS (height_cm / 2.54) STORED
);
```

Ключевое слово `STORED`, определяющее тип хранения генерируемого столбца, является обязательным. За подробностями обратитесь к [CREATE TABLE](#).

Произвести запись непосредственно в генерируемый столбец нельзя. Поэтому в командах `INSERT` или `UPDATE` нельзя задать значение для таких столбцов, хотя ключевое слово `DEFAULT` указать можно.

Примите к сведению следующие отличия генерируемых столбцов от столбцов со значением по умолчанию. Значение столбца по умолчанию вычисляется один раз, когда в таблицу впервые вставляется строка и никакое другое значение не задано; значение же генерируемого столбца может меняться при изменении строки и не может быть переопределено. Выражение значения по умолчанию не может обращаться к другим столбцам таблицы, а генерирующее выражение, напротив, обычно обращается к ним. В выражении значения по умолчанию могут вызываться изменчивые функции, например, `random()` или функции, зависящие от времени, а для генерируемых столбцов это не допускается.

С определением генерируемых столбцов и их содержащими таблицами связан ряд ограничений и особенностей:

- В генерирующем выражении могут использоваться только постоянные функции и не могут фигурировать подзапросы или ссылки на какие-либо значения, не относящиеся к данной строке.
- Генерирующее выражение не может обращаться к другому генерируемому столбцу.
- Генерирующее выражение не может обращаться к системным столбцам, за исключением `tableoid`.

- Для генерируемого столбца нельзя задать значение по умолчанию или свойство идентификации.
- Генерируемый столбец не может быть частью ключа секционирования.
- Генерируемые столбцы могут содержаться в сторонних таблицах. За подробностями обратитесь к [CREATE FOREIGN TABLE](#).
- Применительно к наследованию:
 - Если родительский столбец является генерируемым, дочерний должен генерироваться тем же выражением. Поэтому в определении дочернего столбца нужно опустить предложение `GENERATED`, так как оно будет скопировано из родительского.
 - В случае множественного наследования, если один родительский столбец является генерируемым, все соответствующие ему столбцы в иерархии наследования также должны генерироваться тем же выражением.
 - Если родительский столбец не является генерируемым, дочерний столбец может быть как генерируемым, так и обычным.

Дополнительные замечания касаются использования генерируемых столбцов.

- Права доступа к генерируемым столбцам существуют отдельно от прав для нижележащих базовых столбцов. Поэтому их можно организовать так, чтобы определённый пользователь мог прочитать генерируемый столбец, но не нижележащие базовые столбцы.
- Генерируемые столбцы, в соответствии с концепцией, пересчитываются после выполнения триггеров `BEFORE`. Вследствие этого, в генерируемых столбцах будут отражаться изменения, производимые триггером `BEFORE` в базовых столбцах, а обращаться в коде такого триггера к генерируемым столбцам, напротив, нельзя.
- Генерируемые столбцы не участвуют в логической репликации и не могут быть указаны в списке столбцов `CREATE PUBLICATION`.

5.4. Ограничения

Типы данных сами по себе ограничивают множество данных, которые можно сохранить в таблице. Однако для многих приложений такие ограничения слишком грубые. Например, столбец, содержащий цену продукта, должен, вероятно, принимать только положительные значения. Но такого стандартного типа данных нет. Возможно, вы также захотите ограничить данные столбца по отношению к другим столбцам или строкам. Например, в таблице с информацией о товаре должна быть только одна строка с определённым кодом товара.

Для решения подобных задач SQL позволяет вам определять ограничения для столбцов и таблиц. Ограничения дают вам возможность управлять данными в таблицах так, как вы захотите. Если пользователь попытается сохранить в столбце значение, нарушающее ограничения, возникнет ошибка. Ограничения будут действовать, даже если это значение по умолчанию.

5.4.1. Ограничения-проверки

Ограничение-проверка — наиболее общий тип ограничений. В его определении вы можете указать, что значение данного столбца должно удовлетворять логическому выражению (проверке истинности). Например, цену товара можно ограничить положительными значениями так:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric CHECK (price > 0)
);
```

Как вы видите, ограничение определяется после типа данных, как и значение по умолчанию. Значения по умолчанию и ограничения могут указываться в любом порядке. Ограничение-проверка

состоит из ключевого слова `CHECK`, за которым идёт выражение в скобках. Это выражение должно включать столбец, для которого задаётся ограничение, иначе оно не имеет большого смысла.

Вы можете также присвоить ограничению отдельное имя. Это улучшит сообщения об ошибках и позволит вам ссылаться на это ограничение, когда вам понадобится изменить его. Сделать это можно так:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CONSTRAINT positive_price CHECK (price > 0)  
);
```

То есть, чтобы создать именованное ограничение, напишите ключевое слово `CONSTRAINT`, а за ним идентификатор и собственно определение ограничения. (Если вы не определите имя ограничения таким образом, система выберет для него имя за вас.)

Ограничение-проверка может также ссылаться на несколько столбцов. Например, если вы храните обычную цену и цену со скидкой, так вы можете гарантировать, что цена со скидкой будет всегда меньше обычной:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CHECK (price > 0),  
    discounted_price numeric CHECK (discounted_price > 0),  
    CHECK (price > discounted_price)  
);
```

Первые два ограничения определяются похожим образом, но для третьего используется новый синтаксис. Оно не связано с определённым столбцом, а представлено отдельным элементом в списке. Определения столбцов и такие определения ограничений можно переставлять в произвольном порядке.

Про первые два ограничения можно сказать, что это ограничения столбцов, тогда как третье является ограничением таблицы, так как оно написано отдельно от определений столбцов. Ограничения столбцов также можно записать в виде ограничений таблицы, тогда как обратное не всегда возможно, так как подразумевается, что ограничение столбца ссылается только на связанный столбец. (Хотя Postgres Pro этого не требует, но для совместимости с другими СУБД лучше следовать этому правилу.) Ранее приведённый пример можно переписать и так:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric,  
    CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0),  
    CHECK (price > discounted_price)  
);
```

Или даже так:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0 AND price > discounted_price)  
);
```

Это дело вкуса.

Ограничениям таблицы можно присваивать имена так же, как и ограничениям столбцов:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric,
    CHECK (price > 0),
    discounted_price numeric,
    CHECK (discounted_price > 0),
    CONSTRAINT valid_discount CHECK (price > discounted_price)
);
```

Следует заметить, что ограничение-проверка удовлетворяется, если выражение принимает значение true или NULL. Так как результатом многих выражений с операндами NULL будет значение NULL, такие ограничения не будут препятствовать записи NULL в ограничиваемые столбцы. Чтобы гарантировать, что столбец не содержит значения NULL, можно использовать ограничение NOT NULL, описанное в следующем разделе.

Примечание

Postgres Pro не поддерживает ограничения CHECK, которые обращаются к данным, не относящимся к новой или изменённой строке. Хотя ограничение CHECK, нарушающее это правило, может работать в простых случаях, в общем случае нельзя гарантировать, что база данных не придёт в состояние, когда условие ограничения окажется ложным (вследствие последующих изменений других участвующих в его вычислении строк). В результате восстановления выгруженных данных может оказаться невозможным. Во время восстановления возможен сбой, даже если полное состояние базы данных согласуется с условием ограничения, по причине того, что строки загружаются не в том порядке, в котором это условие будет соблюдаться. Поэтому для определения ограничений, затрагивающих другие строки и другие таблицы, используйте ограничения UNIQUE, EXCLUDE или FOREIGN KEY, если это возможно.

Если вам не нужна постоянно поддерживаемая гарантия целостности, а достаточно разовой проверки добавляемой строки по отношению к другим строкам, вы можете реализовать эту проверку в собственном [триггере](#). (Этот подход исключает вышеописанные проблемы при восстановлении, так как в выгрузке pg_dump триггеры воссоздаются после восстановления данных, и поэтому эта проверка не будет действовать в процессе выгрузки/восстановления.)

Примечание

В Postgres Pro предполагается, что условия ограничений CHECK являются постоянными, то есть при одинаковых данных в строке они всегда выдают одинаковый результат. Именно этим предположением оправдывается то, что ограничения CHECK проверяются только при добавлении или изменении строк, а не при каждом обращении к ним. (Приведённое выше предупреждение о недопустимости обращений к другим таблицам является частным следствием этого предположения.)

Однако это предположение может нарушаться, как часто бывает, когда в выражении CHECK используется пользовательская функция, поведение которой впоследствии меняется. Postgres Pro не запрещает этого, и если строки в таблице перестанут удовлетворять ограничению CHECK, это останется незамеченным. В итоге при попытке загрузить выгруженные позже данные могут возникнуть проблемы. Поэтому подобные изменения рекомендуется осуществлять следующим образом: удалить ограничение (используя ALTER TABLE), изменить определение функции, а затем пересоздать ограничение той же командой, которая при этом перепроверит все строки таблицы.

5.4.2. Ограничения NOT NULL

Ограничение NOT NULL просто указывает, что столбцу нельзя присваивать значение NULL. Пример синтаксиса:

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name text NOT NULL,  
    price numeric  
);
```

Ограничение NOT NULL всегда записывается как ограничение столбца и функционально эквивалентно ограничению CHECK (*имя_столбца* IS NOT NULL), но в Postgres Pro явное ограничение NOT NULL работает более эффективно. Хотя у такой записи есть недостаток — назначить имя таким ограничениям нельзя.

Естественно, для столбца можно определить больше одного ограничения. Для этого их нужно просто указать одно за другим:

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name text NOT NULL,  
    price numeric NOT NULL CHECK (price > 0)  
);
```

Порядок здесь не имеет значения, он не обязательно соответствует порядку проверки ограничений.

Для ограничения NOT NULL есть и обратное: ограничение NULL. Оно не означает, что столбец должен иметь только значение NULL, что конечно было бы бессмысленно. Суть же его в простом указании, что столбец может иметь значение NULL (это поведение по умолчанию). Ограничение NULL отсутствует в стандарте SQL и использовать его в переносимых приложениях не следует. (Оно было добавлено в Postgres Pro только для совместимости с некоторыми другими СУБД.) Однако некоторые пользователи любят его использовать, так как оно позволяет легко переключать ограничения в скрипте. Например, вы можете начать с:

```
CREATE TABLE products (  
    product_no integer NULL,  
    name text NULL,  
    price numeric NULL  
);
```

и затем вставить ключевое слово NOT, где потребуется.

Подсказка

При проектировании баз данных чаще всего большинство столбцов должны быть помечены как NOT NULL.

5.4.3. Ограничения уникальности

Ограничения уникальности гарантируют, что данные в определённом столбце или группе столбцов уникальны среди всех строк таблицы. Ограничение записывается так:

```
CREATE TABLE products (  
    product_no integer UNIQUE,  
    name text,  
    price numeric
```

```
);
```

в виде ограничения столбца и так:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric,
    UNIQUE (product_no)
);
```

в виде ограничения таблицы.

Чтобы определить ограничение уникальности для группы столбцов, запишите его в виде ограничения таблицы, перечислив имена столбцов через запятую:

```
CREATE TABLE example (
    a integer,
    b integer,
    c integer,
    UNIQUE (a, c)
);
```

Такое ограничение указывает, что сочетание значений перечисленных столбцов должно быть уникально во всей таблице, тогда как значения каждого столбца по отдельности не должны быть (и обычно не будут) уникальными.

Вы можете назначить уникальному ограничению имя обычным образом:

```
CREATE TABLE products (
    product_no integer CONSTRAINT must_be_different UNIQUE,
    name text,
    price numeric
);
```

При добавлении ограничения уникальности будет автоматически создан уникальный индекс-B-дерево для столбца или группы столбцов, перечисленных в ограничении. Условие уникальности, распространяющееся только на некоторые строки, нельзя записать в виде ограничения уникальности, однако такое условие можно установить, создав уникальный [частичный индекс](#).

Как правило, ограничение уникальности нарушается, если в таблице оказывается несколько строк, у которых совпадают значения всех столбцов, включённых в ограничение. По умолчанию два значения NULL при таком сравнении не считаются равными. Это означает, что даже при наличии ограничения уникальности в таблице можно сохранить строки с дублирующимися значениями, если они содержат NULL в одном или нескольких ограничиваемых столбцах. Это поведение можно изменить, добавив предложение `NULLS NOT DISTINCT`, например

```
CREATE TABLE products (
    product_no integer UNIQUE NULLS NOT DISTINCT,
    name text,
    price numeric
);
```

или

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric,
    UNIQUE NULLS NOT DISTINCT (product_no)
);
```

Поведение по умолчанию можно выбрать явно, добавив `NULLS DISTINCT`. Согласно стандарту SQL вариант обработки значений `NULL` по умолчанию определяется реализацией, и в других СУБД встречается другое поведение. Это следует учитывать при разработке переносимых приложений.

5.4.4. Первичные ключи

Ограничение первичного ключа означает, что образующий его столбец или группа столбцов может быть уникальным идентификатором строк в таблице. Для этого требуется, чтобы значения были одновременно уникальными и отличными от `NULL`. Таким образом, таблицы со следующими двумя определениями будут принимать одинаковые данные:

```
CREATE TABLE products (
    product_no integer UNIQUE NOT NULL,
    name text,
    price numeric
);

CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);
```

Первичные ключи могут включать несколько столбцов; синтаксис похож на запись ограничений уникальности:

```
CREATE TABLE example (
    a integer,
    b integer,
    c integer,
    PRIMARY KEY (a, c)
);
```

При добавлении первичного ключа автоматически создаётся уникальный индекс-В-дерево для столбца или группы столбцов, перечисленных в первичном ключе, и данные столбцы помечаются как `NOT NULL`.

Таблица может иметь максимум один первичный ключ. (Ограничений уникальности и ограничений `NOT NULL`, которые функционально почти равнозначны первичным ключам, может быть сколько угодно, но назначить ограничением первичного ключа можно только одно.) Теория реляционных баз данных говорит, что первичный ключ должен быть в каждой таблице. В Postgres Pro такого жёсткого требования нет, но обычно лучше ему следовать.

Первичные ключи полезны и для документирования, и для клиентских приложений. Например, графическому приложению с возможностями редактирования содержимого таблицы, вероятно, потребуется знать первичный ключ таблицы, чтобы однозначно идентифицировать её строки. Первичные ключи находят и другое применение в СУБД; в частности, первичный ключ в таблице определяет целевые столбцы по умолчанию для сторонних ключей, ссылающихся на эту таблицу.

5.4.5. Внешние ключи

Ограничение внешнего ключа указывает, что значения столбца (или группы столбцов) должны соответствовать значениям в некоторой строке другой таблицы. Это называется *ссылочной целостностью* двух связанных таблиц.

Пусть у вас уже есть таблица продуктов, которую мы неоднократно использовали ранее:

```
CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
```

```
);
```

Давайте предположим, что у вас есть таблица с заказами этих продуктов. Мы хотим, чтобы в таблице заказов содержались только заказы действительно существующих продуктов. Поэтому мы определим в ней ограничение внешнего ключа, ссылающееся на таблицу продуктов:

```
CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    product_no integer REFERENCES products (product_no),
    quantity integer
);
```

С таким ограничением создать заказ со значением `product_no`, отсутствующим в таблице `products` (и не равным `NULL`), будет невозможно.

В такой схеме таблицу `orders` называют *подчинённой* таблицей, а `products` — *главной*. Соответственно, столбцы называют так же подчинённым и главным (или ссылающимся и целевым).

Предыдущую команду можно сократить так:

```
CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    product_no integer REFERENCES products,
    quantity integer
);
```

то есть, если опустить список столбцов, внешний ключ будет неявно связан с первичным ключом главной таблицы.

Ограничению внешнего ключа можно назначить имя стандартным способом.

Внешний ключ также может ссылаться на группу столбцов. В этом случае его нужно записать в виде обычного ограничения таблицы. Например:

```
CREATE TABLE t1 (
    a integer PRIMARY KEY,
    b integer,
    c integer,
    FOREIGN KEY (b, c) REFERENCES other_table (c1, c2)
);
```

Естественно, число и типы столбцов в ограничении должны соответствовать числу и типам целевых столбцов.

Иногда имеет смысл задать в ограничении внешнего ключа в качестве «другой таблицы» ту же таблицу; такой внешний ключ называется *ссылающимся на себя*. Например, если вы хотите, чтобы строки таблицы представляли узлы древовидной структуры, вы можете написать

```
CREATE TABLE tree (
    node_id integer PRIMARY KEY,
    parent_id integer REFERENCES tree,
    name text,
    ...
);
```

Для узла верхнего уровня `parent_id` будет равен `NULL`, пока записи с отличным от `NULL` `parent_id` будут ссылаться только на существующие строки таблицы.

Таблица может содержать несколько ограничений внешнего ключа. Это полезно для связи таблиц в отношении многие-ко-многим. Скажем, у вас есть таблицы продуктов и заказов, но вы хотите, чтобы один заказ мог содержать несколько продуктов (что невозможно в предыдущей схеме). Для этого вы можете использовать такую схему:

```
CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);

CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    shipping_address text,
    ...
);

CREATE TABLE order_items (
    product_no integer REFERENCES products,
    order_id integer REFERENCES orders,
    quantity integer,
    PRIMARY KEY (product_no, order_id)
);
```

Заметьте, что в последней таблице первичный ключ покрывает внешние ключи.

Мы знаем, что внешние ключи запрещают создание заказов, не относящихся ни к одному продукту. Но что делать, если после создания заказов с определённым продуктом мы захотим удалить его? SQL справится с этой ситуацией. Интуиция подсказывает следующие варианты поведения:

- Запретить удаление продукта
- Удалить также связанные заказы
- Что-то ещё?

Для иллюстрации давайте реализуем следующее поведение в вышеприведённом примере: при попытке удаления продукта, на который ссылаются заказы (через таблицу `order_items`), мы запрещаем эту операцию. Если же кто-то попытается удалить заказ, то удалится и его содержимое:

```
CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);

CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    shipping_address text,
    ...
);

CREATE TABLE order_items (
    product_no integer REFERENCES products ON DELETE RESTRICT,
    order_id integer REFERENCES orders ON DELETE CASCADE,
    quantity integer,
    PRIMARY KEY (product_no, order_id)
);
```

Ограничивающие и каскадные удаления — два наиболее распространённых варианта. `RESTRICT` предотвращает удаление связанной строки. `NO ACTION` означает, что если зависимые строки продолжают существовать при проверке ограничения, возникает ошибка (это поведение по умолчанию). (Главным отличием этих двух вариантов является то, что `NO ACTION` позволяет отложить проверку в процессе транзакции, а `RESTRICT` — нет.) `CASCADE` указывает, что при удалении связанных строк зависимые от них будут так же автоматически удалены. Есть ещё два варианта: `SET NULL` и `SET DEFAULT`. При удалении связанных строк они назначают зависимым столбцам в подчинён-

ной таблице значения NULL или значения по умолчанию, соответственно. Заметьте, что это не будет основанием для нарушения ограничений. Например, если в качестве действия задано SET DEFAULT, но значение по умолчанию не удовлетворяет ограничению внешнего ключа, операция закончится ошибкой.

Какой вариант действия ON DELETE выбрать — зависит от того, какие типы объектов представляются в связанных таблицах. Когда в подчинённой таблице представляется объект, который является составной частью сущности, представленной в главной таблице, и не может существовать независимо, уместно выбрать действие CASCADE. Если две таблицы представляют независимые объекты, более подходящим действием будет RESTRICT или NO ACTION; тогда приложению, которому действительно нужно удалить оба объекта, потребуется сделать это явно и выполнить две команды удаления. В приведённом выше примере позиции заказа являются частью заказа, и будет удобно, если они удалятся автоматически при удалении заказа. Но продукты и заказы — разные вещи, поэтому автоматическое удаление некоторых позиций заказов при удалении продуктов может быть неприемлемым. Если же отношение внешнего ключа представляет необязательную информацию, подходящим может быть действие SET NULL или SET DEFAULT. Например, если в таблице продуктов содержится ссылка на менеджера продукта, и запись менеджера удаляется, может быть полезным установить в поле менеджера продукта значение NULL или значение по умолчанию.

Действия SET NULL и SET DEFAULT могут принимать список столбцов, которые получают соответствующие значения. Обычно значения присваиваются всем столбцам ограничения внешнего ключа; присвоение для подмножества столбцов полезно только в некоторых особых случаях. Рассмотрите следующий пример:

```
CREATE TABLE tenants (
    tenant_id integer PRIMARY KEY
);

CREATE TABLE users (
    tenant_id integer REFERENCES tenants ON DELETE CASCADE,
    user_id integer NOT NULL,
    PRIMARY KEY (tenant_id, user_id)
);

CREATE TABLE posts (
    tenant_id integer REFERENCES tenants ON DELETE CASCADE,
    post_id integer NOT NULL,
    author_id integer,
    PRIMARY KEY (tenant_id, post_id),
    FOREIGN KEY (tenant_id, author_id) REFERENCES users ON DELETE SET NULL (author_id)
);
```

Без указания столбца внешний ключ также установил бы значение NULL для столбца tenant_id, но этот столбец является частью первичного ключа.

Аналогично указанию ON DELETE существует ON UPDATE, которое срабатывает при изменении заданного столбца. При этом возможные действия те же, за исключением того, что для SET NULL и SET DEFAULT нельзя задать списки столбцов. CASCADE в данном случае означает, что изменённые значения связанных столбцов будут скопированы в зависимые строки.

Обычно зависимая строка не должна удовлетворять ограничению внешнего ключа, если один из связанных столбцов содержит NULL. Если в объявление внешнего ключа добавлено MATCH FULL, строка будет удовлетворять ограничению, только если все связанные столбцы равны NULL (то есть при разных значениях (NULL и не NULL) гарантируется невыполнение ограничения MATCH FULL). Если вы хотите, чтобы зависимые строки не могли избежать и этого ограничения, объявите связанные столбцы как NOT NULL.

Внешний ключ должен ссылаться на столбцы, либо являющиеся первичным ключом, либо образующие ограничение уникальности, либо являющиеся столбцами из нечастичного уникального

индекса. Таким образом, для связанных столбцов всегда будет существовать индекс, а значит проверки наличия соответствия для связанной строки будут выполняться эффективно. Так как команды `DELETE` для строк главной таблицы или `UPDATE` для зависимых столбцов потребуют просканировать подчинённую таблицу и найти строки, ссылающиеся на старые значения, полезно будет иметь индекс и для подчинённых столбцов. Но это нужно не всегда, и создать соответствующий индекс можно по-разному, поэтому объявление внешнего ключа не создаёт автоматически индекс по связанным столбцам.

Подробнее об изменении и удалении данных рассказывается в [Главе 6](#). Вы также можете подробнее узнать о синтаксисе ограничений внешнего ключа в справке [CREATE TABLE](#).

5.4.6. Ограничения-исключения

Ограничения-исключения гарантируют, что при сравнении любых двух строк по указанным столбцам или выражениям с помощью заданных операторов, минимум одно из этих сравнений возвратит `false` или `NULL`. Записывается это так:

```
CREATE TABLE circles (
    c circle,
    EXCLUDE USING gist (c WITH &&)
);
```

Подробнее об этом см. [CREATE TABLE ... CONSTRAINT ... EXCLUDE](#).

При добавлении ограничения-исключения будет автоматически создан индекс того типа, который указан в объявлении ограничения.

5.5. Системные столбцы

В каждой таблице есть несколько *системных столбцов*, неявно определённых системой. Как следствие, их имена нельзя использовать в качестве имён пользовательских столбцов. (Заметьте, что это не зависит от того, является ли имя ключевым словом или нет; заключение имени в кавычки не поможет избежать этого ограничения.) Эти столбцы не должны вас беспокоить, вам лишь достаточно знать об их существовании.

`tableoid`

Идентификатор объекта для таблицы, содержащей строку. Этот столбец особенно полезен для запросов, имеющих дело с секционированными таблицами (см. [Раздел 5.11](#)) или иерархией наследования (см. [Раздел 5.10](#)), так как без него сложно определить, из какой таблицы выбрана строка. Связав `tableoid` со столбцом `oid` в таблице `pg_class`, можно будет получить имя таблицы.

`xmin`

Идентификатор (код) транзакции, добавившей строку этой версии. (Версия строки — это её индивидуальное состояние; при каждом изменении создаётся новая версия одной и той же логической строки.)

`cmin`

Номер команды (начиная с нуля) внутри транзакции, добавившей строку.

`xmax`

Идентификатор транзакции, удалившей строку, или 0 для неудалённой версии строки. Значение этого столбца может быть ненулевым и для видимой версии строки. Это обычно означает, что удаляющая транзакция ещё не была зафиксирована, или удаление было отменено.

`ctid`

Номер команды в удаляющей транзакции или ноль.

`ctid`

Физическое расположение данной версии строки в таблице. Заметьте, что хотя по `ctid` можно очень быстро найти версию строки, значение `ctid` изменится при выполнении `VACUUM FULL`. Таким образом, `ctid` нельзя применять в качестве долгосрочного идентификатора строки. Для идентификации логических строк следует использовать первичный ключ.

Идентификаторы транзакций также являются 32-битными. В долгоживущей базе данных они могут пойти по кругу. Это не критично при правильном обслуживании БД; подробнее об этом см. [Главу 23](#). Однако полагаться на уникальность кодов транзакций в течение длительного времени (при более чем миллиарде транзакций) не следует.

Идентификаторы команд также 32-битные. Это создаёт жёсткий лимит на 2^{32} (4 миллиарда) команд SQL в одной транзакции. На практике это не проблема — заметьте, что это лимит числа команд SQL, а не количества обрабатываемых строк. Кроме того, идентификатор получают только те команды, которые фактически изменяют содержимое базы данных.

5.6. Изменение таблиц

Если вы создали таблицы, а затем поняли, что допустили ошибку, или изменились требования вашего приложения, вы можете удалить её и создать заново. Но это будет неудобно, если таблица уже заполнена данными, или если на неё ссылаются другие объекты базы данных (например, по внешнему ключу). Поэтому PostgreSQL предоставляет набор команд для модификации таблиц. Заметьте, что это по сути отличается от изменения данных, содержащихся в таблице: здесь мы обсуждаем модификацию определения, или структуры, таблицы.

Вы можете:

- Добавлять столбцы
- Удалять столбцы
- Добавлять ограничения
- Удалять ограничения
- Изменять значения по умолчанию
- Изменять типы столбцов
- Переименовывать столбцы
- Переименовывать таблицы

Все эти действия выполняются с помощью команды [ALTER TABLE](#); подробнее о ней вы можете узнать в её справке.

5.6.1. Добавление столбца

Добавить столбец вы можете так:

```
ALTER TABLE products ADD COLUMN description text;
```

Новый столбец заполняется заданным для него значением по умолчанию (или значением NULL, если вы не добавите указание `DEFAULT`).

Подсказка

Начиная с PostgreSQL 11, добавление столбца с постоянным значением по умолчанию более не означает, что при выполнении команды `ALTER TABLE` будут изменены все строки таблицы. Вместо этого установленное значение по умолчанию будет просто выдаваться при следующем обращении к строкам, а сохранится в строках при перезаписи таблицы. Благодаря этому операция `ALTER TABLE` и с большими таблицами выполняется очень быстро.

Однако если значение по умолчанию изменчивое (например, это `clock_timestamp()`), в каждую строку нужно будет внести значение, вычисленное в момент выполнения `ALTER TABLE`.

Чтобы избежать потенциально длительной операции изменения всех строк, если вы планируете заполнить столбец в основном не значениями по умолчанию, лучше будет добавить столбец без значения по умолчанию, затем вставить требуемые значения с помощью `UPDATE`, а потом определить значение по умолчанию, как описано ниже.

При этом вы можете сразу определить ограничения столбца, используя обычный синтаксис:

```
ALTER TABLE products ADD COLUMN description text CHECK (description <> '');
```

На самом деле здесь можно использовать все конструкции, допустимые в определении столбца в команде `CREATE TABLE`. Помните однако, что значение по умолчанию должно удовлетворять данным ограничениям, чтобы операция `ADD` выполнялась успешно. Вы также можете сначала заполнить столбец правильно, а затем добавить ограничения (см. ниже).

5.6.2. Удаление столбца

Удалить столбец можно так:

```
ALTER TABLE products DROP COLUMN description;
```

Данные, которые были в этом столбце, исчезают. Вместе со столбцом удаляются и включающие его ограничения таблицы. Однако если на столбец ссылается ограничение внешнего ключа другой таблицы, Postgres Pro не удалит это ограничение неявно. Разрешить удаление всех зависящих от этого столбца объектов можно, добавив указание `CASCADE`:

```
ALTER TABLE products DROP COLUMN description CASCADE;
```

Общий механизм, стоящий за этим, описывается в [Разделе 5.14](#).

5.6.3. Добавление ограничения

Для добавления ограничения используется синтаксис ограничения таблицы. Например:

```
ALTER TABLE products ADD CHECK (name <> ' ');
ALTER TABLE products ADD CONSTRAINT some_name UNIQUE (product_no);
ALTER TABLE products ADD FOREIGN KEY (product_group_id) REFERENCES product_groups;
```

Чтобы добавить ограничение `NOT NULL`, которое нельзя записать в виде ограничения таблицы, используйте такой синтаксис:

```
ALTER TABLE products ALTER COLUMN product_no SET NOT NULL;
```

Ограничение проходит проверку автоматически и будет добавлено, только если ему удовлетворяют данные таблицы.

5.6.4. Удаление ограничения

Для удаления ограничения вы должны знать его имя. Если вы не присваивали ему имя, это неявно сделала система, и вы должны выяснить его. Здесь может быть полезна команда `psql \d имя_таблицы` (или другие программы, показывающие подробную информацию о таблицах). Зная имя, вы можете использовать команду:

```
ALTER TABLE products DROP CONSTRAINT some_name;
```

Как и при удалении столбца, если вы хотите удалить ограничение с зависимыми объектами, добавьте указание `CASCADE`. Примером такой зависимости может быть ограничение внешнего ключа, связанное со столбцами ограничения первичного ключа.

Так можно удалить ограничения любых типов, кроме `NOT NULL`. Чтобы удалить ограничение `NOT NULL`, используйте команду:

```
ALTER TABLE products ALTER COLUMN product_no DROP NOT NULL;
```

(Вспомните, что у ограничений NOT NULL нет имён.)

5.6.5. Изменение значения по умолчанию

Назначить столбцу новое значение по умолчанию можно так:

```
ALTER TABLE products ALTER COLUMN price SET DEFAULT 7.77;
```

Заметьте, что это никак не влияет на существующие строки таблицы, а просто задаёт значение по умолчанию для последующих команд INSERT.

Чтобы удалить значение по умолчанию, выполните:

```
ALTER TABLE products ALTER COLUMN price DROP DEFAULT;
```

При этом по сути значению по умолчанию просто присваивается NULL. Как следствие, ошибки не будет, если вы попытаетесь удалить значение по умолчанию, не определённое явно, так как неявно оно существует и равно NULL.

5.6.6. Изменение типа данных столбца

Чтобы преобразовать столбец в другой тип данных, используйте команду:

```
ALTER TABLE products ALTER COLUMN price TYPE numeric(10,2);
```

Она будет успешна, только если все существующие значения в столбце могут быть неявно приведены к новому типу. Если требуется более сложное преобразование, вы можете добавить указание USING, определяющее, как получить новые значения из старых.

Postgres Pro попытается также преобразовать к новому типу значение столбца по умолчанию (если оно определено) и все связанные с этим столбцом ограничения. Но преобразование может оказаться неправильным, и тогда вы получите неожиданные результаты. Поэтому обычно лучше удалить все ограничения столбца, перед тем как менять его тип, а затем воссоздать модифицированные должным образом ограничения.

5.6.7. Переименование столбца

Чтобы переименовать столбец, выполните:

```
ALTER TABLE products RENAME COLUMN product_no TO product_number;
```

5.6.8. Переименование таблицы

Таблицу можно переименовать так:

```
ALTER TABLE products RENAME TO items;
```

5.7. Права

Когда в базе данных создаётся объект, ему назначается владелец. Владелцем обычно становится роль, с которой был выполнен оператор создания. Для большинства типов объектов в исходном состоянии только владелец (или суперпользователь) может делать с объектом всё, что угодно. Чтобы разрешить использовать его другим ролям, нужно дать им *права*.

Существует несколько типов прав: SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER, CREATE, CONNECT, TEMPORARY, EXECUTE, USAGE, SET и ALTER SYSTEM. Набор прав, применимых к определённому объекту, зависит от типа объекта (таблица, функция и т. д.). Более подробно назначение этих прав описывается ниже. Как применяются эти права, вы также увидите в следующих разделах и главах.

Право изменять или удалять объект является неотъемлемым правом владельца объекта, его нельзя лишиться или передать другому. (Однако как и все другие права, это право наследуют члены роли-владельца, см. [Раздел 20.3](#).)

Объекту можно назначить нового владельца с помощью команды `ALTER` для соответствующего типа объекта, например:

```
ALTER TABLE имя_таблицы OWNER TO новый_владелец;
```

Суперпользователь может делать это без ограничений, а обычный пользователь — только если он является одновременно текущим владельцем объекта (или членом роли владельца) и членом новой роли.

Для назначения прав применяется команда `GRANT`. Например, если в базе данных есть роль `joe` и таблица `accounts`, право на изменение таблицы можно дать этой роли так:

```
GRANT UPDATE ON accounts TO joe;
```

Если вместо конкретного права написать `ALL`, роль получит все права, применимые для объекта этого типа.

Для назначения права всем ролям в системе можно использовать специальное имя «роли»: `PUBLIC`. Также для упрощения управления ролями, когда в базе данных есть множество пользователей, можно настроить «групповые» роли; подробнее об этом см. [Главу 20](#).

Чтобы лишить пользователей ранее данных им прав, используйте команду `REVOKE`:

```
REVOKE ALL ON accounts FROM PUBLIC;
```

Обычно распоряжаться правами может только владелец объекта (или суперпользователь). Однако возможно дать право доступа к объекту «с правом передачи», что позволит получившему такое право назначать его другим. Если такое право передачи впоследствии будет отозвано, то все, кто получил данное право доступа (непосредственно или по цепочке передачи), потеряют его. Подробнее об этом см. справку `GRANT` и `REVOKE`.

Владелец объекта может лишить себя обычных прав, например запретить не только всем остальным, но и себе, вносить изменения в таблицу. Однако владельцы всегда имеют возможность управлять правами, так что они могут в любом случае вернуть себе права, которых лишились.

Все существующие права перечислены ниже:

`SELECT`

Позволяет выполнять `SELECT` для любого столбца или перечисленных столбцов в заданной таблице, представлении, матпредставлении или другом объекте табличного вида. Также позволяет выполнять `COPY TO`. Помимо этого, данное право требуется для обращения к существующим значениям столбцов в `UPDATE`, `DELETE` или `MERGE`. Для последовательностей это право позволяет пользоваться функцией `currval`. Для больших объектов оно позволяет читать содержимое объекта.

`INSERT`

Позволяет вставлять с помощью `INSERT` строки в заданную таблицу, представление и т. п. Может назначаться для отдельных столбцов; в этом случае только этим столбцам можно присваивать значения в команде `INSERT` (другие столбцы получают значения по умолчанию). Также позволяет выполнять `COPY FROM`.

`UPDATE`

Позволяет изменять с помощью `UPDATE` данные во всех, либо только перечисленных, столбцах в заданной таблице, представлении и т. п. (На практике для любой нетривиальной команды `UPDATE` потребуется и право `SELECT`, так как она должна обратиться к столбцам таблицы, чтобы определить, какие строки подлежат изменению, и/или вычислить новые значения столбцов.) Для `SELECT ... FOR UPDATE` и `SELECT ... FOR SHARE` также требуется иметь это право как минимум для одного столбца, помимо права `SELECT`. Для последовательностей это право позволяет пользоваться функциями `nextval` и `setval`. Для больших объектов это право позволяет записывать данные в объект или обрезать его.

DELETE

Позволяет удалять с помощью команды `DELETE` строки из таблицы, представления и т. п. (На практике для любой нетривиальной команды `DELETE` потребуется также право `SELECT`, так как она должна обратиться к колонкам таблицы, чтобы определить, какие строки подлежат удалению.)

TRUNCATE

Позволяет опустошать таблицу с помощью `TRUNCATE`.

REFERENCES

Позволяет создавать ограничение внешнего ключа, обращающееся к таблице или определённым столбцам таблицы.

TRIGGER

Позволяет создавать триггер для таблицы, представления и т. п.

CREATE

Для баз данных это право позволяет создавать схемы и публикации, а также устанавливать доверенные расширения в конкретной базе.

Для схем это право позволяет создавать новые объекты в заданной схеме. Чтобы переименовать существующий объект, необходимо быть его владельцем и иметь это право для схемы, содержащей его.

Для табличных пространств это право позволяет создавать таблицы, индексы и временные файлы в определённом табличном пространстве, а также создавать базы данных, для которых это пространство будет основным.

Заметьте, что факт лишения пользователя этого права не влияет на существование или размещение существующих объектов.

CONNECT

Позволяет подключаться к базе данных. Это право проверяется при установлении соединения (в дополнение к условиям, определённым в конфигурации `pg_hba.conf`).

TEMPORARY

Позволяет создавать временные таблицы в определённой базе данных.

EXECUTE

Позволяет вызывать функцию или процедуру, в том числе использовать любые операторы, реализованные данной функцией. Это единственный тип прав, применимый к функциям и процедурам.

USAGE

Для процедурных языков это право позволяет создавать функции на определённом языке. Это единственный тип прав, применимый к процедурным языкам.

Для схем это право даёт доступ к содержащимся в них объектам (предполагается, что при этом имеются права, необходимые для доступа к самим объектам). По сути это право позволяет субъекту «просматривать» объекты внутри схемы. Без этого разрешения имена объектов всё же можно будет узнать, например, обратившись к системным каталогам. Кроме того, если отозвать это право, в существующих сеансах могут оказаться операторы, для которых просмотр имён объектов был выполнен ранее, так что это право не позволяет абсолютно надёжно перекрыть доступ к объектам.

Для последовательностей это право позволяет использовать функции `currval` и `nextval`.

Для типов и доменов это право позволяет использовать заданный тип или домен при создании таблиц, функций или других объектов схемы. (Заметьте, что это право не ограничивает общее «использование» типа, например обращение к значениям типа в запросах. Без этого права нельзя только создавать объекты, зависящие от заданного типа. Основное предназначение этого права в том, чтобы ограничить круг пользователей, способных создавать зависимости от типа, которые могут впоследствии помешать владельцу типа изменить его.)

Для обёрток сторонних данных это право позволяет создавать использующие их определения сторонних серверов.

Для сторонних серверов это право позволяет создавать использующие их сторонние таблицы. Наделённые этим правом могут также создавать, модифицировать или удалять собственные сопоставления пользователей, связанные с определённым сервером.

SET

Позволяет задать для параметра конфигурации сервера новое значение в текущем сеансе. (Хотя это право может быть предоставлено для любого параметра, оно имеет смысл только для тех параметров, для которых обычно требуются права суперпользователя.)

ALTER SYSTEM

Позволяет присваивать значение параметру конфигурации сервера, используя команду [ALTER SYSTEM](#).

Права, требующиеся для других команд, указаны на страницах справки этих команд.

Postgres Pro по умолчанию назначает роли `PUBLIC` права для некоторых типов объектов, когда эти объекты создаются. Для таблиц, столбцов, последовательностей, обёрток сторонних данных, сторонних серверов, больших объектов, схем, табличных пространств или параметров конфигурации `PUBLIC` по умолчанию никаких прав не получает. Для других типов объектов `PUBLIC` получает следующие права по умолчанию: `CONNECT` и `TEMPORARY` (создание временных таблиц) для баз данных; `EXECUTE` — для функций и процедур; `USAGE` — для языков и типов данных (включая домены). Владелец объекта, конечно же, может отозвать (посредством `REVOKE`) как явно назначенные права, так и права по умолчанию. (Для максимальной безопасности команду `REVOKE` нужно выполнять в транзакции, создающей объект; тогда не образуется окно, в котором другой пользователь сможет обратиться к объекту.) Кроме того, эти изначально назначаемые права по умолчанию можно перепределить, воспользовавшись командой [ALTER DEFAULT PRIVILEGES](#).

В [Таблице 5.1](#) показаны однобуквенные сокращения, которыми обозначаются эти права в списках `ACL` (Access Control List, Список контроля доступа). Вы увидите эти сокращения в выводе перечисленных ниже команд `psql` или в столбцах `ACL` в системных каталогах.

Таблица 5.1. Сокращённые обозначения прав в ACL

Право	Сокращение	Применимые типы объектов
SELECT	r («read», чтение)	LARGE OBJECT, SEQUENCE, TABLE (и объекты, подобным таблицам), столбец таблицы
INSERT	a («append», добавление)	TABLE, столбец таблицы
UPDATE	w («write», запись)	LARGE OBJECT, SEQUENCE, TABLE, столбец таблицы
DELETE	d	TABLE
TRUNCATE	D	TABLE
REFERENCES	x	TABLE, столбец таблицы

Право	Сокращение	Применимые типы объектов
TRIGGER	t	TABLE
CREATE	C	DATABASE, SCHEMA, TABLESPACE
CONNECT	c	DATABASE
TEMPORARY	T	DATABASE
EXECUTE	X	FUNCTION, PROCEDURE
USAGE	U	DOMAIN, FOREIGN DATA WRAPPER, FOREIGN SERVER, LANGUAGE, SCHEMA, SEQUENCE, TYPE
SET	s	PARAMETER
ALTER SYSTEM	A	PARAMETER

В Таблица 5.2 для каждого типа SQL-объекта показаны относящиеся к нему права, с использованием приведённых выше сокращений. Также в ней для каждого типа приведена команда `psql`, которая позволяет узнать, какие права назначены для объекта этого типа.

Таблица 5.2. Сводка прав доступа

Тип объекта	Все права	Права PUBLIC по умолчанию	Команда psql
DATABASE	CTc	Tc	\l
DOMAIN	U	U	\dD+
FUNCTION или PROCEDURE	X	X	\df+
FOREIGN DATA WRAPPER	U	нет	\dew+
FOREIGN SERVER	U	нет	\des+
LANGUAGE	U	U	\dL+
LARGE OBJECT	rw	нет	\dl+
PARAMETER	sA	нет	\dconfig+
SCHEMA	UC	нет	\dn+
SEQUENCE	rwU	нет	\dp
TABLE (и объекты, подобные таблицам)	arwdDxt	нет	\dp
Столбец таблицы	arwx	нет	\dp
TABLESPACE	C	нет	\db+
TYPE	U	U	\dT+

Права, назначенные для определённого объекта, выводятся в виде списка элементов `aclitem`, где каждый `aclitem` имеет такой формат:

правообладатель=аббревиатура-права[].../праводатель*

Каждый `aclitem` содержит разрешения, которые были предоставлены субъекту определённым праводателем. Конкретные права представлены однобуквенными аббревиатурами из таблицы Таблица 5.1 с добавлением *, если право было выдано с параметром передачи. Например, запись `calvin=r*w/hobbes` означает, что роль `calvin` имеет право `SELECT` (`r`) с возможностью передачи (*), а также непередаваемое право `UPDATE` (`w`), и оба эти права даны ему ролью `hobbes`. Если `calvin` имеет для этого объекта и другие права, предоставленные ему другим праводателем, они выводятся в отдельном элементе `aclitem`. Пустое поле правообладателя в `aclitem` соответствует роли `PUBLIC`.

Например, предположим, что пользователь `miriam` создаёт таблицы `mytable` и выполняет:


```
GRANT SELECT ON mytable TO PUBLIC;
GRANT SELECT, UPDATE, INSERT ON mytable TO admin;
GRANT SELECT (col1), UPDATE (col1) ON mytable TO miriam_rw;
```

Тогда команда `\dp` в `psql` покажет:

```
=> \dp mytable
```

Schema	Name	Type	Access privileges	Column privileges	Policies
public	mytable	table	miriam=arwdDxt/miriam+ =r/miriam admin=arw/miriam	col1: miriam_rw=rw/miriam	+

(1 row)

Если столбец «Права доступа» (Access privileges) для данного объекта пуст, это значит, что для объекта действуют стандартные права (то есть запись прав в соответствующем каталоге содержит NULL). Права по умолчанию всегда включают все права для владельца и могут также включать некоторые права для PUBLIC в зависимости от типа объекта, как разъяснялось выше. Первая команда GRANT или REVOKE для объекта приводит к созданию записи прав по умолчанию (например, {miriam=arwdDxt/miriam}), а затем изменяет эту запись в соответствии с заданным запросом. Подобным образом, строки, показанные в столбце «Права доступа к столбцам» (Column privileges), выводятся только для столбцов с нестандартными правами доступа. (Заметьте, что в данном контексте под «стандартными правами» всегда подразумевается встроенный набор прав, предопределённый для типа объекта. Если с объектом связан набор прав по умолчанию, полученный после изменения в результате ALTER DEFAULT PRIVILEGES, изменённые права будут всегда выводиться явно, показывая эффект команды ALTER.)

Заметьте, что право распоряжения правами, которое имеет владелец, не отмечается в выводимой сводке. Знаком * отмечаются только те права с правом передачи, которые были явно назначены кому-либо.

5.8. Политики защиты строк

В дополнение к стандартной [системе прав SQL](#), управляемой командой [GRANT](#), на уровне таблиц можно определить *политики защиты строк*, ограничивающие для пользователей наборы строк, которые могут быть возвращены обычными запросами или добавлены, изменены и удалены командами, изменяющими данные. Это называется также *защитой на уровне строк* (RLS, Row-Level Security). По умолчанию таблицы не имеют политик, так что если система прав SQL разрешает пользователю доступ к таблице, все строки в ней одинаково доступны для чтения или изменения.

Когда для таблицы включается защита строк (с помощью команды [ALTER TABLE ... ENABLE ROW LEVEL SECURITY](#)), все обычные запросы к таблице на выборку или модификацию строк должны разрешаться политикой защиты строк. (Однако на владельца таблицы такие политики обычно не действуют.) Если политика для таблицы не определена, применяется политика запрета по умолчанию, так что никакие строки в этой таблице нельзя увидеть или модифицировать. На операции с таблицей в целом, такие как TRUNCATE и REFERENCES, защита строк не распространяется.

Политики защиты строк могут применяться к определённым командам и/или ролям. Политику можно определить как применяемую к командам ALL (всем), либо SELECT, INSERT, UPDATE и DELETE. Кроме того, политику можно связать с несколькими ролями, при этом действуют обычные правила членства и наследования.

Чтобы определить, какие строки будут видимыми или могут изменяться в таблице, для политики задаётся выражение, возвращающее логический результат. Это выражение будет вычисляться для каждой строки перед другими условиями или функциями, поступающими из запроса пользователя. (Единственным исключением из этого правила являются *герметичные функции*, которые гарантированно не допускают утечки информации; оптимизатор может решить выполнить эти функции до проверок защиты строк.) Строки, для которых это выражение возвращает не true, обрабаты-

ваться не будут. Чтобы независимо управлять набором строк, которые можно видеть, и набором строк, которые можно модифицировать, в политике можно задать отдельные выражения. Выражения политик обрабатываются в составе запроса с правами исполняющего его пользователя, но для обращения к данным, недоступным этому пользователю, в этих выражениях могут применяться функции, определяющие контекст безопасности.

Суперпользователи и роли с атрибутом `BYPASSRLS` всегда обращаются к таблице, минуя систему защиты строк. На владельца таблицы защита строк тоже не действует, хотя он может включить её для себя принудительно, выполнив команду `ALTER TABLE ... FORCE ROW LEVEL SECURITY`.

Неотъемлемое право включать или отключать защиту строк, а также определять политики для таблицы, имеет только её владелец.

Для создания политик предназначена команда `CREATE POLICY`, для изменения — `ALTER POLICY`, а для удаления — `DROP POLICY`. Чтобы включить или отключить защиту строк для определённой таблицы, воспользуйтесь командой `ALTER TABLE`.

Каждой политике назначается имя, при этом для одной таблицы можно определить несколько политик. Так как политики привязаны к таблицам, каждая политика для таблицы должна иметь уникальное имя. В разных таблицах политики могут иметь одинаковые имена.

Когда к определённому запросу применяются несколько политик, они объединяются либо логическим сложением (если политики разрешительные (по умолчанию)), либо умножением (если политики ограничительные). Это подобно тому, как некоторая роль получает права всех ролей, в которые она включена. Разрешительные и ограничительные политики рассматриваются ниже.

В качестве простого примера, создать политику для отношения `account`, позволяющую только членам роли `managers` обращаться к строкам отношения и при этом только к своим, можно так:

```
CREATE TABLE accounts (manager text, company text, contact_email text);

ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;

CREATE POLICY account_managers ON accounts TO managers
    USING (manager = current_user);
```

Эта политика неявно подразумевает и предложение `WITH CHECK`, идентичное предложению `USING`, поэтому указанное ограничение применяется и к строкам, выбираемым командой (так что один менеджер не может выполнить `SELECT`, `UPDATE` или `DELETE` для существующих строк, принадлежащих другому), и к строкам, изменяемым командой (так что командами `INSERT` и `UPDATE` нельзя создать строки, принадлежащие другому менеджеру).

Если роль не задана, либо задано специальное имя пользователя `PUBLIC`, политика применяется ко всем пользователям в данной системе. Чтобы все пользователи могли обратиться только к собственной строке в таблице `users`, можно применить простую политику:

```
CREATE POLICY user_policy ON users
    USING (user_name = current_user);
```

Это работает подобно предыдущему примеру.

Чтобы определить для строк, добавляемых в таблицу, отдельную политику, отличную от политики, ограничивающей видимые строки, можно скомбинировать несколько политик. Следующая пара политик позволит всем пользователям видеть все строки в таблице `users`, но изменять только свою собственную:

```
CREATE POLICY user_sel_policy ON users
    FOR SELECT
    USING (true);
```

```
CREATE POLICY user_mod_policy ON users
    USING (user_name = current_user);
```

Для команды `SELECT` эти две политики объединяются операцией `OR`, так что в итоге это позволяет выбирать все строки. Для команд других типов применяется только вторая политика, и эффект тот же, что и раньше.

Защиту строк можно отключить так же командой `ALTER TABLE`. При отключении защиты, политики, определённые для таблицы, не удаляются, а просто игнорируются. В результате в таблице будут видны и могут модифицироваться все строки, с учётом ограничений стандартной системы прав SQL.

Ниже показан развёрнутый пример того, как этот механизм защиты можно применять в производственной среде. Таблица `passwd` имитирует файл паролей в Unix:

```
-- Простой пример на базе файла passwd
CREATE TABLE passwd (
    user_name          text UNIQUE NOT NULL,
    pwhash             text,
    uid               int  PRIMARY KEY,
    gid               int  NOT NULL,
    real_name         text NOT NULL,
    home_phone        text,
    extra_info        text,
    home_dir          text NOT NULL,
    shell             text NOT NULL
);

CREATE ROLE admin;  -- Администратор
CREATE ROLE bob;    -- Обычный пользователь
CREATE ROLE alice;  -- Обычный пользователь

-- Наполнение таблицы
INSERT INTO passwd VALUES
    ('admin', 'xxx', 0, 0, 'Admin', '111-222-3333', null, '/root', '/bin/dash');
INSERT INTO passwd VALUES
    ('bob', 'xxx', 1, 1, 'Bob', '123-456-7890', null, '/home/bob', '/bin/zsh');
INSERT INTO passwd VALUES
    ('alice', 'xxx', 2, 1, 'Alice', '098-765-4321', null, '/home/alice', '/bin/zsh');

-- Необходимо включить для этой таблицы защиту на уровне строк
ALTER TABLE passwd ENABLE ROW LEVEL SECURITY;

-- Создание политик
-- Администратор может видеть и добавлять любые строки
CREATE POLICY admin_all ON passwd TO admin USING (true) WITH CHECK (true);
-- Обычные пользователи могут видеть все строки
CREATE POLICY all_view ON passwd FOR SELECT USING (true);
-- Обычные пользователи могут изменять собственные данные, но
-- не могут задать произвольную оболочку входа
CREATE POLICY user_mod ON passwd FOR UPDATE
    USING (current_user = user_name)
    WITH CHECK (
        current_user = user_name AND
        shell IN ('/bin/bash', '/bin/sh', '/bin/dash', '/bin/zsh', '/bin/tcsh')
    );

-- Администраторы получают все обычные права
GRANT SELECT, INSERT, UPDATE, DELETE ON passwd TO admin;
```

```
-- Пользователям разрешается чтение только общедоступных столбцов
GRANT SELECT
    (user_name, uid, gid, real_name, home_phone, extra_info, home_dir, shell)
    ON passwd TO public;
-- Пользователям разрешается изменение определённых столбцов
GRANT UPDATE
    (pwhash, real_name, home_phone, extra_info, shell)
    ON passwd TO public;
```

Как и любые средства защиты, важно проверить политики, и убедиться в том, что они работают ожидаемым образом. Применительно к предыдущему примеру, эти команды показывают, что система разрешений работает корректно.

```
-- Администратор может видеть все строки и поля
postgres=> set role admin;
SET
postgres=> table passwd;
 user_name | pwhash | uid | gid | real_name | home_phone | extra_info | home_dir | shell
-----+-----+-----+-----+-----+-----+-----+-----+-----
admin      | xxx    | 0   | 0   | Admin     | 111-222-3333 |           | /root    | /bin/dash
bob        | xxx    | 1   | 1   | Bob       | 123-456-7890 |           | /home/bob | /bin/zsh
alice      | xxx    | 2   | 1   | Alice     | 098-765-4321 |           | /home/alice | /bin/zsh
(3 rows)
```

```
-- Проверим, что может делать Алиса
postgres=> set role alice;
SET
postgres=> table passwd;
ERROR: permission denied for table passwd
postgres=> select user_name,real_name,home_phone,extra_info,home_dir,shell from passwd;
 user_name | real_name | home_phone | extra_info | home_dir | shell
-----+-----+-----+-----+-----+-----
admin      | Admin     | 111-222-3333 |           | /root    | /bin/dash
bob        | Bob       | 123-456-7890 |           | /home/bob | /bin/zsh
alice      | Alice     | 098-765-4321 |           | /home/alice | /bin/zsh
(3 rows)
```

```
postgres=> update passwd set user_name = 'joe';
ERROR: permission denied for table passwd
-- Алиса может изменить своё имя (поле real_name), но не имя кого-либо другого
postgres=> update passwd set real_name = 'Alice Doe';
UPDATE 1
postgres=> update passwd set real_name = 'John Doe' where user_name = 'admin';
UPDATE 0
postgres=> update passwd set shell = '/bin/xx';
ERROR: new row violates WITH CHECK OPTION for "passwd"
postgres=> delete from passwd;
ERROR: permission denied for table passwd
postgres=> insert into passwd (user_name) values ('xxx');
ERROR: permission denied for table passwd
-- Алиса может изменить собственный пароль; попытки поменять другие пароли RLS просто игнорирует
postgres=> update passwd set pwhash = 'abc';
UPDATE 1
```

Все политики, создаваемые до этого, были разрешительными, что значит, что при применении нескольких политик они объединялись логическим оператором «ИЛИ». Хотя можно создать такие разрешительные политики, которые будут только разрешать доступ к строкам при определённых условиях, может быть проще скомбинировать разрешительные политики с ограничительными (которым должны удовлетворять записи и которые объединяются логическим оператором «И»). В развитие предыдущего примера мы добавим ограничительную политику, разрешающую администратору, подключённому через локальный сокет Unix, обращаться к записям таблицы `passwd`:

```
CREATE POLICY admin_local_only ON passwd AS RESTRICTIVE TO admin
    USING (pg_catalog.inet_client_addr() IS NULL);
```

Затем мы можем убедиться, что администратор, подключённый по сети, не увидит никаких записей, благодаря этой ограничительной политике:

```
=> SELECT current_user;
current_user
-----
admin
(1 row)

=> select inet_client_addr();
inet_client_addr
-----
127.0.0.1
(1 row)

=> TABLE passwd;
user_name | pwhash | uid | gid | real_name | home_phone | extra_info | home_dir |
shell
-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)

=> UPDATE passwd set pwhash = NULL;
UPDATE 0
```

На проверки ссылочной целостности, например, на ограничения уникальности и внешние ключи, защита строк никогда не распространяется, чтобы не нарушалась целостность данных. Поэтому организацию и политики защиты на уровне строк необходимо тщательно прорабатывать, чтобы не возникли «скрытые каналы» утечки информации через эти проверки.

В некоторых случаях важно, чтобы защита на уровне строк, наоборот, не действовала. Например, резервное копирование может оказаться провальным, если механизм защиты на уровне строк молча не даст скопировать какие-либо строки. В таком случае вы можете установить для параметра конфигурации `row_security` значение `off`. Это само по себе не отключит защиту строк; при этом просто будет выдана ошибка, если результаты запроса отфильтруются политикой, с тем чтобы можно было изучить причину ошибки и устранить её.

В приведённых выше примерах выражения политики учитывали только текущие значения в запрашиваемой или изменяемой строке. Это самый простой и наиболее эффективный по скорости вариант; по возможности реализацию защиты строк следует проектировать именно так. Если же для принятия решения о доступе необходимо обращаться к другим строкам или другим таблицам, это можно осуществить, применяя в выражениях политик вложенные `SELECT` или функции, содержащие `SELECT`. Однако учтите, что при такой реализации возможны условия гонки, что чревато утечкой информации, если не принять меры предосторожности. Например, рассмотрим следующую конструкцию таблиц:

```
-- определение групп привилегий
CREATE TABLE groups (group_id int PRIMARY KEY,
                    group_name text NOT NULL);
```

```
INSERT INTO groups VALUES
  (1, 'low'),
  (2, 'medium'),
  (5, 'high');

GRANT ALL ON groups TO alice; -- alice является администратором
GRANT SELECT ON groups TO public;

-- определение уровней привилегий для пользователей
CREATE TABLE users (user_name text PRIMARY KEY,
                     group_id int NOT NULL REFERENCES groups);

INSERT INTO users VALUES
  ('alice', 5),
  ('bob', 2),
  ('mallory', 2);

GRANT ALL ON users TO alice;
GRANT SELECT ON users TO public;

-- таблица, содержащая защищаемую информацию
CREATE TABLE information (info text,
                          group_id int NOT NULL REFERENCES groups);

INSERT INTO information VALUES
  ('barely secret', 1),
  ('slightly secret', 2),
  ('very secret', 5);

ALTER TABLE information ENABLE ROW LEVEL SECURITY;

-- строка должна быть доступна для чтения/изменения пользователям с group_id,
-- большим или равным group_id данной строки
CREATE POLICY fp_s ON information FOR SELECT
  USING (group_id <= (SELECT group_id FROM users WHERE user_name = current_user));
CREATE POLICY fp_u ON information FOR UPDATE
  USING (group_id <= (SELECT group_id FROM users WHERE user_name = current_user));

-- мы защищаем таблицу с информацией, полагаясь только на RLS
GRANT ALL ON information TO public;
```

Теперь предположим, что Алиса (роль `alice`) желает записать «слегка секретную» информацию, но при этом не хочет давать `mallory` доступ к ней. Она делает следующее:

```
BEGIN;
UPDATE users SET group_id = 1 WHERE user_name = 'mallory';
UPDATE information SET info = 'secret from mallory' WHERE group_id = 2;
COMMIT;
```

На первый взгляд всё нормально; `mallory` ни при каких условиях не должна увидеть строку «secret from mallory». Однако здесь возможно условие гонки. Если Мэллори (роль `mallory`) параллельно выполняет, скажем:

```
SELECT * FROM information WHERE group_id = 2 FOR UPDATE;
```

и её транзакция в режиме `READ COMMITTED`, она сможет увидеть «secret from mallory». Это произойдёт, если её транзакция дойдёт до строки `information` сразу после того, как эту строку изменит Алиса (роль `alice`). Она заблокируется, ожидая фиксирования транзакции Алисы, а затем прочитает изменённое содержимое строки благодаря предложению `FOR UPDATE`. Однако при этом из-

менённое содержимое `users` *не* будет прочитано неявным запросом `SELECT`, так как этот вложенный `SELECT` выполняется без указания `FOR UPDATE`; вместо этого строка `users` читается из снимка, полученного в начале запроса. Таким образом, выражение политики проверяет старое значение уровня привилегий пользователя `mallory` и позволяет ей видеть изменённую строку.

Обойти эту проблему можно несколькими способами. Первое простое решение заключается в использовании `SELECT ... FOR SHARE` во вложенных запросах `SELECT` в политиках защиты строк. Однако для этого потребуется давать затронутым пользователям право `UPDATE` в целевой таблице (здесь `users`), что может быть нежелательно. (Хотя можно применить ещё одну политику защиты строк, чтобы они не могли практически воспользоваться этим правилом; либо поместить вложенный `SELECT` в функцию, определяющую контекст безопасности.) Кроме этого, активное использование блокировок строк в целевой таблице может повлечь проблемы с производительностью, особенно при частых изменениях. Другое решение, практичное, если целевая таблица изменяется нечасто, заключается в блокировке целевой таблицы в режиме `ACCESS EXCLUSIVE` при изменении, чтобы никакие параллельные транзакции не видели старые значения строк. Либо можно просто дождаться завершения всех параллельных транзакций после изменения в целевой таблице, прежде чем вносить изменения, рассчитанные на новые условия безопасности.

За дополнительными подробностями обратитесь к [CREATE POLICY](#) и [ALTER TABLE](#).

5.9. Схемы

Кластер баз данных Postgres Pro содержит один или несколько именованных экземпляров баз. На уровне кластера создаются роли и некоторые другие объекты. При этом в рамках одного подключения к серверу можно обращаться к данным только одной базы — той, что была выбрана при установлении соединения.

Примечание

Пользователи кластера не обязательно будут иметь доступ ко всем базам данных этого кластера. Тот факт, что роли создаются на уровне кластера, означает только то, что в кластере не может быть двух ролей `joe` в разных базах данных, хотя система позволяет ограничить доступ `joe` только некоторыми базами данных.

База данных содержит одну или несколько именованных схем, которые в свою очередь содержат таблицы. Схемы также содержат именованные объекты других видов, включая типы данных, функции и операторы. Одно и то же имя объекта можно свободно использовать в разных схемах, например `schema1`, и `myschema` могут содержать таблицы с именем `mytable`. В отличие от баз данных, схемы не ограничивают доступ к данным: пользователи могут обращаться к объектам в любой схеме текущей базы данных, если им назначены соответствующие права.

Есть несколько возможных объяснений, для чего стоит применять схемы:

- Чтобы одну базу данных могли использовать несколько пользователей, независимо друг от друга.
- Чтобы объединить объекты базы данных в логические группы для облегчения управления ими.
- Чтобы в одной базе сосуществовали разные приложения, и при этом не возникало конфликтов имён.

Схемы в некотором смысле подобны каталогам в операционной системе, но они не могут быть вложенными.

5.9.1. Создание схемы

Для создания схемы используется команда [CREATE SCHEMA](#). При этом вы определяете имя схемы по своему выбору, например так:

```
CREATE SCHEMA myschema;
```

Чтобы создать объекты в схеме или обратиться к ним, указывайте *полное имя*, состоящее из имён схемы и объекта, разделённых точкой:

схема.таблица

Этот синтаксис работает везде, где ожидается имя таблицы, включая команды модификации таблицы и команды обработки данных, обсуждаемые в следующих главах. (Для краткости мы будем говорить только о таблицах, но всё это распространяется и на другие типы именованных объектов, например, типы и функции.)

Есть ещё более общий синтаксис

база_данных.схема.таблица

но в настоящее время он поддерживается только для формального соответствия стандарту SQL. Если вы указываете базу данных, это может быть только база данных, к которой вы подключены.

Таким образом, создать таблицу в новой схеме можно так:

```
CREATE TABLE myschema.mytable (
    ...
);
```

Чтобы удалить пустую схему (не содержащую объектов), выполните:

```
DROP SCHEMA myschema;
```

Удалить схему со всеми содержащимися в ней объектами можно так:

```
DROP SCHEMA myschema CASCADE;
```

Стоящий за этим общий механизм описан в [Разделе 5.14](#).

Часто бывает нужно создать схему, владельцем которой будет другой пользователь (это один из способов ограничения пользователей пространствами имён). Сделать это можно так:

```
CREATE SCHEMA имя_схемы AUTHORIZATION имя_пользователя;
```

Вы даже можете опустить имя схемы, в этом случае именем схемы станет имя пользователя. Как это можно применять, описано в [Подразделе 5.9.6](#).

Схемы с именами, начинающимися с `pg_`, являются системными; пользователям не разрешено использовать такие имена.

5.9.2. Схема public

До этого мы создавали таблицы, не указывая никакие имена схем. По умолчанию такие таблицы (и другие объекты) автоматически помещаются в схему «public». Она содержится во всех создаваемых базах данных. Таким образом, команда:

```
CREATE TABLE products ( ... );
```

эквивалентна:

```
CREATE TABLE public.products ( ... );
```

5.9.3. Путь поиска схемы

Везде писать полные имена утомительно, и часто всё равно лучше не привязывать приложения к конкретной схеме. Поэтому к таблицам обычно обращаются по *неполному имени*, состоящему просто из имени таблицы. Система определяет, какая именно таблица подразумевается, исполь-

зую *путь поиска*, который представляет собой список просматриваемых схем. Подразумеваемой таблицей считается первая подходящая таблица, найденная в схемах пути. Если подходящая таблица не найдена, возникает ошибка, даже если таблица с таким именем есть в других схемах базы данных.

Возможность создавать одноимённые объекты в разных схемах усложняет написание запросов, которые должны всегда обращаться к конкретным объектам. Это также потенциально позволяет пользователям влиять на поведение запросов других пользователей, злонамеренно или случайно. Ввиду преобладания неполных имён в запросах и их использования внутри Postgres Pro, добавить схему в `search_path` — по сути значит доверять всем пользователям, имеющим право `CREATE` в этой схеме. Когда вы выполняете обычный запрос, злонамеренный пользователь может создать объекты в схеме, включённой в ваш путь поиска, и таким образом перехватывать управление и выполнять произвольные функции SQL как если бы их выполняли вы.

Первая схема в пути поиска называется текущей. Эта схема будет использоваться не только при поиске, но и при создании объектов — она будет включать таблицы, созданные командой `CREATE TABLE` без указания схемы.

Чтобы узнать текущий тип поиска, выполните следующую команду:

```
SHOW search_path;
```

В конфигурации по умолчанию она возвращает:

```
search_path
-----
"$user", public
```

Первый элемент ссылается на схему с именем текущего пользователя. Если такой схемы не существует, ссылка на неё игнорируется. Второй элемент ссылается на схему `public`, которую мы уже видели.

Первая существующая схема в пути поиска также считается схемой по умолчанию для новых объектов. Именно поэтому по умолчанию объекты создаются в схеме `public`. При указании неполной ссылки на объект в любом контексте (при модификации таблиц, изменении данных или в запросах) система просматривает путь поиска, пока не найдёт соответствующий объект. Таким образом, в конфигурации по умолчанию неполные имена могут относиться только к объектам в схеме `public`.

Чтобы добавить в путь нашу новую схему, мы выполняем:

```
SET search_path TO myschema,public;
```

(Мы опускаем компонент `$user`, так как здесь в нём нет необходимости.) Теперь мы можем обращаться к таблице без указания схемы:

```
DROP TABLE mytable;
```

И так как `myschema` — первый элемент в пути, новые объекты будут по умолчанию создаваться в этой схеме.

Мы можем также написать:

```
SET search_path TO myschema;
```

Тогда мы больше не сможем обращаться к схеме `public`, не написав полное имя объекта. Единственное, что отличает схему `public` от других, это то, что она существует по умолчанию, хотя её так же можно удалить.

В [Разделе 9.26](#) вы узнаете, как ещё можно манипулировать путём поиска схем.

Как и для имён таблиц, путь поиска аналогично работает для имён типов данных, имён функций и имён операторов. Имена типов данных и функций можно записать в полном виде так же, как

и имена таблиц. Если же вам нужно использовать в выражении полное имя оператора, для этого есть специальный способ — вы должны написать:

```
OPERATOR (схема.оператор)
```

Такая запись необходима для избежания синтаксической неоднозначности. Пример такого выражения:

```
SELECT 3 OPERATOR(pg_catalog.+) 4;
```

На практике пользователи часто полагаются на путь поиска, чтобы не приходилось писать такие замысловатые конструкции.

5.9.4. Схемы и права

По умолчанию пользователь не может обращаться к объектам в чужих схемах. Чтобы изменить это, владелец схемы должен дать пользователю право `USAGE` для данной схемы. По умолчанию все пользователи имеют это право для схемы `public`. Чтобы пользователи могли использовать объекты схемы, может понадобиться назначить дополнительные права на уровне объектов.

Пользователю также можно разрешить создавать объекты в схеме, не принадлежащей ему. Для этого ему нужно дать право `CREATE` в требуемой схеме. В базах данных, обновлённых с PostgreSQL 14 или более ранней версии, все имеют это право в схеме `public`. Некоторые [шаблоны использования](#) требуют запретить это:

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

(Первое слово «`public`» обозначает схему, а второе означает «каждый пользователь». В первом случае это идентификатор, а во втором — ключевое слово, поэтому они написаны в разном регистре; вспомните указания из [Подраздела 4.1.1.](#))

5.9.5. Схема системного каталога

В дополнение к схеме `public` и схемам, создаваемым пользователями, любая база данных содержит схему `pg_catalog`, в которой находятся системные таблицы и все встроенные типы данных, функции и операторы. `pg_catalog` фактически всегда является частью пути поиска. Если даже эта схема не добавлена в путь явно, она неявно просматривается до всех схем, указанных в пути. Так обеспечивается доступность встроенных имён при любых условиях. Однако вы можете явным образом поместить `pg_catalog` в конец пути поиска, если вам нужно, чтобы пользовательские имена переопределяли встроенные.

Так как имена системных таблиц начинаются с `pg_`, такие имена лучше не использовать во избежание конфликта имён, возможного при появлении в будущем системной таблицы с тем же именем, что и ваша. (С путём поиска по умолчанию неполная ссылка будет воспринята как обращение к системной таблице.) Системные таблицы будут и дальше содержать в имени приставку `pg_`, так что они не будут конфликтовать с неполными именами пользовательских таблиц, если пользователи со своей стороны не будут использовать приставку `pg_`.

5.9.6. Шаблоны использования

Схемам можно найти множество применений. Для защиты от влияния недоверенных пользователей на поведение запросов других пользователей предлагается *шаблон безопасного использования схем*, но если этот шаблон не применяется в базе данных, пользователи, желающие безопасно выполнять в ней запросы, должны будут принимать защитные меры в начале каждого сеанса. В частности, они должны начинать каждый сеанс с присваивания пустого значения переменной `search_path` или каким-либо другим образом удалять из `search_path` схемы, доступные для записи обычным пользователям. С конфигурацией по умолчанию легко реализуются следующие шаблоны использования:

- Ограничить обычных пользователей личными схемами. Для реализации этого подхода сначала убедитесь, что ни у одной схемы нет права `CREATE`. Затем для каждого пользователя, кото-

рый будет создавать не временные объекты, создайте схему с его именем, например `CREATE SCHEMA alice AUTHORIZATION alice`. (Как вы знаете, путь поиска по умолчанию начинается с имени `$user`, вместо которого подставляется имя пользователя. Таким образом, если у всех пользователей будет отдельная схема, они по умолчанию будут обращаться к собственным схемам.) Этот шаблон позволяет безопасно использовать схемы, только если никакой недоверенный пользователь не является владельцем базы данных и не получал право `ADMIN OPTION` для соответствующей роли. В противном случае безопасное использование схем невозможно.

В PostgreSQL 15 и выше этот подход использования поддерживается конфигурацией по умолчанию. В предыдущих версиях или при использовании базы данных, обновлённой с предыдущей версии, необходимо удалить право `CREATE` из схемы `public` (выполнить `REVOKE CREATE ON SCHEMA public FROM PUBLIC`). Затем проверьте, нет ли в схеме `public` объектов с такими же именами, как у объектов в схеме `pg_catalog`.

- Удалить схему `public` из пути поиска по умолчанию, изменив `postgresql.conf` или выполнив команду `ALTER ROLE ALL SET search_path = "$user"`. Затем следует предоставить права на создание объектов в схеме `public`. Выбираться объекты в этой схеме будут только по полному имени. Тогда как обращаться к таблицам по полному имени вполне допустимо, обращения к функциям в общей схеме всё же **будут небезопасными или ненадёжными**. Поэтому если вы создаёте функции или расширения в схеме `public`, применяйте первый шаблон. Если же нет, этот шаблон, как и первый, безопасен при условии, что никакой недоверенный пользователь не является владельцем базы данных и не получал право `ADMIN OPTION` для соответствующей роли.
- Сохранить путь поиска по умолчанию и предоставить права создания объектов в схеме `public`. Все пользователи неявно обращаются к схеме `public`. Тем самым имитируется ситуация с полным отсутствием схем, что позволяет осуществить плавный переход из среды без схем. Однако данный шаблон ни в коем случае нельзя считать безопасным. Он подходит, только если в базе данных имеется всего один либо несколько доверяющих друг другу пользователей. В базах данных, обновлённых с версии PostgreSQL 14 или более ранней, этот шаблон применяется по умолчанию.

При любом подходе, устанавливая совместно используемые приложения (таблицы, которые нужны всем, дополнительные функции сторонних разработчиков и т. д.), помещайте их в отдельные схемы. Не забудьте дать другим пользователям права для доступа к этим схемам. Тогда пользователи смогут обращаться к этим дополнительным объектам по полному имени или при желании добавят эти схемы в свои пути поиска.

5.9.7. Переносимость

Стандарт SQL не поддерживает обращение в одной схеме к разным объектам, принадлежащим разным пользователям. Более того, в ряде реализаций СУБД нельзя создавать схемы с именем, отличным от имени владельца. На практике, в СУБД, реализующих только базовую поддержку схем согласно стандарту, концепции пользователя и схемы очень близки. Таким образом, многие пользователи полагают, что полное имя на самом деле образуется как `имя_пользователя.имя_таблицы`. И именно так будет вести себя Postgres Pro, если вы создадите схемы для каждого пользователя.

В стандарте SQL нет и понятия схемы `public`. Для максимального соответствия стандарту использовать схему `public` не следует.

Конечно, есть СУБД, в которых вообще не реализованы схемы или пространства имён поддерживают (возможно, с ограничениями) обращения к другим базам данных. Если вам потребуется работать с этими системами, максимальной переносимости вы достигнете, вообще не используя схемы.

5.10. Наследование

Postgres Pro реализует наследование таблиц, что может быть полезно для проектировщиков баз данных. (Стандарт SQL:1999 и более поздние версии определяют возможность наследования типов, но это во многом отличается от того, что описано здесь.)

Давайте начнём со следующего примера: предположим, что мы создаём модель данных для городов. В каждом штате есть множество городов, но лишь одна столица. Мы хотим иметь возможность быстро получать город-столицу для любого штата. Это можно сделать, создав две таблицы: одну для столиц штатов, а другую для городов, не являющихся столицами. Однако что делать, если нам нужно получить информацию о любом городе, будь то столица штата или нет? В решении этой проблемы может помочь наследование. Мы определим таблицу `capitals` как наследника `cities`:

```
CREATE TABLE cities (
    name          text,
    population     float,
    elevation      int      -- в футах
);
```

```
CREATE TABLE capitals (
    state          char(2)
) INHERITS (cities);
```

В этом случае таблица `capitals` *наследует* все столбцы своей родительской таблицы, `cities`. Столицы штатов также имеют дополнительный столбец `state`, в котором будет указан штат.

В Postgres Pro таблица может наследоваться от нуля или нескольких других таблиц, а запросы могут выбирать все строки родительской таблицы или все строки родительской и всех дочерних таблиц. По умолчанию принят последний вариант. Например, следующий запрос найдёт названия всех городов, включая столицы штатов, расположенных выше 500 футов:

```
SELECT name, elevation
FROM cities
WHERE elevation > 500;
```

Для данных из введения (см. [Раздел 2.1](#)) он выдаст:

name	elevation
Las Vegas	2174
Mariposa	1953
Madison	845

А следующий запрос находит все города, которые не являются столицами штатов, но также находятся на высоте выше 500 футов:

```
SELECT name, elevation
FROM ONLY cities
WHERE elevation > 500;
```

name	elevation
Las Vegas	2174
Mariposa	1953

Здесь ключевое слово `ONLY` указывает, что запрос должен применяться только к таблице `cities`, но не к таблицам, расположенным ниже `cities` в иерархии наследования. Многие операторы, которые мы уже обсудили, — `SELECT`, `UPDATE` и `DELETE` — поддерживают ключевое слово `ONLY`.

Вы также можете добавить после имени таблицы `*`, чтобы явно указать, что должны включаться и дочерние таблицы:

```
SELECT name, elevation
FROM cities*
WHERE elevation > 500;
```

Указывать `*` не обязательно, так как теперь это поведение всегда подразумевается по умолчанию. Однако такая запись всё ещё поддерживается для совместимости со старыми версиями, где поведение по умолчанию могло быть изменено.

В некоторых ситуациях бывает необходимо узнать, из какой таблицы выбрана конкретная строка. Для этого вы можете воспользоваться системным столбцом `tableoid`, присутствующим в каждой таблице:

```
SELECT c.tableoid, c.name, c.elevation
FROM cities c
WHERE c.elevation > 500;
```

этот запрос выдаст:

tableoid	name	elevation
139793	Las Vegas	2174
139793	Mariposa	1953
139798	Madison	845

(Если вы попытаетесь выполнить его у себя, скорее всего вы получите другие значения OID.) Собственно имена таблиц вы можете получить, обратившись к `pg_class`:

```
SELECT p.relname, c.name, c.elevation
FROM cities c, pg_class p
WHERE c.elevation > 500 AND c.tableoid = p.oid;
```

в результате вы получите:

relname	name	elevation
cities	Las Vegas	2174
cities	Mariposa	1953
capitals	Madison	845

Тот же эффект можно получить другим способом, используя альтернативный тип `regclass`; при этом OID таблицы выводится в символьном виде:

```
SELECT c.tableoid::regclass, c.name, c.elevation
FROM cities c
WHERE c.elevation > 500;
```

Механизм наследования не способен автоматически распределять данные команд `INSERT` или `COPY` по таблицам в иерархии наследования. Поэтому в нашем примере этот оператор `INSERT` не выполнится:

```
INSERT INTO cities (name, population, elevation, state)
VALUES ('Albany', NULL, NULL, 'NY');
```

Мы могли надеяться на то, что данные каким-то образом попадут в таблицу `capitals`, но этого не происходит: `INSERT` всегда вставляет данные непосредственно в указанную таблицу. В некоторых случаях добавляемые данные можно перенаправлять, используя правила (см. [Главу 40](#)). Однако в нашем случае это не поможет, так как таблица `cities` не содержит столбца `state` и команда будет отвергнута до применения правила.

Дочерние таблицы автоматически наследуют от родительской таблицы ограничения-проверки и ограничения `NOT NULL` (если только для них не задано явно `NO INHERIT`). Все остальные ограничения (уникальности, первичный ключ и внешние ключи) не наследуются.

Таблица может наследоваться от нескольких родительских таблиц, в этом случае она будет объединять в себе все столбцы этих таблиц, а также столбцы, описанные непосредственно в её определении. Если в определениях родительских и дочерней таблиц встретятся столбцы с одним именем, эти столбцы будут «объединены», так что в дочерней таблице окажется только один столбец. Чтобы такое объединение было возможно, столбцы должны иметь одинаковый тип данных, в противном случае произойдёт ошибка. Наследуемые ограничения-проверки и ограничения `NOT NULL` объединяются подобным образом. Так, например, объединяемый столбец получит свойство `NOT NULL`, если какое-либо из порождающих его определений имеет свойство `NOT NULL`. Ограниче-

ния-проверки объединяются, если они имеют одинаковые имена; но если их условия различаются, происходит ошибка.

Отношение наследования между таблицами обычно устанавливается при создании дочерней таблицы с использованием предложения `INHERITS` оператора `CREATE TABLE`. Другой способ добавить такое отношение для таблицы, определённой подходящим образом, — использовать `INHERIT` с оператором `ALTER TABLE`. Для этого будущая дочерняя таблица должна уже включать те же столбцы (с совпадающими именами и типами), что и родительская таблица. Также она должна включать аналогичные ограничения-проверки (с теми же именами и выражениями). Удалить отношение наследования можно с помощью указания `NO INHERIT` оператора `ALTER TABLE`. Динамическое добавление и удаление отношений наследования может быть полезно при реализации секционирования таблиц (см. [Раздел 5.11](#)).

Для создания таблицы, которая затем может стать наследником другой, удобно воспользоваться предложением `LIKE` оператора `CREATE TABLE`. Такая команда создаст новую таблицу с теми же столбцами, что имеются в исходной. Если в исходной таблице определены ограничения `CHECK`, для создания полностью совместимой таблицы их тоже нужно скопировать, и это можно сделать, добавив к предложению `LIKE` параметр `INCLUDING CONSTRAINTS`.

Родительскую таблицу нельзя удалить, пока существуют унаследованные от неё. При этом в дочерних таблицах нельзя удалять или модифицировать столбцы или ограничения-проверки, унаследованные от родительских таблиц. Если вы хотите удалить таблицу вместе со всеми её потомками, это легко сделать, добавив в команду удаления родительской таблицы параметр `CASCADE` (см. [Раздел 5.14](#)).

При изменениях определений и ограничений столбцов команда `ALTER TABLE` распространяет эти изменения вниз в иерархии наследования. Однако удалить столбцы, унаследованные дочерними таблицами, можно только с помощью параметра `CASCADE`. При создании отношений наследования команда `ALTER TABLE` следует тем же правилам объединения дублирующихся столбцов, что и `CREATE TABLE`.

В запросах с наследуемыми таблицами проверка прав доступа выполняется только в родительской таблице. Так, например, наличие разрешения `UPDATE` для таблицы `cities` подразумевает право на изменение строк также в таблице `capitals`, когда к ним происходит обращение через таблицу `cities`. Это сохраняет видимость того, что эти данные (также) находятся в родительской таблице. Но изменить таблицу `capitals` напрямую без дополнительного разрешения нельзя. Подобным образом, политики защиты на уровне строк (см. [Раздел 5.8](#)) для родительской таблицы применяются к строкам, получаемым из дочерних таблиц при выполнении запроса с наследованием. Политики же дочерних таблиц, если они определены, действуют только когда такие таблицы явно задействуются в запросе; в этом случае все политики, связанные с родительскими таблицами, игнорируются.

Сторонние таблицы (см. [Раздел 5.12](#)) могут также входить в иерархию наследования как родительские или дочерние таблицы, так же, как и обычные. Если в иерархию наследования входит сторонняя таблица, все операции, не поддерживаемые ей, не будут поддерживаться иерархией в целом.

5.10.1. Ограничения

Заметьте, что не все SQL-команды могут работать с иерархиями наследования. Команды, выполняющие выборку данных, изменение данных или модификацию схемы (например `SELECT`, `UPDATE`, `DELETE`, большинство вариантов `ALTER TABLE`, но не `INSERT` и `ALTER TABLE ... RENAME`), обычно по умолчанию обрабатывают данные дочерних таблиц и могут исключать их, если поддерживают указание `ONLY`. Команды для обслуживания и настройки базы данных (например `REINDEX` и `VACUUM`) обычно работают только с отдельными физическими таблицами и не поддерживают рекурсивную обработку отношений наследования. Соответствующее поведение каждой команды описано в её справке ([Команды SQL](#)).

Возможности наследования серьёзно ограничены тем, что индексы (включая ограничения уникальности) и ограничения внешних ключей относятся только к отдельным таблицам, но не к их по-

томкам. Это касается обеих сторон ограничений внешних ключей. Таким образом, применительно к нашему примеру:

- Если мы объявим `cities.name` с ограничением `UNIQUE` или `PRIMARY KEY`, это не мешает добавить в таблицу `capitals` строки с названиями городов, уже существующими в таблице `cities`. И эти дублирующиеся строки по умолчанию будут выводиться в результате запросов к `cities`. На деле таблица `capitals` по умолчанию вообще не будет содержать ограничение уникальности, так что в ней могут оказаться несколько строк с одним названием. Хотя вы можете добавить в `capitals` соответствующее ограничение, но это не предотвратит дублирование при объединении с `cities`.
- Подобным образом, если мы укажем, что `cities.name` ссылается (`REFERENCES`) на какую-то другую таблицу, это ограничение не будет автоматически распространено на `capitals`. В этом случае решением может стать явное добавление такого же ограничения `REFERENCES` в таблицу `capitals`.
- Если вы сделаете, чтобы столбец другой таблицы ссылался на `cities(name)`, в этом столбце можно будет указывать только названия городов, но не столиц. В этом случае хорошего решения нет.

Некоторая функциональность, не реализованная для иерархий наследования, реализована для декларативного секционирования. Поэтому обязательно взвесьте все за и против, прежде чем применять в своих приложениях секционирование с использованием устаревшего наследования.

5.11. Секционирование таблиц

Postgres Pro поддерживает простое секционирование таблиц. В этом разделе описывается, как и почему бывает полезно применять секционирование при проектировании баз данных.

5.11.1. Обзор

Секционированием называется разбиение данных, логически являющихся одной большой таблицей, на более мелкие физические части (секции). Секционирование может принести следующую пользу:

- В определённых ситуациях оно кардинально увеличивает быстродействие, особенно когда большой процент часто запрашиваемых строк таблицы относится к одной или лишь нескольким секциям. Секционирование по сути заменяет верхние уровни деревьев индексов, что увеличивает вероятность нахождения наиболее востребованных частей индексов в памяти.
- Когда в выборке или изменении данных задействована большая часть одной секции, производительность может возрасти, если будет выполняться последовательное сканирование этой секции, а не поиск по индексу, сопровождаемый произвольным чтением данных, разбросанных по всей таблице.
- Массовую загрузку и удаление данных можно осуществлять, добавляя и удаляя секции, если такой вариант использования был предусмотрен при проектировании секций. Удаление отдельной секции командой `DROP TABLE` и действие `ALTER TABLE DETACH PARTITION` выполняются гораздо быстрее, чем аналогичная массовая операция. Эти команды полностью исключают накладные расходы, связанные с выполнением `VACUUM` после массовой операции `DELETE`.
- Редко используемые данные можно перенести на более дешёвые и медленные носители.

Всё это обычно полезно только для очень больших таблиц. Какие именно таблицы выиграют от секционирования, зависит от конкретного приложения, хотя, как правило, это следует применять для таблиц, размер которых превышает объём ОЗУ сервера.

Postgres Pro предлагает поддержку следующих видов секционирования:

Секционирование по диапазонам

Таблица секционируется по «диапазонам», определённым по ключевому столбцу или набору столбцов, и не пересекающимся друг с другом. Например, можно секционировать данные по диапазонам дат или по диапазонам идентификаторов определённых бизнес-объектов. Границы каждого диапазона считаются включающими нижнее значение и исключаящими верхнее. На-

пример, если для первой секции задан диапазон значений от 1 до 10, а для второй — от 10 до 20, значение 10 относится ко второй секции, а не к первой.

Секционирование по списку

Таблица секционируется с помощью списка, явно указывающего, какие значения ключа должны относиться к каждой секции.

Секционирование по хешу

Таблица секционируется по определённым модулям и остаткам, которые указываются для каждой секции. Каждая секция содержит строки, для которых хеш-значение ключа секционирования, делённое на модуль, равняется заданному остатку.

Если вашему приложению требуются другие формы секционирования, можно также прибегнуть к альтернативным реализациям, с использованием наследования и представлений с `UNION ALL`. Такие подходы дают гибкость, но не дают такого выигрыша в производительности, как встроенное декларативное секционирование.

5.11.2. Декларативное секционирование

Postgres Pro позволяет декларировать, что некоторая таблица разделяется на секции. Разделённая на секции таблица называется *секционированной таблицей*. Декларация секционирования состоит из описанного выше определения *метода секционирования* и списка столбцов или выражений, образующих *ключ секционирования*.

Сама секционированная таблица является «виртуальной» и как таковая не хранится. Хранилище используется её *секциями*, которые являются обычными таблицами, связанными с секционированной. В каждой секции хранится подмножество данных таблицы, определяемое её *границами секции*. Все строки, вставляемые в секционированную таблицу, перенаправляются в соответствующие секции в зависимости от значений столбцов ключа секционирования. Если при изменении значений ключа секционирования в строке она перестаёт удовлетворять ограничениям исходной секции, эта строка перемещается в другую секцию.

Сами секции могут представлять собой секционированные таблицы, таким образом реализуется *вложенное секционирование*. Хотя все секции должны иметь те же столбцы, что и секционированная родительская таблица, в каждой секции независимо от других могут быть определены свои индексы, ограничения и значения по умолчанию. Подробнее о создании секционированных таблиц и секций рассказывается в описании [CREATE TABLE](#).

Преобразовать обычную таблицу в секционированную и наоборот нельзя. Однако в секционированную таблицу можно добавить в качестве секции существующую обычную или секционированную таблицу, а также можно удалить секцию из секционированной таблицы и превратить её в отдельную таблицу; это может ускорить многие процессы обслуживания. Обратитесь к описанию [ALTER TABLE](#), чтобы узнать больше о подкомандах `ATTACH PARTITION` и `DETACH PARTITION`.

Секции также могут быть *сторонними таблицами*, но учтите, что именно пользователь ответствен за соответствие содержимого сторонней таблицы правилу секционирования, это не контролируется автоматически. Также существуют и некоторые другие ограничения. За подробностями обратитесь к описанию [CREATE FOREIGN TABLE](#).

5.11.2.1. Пример

Предположим, что мы создаём базу данных для большой компании, торгующей мороженым. Компания учитывает максимальную температуру и продажи мороженого каждый день в разрезе регионов. По сути нам нужна следующая таблица:

```
CREATE TABLE measurement (
    city_id      int not null,
    logdate      date not null,
    peaktemp     int,
    unitsales    int
```


);

Мы знаем, что большинство запросов будут работать только с данными за последнюю неделю, месяц или квартал, так как в основном эта таблица нужна для формирования текущих отчётов для руководства. Чтобы сократить объём хранящихся старых данных, мы решили оставлять данные только за 3 последних года. Ненужные данные мы будем удалять в начале каждого месяца. В этой ситуации мы можем использовать секционирование для удовлетворения всех наших требований к таблице показателей.

Чтобы использовать декларативное секционирование в этом случае, выполните следующее:

1. Создайте таблицу `measurement` как секционированную таблицу с предложением `PARTITION BY`, указав метод секционирования (в нашем случае `RANGE`) и список столбцов, которые будут образовывать ключ секционирования.

```
CREATE TABLE measurement (
    city_id          int not null,
    logdate          date not null,
    peaktemp         int,
    unitsales        int
) PARTITION BY RANGE (logdate);
```

2. Создайте секции. В определении каждой секции должны задаваться границы, соответствующие методу и ключу секционирования родительской таблицы. Заметьте, что указание границ, при котором множество значений новой секции пересекается со множеством значений в одной или нескольких существующих секциях, будет ошибочным.

Секции, создаваемые таким образом, во всех отношениях являются обычными таблицами Postgres Pro (или, возможно, сторонними таблицами). В частности, для каждой секции можно независимо задать табличное пространство и параметры хранения.

В нашем примере каждая секция должна содержать данные за один месяц, чтобы данные можно было удалять по месяцам согласно требованиям. Таким образом, нужные команды будут выглядеть так:

```
CREATE TABLE measurement_y2006m02 PARTITION OF measurement
    FOR VALUES FROM ('2006-02-01') TO ('2006-03-01');

CREATE TABLE measurement_y2006m03 PARTITION OF measurement
    FOR VALUES FROM ('2006-03-01') TO ('2006-04-01');

...

CREATE TABLE measurement_y2007m11 PARTITION OF measurement
    FOR VALUES FROM ('2007-11-01') TO ('2007-12-01');

CREATE TABLE measurement_y2007m12 PARTITION OF measurement
    FOR VALUES FROM ('2007-12-01') TO ('2008-01-01')
    TABLESPACE fasttablespace;

CREATE TABLE measurement_y2008m01 PARTITION OF measurement
    FOR VALUES FROM ('2008-01-01') TO ('2008-02-01')
    WITH (parallel_workers = 4)
    TABLESPACE fasttablespace;
```

(Как говорилось ранее, границы соседних секций могут определяться одинаковыми значениями, так как верхние границы не включаются в диапазон.)

Если вы хотите реализовать вложенное секционирование, дополнительно укажите предложение `PARTITION BY` в командах, создающих отдельные секции, например:

```
CREATE TABLE measurement_y2006m02 PARTITION OF measurement
    FOR VALUES FROM ('2006-02-01') TO ('2006-03-01')
    PARTITION BY RANGE (peaktemp);
```

Когда будут созданы секции `measurement_y2006m02`, данные, добавляемые в `measurement` и падающие в `measurement_y2006m02` (или данные, которые могут добавляться непосредственно в `measurement_y2006m02` при условии соблюдения ограничения данной секции) будут затем перенаправлены в одну из вложенных секций в зависимости от значения столбца `peaktemp`. Указанный ключ секционирования может пересекаться с ключом секционирования родителя, хотя определять границы вложенной секции нужно осмотрительно, чтобы множество данных, которое она принимает, входило во множество, допускаемое собственными границами секции; система не пытается контролировать это сама.

При добавлении в родительскую таблицу данных, которые не соответствуют ни одной из существующих секций, произойдёт ошибка; подходящую секцию нужно создавать вручную.

Создавать в таблицах ограничения с условиями, задающими границы секций, вручную не требуется. Такие ограничения будут созданы автоматически.

3. Создайте в секционируемой таблице индекс по ключевому столбцу (или столбцам), а также любые другие индексы, которые могут понадобиться. (Индекс по ключу, строго говоря, создавать не обязательно, но в большинстве случаев он будет полезен.) При этом автоматически будет создан соответствующий индекс в каждой секции и все секции, которые вы будете создавать или присоединять позднее, тоже будут содержать такой индекс. Индексы или ограничения уникальности, созданные в секционированной таблице, являются «виртуальными», как и сама секционированная таблица: фактически данные находятся в дочерних индексах отдельных таблиц-секций.

```
CREATE INDEX ON measurement (logdate);
```

4. Убедитесь в том, что параметр конфигурации `enable_partition_pruning` не выключен в `postgresql.conf`. Иначе запросы не будут оптимизироваться должным образом.

В данном примере нам потребуется создавать секцию каждый месяц, так что было бы разумно написать скрипт, который бы формировал требуемый код DDL автоматически.

5.11.2.2. Обслуживание секций

Обычно набор секций, образованный изначально при создании таблиц, не предполагается сохранять неизменным. Чаще наоборот, планируется удалять секции со старыми данными и периодически добавлять секции с новыми. Одно из наиболее важных преимуществ секционирования состоит именно в том, что оно позволяет практически моментально выполнять трудоёмкие операции, изменяя структуру секций, а не физически перемещая большие объёмы данных.

Самый лёгкий способ удалить старые данные — просто удалить секцию, ставшую ненужной:

```
DROP TABLE measurement_y2006m02;
```

Так можно удалить миллионы записей гораздо быстрее, чем удалять их по одной. Заметьте, однако, что приведённая выше команда требует установления блокировки `ACCESS EXCLUSIVE`.

Ещё один часто более предпочтительный вариант — убрать секцию из главной таблицы, но сохранить возможность обращаться к ней как к самостоятельной таблице:

```
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02;
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02 CONCURRENTLY;
```

При этом можно будет продолжать работать с данными, пока таблица не будет удалена. Например, в этом состоянии очень кстати будет сделать резервную копию данных, используя `COPY`, `pg_dump` или подобные средства. Возможно, эти данные также можно будет агрегировать, перевести в компактный формат, выполнить другую обработку или построить отчёты. Первая форма команды требует блокировки `ACCESS EXCLUSIVE` родительской таблицы. Благодаря указанию `CONCURRENTLY`, как во второй форме, для операции отсоединения будет требоваться только блокировка `SHARE UPDATE EXCLUSIVE` родительской таблицы, но при этом действуют ограничения, описанные в `ALTER TABLE ... DETACH PARTITION`.

Аналогичным образом можно добавлять новую секцию с данными. Мы можем создать пустую секцию в главной таблице так же, как мы создавали секции в исходном состоянии до этого:

```
CREATE TABLE measurement_y2008m02 PARTITION OF measurement
    FOR VALUES FROM ('2008-02-01') TO ('2008-03-01')
    TABLESPACE fasttablespace;
```

А иногда удобнее создать новую таблицу вне структуры секций и присоединить её в виде секции позже. При таком подходе новые данные можно будет загрузить, проверить и преобразовать до того, как они появятся в секционированной таблице. Кроме того, операция `ATTACH PARTITION` требует для секционированной таблицы только блокировки `SHARE UPDATE EXCLUSIVE`, в отличие от операции `CREATE TABLE ... PARTITION OF`, требующей блокировки `ACCESS EXCLUSIVE`, поэтому её удобнее использовать для выполнения параллельных операций с секционированной таблицей. Обойтись без кропотливого воспроизведения определения родительской таблицы можно, воспользовавшись функциональностью `CREATE TABLE ... LIKE`:

```
CREATE TABLE measurement_y2008m02
    (LIKE measurement INCLUDING DEFAULTS INCLUDING CONSTRAINTS)
    TABLESPACE fasttablespace;

ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02
    CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE '2008-03-01' );

\copy measurement_y2008m02 from 'measurement_y2008m02'
-- possibly some other data preparation work

ALTER TABLE measurement ATTACH PARTITION measurement_y2008m02
    FOR VALUES FROM ('2008-02-01') TO ('2008-03-01' );
```

Прежде чем выполнять команду `ATTACH PARTITION`, рекомендуется создать ограничение `CHECK` в присоединяемой таблице, соответствующее ожидаемому ограничению секции, как показано выше. Благодаря этому система сможет обойтись без сканирования, необходимого для проверки неявного ограничения секции. Без этого ограничения `CHECK` нужно будет просканировать и убедиться в выполнении ограничения секции, удерживая блокировку `ACCESS EXCLUSIVE` в этой секции. После выполнения команды `ATTACH PARTITION` рекомендуется удалить ограничение `CHECK`, поскольку оно больше не нужно. Если присоединяемая таблица также является секционированной таблицей, то каждая из её секций будет рекурсивно блокироваться и сканироваться до тех пор, пока не встретится подходящее ограничение `CHECK` или не будут достигнуты конечные разделы.

Точно так же, если в секционированной таблице есть секция `DEFAULT`, рекомендуется создать ограничение `CHECK`, которое исключает ограничение секции, подлежащей присоединению. Если этого не сделать, то секция `DEFAULT` будет просканирована, чтобы убедиться, что она не содержит записей, которые должны быть расположены в присоединяемой секции. Эта операция будет выполняться при удержании блокировки `ACCESS EXCLUSIVE` на секции `DEFAULT`. Если секция `DEFAULT` сама является секционированной таблицей, то каждая из её секций будет рекурсивно проверяться таким же образом, как и присоединяемая таблица, как упоминалось выше.

Как говорилось выше, в секционированных таблицах можно создавать индексы так, чтобы они изменялись автоматически ко всей иерархии. Это очень удобно, так как индексироваться будут не только все существующие секции, но и любые секции, создаваемые в будущем. Но есть одно ограничение — такой секционированный индекс нельзя создать в неблокирующем режиме (с указанием `CONCURRENTLY`). Чтобы избежать блокировки на долгое время, для создания индекса в самой секционированной таблице можно использовать команду `CREATE INDEX ON ONLY`; такой индекс будет помечен как нерабочий, и он не будет автоматически применён к секциям. Индексы собственно в секциях можно создать в индивидуальном порядке с указанием `CONCURRENTLY`, а затем *присоединить* их к индексу-родителю, используя команду `ALTER INDEX .. ATTACH PARTITION`. После того как индексы всех секций будут присоединены к родительскому, последний автоматически перейдёт в рабочее состояние. Например:

```
CREATE INDEX measurement_usls_idx ON ONLY measurement (unitsales);

CREATE INDEX CONCURRENTLY measurement_usls_200602_idx
    ON measurement_y2006m02 (unitsales);
```

```
ALTER INDEX measurement_usls_idx
  ATTACH PARTITION measurement_usls_200602_idx;
...
```

Этот приём можно применять и с ограничениями `UNIQUE` и `PRIMARY KEY`; для них индексы создаются неявно при создании ограничения. Например:

```
ALTER TABLE ONLY measurement ADD UNIQUE (city_id, logdate);

ALTER TABLE measurement_y2006m02 ADD UNIQUE (city_id, logdate);
ALTER INDEX measurement_city_id_logdate_key
  ATTACH PARTITION measurement_y2006m02_city_id_logdate_key;
...
```

5.11.2.3. Ограничения

С секционированными таблицами связаны следующие ограничения:

- Создать ограничение уникальности или первичного ключа для секционированной таблицы можно, только если ключи секций не включают никакие выражения или вызовы функций и в ограничение входят все столбцы ключа секционирования. Это условие объясняется тем, что отдельные индексы, образующие ограничение, могут непосредственно обеспечивать уникальность только в своих секциях. Поэтому сама структура секционирования должна гарантировать отсутствие дубликатов в разных секциях.
- Создать ограничение-исключение, охватывающее всю секционированную таблицу, нельзя; можно только поместить такое ограничение в каждую отдельную секцию с данными. И это также является следствием того, что установить ограничения, действующие между секциями, невозможно.
- Триггеры `BEFORE ROW` для `INSERT` не могут менять секцию, в которую в итоге попадёт новая строка.
- Смешивание временных и постоянных отношений в одном дереве секционирования не допускается. Таким образом, если секционированная таблица постоянная, такими же должны быть её секции; с временными таблицами аналогично. В случае с временными отношениями все таблицы дерева секционирования должны быть из одного сеанса.

На уровне реализации отдельные секции связываются с секционированной таблицей средствами наследования. Однако с декларативно секционированными таблицами или их секциями нельзя использовать некоторую общую функциональность наследования, как описано ниже. А именно, секция не может иметь никаких других родителей, кроме секционированной таблицы, к которой она относится, равно как и любая таблица не может наследоваться и от секционированной, и от обычной таблицы. Это означает, что секционированные таблицы и их секции не совмещаются в иерархии наследования с обычными таблицами.

Так как иерархия секционирования, образованная секционированной таблицей и её секциями, является одновременно и иерархией наследования, она содержит `tableoid` и на неё распространяются все обычные правила наследования, описанные в [Разделе 5.10](#), с некоторыми исключениями:

- В секциях не может быть столбцов, отсутствующих в родительской таблице. Такие столбцы невозможно определить ни при создании секций командой `CREATE TABLE`, ни путём последующего добавления в секции командой `ALTER TABLE`. Таблицы могут быть подключены в качестве секций командой `ALTER TABLE ... ATTACH PARTITION`, только если их столбцы в точности соответствуют родительской таблице.
- Ограничения `CHECK` вместе с `NOT NULL`, определённые в секционированной таблице, всегда наследуются всеми её секциями. Ограничения `CHECK` с характеристикой `NO INHERIT` в секционированных таблицах создавать нельзя. Также нельзя удалить ограничение `NOT NULL`, заданное для столбца секции, если такое же ограничение существует в родительской таблице.
- Использование указания `ONLY` при добавлении или удалении ограничения только в секционированной таблице поддерживается, лишь когда в ней нет секций. При наличии секций попыт-

ка использования `ONLY` вызовет ошибку для любых ограничений, кроме `UNIQUE` и `PRIMARY KEY`. С другой стороны, ограничения можно добавлять или удалять непосредственно в секциях (если они отсутствуют в родительской таблице).

- Так как секционированная таблица сама по себе не содержит данные, использование `TRUNCATE ONLY` для секционированной таблицы всегда будет считаться ошибкой.

5.11.3. Секционирование с использованием наследования

Хотя встроенное декларативное секционирование полезно во многих часто возникающих ситуациях, бывают обстоятельства, требующие более гибкого подхода. В этом случае секционирование можно реализовать, применив механизм наследования таблиц, что даст ряд возможностей, неподдерживаемых при декларативном секционировании, например:

- При декларативном секционировании все секции должны иметь в точности тот же набор столбцов, что и секционируемая таблица, тогда как обычное наследование таблиц допускает наличие в дочерних таблицах дополнительных столбцов, отсутствующих в родителе.
- Механизм наследования таблиц поддерживает множественное наследование.
- С декларативным секционированием поддерживается только секционирование по спискам, по диапазонам и по хешу, тогда как с наследованием таблиц данные можно разделять по любому критерию, выбранному пользователем. (Однако заметьте, что если исключение по ограничению не позволяет эффективно устранять дочерние таблицы из планов запросов, производительность запросов будет очень низкой.)

5.11.3.1. Пример

В этом примере будет создана структура секционирования, равнозначная структуре из примера с декларативным секционированием. Выполните следующие действия:

1. Создайте «главную» таблицу, от которой будут наследоваться все «дочерние» таблицы. Главная таблица не будет содержать данные. Не определяйте в ней никакие ограничения-проверки, если только вы не намерены применить их во всех дочерних таблицах. Также не имеет смысла определять в ней какие-либо индексы или ограничения уникальности. В нашем примере главной таблицей будет `measurement` со своим изначальным определением:

```
CREATE TABLE measurement (
    city_id          int not null,
    logdate          date not null,
    peaktemp         int,
    unitsales        int
);
```

2. Создайте несколько «дочерних» таблиц, унаследовав их все от главной. Обычно в таких таблицах не будет никаких дополнительных столбцов, кроме унаследованных. Как и с декларативным секционированием, эти таблицы во всех отношениях будут обычными таблицами Postgres Pro (или сторонними таблицами).

```
CREATE TABLE measurement_y2006m02 () INHERITS (measurement);
CREATE TABLE measurement_y2006m03 () INHERITS (measurement);
...
CREATE TABLE measurement_y2007m11 () INHERITS (measurement);
CREATE TABLE measurement_y2007m12 () INHERITS (measurement);
CREATE TABLE measurement_y2008m01 () INHERITS (measurement);
```

3. Добавьте в дочерние таблицы неперекрывающиеся ограничения, определяющие допустимые значения ключей для каждой из них.

Типичные примеры таких ограничений:

```
CHECK ( x = 1 )
CHECK ( county IN ( 'Oxfordshire', 'Buckinghamshire', 'Warwickshire' ) )
CHECK ( outletID >= 100 AND outletID < 200 )
```

Убедитесь в том, что ограничения не пересекаются, то есть никакие значения ключа не относятся сразу к нескольким дочерним таблицам. Например, часто допускают такую ошибку в определении диапазонов:

```
CHECK ( outletID BETWEEN 100 AND 200 )
CHECK ( outletID BETWEEN 200 AND 300 )
```

Это не будет работать, так как неясно, к какой дочерней таблице должно относиться значение 200. Поэтому диапазоны должны определяться следующим образом:

```
CREATE TABLE measurement_y2006m02 (
    CHECK ( logdate >= DATE '2006-02-01' AND logdate < DATE '2006-03-01' )
) INHERITS (measurement);
```

```
CREATE TABLE measurement_y2006m03 (
    CHECK ( logdate >= DATE '2006-03-01' AND logdate < DATE '2006-04-01' )
) INHERITS (measurement);
```

...

```
CREATE TABLE measurement_y2007m11 (
    CHECK ( logdate >= DATE '2007-11-01' AND logdate < DATE '2007-12-01' )
) INHERITS (measurement);
```

```
CREATE TABLE measurement_y2007m12 (
    CHECK ( logdate >= DATE '2007-12-01' AND logdate < DATE '2008-01-01' )
) INHERITS (measurement);
```

```
CREATE TABLE measurement_y2008m01 (
    CHECK ( logdate >= DATE '2008-01-01' AND logdate < DATE '2008-02-01' )
) INHERITS (measurement);
```

4. Для каждой дочерней таблицы создайте индекс по ключевому столбцу (или столбцам), а также любые другие индексы по своему усмотрению.

```
CREATE INDEX measurement_y2006m02_logdate ON measurement_y2006m02 (logdate);
CREATE INDEX measurement_y2006m03_logdate ON measurement_y2006m03 (logdate);
CREATE INDEX measurement_y2007m11_logdate ON measurement_y2007m11 (logdate);
CREATE INDEX measurement_y2007m12_logdate ON measurement_y2007m12 (logdate);
CREATE INDEX measurement_y2008m01_logdate ON measurement_y2008m01 (logdate);
```

5. Мы хотим, чтобы наше приложение могло сказать INSERT INTO measurement ... и данные оказались в соответствующей дочерней таблице. Мы можем добиться этого, добавив подходящую триггерную функцию в главную таблицу. Если данные всегда будут добавляться только в последнюю дочернюю таблицу, нам будет достаточно очень простой функции:

```
CREATE OR REPLACE FUNCTION measurement_insert_trigger()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO measurement_y2008m01 VALUES (NEW.*);
    RETURN NULL;
END;
$$
LANGUAGE plpgsql;
```

После функции мы создадим вызывающий её триггер:

```
CREATE TRIGGER insert_measurement_trigger
    BEFORE INSERT ON measurement
    FOR EACH ROW EXECUTE FUNCTION measurement_insert_trigger();
```

Мы должны менять определение триггерной функции каждый месяц, чтобы она всегда вставляла данные в текущую дочернюю таблицу. Определение самого триггера, однако, менять не требуется.

Но мы можем также сделать, чтобы сервер автоматически находил дочернюю таблицу, в которую нужно направить добавляемую строку. Для этого нам потребуется более сложная триггерная функция:

```
CREATE OR REPLACE FUNCTION measurement_insert_trigger()
RETURNS TRIGGER AS $$
BEGIN
    IF ( NEW.logdate >= DATE '2006-02-01' AND
        NEW.logdate < DATE '2006-03-01' ) THEN
        INSERT INTO measurement_y2006m02 VALUES (NEW.*);
    ELSIF ( NEW.logdate >= DATE '2006-03-01' AND
        NEW.logdate < DATE '2006-04-01' ) THEN
        INSERT INTO measurement_y2006m03 VALUES (NEW.*);
    ...
    ELSIF ( NEW.logdate >= DATE '2008-01-01' AND
        NEW.logdate < DATE '2008-02-01' ) THEN
        INSERT INTO measurement_y2008m01 VALUES (NEW.*);
    ELSE
        RAISE EXCEPTION
        'Date out of range. Fix the measurement_insert_trigger() function!';
    END IF;
    RETURN NULL;
END;
$$
LANGUAGE plpgsql;
```

Определение триггера остаётся прежним. Заметьте, что все условия `IF` должны в точности отражать ограничения `CHECK` соответствующих дочерних таблиц.

Хотя эта функция сложнее, чем вариант с одним текущим месяцем, её не придётся так часто модифицировать, так как ветви условий можно добавить заранее.

Примечание

На практике будет лучше сначала проверять условие для последней дочерней таблицы, если строки добавляются в неё чаще всего, но для простоты мы расположили проверки триггера в том же порядке, что и в других фрагментах кода для этого примера.

Другой способ перенаправления добавляемых строк в соответствующую дочернюю таблицу можно реализовать, определив для главной таблицы не триггер, а правила. Например:

```
CREATE RULE measurement_insert_y2006m02 AS
ON INSERT TO measurement WHERE
    ( logdate >= DATE '2006-02-01' AND logdate < DATE '2006-03-01' )
DO INSTEAD
    INSERT INTO measurement_y2006m02 VALUES (NEW.*);
...
CREATE RULE measurement_insert_y2008m01 AS
ON INSERT TO measurement WHERE
    ( logdate >= DATE '2008-01-01' AND logdate < DATE '2008-02-01' )
DO INSTEAD
    INSERT INTO measurement_y2008m01 VALUES (NEW.*);
```

С правилами связано гораздо больше накладных расходов, чем с триггером, но они относятся к запросу в целом, а не к каждой строке. Поэтому этот способ может быть более выигрышным при массовом добавлении данных. Однако в большинстве случаев триггеры будут работать быстрее.

Учтите, что команда `COPY` игнорирует правила. Если вы хотите вставить данные с помощью `COPY`, вам придётся копировать их сразу в нужную дочернюю таблицу, а не в главную таблицу. С другой стороны, `COPY` не отменяет триггеры, так что с триггерами вы сможете использовать её обычным образом.

Ещё один недостаток подхода с правилами связан с невозможностью выдать ошибку, если добавляемая строка не подпадает ни под одно из правил; в этом случае данные просто попадут в главную таблицу.

6. Убедитесь в том, что параметр конфигурации `constraint_exclusion` не выключен в `postgresql.conf`. В противном случае дочерние таблицы могут сканироваться, когда это не требуется.

Как уже можно понять, для реализации сложной иерархии таблиц может потребоваться DDL-код значительного объёма. В данном примере нам потребуется создавать дочернюю таблицу каждый месяц, так что было бы разумно написать скрипт, формирующий требуемый код DDL автоматически.

5.11.3.2. Обслуживание таблиц, секционированных через наследование

Чтобы быстро удалить старые данные, просто удалите ставшую ненужной дочернюю таблицу:

```
DROP TABLE measurement_y2006m02;
```

Чтобы удалить дочернюю таблицу из иерархии наследования, но сохранить к ней доступ как к самостоятельной таблице:

```
ALTER TABLE measurement_y2006m02 NO INHERIT measurement;
```

Чтобы добавить новую дочернюю таблицу для новых данных, создайте пустую дочернюю таблицу так же, как до этого создавали начальные:

```
CREATE TABLE measurement_y2008m02 (
    CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE '2008-03-01' )
) INHERITS (measurement);
```

Можно также создать новую таблицу и наполнить её данными до добавления в иерархию таблиц. Это позволит загрузить, проверить и при необходимости преобразовать данные до того, как запросы к главной таблице смогут их увидеть.

```
CREATE TABLE measurement_y2008m02
    (LIKE measurement INCLUDING DEFAULTS INCLUDING CONSTRAINTS);
ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02
    CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE '2008-03-01' );
\copy measurement_y2008m02 from 'measurement_y2008m02'
-- возможна дополнительная подготовка данных
ALTER TABLE measurement_y2008m02 INHERIT measurement;
```

5.11.3.3. Ограничения

С реализацией секционирования через наследование связаны следующие ограничения:

- Система не может проверить автоматически, являются ли все ограничения `CHECK` взаимно исключающими. Поэтому безопаснее будет написать и отладить код для формирования дочерних таблиц и создания и/или изменения связанных объектов, чем делать это вручную.
- Индексы и внешние ключи относятся к определённой таблице, но не к её иерархии наследования, поэтому с ними связаны дополнительные [ограничения](#).
- Показанные здесь схемы подразумевают, что ключевой столбец (или столбцы) в строке никогда не меняется, или меняется не настолько, чтобы строку потребовалось перенести в другую секцию. Если же попытаться выполнить такой оператор `UPDATE`, произойдёт ошибка из-за нарушения ограничения `CHECK`. Если вам нужно обработать и такие случаи, вы можете устано-

вить подходящие триггеры на обновление в дочерних таблицах, но это ещё больше усложнит управление всей конструкцией.

- Если вы выполняете команды `VACUUM` или `ANALYZE` вручную, не забывайте, что их нужно запускать для каждой дочерней таблицы в отдельности. Команда

```
ANALYZE measurement;
```

обработает только главную таблицу.

- Операторы `INSERT` с предложениями `ON CONFLICT` скорее всего не будут работать ожидаемым образом, так как действие `ON CONFLICT` предпринимается только в случае нарушений уникальности в указанном целевом отношении, а не его дочерних отношениях.
- Для направления строк в нужные дочерние таблицы потребуются триггеры или правила, если только приложение не знает непосредственно о схеме секционирования. Разработать триггеры может быть довольно сложно, и они будут работать гораздо медленнее, чем внутреннее распределение кортежей при декларативном секционировании.

5.11.4. Отсечение секций

Отсечение секций — это приём оптимизации запросов, который ускоряет работу с декларативно секционированными таблицами. Например:

```
SET enable_partition_pruning = on;           -- по умолчанию
SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';
```

Без отсечения секций показанный запрос должен будет просканировать все секции таблицы `measurement`. Когда отсечение секций включено, планировщик рассматривает определение каждой секции и может заключить, что какую-либо секцию сканировать не нужно, так как в ней не может быть строк, удовлетворяющих предложению `WHERE` в запросе. Когда планировщик может сделать такой вывод, он исключает (*отсекает*) секцию из плана запроса.

Используя команду `EXPLAIN` и параметр конфигурации [enable_partition_pruning](#), можно увидеть отличие плана, в котором были отсечены секции, от плана без отсечения. Типичный неоптимизированный план для такой конфигурации таблицы будет выглядеть так:

```
SET enable_partition_pruning = off;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';
QUERY PLAN
```

```
Aggregate  (cost=188.76..188.77 rows=1 width=8)
-> Append  (cost=0.00..181.05 rows=3085 width=0)
    -> Seq Scan on measurement_y2006m02  (cost=0.00..33.12 rows=617 width=0)
        Filter: (logdate >= '2008-01-01'::date)
    -> Seq Scan on measurement_y2006m03  (cost=0.00..33.12 rows=617 width=0)
        Filter: (logdate >= '2008-01-01'::date)
...
    -> Seq Scan on measurement_y2007m11  (cost=0.00..33.12 rows=617 width=0)
        Filter: (logdate >= '2008-01-01'::date)
    -> Seq Scan on measurement_y2007m12  (cost=0.00..33.12 rows=617 width=0)
        Filter: (logdate >= '2008-01-01'::date)
    -> Seq Scan on measurement_y2008m01  (cost=0.00..33.12 rows=617 width=0)
        Filter: (logdate >= '2008-01-01'::date)
```

В некоторых или всех секциях может применяться не полное последовательное сканирование, а сканирование по индексу, но основная идея примера в том, что для удовлетворения запроса не нужно сканировать старые секции. И когда мы включаем отсечение секций, мы получаем значительно более эффективный план, дающий тот же результат:

```
SET enable_partition_pruning = on;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';
QUERY PLAN
```

```
Aggregate (cost=37.75..37.76 rows=1 width=8)
-> Seq Scan on measurement_y2008m01 (cost=0.00..33.12 rows=617 width=0)
    Filter: (logdate >= '2008-01-01'::date)
```

Заметьте, что механизм отсечения секций учитывает только ограничения, определённые неявно ключами секционирования, но не наличие индексов. Поэтому определять индексы для столбцов ключа не обязательно. Нужно ли создавать индекс для определённой секции, зависит от того, какую часть секции (меньшую или большую), по вашим представлениям, будут сканировать запросы, обращающиеся к этой секции. Индекс будет полезен в первом случае, но не во втором.

Отсечение секций может производиться не только при планировании конкретного запроса, но и в процессе его выполнения. Благодаря этому может быть отсечено больше секций, когда условные выражения содержат значения, неизвестные во время планирования, например параметры, определённые оператором `PREPARE`, значения, получаемые из подзапросов, или параметризованные значения во внутренней стороне соединения с вложенным циклом. Отсечение секций в процессе выполнения запроса возможно в следующие моменты времени:

- Во время подготовки плана запроса. В этот момент можно отсечь секции, учитывая значения параметров, известные при подготовке выполнения запроса. Секции, отсечённые на этом этапе, не будут видны в выводе `EXPLAIN` или `EXPLAIN ANALYZE`. Их общее количество можно определить по свойству «Subplans Removed» в выводе `EXPLAIN`. Важно отметить, что секции, отсечённые на этом этапе, остаются заблокированными на момент начала выполнения запроса.
- В процессе собственно выполнения плана запроса. Отсечение секций также может выполняться на этом этапе и позволяет отфильтровать секции, используя значения, которые становятся известны, когда запрос выполняется фактически. В частности это могут быть значения из подзапросов и значения параметров времени выполнения, например из параметризованных соединений с вложенными циклами. Так как значения параметров могут меняться многократно при выполнении запроса, отсечение секций выполняется при изменении любого из параметров, анализируемых механизмом отсечения. Чтобы определить, были ли секции отсечены на данном этапе, нужно внимательно изучить свойство `loops` в выводе `EXPLAIN ANALYZE`. Подпланы, соответствующие разным секциям, могут иметь разные значения, в зависимости от того, сколько раз они отсекались во время выполнения. Некоторые из них могут даже иметь значение `(never executed)` (никогда не выполнялись), если они отсекались всегда.

Отсечение секций можно отключить, воспользовавшись параметром [enable_partition_pruning](#).

5.11.5. Секционирование и исключение по ограничению

Исключение по ограничению — приём оптимизации запросов, подобный отсечению секций. Прежде всего он применяется, когда секционирование осуществляется с использованием старого метода наследования, но он может быть полезен и для других целей, включая декларативное секционирование.

Исключение по ограничению работает во многом так же, как и отсечение секций; отличие состоит в том, что оно использует ограничения `CHECK` всех таблиц (поэтому оно так и называется), тогда как для отсечения секций используются границы секции, которые существуют только в случае декларативного секционирования. Ещё одно различие состоит в том, что исключение по ограничению применяется только во время планирования; во время выполнения секции из плана удаляться не будут.

То, что исключение по ограничению использует ограничения `CHECK` (вследствие чего оно работает медленнее отсечения секций), иногда может быть и преимуществом. Ограничения могут быть определены даже для декларативно секционированных таблиц, в дополнение к внутренним границам секций, и тогда исключение по ограничению сможет дополнительно убрать некоторые секции из плана запроса.

По умолчанию параметр [constraint_exclusion](#) имеет значение не `on` и не `off`, а промежуточное (и рекомендуемое) значение `partition`, при котором этот приём будет применяться только к запро-

сам, где предположительно будут задействованы таблицы, секционированные с использованием наследования. Значение `on` обязывает планировщик просматривать ограничения `CHECK` во всех запросах, даже в самых простых, где выигрыш от исключения по ограничению маловероятен.

Применяя исключения по ограничению, необходимо учитывать следующее:

- Исключение по ограничению применяется только при планировании запроса, в отличие от отсечения секций, которое может осуществляться и при выполнении запроса.
- Исключение по ограничению работает только когда предложение `WHERE` в запросе содержит константы (или получаемые извне параметры). Например, сравнение с непостоянной функцией, такой как `CURRENT_TIMESTAMP`, нельзя оптимизировать, так как планировщик не может знать, в какую дочернюю таблицу попадёт значение функции во время выполнения.
- Ограничения секций должны быть простыми, иначе планировщик не сможет вычислить, какие дочерние таблицы не нужно обрабатывать. Для секционирования по спискам используйте простые условия на равенства, а для секционирования по диапазонам — простые проверки диапазонов, подобные показанным в примерах. Рекомендуются создавать ограничения секций, содержащие только такие сравнения секционирующих столбцов с константами, в которых используются операторы, поддерживающие В-деревья. Это объясняется тем, что в ключе секционирования допускаются только такие столбцы, которые могут быть проиндексированы в В-дереве.
- При анализе для исключения по ограничению исследуются все ограничения всех дочерних таблиц, относящихся к главной, так что при большом их количестве время планирования запросов может значительно увеличиться. Поэтому устаревший вариант секционирования, основанный на наследовании, будет работать хорошо, пока количество дочерних таблиц не превышает примерно ста; не пытайтесь применять его с тысячами дочерних таблиц.

5.11.6. Рекомендации по декларативному секционированию

К секционированию таблицы следует подходить продуманно, так как неудачное решение может отрицательно повлиять на скорость планирования и выполнения запросов.

Одним из самых важных факторов является выбор столбца или столбцов, по которым будут секционироваться ваши данные. Часто оптимальным будет секционирование по столбцу или набору столбцов, которые практически всегда присутствуют в предложении `WHERE` в запросах, обращающихся к секционируемой таблице. Предложения `WHERE`, совместимые с ограничениями границ секции, могут применяться для устранения ненужных для выполнения запроса секций. Однако наличие ограничений `PRIMARY KEY` или `UNIQUE` может подтолкнуть к выбору и других столбцов в качестве секционирующих. Также при планировании секционирования следует продумать, как будут удаляться данные. Секцию целиком можно отсоединить очень быстро, поэтому может иметь смысл разработать стратегию секционирования так, чтобы массово удаляемые данные оказывались в одной секции.

Также важно правильно выбрать число секций, на которые будет разбиваться таблица. Если их будет недостаточно много, индексы останутся большими, не улучшится и локальность данных, вследствие чего процент попаданий в кеш окажется низким. Однако и при слишком большом количестве секций возможны проблемы. С большим количеством секций увеличивается время планирования и потребление памяти как при планировании, так и при выполнении запросов, о чём рассказывается далее. Выбирая стратегию секционирования таблицы, также важно учитывать, какие изменения могут произойти в будущем. Например если вы решите создавать отдельные секции для каждого клиента, и в данный момент у вас всего несколько больших клиентов, подумайте, что будет, если через несколько лет у вас будет много мелких клиентов. В этом случае может быть лучше произвести секционирование по хешу (`HASH`) и выбрать разумное количество секций, но не создавать секции по списку (`LIST`) в надежде, что количество клиентов не увеличится до такой степени, что секционирование данных окажется непрактичным.

Вложенное секционирование позволяет дополнительно разделить те секции, которые предположительно окажутся больше других. Также можно использовать секционирование по диапазонам с

ключом, состоящим из нескольких столбцов. Но при любом из подходов легко может получиться слишком много секций, так что рекомендуется использовать их осмотрительно.

Важно учитывать издержки секционирования, которые отражаются на планировании и выполнении запросов. Планировщик запросов обычно довольно неплохо справляется с иерархиями, включающими до нескольких тысяч секций, если при выполнении типичных запросов ему удаётся отсечь почти все секции. Однако когда после отсечения остаётся большое количество секций, возрастает и время планирования запросов, и объём потребляемой памяти. Наличие большого количества секций нежелательно ещё и потому, что потребление памяти сервером может значительно возрасти со временем, особенно когда множество сеансов обращаются ко множеству секций. Это объясняется тем, что в локальную память каждого сеанса, который обращается к секциям, должны быть загружены метаданные всех этих секций.

С нагрузкой, присущей информационным хранилищам, может иметь смысл создавать больше секций, чем с нагрузкой OLTP. Как правило, в информационных хранилищах время планирования запроса второстепенно, так как гораздо больше времени тратится на выполнение запроса. Однако при любом типе нагрузки важно принимать правильные решения на ранней стадии реализации, так как процесс пересекционирования таблиц большого объёма может оказаться чрезвычайно длительным. Для оптимизации стратегии секционирования часто бывает полезно предварительно эмулировать ожидаемую нагрузку. Но ни в коем случае нельзя полагать, что чем больше секций, тем лучше, равно как и наоборот.

5.12. Сторонние данные

Postgres Pro частично реализует спецификацию SQL/MED, позволяя вам обращаться к данным, находящимся снаружи, используя обычные SQL-запросы. Такие данные называются *сторонними*.

Сторонние данные доступны в PostgreSQL через *обёртку сторонних данных*. Обёртка сторонних данных — это библиотека, взаимодействующая с внешним источником данных и скрывающая в себе внутренние особенности подключения и получения данных. Несколько готовых обёрток предоставляются в виде модулей `contrib`; см. [Приложение F](#). Также вы можете найти другие обёртки, выпускаемые как дополнительные продукты. Если ни одна из существующих обёрток вас не устраивает, вы можете написать свою собственную (см. [Главу 56](#)).

Чтобы обратиться к сторонним данным, вы должны создать объект *сторонний сервер*, в котором настраивается подключение к внешнему источнику данных, определяются параметры соответствующей обёртки сторонних данных. Затем вы должны создать одну или несколько *сторонних таблиц*, определив тем самым структуру внешних данных. Сторонние таблицы можно использовать в запросах так же, как и обычные, но их данные не хранятся на сервере Postgres Pro. При каждом запросе Postgres Pro обращается к обёртке сторонних данных, которая, в свою очередь, получает данные из внешнего источника или передаёт их ему (в случае команд INSERT или UPDATE).

При обращении к внешним данным удалённый источник может потребовать аутентификации клиента. Соответствующие учётные данные можно предоставить с помощью *сопоставлений пользователей*, позволяющих определить в частности имена и пароли, в зависимости от текущей роли пользователя Postgres Pro.

Дополнительную информацию вы найдёте в [CREATE FOREIGN DATA WRAPPER](#), [CREATE SERVER](#), [CREATE USER MAPPING](#), [CREATE FOREIGN TABLE](#) и [IMPORT FOREIGN SCHEMA](#).

5.13. Другие объекты баз данных

Таблицы — центральные объекты в структуре реляционной базы данных, так как они содержат ваши данные. Но это не единственные объекты, которые могут в ней существовать. Помимо них вы можете создавать и использовать объекты и других типов, призванные сделать управление данными эффективнее и удобнее. Они не обсуждаются в этой главе, но мы просто перечислим некоторые из них, чтобы вы знали об их существовании:

- Представления

- Функции, процедуры и операторы
- Типы данных и домены
- Триггеры и правила перезаписи

Подробнее соответствующие темы освещаются в [Части V](#).

5.14. Отслеживание зависимостей

Когда вы создаёте сложные структуры баз данных, включающие множество таблиц с внешними ключами, представлениями, триггерами, функциями и т. п., вы неявно создаёте сеть зависимостей между объектами. Например, таблица с ограничением внешнего ключа зависит от таблицы, на которую она ссылается.

Для сохранения целостности структуры всей базы данных, Postgres Pro не позволяет удалять объекты, от которых зависят другие. Например, попытка удалить таблицу `products` (мы рассматривали её в [Подразделе 5.4.5](#)), от которой зависит таблица `orders`, приведёт к ошибке примерно такого содержания:

```
DROP TABLE products;
```

ОШИБКА: удалить объект "таблица products" нельзя, так как от него зависят другие

ПОДРОБНОСТИ: ограничение `orders_product_no_fkey` в отношении "таблица orders" зависит от объекта "таблица products"

ПОДСКАЗКА: Для удаления зависимых объектов используйте `DROP ... CASCADE`.

Сообщение об ошибке даёт полезную подсказку: если вы не хотите заниматься ликвидацией зависимостей по отдельности, можно выполнить:

```
DROP TABLE products CASCADE;
```

и все зависимые объекты, а также объекты, зависящие от них, будут удалены рекурсивно. В этом случае таблица `orders` останется, а удалено будет только её ограничение внешнего ключа. Удаление не распространится на другие объекты, так как ни один объект не зависит от этого ограничения. (Если вы хотите проверить, что произойдёт при выполнении `DROP ... CASCADE`, запустите `DROP` без `CASCADE` и прочитайте [ПОДРОБНОСТИ \(DETAIL\)](#).)

Почти все команды `DROP` в Postgres Pro поддерживают указание `CASCADE`. Конечно, вид возможных зависимостей зависит от типа объекта. Вы также можете написать `RESTRICT` вместо `CASCADE`, чтобы включить поведение по умолчанию, когда объект можно удалить, только если от него не зависят никакие другие.

Примечание

Стандарт SQL требует явного указания `RESTRICT` или `CASCADE` в команде `DROP`. Но это требование на самом деле не выполняется ни в одной СУБД, при этом одни системы по умолчанию подразумевают `RESTRICT`, а другие — `CASCADE`.

Если в команде `DROP` перечисляются несколько объектов, `CASCADE` требуется указывать, только когда есть зависимости вне заданной группы. Например, в команде `DROP TABLE tab1, tab2` при наличии внешнего ключа, ссылающегося на `tab1` из `tab2`, можно не указывать `CASCADE`, чтобы она выполнялась успешно.

Для пользовательских функций или процедур, тело которых задано строковой константой, Postgres Pro отслеживает зависимости, связанные с внешне видимыми свойствами функции, такими как типы аргументов и результата, но не зависимости, которые могут быть выявлены только при анализе тела функции. В качестве примера рассмотрите следующий сценарий:

```
CREATE TYPE rainbow AS ENUM ('red', 'orange', 'yellow',
                             'green', 'blue', 'purple');
```

```
CREATE TABLE my_colors (color rainbow, note text);
```

```
CREATE FUNCTION get_color_note (rainbow) RETURNS text AS
  'SELECT note FROM my_colors WHERE color = $1'
  LANGUAGE SQL;
```

(Описание функций языка SQL можно найти в [Разделе 37.5](#).) Postgres Pro будет понимать, что функция `get_color_note` зависит от типа `rainbow`: при удалении типа будет принудительно удалена функция, так как тип её аргумента оказывается неопределённым. Но Postgres Pro не будет учитывать зависимость `get_color_note` от таблицы `my_colors` и не удалит функцию при удалении таблицы. Но у этого подхода есть не только минус, но и плюс. В случае отсутствия таблицы эта функция останется рабочей в некотором смысле: хотя при попытке выполнить её возникнет ошибка, но при создании новой таблицы с тем же именем функция снова будет работать.

С другой стороны, при определении функции или процедуры в стиле стандарта SQL её тело разбирается, и все зависимости, распознаваемые синтаксическим анализатором, сохраняются. Таким образом, если записать приведённую выше функцию как

```
CREATE FUNCTION get_color_note (rainbow) RETURNS text
BEGIN ATOMIC
  SELECT note FROM my_colors WHERE color = $1;
END;
```

, то будет известна зависимость функции от таблицы `my_colors` и команда `DROP` её обработает.

Глава 6. Модификация данных

В предыдущей главе мы обсуждали, как создавать таблицы и другие структуры для хранения данных. Теперь пришло время заполнить таблицы данными. В этой главе мы расскажем, как добавлять, изменять и удалять данные из таблиц. А из следующей главы вы наконец узнаете, как извлекать нужные вам данные из базы данных.

6.1. Добавление данных

Сразу после создания таблицы она не содержит никаких данных. Поэтому, чтобы она была полезна, в неё прежде всего нужно добавить данные. По сути данные добавляются в таблицу по одной строке. И хотя вы конечно можете добавить в таблицу несколько строк, добавить в неё меньше, чем строку, невозможно. Даже если вы указываете значения только некоторых столбцов, создаётся полная строка.

Чтобы создать строку, вы будете использовать команду **INSERT**. В этой команде необходимо указать имя таблицы и значения столбцов. Например, рассмотрим таблицу товаров из [Главы 5](#):

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric  
);
```

Добавить в неё строку можно было бы так:

```
INSERT INTO products VALUES (1, 'Cheese', 9.99);
```

Значения данных перечисляются в порядке столбцов в таблице и разделяются запятыми. Обычно в качестве значений указываются константы, но это могут быть и скалярные выражения.

Показанная выше запись имеет один недостаток — вам необходимо знать порядок столбцов в таблице. Чтобы избежать этого, можно перечислить столбцы явно. Например, следующие две команды дадут тот же результат, что и показанная выше:

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);  
INSERT INTO products (name, price, product_no) VALUES ('Cheese', 9.99, 1);
```

Многие считают, что лучше всегда явно указывать имена столбцов.

Если значения определяются не для всех столбцов, лишние столбцы можно опустить. В таком случае эти столбцы получают значения по умолчанию. Например:

```
INSERT INTO products (product_no, name) VALUES (1, 'Cheese');  
INSERT INTO products VALUES (1, 'Cheese');
```

Вторая форма является расширением Postgres Pro. Она заполняет столбцы слева по числу переданных значений, а все остальные столбцы принимают значения по умолчанию.

Для ясности можно также явно указать значения по умолчанию для отдельных столбцов или всей строки:

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', DEFAULT);  
INSERT INTO products DEFAULT VALUES;
```

Одна команда может вставить сразу несколько строк:

```
INSERT INTO products (product_no, name, price) VALUES  
    (1, 'Cheese', 9.99),  
    (2, 'Bread', 1.99),  
    (3, 'Milk', 2.99);
```

Также возможно вставить результат запроса (который может не содержать строк либо содержать одну или несколько):


```
INSERT INTO products (product_no, name, price)
  SELECT product_no, name, price FROM new_products
  WHERE release_date = 'today';
```

Это позволяет использовать все возможности механизма запросов SQL (см. [Главу 7](#)) для вычисления вставляемых строк.

Подсказка

Когда нужно добавить сразу множество строк, возможно будет лучше использовать команду **COPY**. Она не такая гибкая, как **INSERT**, но гораздо эффективнее. Дополнительно об ускорении массовой загрузки данных можно узнать в [Разделе 14.4](#).

6.2. Изменение данных

Модификация данных, уже сохранённых в БД, называется изменением. Изменить можно все строки таблицы, либо подмножество всех строк, либо только избранные строки. Каждый столбец при этом можно изменять независимо от других.

Для изменения данных в существующих строках используется команда **UPDATE**. Ей требуется следующая информация:

1. Имя таблицы и изменяемого столбца
2. Новое значение столбца
3. Критерий отбора изменяемых строк

Если вы помните, в [Главе 5](#) говорилось, что в SQL в принципе нет уникального идентификатора строк. Таким образом, не всегда возможно явно указать на строку, которую требуется изменить. Поэтому необходимо указать условия, каким должны соответствовать требуемая строка. Только если в таблице есть первичный ключ (вне зависимости от того, объявляли вы его или нет), можно однозначно адресовать отдельные строки, определив условие по первичному ключу. Этим пользуются графические инструменты для работы с базой данных, дающие возможность редактировать данные по строкам.

Например, следующая команда увеличивает цену всех товаров, имевших до этого цену 5, до 10:

```
UPDATE products SET price = 10 WHERE price = 5;
```

В результате может измениться ноль, одна или множество строк. И если этому запросу не будет удовлетворять ни одна строка, это не будет ошибкой.

Давайте рассмотрим эту команду подробнее. Она начинается с ключевого слова **UPDATE**, за которым идёт имя таблицы. Как обычно, имя таблицы может быть записано в полной форме, в противном случае она будет найдена по пути. Затем идёт ключевое слово **SET**, за которым следует имя столбца, знак равенства и новое значение столбца. Этим значением может быть любое скалярное выражение, а не только константа. Например, если вы захотите поднять цену всех товаров на 10%, это можно сделать так:

```
UPDATE products SET price = price * 1.10;
```

Как видно из этого примера, выражение нового значения может ссылаться на существующие значения столбцов в строке. Мы также опустили в нём предложение **WHERE**. Это означает, что будут изменены все строки в таблице. Если же это предложение присутствует, изменяются только строки, которые соответствуют условию **WHERE**. Заметьте, что хотя знак равенства в предложении **SET** обозначает операцию присваивания, а такой же знак в предложении **WHERE** используется для сравнения, это не приводит к неоднозначности. И конечно, в условии **WHERE** не обязательно должна быть проверка равенства, а могут применяться и другие операторы (см. [Главу 9](#)). Необходимо только, чтобы это выражение возвращало логический результат.

В команде **UPDATE** можно изменить значения сразу нескольких столбцов, перечислив их в предложении **SET**. Например:

```
UPDATE mytable SET a = 5, b = 3, c = 1 WHERE a > 0;
```

6.3. Удаление данных

Мы рассказали о том, как добавлять данные в таблицы и как изменять их. Теперь вам осталось узнать, как удалить данные, которые оказались не нужны. Так же, как добавлять данные можно только целыми строками, удалять их можно только по строкам. В предыдущем разделе мы отметили, что в SQL нет возможности напрямую адресовать отдельные строки, так что удалить избранные строки можно, только сформулировав для них подходящие условия. Но если в таблице есть первичный ключ, с его помощью можно однозначно выделить определённую строку. При этом можно так же удалить группы строк, соответствующие условию, либо сразу все строки таблицы.

Для удаления строк используется команда **DELETE**; её синтаксис очень похож на синтаксис команды **UPDATE**. Например, удалить все строки из таблицы с товарами, имеющими цену 10, можно так:

```
DELETE FROM products WHERE price = 10;
```

Если вы напишете просто:

```
DELETE FROM products;
```

будут удалены все строки таблицы! Будьте осторожны!

6.4. Возврат данных из изменённых строк

Иногда бывает полезно получать данные из модифицируемых строк в процессе их обработки. Это возможно с использованием предложения **RETURNING**, которое можно задать для команд **INSERT**, **UPDATE** и **DELETE**. Применение **RETURNING** позволяет обойтись без дополнительного запроса к базе для сбора данных и это особенно ценно, когда как-то иначе трудно получить изменённые строки надёжным образом.

В предложении **RETURNING** допускается то же содержимое, что и в выходном списке команды **SELECT** (см. [Раздел 7.3](#)). Оно может содержать имена столбцов целевой таблицы команды или значения выражений с этими столбцами. Также часто применяется краткая запись **RETURNING ***, выбирающая все столбцы целевой таблицы по порядку.

В команде **INSERT** данные, выдаваемые в **RETURNING**, образуются из строки в том виде, в каком она была вставлена. Это не очень полезно при простом добавлении, так как в результате будут получены те же данные, что были переданы клиентом. Но это может быть очень удобно при использовании вычисляемых значений по умолчанию. Например, если в таблице есть столбец **serial**, в котором генерируются уникальные идентификаторы, команда **RETURNING** может вернуть идентификатор, назначенный новой строке:

```
CREATE TABLE users (firstname text, lastname text, id serial primary key);
```

```
INSERT INTO users (firstname, lastname) VALUES ('Joe', 'Cool') RETURNING id;
```

Предложение **RETURNING** также очень полезно с **INSERT ... SELECT**.

В команде **UPDATE** данные, выдаваемые в **RETURNING**, образуются новым содержимым изменённой строки. Например:

```
UPDATE products SET price = price * 1.10
WHERE price <= 99.99
RETURNING name, price AS new_price;
```

В команде **DELETE** данные, выдаваемые в **RETURNING**, образуются содержимым удалённой строки. Например:

```
DELETE FROM products
WHERE obsoletion_date = 'today'
```

`RETURNING *;`

Если для целевой таблицы заданы триггеры (см. [Главу 38](#)), в `RETURNING` выдаются данные из строки, изменённой триггерами. Таким образом, `RETURNING` часто применяется и для того, чтобы проверить содержимое столбцов, изменяемых триггерами.

Глава 7. Запросы

В предыдущих главах рассказывалось, как создать таблицы, как заполнить их данными и как изменить эти данные. Теперь мы наконец обсудим, как получить данные из базы данных.

7.1. Обзор

Процесс или команда получения данных из базы данных называется *запросом*. В SQL запросы формулируются с помощью команды `SELECT`. В общем виде команда `SELECT` записывается так:

```
[WITH запросы_with] SELECT список_выборки FROM табличное_выражение
[определение_сортировки]
```

В следующих разделах подробно описываются список выборки, табличное выражение и определение сортировки. Запросы `WITH` являются расширенной возможностью PostgreSQL и будут рассмотрены в последнюю очередь.

Простой запрос выглядит так:

```
SELECT * FROM table1;
```

Если предположить, что в базе данных есть таблица `table1`, эта команда получит все строки с содержимым всех столбцов из `table1`. (Метод выдачи результата определяет клиентское приложение. Например, программа `psql` выведет на экране ASCII-таблицу, хотя клиентские библиотеки позволяют извлекать отдельные значения из результата запроса.) Здесь список выборки задан как `*`, это означает, что запрос должен вернуть все столбцы табличного выражения. В списке выборки можно также указать подмножество доступных столбцов или составить выражения с этими столбцами. Например, если в `table1` есть столбцы `a`, `b` и `c` (и возможно, другие), вы можете выполнить следующий запрос:

```
SELECT a, b + c FROM table1;
```

(в предположении, что столбцы `b` и `c` имеют числовой тип данных). Подробнее это описано в [Разделе 7.3](#).

`FROM table1` — это простейший тип табличного выражения, в котором просто читается одна таблица. Вообще табличные выражения могут быть сложными конструкциями из базовых таблиц, соединений и подзапросов. А можно и вовсе опустить табличное выражение и использовать команду `SELECT` как калькулятор:

```
SELECT 3 * 4;
```

В этом может быть больше смысла, когда выражения в списке выборки возвращают меняющиеся результаты. Например, можно вызвать функцию так:

```
SELECT random();
```

7.2. Табличные выражения

Табличное выражение вычисляет таблицу. Это выражение содержит предложение `FROM`, за которым могут следовать предложения `WHERE`, `GROUP BY` и `HAVING`. Тривиальные табличные выражения просто ссылаются на физическую таблицу, её называют также базовой, но в более сложных выражениях такие таблицы можно преобразовывать и комбинировать самыми разными способами.

Необязательные предложения `WHERE`, `GROUP BY` и `HAVING` в табличном выражении определяют последовательность преобразований, осуществляемых с данными таблицы, полученной в предложении `FROM`. В результате этих преобразований образуется виртуальная таблица, строки которой передаются списку выборки, вычисляющему выходные строки запроса.

7.2.1. Предложение FROM

Предложение `FROM` образует таблицу из одной или нескольких ссылок на таблицы, разделённых запятыми.

```
FROM табличная_ссылка [, табличная_ссылка [, ...]]
```

Здесь табличной ссылкой может быть имя таблицы (возможно, с именем схемы), производная таблица, например подзапрос, соединение таблиц или сложная комбинация этих вариантов. Если в предложении `FROM` перечисляются несколько ссылок, для них применяется перекрёстное соединение (то есть декартово произведение их строк; см. ниже). Список `FROM` преобразуется в промежуточную виртуальную таблицу, которая может пройти через преобразования `WHERE`, `GROUP BY` и `HAVING`, и в итоге определит результат табличного выражения.

Когда в табличной ссылке указывается таблица, являющаяся родительской в иерархии наследования, в результате будут получены строки не только этой таблицы, но и всех её дочерних таблиц. Чтобы выбрать строки только одной родительской таблицы, перед её именем нужно добавить ключевое слово `ONLY`. Учтите, что при этом будут получены только столбцы указанной таблицы — дополнительные столбцы дочерних таблиц не попадут в результат.

Если же вы не добавляете `ONLY` перед именем таблицы, вы можете дописать после него `*`, тем самым указав, что должны обрабатываться и все дочерние таблицы. Практических причин использовать этот синтаксис больше нет, так как поиск в дочерних таблицах теперь производится по умолчанию. Однако эта запись поддерживается для совместимости со старыми версиями.

7.2.1.1. Соединённые таблицы

Соединённая таблица — это таблица, полученная из двух других (реальных или производных от них) таблиц в соответствии с правилами соединения конкретного типа. Общий синтаксис описания соединённой таблицы:

```
T1 тип_соединения T2 [ условие_соединения ]
```

Соединения любых типов могут вкладываться друг в друга или объединяться: и `T1`, и `T2` могут быть результатами соединения. Для однозначного определения порядка соединений предложения `JOIN` можно заключать в скобки. Если скобки отсутствуют, предложения `JOIN` обрабатываются слева направо.

Типы соединений

Перекрёстное соединение

```
T1 CROSS JOIN T2
```

Соединённую таблицу образуют все возможные сочетания строк из `T1` и `T2` (т. е. их декартово произведение), а набор её столбцов объединяет в себе столбцы `T1` со следующими за ними столбцами `T2`. Если таблицы содержат `N` и `M` строк, соединённая таблица будет содержать `N * M` строк.

`FROM T1 CROSS JOIN T2` равнозначно `FROM T1 INNER JOIN T2 ON TRUE` (см. ниже). Эта запись также равнозначна `FROM T1, T2`.

Примечание

Последняя запись не полностью эквивалентна первым при указании более чем двух таблиц, так как `JOIN` связывает таблицы сильнее, чем запятая. Например, `FROM T1 CROSS JOIN T2 INNER JOIN T3 ON условие` не равнозначно `FROM T1, T2 INNER JOIN T3 ON условие`, так как `условие` может ссылаться на `T1` в первом случае, но не во втором.

Соединения с сопоставлениями строк

```
T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2 ON логическое_выражение
T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2 USING ( список_столбцов_соединения )
T1 NATURAL { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2
```

Слова `INNER` и `OUTER` необязательны во всех формах. По умолчанию подразумевается `INNER` (внутреннее соединение), а при указании `LEFT`, `RIGHT` и `FULL` — внешнее соединение.

Условие соединения указывается в предложении `ON` или `USING`, либо неявно задаётся ключевым словом `NATURAL`. Это условие определяет, какие строки двух исходных таблиц считаются «соответствующими» друг другу (это подробно рассматривается ниже).

Возможные типы соединений с сопоставлениями строк:

`INNER JOIN`

Для каждой строки `R1` из `T1` в результирующей таблице содержится строка для каждой строки в `T2`, удовлетворяющей условию соединения с `R1`.

`LEFT OUTER JOIN`

Сначала выполняется внутреннее соединение (`INNER JOIN`). Затем в результат добавляются все строки из `T1`, которым не соответствуют никакие строки в `T2`, а вместо значений столбцов `T2` вставляются `NULL`. Таким образом, в результирующей таблице всегда будет минимум одна строка для каждой строки из `T1`.

`RIGHT OUTER JOIN`

Сначала выполняется внутреннее соединение (`INNER JOIN`). Затем в результат добавляются все строки из `T2`, которым не соответствуют никакие строки в `T1`, а вместо значений столбцов `T1` вставляются `NULL`. Это соединение является обратным к левому (`LEFT JOIN`): в результирующей таблице всегда будет минимум одна строка для каждой строки из `T2`.

`FULL OUTER JOIN`

Сначала выполняется внутреннее соединение. Затем в результат добавляются все строки из `T1`, которым не соответствуют никакие строки в `T2`, а вместо значений столбцов `T2` вставляются `NULL`. И наконец, в результат включаются все строки из `T2`, которым не соответствуют никакие строки в `T1`, а вместо значений столбцов `T1` вставляются `NULL`.

Предложение `ON` определяет наиболее общую форму условия соединения: в нём указываются выражения логического типа, подобные тем, что используются в предложении `WHERE`. Пара строк из `T1` и `T2` соответствуют друг другу, если выражение `ON` возвращает для них `true`.

`USING` — это сокращённая запись условия, полезная в ситуации, когда с обеих сторон соединения столбцы имеют одинаковые имена. Она принимает список общих имён столбцов через запятую и формирует условие соединения с равенством этих столбцов. Например, запись соединения `T1` и `T2` с `USING (a, b)` формирует условие `ON T1.a = T2.a AND T1.b = T2.b`.

Более того, при выводе `JOIN USING` исключаются избыточные столбцы: оба сопоставленных столбца выводить не нужно, так как они содержат одинаковые значения. Тогда как `JOIN ON` выдаёт все столбцы из `T1`, а за ними все столбцы из `T2`, `JOIN USING` выводит один столбец для каждой пары (в указанном порядке), за ними все оставшиеся столбцы из `T1` и, наконец, все оставшиеся столбцы `T2`.

Наконец, `NATURAL` — сокращённая форма `USING`: она образует список `USING` из всех имён столбцов, существующих в обеих входных таблицах. Как и с `USING`, эти столбцы оказываются в выходной таблице в единственном экземпляре. Если столбцов с одинаковыми именами не находится, `NATURAL JOIN` действует как `JOIN ... ON TRUE` и выдаёт декартово произведение строк.

Примечание

Предложение `USING` разумно защищено от изменений в соединяемых отношениях, так как оно связывает только явно перечисленные столбцы. `NATURAL` считается более рискованным, так как при любом изменении схемы в одном или другом отношении, когда по-

являются столбцы с совпадающими именами, при соединении будут связываться и эти новые столбцы.

Для наглядности предположим, что у нас есть таблицы t1:

num	name
1	a
2	b
3	c

и t2:

num	value
1	xxx
3	yyy
5	zzz

С ними для разных типов соединений мы получим следующие результаты:

=> **SELECT * FROM t1 CROSS JOIN t2;**

num	name	num	value
1	a	1	xxx
1	a	3	yyy
1	a	5	zzz
2	b	1	xxx
2	b	3	yyy
2	b	5	zzz
3	c	1	xxx
3	c	3	yyy
3	c	5	zzz

(9 rows)

=> **SELECT * FROM t1 INNER JOIN t2 ON t1.num = t2.num;**

num	name	num	value
1	a	1	xxx
3	c	3	yyy

(2 rows)

=> **SELECT * FROM t1 INNER JOIN t2 USING (num);**

num	name	value
1	a	xxx
3	c	yyy

(2 rows)

=> **SELECT * FROM t1 NATURAL INNER JOIN t2;**

num	name	value
1	a	xxx
3	c	yyy

(2 rows)

=> **SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num;**

num	name	num	value
1	a	1	xxx

```

 2 | b      |      |
 3 | c      |      | 3 | yyy
(3 rows)

```

```
=> SELECT * FROM t1 LEFT JOIN t2 USING (num);
```

```

num | name | value
-----+-----+-----
 1 | a    | xxx
 2 | b    |
 3 | c    | yyy
(3 rows)

```

```
=> SELECT * FROM t1 RIGHT JOIN t2 ON t1.num = t2.num;
```

```

num | name | num | value
-----+-----+-----+-----
 1 | a    | 1   | xxx
 3 | c    | 3   | yyy
   |      | 5   | zzz
(3 rows)

```

```
=> SELECT * FROM t1 FULL JOIN t2 ON t1.num = t2.num;
```

```

num | name | num | value
-----+-----+-----+-----
 1 | a    | 1   | xxx
 2 | b    |     |
 3 | c    | 3   | yyy
   |      | 5   | zzz
(4 rows)

```

Условие соединения в предложении `ON` может также содержать выражения, не связанные непосредственно с соединением. Это может быть полезно в некоторых запросах, но не следует использовать это необдуманно. Рассмотрим следующий запрос:

```
=> SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num AND t2.value = 'xxx';
```

```

num | name | num | value
-----+-----+-----+-----
 1 | a    | 1   | xxx
 2 | b    |     |
 3 | c    |     |
(3 rows)

```

Заметьте, что если поместить ограничение в предложение `WHERE`, вы получите другой результат:

```
=> SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num WHERE t2.value = 'xxx';
```

```

num | name | num | value
-----+-----+-----+-----
 1 | a    | 1   | xxx
(1 row)

```

Это связано с тем, что ограничение, помещённое в предложение `ON`, обрабатывается *до* операции соединения, тогда как ограничение в `WHERE` — *после*. Это не имеет значения при внутренних соединениях, но важно при внешних.

7.2.1.2. Псевдонимы таблиц и столбцов

Таблицам и ссылкам на сложные таблицы в запросе можно дать временное имя, по которому к ним можно будет обращаться в рамках запроса. Такое имя называется *псевдонимом таблицы*.

Определить псевдоним таблицы можно, написав

```
FROM табличная_ссылка AS псевдоним
```


или

```
FROM табличная_ссылка псевдоним
```

Ключевое слово `AS` является необязательным. Вместо *псевдоним* здесь может быть любой идентификатор.

Псевдонимы часто применяются для назначения коротких идентификаторов длинным именам таблиц с целью улучшения читаемости запросов. Например:

```
SELECT * FROM "очень_длинное_имя_таблицы" s JOIN "другое_длинное_имя" a ON s.id = a.num;
```

Псевдоним становится новым именем таблицы в рамках текущего запроса, т. е. после назначения псевдонима использовать исходное имя таблицы в другом месте запроса нельзя. Таким образом, следующий запрос недопустим:

```
SELECT * FROM my_table AS m WHERE my_table.a > 5;      -- неправильно
```

Хотя в основном псевдонимы используются для удобства, они бывают необходимы, когда таблица соединяется сама с собой, например:

```
SELECT * FROM people AS mother JOIN people AS child ON mother.id = child.mother_id;
```

Кроме того, псевдонимы обязательно нужно назначать подзапросам (см. [Подраздел 7.2.1.3](#)).

В случае неоднозначности определения псевдонимов можно использовать скобки. В следующем примере первый оператор назначает псевдоним `b` второму экземпляру `my_table`, а второй оператор назначает псевдоним результату соединения:

```
SELECT * FROM my_table AS a CROSS JOIN my_table AS b ...
SELECT * FROM (my_table AS a CROSS JOIN my_table) AS b ...
```

В другой форме назначения псевдонима временные имена даются не только таблицам, но и её столбцам:

```
FROM табличная_ссылка [AS] псевдоним ( столбец1 [, столбец2 [, ...]] )
```

Если псевдонимов столбцов оказывается меньше, чем фактически столбцов в таблице, остальные столбцы сохраняют свои исходные имена. Эта запись особенно полезна для замкнутых соединений или подзапросов.

Когда псевдоним применяется к результату `JOIN`, он скрывает оригинальные имена таблиц внутри `JOIN`. Например, это допустимый SQL-запрос:

```
SELECT a.* FROM my_table AS a JOIN your_table AS b ON ...
```

а запрос:

```
SELECT a.* FROM (my_table AS a JOIN your_table AS b ON ...) AS c
```

ошибочный, так как псевдоним таблицы `a` не виден снаружи определения псевдонима `c`.

7.2.1.3. Подзапросы

Подзапросы, образующие таблицы, должны заключаться в скобки и им *обязательно* должны назначаться псевдонимы (как описано в [Подразделе 7.2.1.2](#)). Например:

```
FROM (SELECT * FROM table1) AS псевдоним
```

Этот пример равносильен записи `FROM table1 AS псевдоним`. Более интересные ситуации, которые нельзя свести к простому соединению, возникают, когда в подзапросе используются агрегирующие функции или группировка.

Подзапросом может также быть список `VALUES`:

```
FROM (VALUES ('anne', 'smith'), ('bob', 'jones'), ('joe', 'blow'))
      AS names(first, last)
```

Такому подзапросу тоже требуется псевдоним. Назначать псевдонимы столбцам списка VALUES не требуется, но вообще это хороший приём. Подробнее это описано в [Разделе 7.7](#).

7.2.1.4. Табличные функции

Табличные функции — это функции, выдающие набор строк, содержащих либо базовые типы данных (скалярных типов), либо составные типы (табличные строки). Они применяются в запросах как таблицы, представления или подзапросы в предложении FROM. Столбцы, возвращённые табличными функциями, можно включить в выражения SELECT, JOIN или WHERE так же, как столбцы таблиц, представлений или подзапросов.

Табличные функции можно также скомбинировать, используя запись ROWS FROM. Результаты функций будут возвращены в параллельных столбцах; число строк в этом случае будет наибольшим из результатов всех функций, а результаты функций с меньшим количеством строк будут дополнены значениями NULL.

```
вызов_функции [WITH ORDINALITY] [[AS] псевдоним_таблицы [(псевдоним_столбца [, ...])]]
ROWS FROM( вызов_функции [, ...] ) [WITH ORDINALITY] [[AS] псевдоним_таблицы
      [(псевдоним_столбца [, ...])]]
```

Если указано предложение WITH ORDINALITY, к столбцам результатов функций будет добавлен ещё один, с типом bigint. В этом столбце нумеруются строки результирующего набора, начиная с 1. (Это обобщение стандартного SQL-синтаксиса UNNEST ... WITH ORDINALITY.) По умолчанию, этот столбец называется ordinality, но ему можно присвоить и другое имя с помощью указания AS.

Специальную табличную функцию UNNEST можно вызвать с любым числом параметров-массивов, а возвращает она соответствующее число столбцов, как если бы UNNEST ([Раздел 9.19](#)) вызывалась для каждого параметра в отдельности, а результаты объединялись с помощью конструкции ROWS FROM.

```
UNNEST( выражение_массива [, ...] ) [WITH ORDINALITY] [[AS] псевдоним_таблицы
      [(псевдоним_столбца [, ...])]]
```

Если псевдоним_таблицы не указан, в качестве имени таблицы используется имя функции; в случае с конструкцией ROWS FROM() — имя первой функции.

Если псевдонимы столбцов не указаны, то для функции, возвращающей базовый тип данных, именем столбца будет имя функции. Для функций, возвращающих составной тип, имена результирующих столбцов определяются индивидуальными атрибутами типа.

Несколько примеров:

```
CREATE TABLE foo (fooid int, foosubid int, fooname text);
```

```
CREATE FUNCTION getfoo(int) RETURNS SETOF foo AS $$
  SELECT * FROM foo WHERE fooid = $1;
$$ LANGUAGE SQL;
```

```
SELECT * FROM getfoo(1) AS t1;
```

```
SELECT * FROM foo
  WHERE foosubid IN (
                        SELECT foosubid
                        FROM getfoo(foo.fooid) z
                        WHERE z.fooid = foo.fooid
                      );
```

```
CREATE VIEW vw_getfoo AS SELECT * FROM getfoo(1);
```

```
SELECT * FROM vw_getfoo;
```

В некоторых случаях бывает удобно определить табличную функцию, возвращающую различные наборы столбцов при разных вариантах вызова. Это можно сделать, объявив функцию, не имеющую выходных параметров (OUT) и возвращающую псевдотип `record`. Используя такую функцию, ожидаемую структуру строк нужно описать в самом запросе, чтобы система знала, как разобрать запрос и составить его план. Записывается это так:

```
вызов_функции [AS] псевдоним (определение_столбца [, ...])
вызов_функции AS [псевдоним] (определение_столбца [, ...])
ROWS FROM( ... вызов_функции AS (определение_столбца [, ...]) [, ...] )
```

Без `ROWS FROM()` список `определения_столбцов` заменяет список псевдонимов, который можно также добавить в предложении `FROM`; имена в определениях столбцов служат псевдонимами. С `ROWS FROM()` список `определения_столбцов` можно добавить к каждой функции отдельно, либо в случае с одной функцией и без предложения `WITH ORDINALITY`, список `определения_столбцов` можно записать вместо списка с псевдонимами столбцов после `ROWS FROM()`.

Взгляните на этот пример:

```
SELECT *
FROM dblink('dbname=mydb', 'SELECT proname, prosrc FROM pg_proc')
AS t1(proname name, prosrc text)
WHERE proname LIKE 'bytea%';
```

Здесь функция `dblink` (из модуля `dblink`) выполняет удалённый запрос. Она объявлена как функция, возвращающая тип `record`, так как он подойдёт для запроса любого типа. В этом случае фактический набор столбцов функции необходимо описать в вызывающем её запросе, чтобы анализатор запроса знал, например, как преобразовать `*`.

В этом примере используется конструкция `ROWS FROM`:

```
SELECT *
FROM ROWS FROM
(
    json_to_recordset('[{ "a":40, "b":"foo"}, { "a":100, "b":"bar"} ]')
    AS (a INTEGER, b TEXT),
    generate_series(1, 3)
) AS x (p, q, s)
ORDER BY p;
```

p	q	s
40	foo	1
100	bar	2
		3

Она объединяет результаты двух функций в одном отношении `FROM`. В данном случае `json_to_recordset()` должна выдавать два столбца, первый `integer` и второй `text`, а результат `generate_series()` используется непосредственно. Предложение `ORDER BY` упорядочивает значения первого столбца как целочисленные.

7.2.1.5. Подзапросы LATERAL

Перед подзапросами в предложении `FROM` можно добавить ключевое слово `LATERAL`. Это позволит ссылаться в них на столбцы предшествующих элементов списка `FROM`. (Без `LATERAL` каждый подзапрос выполняется независимо и поэтому не может обращаться к другим элементам `FROM`.)

Перед табличными функциями в предложении `FROM` также можно указать `LATERAL`, но для них это ключевое слово необязательно; в аргументах функций в любом случае можно обращаться к столбцам в предыдущих элементах `FROM`.

Элемент `LATERAL` может находиться на верхнем уровне списка `FROM` или в дереве `JOIN`. В последнем случае он может также ссылаться на любые элементы в левой части `JOIN`, справа от которого он находится.

Когда элемент `FROM` содержит ссылки `LATERAL`, запрос выполняется следующим образом: сначала для строки элемента `FROM` с целевыми столбцами, или набора строк из нескольких элементов `FROM`, содержащих целевые столбцы, вычисляется элемент `LATERAL` со значениями этих столбцов. Затем результирующие строки обычным образом соединяются со строками, из которых они были вычислены. Эта процедура повторяется для всех строк исходных таблиц.

`LATERAL` можно использовать так:

```
SELECT * FROM foo, LATERAL (SELECT * FROM bar WHERE bar.id = foo.bar_id) ss;
```

Здесь это не очень полезно, так как тот же результат можно получить более простым и привычным способом:

```
SELECT * FROM foo, bar WHERE bar.id = foo.bar_id;
```

Применять `LATERAL` имеет смысл в основном, когда для вычисления соединяемых строк необходимо обратиться к столбцам других таблиц. В частности, это полезно, когда нужно передать значение функции, возвращающей набор данных. Например, если предположить, что `vertices(polygon)` возвращает набор вершин многоугольника, близкие вершины многоугольников из таблицы `polygons` можно получить так:

```
SELECT p1.id, p2.id, v1, v2
FROM polygons p1, polygons p2,
     LATERAL vertices(p1.poly) v1,
     LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

Этот запрос можно записать и так:

```
SELECT p1.id, p2.id, v1, v2
FROM polygons p1 CROSS JOIN LATERAL vertices(p1.poly) v1,
     polygons p2 CROSS JOIN LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

или переформулировать другими способами. (Как уже упоминалось, в данном примере ключевое слово `LATERAL` не требуется, но мы добавили его для ясности.)

Особенно полезно бывает использовать `LEFT JOIN` с подзапросом `LATERAL`, чтобы исходные строки оказывались в результате, даже если подзапрос `LATERAL` не возвращает строк. Например, если функция `get_product_names()` выдаёт названия продуктов, выпущенных определённым производителем, но о продукции некоторых производителей информации нет, мы можем найти, каких именно, примерно так:

```
SELECT m.name
FROM manufacturers m LEFT JOIN LATERAL get_product_names(m.id) pname ON true
WHERE pname IS NULL;
```

7.2.2. Предложение `WHERE`

Предложение `WHERE` записывается так:

```
WHERE условие_ограничения
```

где `условие_ограничения` — любое выражение значения (см. [Раздел 4.2](#)), выдающее результат типа `boolean`.

После обработки предложения `FROM` каждая строка полученной виртуальной таблицы проходит проверку по условию ограничения. Если результат условия равен `true`, эта строка остаётся в выходной таблице, а иначе (если результат равен `false` или `NULL`) отбрасывается. В условии ограничения, как правило, задействуется минимум один столбец из таблицы, полученной на выходе `FROM`.

Хотя строго говоря, это не требуется, но в противном случае предложение `WHERE` будет бессмысленным.

Примечание

Условие для внутреннего соединения можно записать как в предложении `WHERE`, так и в предложении `JOIN`. Например, это выражение:

```
FROM a, b WHERE a.id = b.id AND b.val > 5
```

равнозначно этому:

```
FROM a INNER JOIN b ON (a.id = b.id) WHERE b.val > 5
```

и возможно, даже этому:

```
FROM a NATURAL JOIN b WHERE b.val > 5
```

Какой вариант выбрать, в основном дело вкуса и стиля. Вариант с `JOIN` внутри предложения `FROM`, возможно, не лучший с точки зрения совместимости с другими СУБД, хотя он и описан в стандарте SQL. Но для внешних соединений других вариантов нет: их можно записывать только во `FROM`. Предложения `ON` и `USING` во внешних соединениях *не* равнозначны условию `WHERE`, так как они могут добавлять строки (для входных строк без соответствия), а также удалять их из конечного результата.

Несколько примеров запросов с `WHERE`:

```
SELECT ... FROM fdt WHERE c1 > 5
```

```
SELECT ... FROM fdt WHERE c1 IN (1, 2, 3)
```

```
SELECT ... FROM fdt WHERE c1 IN (SELECT c1 FROM t2)
```

```
SELECT ... FROM fdt WHERE c1 IN (SELECT c3 FROM t2 WHERE c2 = fdt.c1 + 10)
```

```
SELECT ... FROM fdt WHERE c1 BETWEEN (SELECT c3 FROM t2 WHERE c2 = fdt.c1 + 10) AND 100
```

```
SELECT ... FROM fdt WHERE EXISTS (SELECT c1 FROM t2 WHERE c2 > fdt.c1)
```

`fdt` — название таблицы, порождённой в предложении `FROM`. Строки, которые не соответствуют условию `WHERE`, исключаются из `fdt`. Обратите внимание, как в качестве выражений значения используются скалярные подзапросы. Как и любые другие запросы, подзапросы могут содержать сложные табличные выражения. Заметьте также, что `fdt` используется в подзапросах. Дополнение имени `c1` в виде `fdt.c1` необходимо только, если в порождённой таблице в подзапросе также оказывается столбец `c1`. Полное имя придаёт ясность даже там, где без него можно обойтись. Этот пример показывает, как область именования столбцов внешнего запроса распространяется на все вложенные в него внутренние запросы.

7.2.3. Предложения `GROUP BY` и `HAVING`

Строки порождённой входной таблицы, прошедшие фильтр `WHERE`, можно сгруппировать с помощью предложения `GROUP BY`, а затем оставить в результате только нужные группы строк, используя предложение `HAVING`.

```
SELECT список_выборки
FROM ...
[WHERE ...]
GROUP BY группирующий_столбец [, группирующий_столбец]...
```

Предложение `GROUP BY` группирует строки таблицы, объединяя их в одну группу при совпадении значений во всех перечисленных столбцах. Порядок, в котором указаны столбцы, не имеет значе-

ния. В результате наборы строк с одинаковыми значениями преобразуются в отдельные строки, представляющие все строки группы. Это может быть полезно для устранения избыточности выходных данных и/или для вычисления агрегатных функций, применённых к этим группам. Например:

```
=> SELECT * FROM test1;
```

```
x | y
---+---
a | 3
c | 2
b | 5
a | 1
(4 rows)
```

```
=> SELECT x FROM test1 GROUP BY x;
```

```
x
---
a
b
c
(3 rows)
```

Во втором запросе мы не могли написать `SELECT * FROM test1 GROUP BY x`, так как для столбца `y` нет единого значения, связанного с каждой группой. Однако столбцы, по которым выполняется группировка, можно использовать в списке выборки, так как они имеют единственное значение в каждой группе.

Вообще говоря, в группированной таблице столбцы, не включённые в список `GROUP BY`, можно использовать только в агрегатных выражениях. Пример такого агрегатного выражения:

```
=> SELECT x, sum(y) FROM test1 GROUP BY x;
```

```
x | sum
---+-----
a |    4
b |    5
c |    2
(3 rows)
```

Здесь `sum` — агрегатная функция, вычисляющая единственное значение для всей группы. Подробную информацию о существующих агрегатных функциях можно найти в [Разделе 9.21](#).

Подсказка

Группировка без агрегатных выражений по сути выдаёт набор различающихся значений столбцов. Этот же результат можно получить с помощью предложения `DISTINCT` (см. [Подраздел 7.3.3](#)).

Взгляните на следующий пример: в нём вычисляется общая сумма продаж по каждому продукту (а не общая сумма по всем продуктам):

```
SELECT product_id, p.name, (sum(s.units) * p.price) AS sales
FROM products p LEFT JOIN sales s USING (product_id)
GROUP BY product_id, p.name, p.price;
```

В этом примере столбцы `product_id`, `p.name` и `p.price` должны присутствовать в списке `GROUP BY`, так как они используются в списке выборки. Столбец `s.units` может отсутствовать в списке `GROUP BY`, так как он используется только в агрегатном выражении (`sum(...)`), вычисляющем сумму продаж. Для каждого продукта этот запрос возвращает строку с итоговой суммой по всем продажам данного продукта.

Если бы в таблице `products` по столбцу `product_id` был создан первичный ключ, тогда в данном примере было бы достаточно сгруппировать строки по `product_id`, так как название и цена продукта *функционально зависят* от кода продукта и можно однозначно определить, какое название и цену возвращать для каждой группы по ID.

В стандарте SQL `GROUP BY` может группировать только по столбцам исходной таблицы, но расширение Postgres Pro позволяет использовать в `GROUP BY` столбцы из списка выборки. Также возможна группировка по выражениям, а не просто именам столбцов.

Если таблица была сгруппирована с помощью `GROUP BY`, но интерес представляют только некоторые группы, отфильтровать их можно с помощью предложения `HAVING`, действующего подобно `WHERE`. Записывается это так:

```
SELECT список_выборки FROM ... [WHERE ...] GROUP BY ... HAVING логическое_выражение
```

В предложении `HAVING` могут использоваться и группирующие выражения, и выражения, не участвующие в группировке (в этом случае это должны быть агрегирующие функции).

Пример:

```
=> SELECT x, sum(y) FROM test1 GROUP BY x HAVING sum(y) > 3;
```

```
x | sum
---+-----
a |    4
b |    5
(2 rows)
```

```
=> SELECT x, sum(y) FROM test1 GROUP BY x HAVING x < 'c';
```

```
x | sum
---+-----
a |    4
b |    5
(2 rows)
```

И ещё один более реалистичный пример:

```
SELECT product_id, p.name, (sum(s.units) * (p.price - p.cost)) AS profit
FROM products p LEFT JOIN sales s USING (product_id)
WHERE s.date > CURRENT_DATE - INTERVAL '4 weeks'
GROUP BY product_id, p.name, p.price, p.cost
HAVING sum(p.price * s.units) > 5000;
```

В данном примере предложение `WHERE` выбирает строки по столбцу, не включённому в группировку (выражение истинно только для продаж за последние четыре недели), тогда как предложение `HAVING` отфильтровывает группы с общей суммой продаж больше 5000. Заметьте, что агрегатные выражения не обязательно должны быть одинаковыми во всех частях запроса.

Если в запросе есть вызовы агрегатных функций, но нет предложения `GROUP BY`, строки всё равно будут группироваться: в результате окажется одна строка группы (или возможно, ни одной строки, если эта строка будет отброшена предложением `HAVING`). Это справедливо и для запросов, которые содержат только предложение `HAVING`, но не содержат вызовы агрегатных функций и предложение `GROUP BY`.

7.2.4. GROUPING SETS, CUBE И ROLLUP

Более сложные, чем описанные выше, операции группировки возможны с концепцией *наборов группирования*. Данные, выбранные предложениями `FROM` и `WHERE`, группируются отдельно для каждого заданного набора группирования, затем для каждой группы вычисляются агрегатные функции как для простых предложений `GROUP BY`, и в конце возвращаются результаты. Например:

```
=> SELECT * FROM items_sold;
```

brand	size	sales
Foo	L	10
Foo	M	20
Bar	M	15
Bar	L	5

(4 rows)

```
=> SELECT brand, size, sum(sales) FROM items_sold GROUP BY GROUPING SETS ((brand),
    (size), ());
```

brand	size	sum
Foo		30
Bar		20
	L	15
	M	35
		50

(5 rows)

В каждом внутреннем списке `GROUPING SETS` могут задаваться ноль или более столбцов или выражений, которые воспринимаются так же, как если бы они были непосредственно записаны в предложении `GROUP BY`. Пустой набор группировки означает, что все строки сводятся к одной группе (которая выводится, даже если входных строк нет), как описано выше для агрегатных функций без предложения `GROUP BY`.

Ссылки на группирующие столбцы или выражения заменяются в результирующих строках значениями `NULL` для тех группирующих наборов, в которых эти столбцы отсутствуют. Чтобы можно было понять, результатом какого группирования стала конкретная выходная строка, предназначена функция, описанная в [Таблице 9.66](#).

Для указания двух распространённых видов наборов группирования предусмотрена краткая запись. Предложение формы

```
ROLLUP ( e1, e2, e3, ... )
```

представляет заданный список выражений и всех префиксов списка, включая пустой список; то есть оно равнозначно записи

```
GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)
```

Оно часто применяется для анализа иерархических данных, например, для суммирования зарплаты по отделам, подразделениям и компании в целом.

Предложение формы

```
CUBE ( e1, e2, ... )
```

представляет заданный список и все его возможные подмножества (степень множества). Таким образом, запись

```
CUBE ( a, b, c )
```

равнозначна

```
GROUPING SETS (
    ( a, b, c ),
    ( a, b ),
    ( a ),
    ( )
)
```



```

( a,      c ),
( a              ),
(      b, c ),
(      b        ),
(              c ),
(              )
)

```

Элементами предложений CUBE и ROLLUP могут быть либо отдельные выражения, либо вложенные списки элементов в скобках. Вложенные списки обрабатываются как атомарные единицы, с которыми формируются отдельные наборы группирования. Например:

```
CUBE ( (a, b), (c, d) )
```

равнозначно

```

GROUPING SETS (
  ( a, b, c, d ),
  ( a, b          ),
  (              c, d ),
  (              )
)

```

и

```
ROLLUP ( a, (b, c), d )
```

равнозначно

```

GROUPING SETS (
  ( a, b, c, d ),
  ( a, b, c      ),
  ( a              ),
  (              )
)

```

Конструкции CUBE и ROLLUP могут применяться либо непосредственно в предложении GROUP BY, либо вкладываться внутрь предложения GROUPING SETS. Если одно предложение GROUPING SETS вкладывается внутрь другого, результат будет таким же, как если бы все элементы внутреннего предложения были записаны непосредственно во внешнем.

Если в одном предложении GROUP BY задаётся несколько элементов группирования, окончательный список наборов группирования образуется как прямое произведение этих элементов. Например:

```
GROUP BY a, CUBE (b, c), GROUPING SETS ((d), (e))
```

равнозначно

```

GROUP BY GROUPING SETS (
  (a, b, c, d), (a, b, c, e),
  (a, b, d),   (a, b, e),
  (a, c, d),   (a, c, e),
  (a, d),     (a, e)
)

```

Если задаётся несколько элементов группирования, окончательный список наборов группирования может содержать дублирующие результаты. Например:

```
GROUP BY ROLLUP (a, b), ROLLUP (a, c)
```

равнозначно

```
GROUP BY GROUPING SETS (
```

```
(a, b, c),
(a, b),
(a, b),
(a, c),
(a),
(a),
(a, c),
(a),
()
```

Если эти дублирующиеся результаты нежелательны, их можно убрать, добавив `DISTINCT` непосредственно в предложение `GROUP BY`. Таким образом:

```
GROUP BY DISTINCT ROLLUP (a, b), ROLLUP (a, c)
```

равнозначно

```
GROUP BY GROUPING SETS (
  (a, b, c),
  (a, b),
  (a, c),
  (a),
  ()
)
```

Это не то же самое, что `SELECT DISTINCT`, поскольку результирующие строки всё же могут содержать дублирующиеся записи. Если любой из столбцов, не участвующих в группировании, содержит значение `NULL`, в результате оно не будет отличаться от `NULL`, полученного при группировании этого столбца.

Примечание

Конструкция `(a, b)` обычно воспринимается в выражениях как [конструктор строки](#). Однако в предложении `GROUP BY` на верхнем уровне выражений запись `(a, b)` воспринимается как список выражений, как описано выше. Если вам по какой-либо причине *нужен* именно конструктор строки в выражении группирования, используйте запись `ROW(a, b)`.

7.2.5. Обработка оконных функций

Если запрос содержит оконные функции (см. [Раздел 3.5](#), [Раздел 9.22](#) и [Подраздел 4.2.8](#)), эти функции вычисляются после каждой группировки, агрегатных выражений и фильтрации `HAVING`. Другими словами, если в запросе есть агрегатные функции, предложения `GROUP BY` или `HAVING`, оконные функции видят не исходные строки, полученные из `FROM/WHERE`, а сгруппированные.

Когда используются несколько оконных функций, все оконные функции, имеющие в своих определениях равнозначные предложения `PARTITION BY` и `ORDER BY`, гарантированно видят один порядок сортировки входных строк, даже если `ORDER BY` не определяет этот порядок однозначно. Однако относительно функций с разными формулировками `PARTITION BY` и `ORDER BY` никаких гарантий не даётся. (В таких случаях между проходами вычислений оконных функций обычно требуется дополнительный этап сортировки и эта сортировка может не сохранять порядок строк, равнозначный с точки зрения `ORDER BY`.)

В настоящее время оконные функции всегда требуют предварительно отсортированных данных, так что результат запроса будет отсортирован согласно тому или иному предложению `PARTITION BY/ORDER BY` оконных функций. Однако полагаться на это не следует. Если вы хотите, чтобы результаты сортировались определённым образом, явно добавьте предложение `ORDER BY` на верхнем уровне запроса.

7.3. Списки выборки

Как говорилось в предыдущем разделе, табличное выражение в `SELECT` создаёт промежуточную виртуальную таблицу, возможно объединяя таблицы, представления, группируя и исключая лишние строки и т. д. Полученная таблица передаётся для обработки в *список выборки*. Этот список выбирает, какие *столбцы* промежуточной таблицы должны выводиться в результате и как именно.

7.3.1. Элементы списка выборки

Простейший список выборки образует элемент `*`, который выбирает все столбцы из полученного табличного выражения. Список выборки также может содержать список выражений значения через запятую (как определено в [Разделе 4.2](#)). Например, это может быть список имён столбцов:

```
SELECT a, b, c FROM ...
```

Имена столбцов `a`, `b` и `c` представляют либо фактические имена столбцов таблиц, перечисленных в предложении `FROM`, либо их псевдонимы, определённые как описано в [Подразделе 7.2.1.2](#). Пространство имён в списке выборки то же, что и в предложении `WHERE`, если не используется группировка. В противном случае оно совпадает с пространством имён предложения `HAVING`.

Если столбец с заданным именем есть в нескольких таблицах, необходимо также указать имя таблицы, например так:

```
SELECT tbl1.a, tbl2.a, tbl1.b FROM ...
```

Обращаясь к нескольким таблицам, бывает удобно получить сразу все столбцы одной из таблиц:

```
SELECT tbl1.*, tbl2.a FROM ...
```

Подробнее запись `имя_таблицы.*` описывается в [Подразделе 8.16.5](#).

Если в списке выборки используется обычное выражение значения, по сути при этом в возвращаемую таблицу добавляется новый виртуальный столбец. Выражение значения вычисляется один раз для каждой строки результата со значениями столбцов в данной строке. Хотя выражения в списке выборки не обязательно должны обращаться к столбцам табличного выражения из предложения `FROM`; они могут содержать, например и простые арифметические выражения.

7.3.2. Метки столбцов

Элементам в списке выборки можно назначить имена для последующей обработки, например, для указания в предложении `ORDER BY` или для вывода в клиентском приложении. Например:

```
SELECT a AS value, b + c AS sum FROM ...
```

Если выходное имя столбца не определено (с помощью `AS`), система назначает имя сама. Для простых ссылок на столбцы этим именем становится имя целевого столбца, а для вызовов функций это имя функции. Для сложных выражений система генерирует некоторое подходящее имя.

Слово `AS` обычно можно опустить, но в некоторых случаях, когда желаемое имя столбца совпадает с ключевым словом `Postgres Pro`, нужно написать `AS` или заключить имя столбца в кавычки во избежание неоднозначности. (В [Приложении С](#) перечислены ключевые слова, для которых нужно использовать `AS` в метке столбца.) Например, `FROM` — ключевое слово, поэтому такой вариант не будет работать:

```
SELECT a from, b + c AS sum FROM ...
```

а любой из этих сработает:

```
SELECT a AS from, b + c AS sum FROM ...  
SELECT a "from", b + c AS sum FROM ...
```

Для предотвращения конфликта с ключевыми словами, которые могут появиться в будущем, рекомендуется всегда писать `AS` или заключать метки выходных столбцов в кавычки.

Примечание

Именование выходных столбцов отличается от того, что происходит в предложении `FROM` (см. [Подраздел 7.2.1.2](#)). Один столбец можно переименовать дважды, но на выходе окажется имя, назначенное в списке выборки.

7.3.3. DISTINCT

После обработки списка выборки в результирующей таблице можно дополнительно исключить дублирующиеся строки. Для этого сразу после `SELECT` добавляется ключевое слово `DISTINCT`:

```
SELECT DISTINCT список_выборки ...
```

(Чтобы явно включить поведение по умолчанию, когда возвращаются все строки, вместо `DISTINCT` можно указать ключевое слово `ALL`.)

Две строки считаются разными, если они содержат различные значения минимум в одном столбце. При этом значения `NULL` полагаются равными.

Кроме того, можно явно определить, какие строки будут считаться различными, следующим образом:

```
SELECT DISTINCT ON (выражение [, выражение ...]) список_выборки ...
```

Здесь *выражение* — обычное выражение значения, вычисляемое для всех строк. Строки, для которых перечисленные выражения дают один результат, считаются дублирующимися и возвращается только первая строка из такого набора. Заметьте, что «первая строка» набора может быть любой, если только запрос не включает сортировку, гарантирующую однозначный порядок строк, поступающих в фильтр `DISTINCT`. (Обработка `DISTINCT ON` производится после сортировки `ORDER BY`.)

Предложение `DISTINCT ON` не описано в стандарте SQL и иногда его применение считается плохим стилем из-за возможной неопределённости в результатах. При разумном использовании `GROUP BY` и подзапросов во `FROM` можно обойтись без этой конструкции, но часто она бывает удобнее.

7.4. Сочетание запросов (UNION, INTERSECT, EXCEPT)

Результаты двух запросов можно обработать, используя операции над множествами: объединение, пересечение и вычитание. Эти операции записываются соответственно так:

```
запрос1 UNION [ALL] запрос2
запрос1 INTERSECT [ALL] запрос2
запрос1 EXCEPT [ALL] запрос2
```

Здесь *запрос1* и *запрос2* — это запросы, в которых могут использоваться все возможности, рассмотренные до этого.

`UNION` по сути добавляет результаты второго запроса к результатам первого (хотя никакой порядок возвращаемых строк при этом не гарантируется). Более того, эта операция убирает дублирующиеся строки из результата так же, как это делает `DISTINCT`, если только не указано `UNION ALL`.

`INTERSECT` возвращает все строки, содержащиеся в результате и первого, и второго запроса. Дублирующиеся строки отфильтровываются, если не указано `ALL`.

`EXCEPT` возвращает все строки, которые есть в результате первого запроса, но отсутствуют в результате второго. (Иногда это называют *разницей* двух запросов.) И здесь дублирующиеся строки отфильтровываются, если не указано `ALL`.

Чтобы можно было вычислить объединение, пересечение или разницу результатов двух запросов, эти запросы должны быть «совместимыми для объединения», что означает, что они должны иметь

одинаковое число столбцов и соответствующие столбцы должны быть совместимых типов, как описывается в [Разделе 10.5](#).

Операции над множествами можно объединять, например

```
запрос1 UNION запрос2 EXCEPT запрос3
```

что равнозначно

```
(запрос1 UNION запрос2) EXCEPT запрос3
```

Как показано выше, вы можете использовать круглые скобки, чтобы управлять очередью обработки. Без круглых скобок UNION и EXCEPT объединяются слева направо, но оператор INTERSECT имеет больший приоритет, чем первые два. Таким образом,

```
запрос1 UNION запрос2 INTERSECT запрос3
```

означает

```
запрос1 UNION (запрос2 INTERSECT запрос3)
```

Вы также можете заключить отдельный запрос в круглые скобки. Это важно, если в запросе необходимо использовать любое из предложений, обсуждаемых в следующих разделах, например LIMIT. Без круглых скобок возникнет синтаксическая ошибка или предложение будет восприниматься как относящееся к результату операции над множествами, а не к её аргументам. Например,

```
SELECT a FROM b UNION SELECT x FROM y LIMIT 10
```

не вызовет ошибку, но будет означать

```
(SELECT a FROM b UNION SELECT x FROM y) LIMIT 10
```

а не

```
SELECT a FROM b UNION (SELECT x FROM y LIMIT 10)
```

7.5. Сортировка строк (ORDER BY)

После того как запрос выдал таблицу результатов (после обработки списка выборки), её можно отсортировать. Если сортировка не задана, строки возвращаются в неопределённом порядке. Фактический порядок строк в этом случае будет зависеть от плана соединения и сканирования, а также от порядка данных на диске, поэтому полагаться на него нельзя. Определённый порядок выводимых строк гарантируется, только если этап сортировки задан явно.

Порядок сортировки определяет предложение ORDER BY:

```
SELECT список_выборки
FROM табличное_выражение
ORDER BY выражение_сортировки1 [ASC | DESC] [NULLS { FIRST | LAST }]
        [, выражение_сортировки2 [ASC | DESC] [NULLS { FIRST | LAST }] ...]
```

Выражениями сортировки могут быть любые выражения, допустимые в списке выборки запроса. Например:

```
SELECT a, b FROM table1 ORDER BY a + b, c;
```

Когда указывается несколько выражений, последующие значения позволяют отсортировать строки, в которых совпали все предыдущие значения. Каждое выражение можно дополнить ключевыми словами ASC или DESC, которые выбирают сортировку соответственно по возрастанию или убыванию. По умолчанию принят порядок по возрастанию (ASC). При сортировке по возрастанию сначала идут меньшие значения, где понятие «меньше» определяется оператором <. Подобным образом, сортировка по возрастанию определяется оператором >.¹

Для определения места значений NULL можно использовать указания NULLS FIRST и NULLS LAST, которые помещают значения NULL соответственно до или после значений не NULL. По умолча-

¹На деле Postgres Pro определяет порядок сортировки для ASC и DESC по классу оператора B-дерева по умолчанию для типа данных выражения. Обычно типы данных создаются так, что этому порядку соответствуют операторы < и >, но возможно разработать собственный тип данных, который будет вести себя по-другому.

нию значения NULL считаются больше любых других, то есть подразумевается NULLS FIRST для порядка DESC и NULLS LAST в противном случае.

Заметьте, что порядки сортировки определяются независимо для каждого столбца. Например, ORDER BY x, y DESC означает ORDER BY x ASC, y DESC, и это не то же самое, что ORDER BY x DESC, y DESC.

Здесь *выражение_сортировки* может быть меткой столбца или номером выводимого столбца, как в данном примере:

```
SELECT a + b AS sum, c FROM table1 ORDER BY sum;
SELECT a, max(b) FROM table1 GROUP BY a ORDER BY 1;
```

Оба эти запроса сортируют результат по первому столбцу. Заметьте, что имя выводимого столбца должно оставаться само по себе, его нельзя использовать в выражении. Например, это *ошибка*:

```
SELECT a + b AS sum, c FROM table1 ORDER BY sum + c;           -- неправильно
```

Это ограничение позволяет уменьшить неоднозначность. Тем не менее неоднозначность возможна, когда в ORDER BY указано простое имя, но оно соответствует и имени выходного столбца, и столбцу из табличного выражения. В этом случае используется выходной столбец. Эта ситуация может возникнуть, только когда с помощью AS выходному столбцу назначается то же имя, что имеет столбец в другой таблице.

ORDER BY можно применить к результату комбинации UNION, INTERSECT и EXCEPT, но в этом случае возможна сортировка только по номерам или именам столбцов, но не по выражениям.

7.6. LIMIT и OFFSET

Указания LIMIT и OFFSET позволяют получить только часть строк из тех, что выдал остальной запрос:

```
SELECT список_выборки
FROM табличное_выражение
[ ORDER BY ... ]
[ LIMIT { число | ALL } ] [ OFFSET число ]
```

Если указывается число LIMIT, в результате возвращается не больше заданного числа строк (меньше может быть, если сам запрос выдал меньшее количество строк). LIMIT ALL равносильно отсутствию указания LIMIT, как и LIMIT с аргументом NULL.

OFFSET указывает пропустить указанное число строк, прежде чем начать выдавать строки. OFFSET 0 равносильно отсутствию указания OFFSET, как и OFFSET с аргументом NULL.

Если указано и OFFSET, и LIMIT, сначала система пропускает OFFSET строк, а затем начинает подсчитывать строки для ограничения LIMIT.

Применяя LIMIT, важно использовать также предложение ORDER BY, чтобы строки результата выдавались в определённом порядке. Иначе будут возвращаться непредсказуемые подмножества строк. Вы можете запросить строки с десятой по двадцатую, но какой порядок вы имеете в виду? Порядок будет неизвестен, если не добавить ORDER BY.

Оптимизатор запроса учитывает ограничение LIMIT, строя планы выполнения запросов, поэтому вероятнее всего планы (а значит и порядок строк) будут меняться при разных LIMIT и OFFSET. Таким образом, различные значения LIMIT/OFFSET, выбирающие разные подмножества результатов запроса, *приведут к несогласованности результатов*, если не установить предсказуемую сортировку с помощью ORDER BY. Это не ошибка, а неизбежное следствие того, что SQL не гарантирует вывод результатов запроса в некотором порядке, если порядок не определён явно предложением ORDER BY.

Строки, пропускаемые согласно предложению OFFSET, тем не менее должны вычисляться на сервере. Таким образом, при больших значениях OFFSET работает неэффективно.

7.7. Списки VALUES

Предложение `VALUES` позволяет создать «постоянную таблицу», которую можно использовать в запросе, не создавая и не наполняя таблицу в БД. Синтаксис предложения:

```
VALUES ( выражение [, ...] ) [, ...]
```

Для каждого списка выражений в скобках создаётся строка таблицы. Все списки должны иметь одинаковое число элементов (т. е. число столбцов в таблице) и соответствующие элементы во всех списках должны иметь совместимые типы данных. Фактический тип данных столбцов результата определяется по тем же правилам, что и для `UNION` (см. [Раздел 10.5](#)).

Как пример:

```
VALUES (1, 'one'), (2, 'two'), (3, 'three');
```

вернёт таблицу из двух столбцов и трёх строк. Это равносильно такому запросу:

```
SELECT 1 AS column1, 'one' AS column2
UNION ALL
SELECT 2, 'two'
UNION ALL
SELECT 3, 'three';
```

По умолчанию Postgres Pro назначает столбцам таблицы `VALUES` имена `column1`, `column2` и т. д. Имена столбцов не определены в стандарте SQL и в другой СУБД они могут быть другими, поэтому обычно лучше переопределить имена списком псевдонимов, например так:

```
=> SELECT * FROM (VALUES (1, 'one'), (2, 'two'), (3, 'three')) AS t (num, letter);
 num | letter
-----+-----
   1 | one
   2 | two
   3 | three
(3 rows)
```

Синтаксически список `VALUES` с набором выражений равнозначен:

```
SELECT список_выборки FROM табличное_выражение
```

и допускается везде, где допустим `SELECT`. Например, вы можете использовать его в составе `UNION` или добавить к нему *определение сортировки* (`ORDER BY`, `LIMIT` и/или `OFFSET`). `VALUES` чаще всего используется как источник данных для команды `INSERT`, а также как подзапрос.

За дополнительными сведениями обратитесь к справке [VALUES](#).

7.8. Запросы WITH (Общие табличные выражения)

`WITH` предоставляет способ записывать дополнительные операторы для применения в больших запросах. Эти операторы, которые также называют общими табличными выражениями (Common Table Expressions, CTE), можно представить как определения временных таблиц, существующих только для одного запроса. Дополнительным оператором в предложении `WITH` может быть `SELECT`, `INSERT`, `UPDATE` или `DELETE`, а само предложение `WITH` присоединяется к основному оператору, которым может быть `SELECT`, `INSERT`, `UPDATE`, `DELETE` или `MERGE`.

7.8.1. SELECT в WITH

Основное предназначение `SELECT` в предложении `WITH` заключается в разбиении сложных запросов на простые части. Например, запрос:

```
WITH regional_sales AS (
  SELECT region, SUM(amount) AS total_sales
  FROM orders
```

```

    GROUP BY region
), top_regions AS (
    SELECT region
    FROM regional_sales
    WHERE total_sales > (SELECT SUM(total_sales)/10 FROM regional_sales)
)
SELECT region,
       product,
       SUM(quantity) AS product_units,
       SUM(amount) AS product_sales
FROM orders
WHERE region IN (SELECT region FROM top_regions)
GROUP BY region, product;

```

выводит итоги по продажам только для передовых регионов. Предложение `WITH` определяет два дополнительных оператора `regional_sales` и `top_regions` так, что результат `regional_sales` используется в `top_regions`, а результат `top_regions` используется в основном запросе `SELECT`. Этот пример можно было бы переписать без `WITH`, но тогда нам понадобятся два уровня вложенных подзапросов `SELECT`. Показанным выше способом это можно сделать немного проще.

7.8.2. Рекурсивные запросы

Необязательное указание `RECURSIVE` превращает `WITH` из просто удобной синтаксической конструкции в средство реализации того, что невозможно в стандартном SQL. Используя `RECURSIVE`, запрос `WITH` может обращаться к собственному результату. Очень простой пример, суммирующий числа от 1 до 100:

```

WITH RECURSIVE t(n) AS (
    VALUES (1)
    UNION ALL
    SELECT n+1 FROM t WHERE n < 100
)
SELECT sum(n) FROM t;

```

В общем виде рекурсивный запрос `WITH` всегда записывается как *не рекурсивная часть*, потом `UNION` (или `UNION ALL`), а затем *рекурсивная часть*, где только в рекурсивной части можно обратиться к результату запроса. Такой запрос выполняется следующим образом:

Вычисление рекурсивного запроса

1. Вычисляется не рекурсивная часть. Для `UNION` (но не `UNION ALL`) отбрасываются дублирующиеся строки. Все оставшиеся строки включаются в результат рекурсивного запроса и также помещаются во временную *рабочую таблицу*.
2. Пока рабочая таблица не пуста, повторяются следующие действия:
 - a. Вычисляется рекурсивная часть так, что рекурсивная ссылка на сам запрос обращается к текущему содержимому рабочей таблицы. Для `UNION` (но не `UNION ALL`) отбрасываются дублирующиеся строки и строки, дублирующие ранее полученные. Все оставшиеся строки включаются в результат рекурсивного запроса и также помещаются во временную *промежуточную таблицу*.
 - b. Содержимое рабочей таблицы заменяется содержимым промежуточной таблицы, а затем промежуточная таблица очищается.

Примечание

Хотя указание `RECURSIVE` позволяет определять рекурсивные запросы, внутри такие запросы обрабатываются итерационно.

В показанном выше примере в рабочей таблице на каждом этапе содержится всего одна строка и в ней последовательно накапливаются значения от 1 до 100. На сотом шаге, благодаря условию `WHERE`, не возвращается ничего, так что вычисление запроса завершается.

Рекурсивные запросы обычно применяются для работы с иерархическими или древовидными структурами данных. В качестве полезного примера можно привести запрос, находящий все непосредственные и косвенные составные части продукта, используя только таблицу с прямыми связями:

```
WITH RECURSIVE included_parts(sub_part, part, quantity) AS (
    SELECT sub_part, part, quantity FROM parts WHERE part = 'our_product'
    UNION ALL
    SELECT p.sub_part, p.part, p.quantity * pr.quantity
    FROM included_parts pr, parts p
    WHERE p.part = pr.sub_part
)
SELECT sub_part, SUM(quantity) as total_quantity
FROM included_parts
GROUP BY sub_part
```

7.8.2.1. Порядок поиска

Когда при выполнении рекурсивного запроса производится обход дерева, результаты можно получать в разном порядке: «сначала в глубину» или «сначала в ширину». Для этого можно в дополнение к другим столбцам вычислить упорядочивающий столбец и использовать его для сортировки результатов. Обратите внимание, что такой столбец не определяет порядок обхода строк запросом — этот порядок, как всегда, зависит от реализации SQL. Упорядочивающий столбец лишь позволяет удобным образом упорядочить полученные результаты.

Чтобы отсортировать результаты в порядке «сначала в глубину», для каждой строки результата вычисляется массив уже просмотренных строк. Например, рассмотрим следующий запрос, который выполняет поиск в таблице `tree` по полю `link`:

```
WITH RECURSIVE search_tree(id, link, data) AS (
    SELECT t.id, t.link, t.data
    FROM tree t
    UNION ALL
    SELECT t.id, t.link, t.data
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree;
```

Чтобы добавить информацию для упорядочивания «сначала в глубину», вы можете написать так:

```
WITH RECURSIVE search_tree(id, link, data, path) AS (
    SELECT t.id, t.link, t.data, ARRAY[t.id]
    FROM tree t
    UNION ALL
    SELECT t.id, t.link, t.data, path || t.id
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;
```

В общем случае, когда для выявления строки нужно использовать несколько полей, следует использовать массив строк. Например, если нужно отследить поля `f1` и `f2`:

```
WITH RECURSIVE search_tree(id, link, data, path) AS (
    SELECT t.id, t.link, t.data, ARRAY[ROW(t.f1, t.f2)]
    FROM tree t
```

```

UNION ALL
  SELECT t.id, t.link, t.data, path || ROW(t.f1, t.f2)
  FROM tree t, search_tree st
  WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;

```

Подсказка

ROW() можно опустить, когда нужно отслеживать только одно поле (как обычно и бывает). При этом будет использоваться не массив данных составного типа, а простой массив, что более эффективно.

Чтобы получить результаты, отсортированные «сначала в ширину», можно добавить столбец, отслеживающий глубину поиска, например:

```

WITH RECURSIVE search_tree(id, link, data, depth) AS (
  SELECT t.id, t.link, t.data, 0
  FROM tree t
  UNION ALL
  SELECT t.id, t.link, t.data, depth + 1
  FROM tree t, search_tree st
  WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY depth;

```

Для обеспечения стабильности сортировки добавьте столбцы данных в качестве столбцов вторичной сортировки.

Подсказка

Этот алгоритм рекурсивного вычисления запроса выдаёт в результате узлы, упорядоченные «сначала в ширину». И всё же это зависит от реализации, поэтому полагаться на это поведение не следует. Порядок строк внутри каждого уровня, конечно, не определён, поэтому в любом случае может потребоваться явное упорядочивание.

Для определения столбца, который будет вычисляться для упорядочивания «сначала в глубину» или «сначала в ширину», есть встроенный синтаксис. Например:

```

WITH RECURSIVE search_tree(id, link, data) AS (
  SELECT t.id, t.link, t.data
  FROM tree t
  UNION ALL
  SELECT t.id, t.link, t.data
  FROM tree t, search_tree st
  WHERE t.id = st.link
) SEARCH DEPTH FIRST BY id SET ordercol
SELECT * FROM search_tree ORDER BY ordercol;

```

```

WITH RECURSIVE search_tree(id, link, data) AS (
  SELECT t.id, t.link, t.data
  FROM tree t
  UNION ALL
  SELECT t.id, t.link, t.data
  FROM tree t, search_tree st
  WHERE t.id = st.link
) SEARCH BREADTH FIRST BY id SET ordercol

```

```
SELECT * FROM search_tree ORDER BY ordercol;
```

Внутри этот синтаксис преобразуется в формы, подобные вышеприведённым формам, составленным вручную. В предложении `SEARCH` указывается, какой тип поиска необходим — «сначала в глубину» или «сначала в ширину», список отслеживаемых для сортировки столбцов и имя столбца, который будет содержать данные, используемые для упорядочивания. Этот столбец будет неявно добавлен в результирующие строки в СТЕ.

7.8.2.2. Выявление циклов

Работая с рекурсивными запросами, важно обеспечить, чтобы рекурсивная часть запроса в конце концов не выдала никаких кортежей (строк), в противном случае цикл будет бесконечным. Иногда для этого достаточно применять `UNION` вместо `UNION ALL`, так как при этом будут отбрасываться строки, которые уже есть в результате. Однако часто в цикле выдаются строки, не совпадающие полностью с предыдущими: в таких случаях может иметь смысл проверить одно или несколько полей, чтобы определить, не была ли текущая точка достигнута раньше. Стандартный способ решения подобных задач — вычислить массив с уже обработанными значениями. Например, снова рассмотрим следующий запрос, просматривающий таблицу `graph` по полю `link`:

```
WITH RECURSIVE search_graph(id, link, data, depth) AS (
    SELECT g.id, g.link, g.data, 0
    FROM graph g
    UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1
    FROM graph g, search_graph sg
    WHERE g.id = sg.link
)
SELECT * FROM search_graph;
```

Этот запрос заикнется, если связи `link` содержат циклы. Так как нам нужно получать в результате «depth», одно лишь изменение `UNION ALL` на `UNION` не позволит избежать заикливания. Вместо этого мы должны как-то определить, что уже достигали текущей строки, пройдя некоторый путь. Для этого, добавив два столбца `is_cycle` и `path`, мы получаем запрос, защищённый от заикливания:

```
WITH RECURSIVE search_graph(id, link, data, depth, is_cycle, path) AS (
    SELECT g.id, g.link, g.data, 0,
           false,
           ARRAY[g.id]
    FROM graph g
    UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1,
           g.id = ANY(path),
           path || g.id
    FROM graph g, search_graph sg
    WHERE g.id = sg.link AND NOT is_cycle
)
SELECT * FROM search_graph;
```

Помимо предотвращения циклов, значения массива часто бывают полезны сами по себе для представления «пути», приведшего к определённой строке.

В общем случае, когда для выявления цикла нужно проверять несколько полей, следует использовать массив строк. Например, если нужно сравнить поля `f1` и `f2`:

```
WITH RECURSIVE search_graph(id, link, data, depth, is_cycle, path) AS (
    SELECT g.id, g.link, g.data, 0,
           false,
           ARRAY[ROW(g.f1, g.f2)]
    FROM graph g
    UNION ALL
```

```

SELECT g.id, g.link, g.data, sg.depth + 1,
       ROW(g.f1, g.f2) = ANY(path),
       path || ROW(g.f1, g.f2)
FROM graph g, search_graph sg
WHERE g.id = sg.link AND NOT is_cycle
)
SELECT * FROM search_graph;

```

Подсказка

Часто для выявления цикла достаточно одного поля, и тогда ROW() можно опустить. При этом будет использоваться не массив данных составного типа, а простой массив, что более эффективно.

Для упрощения выявления циклов есть встроенный синтаксис. Запрос выше можно записать так:

```

WITH RECURSIVE search_graph(id, link, data, depth) AS (
  SELECT g.id, g.link, g.data, 1
  FROM graph g
  UNION ALL
  SELECT g.id, g.link, g.data, sg.depth + 1
  FROM graph g, search_graph sg
  WHERE g.id = sg.link
) CYCLE id SET is_cycle USING path
SELECT * FROM search_graph;

```

и внутри он будет приведён к вышеуказанной форме. В предложении CYCLE сначала указывается список столбцов, отслеживаемых для выявления цикла, затем имя столбца, показывающего признак выявления цикла, и, наконец, имя ещё одного столбца, в котором будет отслеживаться путь. Столбцы, указывающие цикл и путь, будут неявно добавлены в результирующие строки в CTE.

Подсказка

Столбец с указанием пути цикла вычисляется так же, как и столбец для упорядочивания результатов «сначала в глубину», что показано в предыдущем подразделе. Запрос может содержать одновременно SEARCH и CYCLE, но в этом случае для упорядочивания «сначала в глубину» будут производиться лишние вычисления, поэтому эффективнее использовать только CYCLE и сортировать результаты по столбцу пути. Однако использование в запросе и SEARCH, и CYCLE оправдано, когда нужен порядок «сначала в ширину».

Для тестирования запросов, которые могут заикливаться, есть хороший приём — добавить LIMIT в родительский запрос. Например, следующий запрос заиклится, если не добавить предложение LIMIT:

```

WITH RECURSIVE t(n) AS (
  SELECT 1
  UNION ALL
  SELECT n+1 FROM t
)
SELECT n FROM t LIMIT 100;

```

Но в данном случае этого не происходит, так как в Postgres Pro запрос WITH выдаёт столько строк, сколько фактически принимает родительский запрос. В производственной среде использовать этот приём не рекомендуется, так как другие системы могут вести себя по-другому. Кроме того, это не будет работать, если внешний запрос сортирует результаты рекурсивного запроса или соединяет их с другой таблицей, так как в подобных случаях внешний запрос обычно всё равно выбирает результат запроса WITH полностью.

7.8.3. Материализация общих табличных выражений

Запросы `WITH` имеют полезное свойство — обычно они вычисляются только раз для всего родительского запроса, даже если этот запрос или соседние запросы `WITH` обращаются к ним неоднократно. Таким образом, сложные вычисления, результаты которых нужны в нескольких местах, можно выносить в запросы `WITH` в целях оптимизации. Кроме того, такие запросы позволяют избежать нежелательных вычислений функций с побочными эффектами. Однако есть и обратная сторона — оптимизатор не может распространить ограничения родительского запроса на неоднократно задействуемый запрос `WITH`, так как это может повлиять на использование результата `WITH` во всех местах, тогда как должно повлиять только в одном. Многократно задействуемый запрос `WITH` будет выполняться буквально и возвращать все строки, включая те, что потом может отбросить родительский запрос. (Но как было сказано выше, вычисление может остановиться раньше, если в ссылке на этот запрос затребуются только ограниченное число строк.)

Однако если запрос `WITH` является нерекурсивным и свободным от побочных эффектов (то есть это `SELECT`, не вызывающий изменчивых функций), он может быть свёрнут в родительский запрос, что позволит оптимизировать совместно два уровня запросов. По умолчанию это происходит, только если запрос `WITH` задействуется в родительском запросе всего в одном месте, а не в нескольких. Это поведение можно переопределить, добавив указание `MATERIALIZED`, чтобы выделить вычисление запроса `WITH`, или указание `NOT MATERIALIZED`, чтобы принудительно свернуть его в родительский запрос. В последнем случае возникает риск многократного вычисления запроса `WITH`, но в итоге это может быть выгодно, если в каждом случае использования `WITH` из всего результата запроса остаётся только небольшая часть.

Простой пример для демонстрации этих правил:

```
WITH w AS (
    SELECT * FROM big_table
)
SELECT * FROM w WHERE key = 123;
```

Этот запрос `WITH` будет свёрнут в родительский и будет выполнен с тем же планом, что и запрос:

```
SELECT * FROM big_table WHERE key = 123;
```

В частности, если в таблице создан индекс по столбцу `key`, он может использоваться для получения строк с `key = 123`. В другом случае:

```
WITH w AS (
    SELECT * FROM big_table
)
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref
WHERE w2.key = 123;
```

запрос `WITH` будет материализован, то есть создастся временная копия таблицы `big_table`, которая будет соединена с собой же, без использования какого-либо индекса. Этот запрос будет выполняться гораздо эффективнее в таком виде:

```
WITH w AS NOT MATERIALIZED (
    SELECT * FROM big_table
)
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref
WHERE w2.key = 123;
```

В этом случае ограничения родительского запроса могут применяться непосредственно при сканировании `big_table`.

Пример, в котором вариант `NOT MATERIALIZED` может быть нежелательным:

```
WITH w AS (
    SELECT key, very_expensive_function(val) as f FROM some_table
)
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.f = w2.f;
```

В данном случае благодаря материализации запроса `WITH` ресурсоёмкая функция `very_expensive_function` вычисляется только один раз для строки таблицы, а не дважды.

Примеры выше показывают только предложение `WITH` с `SELECT`, но таким же образом его можно использовать с командами `INSERT`, `UPDATE`, `DELETE` и `MERGE`. В каждом случае он по сути создаёт временную таблицу, к которой можно обратиться в основной команде.

7.8.4. Изменение данных в `WITH`

В предложении `WITH` можно также использовать большинство операторов, изменяющих данные (`INSERT`, `UPDATE` или `DELETE`, но не `MERGE`). Это позволяет выполнять в одном запросе сразу несколько разных операций. Например:

```
WITH moved_rows AS (  
    DELETE FROM products  
    WHERE  
        "date" >= '2010-10-01' AND  
        "date" < '2010-11-01'  
    RETURNING *  
)  
INSERT INTO products_log  
SELECT * FROM moved_rows;
```

Этот запрос фактически перемещает строки из `products` в `products_log`. Оператор `DELETE` в `WITH` удаляет указанные строки из `products` и возвращает их содержимое в предложении `RETURNING`; а затем главный запрос читает это содержимое и вставляет в таблицу `products_log`.

Следует заметить, что предложение `WITH` в данном случае присоединяется к оператору `INSERT`, а не к `SELECT`, вложенному в `INSERT`. Это необходимо, так как `WITH` может содержать операторы, изменяющие данные, только на верхнем уровне запроса. Однако при этом применяются обычные правила видимости `WITH`, так что к результату `WITH` можно обратиться и из вложенного оператора `SELECT`.

Операторы, изменяющие данные, в `WITH` обычно дополняются предложением `RETURNING` (см. [Раздел 6.4](#)), как показано в этом примере. Важно понимать, что временная таблица, которую можно будет использовать в остальном запросе, создаётся из результата `RETURNING`, а не целевой таблицы оператора. Если оператор, изменяющий данные, в `WITH` не дополнен предложением `RETURNING`, временная таблица не создаётся и обращаться к ней в остальном запросе нельзя. Однако такой запрос всё равно будет выполнен. Например, допустим следующий не очень практичный запрос:

```
WITH t AS (  
    DELETE FROM foo  
)  
DELETE FROM bar;
```

Он удалит все строки из таблиц `foo` и `bar`. При этом число задействованных строк, которое получит клиент, будет подсчитываться только по строкам, удалённым из `bar`.

Рекурсивные ссылки в операторах, изменяющих данные, не допускаются. В некоторых случаях это ограничение можно обойти, обратившись к конечному результату рекурсивного `WITH`, например так:

```
WITH RECURSIVE included_parts(sub_part, part) AS (  
    SELECT sub_part, part FROM parts WHERE part = 'our_product'  
    UNION ALL  
    SELECT p.sub_part, p.part  
    FROM included_parts pr, parts p  
    WHERE p.part = pr.sub_part  
)  
DELETE FROM parts  
WHERE part IN (SELECT part FROM included_parts);
```

Этот запрос удаляет все непосредственные и косвенные составные части продукта.

Операторы, изменяющие данные в `WITH`, выполняются только один раз и всегда полностью, вне зависимости от того, принимает ли их результат основной запрос. Заметьте, что это отличается от поведения `SELECT` в `WITH`: как говорилось в предыдущем разделе, `SELECT` выполняется только до тех пор, пока его результаты востребованы основным запросом.

Вложенные операторы в `WITH` выполняются одновременно друг с другом и с основным запросом. Таким образом, порядок, в котором операторы в `WITH` будут фактически изменять данные, непредсказуем. Все эти операторы выполняются с одним *снимком данных* (см. [Главу 13](#)), так что они не могут «видеть», как каждый из них меняет целевые таблицы. Это уменьшает эффект непредсказуемости фактического порядка изменения строк и означает, что `RETURNING` — единственный вариант передачи изменений от вложенных операторов `WITH` основному запросу. Например, в данном случае:

```
WITH t AS (  
    UPDATE products SET price = price * 1.05  
    RETURNING *  
)  
SELECT * FROM products;
```

внешний оператор `SELECT` выдаст цены, которые были до действия `UPDATE`, тогда как в запросе

```
WITH t AS (  
    UPDATE products SET price = price * 1.05  
    RETURNING *  
)  
SELECT * FROM t;
```

внешний `SELECT` выдаст изменённые данные.

Неоднократное изменение одной и той же строки в рамках одного оператора не поддерживается. Иметь место будет только одно из нескольких изменений и надёжно определить, какое именно, часто довольно сложно (а иногда и вовсе невозможно). Это так же касается случая, когда строка удаляется и изменяется в том же операторе: в результате может быть выполнено только обновление. Поэтому в общем случае следует избегать подобного наложения операций. В частности, избегайте подзапросов `WITH`, которые могут повлиять на строки, изменяемые основным оператором или операторами, вложенные в него. Результат действия таких запросов будет непредсказуемым.

В настоящее время, для оператора, изменяющего данные в `WITH`, в качестве целевой нельзя использовать таблицу, для которой определено условное правило или правило `ALSO` или `INSTEAD`, если оно состоит из нескольких операторов.

Глава 8. Типы данных

Postgres Pro предоставляет пользователям богатый ассортимент встроенных типов данных. Кроме того, пользователи могут создавать свои типы в Postgres Pro, используя команду [CREATE TYPE](#).

[Таблица 8.1](#) содержит все встроенные типы данных общего пользования. Многие из альтернативных имён, приведённых в столбце «Псевдонимы», используются внутри Postgres Pro по историческим причинам. В этот список не включены некоторые устаревшие типы и типы для внутреннего применения.

Таблица 8.1. Типы данных

Имя	Псевдонимы	Описание
bigint	int8	знаковое целое из 8 байт
bigserial	serial8	восьмибайтное целое с автоувеличением
bit [(n)]		битовая строка фиксированной длины
bit varying [(n)]	varbit [(n)]	битовая строка переменной длины
boolean	bool	логическое значение (true/false)
box		прямоугольник в плоскости
bytea		двоичные данные («массив байт»)
character [(n)]	char [(n)]	символьная строка фиксированной длины
character varying [(n)]	varchar [(n)]	символьная строка переменной длины
cidr		сетевой адрес IPv4 или IPv6
circle		круг в плоскости
date		календарная дата (год, месяц, день)
double precision	float, float8	число двойной точности с плавающей точкой (8 байт)
inet		адрес узла IPv4 или IPv6
integer	int, int4	знаковое четырёхбайтное целое
interval [поля] [(p)]		интервал времени
json		текстовые данные JSON
jsonb		двоичные данные JSON, разобранные
line		прямая в плоскости
lseg		отрезок в плоскости
macaddr		MAC-адрес
macaddr8		адрес MAC (Media Access Control) (в формате EUI-64)
money		денежная сумма
numeric [(p, s)]	decimal [(p, s)]	вещественное число заданной точности
path		геометрический путь в плоскости
pg_lsn		последовательный номер в журнале Postgres Pro
pg_snapshot		снимок идентификатора транзакций

Имя	Псевдонимы	Описание
point		геометрическая точка в плоскости
polygon		замкнутый геометрический путь в плоскости
real	float4	число одинарной точности с плавающей точкой (4 байта)
smallint	int2	знаковое двухбайтное целое
smallserial	serial2	двухбайтное целое с автоувеличением
serial	serial4	четырёхбайтное целое с автоувеличением
text		символьная строка переменной длины
time [(p)] [without time zone]		время суток (без часового пояса)
time [(p)] with time zone	timetz	время суток с учётом часового пояса
timestamp [(p)] [without time zone]		дата и время (без часового пояса)
timestamp [(p)] with time zone	timestampz	дата и время с учётом часового пояса
tsquery		запрос текстового поиска
tsvector		документ для текстового поиска
txid_snapshot		снимок идентификаторов транзакций для пользовательского уровня (устаревший тип; см. <code>pg_snapshot</code>)
uuid		универсальный уникальный идентификатор
xml		XML-данные

Совместимость

В стандарте SQL описаны следующие типы (или их имена): `bigint`, `bit`, `bit varying`, `boolean`, `char`, `character varying`, `character`, `varchar`, `date`, `double precision`, `integer`, `interval`, `numeric`, `decimal`, `real`, `smallint`, `time` (с часовым поясом и без), `timestamp` (с часовым поясом и без), `xml`.

Каждый тип данных имеет внутреннее представление, скрытое функциями ввода и вывода. При этом многие встроенные типы стандартны и имеют очевидные внешние форматы. Однако есть типы, уникальные для Postgres Pro, например геометрические пути, и есть типы, которые могут иметь разные форматы, например, дата и время. Некоторые функции ввода и вывода не являются в точности обратными друг к другу, то есть результат функции вывода может не совпадать со входным значением из-за потери точности.

8.1. Числовые типы

Числовые типы включают двух-, четырёх- и восьмибайтные целые, четырёх- и восьмибайтные числа с плавающей точкой, а также десятичные числа с задаваемой точностью. Все эти типы перечислены в [Таблице 8.2](#).

Таблица 8.2. Числовые типы

Имя	Размер	Описание	Диапазон
<code>smallint</code>	2 байта	целое в небольшом диапазоне	-32768 .. +32767
<code>integer</code>	4 байта	типичный выбор для целых чисел	-2147483648 .. +2147483647
<code>bigint</code>	8 байт	целое в большом диапазоне	-9223372036854775808 .. 9223372036854775807
<code>decimal</code>	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
<code>numeric</code>	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
<code>real</code>	4 байта	вещественное число с переменной точностью	точность в пределах 6 десятичных цифр
<code>double precision</code>	8 байт	вещественное число с переменной точностью	точность в пределах 15 десятичных цифр
<code>smallserial</code>	2 байта	небольшое целое с автоувеличением	1 .. 32767
<code>serial</code>	4 байта	целое с автоувеличением	1 .. 2147483647
<code>bigserial</code>	8 байт	большое целое с автоувеличением	1 .. 9223372036854775807

Синтаксис констант числовых типов описан в [Подразделе 4.1.2](#). Для этих типов определён полный набор соответствующих арифметических операторов и функций. За дополнительными сведениями обратитесь к [Главе 9](#). Подробнее эти типы описаны в следующих разделах.

8.1.1. Целочисленные типы

Типы `smallint`, `integer` и `bigint` хранят целые числа, то есть числа без дробной части, имеющие разные допустимые диапазоны. Попытка сохранить значение, выходящее за рамки диапазона, приведёт к ошибке.

Чаще всего используется тип `integer`, как наиболее сбалансированный выбор ширины диапазона, размера и быстродействия. Тип `smallint` обычно применяется, только когда крайне важно уменьшить размер данных на диске. Тип `bigint` предназначен для тех случаев, когда числа не уместятся в диапазон типа `integer`.

В SQL определены только типы `integer` (или `int`), `smallint` и `bigint`. Имена типов `int2`, `int4` и `int8` выходят за рамки стандарта, хотя могут работать и в некоторых других СУБД.

8.1.2. Числа с произвольной точностью

Тип `numeric` позволяет хранить числа с очень большим количеством цифр. Он особенно рекомендуется для хранения денежных сумм и других величин, где важна точность. Вычисления с типом `numeric` дают точные результаты, где это возможно, например, при сложении, вычитании и умножении. Однако операции со значениями `numeric` выполняются гораздо медленнее, чем с целыми числами или с типами с плавающей точкой, описанными в следующем разделе.

Ниже мы используем следующие термины: *масштаб* значения `numeric` определяет количество десятичных цифр в дробной части, справа от десятичной точки, а *точность* — общее количество значимых цифр в числе, т. е. количество цифр по обе стороны десятичной точки. Например, число

23.5141 имеет точность 6 и масштаб 4. Целочисленные значения можно считать числами с масштабом 0.

Для столбца типа `numeric` можно настроить и максимальную точность, и максимальный масштаб. Столбец типа `numeric` объявляется следующим образом:

```
NUMERIC (точность, масштаб)
```

Точность должна быть положительной, а масштаб может быть положительным или отрицательным (см. ниже). Альтернативный вариант

```
NUMERIC (точность)
```

устанавливает масштаб 0. Форма:

```
NUMERIC
```

без указания точности и масштаба создаёт столбец типа «неограниченное число», в котором можно сохранять числовые значения любой длины до предела, обусловленного реализацией. В столбце этого типа входные значения не будут приводиться к какому-либо масштабу, тогда как в столбцах `numeric` с явно заданным масштабом значения подгоняются под этот масштаб. (Стандарт SQL утверждает, что по умолчанию должен устанавливаться масштаб 0, т. е. значения должны приводиться к целым числам. Однако мы считаем это не очень полезным. Если для вас важна переносимость, всегда указывайте точность и масштаб явно.)

Примечание

Максимальная точность, которую можно указать в объявлении типа `numeric`, равна 1000. Для столбцов типа «неограниченный `numeric`» существуют пределы, описанные в [Таблице 8.2](#).

Если масштаб значения, которое нужно сохранить, превышает объявленный масштаб столбца, система округлит его до заданного количества цифр после точки. Если же после этого количество цифр слева в сумме с масштабом превысит объявленную точность, произойдёт ошибка. Например, столбец, объявленный как

```
NUMERIC (3, 1)
```

, будет содержать значения, округлённые до одного десятичного знака, от -99.9 до 99.9 включительно.

Начиная с PostgreSQL 15, столбец типа `numeric` можно объявить с отрицательным масштабом. В этом случае значения будут округляться до позиции слева от десятичной точки. При этом точность так же задаёт максимальное число цифр, без учёта округления. Таким образом, в столбце, объявленном как

```
NUMERIC (2, -3)
```

, число будет округляться до ближайшей тысячи, и в нём можно будет хранить значения от -99000 до 99000 включительно. Также вполне возможно задать масштаб, превышающий объявленную точность. Такой столбец сможет содержать только дробные значения, в которых количество нулей сразу после точки должно равняться разности масштаба и точности. Например, столбец, объявленный как

```
NUMERIC (3, 5)
```

, будет округлять значения до пяти знаков после точки и сможет хранить значения от -0.00999 до 0.00999 включительно.

Примечание

PostgreSQL позволяет использовать в объявлении типа `numeric` любое значение масштаба в диапазоне от -1000 до 1000. Однако стандарт SQL требует, чтобы масштаб находился в диа-

пазоне от 0 до значения *точности*. Масштабы вне этого диапазона могут не поддерживаться другими системами баз данных.

Числовые значения физически хранятся без каких-либо дополняющих нулей слева или справа. Таким образом, объявляемые точность и масштаб столбца определяют максимальный, а не фиксированный размер хранения. (В этом смысле тип `numeric` больше похож на тип `varchar(n)`, чем на `char(n)`.) Действительный размер хранения такого значения складывается из двух байт для каждой группы из четырёх цифр и дополнительных трёх-восьми байт.

В дополнение к обычным числовым значениям тип `numeric` принимает следующие специальные значения:

Infinity
-Infinity
NaN

Они представляют особые значения, описанные в IEEE 754, соответственно «бесконечность», «минус бесконечность» и «не число». Записывая эти значения в виде констант в команде SQL, их нужно заключать в апострофы, например так: `UPDATE table SET x = '-Infinity'`. Регистр символов в этих строках не важен. В качестве альтернативы значения бесконечности могут быть записаны как `inf` и `-inf`.

Значения бесконечности соответствуют ожиданиям с точки зрения математики. Например, `Infinity` плюс любое конечное значение равно `Infinity`, как и `Infinity` плюс `Infinity`; но `Infinity` минус `Infinity` даёт `NaN` (не число), потому что в результате получается неопределённость. Обратите внимание, что бесконечность может быть сохранена только в столбце типа «неограниченный `numeric`», потому что она теоретически превышает любой конечный предел точности.

Значение `NaN` (не число) используется для представления неопределённых результатов вычислений. Вообще, любая операция с `NaN` выдаёт в результате тоже `NaN`. Единственное исключение составляют операции, которые дают один и тот же результат для любого числового значения, конечного или бесконечного — этот результат будет выдаваться и для `NaN`. (Пример этого принципа: результатом возведения `NaN` в нулевую степень будет единица.)

Примечание

В большинстве реализаций «не число» (`NaN`) считается не равным любому другому значению (в том числе и самому `NaN`). Чтобы значения `numeric` можно было сортировать и использовать в древовидных индексах, Postgres Pro считает, что значения `NaN` равны друг другу и при этом больше любых числовых значений (не `NaN`).

Типы `decimal` и `numeric` равнозначны. Оба эти типа описаны в стандарте SQL.

При округлении значений тип `numeric` выдаёт число, большее по модулю, тогда как (на большинстве платформ) типы `real` и `double precision` выдают ближайшее чётное число. Например:

```
SELECT x,
       round(x::numeric) AS num_round,
       round(x::double precision) AS dbl_round
FROM generate_series(-3.5, 3.5, 1) as x;
 x  | num_round | dbl_round
-----+-----+-----
-3.5 |         -4 |        -4
-2.5 |         -3 |        -2
-1.5 |         -2 |        -2
-0.5 |         -1 |         0
```

0.5	1	0
1.5	2	2
2.5	3	2
3.5	4	4

(8 rows)

8.1.3. Типы с плавающей точкой

Типы данных `real` и `double precision` хранят приближённые числовые значения с переменной точностью. На всех поддерживаемых в настоящее время платформах эти типы реализуют стандарт IEEE 754 для двоичной арифметики с плавающей точкой (с одинарной и двойной точностью соответственно), в той мере, в какой его поддерживают процессор, операционная система и компилятор.

Неточность здесь выражается в том, что некоторые значения, которые нельзя преобразовать во внутренний формат, сохраняются приближённо, так что полученное значение может несколько отличаться от записанного. Управление подобными ошибками и их распространение в процессе вычислений является предметом изучения целого раздела математики и компьютерной науки, и здесь не рассматривается. Мы отметим только следующее:

- Если вам нужна точность при хранении и вычислениях (например, для денежных сумм), используйте вместо этого тип `numeric`.
- Если вы хотите выполнять с этими типами сложные вычисления, имеющие большую важность, тщательно изучите реализацию операций в вашей среде и особенно поведение в крайних случаях (бесконечность, антипереполнение).
- Проверка равенства двух чисел с плавающей точкой может не всегда давать ожидаемый результат.

На всех поддерживаемых сейчас платформах тип `real` может сохранить значения примерно от $1\text{E}-37$ до $1\text{E}+37$ с точностью не меньше 6 десятичных цифр. Тип `double precision` предлагает значения в диапазоне приблизительно от $1\text{E}-307$ до $1\text{E}+308$ и с точностью не меньше 15 цифр. Попытка сохранить слишком большие или слишком маленькие значения приведёт к ошибке. Если точность вводимого числа слишком велика, оно будет округлено. При попытке сохранить число, близкое к 0, но непредставимое как отличное от 0, произойдёт ошибка антипереполнения.

По умолчанию числа с плавающей точкой выводятся в текстовом виде в кратчайшем точном десятичном представлении; выводимое десятичное значение оказывается более близким к изначальному двоичному числу, чем любое другое значение, представимое с той же двоичной точностью. (Однако выводимое значение в текущей реализации никогда не находится *точно* посередине между двумя представимыми двоичными значениями, во избежание распространённой ошибки с функциями ввода, не учитывающими корректно правило округления до ближайшего чётного.) Выводимое значение может занимать не больше 17 значащих десятичных цифр для типа `float8` и не больше 9 цифр для типа `float4`.

Примечание

Преобразование в кратчайший точный вид производится гораздо быстрее, чем в традиционное представление с округлением.

Для совместимости с результатами, выдаваемыми старыми версиями Postgres Pro, и уменьшения точности выводимых чисел, когда это требуется, в параметре `extra_float_digits` можно выбрать также вариант округлённого десятичного вывода. Со значением 0 восстанавливается действовавшее ранее по умолчанию округление числа до 6 (для типа `float4`) или 15 (для `float8`) значащих десятичных цифр. При отрицательных значениях число значащих цифр уменьшается дополнительно; например, при -2 результат будет округлён до 4 или 13 цифр, соответственно.

При любом значении `extra_float_digits`, большем 0, выбирается кратчайшее точное представление.

Примечание

Приложения, которым были нужны точные числовые значения, раньше задавали для параметра `extra_float_digits` значение 3, чтобы получить их. Они могут продолжать использовать это значение для максимальной совместимости с разными версиями.

В дополнение к обычным числовым значениям типы с плавающей точкой принимают следующие специальные значения:

Infinity
-Infinity
NaN

Они представляют особые значения, описанные в IEEE 754, соответственно «бесконечность», «минус бесконечность» и «не число». Записывая эти значения в виде констант в команде SQL, их нужно заключать в апострофы, например так: `UPDATE table SET x = '-Infinity'`. Регистр символов в этих строках не важен. В качестве альтернативы значения бесконечности могут быть записаны как `inf` и `-inf`.

Примечание

Согласно IEEE 754, NaN не должно считаться равным любому другому значению с плавающей точкой (в том числе и самому NaN). Чтобы значения с плавающей точкой можно было сортировать и использовать в древовидных индексах, Postgres Pro считает, что значения NaN равны друг другу, и при этом больше любых числовых значений (не NaN).

Postgres Pro также поддерживает форматы `float` и `float(p)`, оговорённые в стандарте SQL, для указания неточных числовых типов. Здесь `p` определяет минимально допустимую точность в двоичных цифрах. Postgres Pro воспринимает запись от `float(1)` до `float(24)` как выбор типа `real`, а запись от `float(25)` до `float(53)` как выбор типа `double precision`. Значения `p` вне допустимого диапазона вызывают ошибку. Если `float` указывается без точности, подразумевается тип `double precision`.

8.1.4. Последовательные типы

Примечание

В этом разделе описывается специфичный для Postgres Pro способ создания столбца с автоувеличением. Другой способ, соответствующий стандарту SQL, заключается в использовании столбцов идентификации и рассматривается в описании [CREATE TABLE](#).

Типы данных `smallserial`, `serial` и `bigserial` не являются настоящими типами, а представляют собой просто удобное средство для создания столбцов с уникальными идентификаторами (подобное свойству `AUTO_INCREMENT` в некоторых СУБД). В текущей реализации запись:

```
CREATE TABLE имя_таблицы (
    имя_столбца SERIAL
);
```

равнозначна следующим командам:

```
CREATE SEQUENCE имя_таблицы_имя_столбца_seq AS integer;
CREATE TABLE имя_таблицы (
    имя_столбца integer NOT NULL DEFAULT nextval('имя_таблицы_имя_столбца_seq')
);
ALTER SEQUENCE имя_таблицы_имя_столбца_seq OWNED BY имя_таблицы.имя_столбца;
```

То есть при определении такого типа создаётся целочисленный столбец со значением по умолчанию, извлекаемым из генератора последовательности. Чтобы в столбец нельзя было вставить NULL, в его определение добавляется ограничение NOT NULL. (Во многих случаях также имеет смысл добавить для этого столбца ограничения UNIQUE или PRIMARY KEY для защиты от ошибочного добавления дублирующихся значений, но автоматически это не происходит.) Последняя команда определяет, что последовательность «принадлежит» столбцу, так что она будет удалена при удалении столбца или таблицы.

Примечание

Так как типы `smallserial`, `serial` и `bigserial` реализованы через последовательности, в числовом ряду значений столбца могут образовываться пропуски (или «дыры»), даже если никакие строки не удалялись. Значение, выделенное из последовательности, считается «задействованным», даже если строку с этим значением не удалось вставить в таблицу. Это может произойти, например, при откате транзакции, добавляющей данные. См. описание `nextval()` в [Разделе 9.17](#).

Чтобы вставить в столбец `serial` следующее значение последовательности, ему нужно присвоить значение по умолчанию. Это можно сделать, либо исключив его из списка столбцов в операторе INSERT, либо с помощью ключевого слова DEFAULT.

Имена типов `serial` и `serial4` равнозначны: они создают столбцы `integer`. Так же являются синонимами имена `bigserial` и `serial8`, но они создают столбцы `bigint`. Тип `bigserial` следует использовать, если за всё время жизни таблицы планируется использовать больше чем 2^{31} значений. И наконец, синонимами являются имена типов `smallserial` и `serial2`, но они создают столбец `smallint`.

Последовательность, созданная для столбца `serial`, автоматически удаляется при удалении связанного столбца. Последовательность можно удалить и отдельно от столбца, но при этом также будет удалено определение значения по умолчанию.

8.2. Денежные типы

Тип `money` хранит денежную сумму с фиксированной дробной частью; см. [Таблицу 8.3](#). Точность дробной части определяется на уровне базы данных параметром `lc_monetary`. Для диапазона, показанного в таблице, предполагается, что число содержит два знака после запятой. Входные данные могут быть записаны по-разному, в том числе в виде целых и дробных чисел, а также в виде строки в денежном формате, например `'$1,000.00'`. Выводятся эти значения обычно в денежном формате, зависящем от региональных стандартов.

Таблица 8.3. Денежные типы

Имя	Размер	Описание	Диапазон
<code>money</code>	8 байт	денежная сумма	-92233720368547758.08 .. +92233720368547758.07

Так как выводимые значения этого типа зависят от региональных стандартов, попытка загрузить данные типа `money` в базу данных с другим параметром `lc_monetary` может быть неудачной. Во избежание подобных проблем, прежде чем восстанавливать копию в новую базу данных, убедитесь в том, что параметр `lc_monetary` в этой базе данных имеет то же значение, что и в исходной.

Значения типов `numeric`, `int` и `bigint` можно привести к типу `money`. Преобразования типов `real` и `double precision` так же возможны через тип `numeric`, например:

```
SELECT '12.34'::float8::numeric::money;
```

Однако использовать числа с плавающей точкой для денежных сумм не рекомендуется из-за возможных ошибок округления.

Значение `money` можно привести к типу `numeric` без потери точности. Преобразование в другие типы может быть неточным и также должно выполняться в два этапа:

```
SELECT '52093.89'::money::numeric::float8;
```

При делении значения типа `money` на целое число выполняется отбрасывание дробной части и получается целое, ближайшее к нулю. Чтобы получить результат с округлением, выполните деление значения с плавающей точкой или приведите значение типа `money` к `numeric` до деления, а затем приведите результат к типу `money`. (Последний вариант предпочтительнее, так как исключает риск потери точности.) Когда значение `money` делится на другое значение `money`, результатом будет значение типа `double precision` (то есть просто число, не денежная величина); денежные единицы измерения при делении сокращаются.

8.3. Символьные типы

Таблица 8.4. Символьные типы

Имя	Описание
<code>character varying(n), varchar(n)</code>	строка ограниченной переменной длины
<code>character(n), char(n)</code>	строка фиксированной длины, дополненная пробелами
<code>text</code>	строка неограниченной переменной длины

В Таблице 8.4 перечислены символьные типы общего назначения, доступные в Postgres Pro.

SQL определяет два основных символьных типа: `character varying( n )` и `character( n )`, где `n` — положительное число. Оба эти типа могут хранить текстовые строки длиной до `n` символов (не байт). Попытка сохранить в столбце такого типа более длинную строку приведёт к ошибке, если только все лишние символы не являются пробелами (тогда они будут усечены до максимально допустимой длины). (Это несколько странное исключение продиктовано стандартом SQL.) Если длина сохраняемой строки оказывается меньше объявленной, значения типа `character` будут дополняться пробелами; а тип `character varying` просто сохранит короткую строку.

При попытке явно привести значение к типу `character varying( n )` или `character( n )`, часть строки, выходящая за границу в `n` символов, удаляется, не вызывая ошибки. (Это также продиктовано стандартом SQL.)

Записи `varchar( n )` и `char( n )` являются синонимами `character varying( n )` и `character( n )` соответственно. Если `n` задано, оно должно быть больше нуля и меньше или равно 10485760. Записи `character` без указания длины соответствует `character(1)`. Если же длина не указывается для `character varying`, этот тип будет принимать строки любого размера. Это поведение является расширением Postgres Pro.

Помимо этого, Postgres Pro предлагает тип `text`, в котором можно хранить строки произвольной длины. Хотя тип `text` не описан в стандарте SQL, его поддерживают и некоторые другие СУБД SQL.

Значения типа `character` физически дополняются пробелами до `n` символов и хранятся, а затем отображаются в таком виде. Однако при сравнении двух значений типа `character` дополняющие пробелы считаются незначущими и игнорируются. С правилами сортировки, где пробельные символы являются значущими, это поведение может приводить к неожиданным результатам, например `SELECT 'a '::CHAR(2) collate "C" < E'a\n'::CHAR(2)` вернёт `true` (условие будет истинным), хотя в локали `C` символ пробела считается больше символа новой строки. При приведении значения `character` к другому символьному типу дополняющие пробелы отбрасываются. Заметьте, что эти пробелы *несут* смысловую нагрузку в типах `character varying` и `text` и в проверках по шаблонам, то есть в `LIKE` и регулярных выражениях.

Какие именно символы можно сохранить в этих типах данных, зависит от того, какой набор символов был выбран при создании базы данных. Однако символ с кодом 0 (иногда называемый `NUL`)

сохранить нельзя, вне зависимости от выбранного набора символов. За подробностями обратитесь к [Разделу 22.3](#).

Для хранения короткой строки (до 126 байт) требуется дополнительный 1 байт плюс размер самой строки, включая дополняющие пробелы для типа `character`. Для строк длиннее требуется не 1, а 4 дополнительных байта. Система может автоматически сжимать длинные строки, так что физический размер на диске может быть меньше. Очень длинные текстовые строки переносятся в отдельные таблицы, чтобы они не замедляли работу с другими столбцами. В любом случае максимально возможный размер строки составляет около 1 ГБ. (Допустимое значение n в объявлении типа данных меньше этого числа. Это объясняется тем, что в зависимости от кодировки каждый символ может занимать несколько байт. Если вы желаете сохранять строки без определённого предела длины, используйте типы `text` или `character varying` без указания длины, а не задавайте какое-либо большое максимальное значение.)

Подсказка

По быстродействию эти три типа практически не отличаются друг от друга, не считая большего размера хранения для типа с дополняющими пробелами и нескольких машинных операций для проверки длины при сохранении строк в столбце с ограниченной длиной. Хотя в некоторых СУБД тип `character(n)` работает быстрее других, в Postgres Pro это не так; на деле `character(n)` обычно оказывается медленнее остальных типов из-за большего размера данных и более медленной сортировки. В большинстве случаев вместо него лучше применять `text` или `character varying`.

За информацией о синтаксисе строковых констант обратитесь к [Подразделу 4.1.2.1](#), а об имеющихся операторах и функциях вы можете узнать в [Главе 9](#).

Пример 8.1. Использование символьных типов

```
CREATE TABLE test1 (a character(4));
INSERT INTO test1 VALUES ('ok');
SELECT a, char_length(a) FROM test1; -- 1
```

a	char_length
ok	2

```
CREATE TABLE test2 (b varchar(5));
INSERT INTO test2 VALUES ('ok');
INSERT INTO test2 VALUES ('good');
INSERT INTO test2 VALUES ('too long');
ОШИБКА: значение не укладывается в тип character varying(5)
INSERT INTO test2 VALUES ('too long'::varchar(5)); -- явное усеечение
SELECT b, char_length(b) FROM test2;
```

b	char_length
ok	2
good	5
too l	5

1 Функция `char_length` рассматривается в [Разделе 9.4](#).

В Postgres Pro есть ещё два символьных типа фиксированной длины, приведённые в [Таблице 8.5](#). Они предназначены не для широкого применения, а для использования только во внутренних системных каталогах. Тип `name` создан для хранения идентификаторов. В настоящее время его длина составляет 64 байта (63 ASCII-символа плюс завершающий ноль), но в исходном коде с она задаётся константой `NAMEDATALEN`. Эта константа определяется во время компиляции (и её можно ме-

нять в особых случаях), а кроме того, максимальная длина по умолчанию может быть увеличена в следующих версиях. Тип "char" (обратите внимание на кавычки) отличается от char(1) тем, что он занимает только один байт хранилища и поэтому может хранить только один ASCII-символ. Он используется в системных каталогах для простых перечислений.

Таблица 8.5. Специальные символьные типы

Имя	Размер	Описание
"char"	1 байт	внутренний однобайтный тип
name	64 байта	внутренний тип для имён объектов

8.4. Двоичные типы данных

Для хранения двоичных данных предназначен тип bytea; см. [Таблицу 8.6](#).

Таблица 8.6. Двоичные типы данных

Имя	Размер	Описание
bytea	1 или 4 байта плюс сама двоичная строка	двоичная строка переменной длины

Двоичные строки представляют собой последовательность октетов (байт) и имеют два отличия от текстовых строк. Во-первых, в двоичных строках можно хранить байты с кодом 0 и другими «непечатаемыми» значениями (обычно это значения вне десятичного диапазона 32..126). В текстовых строках нельзя сохранять нулевые байты, а также значения и последовательности значений, не соответствующие выбранной кодировке базы данных. Во-вторых, в операциях с двоичными строками обрабатываются байты в чистом виде, тогда как текстовые строки обрабатываются в зависимости от языковых стандартов. То есть, двоичные строки больше подходят для данных, которые программист видит как «просто байты», а символьные строки — для хранения текста.

Тип bytea поддерживает два формата ввода и вывода: «шестнадцатеричный» и традиционный для PostgreSQL формат «спецпоследовательностей». Входные данные принимаются в обоих форматах, а формат выходных данных зависит от параметра конфигурации `bytea_output`; по умолчанию выбран шестнадцатеричный. (Заметьте, что шестнадцатеричный формат был введён в PostgreSQL 9.0; в ранних версиях и некоторых программах он не будет работать.)

Стандарт SQL определяет другой тип двоичных данных, BLOB (BINARY LARGE OBJECT, большой двоичный объект). Его входной формат отличается от форматов bytea, но функции и операторы в основном те же.

8.4.1. Шестнадцатеричный формат bytea

В «шестнадцатеричном» формате двоичные данные кодируются двумя шестнадцатеричными цифрами на байт, при этом первая цифра соответствует старшим 4 битам. К полученной строке добавляется префикс \x (чтобы она отличалась от формата спецпоследовательности). В некоторых контекстах обратную косую черту нужно экранировать, продублировав её (см. [Подраздел 4.1.2.1](#)). Вводимые шестнадцатеричные цифры могут быть в любом регистре, а между парами цифр допускаются пробельные символы (но не внутри пары и не в начале последовательности \x). Этот формат совместим со множеством внешних приложений и протоколов, к тому же обычно преобразуется быстрее, поэтому предпочтительнее использовать его.

Пример:

```
SET bytea_output = 'hex';

SELECT '\xDEADBEEF'::bytea;
   bytea
-----
 \xdeadbeef
```

8.4.2. Формат спецпоследовательностей bytea

Формат «спецпоследовательностей» традиционно использовался в Postgres Pro для значений типа `bytea`. В нём двоичная строка представляется в виде последовательности ASCII-символов, а байты, непредставимые в виде ASCII-символов, передаются в виде спецпоследовательностей. Этот формат может быть удобен, если с точки зрения приложения представление байт в виде символов имеет смысл. Но на практике это обычно создаёт путаницу, так как двоичные и символьные строки могут выглядеть одинаково, а кроме того выбранный механизм спецпоследовательностей довольно неуклюж. Поэтому в новых приложениях этот формат обычно не стоит использовать.

Передавая значения `bytea` в формате спецпоследовательности, байты с определёнными значениями *необходимо* записывать специальным образом, хотя так *можно* записывать и все значения. В общем виде для этого значение байта нужно преобразовать в трёхзначное восьмеричное число и добавить перед ним обратную косую черту. Саму обратную косую черту (символ с десятичным кодом 92) можно записать в виде двух таких символов. В [Таблице 8.7](#) перечислены символы, которые нужно записывать спецпоследовательностями, и приведены альтернативные варианты записи, если они возможны.

Таблица 8.7. Спецпоследовательности записи значений `bytea`

Десятичное значение байта	Описание	Спецпоследовательность ввода	Пример	Шестнадцатеричное представление
0	нулевой байт	'\000'	'\000'::bytea	\x00
39	апостроф	'\'' или '\047'	'\''::bytea	\x27
92	обратная косая черта	'\\' или '\134'	'\\'::bytea	\x5c
от 0 до 31 и от 127 до 255	«непечатаемые» байты	E'\xxx' (восьмеричное значение)	'\001'::bytea	\x01

Требования экранирования *непечатаемых* символов определяются языковыми стандартами. Иногда такие символы могут восприниматься и без спецпоследовательностей.

Апострофы должны дублироваться, как показано в [Таблице 8.7](#), потому что это обязательно для любой текстовой строки в команде SQL. При общем разборе текстовой строки внешние апострофы убираются, а каждая пара внутренних сводится к одному символу. Таким образом, функция ввода `bytea` видит всего один апостроф, который она обрабатывает как обычный символ в данных. Дублировать же обратную косую черту при вводе `bytea` не требуется: этот символ считается особым и меняет поведение функции ввода, как показано в [Таблице 8.7](#).

В некоторых контекстах обратная косая черта должна дублироваться (относительно примеров выше), так как при общем разборе строковых констант пара таких символов будет сведена к одному; см. [Подраздел 4.1.2.1](#).

Данные `Bytea` по умолчанию выводятся в шестнадцатеричном формате (`hex`). Если поменять значение `bytea_output` на `escape`, «непечатаемые» байты представляются в виде соответствующих трёхзначных восьмеричных значений, которые предваряются одной обратной косой чертой. Большинство «печатаемых» байтов представляются обычными символами из клиентского набора символов, например:

```
SET bytea_output = 'escape';

SELECT 'abc \153\154\155 \052\251\124'::bytea;
      bytea
-----
abc klm *\251T
```

Байт с десятичным кодом 92 (обратная косая черта) при выводе дублируется. Это иллюстрирует [Таблица 8.8](#).

Таблица 8.8. Спецпоследовательности выходных значений `bytea`

Десятичное значение байта	Описание	Спецпоследовательность вывода	Пример	Выводимый результат
92	обратная косая черта	\\	'\134'::bytea	\\
от 0 до 31 и от 127 до 255	«непечатаемые» байты	\xxx (значение байта)	'\001'::bytea	\001
от 32 до 126	«печатаемые» байты	представление из клиентского набора символов	'\176'::bytea	~

В зависимости от применяемой клиентской библиотеки Postgres Pro, для преобразования значений `bytea` в спецстроки и обратно могут потребоваться дополнительные действия. Например, если приложение сохраняет в строках символы перевода строк, возможно их также нужно будет представить спецпоследовательностями.

8.5. Типы даты/времени

Postgres Pro поддерживает полный набор типов даты и времени SQL, показанный в [Таблице 8.9](#). Операции, возможные с этими типами данных, описаны в [Разделе 9.9](#). Все даты считаются по Григорианскому календарю, даже для времени до его введения (за дополнительными сведениями обратитесь к [Разделу В.6](#)).

Таблица 8.9. Типы даты/времени

Имя	Размер	Описание	Наименьшее значение	Наибольшее значение	Точность
<code>timestamp [(p)] [without time zone]</code>	8 байт	дата и время (без часового пояса)	4713 до н. э.	294276 н. э.	1 микросекунда
<code>timestamp [(p)] with time zone</code>	8 байт	дата и время (с часовым поясом)	4713 до н. э.	294276 н. э.	1 микросекунда
<code>date</code>	4 байта	дата (без времени суток)	4713 до н. э.	5874897 н. э.	1 день
<code>time [(p)] [without time zone]</code>	8 байт	время суток (без даты)	00:00:00	24:00:00	1 микросекунда
<code>time [(p)] with time zone</code>	12 байт	время дня (без даты), с часовым поясом	00:00:00+1559	24:00:00-1559	1 микросекунда
<code>interval [поля] [(p)]</code>	16 байт	временной интервал	-178000000 лет	178000000 лет	1 микросекунда

Примечание

Стандарт SQL требует, чтобы тип `timestamp` подразумевал `timestamp without time zone` (время без часового пояса), и Postgres Pro следует этому. Для краткости `timestamp with time zone` можно записать как `timestamptz`; это расширение Postgres Pro.

Типы `time`, `timestamp` и `interval` принимают необязательное значение точности p , определяющее, сколько знаков после запятой должно сохраняться в секундах. По умолчанию точность не ограничивается. Допустимые значения p лежат в интервале от 0 до 6.

Тип `interval` дополнительно позволяет ограничить набор сохраняемых полей следующими фразами:

```
YEAR
MONTH
DAY
HOUR
MINUTE
SECOND
YEAR TO MONTH
DAY TO HOUR
DAY TO MINUTE
DAY TO SECOND
HOUR TO MINUTE
HOUR TO SECOND
MINUTE TO SECOND
```

Заметьте, что если указаны и поля, и точность p , указание поля должно включать `SECOND`, так как точность применима только к секундам.

Тип `time with time zone` определён стандартом SQL, но в его определении описаны свойства сомнительной ценности. В большинстве случаев сочетание типов `date`, `time`, `timestamp without time zone` и `timestamp with time zone` удовлетворяет все потребности в функциональности дат/времени, возникающие в приложениях.

8.5.1. Ввод даты/времени

Значения даты и времени принимаются практически в любом разумном формате, включая ISO 8601, SQL-совместимый, традиционный формат POSTGRES и другие. В некоторых форматах порядок даты, месяца и года во вводимой дате неоднозначен и поэтому поддерживается явное определение формата. Для этого предназначен параметр `DateStyle`. Когда он имеет значение `MDY`, выбирается интерпретация месяц-день-год, значению `DMY` соответствует день-месяц-год, а `YMD` — год-месяц-день.

Postgres Pro обрабатывает вводимые значения даты/времени более гибко, чем того требует стандарт SQL. Точные правила разбора даты/времени и распознаваемые текстовые поля, в том числе названия месяцев, дней недели и часовых поясов описаны в [Приложении В](#).

Помните, что любые вводимые значения даты и времени нужно заключать в апострофы, как текстовые строки. За дополнительной информацией обратитесь к [Подразделу 4.1.2.7](#). SQL предусматривает следующий синтаксис:

```
тип [ (p) ] 'значение'
```

Здесь p — необязательное указание точности, определяющее число знаков после точки в секундах. Точность может быть определена для типов `time`, `timestamp` и `interval` в интервале от 0 до 6. Если в определении константы точность не указана, она считается равной точности значения в строке (но не больше 6 цифр).

8.5.1.1. Даты

В [Таблице 8.10](#) приведены некоторые допустимые значения типа `date`.

Таблица 8.10. Вводимые даты

Пример	Описание
1999-01-08	ISO 8601; 8 января в любом режиме (рекомендуемый формат)

Пример	Описание
January 8, 1999	воспринимается однозначно в любом режиме <code>datestyle</code>
1/8/1999	8 января в режиме <code>MDY</code> и 1 августа в режиме <code>DMY</code>
1/18/1999	18 января в режиме <code>MDY</code> ; недопустимая дата в других режимах
01/02/03	2 января 2003 г. в режиме <code>MDY</code> ; 1 февраля 2003 г. в режиме <code>DMY</code> и 3 февраля 2001 г. в режиме <code>YMD</code>
1999-Jan-08	8 января в любом режиме
Jan-08-1999	8 января в любом режиме
08-Jan-1999	8 января в любом режиме
99-Jan-08	8 января в режиме <code>YMD</code> ; ошибка в других режимах
08-Jan-99	8 января; ошибка в режиме <code>YMD</code>
Jan-08-99	8 января; ошибка в режиме <code>YMD</code>
19990108	ISO 8601; 8 января 1999 в любом режиме
990108	ISO 8601; 8 января 1999 в любом режиме
1999.008	год и день года
J2451187	юлианский день
January 8, 99 BC	99 до н. э.

8.5.1.2. Время

Для хранения времени суток без даты предназначены типы `time [ (p) ] without time zone` и `time [ (p) ] with time zone`. Тип `time` без уточнения эквивалентен типу `time without time zone`.

Допустимые вводимые значения этих типов состоят из записи времени суток и необязательного указания часового пояса. (См. [Таблицу 8.11](#) и [Таблицу 8.12](#).) Если в значении для типа `time without time zone` указывается часовой пояс, он просто игнорируется. Так же будет игнорироваться дата, если её указать, за исключением случаев, когда в указанном часовом поясе принят переход на летнее время, например `America/New_York`. В данном случае указать дату необходимо, чтобы система могла определить, применяется ли обычное или летнее время. Соответствующее смещение часового пояса записывается в значении `time with time zone` и выводится без изменений, оно не привязано к активному часовому поясу.

Таблица 8.11. Вводимое время

Пример	Описание
04:05:06.789	ISO 8601
04:05:06	ISO 8601
04:05	ISO 8601
040506	ISO 8601
04:05 AM	то же, что и 04:05; AM не меняет значение времени
04:05 PM	то же, что и 16:05; часы должны быть ≤ 12
04:05:06.789-8	ISO 8601, с часовым поясом в виде смещения от UTC
04:05:06-08:00	ISO 8601, с часовым поясом в виде смещения от UTC
04:05-08:00	ISO 8601, с часовым поясом в виде смещения от UTC

Пример	Описание
040506-08	ISO 8601, с часовым поясом в виде смещения от UTC
040506+0730	ISO 8601, с часовым поясом, задаваемым нецелочисленным смещением от UTC
040506+07:30:00	смещение от UTC, заданное до секунд (не допускается в ISO 8601)
04:05:06 PST	часовой пояс задаётся аббревиатурой
2003-04-12 04:05:06 America/New_York	часовой пояс задаётся полным названием

Таблица 8.12. Вводимый часовой пояс

Пример	Описание
PST	аббревиатура (Pacific Standard Time, Стандартное тихоокеанское время)
America/New_York	полное название часового пояса
PST8PDT	указание часового пояса в стиле POSIX
-8:00:00	смещение часового пояса PST от UTC
-8:00	смещение часового пояса PST от UTC (расширенный формат ISO 8601)
-800	смещение часового пояса PST от UTC (стандартный формат ISO 8601)
-8	смещение часового пояса PST от UTC (стандартный формат ISO 8601)
zulu	принятое у военных сокращение UTC
z	краткая форма zulu (также определена в ISO 8601)

Подробнее узнать о том, как указывается часовой пояс, можно в [Подразделе 8.5.3](#).

8.5.1.3. Даты и время

Допустимые значения типов timestamp состоят из записи даты и времени, после которого может указываться часовой пояс и необязательное уточнение BC или AD, определяющее эпоху до нашей эры и нашу эру соответственно. (BC/AD можно указать и перед часовым поясом, но предпочтительнее первый вариант.) Таким образом:

1999-01-08 04:05:06

и

1999-01-08 04:05:06 -8:00

допустимые варианты, соответствующие стандарту ISO 8601. В дополнение к этому поддерживается распространённый формат:

January 8 04:05:06 1999 PST

Стандарт SQL различает константы типов timestamp without time zone и timestamp with time zone по знаку «+» или «-» и смещению часового пояса, добавленному после времени. Следовательно, согласно стандарту, записи

TIMESTAMP '2004-10-19 10:23:54'

должен соответствовать тип `timestamp without time zone`, а

```
TIMESTAMP '2004-10-19 10:23:54+02'
```

тип `timestamp with time zone`. Postgres Pro никогда не анализирует содержимое текстовой строки, чтобы определить тип значения, и поэтому обе записи будут обработаны как значения типа `timestamp without time zone`. Чтобы текстовая константа обрабатывалась как `timestamp with time zone`, укажите этот тип явно:

```
TIMESTAMP WITH TIME ZONE '2004-10-19 10:23:54+02'
```

В значении типа `timestamp without time zone` Postgres Pro просто игнорирует часовой пояс. То есть результирующее значение вычисляется только из полей даты/времени и не подстраивается под указанный часовой пояс.

Для значений `timestamp with time zone`, входная строка, содержащая явное указание часового пояса, будет преобразована в UTC (*Universal Coordinated Time*, Всемирное координированное время) с учётом смещения данного часового пояса. Если во входной строке не указан часовой пояс, подразумевается часовой пояс, заданный системным параметром `TimeZone` и время так же пересчитывается в UTC со смещением `timezone`. В обоих случаях значение хранится внутри системы как UTC, а изначально указанный или предполагаемый часовой пояс не сохраняется.

Когда значение `timestamp with time zone` выводится, оно всегда преобразуется из UTC в текущий часовой пояс `timezone` и отображается как локальное время. Чтобы получить время для другого часового пояса, нужно либо изменить `timezone`, либо воспользоваться конструкцией `AT TIME ZONE` (см. [Подраздел 9.9.4](#)).

В преобразованиях между `timestamp without time zone` и `timestamp with time zone` обычно предполагается, что значение `timestamp without time zone` содержит местное время (для часового пояса `timezone`). Другой часовой пояс для преобразования можно задать с помощью `AT TIME ZONE`.

8.5.1.4. Специальные значения

Postgres Pro для удобства поддерживает несколько специальных значений даты/времени, перечисленных в [Таблице 8.13](#). Значения `infinity` и `-infinity` имеют особое представление в системе и они отображаются в том же виде, тогда как другие варианты при чтении преобразуются в значения даты/времени. (В частности, `now` и подобные строки преобразуются в актуальные значения времени в момент чтения.) Чтобы использовать эти значения в качестве констант в командах SQL, их нужно заключать в апострофы.

Таблица 8.13. Специальные значения даты/времени

Вводимая строка	Допустимые типы	Описание
<code>epoch</code>	<code>date</code> , <code>timestamp</code>	1970-01-01 00:00:00+00 (точка отсчёта времени в Unix)
<code>infinity</code>	<code>date</code> , <code>timestamp</code>	время после максимальной допустимой даты
<code>-infinity</code>	<code>date</code> , <code>timestamp</code>	время до минимальной допустимой даты
<code>now</code>	<code>date</code> , <code>time</code> , <code>timestamp</code>	время начала текущей транзакции
<code>today</code>	<code>date</code> , <code>timestamp</code>	время начала текущих суток (00:00)
<code>tomorrow</code>	<code>date</code> , <code>timestamp</code>	время начала следующих суток (00:00)
<code>yesterday</code>	<code>date</code> , <code>timestamp</code>	время начала предыдущих суток (00:00)
<code>allballs</code>	<code>time</code>	00:00:00.00 UTC

Для получения текущей даты/времени соответствующего типа можно также использовать следующие SQL-совместимые функции: `CURRENT_DATE`, `CURRENT_TIME`, `CURRENT_TIMESTAMP`, `LOCALTIME` и `LOCALTIMESTAMP`. (См. [Подраздел 9.9.5.](#)) Заметьте, что во входных строках эти SQL-функции *не* распознаются.

Внимание

Входные значения `now`, `today`, `tomorrow` и `yesterday` вполне корректно работают в интерактивных SQL-командах, но когда команды сохраняются для последующего выполнения, например в подготовленных операторах, представлениях или определениях функций, их поведение может быть неожиданным. Такая строка может преобразоваться в конкретное значение времени, которое затем будет использоваться гораздо позже момента, когда оно было получено. В таких случаях следует использовать одну из SQL-функций. Например, `CURRENT_DATE + 1` будет работать надёжнее, чем `'tomorrow'::date`.

8.5.2. Вывод даты/времени

В качестве выходного формата типов даты/времени можно использовать один из четырёх стилей: ISO 8601, SQL (Ingres), традиционный формат POSTGRES (формат `date` в Unix) или German. По умолчанию выбран формат ISO. (Стандарт SQL требует, чтобы использовался именно ISO 8601. Другой формат называется «SQL» исключительно по историческим причинам.) Примеры всех стилей вывода перечислены в [Таблице 8.14](#). Вообще со значениями типов `date` и `time` выводилась бы только часть даты или времени из показанных примеров, но со стилем POSTGRES значение даты без времени выводится в формате ISO.

Таблица 8.14. Стили вывода даты/время

Стиль	Описание	Пример
ISO	ISO 8601, стандарт SQL	1997-12-17 07:37:16-08
SQL	традиционный стиль	12/17/1997 07:37:16.00 PST
Postgres	изначальный стиль	Wed Dec 17 07:37:16 1997 PST
German	региональный стиль	17.12.1997 07:37:16.00 PST

Примечание

ISO 8601 указывает, что дата должна отделяться от времени буквой `T` в верхнем регистре. Postgres Pro принимает этот формат при вводе, но при выводе вставляет вместо `T` пробел, как показано выше. Это сделано для улучшения читаемости и для совместимости с [RFC 3339](#) и другими СУБД.

В стилях SQL и POSTGRES день выводится перед месяцем, если установлен порядок DMY, а в противном случае месяц выводится перед днём. (Как этот параметр также влияет на интерпретацию входных значений, описано в [Подразделе 8.5.1](#)) Соответствующие примеры показаны в [Таблице 8.15](#).

Таблица 8.15. Соглашения о порядке компонентов даты

Параметр <code>datestyle</code>	Порядок при вводе	Пример вывода
SQL, DMY	день/месяц/год	17/12/1997 15:37:16.00 CET
SQL, MDY	месяц/день/год	12/17/1997 07:37:16.00 PST
Postgres, DMY	день/месяц/год	Wed 17 Dec 07:37:16 1997 PST

В формате ISO часовой пояс всегда отображается в виде числового смещения со знаком относительно всемирного координированного времени (UTC); для зон к востоку от Гринвича знак смещения положительный. Смещение будет отображаться как *hh* (только часы), если оно задаётся целым числом часов, как *hh:mm*, если оно задаётся целым числом минут, или как *hh:mm:ss*. (Третий вариант невозможен в современных стандартах часовых поясов, но он может понадобиться при работе с отметками времени, предшествующими принятию стандартизированных часовых поясов.) В других форматах даты часовой пояс отображается как буквенное сокращение, если оно принято в текущем поясе. В противном случае он отображается как числовое смещение со знаком в стандартном формате ISO 8601 (*hh* или *hhmm*).

Стиль даты/времени пользователь может выбрать с помощью команды `SET datestyle`, параметра `DateStyle` в файле конфигурации `postgresql.conf` или переменной окружения `PGDATESTYLE` на сервере или клиенте.

Для большей гибкости при форматировании выводимой даты/времени можно использовать функцию `to_char` (см. [Раздел 9.8](#)).

8.5.3. Часовые пояса

Часовые пояса и правила их применения определяются, как вы знаете, не только по географическим, но и по политическим соображениям. Часовые пояса во всём мире были более-менее стандартизированы в начале прошлого века, но они продолжают претерпевать изменения, в частности это касается перехода на летнее время. Для расчёта времени в прошлом Postgres Pro получает исторические сведения о правилах часовых поясов из распространённой базы данных IANA (Olson). Для будущего времени предполагается, что в заданном часовом поясе будут продолжать действовать последние принятые правила.

Postgres Pro стремится к совместимости со стандартом SQL в наиболее типичных случаях. Однако стандарт SQL допускает некоторые странности при смешивании типов даты и времени. Две очевидные проблемы:

- Хотя для типа `date` часовой пояс указать нельзя, это можно сделать для типа `time`. В реальности это не очень полезно, так как без даты нельзя точно определить смещение при переходе на летнее время.
- По умолчанию часовой пояс задаётся постоянным смещением от UTC. Это также не позволяет учесть летнее время при арифметических операций с датами, пересекающими границы летнего времени.

Поэтому мы советуем использовать часовой пояс с типами, включающими и время, и дату. Мы *не* рекомендуем использовать тип `time with time zone` (хотя Postgres Pro поддерживает его для старых приложений и совместимости со стандартом SQL). Для типов, включающих только дату или только время, в Postgres Pro предполагается местный часовой пояс.

Все значения даты и времени с часовым поясом представляются внутри в UTC, а при передаче клиентскому приложению они переводятся в местное время, при этом часовой пояс по умолчанию определяется параметром конфигурации `TimeZone`.

Postgres Pro позволяет задать часовой пояс тремя способами:

- Полное название часового пояса, например `America/New_York`. Все допустимые названия перечислены в представлении `pg_timezone_names` (см. [Раздел 53.32](#)). Определения часовых поясов Postgres Pro берёт из широко распространённой базы IANA, так что имена часовых поясов Postgres Pro будут воспринимать и другие приложения.
- Аббревиатура часового пояса, например `PST`. Такое определение просто задаёт смещение от UTC, в отличие от полных названий поясов, которые кроме того подразумевают и правила перехода на летнее время. Распознаваемые аббревиатуры перечислены в представлении `pg_timezone_abbrevs` (см. [Раздел 53.31](#)). Аббревиатуры можно использовать во вводимых значениях даты/времени и в операторе `AT TIME ZONE`, но не в параметрах конфигурации `TimeZone` и `log_timezone`.

- Помимо аббревиатур и названий часовых поясов Postgres Pro принимает указания часовых поясов в стиле POSIX, как описано в [Разделе В.5](#). Этот вариант обычно менее предпочтителен, чем использование именованного часового пояса, но он может быть единственным возможным, если для нужного часового пояса нет записи в базе данных IANA.

Вкратце, различие между аббревиатурами и полными названиями заключается в следующем: аббревиатуры представляют определённый сдвиг от UTC, а полное название подразумевает ещё и местное правило по переходу на летнее время, то есть, возможно, два сдвига от UTC. Например, 2014-06-04 12:00 America/New_York представляет полдень по местному времени в Нью-Йорк, что для данного дня было бы летним восточным временем (EDT или UTC-4). Так что 2014-06-04 12:00 EDT обозначает тот же момент времени. Но 2014-06-04 12:00 EST задаёт стандартное восточное время (UTC-5), не зависящее от того, действовало ли летнее время в этот день.

Мало того, в некоторых юрисдикциях одна и та же аббревиатура часового пояса означала разные сдвиги UTC в разное время; например, аббревиатура московского времени MSK несколько лет означала UTC+3, а затем стала означать UTC+4. Postgres Pro обрабатывает такие аббревиатуры в соответствии с их значениями на заданную дату, но, как и с примером выше EST, это не обязательно будет соответствовать местному гражданскому времени в этот день.

Независимо от формы, регистр в названиях и аббревиатурах часовых поясов не важен. (В PostgreSQL до версии 8.2 он где-то имел значение, а где-то нет.)

Ни названия, ни аббревиатуры часовых поясов, не зашиты в самом сервере; они считываются из файлов конфигурации, находящихся в путях `.../share/timezone/` и `.../share/timezonesets/` относительно каталога установки (см. [Раздел В.4](#)).

Параметр конфигурации [TimeZone](#) можно установить в `postgresql.conf` или любым другим стандартным способом, описанным в [Главе 18](#). Часовой пояс может быть также определён следующими специальными способами:

- Часовой пояс для текущего сеанса можно установить с помощью SQL-команды `SET TIME ZONE`. Это альтернативная запись команды `SET TIMEZONE TO`, более соответствующая SQL-стандарту.
- Если установлена переменная окружения `PGTZ`, клиенты `libpq` используют её значение, выполняя при подключении к серверу команду `SET TIME ZONE`.

8.5.4. Ввод интервалов

Значения типа `interval` могут быть записаны в следующей расширенной форме:

`[@] количество единица [количество единица...] [направление]`

где *количество* — это число (возможно, со знаком); *единица* — одно из значений: `microsecond`, `millisecond`, `second`, `minute`, `hour`, `day`, `week`, `month`, `year`, `decade`, `century`, `millennium` (которые обозначают соответственно микросекунды, миллисекунды, секунды, минуты, часы, дни, недели, месяцы, годы, десятилетия, века и тысячелетия), либо эти же слова во множественном числе, либо их сокращения; *направление* может принимать значение `ago` (назад) или быть пустым. Знак `@` является необязательным. Все заданные величины различных единиц суммируются вместе с учётом знака чисел. Указание `ago` меняет знак всех полей на противоположный. Этот синтаксис также используется при выводе интервала, если параметр [IntervalStyle](#) имеет значение `postgres_verbose`.

Количества дней, часов, минут и секунд можно определить, не указывая явно соответствующие единицы. Например, запись `'1 12:59:10'` равнозначна `'1 day 12 hours 59 min 10 sec'`. Сочетание года и месяца также можно записать через минус; например `'200-10'` означает то, же что и `'200 years 10 months'`. (На самом деле только эти краткие формы разрешены стандартом SQL и они используются при выводе, когда `IntervalStyle` имеет значение `sql_standard`.)

Интервалы можно также записывать в виде, определённом в ISO 8601, либо в «формате с кодами», описанном в разделе 4.4.3.2 этого стандарта, либо в «альтернативном формате», описанном в разделе 4.4.3.3. Формат с кодами выглядит так:

`P количество единица [ количество единица ...] [ T [ количество единица ...]]`

Строка должна начинаться с символа **P** и может включать также **T** перед временем суток. Допустимые коды единиц перечислены в [Таблице 8.16](#). Коды единиц можно опустить или указать в любом порядке, но компоненты времени суток должны идти после символа **T**. В частности, значение кода **M** зависит от того, располагается ли он до или после **T**.

Таблица 8.16. Коды единиц временных интервалов ISO 8601

Код	Значение
Y	годы
M	месяцы (в дате)
W	недели
D	дни
H	часы
M	минуты (во времени)
S	секунды

В альтернативном формате:

P [год-месяц-день] [**T** часы:минуты:секунды]

строка должна начинаться с **P**, а **T** разделяет компоненты даты и времени. Значения выражаются числами так же, как и в датах ISO 8601.

При записи интервальной константы с указанием *полей* или присваивании столбцу типа `interval` строки с *полями*, интерпретация непомеченных величин зависит от *полей*. Например, `INTERVAL '1' YEAR` воспринимается как 1 год, а `INTERVAL '1'` — как 1 секунда. Кроме того, значения «справа» от меньшего значащего поля, заданного в определении *полей*, просто отбрасываются. Например, в записи `INTERVAL '1 day 2:03:04' HOUR TO MINUTE` будут отброшены секунды, но не день.

Согласно стандарту SQL, все компоненты значения `interval` должны быть одного знака, и ведущий минус применяется ко всем компонентам; например, минус в записи `'-1 2:03:04'` применяется и к дню, и к часам/минутам/секундам. Postgres Pro позволяет задавать для разных компонентов разные знаки и традиционно обрабатывает знак каждого компонента в текстовом представлении отдельно от других, так что в данном случае часы/минуты/секунды будут считаться положительными. Если параметр `IntervalStyle` имеет значение `sql_standard`, ведущий знак применяется ко всем компонентам (но только если они не содержат знаки явно). В противном случае действуют традиционные правила Postgres Pro. Во избежание неоднозначности рекомендуется добавлять знак к каждому компоненту с отрицательным значением.

Значения `interval` хранятся в виде трёх целочисленных полей: месяцы, дни и микросекунды. Эти поля хранятся отдельно, поскольку количество дней в месяце варьируется, а день может состоять из 23 или 25 часов при переходе на летнее/зимнее время. Входная строка интервала, в которой используются другие единицы измерения, нормализуется в этот формат, а затем стандартизированным образом реконструируется для вывода, например:

```
SELECT '2 years 15 months 100 weeks 99 hours 123456789 milliseconds'::interval;
interval
-----
3 years 3 mons 700 days 133:17:36.789
```

Здесь недели, под которыми понимаются «7 дней», были сохранены отдельно, а меньшие и большие единицы измерения времени были объединены и нормализованы.

Значения полей могут содержать дробную часть, например `'1.5 weeks'` или `'01:02:03.45'`. Однако поскольку `interval` содержит только целые единицы, дробные значения должны быть преобразованы в более мелкие единицы. Дробные части единиц, превышающих месяцы, округляются до целого числа месяцев, например `'1.5 years'` преобразуется в `'1 year 6 mons'`. Дробные части недель и дней пересчитываются в целое число дней и микросекунд, из расчёта, что в месяце 30

дней, а в сутках — 24 часа, например, '1.75 months' становится 1 mon 22 days 12:00:00. На выходе только секунды могут иметь дробную часть.

В [Таблице 8.17](#) показано несколько примеров допустимых вводимых значений типа `interval`.

Таблица 8.17. Ввод интервалов

Пример	Описание
1-2	Стандартный формат SQL: 1 год и 2 месяца
3 4:05:06	Стандартный формат SQL: 3 дня 4 часа 5 минут 6 секунд
1 year 2 months 3 days 4 hours 5 minutes 6 seconds	Традиционный формат Postgres: 1 год 2 месяца 3 дня 4 часа 5 минут 6 секунд
P1Y2M3DT4H5M6S	«Формат с кодами» ISO 8601: то же значение, что и выше
P0001-02-03T04:05:06	«Альтернативный формат» ISO 8601: то же значение, что и выше

8.5.5. Вывод интервалов

Как объяснялось ранее, PostgreSQL хранит значения `interval` в виде месяцев, дней и микросекунд. При выводе значение месяцев преобразуется в годы и месяцы путём деления на 12. Значение в днях отображается как есть. Значение в микросекундах преобразуется в часы, минуты, секунды и доли секунды. Таким образом, месяцы, минуты и секунды никогда не будут выводиться вне диапазонов 0-11, 0-59 и 0-59 соответственно, тогда как отображаемые значения в годах, днях и часах могут быть довольно большими. (Чтобы перенести большие значения дней и часов в следующее поле, можно использовать функции `justify_days` и `justify_hours`.)

Формат вывода типа `interval` может определяться одним из четырёх стилей: `sql_standard`, `postgres`, `postgres_verbose` и `iso_8601`. Выбрать нужный стиль позволяет команда `SET intervalstyle` (по умолчанию выбран `postgres`). Примеры форматов разных стилей показаны в [Таблице 8.18](#).

Стиль `sql_standard` выдаёт результат, соответствующий стандарту SQL, если значение интервала удовлетворяет ограничениям стандарта (и содержит либо только год и месяц, либо только день и время, и при этом все его компоненты одного знака). В противном случае выводится год-месяц, за которым идёт дата-время, а в компоненты для однозначности явно добавляются знаки.

Вывод в стиле `postgres` соответствует формату, который был принят в PostgreSQL до версии 8.4, когда параметр `DateStyle` имел значение `ISO`.

Вывод в стиле `postgres_verbose` соответствует формату, который был принят в PostgreSQL до версии 8.4, когда значением параметром `DateStyle` было не `ISO`.

Вывод в стиле `iso_8601` соответствует «формату с кодами» описанному в разделе 4.4.3.2 формата ISO 8601.

Таблица 8.18. Примеры стилей вывода интервалов

Стиль	Интервал год-месяц	Интервал день-время	Смешанный интервал
<code>sql_standard</code>	1-2	3 4:05:06	-1-2 +3 -4:05:06
<code>postgres</code>	1 year 2 mons	3 days 04:05:06	-1 year -2 mons +3 days -04:05:06
<code>postgres_verbose</code>	@ 1 year 2 mons	@ 3 days 4 hours 5 mins 6 secs	@ 1 year 2 mons -3 days 4 hours 5 mins 6 secs ago

Стиль	Интервал год-месяц	Интервал день-время	Смешанный интервал
iso_8601	P1Y2M	P3DT4H5M6S	P-1Y-2M3DT-4H-5M-6S

8.6. Логический тип

В Postgres Pro есть стандартный SQL-тип `boolean`; см. [Таблицу 8.19](#). Тип `boolean` может иметь следующие состояния: «true», «false» и третье состояние, «unknown», которое представляется SQL-значением `NULL`.

Таблица 8.19. Логический тип данных

Имя	Размер	Описание
<code>boolean</code>	1 байт	состояние: истина или ложь

Логические константы могут представляться в SQL-запросах следующими ключевыми словами SQL: `TRUE`, `FALSE` и `NULL`.

Функция ввода данных типа `boolean` воспринимает следующие строковые представления состояния «true»:

```
true
yes
on
1
```

и следующие представления состояния «false»:

```
false
no
off
0
```

Также воспринимаются уникальные префиксы этих строк, например `t` или `n`. Регистр символов не имеет значения, а пробельные символы в начале и в конце строки игнорируются.

Функция вывода данных типа `boolean` всегда выдаёт `t` или `f`, как показано в [Примере 8.2](#).

Пример 8.2. Использование типа `boolean`

```
CREATE TABLE test1 (a boolean, b text);
INSERT INTO test1 VALUES (TRUE, 'sic est');
INSERT INTO test1 VALUES (FALSE, 'non est');
SELECT * FROM test1;
 a |      b
---+-----
 t | sic est
 f | non est

SELECT * FROM test1 WHERE a;
 a |      b
---+-----
 t | sic est
```

Ключевые слова `TRUE` и `FALSE` являются предпочтительными (соответствующими стандарту SQL) для записи логических констант в SQL-запросах. Но вы также можете использовать строковые представления, которые допускает синтаксис строковых констант, описанный в [Подразделе 4.1.2.7](#), например, `'yes'::boolean`.

Заметьте, что при анализе запроса `TRUE` и `FALSE` автоматически считаются значениями типа `boolean`, но для `NULL` это не так, потому что ему может соответствовать любой тип. Поэтому в некоторых контекстах может потребоваться привести `NULL` к типу `boolean` явно, например так: `NULL::boolean`. С другой стороны, приведение строковой константы к логическому типу можно опустить в тех контекстах, где анализатор запроса может понять, что буквальное значение должно иметь тип `boolean`.

8.7. Типы перечислений

Типы перечислений (`enum`) определяют статический упорядоченный набор значений, так же как и типы `enum`, существующие в ряде языков программирования. В качестве перечисления можно привести дни недели или набор состояний.

8.7.1. Объявление перечислений

Тип перечислений создаются с помощью команды `CREATE TYPE`, например так:

```
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
```

Созданные типы `enum` можно использовать в определениях таблиц и функций, как и любые другие:

```
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
CREATE TABLE person (
    name text,
    current_mood mood
);
INSERT INTO person VALUES ('Moe', 'happy');
SELECT * FROM person WHERE current_mood = 'happy';
name | current_mood
-----+-----
Moe  | happy
(1 row)
```

8.7.2. Порядок

Порядок значений в перечислении определяется последовательностью, в которой были указаны значения при создании типа. Перечисления поддерживаются всеми стандартными операторами сравнения и связанными агрегатными функциями. Например:

```
INSERT INTO person VALUES ('Larry', 'sad');
INSERT INTO person VALUES ('Curly', 'ok');
SELECT * FROM person WHERE current_mood > 'sad';
name | current_mood
-----+-----
Moe  | happy
Curly | ok
(2 rows)

SELECT * FROM person WHERE current_mood > 'sad' ORDER BY current_mood;
name | current_mood
-----+-----
Curly | ok
Moe    | happy
(2 rows)

SELECT name
FROM person
WHERE current_mood = (SELECT MIN(current_mood) FROM person);
name
-----
```

```
Larry
(1 row)
```

8.7.3. Безопасность типа

Все типы перечислений считаются уникальными и поэтому значения разных типов нельзя сравнивать. Взгляните на этот пример:

```
CREATE TYPE happiness AS ENUM ('happy', 'very happy', 'ecstatic');
CREATE TABLE holidays (
    num_weeks integer,
    happiness happiness
);
INSERT INTO holidays(num_weeks,happiness) VALUES (4, 'happy');
INSERT INTO holidays(num_weeks,happiness) VALUES (6, 'very happy');
INSERT INTO holidays(num_weeks,happiness) VALUES (8, 'ecstatic');
INSERT INTO holidays(num_weeks,happiness) VALUES (2, 'sad');
ОШИБКА: неверное значение для перечисления happiness: "sad"
SELECT person.name, holidays.num_weeks FROM person, holidays
    WHERE person.current_mood = holidays.happiness;
ОШИБКА: оператор не существует: mood = happiness
```

Если вам действительно нужно сделать что-то подобное, вы можете либо реализовать собственный оператор, либо явно преобразовать типы в запросе:

```
SELECT person.name, holidays.num_weeks FROM person, holidays
    WHERE person.current_mood::text = holidays.happiness::text;
name | num_weeks
-----+-----
Moe  |         4
(1 row)
```

8.7.4. Тонкости реализации

В метках значений регистр имеет значение, т. е. 'happy' и 'HAPPY' — не одно и то же. Также в метках имеют значение пробелы.

Хотя типы-перечисления предназначены прежде всего для статических наборов значений, имеется возможность добавлять новые значения в существующий тип-перечисление и переименовывать значения (см. [ALTER TYPE](#)). Однако удалять существующие значения из перечисления, а также изменять их порядок, нельзя — для получения нужного результата придётся удалить и воссоздать это перечисление.

Значение enum занимает на диске 4 байта. Длина текстовой метки значения ограничена параметром компиляции NAMEDATALEN; в стандартных сборках Postgres Pro он ограничивает длину 63 байтами.

Сопоставления внутренних значений enum с текстовыми метками хранятся в системном каталоге [pg_enum](#). Он может быть полезен в ряде случаев.

8.8. Геометрические типы

Геометрические типы данных представляют объекты в двумерном пространстве. Все существующие в Postgres Pro геометрические типы перечислены в [Таблице 8.20](#).

Таблица 8.20. Геометрические типы

Имя	Размер	Описание	Представление
point	16 байт	Точка на плоскости	(x,y)
line	24 байта	Бесконечная прямая	{A,B,C}

Имя	Размер	Описание	Представление
lseg	32 байта	Отрезок	((x1,y1),(x2,y2))
box	32 байта	Прямоугольник	((x1,y1),(x2,y2))
path	16+16n байт	Закрытый путь (подобный многоугольнику)	((x1,y1),...)
path	16+16n байт	Открытый путь	[(x1,y1),...]
polygon	40+16n байт	Многоугольник (подобный закрытому пути)	((x1,y1),...)
circle	24 байта	Окружность	<(x,y),r> (центр окружности и радиус)

Во всех этих типах отдельные координаты хранятся как числа типа `double precision (float8)`.

Для выполнения различных геометрических операций, в частности масштабирования, вращения и определения пересечений, PostgreSQL предлагает богатый набор функций и операторов. Они рассматриваются в [Разделе 9.11](#).

8.8.1. Точки

Точки — это основной элемент, на базе которого строятся все остальные геометрические типы. Значения типа `point` записываются в одном из двух форматов:

```
( x , y )
x , y
```

где x и y — координаты точки на плоскости, выраженные числами с плавающей точкой.

Выводятся точки в первом формате.

8.8.2. Прямые

Прямые представляются линейным уравнением $ax + by + c = 0$, где a и b не могут быть равны 0 одновременно. Значения типа `line` вводятся и выводятся в следующем виде:

```
{ A, B, C }
```

Кроме того, для ввода может использоваться любая из этих форм:

```
[ ( x1 , y1 ) , ( x2 , y2 ) ]
( ( x1 , y1 ) , ( x2 , y2 ) )
( x1 , y1 ) , ( x2 , y2 )
x1 , y1 , x2 , y2
```

где $(x1, y1)$ и $(x2, y2)$ — две различные точки на данной прямой.

8.8.3. Отрезки

Отрезок представляется парой точек, определяющих концы отрезка. Значения типа `lseg` записываются в одной из следующих форм:

```
[ ( x1 , y1 ) , ( x2 , y2 ) ]
( ( x1 , y1 ) , ( x2 , y2 ) )
( x1 , y1 ) , ( x2 , y2 )
x1 , y1 , x2 , y2
```

где $(x1, y1)$ и $(x2, y2)$ — концы отрезка.

Выводятся отрезки в первом формате.

8.8.4. Прямоугольники

Прямоугольник представляется двумя точками, находящимися в противоположных его углах. Значения типа `box` записываются в одной из следующих форм:

```
( ( x1 , y1 ) , ( x2 , y2 ) )
( x1 , y1 ) , ( x2 , y2 )
x1 , y1 , x2 , y2
```

где $(x1, y1)$ и $(x2, y2)$ — противоположные углы прямоугольника.

Выводятся прямоугольники во второй форме.

Во вводимом значении могут быть указаны любые два противоположных угла, но затем они будут упорядочены, так что внутри сохранятся правый верхний и левый нижний углы, в таком порядке.

8.8.5. Пути

Пути представляют собой списки соединённых точек. Пути могут быть *закрытыми*, когда подразумевается, что первая и последняя точка в списке соединены, или *открытыми*, в противном случае.

Значения типа `path` записываются в одной из следующих форм:

```
[ ( x1 , y1 ) , ... , ( xn , yn ) ]
( ( x1 , y1 ) , ... , ( xn , yn ) )
( x1 , y1 ) , ... , ( xn , yn )
( x1 , y1 , ... , xn , yn )
x1 , y1 , ... , xn , yn
```

где точки задают узлы сегментов, составляющих путь. Квадратные скобки `[]` указывают, что путь открытый, а круглые `()` — закрытый. Когда внешние скобки опускаются, как в показанных выше последних трёх формах, считается, что путь закрытый.

Пути выводятся в первой или второй форме, в соответствии с типом.

8.8.6. Многоугольники

Многоугольники представляются списками точек (вершин). Многоугольники похожи на закрытые пути, но важное семантическое отличие состоит в том, что многоугольник считается содержащим область внутри него, а путь — нет.

Важное различие в реализации между многоугольниками и путями заключается в том, что сохранённое представление многоугольника включает его наименьший окружающий прямоугольник. Это ускоряет определённые операции поиска, хотя вычисление окружающего прямоугольника добавляет издержки при построении новых многоугольников.

Значения типа `polygon` записываются в одной из следующих форм:

```
( ( x1 , y1 ) , ... , ( xn , yn ) )
( x1 , y1 ) , ... , ( xn , yn )
( x1 , y1 , ... , xn , yn )
x1 , y1 , ... , xn , yn
```

где точки задают узлы сегментов, образующих границу многоугольника.

Выводятся многоугольники в первом формате.

8.8.7. Окружности

Окружности задаются координатами центра и радиусом. Значения типа `circle` записываются в одном из следующих форматов:

```
< ( x , y ) , r >
( ( x , y ) , r )
( x , y ) , r
```

x, y, r

где (x, y) — центр окружности, а r — её радиус.

Выводятся окружности в первом формате.

8.9. Типы, описывающие сетевые адреса

Postgres Pro предлагает типы данных для хранения адресов IPv4, IPv6 и MAC, показанные в [Таблице 8.21](#). Для хранения сетевых адресов лучше использовать эти типы, а не простые текстовые строки, так как Postgres Pro проверяет вводимые значения данных типов и предоставляет специализированные операторы и функции для работы с ними (см. [Раздел 9.12](#)).

Таблица 8.21. Типы, описывающие сетевые адреса

Имя	Размер	Описание
<code>cidr</code>	7 или 19 байт	Сети IPv4 и IPv6
<code>inet</code>	7 или 19 байт	Узлы и сети IPv4 и IPv6
<code>macaddr</code>	6 байт	MAC-адреса
<code>macaddr8</code>	8 байт	MAC-адреса (в формате EUI-64)

При сортировке типов `inet` и `cidr`, адреса IPv4 всегда идут до адресов IPv6, в том числе адреса IPv4, включённые в IPv6 или сопоставленные с ними, например `::10.2.3.4` или `::ffff:10.4.3.2`.

8.9.1. `inet`

Тип `inet` содержит IPv4- или IPv6-адрес узла и может также содержать его подсеть, всё в одном поле. Подсеть представляется числом бит, определяющих адрес сети в адресе узла (или «маску сети»). Если маска сети равна 32 для адреса IPv4, такое значение представляет не подсеть, а определённый узел. Адреса IPv6 имеют длину 128 бит, поэтому уникальный адрес узла задаётся с маской 128 бит. Заметьте, что когда нужно, чтобы принимались только адреса сетей, следует использовать тип `cidr`, а не `inet`.

Вводимые значения такого типа должны иметь формат `адрес/у`, где `адрес` — адрес IPv4 или IPv6, а `у` — число бит в маске сети. Если компонент `/у` опущен, маска сети считается равной 32 для IPv4 и 128 для IPv6, так что это значение будет представлять один узел. При выводе компонент `/у` опускается, если сетевой адрес определяет адрес одного узла.

8.9.2. `cidr`

Тип `cidr` содержит определение сети IPv4 или IPv6. Входные и выходные форматы соответствуют соглашениям CIDR (Classless Internet Domain Routing, Бесклассовая межсетевая адресация). Определение сети записывается в формате `адрес/у`, где `адрес` — минимальный адрес в сети, представленный в виде адреса IPv4 или IPv6, а `у` — число бит в маске сети. Если `у` не указывается, это значение вычисляется по старой классовой схеме нумерации сетей, но при этом оно может быть увеличено, чтобы в него вошли все байты введённого адреса. Если в сетевом адресе справа от маски сети окажутся биты со значением 1, он будет считаться ошибочным.

В [Таблице 8.22](#) показаны несколько примеров адресов.

Таблица 8.22. Примеры допустимых значений типа `cidr`

Вводимое значение <code>cidr</code>	Выводимое значение <code>cidr</code>	<code>abbrev(cidr)</code>
192.168.100.128/25	192.168.100.128/25	192.168.100.128/25
192.168/24	192.168.0.0/24	192.168.0/24
192.168/25	192.168.0.0/25	192.168.0.0/25
192.168.1	192.168.1.0/24	192.168.1/24

Вводимое значение cidr	Выводимое значение cidr	abbrev(cidr)
192.168	192.168.0.0/24	192.168.0/24
128.1	128.1.0.0/16	128.1/16
128	128.0.0.0/16	128.0/16
128.1.2	128.1.2.0/24	128.1.2/24
10.1.2	10.1.2.0/24	10.1.2/24
10.1	10.1.0.0/16	10.1/16
10	10.0.0.0/8	10/8
10.1.2.3/32	10.1.2.3/32	10.1.2.3/32
2001:4f8:3:ba::/64	2001:4f8:3:ba::/64	2001:4f8:3:ba/64
2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128	2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128	2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128
::ffff:1.2.3.0/120	::ffff:1.2.3.0/120	::ffff:1.2.3/120
::ffff:1.2.3.0/128	::ffff:1.2.3.0/128	::ffff:1.2.3.0/128

8.9.3. Различия inet и cidr

Существенным различием типов данных `inet` и `cidr` является то, что `inet` принимает значения с ненулевыми битами справа от маски сети, а `cidr` — нет. Например, значение `192.168.0.1/24` является допустимым для типа `inet`, но не для `cidr`.

Подсказка

Если вас не устраивает выходной формат значений `inet` или `cidr`, попробуйте функции `host`, `text` и `abbrev`.

8.9.4. macaddr

Тип `macaddr` предназначен для хранения MAC-адреса, примером которого является адрес сетевой платы Ethernet (хотя MAC-адреса применяются и для других целей). Вводимые значения могут задаваться в следующих форматах:

```
'08:00:2b:01:02:03'
'08-00-2b-01-02-03'
'08002b:010203'
'08002b-010203'
'0800.2b01.0203'
'0800-2b01-0203'
'08002b010203'
```

Все эти примеры определяют один и тот же адрес. Шестнадцатеричные цифры от `a` до `f` могут быть и в нижнем, и в верхнем регистре. Выводятся MAC-адреса всегда в первой форме.

В стандарте IEEE 802-2001 стандартной считается вторая форма MAC-адресов (с минусами), а первая (с двоеточиями) предполагает обратный порядок бит, от старшего к младшему, так что `08-00-2b-01-02-03` = `10:00:D4:80:40:C0`. В настоящее время этому соглашению практически никто не следует, и уместно оно было только для устаревших сетевых протоколов (таких как Token Ring). Postgres Pro не меняет порядок бит и во всех принимаемых форматах подразумевается традиционный порядок LSB.

Последние пять входных форматов не описаны ни в каком стандарте.

8.9.5. macaddr8

Тип `macaddr8` хранит MAC-адреса в формате EUI-64, применяющиеся, например, для аппаратных адресов плат Ethernet (хотя MAC-адреса используются и для других целей). Этот тип может принять и 6-байтовые, и 8-байтовые адреса MAC и сохраняет их в 8 байтах. MAC-адреса, заданные в 6-байтовом формате, хранятся в формате 8 байт, а 4-й и 5-й байт содержат FF и FE, соответственно. Заметьте, что для IPv6 используется модифицированный формат EUI-64, в котором 7-й бит должен быть установлен в 1 после преобразования из EUI-48. Для выполнения этого изменения предоставляется функция `macaddr8_set7bit`. Вообще говоря, этот тип принимает любые строки, состоящие из пар шестнадцатеричных цифр (выровненных по границам байт), которые могут согласованно разделяться одинаковыми символами ':', '-' или '.'. Шестнадцатеричных цифр должно быть либо 16 (для 8 байт), либо 12 (для 6 байт). Начальные и конечные пробелы игнорируются. Ниже показаны примеры допустимых входных строк:

```
'08:00:2b:01:02:03:04:05'
'08-00-2b-01-02-03-04-05'
'08002b:0102030405'
'08002b-0102030405'
'0800.2b01.0203.0405'
'0800-2b01-0203-0405'
'08002b01:02030405'
'08002b0102030405'
```

Во всех этих примерах задаётся один и тот же адрес. Для цифр с `a` по `f` принимаются буквы и в верхнем, и в нижнем регистре. Вывод всегда представляется в первом из показанных форматов.

Последние шесть входных форматов из показанных выше не являются стандартизированными.

Чтобы преобразовать традиционный 48-битный MAC-адрес в формате EUI-48 в модифицированный формат EUI-64 для включения в состав адреса IPv6 в качестве адреса узла, используйте функцию `macaddr8_set7bit` следующим образом:

```
SELECT macaddr8_set7bit('08:00:2b:01:02:03');

 macaddr8_set7bit
-----
 0a:00:2b:ff:fe:01:02:03
(1 row)
```

8.10. Битовые строки

Битовые строки представляют собой последовательности из 1 и 0. Их можно использовать для хранения или отображения битовых масок. В SQL есть два битовых типа: `bit(n)` и `bit varying(n)`, где n — положительное целое число.

Длина значения типа `bit` должна в точности равняться n ; при попытке сохранить данные длиннее или короче произойдёт ошибка. Данные типа `bit varying` могут иметь переменную длину, но не превышающую n ; строки большей длины не будут приняты. Запись `bit` без указания длины равнозначна записи `bit(1)`, тогда как `bit varying` без указания длины подразумевает строку неограниченной длины.

Примечание

При попытке привести значение битовой строки к типу `bit(n)`, оно будет усечено или дополнено нулями справа до длины ровно n бит, ошибки при этом не будет. Подобным образом, если явно привести значение битовой строки к типу `bit varying(n)`, она будет усечена справа, если её длина превышает n бит.

Синтаксис констант битовых строк описан в [Подразделе 4.1.2.5](#), а все доступные битовые операторы и функции перечислены в [Разделе 9.6](#).

Пример 8.3. Использование битовых строк

```
CREATE TABLE test (a BIT(3), b BIT VARYING(5));
INSERT INTO test VALUES (B'101', B'00');
INSERT INTO test VALUES (B'10', B'101');
```

ОШИБКА: длина битовой строки (2) не соответствует типу bit(3)

```
INSERT INTO test VALUES (B'10'::bit(3), B'101');
SELECT * FROM test;
```

a	b
101	00
100	101

Для хранения битовой строки используется по 1 байту для каждой группы из 8 бит, плюс 5 или 8 байт дополнительно в зависимости от длины строки (но длинные строки могут быть сжаты или вынесены отдельно, как описано в [Разделе 8.3](#) применительно к символьным строкам).

8.11. Типы, предназначенные для текстового поиска

Postgres Pro предоставляет два типа данных для поддержки полнотекстового поиска. Текстовым поиском называется операция анализа набора *документов* с текстом на естественном языке, в результате которой находятся фрагменты, наиболее соответствующие *запросу*. Тип `tsvector` представляет документ в виде, оптимизированном для текстового поиска, а `tsquery` представляет запрос текстового поиска в подобном виде. Более подробно это описывается в [Главе 12](#), а все связанные функции и операторы перечислены в [Разделе 9.13](#).

8.11.1. tsvector

Значение типа `tsvector` содержит отсортированный список неповторяющихся *лексем*, т. е. слов, *нормализованных* так, что все словоформы сводятся к одной (подробнее это описано в [Главе 12](#)). Сортировка и исключение повторяющихся слов производится автоматически при вводе значения, как показано в этом примере:

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::tsvector;
           tsvector
-----
'a' 'and' 'ate' 'cat' 'fat' 'mat' 'on' 'rat' 'sat'
```

Для представления в виде лексем пробелов или знаков препинания их нужно заключить в апострофы:

```
SELECT $$the lexeme ' ' contains spaces$$::tsvector;
           tsvector
-----
' ' 'contains' 'lexeme' 'spaces' 'the'
```

(В данном и следующих примерах мы используем строку в долларах, чтобы не дублировать все апострофы в таких строках.) При этом включаемый апостроф или обратную косую черту нужно продублировать:

```
SELECT $$the lexeme 'Joe''s' contains a quote$$::tsvector;
           tsvector
-----
'Joe''s' 'a' 'contains' 'lexeme' 'quote' 'the'
```

Также для лексем можно указать их целочисленные *позиции*:

```
SELECT 'a:1 fat:2 cat:3 sat:4 on:5 a:6 mat:7 and:8 ate:9 a:10 fat:11 rat:12'::tsvector;
           tsvector
-----
```

```
'a':1,6,10 'and':8 'ate':9 'cat':3 'fat':2,11 'mat':7 'on':5 'rat':12 'sat':4
```

Позиция обычно указывает положение исходного слова в документе. Информация о расположении слов затем может использоваться для *оценки близости*. Позиция может задаваться числом от 1 до 16383; большие значения просто заменяются на 16383. Если для одной лексемы дважды указывается одно положение, такое повторение отбрасывается.

Лексемам, для которых заданы позиции, также можно назначить *вес*, выраженный буквами A, B, C или D. Вес D подразумевается по умолчанию и поэтому не показывается при выводе:

```
SELECT 'a:1A fat:2B,4C cat:5D'::tsvector;
      tsvector
-----
'a':1A 'cat':5 'fat':2B,4C
```

Веса обычно применяются для отражения структуры документа, например для придания особого значения словам в заголовке по сравнению со словами в обычном тексте. Назначенным весам можно сопоставить числовые приоритеты в функциях ранжирования результатов.

Важно понимать, что тип `tsvector` сам по себе не выполняет нормализацию слов; предполагается, что в сохраняемом значении слова уже нормализованы приложением. Например:

```
SELECT 'The Fat Rats'::tsvector;
      tsvector
-----
'Fat' 'Rats' 'The'
```

Для большинства англоязычных приложений приведённые выше слова будут считаться ненормализованными, но для `tsvector` это не важно. Поэтому исходный документ обычно следует обработать функцией `to_tsvector`, нормализующей слова для поиска:

```
SELECT to_tsvector('english', 'The Fat Rats');
      to_tsvector
-----
'fat':2 'rat':3
```

И это подробнее описано в [Главе 12](#).

8.11.2. tsquery

Значение `tsquery` содержит искомые лексемы, объединяемые логическими операторами `&` (И), `|` (ИЛИ) и `!` (НЕ), а также оператором поиска фраз `<->` (ПРЕДШЕСТВУЕТ). Также допускается вариация оператора ПРЕДШЕСТВУЕТ вида `<N>`, где *N* — целочисленная константа, задающая расстояние между двумя искомыми лексемами. Запись оператора `<->` равнозначна `<1>`.

Для группировки операторов могут использоваться скобки. Без скобок эти операторы имеют разные приоритеты, в порядке убывания: `!` (НЕ), `<->` (ПРЕДШЕСТВУЕТ), `&` (И) и `|` (ИЛИ).

Несколько примеров:

```
SELECT 'fat & rat'::tsquery;
      tsquery
-----
'fat' & 'rat'

SELECT 'fat & (rat | cat)'::tsquery;
      tsquery
-----
'fat' & ( 'rat' | 'cat' )

SELECT 'fat & rat & ! cat'::tsquery;
      tsquery
-----
```

```
'fat' & 'rat' & '!cat'
```

Лексемам в `tsquery` можно дополнительно сопоставить буквы весов, при этом они будут соответствовать только тем лексемам в `tsvector`, которые имеют какой-либо из этих весов:

```
SELECT 'fat:ab & cat':::tsquery;
      tsquery
-----
'fat':AB & 'cat'
```

Кроме того, в лексемах `tsquery` можно использовать знак `*` для поиска по префиксу:

```
SELECT 'super:*':::tsquery;
      tsquery
-----
'super':*
```

Этот запрос найдёт все слова в `tsvector`, начинающиеся с приставки «super».

Апострофы в лексемах этого типа можно использовать так же, как и в лексемах в `tsvector`; и так же, как и для типа `tsvector`, необходимая нормализация слова должна выполняться до приведения значения к типу `tsquery`. Для такой нормализации удобно использовать функцию `to_tsquery`:

```
SELECT to_tsquery('Fat:ab & Cats');
      to_tsquery
-----
'fat':AB & 'cat'
```

Заметьте, что функция `to_tsquery` будет обрабатывать префиксы подобно другим словам, поэтому следующее сравнение возвращает `true`:

```
SELECT to_tsvector( 'postgraduate' ) @@ to_tsquery( 'postgres:*' );
?column?
-----
t
```

так как `postgres` преобразуется стеммером в `postgr`:

```
SELECT to_tsvector( 'postgraduate' ), to_tsquery( 'postgres:*' );
      to_tsvector | to_tsquery
-----+-----
'postgradu':1 | 'postgr':*
```

и эта приставка находится в преобразованной форме слова `postgraduate`.

8.12. Тип UUID

Тип данных `uuid` сохраняет универсальные уникальные идентификаторы (Universally Unique Identifiers, UUID), определённые в [RFC 4122](#), ISO/IEC 9834-8:2005 и связанных стандартах. (В некоторых системах это называется GUID, глобальным уникальным идентификатором.) Этот идентификатор представляет собой 128-битное значение, генерируемое специальным алгоритмом, практически гарантирующим, что этим же алгоритмом оно не будет получено больше нигде в мире. Таким образом, эти идентификаторы будут уникальными и в распределённых системах, а не только в единственной базе данных, как значения генераторов последовательностей.

UUID записывается в виде последовательности шестнадцатеричных цифр в нижнем регистре, разделённых знаками минуса на несколько групп, в таком порядке: группа из 8 цифр, за ней три группы из 4 цифр и, наконец, группа из 12 цифр, что в сумме составляет 32 цифры и представляет 128 бит. Пример UUID в этом стандартном виде:

```
a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11
```

Postgres Pro также принимает альтернативные варианты: цифры в верхнем регистре, стандартную запись, заключённую в фигурные скобки, запись без минусов или с минусами, разделяющими любые группы из четырёх цифр. Например:


```
A0EEBC99-9C0B-4EF8-BB6D-6BB9BD380A11
{a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11}
a0eebc999c0b4ef8bb6d6bb9bd380a11
a0ee-bc99-9c0b-4ef8-bb6d-6bb9-bd38-0a11
{a0eebc99-9c0b4ef8-bb6d6bb9-bd380a11}
```

Выводится значение этого типа всегда в стандартном виде.

Как можно сгенерировать UUID в PostgreSQL, описывается в [Разделе 9.14](#).

8.13. Тип XML

Тип `xml` предназначен для хранения XML-данных. Его преимущество по сравнению с обычным типом `text` в том, что он проверяет вводимые значения на допустимость по правилам XML и для работы с ним есть типобезопасные функции; см. [Раздел 9.15](#). Для использования этого типа дистрибутив должен быть скомпилирован в конфигурации `configure --with-libxml`.

Тип `xml` может сохранять правильно оформленные «документы», в соответствии со стандартом XML, а также фрагменты «содержимого», определяемые как менее ограниченные «узлы документа» в модели данных XQuery и XPath. Другими словами, это означает, что во фрагментах содержимого может быть несколько элементов верхнего уровня или текстовых узлов. Является ли некоторое значение типа `xml` полным документом или фрагментом содержимого, позволяет определить выражение `xml-значение IS DOCUMENT`.

Информацию о совместимости и ограничениях типа данных `xml` можно найти в [Разделе D.3](#).

8.13.1. Создание XML-значений

Чтобы получить значение типа `xml` из текстовой строки, используйте функцию `xmlparse`:

```
XMLPARSE ( { DOCUMENT | CONTENT } value)
```

Примеры:

```
XMLPARSE (DOCUMENT '<?xml version="1.0"?><book><title>Manual</title><chapter>...</chapter></book>')
XMLPARSE (CONTENT 'abc<foo>bar</foo><bar>foo</bar>')
```

Хотя в стандарте SQL описан только один способ преобразования текстовых строк в XML-значения, специфический синтаксис Postgres Pro:

```
xml '<foo>bar</foo>'
'<foo>bar</foo>'::xml
```

тоже допустим.

Тип `xml` не проверяет вводимые значения по схеме DTD (Document Type Declaration, Объявления типа документа), даже если в них присутствуют ссылка на DTD. В настоящее время в PostgreSQL также нет встроенной поддержки других разновидностей схем, например XML Schema.

Обратная операция, получение текстовой строки из `xml`, выполняется с помощью функции `xmlserialize`:

```
XMLSERIALIZE ( { DOCUMENT | CONTENT } значение AS тип )
```

Здесь допустимый *тип* — `character`, `character varying` или `text` (или их псевдонимы). И в данном случае стандарт SQL предусматривает только один способ преобразования `xml` в тип текстовых строк, но Postgres Pro позволяет просто привести значение к нужному типу.

При преобразовании текстовой строки в тип `xml` или наоборот без использования функций `XMLPARSE` и `XMLSERIALIZE`, выбор режима `DOCUMENT` или `CONTENT` определяется параметром конфигурации сеанса «XML option», установить который можно следующей стандартной командой:

```
SET XML OPTION { DOCUMENT | CONTENT };
```

или такой командой в духе Postgres Pro:

```
SET xmloption TO { DOCUMENT | CONTENT };
```

По умолчанию этот параметр имеет значение `CONTENT`, так что допускаются все формы XML-данных.

8.13.2. Обработка кодировки

Если на стороне сервера и клиента и в XML-данных используются разные кодировки символов, с этим могут возникать проблемы. Когда запросы передаются на сервер, а их результаты возвращаются клиенту в обычном текстовом режиме, Postgres Pro преобразует все передаваемые текстовые данные в кодировку для соответствующей стороны; см. [Раздел 22.3](#). В том числе это происходит и со строковыми представлениями XML-данных, подобными тем, что показаны в предыдущих примерах. Обычно это означает, что объявления кодировки, содержащиеся в XML-данных, могут не соответствовать действительности, когда текстовая строка преобразуется из одной кодировки в другую при передаче данных между клиентом и сервером, так как подобные включённые в данные объявления не будут изменены автоматически. Для решения этой проблемы объявления кодировки, содержащиеся в текстовых строках, вводимых в тип `xml`, просто *игнорируются* и предполагается, что XML-содержимое представлено в текущей кодировке сервера. Как следствие, для правильной обработки таких строк с XML-данными клиент должен передавать их в своей текущей кодировке. Для сервера не важно, будет ли клиент для этого преобразовывать документы в свою кодировку, или изменит её, прежде чем передавать ему данные. При выводе значения типа `xml` не содержат объявления кодировки, а клиент должен предполагать, что все данные поступают в его текущей кодировке.

Если параметры запроса передаются на сервер и он возвращает результаты клиенту в двоичном режиме, кодировка символов не преобразуется, так что возникает другая ситуация. В этом случае объявление кодировки в XML принимается во внимание, а если его нет, то предполагается, что данные закодированы в UTF-8 (это соответствует стандарту XML; заметьте, что Postgres Pro не поддерживает UTF-16). При выводе в данные будет добавлено объявление кодировки, выбранной на стороне клиента (но если это UTF-8, объявление будет опущено).

Само собой, XML-данные в Postgres Pro будут обрабатываться гораздо эффективнее, когда и в XML-данных, и на стороне клиента, и на стороне сервера используется одна кодировка. Так как внутри XML-данные представляются в UTF-8, оптимальный вариант, когда на сервере также выбрана кодировка UTF-8.

Внимание

Некоторые XML-функции способны работать исключительно с ASCII-данными, если кодировка сервера не UTF-8. В частности, это известная особенность функций `xmltable()` и `xpath()`.

8.13.3. Обращение к XML-значениям

Тип `xml` отличается от других тем, что для него не определены никакие операторы сравнения, так как чётко определённого и универсального алгоритма сравнения XML-данных не существует. Одно из следствий этого — нельзя отфильтровать строки таблицы, сравнив столбец `xml` с искомым значением. Поэтому обычно XML-значения должны дополняться отдельным ключевым полем, например ID. Можно также сравнивать XML-значения, преобразовав их сначала в текстовые строки, но заметьте, что с учётом специфики XML-данных этот метод практически бесполезен.

Из-за отсутствия операторов сравнения для типа `xml`, для столбца этого типа также нельзя создать индекс. Поэтому, когда требуется быстрый поиск в XML данных, обойти это ограничение можно, приведя данные к типу текстовой строки и проиндексировав эти строки, либо проиндексировав выражение XPath. Конечно сам запрос при этом следует изменить, чтобы поиск выполнялся по индексированному выражению.

Для ускорения поиска в XML-данных также можно использовать функции полнотекстового поиска в Postgres Pro. Однако это требует определённой подготовки данных, что дистрибутив Postgres Pro пока не поддерживает.

8.14. Типы JSON

Типы JSON предназначены для хранения данных JSON (JavaScript Object Notation, Запись объекта JavaScript) согласно стандарту [RFC 7159](#). Такие данные можно хранить и в типе `text`, но типы JSON лучше тем, что проверяют, соответствует ли вводимое значение формату JSON. Для работы с ними есть также несколько специальных функций и операторов; см. [Раздел 9.16](#).

В Postgres Pro имеются два типа для хранения данных JSON: `json` и `jsonb`. Для реализации эффективного механизма запросов к этим типам данных в Postgres Pro также имеется тип `jsonpath`, описанный в [Подразделе 8.14.7](#).

Типы данных `json` и `jsonb` принимают на вход *почти* одинаковые наборы значений, а отличаются они главным образом с точки зрения эффективности. Тип `json` сохраняет точную копию введённого текста, которую функции обработки должны разбирать заново при каждом выполнении запроса, тогда как данные `jsonb` сохраняются в разобранном двоичном формате, что несколько замедляет ввод из-за преобразования, но значительно ускоряет обработку, не требуя многократного разбора текста. Кроме того, `jsonb` поддерживает индексацию, что тоже может быть очень полезно.

Так как тип `json` сохраняет точную копию введённого текста, он сохраняет семантически незначимые пробелы между элементами, а также порядок ключей в JSON-объектах. И если JSON-объект внутри содержит повторяющиеся ключи, этот тип сохранит все пары ключ/значение. (Функции обработки будут считать действительной последнюю пару.) Тип `jsonb`, напротив, не сохраняет пробелы, порядок ключей и значения с дублирующимися ключами. Если во входных данных оказываются дублирующиеся ключи, сохраняется только последнее значение.

Для большинства приложений предпочтительнее хранить данные JSON в типе `jsonb` (если нет особых противопоказаний, например важны прежние предположения о порядке ключей объектов).

В RFC 7159 говорится, что строки JSON должны быть представлены в кодировке UTF-8. Поэтому данные JSON не будут полностью соответствовать спецификации, если кодировка базы данных не UTF-8. При этом нельзя будет вставить в JSON символы, непредставимые в кодировке сервера, и наоборот, допустимыми будут символы, представимые в кодировке сервера, но не в UTF-8.

RFC 7159 разрешает включать в строки JSON спецпоследовательности Unicode в виде `\uXXXX`. В функцию ввода для типа `json` эти спецпоследовательности допускаются вне зависимости от кодировки базы данных, и проверяется только правильность их синтаксиса (за `\u` должны следовать четыре шестнадцатеричных цифры). Однако функция ввода для типа `jsonb` более строгая: она не допускает спецпоследовательности Unicode для символов, которые не могут быть представлены в кодировке базы. Тип `jsonb` также не принимает `\u0000` (так как это значение не может быть представлено в типе `text` Postgres Pro) и требует, чтобы суррогатные пары Unicode использовались для представления символов вне основной многоязыковой плоскости (BMP) правильно. Корректные спецпоследовательности Unicode преобразуются для хранения в один соответствующий символ (это подразумевает сворачивание суррогатных пар в один символ).

Примечание

Многие из функций обработки JSON, описанные в [Разделе 9.16](#), преобразуют спецпоследовательности Unicode в обычные символы, поэтому могут выдавать подобные ошибки, даже если им на вход поступает тип `json`, а не `jsonb`. То, что функция ввода в тип `json` не производит этих проверок, можно считать историческим артефактом, хотя это и позволяет просто сохранять (но не обрабатывать) в JSON спецкоды Unicode в базе данных с кодировкой, в которой представленные таким образом символы отсутствуют.

При преобразовании вводимого текста JSON в тип `jsonb`, примитивные типы, описанные в RFC 7159, по сути отображаются в собственные типы Postgres Pro как показано в [Таблице 8.23](#). Таким образом, к содержимому типа `jsonb` предъявляются некоторые дополнительные требования, продиктованные ограничениями представления нижележащего типа данных, которые не распространяются ни на тип `json`, ни на формат JSON вообще. В частности, тип `jsonb` не принимает числа, выходящие за диапазон типа данных Postgres Pro `numeric`, тогда как с `json` такого ограничения нет. Такие ограничения, накладываемые реализацией, допускаются согласно RFC 7159. Однако на практике такие проблемы более вероятны в других реализациях, так как обычно примитивный тип JSON `number` представляется в виде числа с плавающей точкой двойной точности IEEE 754 (что RFC 7159 явно признаёт и допускает). При использовании JSON в качестве формата обмена данными с такими системами следует учитывать риски потери точности чисел, хранившихся в Postgres Pro.

И напротив, как показано в таблице, есть некоторые ограничения в формате ввода примитивных типов JSON, не актуальные для соответствующих типов Postgres Pro.

Таблица 8.23. Примитивные типы JSON и соответствующие им типы Postgres Pro

Примитивный тип JSON	Тип Postgres Pro	Примечания
<code>string</code>	<code>text</code>	<code>\u0000</code> не допускается как спецпоследовательность Unicode, представляющая символ, который отсутствует в кодировке базы
<code>number</code>	<code>numeric</code>	Значения NaN и infinity не допускаются
<code>boolean</code>	<code>boolean</code>	Допускаются только варианты <code>true</code> и <code>false</code> (в нижнем регистре)
<code>null</code>	(нет)	NULL в SQL имеет другой смысл

8.14.1. Синтаксис вводимых и выводимых значений JSON

Синтаксис ввода-вывода типов данных JSON соответствует стандарту RFC 7159.

Примеры допустимых выражений с типом `json` (или `jsonb`):

```
-- Простое скалярное/примитивное значение
-- Простыми значениями могут быть числа, строки в кавычках, true, false или null
SELECT '5'::json;

-- Массив из нуля и более элементов (элементы могут быть разных типов)
SELECT '[1, 2, "foo", null]'::json;

-- Объект, содержащий пары ключей и значений
-- Заметьте, что ключи объектов — это всегда строки в кавычках
SELECT '{"bar": "baz", "balance": 7.77, "active": false}'::json;

-- Массивы и объекты могут вкладываться произвольным образом
SELECT '{"foo": [true, "bar"], "tags": {"a": 1, "b": null}}'::json;
```

Как было сказано ранее, когда значение JSON вводится и затем выводится без дополнительной обработки, тип `json` выводит тот же текст, что поступил на вход, а `jsonb` не сохраняет семантически незначимые детали, такие как пробелы. Например, посмотрите на эти различия:

```
SELECT '{"bar": "baz", "balance": 7.77, "active":false}'::json;
           json
-----
{"bar": "baz", "balance": 7.77, "active":false}
(1 row)
```

```
SELECT '{"bar": "baz", "balance": 7.77, "active":false}':::jsonb;
           jsonb
```

```
-----
{"bar": "baz", "active": false, "balance": 7.77}
(1 row)
```

Первая семантически незначимая деталь, заслуживающая внимания: с `jsonb` числа выводятся по правилам нижележащего типа `numeric`. На практике это означает, что числа, заданные в записи с `E`, будут выведены без неё, например:

```
SELECT '{"reading": 1.230e-5}':::json, '{"reading": 1.230e-5}':::jsonb;
           json           |           jsonb
-----+-----
{"reading": 1.230e-5} | {"reading": 0.00001230}
(1 row)
```

Однако как видно из этого примера, `jsonb` сохраняет конечные нули дробного числа, хотя они и не имеют семантической значимости, в частности для проверки на равенство.

Список встроенных функций и операторов, позволяющих создавать и обрабатывать значения JSON, приведён в [Разделе 9.16](#).

8.14.2. Проектирование документов JSON

Представлять данные в JSON можно гораздо более гибко, чем в традиционной реляционной модели данных, что очень привлекательно там, где нет жёстких условий. И оба этих подхода вполне могут сосуществовать и дополнять друг друга в одном приложении. Однако даже для приложений, которым нужна максимальная гибкость, рекомендуется, чтобы документы JSON имели некоторую фиксированную структуру. Эта структура обычно не навязывается жёстко (хотя можно декларативно диктовать некоторые бизнес-правила), но когда она предсказуема, становится гораздо проще писать запросы, которые извлекают полезные данные из набора «документов» (информации) в таблице.

Данные JSON, как и данные любых других типов, хранящиеся в таблицах, находятся под контролем механизма параллельного доступа. Хотя хранить большие документы вполне возможно, не забывайте, что при любом изменении устанавливается блокировка всей строки (на уровне строки). Поэтому для оптимизации блокировок транзакций, изменяющих данные, стоит ограничить размер документов JSON разумными пределами. В идеале каждый документ JSON должен собой представлять атомарный информационный блок, который, согласно бизнес-логике, нельзя разделить на меньшие, индивидуально изменяемые блоки.

8.14.3. Проверки на вхождение и существование `jsonb`

Проверка *вхождения* — важная особенность типа `jsonb`, не имеющая аналога для типа `json`. Эта проверка определяет, входит ли один документ `jsonb` в другой. В следующих примерах возвращается истинное значение (кроме упомянутых исключений):

```
-- Простые скалярные/примитивные значения включают только одно идентичное значение:
SELECT '"foo"':::jsonb @> '"foo"':::jsonb;
```

```
-- Массив с правой стороны входит в массив слева:
SELECT '[1, 2, 3]':::jsonb @> '[1, 3]':::jsonb;
```

```
-- Порядок элементов в массиве не важен, поэтому это условие тоже выполняется:
SELECT '[1, 2, 3]':::jsonb @> '[3, 1]':::jsonb;
```

```
-- А повторяющиеся элементы массива не имеют значения:
SELECT '[1, 2, 3]':::jsonb @> '[1, 2, 2]':::jsonb;
```

```
-- Объект с одной парой справа входит в объект слева:
```

```
SELECT '{"product": "PostgreSQL", "version": 9.4, "jsonb": true}'::jsonb @>
 '{"version": 9.4}'::jsonb;
```

-- Массив справа **не** считается входящим в

-- массив слева, хотя в последний и вложен подобный массив:

```
SELECT '[1, 2, [1, 3]]'::jsonb @> '[1, 3]'::jsonb; -- выдаёт false
```

-- Но если добавить уровень вложенности, проверка на вхождение выполняется:

```
SELECT '[1, 2, [1, 3]]'::jsonb @> '[[1, 3]]'::jsonb;
```

-- Аналогично, это вхождением не считается:

```
SELECT '{"foo": {"bar": "baz"}}'::jsonb @> '{"bar": "baz"}'::jsonb; -- выдаёт false
```

-- Ключ с пустым объектом на верхнем уровне входит в объект с таким ключом:

```
SELECT '{"foo": {"bar": "baz"}}'::jsonb @> '{"foo": {}}'::jsonb;
```

Общий принцип этой проверки в том, что входящий объект должен соответствовать объекту, содержащему его, по структуре и данным, возможно, после исключения из содержащего объекта лишних элементов массива или пар ключ/значение. Но помните, что порядок элементов массива для проверки на вхождение не имеет значения, а повторяющиеся элементы массива считаются только один раз.

В качестве особого исключения для требования идентичности структур, массив может содержать примитивное значение:

-- В этот массив входит примитивное строковое значение:

```
SELECT '["foo", "bar"]'::jsonb @> '"bar"'::jsonb;
```

-- Это исключение действует только в одну сторону -- здесь вхождения нет:

```
SELECT '"bar"'::jsonb @> '["bar"]'::jsonb; -- выдаёт false
```

Для типа jsonb введён также оператор *существования*, который является вариацией на тему вхождения: он проверяет, является ли строка (заданная в виде значения text) ключом объекта или элементом массива на верхнем уровне значения jsonb. В следующих примерах возвращается истинное значение (кроме упомянутых исключений):

-- Строка существует в качестве элемента массива:

```
SELECT '["foo", "bar", "baz"]'::jsonb ? 'bar';
```

-- Строка существует в качестве ключа объекта:

```
SELECT '{"foo": "bar"}'::jsonb ? 'foo';
```

-- Значения объектов не рассматриваются:

```
SELECT '{"foo": "bar"}'::jsonb ? 'bar'; -- выдаёт false
```

-- Как и вхождение, существование определяется на верхнем уровне:

```
SELECT '{"foo": {"bar": "baz"}}'::jsonb ? 'bar'; -- выдаёт false
```

-- Строка считается существующей, если она соответствует примитивной строке JSON:

```
SELECT '"foo"'::jsonb ? 'foo';
```

Объекты JSON для проверок на существование и вхождение со множеством ключей или элементов подходят больше, чем массивы, так как, в отличие от массивов, они внутри оптимизируются для поиска, и поиск элемента не будет линейным.

Подсказка

Так как вхождение в JSON проверяется с учётом вложенности, правильно написанный запрос может заменить явную выборку внутренних объектов. Например, предположим, что у нас есть столбец doc, содержащий объекты на верхнем уровне, и большинство этих объек-

тов содержит поля `tags` с массивами вложенных объектов. Данный запрос найдёт записи, в которых вложенные объекты содержат ключи `"term": "paris"` и `"term": "food"`, и при этом пропустит такие ключи, находящиеся вне массива `tags`:

```
SELECT doc->'site_name' FROM websites
WHERE doc @> '{"tags":[{"term":"paris"}, {"term":"food"}]}';
```

Этого же результата можно добиться, например, так:

```
SELECT doc->'site_name' FROM websites
WHERE doc->'tags' @> '[{"term":"paris"}, {"term":"food"}]';
```

Но данный подход менее гибкий и часто также менее эффективный.

С другой стороны, оператор существования JSON не учитывает вложенность: он будет искать заданный ключ или элемент массива только на верхнем уровне значения JSON.

Различные операторы вхождения и существования, а также все другие операторы и функции для работы с JSON документированы в [Разделе 9.16](#).

8.14.4. Индексация `jsonb`

Для эффективного поиска ключей или пар ключ/значение в большом количестве документов `jsonb` можно успешно применять индексы GIN. Для этого предоставляются два «класса операторов» GIN, предлагающие выбор между производительностью и гибкостью.

Класс операторов GIN по умолчанию для `jsonb` поддерживает запросы с операторами существования ключа (`?`, `?|` и `?&`), оператором включения (`@>`) и операторами соответствия для `jsonpath` (`@?` и `@@`). (Подробнее семантика, реализуемая этими операторами, описана в [Таблице 9.46](#).) Пример создания индекса с этим классом операторов:

```
CREATE INDEX idxgin ON api USING GIN (jdoc);
```

Дополнительный класс операторов GIN `jsonb_path_ops` не поддерживает операторы существования ключа, но поддерживает `@>`, `@?` и `@@`. Пример создания индекса с этим классом операторов:

```
CREATE INDEX idxginp ON api USING GIN (jdoc jsonb_path_ops);
```

Рассмотрим пример таблицы, в которой хранятся документы JSON, получаемые от сторонней веб-службы, с документированным определением схемы. Типичный документ:

```
{
  "guid": "9c36adc1-7fb5-4d5b-83b4-90356a46061a",
  "name": "Angela Barton",
  "is_active": true,
  "company": "MagnaFone",
  "address": "178 Howard Place, Gulf, Washington, 702",
  "registered": "2009-11-07T08:53:22 +08:00",
  "latitude": 19.793713,
  "longitude": 86.513373,
  "tags": [
    "enim",
    "aliquip",
    "qui"
  ]
}
```

Мы сохраняем эти документы в таблице `api`, в столбце `jdoc` типа `jsonb`. Если по этому столбцу создаётся GIN-индекс, он может применяться в подобных запросах:

```
-- Найти документы, в которых ключ "company" имеет значение "MagnaFone"
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @> '{"company": "MagnaFone"}';
```


Однако в следующих запросах он не будет использоваться, потому что, несмотря на то, что оператор `?` — индексируемый, он применяется не к индексированному столбцу `jdoc` непосредственно:

```
-- Найти документы, в которых ключ "tags" содержит ключ или элемент массива "qui"
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc -> 'tags' ? 'qui';
```

И всё же, правильно применяя индексы выражений, в этом запросе можно задействовать индекс. Если запрос определённых элементов в ключе `"tags"` выполняется часто, вероятно стоит определить такой индекс:

```
CREATE INDEX idxgintags ON api USING GIN ((jdoc -> 'tags'));
```

Теперь предложение `WHERE jdoc -> 'tags' ? 'qui'` будет выполняться как применение индексируемого оператора `?` к индексируемому выражению `jdoc -> 'tags'`. (Подробнее об индексах выражений можно узнать в [Разделе 11.7.](#))

Ещё один подход к использованию проверок на существование:

```
-- Найти документы, в которых ключ "tags" содержит элемент массива "qui"
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @> '{"tags": ["qui"]}';
```

Этот запрос может задействовать простой GIN-индекс по столбцу `jdoc`. Но заметьте, что такой индекс будет хранить копии всех ключей и значений в поле `jdoc`, тогда как индекс выражения из предыдущего примера хранит только данные внутри объекта с ключом `tags`. Хотя подход с простым индексом гораздо более гибкий (так как он поддерживает запросы по любому ключу), индексы конкретных выражений скорее всего будут меньше и быстрее, чем простые индексы.

Также индексы GIN поддерживают операторы `@?` и `@@`, которые сопоставляют `jsonpath` с данными. Примеры:

```
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @? '$.tags[*] ? (@ == "qui")';
SELECT jdoc->'guid', jdoc->'name' FROM api WHERE jdoc @@ '$.tags[*] == "qui";
```

Для этих операторов индекс GIN извлекает из `jsonpath` предложения вида *цепочка_обращения = константа* и выполняет поиск по ключам и значениям из этих предложений. Цепочка обращения может включать указания обращения *.ключ*, *[*]* и *[индекс]*. Класс операторов `jsonb_ops` также поддерживает указания *.** и *.***, которые не поддерживает класс операторов `jsonb_path_ops`.

Класс операторов `jsonb_path_ops` поддерживает только запросы с операторами `@>`, `@?` и `@@`, но он значительно производительнее класса по умолчанию `jsonb_ops`. Индекс `jsonb_path_ops` обычно гораздо меньше индекса `jsonb_ops` для тех же данных и более точен при поиске, особенно если запросы обращаются к ключам, часто встречающимся в данных. Таким образом, с ним операции поиска выполняются гораздо эффективнее, чем с классом операторов по умолчанию.

Техническое различие между GIN-индексами `jsonb_ops` и `jsonb_path_ops` состоит в том, что для первых создаются независимые элементы индекса для каждого ключа/значения в данных, тогда как для вторых создаются элементы только для значений.¹ По сути, каждый элемент индекса `jsonb_path_ops` представляет собой хеш значения и ключа(ей), приводящего к нему; например, при индексации `{"foo": {"bar": "baz"}}` будет создан один элемент индекса с хешем, рассчитанным по всем трём значениям: `foo`, `bar` и `baz`. Таким образом, проверка на вхождение этой структуры будет использовать крайне точный поиск по индексу, но определить, является ли `foo` ключом, с помощью такого индекса нельзя. С другой стороны, индекс `jsonb_ops` создаст три отдельных элемента индекса, представляющих `foo`, `bar` и `baz` по отдельности; для выполнения проверки на вхождение будут проверены строки таблицы, содержащие все эти три значения. Хотя GIN-индексы позволяют вычислить AND довольно эффективно, такой поиск всё же будет менее точным и более медленным, чем равнозначный поиск с `jsonb_path_ops`, особенно если любое одно из этих трёх значений содержится в большом количестве строк.

Недостаток класса `jsonb_path_ops` заключается в том, что он не учитывает в индексе структуры JSON, не содержащие никаких значений `{"a": {}}`. Для поиска по документам, содержащим такие

¹Поэтому понятие «значение» включает и элементы массивов, хотя в терминологии JSON иногда элементы массивов считаются отличными от значений внутри объектов.

структуры, потребуется выполнить полное сканирование индекса, что довольно долго, поэтому `jsonb_path_ops` не очень подходит для приложений, часто выполняющих такие запросы.

Тип `jsonb` также поддерживает индексы `btree` и `hash`. Они полезны, только если требуется проверять равенство JSON-документов в целом. Порядок сортировки `btree` для типа `jsonb` редко имеет большое значение, но для полноты он приводится ниже:

Объект > Массив > Логическое значение > Число > Строка > null

Объект с n парами > Объект с n - 1 парами

Массив с n элементами > Массив с n - 1 элементами

за исключением того, что (по историческим причинам) пустой массив верхнего уровня сортируется как меньший, чем `null`. Объекты с равным количеством пар сравниваются в таком порядке:

ключ-1, значение-1, ключ-2 ...

Заметьте, что ключи объектов сравниваются согласно порядку при хранении; в частности, из-за того, что короткие ключи хранятся перед длинными, результаты могут оказаться несколько не интуитивными:

```
{ "aa": 1, "c": 1 } > { "b": 1, "d": 1 }
```

Массивы с равным числом элементов упорядочиваются аналогично:

элемент-1, элемент-2 ...

Примитивные значения JSON сравниваются по тем же правилам сравнения, что и нижележащие типы данных Postgres Pro. Строки сравниваются с учётом порядка сортировки по умолчанию в текущей базе данных.

8.14.5. Обращение по индексу к элементам `jsonb`

Тип данных `jsonb` поддерживает извлечение и изменение элементов в стиле обращения к элементам массива. Указывать на вложенные значения можно, задавая цепочку обращений к элементам, при этом будут действовать правила использования аргумента `path` функции `jsonb_set`. Если значение `jsonb` является массивом, числовые индексы начинаются с нуля, а отрицательные целые числа отсчитывают элементы с конца массива к началу. Обращения к срезам массивов не поддерживаются. Результат обращения по индексу всегда имеет тип `jsonb`.

Используя операцию обращения по индексу в предложении `SET` оператора `UPDATE`, можно изменять значения `jsonb`. Путь такого обращения должен быть «проходимым» для всех указанных значений, если они существуют. Например, путь `val['a']['b']['c']` можно пройти полностью до `c`, если `val`, `val['a']` и `val['a']['b']` — объекты. Если же значение `val['a']` или `val['a']['b']` не определено, будет создан пустой объект, заполняемый по мере необходимости. Однако, если значение собственно `val` или любое из промежуточных значений существует и является не объектом, а, например, строкой, числом или сущностью `jsonb null`, пройти этот путь невозможно, поэтому возникает ошибка и транзакция прерывается.

Пример синтаксиса обращения по индексу:

```
-- Извлечь значение объекта по ключу
SELECT ('{"a": 1}'::jsonb)['a'];

-- Извлечь значение вложенного объекта по пути ключа
SELECT ('{"a": {"b": {"c": 1}}}'::jsonb)['a']['b']['c'];

-- Извлечь элемент массива по индексу
SELECT ('[1, "2", null]'::jsonb)[1];

-- Изменить значение объекта по ключу. Обратите внимание на апострофы вокруг '1':
  присваиваемое
-- значение также должно быть типа jsonb
```

```
UPDATE table_name SET jsonb_field['key'] = '1';

-- Это вызовет ошибку, если jsonb_field['a']['b'] в какой-либо записи является не
  объектом.
-- Например, в {"a": 1} ключу 'a' соответствует числовое значение
-- ключа 'a'.
UPDATE table_name SET jsonb_field['a']['b']['c'] = '1';

-- Отфильтровать записи предложением WHERE с обращением по ключу. Поскольку результат
-- обращения по ключу будет иметь тип jsonb, такой же тип должно иметь сравниваемое с
  ним значение.
-- Двойные кавычки добавлены, чтобы строка "value" стала допустимой строкой jsonb.
SELECT * FROM table_name WHERE jsonb_field['key'] = '"value"';
```

Присваивание jsonb при обращении по индексу в некоторых особых случаях работает не так, как с функцией jsonb_set. Когда исходное значение jsonb — NULL, присваивание при обращении по ключу будет работать, как будто это значение — пустое значение JSON (типа массив или объект, в зависимости от типа ключа):

```
-- Там, где поле jsonb_field было NULL, оно станет {"a": 1}
UPDATE table_name SET jsonb_field['a'] = '1';

-- Там, где поле jsonb_field было NULL, оно станет [1]
UPDATE table_name SET jsonb_field[0] = '1';
```

Если индекс указан для массива, содержащего недостаточно элементов, в него будут добавляться элементы со значением NULL до тех пор, пока индекс не станет достижимым и пока не станет возможным задать значение.

```
-- Там, где поле jsonb_field было [], оно станет [null, null, 2];
-- там, где поле jsonb_field было [0], оно станет [0, null, 2]
UPDATE table_name SET jsonb_field[2] = '2';
```

Значение jsonb будет принимать присваивания по несуществующим путям, если последний существующий элемент, который нужно пройти, — объект или массив, в зависимости от соответствующего ключа (элемент, на который указывает последний компонент пути, не проходится и может быть любым). Вложенные структуры массивов и объектов будут созданы и для первых дополнятся элементами null до нужной позиции (заданной в пути), в которую может быть помещено присваиваемое значение.

```
-- Там, где поле jsonb_field было {}, оно станет {"a": [{"b": 1}]}
UPDATE table_name SET jsonb_field['a'][0]['b'] = '1';

-- Там, где поле jsonb_field было [], оно станет [null, {"a": 1}]
UPDATE table_name SET jsonb_field[1]['a'] = '1';
```

8.14.6. Трансформации

Для различных процедурных языков представлены дополнительные расширения, реализующие трансформации для типа jsonb.

Расширения для PL/Perl называются jsonb_plperl и jsonb_plperlu. Когда они используются, значения jsonb отображаются в соответствующие структуры Perl: массивы, хеши или скаляры.

Расширение для PL/Python называется jsonb_plpython3u. Когда оно используется, значения jsonb отображаются в соответствующие структуры Python: словари, списки или скаляры.

Из этих расширений «доверенным» считается jsonb_plperl, то есть его могут устанавливать обычные пользователи, имеющие право CREATE в текущей базе данных. Остальные расширения могут устанавливать только суперпользователи.

8.14.7. Тип jsonpath

Тип `jsonpath` предназначен для реализации поддержки языка путей SQL/JSON в Postgres Pro, позволяющего эффективно выполнять запросы к данным JSON. Он обеспечивает двоичное представление разобранного выражения пути SQL/JSON, определяющего, какие элементы должны извлекаться из данных JSON для дальнейшей обработки в функциях SQL/JSON.

Семантика предикатов и операторов языка путей SQL/JSON в целом соответствует SQL. В то же время, чтобы с данными JSON можно было оперировать естественным образом, в синтаксисе путей SQL/JSON приняты некоторые соглашения JavaScript:

- Точка (.) применяется для доступа к члену объекта.
- Квадратные скобки ([]) применяются для обращения к массиву.
- Элементы массивов в SQL/JSON нумеруются с 0, тогда как обычные массивы SQL — с 1.

Выражение пути SQL/JSON обычно записывается в SQL-запросе в виде символьной константы SQL, и поэтому должно заключаться в апострофы, а любой апостроф, который нужно заключить в это значение, должен дублироваться (см. [Подраздел 4.1.2.1](#)). Нередко строковые константы требуется использовать и внутри выражений путей. На такие константы распространяются соглашения JavaScript/ECMAScript: они должны заключаться в двойные кавычки, а для представления символов, которые сложно ввести иначе, используются спецпоследовательности с обратной косой чертой. В частности, символ двойных кавычек внутри строковой константы записывается как `\`, а собственно обратная косая черта как `\\`. В число других спецпоследовательностей, воспринимаемых в строках JavaScript, входят: `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, выражающие различные управляющие символы ASCII, `\xNN`, выражающая символ кодом в виде только двух шестнадцатеричных цифр, `\uNNNN`, выражающая символ Unicode кодом в виде четырёх шестнадцатеричных цифр, а также `\u{N...}`, позволяющая для записи кода символа Unicode использовать от 1 до 6 шестнадцатеричных цифр.

Выражение пути состоит из последовательности элементов пути, которые могут быть следующими:

- Константы примитивных типов JSON: текст Unicode, числа и значения `true`, `false` и `null`.
- Переменные пути перечислены в [Таблице 8.24](#).
- Операторы обращения перечислены в [Таблице 8.25](#).
- Операторы и методы `jsonpath` перечислены в [Подразделе 9.16.2.2](#).
- Скобки, применяющиеся для образования выражений фильтра и изменения порядка вычисления пути.

Более подробно использование выражений `jsonpath` с функциями запросов SQL/JSON описано в [Подразделе 9.16.2](#).

Таблица 8.24. Переменные `jsonpath`

Переменная	Описание
\$	Переменная, представляющая значение JSON, фигурирующее в запросе (<i>элемент контекста</i>).
\$varname	Именованная переменная. Её значение может быть задано в параметре <code>vars</code> , который принимают различные функции обработки JSON; подробности в Таблице 9.48 .
@	Переменная, представляющая результат вычисления пути в выражениях фильтров.

Таблица 8.25. Операторы обращения в `jsonpath`

Оператор обращения	Описание
.key	Оператор обращения к члену объекта, выбираемому по заданному ключу. Если имя ключа совпадает с именем какой-либо переменной, начинающимся с \$, или не соответствует действию-
."\$имя_переменной"	

Оператор обращения	Описание
	ющим в JavaScript требованиям к идентификаторам, оно должно заключаться в двойные кавычки и таким образом представляться как строковая константа.
<code>.*</code>	Оператор обращения по звёздочке, который возвращает значения всех членов, находящихся на верхнем уровне объекта.
<code>.**</code>	Рекурсивный оператор обращения по звёздочке, который проходит по всем уровням иерархии JSON текущего объекта и возвращает все значения членов, вне зависимости от их уровня вложенности. Это реализованное в Postgres Pro расширение стандарта SQL/JSON.
<code>.**{уровень}</code> <code>.**{начальный_уровень to конечный_уровень }</code>	Этот оператор подобен <code>.**</code> , но выбирает только указанные уровни иерархии JSON. Уровни вложенности задаются целыми числами, при этом нулевой уровень соответствует текущему объекту. Для обращения к самому нижнему уровню вложенности можно использовать ключевое слово <code>last</code> . Это реализованное в Postgres Pro расширение стандарта SQL/JSON.
<code>[селектор, ...]</code>	Оператор обращения к элементу массива. <i>Селектор</i> может задаваться в двух формах: <i>индекс</i> или <i>начальный_индекс to конечный_индекс</i> . Первая форма выбирает единственный элемент по индексу. Вторая форма выбирает срез массива по двум индексам, включающий крайние элементы, соответствующие значениям <i>начальный_индекс</i> и <i>конечный_индекс</i> . Задаваемый <i>индекс</i> может быть целочисленным значением или выражением, возвращающим единственное число, которое автоматически приводится к целому. Индекс 0 соответствует первому элементу массива. Также в качестве индекса принимается ключевое слово <code>last</code> , обозначающее индекс последнего элемента массива, что полезно при обработке массивов неизвестной длины.
<code>[*]</code>	Оператор обращения к элементам массива по звёздочке, возвращающий все элементы массива.

8.15. Массивы

Postgres Pro позволяет определять столбцы таблицы как многомерные массивы переменной длины. Элементами массивов могут быть любые встроенные или определённые пользователями базовые типы, перечисления, составные типы, типы-диапазоны или домены.

8.15.1. Объявления типов массивов

Чтобы проиллюстрировать использование массивов, мы создадим такую таблицу:

```
CREATE TABLE sal_emp (
    name            text,
    pay_by_quarter  integer[],
    schedule        text[][]
);
```

Как показано, для объявления типа массива к названию типа элементов добавляются квадратные скобки (`[]`). Показанная выше команда создаст таблицу `sal_emp` со столбцами типов `text` (`name`), одномерный массив с элементами `integer` (`pay_by_quarter`), представляющий квартальную зарплату работников, и двумерный массив с элементами `text` (`schedule`), представляющий недельный график работника.

Команда `CREATE TABLE` позволяет также указать точный размер массивов, например так:

```
CREATE TABLE tictactoe (
    squares    integer[3][3]
);
```

Однако текущая реализация игнорирует все указанные размеры, т. е. фактически размер массива остаётся неопределённым.

Текущая реализация также не ограничивает число размерностей. Все элементы массивов считаются одного типа, вне зависимости от его размера и числа размерностей. Поэтому явно указывать число элементов или размерностей в команде `CREATE TABLE` имеет смысл только для документирования, на механизм работы с массивом это не влияет.

Для объявления одномерных массивов можно применять альтернативную запись с ключевым словом `ARRAY`, соответствующую стандарту SQL. Столбец `pay_by_quarter` можно было бы определить так:

```
pay_by_quarter    integer ARRAY[4],
```

Или без указания размера массива:

```
pay_by_quarter    integer ARRAY,
```

Заметьте, что и в этом случае Postgres Pro не накладывает ограничения на фактический размер массива.

8.15.2. Ввод значения массива

Чтобы записать значение массива в виде буквальной константы, заключите значения элементов в фигурные скобки и разделите их запятыми. (Если вам знаком C, вы найдёте, что это похоже на синтаксис инициализации структур в C.) Вы можете заключить значение любого элемента в двойные кавычки, а если он содержит запятые или фигурные скобки, это обязательно нужно сделать. (Подробнее это описано ниже.) Таким образом, общий формат константы массива выглядит так:

```
'{ значение1 разделитель значение2 разделитель ... }'
```

где *разделитель* — символ, указанный в качестве разделителя в соответствующей записи в таблице `pg_type`. Для стандартных типов данных, существующих в дистрибутиве Postgres Pro, разделителем является запятая (,), за исключением лишь типа `box`, в котором разделитель — точка с запятой (;). Каждое *значение* здесь — это либо константа типа элемента массива, либо вложенный массив. Например, константа массива может быть такой:

```
'{{1,2,3},{4,5,6},{7,8,9}}'
```

Эта константа определяет двухмерный массив 3x3, состоящий из трёх вложенных массивов целых чисел.

Чтобы присвоить элементу массива значение `NULL`, достаточно просто написать `NULL` (регистр символов при этом не имеет значения). Если же требуется добавить в массив строку, содержащую «NULL», это слово нужно заключить в двойные кавычки.

(Такого рода константы массивов на самом деле представляют собой всего лишь частный случай констант, описанных в [Подразделе 4.1.2.7](#). Константа изначально воспринимается как строка и передаётся процедуре преобразования вводимого массива. При этом может потребоваться явно указать целевой тип.)

Теперь мы можем показать несколько операторов `INSERT`:

```
INSERT INTO sal_emp
VALUES ('Bill',
    '{10000, 10000, 10000, 10000}',
    '{{"meeting", "lunch"}, {"training", "presentation"}}');

INSERT INTO sal_emp
VALUES ('Carol',
```

```
{20000, 25000, 25000, 25000}',
{{"breakfast", "consulting"}, {"meeting", "lunch"}}}');
```

Результат двух предыдущих команд:

```
SELECT * FROM sal_emp;
name |      pay_by_quarter      |      schedule
-----+-----+-----
Bill | {10000,10000,10000,10000}| {{meeting,lunch},{training,presentation}}
Carol| {20000,25000,25000,25000}| {{breakfast,consulting},{meeting,lunch}}
(2 rows)
```

В многомерных массивов число элементов в каждой размерности должно быть одинаковым; в противном случае возникает ошибка. Например:

```
INSERT INTO sal_emp
VALUES ('Bill',
       '{10000, 10000, 10000, 10000}',
       '{{"meeting", "lunch"}, {"meeting"}}}');
```

ОШИБКА: для многомерных массивов должны задаваться выражения с соответствующими размерностями

Также можно использовать синтаксис конструктора ARRAY:

```
INSERT INTO sal_emp
VALUES ('Bill',
       ARRAY[10000, 10000, 10000, 10000],
       ARRAY[['meeting', 'lunch'], ['training', 'presentation']]);
```

```
INSERT INTO sal_emp
VALUES ('Carol',
       ARRAY[20000, 25000, 25000, 25000],
       ARRAY[['breakfast', 'consulting'], ['meeting', 'lunch']]);
```

Заметьте, что элементы массива здесь — это простые SQL-константы или выражения; и поэтому, например строки будут заключаться в одинарные апострофы, а не в двойные, как в буквальной константе массива. Более подробно конструктор ARRAY обсуждается в [Подразделе 4.2.12](#).

8.15.3. Обращение к массивам

Добавив данные в таблицу, мы можем перейти к выборкам. Сначала мы покажем, как получить один элемент массива. Этот запрос получает имена сотрудников, зарплата которых изменилась во втором квартале:

```
SELECT name FROM sal_emp WHERE pay_by_quarter[1] <> pay_by_quarter[2];
```

```
name
-----
Carol
(1 row)
```

Индексы элементов массива записываются в квадратных скобках. По умолчанию в Postgres Pro действует соглашение о нумерации элементов массива с 1, то есть в массиве из n элементов первым считается `array[1]`, а последним — `array[n]`.

Этот запрос выдаёт зарплату всех сотрудников в третьем квартале:

```
SELECT pay_by_quarter[3] FROM sal_emp;
```

```
pay_by_quarter
-----
10000
25000
```

(2 rows)

Мы также можем получать обычные прямоугольные срезы массива, то есть подмассивы. Срез массива обозначается как *нижняя-граница:верхняя-граница* для одной или нескольких размерностей. Например, этот запрос получает первые пункты в графике Билла в первые два дня недели:

```
SELECT schedule[1:2][1:1] FROM sal_emp WHERE name = 'Bill';
```

```

      schedule
-----
{{meeting},{training}}
(1 row)

```

Если одна из размерностей записана в виде среза, то есть содержит двоеточие, тогда срез распространяется на все размерности. Если при этом для размерности указывается только одно число (без двоеточия), в срез войдут элемент от 1 до заданного номера. Например, в этом примере [2] будет равнозначно [1:2]:

```
SELECT schedule[1:2][2] FROM sal_emp WHERE name = 'Bill';
```

```

      schedule
-----
{{meeting,lunch},{training,presentation}}
(1 row)

```

Во избежание путаницы с обращением к одному элементу, срезы лучше всегда записывать явно для всех измерений, например [1:2][1:1] вместо [2][1:1].

Значения *нижняя-граница* и/или *верхняя-граница* в указании среза можно опустить; опущенная граница заменяется нижним или верхним пределом индексов массива. Например:

```
SELECT schedule[:2][2:] FROM sal_emp WHERE name = 'Bill';
```

```

      schedule
-----
{{lunch},{presentation}}
(1 row)

```

```
SELECT schedule[:] [1:1] FROM sal_emp WHERE name = 'Bill';
```

```

      schedule
-----
{{meeting},{training}}
(1 row)

```

Выражение обращения к элементу массива возвратит NULL, если сам массив или одно из выражений индексов элемента равны NULL. Значение NULL также возвращается, если индекс выходит за границы массива (это не считается ошибкой). Например, если `schedule` в настоящее время имеет размерности [1:3][1:2], результатом обращения к `schedule[3][3]` будет NULL. Подобным образом, при обращении к элементу массива с неправильным числом индексов возвращается NULL, а не ошибка.

Аналогично, NULL возвращается при обращении к срезу массива, если сам массив или одно из выражений, определяющих индексы элементов, равны NULL. Однако в других случаях, например, когда границы среза выходят за рамки массива, возвращается не NULL, а пустой массив (с размерностью 0). (Так сложилось исторически, что в этом срезы отличаются от обращений к обычным элементам.) Если запрошенный срез пересекает границы массива, тогда возвращается не NULL, а срез, сокращённый до области пересечения.

Текущие размеры значения массива можно получить с помощью функции `array_dims`:

```
SELECT array_dims(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
array_dims
-----
[1:2][1:2]
(1 row)
```

`array_dims` выдаёт результат типа `text`, что удобно скорее для людей, чем для программ. Размеры массива также можно получить с помощью функций `array_upper` и `array_lower`, которые возвращают соответственно верхнюю и нижнюю границу для указанной размерности:

```
SELECT array_upper(schedule, 1) FROM sal_emp WHERE name = 'Carol';
```

```
array_upper
-----
                2
(1 row)
```

`array_length` возвращает число элементов в указанной размерности массива:

```
SELECT array_length(schedule, 1) FROM sal_emp WHERE name = 'Carol';
```

```
array_length
-----
                2
(1 row)
```

`cardinality` возвращает общее число элементов массива по всем измерениям. Фактически это число строк, которое вернёт функция `unnest`:

```
SELECT cardinality(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
cardinality
-----
                4
(1 row)
```

8.15.4. Изменение массивов

Значение массива можно заменить полностью так:

```
UPDATE sal_emp SET pay_by_quarter = '{25000,25000,27000,27000}'
WHERE name = 'Carol';
```

или используя синтаксис `ARRAY`:

```
UPDATE sal_emp SET pay_by_quarter = ARRAY[25000,25000,27000,27000]
WHERE name = 'Carol';
```

Также можно изменить один элемент массива:

```
UPDATE sal_emp SET pay_by_quarter[4] = 15000
WHERE name = 'Bill';
```

или срез:

```
UPDATE sal_emp SET pay_by_quarter[1:2] = '{27000,27000}'
WHERE name = 'Carol';
```

При этом в указании среза может быть опущена *нижняя-граница* и/или *верхняя-граница*, но только для массива, отличного от `NULL`, и имеющего ненулевую размерность (иначе неизвестно, какие граничные значения должны подставляться вместо опущенных).

Сохранённый массив можно расширить, определив значения ранее отсутствовавших в нём элементов. При этом все элементы, располагающиеся между существовавшими ранее и новыми, принимают значения `NULL`. Например, если массив `myarray` содержит 4 элемента, после присваивания значения элементу `myarray[6]` его длина будет равна 6, а `myarray[5]` будет содержать `NULL`. В

настоящее время подобное расширение поддерживается только для одномерных, но не многомерных массивов.

Определяя элементы по индексам, можно создавать массивы, в которых нумерация элементов может начинаться не с 1. Например, можно присвоить значение выражению `myarray[-2:7]` и таким образом создать массив, в котором будут элементы с индексами от -2 до 7.

Значения массива также можно сконструировать с помощью оператора конкатенации, `||`:

```
SELECT ARRAY[1,2] || ARRAY[3,4];
?column?
-----
{1,2,3,4}
(1 row)

SELECT ARRAY[5,6] || ARRAY[[1,2],[3,4]];
?column?
-----
{{5,6},{1,2},{3,4}}
(1 row)
```

Оператор конкатенации позволяет вставить один элемент в начало или в конец одномерного массива. Он также может принять два N -мерных массива или массивы размерностей N и $N+1$.

Когда в начало или конец одномерного массива вставляется один элемент, в образованном в результате массиве будет та же нижняя граница, что и в массиве-операнде. Например:

```
SELECT array_dims(1 || '[0:1]={2,3}'::int[]);
array_dims
-----
[0:2]
(1 row)

SELECT array_dims(ARRAY[1,2] || 3);
array_dims
-----
[1:3]
(1 row)
```

Когда складываются два массива одинаковых размерностей, в результате сохраняется нижняя граница внешней размерности левого операнда. Выходной массив включает все элементы левого операнда, после которых добавляются все элементы правого. Например:

```
SELECT array_dims(ARRAY[1,2] || ARRAY[3,4,5]);
array_dims
-----
[1:5]
(1 row)

SELECT array_dims(ARRAY[[1,2],[3,4]] || ARRAY[[5,6],[7,8],[9,0]]);
array_dims
-----
[1:5][1:2]
(1 row)
```

Когда к массиву размерности $N+1$ спереди или сзади добавляется N -мерный массив, он вставляется аналогично тому, как в массив вставляется элемент (это было описано выше). Любой N -мерный массив по сути является элементом во внешней размерности массива, имеющего размерность $N+1$. Например:

```
SELECT array_dims(ARRAY[1,2] || ARRAY[[3,4],[5,6]]);
```

```
array_dims
-----
[1:3][1:2]
(1 row)
```

Массив также можно сконструировать с помощью функций `array_prepend`, `array_append` и `array_cat`. Первые две функции поддерживают только одномерные массивы, а `array_cat` поддерживает и многомерные. Несколько примеров:

```
SELECT array_prepend(1, ARRAY[2,3]);
array_prepend
-----
{1,2,3}
(1 row)
```

```
SELECT array_append(ARRAY[1,2], 3);
array_append
-----
{1,2,3}
(1 row)
```

```
SELECT array_cat(ARRAY[1,2], ARRAY[3,4]);
array_cat
-----
{1,2,3,4}
(1 row)
```

```
SELECT array_cat(ARRAY[[1,2],[3,4]], ARRAY[5,6]);
array_cat
-----
{{1,2},{3,4},{5,6}}
(1 row)
```

```
SELECT array_cat(ARRAY[5,6], ARRAY[[1,2],[3,4]]);
array_cat
-----
{{5,6},{1,2},{3,4}}
```

В простых случаях описанный выше оператор конкатенации предпочтительнее непосредственного вызова этих функций. Однако так как оператор конкатенации перегружен для решения всех трёх задач, возможны ситуации, когда лучше применить одну из этих функций во избежание неоднозначности. Например, рассмотрите:

```
SELECT ARRAY[1, 2] || '{3, 4}'; -- нетипизированная строка воспринимается как массив
?column?
-----
{1,2,3,4}
```

```
SELECT ARRAY[1, 2] || '7'; -- как и эта
ERROR:  malformed array literal: "7"
```

```
SELECT ARRAY[1, 2] || NULL; -- как и буквальный NULL
?column?
-----
{1,2}
(1 row)
```

```
SELECT array_append(ARRAY[1, 2], NULL); -- это могло иметься в виду на самом деле
array_append
```

```
-----
{1, 2, NULL}
```

В показанных примерах анализатор запроса видит целочисленный массив с одной стороны оператора конкатенации и константу неопределённого типа с другой. Согласно своим правилам разрешения типа констант, он полагает, что она имеет тот же тип, что и другой операнд — в данном случае целочисленный массив. Поэтому предполагается, что оператор конкатенации здесь представляет функцию `array_cat`, а не `array_append`. Если это решение оказывается неверным, его можно скорректировать, приведя константу к типу элемента массива; однако может быть лучше явно использовать функцию `array_append`.

8.15.5. Поиск значений в массивах

Чтобы найти значение в массиве, необходимо проверить все его элементы. Это можно сделать вручную, если вы знаете размер массива. Например:

```
SELECT * FROM sal_emp WHERE pay_by_quarter[1] = 10000 OR
                             pay_by_quarter[2] = 10000 OR
                             pay_by_quarter[3] = 10000 OR
                             pay_by_quarter[4] = 10000;
```

Однако с большим массивами этот метод становится утомительным, и к тому же он не работает, когда размер массива неизвестен. Альтернативный подход описан в [Разделе 9.24](#). Показанный выше запрос можно было переписать так:

```
SELECT * FROM sal_emp WHERE 10000 = ANY (pay_by_quarter);
```

А так можно найти в таблице строки, в которых массивы содержат только значения, равные 10000:

```
SELECT * FROM sal_emp WHERE 10000 = ALL (pay_by_quarter);
```

Кроме того, для обращения к элементам массива можно использовать функцию `generate_subscripts`. Например так:

```
SELECT * FROM
  (SELECT pay_by_quarter,
         generate_subscripts(pay_by_quarter, 1) AS s
   FROM sal_emp) AS foo
WHERE pay_by_quarter[s] = 10000;
```

Эта функция описана в [Таблице 9.69](#).

Также искать в массиве значения можно, используя оператор `&&`, который проверяет, перекрывается ли левый операнд с правым. Например:

```
SELECT * FROM sal_emp WHERE pay_by_quarter && ARRAY[10000];
```

Этот и другие операторы для работы с массивами описаны в [Разделе 9.19](#). Он может быть ускорен с помощью подходящего индекса, как описано в [Разделе 11.2](#).

Вы также можете искать определённые значения в массиве, используя функции `array_position` и `array_positions`. Первая функция возвращает позицию первого вхождения значения в массив, а вторая — массив позиций всех его вхождений. Например:

```
SELECT array_position(ARRAY['sun', 'mon', 'tue', 'wed', 'thu', 'fri', 'sat'], 'mon');
array_position
-----
2
(1 row)

SELECT array_positions(ARRAY[1, 4, 3, 1, 3, 4, 2, 1], 1);
array_positions
-----
{1, 4, 8}
```

(1 row)

Подсказка

Массивы — это не множества; необходимость поиска определённых элементов в массиве может быть признаком неудачно сконструированной базы данных. Возможно, вместо массива лучше использовать отдельную таблицу, строки которой будут содержать данные элементов массива. Это может быть лучше и для поиска, и для работы с большим количеством элементов.

8.15.6. Синтаксис вводимых и выводимых значений массива

Внешнее текстовое представление значения массива состоит из записи элементов, интерпретируемых по правилам ввода-вывода для типа элемента массива, и оформления структуры массива. Оформление состоит из фигурных скобок (`{` и `}`), окружающих значение массива, и знаков-разделителей между его элементами. В качестве знака-разделителя обычно используется запятая (`,`), но это может быть и другой символ; он определяется параметром `typdelim` для типа элемента массива. Для стандартных типов данных, существующих в дистрибутиве Postgres Pro, разделителем является запятая (`,`), за исключением лишь типа `box`, в котором разделитель — точка с запятой (`;`). В многомерном массиве у каждой размерности (ряд, плоскость, куб и т. д.) есть свой уровень фигурных скобок, а соседние значения в фигурных скобках на одном уровне должны отделяться разделителями.

Функция вывода массива включает значение элемента в кавычки, если это пустая строка или оно содержит фигурные скобки, знаки-разделители, кавычки, обратную косую черту, пробельный символ или это текст `NULL`. Кавычки и обратная косая черта, включённые в такие значения, преобразуются в спецпоследовательность с обратной косой чертой. Для числовых типов данных можно рассчитывать на то, что значения никогда не будут выводиться в кавычках, но для текстовых типов следует быть готовым к тому, что выводимое значение массива может содержать кавычки.

По умолчанию нижняя граница всех размерностей массива равна одному. Чтобы представить массивы с другими нижними границами, перед содержимым массива можно указать диапазоны индексов. Такое оформление массива будет содержать квадратные скобки (`[]`) вокруг нижней и верхней границ каждой размерности с двоеточием (`:`) между ними. За таким указанием размерности следует знак равно (`=`). Например:

```
SELECT f1[1][-2][3] AS e1, f1[1][-1][5] AS e2
FROM (SELECT '[1:1][-2:-1][3:5]={{{1,2,3},{4,5,6}}}'::int[] AS f1) AS ss;
```

```
e1 | e2
----+----
 1 |  6
(1 row)
```

Процедура вывода массива включает в результат явное указание размерностей, только если нижняя граница в одной или нескольких размерностях отличается от 1.

Если в качестве значения элемента задаётся `NULL` (в любом регистре), этот элемент считается равным непосредственно `NULL`. Если же оно включает кавычки или обратную косую черту, элементу присваивается текстовая строка «`NULL`». Кроме того, для обратной совместимости с версиями PostgreSQL до 8.2, параметр конфигурации [array_nulls](#) можно выключить (присвоив ему `off`), чтобы строки `NULL` не воспринимались как значения `NULL`.

Как было показано ранее, записывая значение массива, любой его элемент можно заключить в кавычки. Это нужно делать, если при разборе значения массива без кавычек возможна неоднозначность. Например, в кавычки необходимо заключать элементы, содержащие фигурные скобки, запятую (или разделитель, определённый для данного типа), кавычки, обратную косую черту, а также пробельные символы в начале или конце строки. Пустые строки и строки, содержащие од-

но слово `NULL`, также нужно заключать в кавычки. Чтобы включить кавычки или обратную косую черту в значение, заключённое в кавычки, добавьте обратную косую черту перед таким символом. С другой стороны, чтобы обойтись без кавычек, таким экранированием можно защитить все символы в данных, которые могут быть восприняты как часть синтаксиса массива.

Перед открывающей и после закрывающей скобки можно добавлять пробельные символы. Пробелы также могут окружать каждую отдельную строку значения. Во всех случаях такие пробельные символы игнорируются. Однако все пробелы в строках, заключённых в кавычки, или окружённые не пробельными символами, напротив, учитываются.

Подсказка

Записывать значения массивов в командах SQL часто бывает удобнее с помощью конструктора `ARRAY` (см. [Подраздел 4.2.12](#)). В `ARRAY` отдельные значения элементов записываются так же, как если бы они не были членами массива.

8.16. Составные типы

Составной тип представляет структуру табличной строки или записи; по сути это просто список имён полей и соответствующих типов данных. Postgres Pro позволяет использовать составные типы во многом так же, как и простые типы. Например, в определении таблицы можно объявить столбец составного типа.

8.16.1. Объявление составных типов

Ниже приведены два простых примера определения составных типов:

```
CREATE TYPE complex AS (
    r      double precision,
    i      double precision
);
```

```
CREATE TYPE inventory_item AS (
    name          text,
    supplier_id   integer,
    price         numeric
);
```

Синтаксис очень похож на `CREATE TABLE`, за исключением того, что он допускает только названия полей и их типы, какие-либо ограничения (такие как `NOT NULL`) в настоящее время не поддерживаются. Заметьте, что ключевое слово `AS` здесь имеет значение; без него система будет считать, что подразумевается другой тип команды `CREATE TYPE`, и выдаст неожиданную синтаксическую ошибку.

Определив такие типы, мы можем использовать их в таблицах:

```
CREATE TABLE on_hand (
    item      inventory_item,
    count     integer
);
```

```
INSERT INTO on_hand VALUES (ROW('fuzzy dice', 42, 1.99), 1000);
```

или функциях:

```
CREATE FUNCTION price_extension(inventory_item, integer) RETURNS numeric
AS 'SELECT $1.price * $2' LANGUAGE SQL;
```

```
SELECT price_extension(item, 10) FROM on_hand;
```

Всякий раз, когда создаётся таблица, вместе с ней автоматически создаётся составной тип. Этот тип представляет тип строки таблицы, и его именем становится имя таблицы. Например, при выполнении команды:

```
CREATE TABLE inventory_item (
    name          text,
    supplier_id   integer REFERENCES suppliers,
    price         numeric CHECK (price > 0)
);
```

в качестве побочного эффекта будет создан составной тип `inventory_item`, в точности соответствующий тому, что был показан выше, и использовать его можно так же. Однако заметьте, что в текущей реализации есть один недостаток: так как с составным типом не могут быть связаны ограничения, то описанные в определении таблицы ограничения *не применяются* к значениям составного типа вне таблицы. (Чтобы обойти этот недостаток, создайте [домен](#) поверх составного типа и добавьте желаемые ограничения в виде ограничений `CHECK` для данного домена.)

8.16.2. Конструирование составных значений

Чтобы записать значение составного типа в виде текстовой константы, его поля нужно заключить в круглые скобки и разделить их запятыми. Значение любого поля можно заключить в кавычки, а если оно содержит запятые или скобки, это делать обязательно. (Подробнее об этом говорится [ниже](#).) Таким образом, в общем виде константа составного типа записывается так:

```
'( значение1 , значение2 , ... )'
```

Например, эта запись:

```
'("fuzzy dice", 42, 1.99)'
```

будет допустимой для описанного выше типа `inventory_item`. Чтобы присвоить `NULL` одному из полей, в соответствующем месте в списке нужно оставить пустое место. Например, эта константа задаёт значение `NULL` для третьего поля:

```
'("fuzzy dice", 42,)'
```

Если же вместо `NULL` требуется вставить пустую строку, нужно записать пару кавычек:

```
'("", 42,)'
```

Здесь в первом поле окажется пустая строка, а в третьем — `NULL`.

(Такого рода константы массивов на самом деле представляют собой всего лишь частный случай констант, описанных в [Подразделе 4.1.2.7](#). Константа изначально воспринимается как строка и передаётся процедуре преобразования составного типа. При этом может потребоваться явно указать тип, к которому будет приведена константа.)

Значения составных типов также можно конструировать, используя синтаксис выражения `ROW`. В большинстве случаев это значительно проще, чем записывать значения в строке, так как при этом не нужно беспокоиться о вложенности кавычек. Мы уже обсуждали этот метод ранее:

```
ROW('fuzzy dice', 42, 1.99)
ROW('', 42, NULL)
```

Ключевое слово `ROW` на самом деле может быть необязательным, если в выражении определяются несколько полей, так что эту запись можно упростить до:

```
('fuzzy dice', 42, 1.99)
('', 42, NULL)
```

Синтаксис выражения `ROW` более подробно рассматривается в [Подразделе 4.2.13](#).

8.16.3. Обращение к составным типам

Чтобы обратиться к полю столбца составного типа, после имени столбца нужно добавить точку и имя поля, подобно тому, как указывается столбец после имени таблицы. На самом деле, эти обращения неотличимы, так что часто бывает необходимо использовать скобки, чтобы команда была

разобрана правильно. Например, можно попытаться выбрать поле столбца из тестовой таблицы `on_hand` таким образом:

```
SELECT item.name FROM on_hand WHERE item.price > 9.99;
```

Но это не будет работать, так как согласно правилам SQL имя `item` здесь воспринимается как имя таблицы, а не столбца в таблице `on_hand`. Поэтому этот запрос нужно переписать так:

```
SELECT (item).name FROM on_hand WHERE (item).price > 9.99;
```

либо указать также и имя таблицы (например, в запросе с многими таблицами), примерно так:

```
SELECT (on_hand.item).name FROM on_hand WHERE (on_hand.item).price > 9.99;
```

В результате объект в скобках будет правильно интерпретирован как ссылка на столбец `item`, из которого выбирается поле.

При выборке поля из значения составного типа также возможны подобные синтаксические казусы. Например, чтобы выбрать одно поле из результата функции, возвращающей составное значение, потребуется написать что-то подобное:

```
SELECT (my_func(...)).field FROM ...
```

Без дополнительных скобок в этом запросе произойдёт синтаксическая ошибка.

Специальное имя поля `*` означает «все поля»; подробнее об этом рассказывается в [Подразделе 8.16.5](#).

8.16.4. Изменение составных типов

Ниже приведены примеры правильных команд добавления и изменения значений составных столбцов. Первые команды иллюстрируют добавление или изменение всего столбца:

```
INSERT INTO mytab (complex_col) VALUES((1.1,2.2));
```

```
UPDATE mytab SET complex_col = ROW(1.1,2.2) WHERE ...;
```

В первом примере опущено ключевое слово `ROW`, а во втором оно есть; присутствовать или отсутствовать оно может в обоих случаях.

Мы можем изменить также отдельное поле составного столбца:

```
UPDATE mytab SET complex_col.r = (complex_col).r + 1 WHERE ...;
```

Заметьте, что при этом не нужно (и на самом деле даже нельзя) заключать в скобки имя столбца, следующее сразу за предложением `SET`, но в ссылке на тот же столбец в выражении, находящемся по правую сторону знака равенства, скобки обязательны.

И мы также можем указать поля в качестве цели команды `INSERT`:

```
INSERT INTO mytab (complex_col.r, complex_col.i) VALUES(1.1, 2.2);
```

Если при этом мы не укажем значения для всех полей столбца, оставшиеся поля будут заполнены значениями `NULL`.

8.16.5. Использование составных типов в запросах

С составными типами в запросах связаны особые правила синтаксиса и поведение. Эти правила образуют полезные конструкции, но они могут быть неочевидными, если не понимать стоящую за ними логику.

В `Postgres Pro` ссылка на имя таблицы (или её псевдоним) в запросе по сути является ссылкой на составное значение текущей строки в этой таблице. Например, имея таблицу `inventory_item`, показанную [выше](#), мы можем написать:

```
SELECT c FROM inventory_item c;
```

Этот запрос выдаёт один столбец с составным значением, и его результат может быть таким:

c

```
-----
("fuzzy dice", 42, 1.99)
(1 row)
```

Заметьте, однако, что простые имена сопоставляются сначала с именами столбцов, и только потом с именами таблиц, так что такой результат получается только потому, что в таблицах запроса не оказалось столбца с именем `c`.

Обычную запись полного имени столбца вида `имя_таблицы.имя_столбца` можно понимать как применение [выбора поля](#) к составному значению текущей строки таблицы. (Из соображений эффективности на самом деле это реализовано по-другому.)

Когда мы пишем

```
SELECT c.* FROM inventory_item c;
```

то, согласно стандарту SQL, мы должны получить содержимое таблицы, развёрнутое в отдельные столбцы:

```
name      | supplier_id | price
-----+-----+-----
fuzzy dice |           42 |   1.99
(1 row)
```

как с запросом

```
SELECT c.name, c.supplier_id, c.price FROM inventory_item c;
```

Postgres Pro применяет такое развёртывание для любых выражений с составными значениями, хотя как показано [выше](#), необходимо заключить в скобки значение, к которому применяется `.*`, если только это не простое имя таблицы. Например, если `myfunc()` — функция, возвращающая составной тип со столбцами `a`, `b` и `c`, то эти два запроса выдадут одинаковый результат:

```
SELECT (myfunc(x)).* FROM some_table;
SELECT (myfunc(x)).a, (myfunc(x)).b, (myfunc(x)).c FROM some_table;
```

Подсказка

Postgres Pro осуществляет развёртывание столбцов фактически переводя первую форму во вторую. Таким образом, в данном примере `myfunc()` будет вызываться три раза для каждой строки и с одним, и с другим синтаксисом. Если это дорогостоящая функция, и вы хотите избежать лишних вызовов, можно использовать такой запрос:

```
SELECT m.* FROM some_table, LATERAL myfunc(x) AS m;
```

Размещение вызова функции в элементе `FROM LATERAL` гарантирует, что она будет вызываться для строки не более одного раза. Конструкция `m.*` так же разворачивается в `m.a`, `m.b`, `m.c`, но теперь эти переменные просто ссылаются на выходные значения `FROM`. (Ключевое слово `LATERAL` здесь является необязательным, но мы добавили его, чтобы подчеркнуть, что функция получает `x` из `some_table`.)

Запись `составное_значение.*` приводит к такому развёртыванию столбцов, когда она фигурирует на верхнем уровне [выходного списка SELECT](#), в [списке RETURNING](#) команд `INSERT/UPDATE/DELETE`, в [предложении VALUES](#) или в [конструкторе строки](#). Во всех других контекстах (включая вложенные в одну из этих конструкций), добавление `.*` к составному значению не меняет это значение, так как это воспринимается как «все столбцы» и поэтому выдаётся то же составное значение. Например, если функция `somefunc()` принимает в качестве аргумента составное значение, эти запросы равносильны:

```
SELECT somefunc(c.*) FROM inventory_item c;
SELECT somefunc(c) FROM inventory_item c;
```


В обоих случаях текущая строка таблицы `inventory_item` передаётся функции как один аргумент с составным значением. И хотя дополнение `.*` в этих случаях не играет роли, использовать его считается хорошим стилем, так как это ясно указывает на использование составного значения. В частности анализатор запроса воспримет `c` в записи `c.*` как ссылку на имя или псевдоним таблицы, а не имя столбца, что избавляет от неоднозначности; тогда как без `.*` неясно, означает ли `c` имя таблицы или имя столбца, и на самом деле при наличии столбца с именем `c` будет выбрано второе прочтение.

Эту концепцию демонстрирует и следующий пример, все запросы в котором действуют одинаково:

```
SELECT * FROM inventory_item c ORDER BY c;
SELECT * FROM inventory_item c ORDER BY c.*;
SELECT * FROM inventory_item c ORDER BY ROW(c.*);
```

Все эти предложения `ORDER BY` обращаются к составному значению строки, вследствие чего строки сортируются по правилам, описанным в [Подразделе 9.24.6](#). Однако если в `inventory_item` содержится столбец с именем `c`, первый запрос будет отличаться от других, так как в нём выполнится сортировка только по данному столбцу. С показанными выше именами столбцов предыдущим запросам также равнозначны следующие:

```
SELECT * FROM inventory_item c ORDER BY ROW(c.name, c.supplier_id, c.price);
SELECT * FROM inventory_item c ORDER BY (c.name, c.supplier_id, c.price);
```

(В последнем случае используется конструктор строки, в котором опущено ключевое слово `ROW`.)

Другая особенность синтаксиса, связанная с составными значениями, состоит в том, что мы можем использовать *функциональную запись* для извлечения поля составного значения. Это легко можно объяснить тем, что записи *поле(таблица)* и *таблица.поле* взаимозаменяемы. Например, следующие запросы равнозначны:

```
SELECT c.name FROM inventory_item c WHERE c.price > 1000;
SELECT name(c) FROM inventory_item c WHERE price(c) > 1000;
```

Более того, если у нас есть функция, принимающая один аргумент составного типа, мы можем вызвать её в любой записи. Все эти запросы равносильны:

```
SELECT somefunc(c) FROM inventory_item c;
SELECT somefunc(c.*) FROM inventory_item c;
SELECT c.somefunc FROM inventory_item c;
```

Эта равнозначность записи с полем и функциональной записи позволяет использовать с составными типами функции, реализующие «вычисляемые поля». При этом приложению, использующему последний из предыдущих запросов, не нужно знать, что фактически `somefunc` — не настоящий столбец таблицы.

Подсказка

Учитывая такое поведение, будет неразумно давать функции, принимающей один аргумент составного типа, то же имя, что и одному из полей данного составного типа. В случае неоднозначности прочтение имени поля будет выбрано при использовании синтаксиса обращения к полю, а прочтение имени функции — если используется синтаксис вызова функции. Однако в PostgreSQL до версии 11 всегда выбиралось прочтение имени поля, если только синтаксис вызова не подталкивал к прочтению имени функции. Чтобы принудительно выбрать прочтение имени функции, в предыдущих версиях надо было дополнить это имя схемой, то есть написать *схема.функция(составное_значение)*.

8.16.6. Синтаксис вводимых и выводимых значений составного типа

Внешнее текстовое представление составного значения состоит из записи элементов, интерпретируемых по правилам ввода-вывода для соответствующих типов полей, и оформления структуры

составного типа. Оформление состоит из круглых скобок ((и)) окружающих всё значение, и запятых (,) между его элементами. Пробельные символы вне скобок игнорируются, но внутри они считаются частью соответствующего элемента и могут учитываться или не учитываться в зависимости от правил преобразования вводимых данных для типа этого элемента. Например, в записи:

```
' ( 42) '
```

пробелы будут игнорироваться, если соответствующее поле имеет целочисленный тип, но не текстовый.

Как было показано ранее, записывая составное значение, любой его элемент можно заключить в кавычки. Это *нужно* делать, если при разборе этого значения без кавычек возможна неоднозначность. Например, в кавычки нужно заключать элементы, содержащие скобки, кавычки, запятую или обратную косую черту. Чтобы включить в поле составного значения, заключённое в кавычки, такие символы, как кавычки или обратная косая черта, перед ними нужно добавить обратную косую черту. (Кроме того, продублированные кавычки в значении поля, заключённого в кавычки, воспринимаются как одинарные, подобно апострофам в строках SQL.) С другой стороны, можно обойтись без кавычек, защитив все символы в данных, которые могут быть восприняты как часть синтаксиса составного значения, с помощью спецпоследовательностей.

Значение NULL в этой записи представляется пустым местом (когда между запятыми или скобками нет никаких символов). Чтобы ввести именно пустую строку, а не NULL, нужно написать "".

Функция вывода составного значения заключает значения полей в кавычки, если они представляют собой пустые строки, либо содержат скобки, запятые, кавычки или обратную косую черту, либо состоят из одних пробелов. (В последнем случае можно обойтись без кавычек, но они добавляются для удобочитаемости.) Кавычки и обратная косая черта, заключённые в значения полей, при выводе дублируются.

Примечание

Помните, что написанная SQL-команда прежде всего интерпретируется как текстовая строка, а затем как составное значение. Вследствие этого число символов обратной косой черты удваивается (если используются спецпоследовательности). Например, чтобы ввести в поле составного столбца значение типа `text` с обратной косой чертой и кавычками, команду нужно будет записать так:

```
INSERT ... VALUES ('\"\"\\\"') ;
```

Сначала обработчик спецпоследовательностей удаляет один уровень обратной косой черты, так что анализатор составного значения получает на вход (\"\"\\\"). В свою очередь, он передаёт эту строку процедуре ввода значения типа `text`, где она преобразуется в \"\". (Если бы мы работали с типом данных, процедура ввода которого также интерпретирует обратную косую черту особым образом, например `bytea`, нам могло бы понадобиться уже восемь таких символов, чтобы сохранить этот символ в поле составного значения.) Во избежание такого дублирования спецсимволов строки можно заключать в доллары (см. [Подраздел 4.1.2.4](#)).

Подсказка

Записывать составные значения в командах SQL часто бывает удобнее с помощью конструктора `ROW`. В `ROW` отдельные значения элементов записываются так же, как если бы они не были членами составного выражения.

8.17. Диапазонные типы

Диапазонные типы представляют диапазоны значений некоторого типа данных (он также называется *подтипом* диапазона). Например, диапазон типа `timestamp` может представлять временной

интервал, когда зарезервирован зал заседаний. В данном случае типом данных будет `tsrange` (сокращение от «timestamp range»), а подтипом — `timestamp`. Подтип должен быть полностью упорядочиваемым, чтобы можно было однозначно определить, где находится значение по отношению к диапазону: внутри, до или после него.

Диапазонные типы полезны тем, что позволяют представить множество возможных значений в одной структуре данных и чётко выразить такие понятия, как пересечение диапазонов. Наиболее очевидный вариант их использования — применять диапазоны даты и времени для составления расписания, но также полезными могут оказаться диапазоны цен, интервалы измерений и т. д.

Для каждого диапазонного типа есть соответствующий мультидиапазонный тип. Мультидиапазон представляет собой упорядоченный список несмежных, непустых и отличных от `NULL` диапазонов. Большинство диапазонных операторов работают и с мультидиапазонами. Кроме того, есть функции для работы именно с мультидиапазонными типами.

8.17.1. Встроенные диапазонные и мультидиапазонные типы

Postgres Pro имеет следующие встроенные диапазонные типы:

- `int4range` — диапазон подтипа `integer`, `int4multirange` — соответствующий мультидиапазон
- `int8range` — диапазон подтипа `bigint`, `int8multirange` — соответствующий мультидиапазон
- `numrange` — диапазон подтипа `numeric`, `nummultirange` — соответствующий мультидиапазон
- `tsrange` — диапазон подтипа `timestamp without time zone`, `tsmultirange` — соответствующий мультидиапазон
- `tstzrange` — диапазон подтипа `timestamp with time zone`, `tstzmultirange` — соответствующий мультидиапазон
- `daterange` — диапазон подтипа `date`, `datemultirange` — соответствующий мультидиапазон

Помимо этого, вы можете определять собственные типы; подробнее это описано в [CREATE TYPE](#).

8.17.2. Примеры

```
CREATE TABLE reservation (room int, during tsrange);
INSERT INTO reservation VALUES
    (1108, '[2010-01-01 14:30, 2010-01-01 15:30)');

-- Вхождение
SELECT int4range(10, 20) @> 3;

-- Перекрытие
SELECT numrange(11.1, 22.2) && numrange(20.0, 30.0);

-- Получение верхней границы
SELECT upper(int8range(15, 25));

-- Вычисление пересечения
SELECT int4range(10, 20) * int4range(15, 25);

-- Является ли диапазон пустым?
SELECT isempty(numrange(1, 5));
```

Полный список операторов и функций, предназначенных для диапазонных типов, приведён в [Таблице 9.58](#) и [Таблице 9.60](#).

8.17.3. Включение и исключение границ

Любой непустой диапазон имеет две границы, верхнюю и нижнюю, и включает все точки между этими значениями. В него также может входить точка, лежащая на границе, если диапазон вклю-

чает эту границу. И наоборот, если диапазон не включает границу, считается, что точка, лежащая на этой границе, в него не входит.

В текстовой записи диапазона включение нижней границы обозначается символом «[», а исключением — символом «(». Для верхней границы включение обозначается аналогично, символом «]», а исключение — символом «)». (Подробнее это описано в [Подразделе 8.17.5.](#))

Для проверки, включается ли нижняя или верхняя граница в диапазон, предназначены функции `lower_inc` и `upper_inc`, соответственно.

8.17.4. Неограниченные (бесконечные) диапазоны

Нижнюю границу диапазона можно опустить и определить тем самым диапазон, включающий все значения, лежащие ниже верхней границы, например: `(, 3]`. Подобным образом, если не определить верхнюю границу, в диапазон войдут все значения, лежащие выше нижней границы. Если же опущена и нижняя, и верхняя границы, такой диапазон будет включать все возможные значения своего подтипа. Указание отсутствующей границы как включаемой в диапазон автоматически преобразуется в исключающее; например, `[, ]` преобразуется в `(, )`. Можно воспринимать отсутствующие значения как плюс/минус бесконечность, но всё же это особые значения диапазонного типа, которые охватывают и возможные для подтипа значения плюс/минус бесконечность.

Для подтипов, в которых есть понятие «бесконечность», `infinity` может использоваться в качестве явного значения границы. При этом, например, в диапазон `[today, infinity)` с подтипом `timestamp` не будет входить специальное значение `infinity` данного подтипа, однако это значение будет входить в диапазон `[today, infinity]`, как и в диапазоны `[today, )` и `[today, ]`.

Проверить, определена ли верхняя или нижняя граница, можно с помощью функций `lower_inf` и `upper_inf`, соответственно.

8.17.5. Ввод-вывод диапазонов

Вводимое значение диапазона должно записываться в одной из следующих форм:

```
(нижняя-граница, верхняя-граница)
(нижняя-граница, верхняя-граница]
[нижняя-граница, верхняя-граница)
[нижняя-граница, верхняя-граница]
empty
```

Тип скобок (квадратные или круглые) определяет, включаются ли в диапазон соответствующие границы, как описано выше. Заметьте, что последняя форма содержит только слово `empty` и определяет пустой диапазон (диапазон, не содержащий точек).

Здесь *нижняя-граница* может быть строкой с допустимым значением подтипа или быть пустой (тогда диапазон будет без нижней границы). Аналогично, *верхняя-граница* может задаваться одним из значений подтипа или быть неопределённой (пустой).

Любое значение границы диапазона можно заключить в кавычки ("). А если значение содержит круглые или квадратные скобки, запятые, кавычки или обратную косую черту, использовать кавычки необходимо, чтобы эти символы не рассматривались как часть синтаксиса диапазона. Чтобы включить в значение границы диапазона, заключённое в кавычки, такие символы, как кавычки или обратная косая черта, перед ними нужно добавить обратную косую черту. (Кроме того, продублированные кавычки в значении диапазона, заключённого в кавычки, воспринимаются как одинарные, подобно апострофам в строках SQL.) С другой стороны, можно обойтись без кавычек, защитив все символы в данных, которые могут быть восприняты как часть синтаксиса диапазона, с помощью спецпоследовательностей. Чтобы задать в качестве границы пустую строку, нужно ввести "", так как пустая строка без кавычек будет означать отсутствие границы.

Пробельные символы до и после определения диапазона игнорируются, но когда они присутствуют внутри скобок, они воспринимаются как часть значения верхней или нижней границы. (Хотя они могут также игнорироваться в зависимости от подтипа диапазона.)

Примечание

Эти правила очень похожи на правила записи значений для полей составных типов. Дополнительные замечания приведены в [Подразделе 8.16.6](#).

Примеры:

```
-- в диапазон включается 3, не включается 7 и включаются все точки между ними
SELECT '[3,7) '::int4range;

-- в диапазон не включаются 3 и 7, но включаются все точки между ними
SELECT '(3,7) '::int4range;

-- в диапазон включается только одно значение 4
SELECT '[4,4] '::int4range;

-- диапазон не включает никаких точек (нормализация заменит его определение на 'empty')
SELECT '[4,4) '::int4range;
```

Вводимое значение мультидиапазона заключается в фигурные скобки ({ и }) и содержит ноль или более диапазонов, разделённых запятыми. До и после скобок и запятых допускаются пробельные символы. Синтаксис сделан похожим на синтаксис массивов, но мультидиапазоны намного проще: они имеют только одну размерность, а их содержимое не нужно заключать в кавычки. (Как показано выше, границы диапазонов могут заключаться в кавычки.)

Примеры:

```
SELECT '{} '::int4multirange;
SELECT '{[3,7)} '::int4multirange;
SELECT '{[3,7), [8,9)} '::int4multirange;
```

8.17.6. Конструирование диапазонов и мультидиапазонов

Для каждого диапазонного типа определена функция конструктора, имеющая то же имя, что и данный тип. Использовать этот конструктор обычно удобнее, чем записывать текстовую константу диапазона, так как это избавляет от потребности в дополнительных кавычках. Функция конструктора может принимать два или три параметра. Вариант с двумя параметрами создаёт диапазон в стандартной форме (нижняя граница включается, верхняя исключается), тогда как для варианта с тремя параметрами включение границ определяется третьим параметром. Третий параметр должен содержать одну из строк: «()», «[]», «[]» или «[]». Например:

```
-- Полная форма: нижняя граница, верхняя граница и текстовая строка, определяющая
-- включение/исключение границ.
SELECT numrange(1.0, 14.0, '()');

-- Если третий аргумент опущен, подразумевается '[]'.
SELECT numrange(1.0, 14.0);

-- Хотя здесь указывается '()', при выводе значение будет приведено к
-- каноническому виду, так как int8range — тип дискретного диапазона (см. ниже).
SELECT int8range(1, 14, '()');

-- Когда вместо любой границы указывается NULL, соответствующей границы у диапазона не
-- будет.
SELECT numrange(NULL, 2.2);
```

Для каждого диапазонного типа также есть конструктор мультидиапазона с тем же именем, что и у мультидиапазонного типа. Функция-конструктор принимает ноль или более аргументов, представляющих собой диапазоны соответствующего типа. Например:

```
SELECT nummultirange();
SELECT nummultirange(numrange(1.0, 14.0));
SELECT nummultirange(numrange(1.0, 14.0), numrange(20.0, 25.0));
```

8.17.7. Типы дискретных диапазонов

Дискретным диапазоном считается диапазон, для подтипа которого однозначно определён «шаг», как, например, для типов `integer` и `date`. Значения этих двух типов можно назвать соседними, когда между ними нет никаких других значений. В непрерывных диапазонах, напротив, всегда (или почти всегда) можно найти ещё одно значение между двумя данными. Например, непрерывным диапазоном будет диапазон с подтипами `numeric` и `timestamp`. (Хотя `timestamp` имеет ограниченную точность, то есть теоретически он является дискретным, но всё же лучше считать его непрерывным, так как шаг его обычно не определён.)

Можно также считать дискретным подтип диапазона, в котором чётко определены понятия «следующего» и «предыдущего» элемента для каждого значения. Такие определения позволяют преобразовывать границы диапазона из включаемых в исключаемые, выбирая следующий или предыдущий элемент вместо заданного значения. Например, диапазоны целочисленного типа `[4, 8]` и `(3, 9)` описывают одно и то же множество значений; но для диапазона подтипа `numeric` это не так.

Для типа дискретного диапазона определяется функция *канонизации*, учитывающая размер шага для данного подтипа. Задача этой функции — преобразовать равнозначные диапазоны к единственному представлению, в частности нормализовать включаемые и исключаемые границы. Если функция канонизации не определена, диапазоны с различным определением будут всегда считаться разными, даже когда они на самом деле представляют одно множество значений.

Для встроенных типов `int4range`, `int8range` и `daterange` каноническое представление включает нижнюю границу и не включает верхнюю; то есть диапазон приводится к виду `[ )`. Однако для нестандартных типов можно использовать и другие соглашения.

8.17.8. Определение новых диапазонных типов

Пользователи могут определять собственные диапазонные типы. Это может быть полезно, когда нужно использовать диапазоны с подтипами, для которых нет встроенных диапазонных типов. Например, можно определить новый тип диапазона для подтипа `float8`:

```
CREATE TYPE floatrange AS RANGE (
    subtype = float8,
    subtype_diff = float8mi
);

SELECT '[1.234, 5.678]'::floatrange;
```

Так как для `float8` осмысленное значение «шага» не определено, функция канонизации в данном примере не задаётся.

Когда вы определяете собственный диапазон, система автоматически создаёт мультидиапазонный тип.

Определяя собственный диапазонный тип, вы также можете выбрать другие правила сортировки или класс оператора B-дерева для его подтипа, что позволит изменить порядок значений, от которого зависит, какие значения попадают в заданный диапазон.

Если подтип можно рассматривать как дискретный, а не непрерывный, в команде `CREATE TYPE` следует также задать функцию канонизации. Этой функции будет передаваться значение диапазона, а она должна вернуть равнозначное значение, но, возможно, с другими границами и форматированием. Для двух диапазонов, представляющих одно множество значений, например, целочисленные диапазоны `[1, 7]` и `[1, 8)`, функция канонизации должна выдавать один результат. Какое именно представление будет считаться каноническим, не имеет значения — главное, чтобы два равнозначных диапазона, отформатированных по-разному, всегда преобразовывались в одно зна-

чение с одинаковым форматированием. Помимо исправления формата включаемых/исключаемых границ, функция канонизации может округлять значения границ, если размер шага превышает точность хранения подтипа. Например, в типе диапазона для подтипа `timestamp` можно определить размер шага, равный часу, тогда функция канонизации должна будет округлить границы, заданные, например с точностью до минут, либо вместо этого выдать ошибку.

Помимо этого, для любого диапазонного типа, ориентированного на использование с индексами GiST или SP-GiST, должна быть определена разница значений подтипов, функция `subtype_diff`. (Индекс сможет работать и без `subtype_diff`, но в большинстве случаев это будет не так эффективно.) Эта функция принимает на вход два значения подтипа и возвращает их разницу (т. е. x минус y) в значении типа `float8`. В показанном выше примере может использоваться функция `float8mi`, определяющая нижележащую реализацию обычного оператора «минус» для типа `float8`, но для другого подтипа могут потребоваться дополнительные преобразования. Иногда для представления разницы в числовом виде требуется ещё и творческий подход. Функция `subtype_diff`, насколько это возможно, должна быть согласована с порядком сортировки, вытекающим из выбранных правил сортировки и класса оператора; то есть, её результат должен быть положительным, если согласно порядку сортировки первый её аргумент больше второго.

Ещё один, не столь тривиальный пример функции `subtype_diff`:

```
CREATE FUNCTION time_subtype_diff(x time, y time) RETURNS float8 AS
'SELECT EXTRACT(EPOCH FROM (x - y))' LANGUAGE sql STRICT IMMUTABLE;
```

```
CREATE TYPE timerange AS RANGE (
    subtype = time,
    subtype_diff = time_subtype_diff
);
```

```
SELECT '[11:10, 23:00]':timerange;
```

Дополнительные сведения о создании диапазонных типов можно найти в описании [CREATE TYPE](#).

8.17.9. Индексация

Для столбцов, имеющих диапазонный тип, можно создать индексы GiST и SP-GiST. Также индексы GiST можно создавать для столбцов мультидиапазонных типов. Например, так создаётся индекс GiST:

```
CREATE INDEX reservation_idx ON reservation USING GIST (during);
```

Индекс GiST или SP-GiST для диапазонов помогает ускорить запросы со следующими операторами: `=`, `&&`, `<@`, `@>`, `<<`, `>>`, `-|-`, `&<` и `&>` (дополнительно о них можно узнать в [Таблице 9.58](#). Индекс GiST для мультидиапазонов может ускорить запросы, задействующие один набор мультидиапазонных операторов. Индекс GiST для диапазонов и индекс GiST для мультидиапазонов могут также соответственно ускорить запросы, задействующие следующие межтиповые операторы диапазон-мультидиапазон и мультидиапазон-диапазон: `&&`, `<@`, `@>`, `<<`, `>>`, `-|-`, `&<` и `&>`. Дополнительные сведения можно найти в [Таблице 9.58](#).

Кроме того, для таких столбцов можно создать индексы на основе хеша и В-деревьев. Для индексов таких типов полезен по сути только один оператор диапазона — равно. Порядок сортировки В-дерева определяется для значений диапазона соответствующими операторами `<` и `>`, но этот порядок может быть произвольным и он не очень важен в реальном мире. Поддержка В-деревьев и хешей диапазонными типами нужна в основном для сортировки и хеширования при выполнении запросов, но не для создания самих индексов.

8.17.10. Ограничения для диапазонов

Тогда как для скалярных значений естественным ограничением является `UNIQUE`, оно обычно не подходит для диапазонных типов. Вместо этого чаще оказываются полезнее ограничения-исключения (см. [CREATE TABLE ... CONSTRAINT ... EXCLUDE](#)). Такие ограничения позволяют, например определить условие «непересечения» диапазонов. Например:


```
CREATE TABLE reservation (
    during tsrange,
    EXCLUDE USING GIST (during WITH &&)
);
```

Это ограничение не позволит одновременно сохранить в таблице несколько диапазонов, которые накладываются друг на друга:

```
INSERT INTO reservation VALUES
    ('[2010-01-01 11:30, 2010-01-01 15:00)');
INSERT 0 1
```

```
INSERT INTO reservation VALUES
    ('[2010-01-01 14:45, 2010-01-01 15:45)');
```

ОШИБКА: конфликтующее значение ключа нарушает ограничение-исключение
"reservation_during_excl"

ПОДРОБНОСТИ: Ключ (during)=(["2010-01-01 14:45:00", "2010-01-01 15:45:00"))
конфликтует с существующим ключом (during)=(["2010-01-01 11:30:00", "2010-01-01 15:00:00")).

Для максимальной гибкости в ограничении-исключении можно сочетать простые скалярные типы данных с диапазонами, используя расширение `btree_gist`. Например, если `btree_gist` установлено, следующее ограничение не будет допускать пересекающиеся диапазоны, только если совпадают также и номера комнат:

```
CREATE EXTENSION btree_gist;
CREATE TABLE room_reservation (
    room text,
    during tsrange,
    EXCLUDE USING GIST (room WITH =, during WITH &&)
);
```

```
INSERT INTO room_reservation VALUES
    ('123A', '[2010-01-01 14:00, 2010-01-01 15:00)');
INSERT 0 1
```

```
INSERT INTO room_reservation VALUES
    ('123A', '[2010-01-01 14:30, 2010-01-01 15:30)');
```

ОШИБКА: конфликтующее значение ключа нарушает ограничение-исключение
"room_reservation_room_during_excl"

ПОДРОБНОСТИ: Ключ (room, during)=(123A, [2010-01-01 14:30:00, 2010-01-01 15:30:00))
конфликтует
с существующим ключом (room, during)=(123A, ["2010-01-01 14:00:00", "2010-01-01 15:00:00")).

```
INSERT INTO room_reservation VALUES
    ('123B', '[2010-01-01 14:30, 2010-01-01 15:30)');
INSERT 0 1
```

8.18. Типы доменов

Домен — пользовательский тип данных, основанный на другом *нижележащем типе*. Он может быть определён с условиями, ограничивающими множество допустимых значений подмножеством значений нижележащего типа. В остальном он ведёт себя как нижележащий тип — например, с доменными типом будут работать любые операторы или функции, применимые к нижележащему типу. Нижележащим типом может быть любой встроенный или пользовательский базовый тип, тип-перечисление, массив, составной тип, диапазон или другой домен.

Например, мы можем создать домен поверх целых чисел, принимающий только положительные числа:


```
CREATE DOMAIN posint AS integer CHECK (VALUE > 0);
CREATE TABLE mytable (id posint);
INSERT INTO mytable VALUES(1);    -- работает
INSERT INTO mytable VALUES(-1);  -- ошибка
```

Когда к значению доменного типа применяются операторы или функции, предназначенные для нижележащего типа, домен автоматически приводится к нижележащему типу. Так, например, результат операции `mytable.id - 1` будет считаться имеющим тип `integer`, а не `posint`. Мы могли бы записать `(mytable.id - 1)::posint`, чтобы снова привести результат к типу `posint`, что повлечёт перепроверку ограничений домена. В этом случае, если данное выражение будет применено к `id`, равному 1, произойдёт ошибка. Значение нижележащего типа можно присвоить полю или переменной доменного типа, не записывая приведение явно, но и в этом случае ограничения домена будут проверяться.

За дополнительными сведениями обратитесь к описанию [CREATE DOMAIN](#).

8.19. Идентификаторы объектов

Идентификатор объекта (Object Identifier, OID) используется внутри Postgres Pro в качестве первичного ключа различных системных таблиц. Идентификатор объекта представляется в типе `oid`. Также существуют различные типы-псевдонимы для `oid`, с именами *регрессность*. Обзор этих типов приведён в [Таблице 8.26](#).

В настоящее время тип `oid` реализован как четырёхбайтное целое число без знака. Таким образом, значение этого типа может быть недостаточно большим для обеспечения уникальности в базе данных или даже в отдельных больших таблицах.

Для самого типа `oid` помимо сравнения определены всего несколько операторов. Однако его можно привести к целому и затем задействовать в обычных целочисленных вычислениях. (При этом следует опасаться путаницы со знаковыми/беззнаковыми значениями.)

Типы-псевдонимы OID сами по себе не вводят новых операций и отличаются только специализированными функциями ввода-вывода. Эти функции могут принимать и выводить не просто числовые значения, как тип `oid`, а символические имена системных объектов. Эти типы позволяют упростить поиск объектов по значениям OID. Например, чтобы выбрать из `pg_attribute` строки, относящиеся к таблице `mytable`, можно написать:

```
SELECT * FROM pg_attribute WHERE attrelid = 'mytable'::regclass;
```

вместо:

```
SELECT * FROM pg_attribute
WHERE attrelid = (SELECT oid FROM pg_class WHERE relname = 'mytable');
```

Хотя второй вариант выглядит не таким уж плохим, но это лишь очень простой запрос. Если же потребуется выбрать правильный OID, когда таблица `mytable` есть в нескольких схемах, вложенный подзапрос будет гораздо сложнее. Преобразователь вводимого значения типа `regclass` находит таблицу согласно заданному пути поиска схем, так что он делает «всё правильно» автоматически. Аналогично, приведя идентификатор таблицы к типу `regclass`, можно получить символическое представление числового кода.

Таблица 8.26. Идентификаторы объектов

Имя	Ссылки	Описание	Пример значения
<code>oid</code>	<code>any</code>	числовой идентификатор объекта	564182
<code>regclass</code>	<code>pg_class</code>	имя отношения	<code>pg_type</code>
<code>regcollation</code>	<code>pg_collation</code>	имя правила сортировки	"POSIX"

Имя	Ссылки	Описание	Пример значения
regconfig	pg_ts_config	конфигурация текстового поиска	english
regdictionary	pg_ts_dict	словарь текстового поиска	simple
regnamespace	pg_namespace	пространство имён	pg_catalog
regoper	pg_operator	имя оператора	+
regoperator	pg_operator	оператор с типами аргументов	*(integer, integer) или -(NONE, integer)
regproc	pg_proc	имя функции	sum
regprocedure	pg_proc	функция с типами аргументов	sum(int4)
regprofile	pg_profile	имя профиля	default
regrole	pg_authid	имя роли	smith
regtype	pg_type	имя типа данных	integer

Все типы псевдонимов OID для объектов, сгруппированных в пространство имён, принимают имена, дополненные именем схемы, и выводят имена со схемой, если данный объект нельзя будет найти в текущем пути поиска без имени схемы. Например, `myschema.mytable` является приемлемым входным значением для `regclass` (если существует такая таблица). Это значение может выводиться как `myschema.mytable` или просто `mytable`, в зависимости от текущего пути поиска. Типы `regproc` и `regoper` принимают только уникальные вводимые имена (не перегруженные), что ограничивает их применимость; в большинстве случаев лучше использовать `regprocedure` или `regoperator`. Для типа `regoperator` в записи унарного оператора неиспользуемый операнд заменяется словом `NONE`.

Функции ввода для данных типов допускают пробелы между компонентами и приводят буквы верхнего регистра к нижнему, за исключением строки в двойных кавычках; это сделано для того, чтобы правила записи были похожи на принятые для записи имён объектов в SQL. И наоборот, функции вывода будут добавлять двойные кавычки, если это необходимо, чтобы выводимая строка была допустимым идентификатором SQL. Например, OID функции с именем `foo` (с `F` в верхнем регистре), принимающей два целочисленных аргумента, можно ввести как `' "foo" ( int, integer ) '::regprocedure`. Результат будет выглядеть как `"foo"(integer, integer)`. И имя функции, и имена типов аргументов также могут быть дополнены схемой.

Многие встроенные функции PostgreSQL принимают OID таблицы или другого типа объекта БД и для удобства объявляются как принимающие `regclass` (или соответствующий тип-псевдоним OID). Это означает, что вам не нужно искать OID объекта вручную, а можно просто ввести его имя в виде строки. Например, функция `nextval (regclass)` принимает OID отношения последовательности, поэтому её можно вызвать так:

<code>nextval('foo')</code>	обращается к последовательности <code>foo</code>
<code>nextval('FOO')</code>	то же самое
<code>nextval('"foo"')</code>	обращается к последовательности <code>foo</code>
<code>nextval('myschema.foo')</code>	обращается к <code>myschema.foo</code>
<code>nextval('"myschema".foo')</code>	то же самое
<code>nextval('foo')</code>	ищет <code>foo</code> в пути поиска

Примечание

Когда аргумент такой функции записывается как текстовая строка в чистом виде, она становится константой типа `regclass`. Так как фактически это будет просто значение OID, оно будет привязано к изначально идентифицированной последовательности, несмотря на то,

что она может быть переименована, перенесена в другую схему и т. д. Такое «раннее связывание» обычно желательно для ссылок на последовательности в значениях столбцов по умолчанию и представлениях. Но иногда возникает необходимость в «позднем связывании», когда ссылки на последовательности распознаются в процессе выполнения. Чтобы получить такое поведение, нужно принудительно изменить тип константы с `regclass` на `text`:

```
nextval('foo'::text)      foo распознаётся во время выполнения
```

Для поиска во время выполнения также может использоваться функция `to_regclass()` и подобные. См. [Таблицу 9.75](#).

Другой практический пример использования `regclass` — поиск OID таблицы, отображённой в представлениях `information_schema`, которые не предоставляют такие OID напрямую. Например, можно вызвать функцию `pg_relation_size()`, для которой требуется OID таблицы. С учётом указанных выше правил, далее представлен корректный способ вызова данной функции

```
SELECT table_schema, table_name,
       pg_relation_size((quote_ident(table_schema) || '.' ||
                           quote_ident(table_name))::regclass)
FROM information_schema.tables
WHERE ...
```

. Функция `quote_ident()` заключит идентификатор в двойные кавычки, когда это необходимо. Более простым способом кажется

```
SELECT pg_relation_size(table_name)
FROM information_schema.tables
WHERE ...
```

но использовать его *не рекомендуется*, потому что он не сработает для таблиц, которые не входят в заданный путь поиска или имена которых нужно заключать в кавычки.

Дополнительным свойством большинства типов псевдонимов OID является образование зависимостей. Когда в сохранённом выражении фигурирует константа одного из этих типов (например, в представлении или в значении столбца по умолчанию), это создаёт зависимость от целевого объекта. Например, если значение по умолчанию определяется выражением `nextval('my_seq'::regclass)`, Postgres Pro понимает, что это выражение зависит от последовательности `my_seq`, и не позволит удалить последовательность раньше, чем будет удалено это выражение. Альтернативная запись `nextval('my_seq'::text)` не создаёт зависимость. (Исключениями являются типы `regprofile` и `regrole`. Константы этих типов в таких выражениях не допускаются.)

Есть ещё один тип системных идентификаторов, `xid`, представляющий идентификатор транзакции (сокращённо *хаст*). Этот тип имеют системные столбцы `xmin` и `xmax`. Идентификаторы транзакций определяются 32-битными числами. В некоторых контекстах используется 64-битный вариант `xid8`. В отличие от `xid`, значения `xid8` увеличиваются строго монотонно и никогда не повторяются на протяжении всего существования кластера баз данных.

Третий тип идентификаторов, используемых в системе, — `cid`, идентификатор команды (`command identifier`). Этот тип данных имеют системные столбцы `cmin` и `cmay`. Идентификаторы команд — это тоже 32-битные числа.

И наконец, последний тип системных идентификаторов — `tid`, идентификатор строки/кортежа (`tuple identifier`). Этот тип данных имеет системный столбец `ctid`. Идентификатор кортежа представляет собой пару (из номера блока и индекса кортежа в блоке), идентифицирующую физическое расположение строки в таблице.

(Подробнее о системных столбцах рассказывается в [Разделе 5.5](#).)

8.20. Тип `pg_lsn`

Тип данных `pg_lsn` может применяться для хранения значения LSN (последовательный номер в журнале, Log Sequence Number), которое представляет собой указатель на позицию в журнале WAL. Этот тип содержит `XLogRecPtr` и является внутренним системным типом Postgres Pro.

Технически LSN — это 64-битное целое число, представляющее байтовое смещение в потоке журнала предзаписи. Он выводится в виде двух шестнадцатеричных чисел до 8 цифр каждое, через косую черту, например: `16/B374D848`. Тип `pg_lsn` поддерживает стандартные операторы сравнения, такие как `=` и `>`. Можно также вычесть один LSN из другого с помощью оператора `-`; результатом будет число байт между этими двумя позициями в журнале предзаписи. Также число байт можно добавлять и вычитать из LSN с помощью операторов `+(pg_lsn, numeric)` и `-(pg_lsn, numeric)` соответственно. Заметьте, что вычисленный LSN должен находиться в допустимом для типа `pg_lsn` диапазоне, то есть между `0/0` и `FFFFFFFF/FFFFFFFF`.

8.21. Псевдотипы

В систему типов Postgres Pro включены несколько специальных элементов, которые в совокупности называются *псевдотипами*. Псевдотип нельзя использовать в качестве типа данных столбца, но можно объявить функцию с аргументом или результатом такого типа. Каждый из существующих псевдотипов полезен в ситуациях, когда характер функции не позволяет просто получить или вернуть определённый тип данных SQL. Все существующие псевдотипы перечислены в [Таблице 8.27](#).

Таблица 8.27. Псевдотипы

Имя	Описание
<code>any</code>	Указывает, что функция принимает любой вводимый тип данных.
<code>anyelement</code>	Указывает, что функция принимает любой тип данных (см. Подраздел 37.2.5).
<code>anyarray</code>	Указывает, что функция принимает любой тип массива (см. Подраздел 37.2.5).
<code>anynonarray</code>	Указывает, что функция принимает любой тип данных, кроме массивов (см. Подраздел 37.2.5).
<code>anyenum</code>	Указывает, что функция принимает любое перечисление (см. Подраздел 37.2.5 и Раздел 8.7).
<code>anyrange</code>	Указывает, что функция принимает любой диапазонный тип данных (см. Подраздел 37.2.5 и Раздел 8.17).
<code>anymultirange</code>	Указывает, что функция принимает любой мультидиапазонный тип данных (см. Подраздел 37.2.5 и Раздел 8.17).
<code>anycompatible</code>	Указывает, что функция принимает любой тип данных и может автоматически приводить различные аргументы к общему типу данных (см. Подраздел 37.2.5).
<code>anycompatiblearray</code>	Указывает, что функция принимает любой тип массива и может автоматически приводить различные аргументы к общему типу данных (см. Подраздел 37.2.5).
<code>anycompatiblenonarray</code>	Указывает, что функция принимает любой тип, отличный от массива, и может автоматически приводить различные аргументы к общему типу данных (см. Подраздел 37.2.5).
<code>anycompatiblerange</code>	Указывает, что функция принимает любой тип диапазонных данных и может автоматически приводить различные аргументы к общему типу данных (см. Подраздел 37.2.5 и Раздел 8.17).
<code>anycompatiblemultirange</code>	Указывает, что функция принимает любой мультидиапазонный тип данных и может автоматически приводить

Имя	Описание
	различные аргументы к общему типу данных (см. Подраздел 37.2.5 и Раздел 8.17).
cstring	Указывает, что функция принимает или возвращает строку в стиле C.
internal	Указывает, что функция принимает или возвращает внутренний серверный тип данных.
language_handler	Обработчик процедурного языка объявляется как возвращающий тип <code>language_handler</code> .
fdw_handler	Обработчик обёртки сторонних данных объявляется как возвращающий тип <code>fdw_handler</code> .
table_am_handler	Обработчик табличного метода доступа объявляется как возвращающий тип <code>table_am_handler</code> .
index_am_handler	Обработчик метода доступа индекса объявляется как возвращающий тип <code>index_am_handler</code> .
tsm_handler	Обработчик метода выборки из таблицы объявляется как возвращающий тип <code>tsm_handler</code> .
record	Указывает, что функция принимает или возвращает неопределённый тип строки.
trigger	Триггерная функция объявляется как возвращающая тип <code>trigger</code> .
event_trigger	Функция событийного триггера объявляется как возвращающая тип <code>event_trigger</code> .
pg_ddl_command	Обозначает представление команд DDL, доступное событийным триггерам.
void	Указывает, что функция не возвращает значение.
unknown	Обозначает ещё не распознанный тип, например простую строковую константу.

Функции, написанные на языке C (встроенные или динамически загружаемые), могут быть объявлены с параметрами или результатами любого из этих псевдотипов. Ответственность за безопасное поведение функции с аргументами таких типов ложится на разработчика функции.

Функции, написанные на процедурных языках, могут использовать псевдотипы, только если это позволяет соответствующий язык. В настоящее время большинство процедурных языков запрещают использовать псевдотипы в качестве типа аргумента и позволяют использовать для результатов только типы `void` и `record` (и `trigger` или `event_trigger`, когда функция реализует триггер или событийный триггер). Некоторые языки также поддерживают полиморфные функции с полиморфными псевдотипами, подробно рассмотренными в [Подразделе 37.2.5](#).

Псевдотип `internal` используется в объявлениях функций, предназначенных только для внутреннего использования в СУБД, но не для прямого вызова в запросах SQL. Если у функции есть как хотя бы один аргумент типа `internal`, её нельзя будет вызывать из SQL. Чтобы сохранить типобезопасность при таком ограничении, следуйте важному правилу: не создавайте функцию, возвращающую результат типа `internal`, если у неё нет ни одного аргумента `internal`.

Глава 9. Функции и операторы

Postgres Pro предоставляет огромное количество функций и операторов для встроенных типов данных. В этой главе описаны основные из них, тогда как некоторые специальные функции описываются в других разделах документации. Кроме того, пользователи могут определять свои функции и операторы, как описано в [Части V](#). Просмотреть все существующие функции и операторы можно в `psql` с помощью команд `\df` и `\do`, соответственно.

В этой главе типы аргументов и результата функции обозначаются так:

```
repeat ( text, integer ) → text
```

В данном случае она говорит, что функция `repeat` принимает один текстовый и один целочисленный аргумент и возвращает результат текстового типа. Стрелка вправо также указывает на результат в примере использования, например:

```
repeat ('Pg', 4) → PgPgPgPg
```

Если для вас важна переносимость, учтите, что практически все функции и операторы, описанные в этой главе, за исключением простейших арифметических и операторов сравнения, а также явно отмеченных функций, не описаны в стандарте SQL. Тем не менее частично эта расширенная функциональность присутствует и в других СУБД SQL и во многих случаях различные реализации одинаковых функций оказываются аналогичными и совместимыми.

9.1. Логические операторы

Набор логических операторов включает обычные:

```
boolean AND boolean → boolean
```

```
boolean OR boolean → boolean
```

```
NOT boolean → boolean
```

В SQL работает логическая система с тремя состояниями: `true` (истина), `false` (ложь) и `NULL`, «неопределённое» состояние. Рассмотрите следующие таблицы истинности:

a	b	a AND b	a OR b
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	NULL	NULL	TRUE
FALSE	FALSE	FALSE	FALSE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

a	NOT a
TRUE	FALSE
FALSE	TRUE
NULL	NULL

Операторы `AND` и `OR` коммутативны, то есть от перемены мест операндов результат не меняется. (Однако левый операнд не обязательно будет вычисляться перед правым. Более подробно порядок вычисления подвыражений описывается в [Подразделе 4.2.14.](#))

9.2. Функции и операторы сравнения

Набор операторов сравнения включает обычные операторы, перечисленные в [Таблице 9.1.](#)

Таблица 9.1. Операторы сравнения

Оператор	Описание
<code>тип_данных < тип_данных → boolean</code>	Меньше
<code>тип_данных > тип_данных → boolean</code>	Больше
<code>тип_данных <= тип_данных → boolean</code>	Меньше или равно
<code>тип_данных >= тип_данных → boolean</code>	Больше или равно
<code>тип_данных = тип_данных → boolean</code>	Равно
<code>тип_данных <> тип_данных → boolean</code>	Не равно
<code>тип_данных != тип_данных → boolean</code>	Не равно

Примечание

В стандарте SQL для условия «не равно» принята запись `<>`. Синонимичная ей запись `!=` преобразуется в `<>` на самой ранней стадии разбора запроса. Как следствие, реализовать операторы `!=` и `<>` так, чтобы они работали по-разному, невозможно.

Эти операторы сравнения имеются для всех встроенных типов данных, значения которых сортируются естественным образом, включая числовые, строковые типы, а также типы даты/времени. Кроме того, сравниваться могут массивы, составные типы и диапазоны, если типы данных их компонентов являются сравниваемыми.

Обычно можно сравнивать также значения связанных типов данных; например, возможно сравнение `integer > bigint`. Некоторые подобные операции реализуются непосредственно «межтиповыми» операторами сравнения, но если такого оператора нет, анализатор запроса попытается привести частные типы к более общим и применить подходящий для них оператор сравнения.

Как показано выше, все операторы сравнения являются бинарными и возвращают значения типа `boolean`. Таким образом, выражения вида `1 < 2 < 3` недопустимы (так как не существует оператора `<`, который бы сравнивал булево значение с 3). Для проверки нахождения значения в интервале, воспользуйтесь предикатом `BETWEEN`, описанным ниже.

Существует также несколько предикатов сравнения; они приведены в [Таблице 9.2](#). Они работают подобно операторам, но имеют особый синтаксис, установленный стандартом SQL.

Таблица 9.2. Предикаты сравнения

Предикат Описание Пример(ы)
<code>тип_данных BETWEEN тип_данных AND тип_данных → boolean</code> Между (включая границы интервала). <code>2 BETWEEN 1 AND 3 → t</code> <code>2 BETWEEN 3 AND 1 → f</code>
<code>тип_данных NOT BETWEEN тип_данных AND тип_данных → boolean</code> Не между (обратное к <code>BETWEEN</code>). <code>2 NOT BETWEEN 1 AND 3 → f</code>
<code>тип_данных BETWEEN SYMMETRIC тип_данных AND тип_данных → boolean</code> Между, после сортировки граничных значений. <code>2 BETWEEN SYMMETRIC 3 AND 1 → t</code>
<code>тип_данных NOT BETWEEN SYMMETRIC тип_данных AND тип_данных → boolean</code>

Предикат Описание Пример(ы)
Не между, после сортировки граничных значений. <code>2 NOT BETWEEN SYMMETRIC 3 AND 1 → f</code>
<code>тип_данных IS DISTINCT FROM тип_данных → boolean</code> Не равно, при этом NULL воспринимается как обычное значение. <code>1 IS DISTINCT FROM NULL → t (а не NULL)</code> <code>NULL IS DISTINCT FROM NULL → f (а не NULL)</code>
<code>тип_данных IS NOT DISTINCT FROM тип_данных → boolean</code> Равно, при этом NULL воспринимается как обычное значение. <code>1 IS NOT DISTINCT FROM NULL → f (а не NULL)</code> <code>NULL IS NOT DISTINCT FROM NULL → t (а не NULL)</code>
<code>тип_данных IS NULL → boolean</code> Проверяет, является ли значение эквивалентным NULL. <code>1.5 IS NULL → f</code>
<code>тип_данных IS NOT NULL → boolean</code> Проверяет, отличается ли значение от NULL. <code>'null' IS NOT NULL → t</code>
<code>тип_данных ISNULL → boolean</code> Проверяет, является ли значение эквивалентным NULL (нестандартный синтаксис).
<code>тип_данных NOTNULL → boolean</code> Проверяет, отличается ли значение от NULL (нестандартный синтаксис).
<code>boolean IS TRUE → boolean</code> Проверяет, является ли результат логического выражения значением true. <code>true IS TRUE → t</code> <code>NULL::boolean IS TRUE → f (а не NULL)</code>
<code>boolean IS NOT TRUE → boolean</code> Проверяет, является ли результат логического выражения значением false или неизвестным. <code>true IS NOT TRUE → f</code> <code>NULL::boolean IS NOT TRUE → t (а не NULL)</code>
<code>boolean IS FALSE → boolean</code> Проверяет, является ли результат логического выражения значением false. <code>true IS FALSE → f</code> <code>NULL::boolean IS FALSE → f (а не NULL)</code>
<code>boolean IS NOT FALSE → boolean</code> Проверяет, является ли результат логического выражения значением true или неизвестным. <code>true IS NOT FALSE → t</code> <code>NULL::boolean IS NOT FALSE → t (а не NULL)</code>
<code>boolean IS UNKNOWN → boolean</code> Проверяет, является ли результат логического выражения неизвестным значением. <code>true IS UNKNOWN → f</code> <code>NULL::boolean IS UNKNOWN → t (а не NULL)</code>

Предикат	Описание	Пример(ы)
	<code>boolean IS NOT UNKNOWN</code>	<code>→ boolean</code>
	Проверяет, является ли результат логического выражения значением <code>true</code> или <code>false</code> .	
	<code>true IS NOT UNKNOWN</code>	<code>→ t</code>
	<code>NULL::boolean IS NOT UNKNOWN</code>	<code>→ f (а не NULL)</code>

Предикат `BETWEEN` упрощает проверки интервала:

`a BETWEEN x AND y`

равнозначно

`a >= x AND a <= y`

Заметьте, что `BETWEEN` считает, что границы интервала включаются в интервал. Предикат `BETWEEN SYMMETRIC` аналогичен `BETWEEN`, за исключением того, что аргумент слева от `AND` не обязательно должен быть меньше или равен аргументу справа. Если это не так, аргументы автоматически меняются местами, так что всегда подразумевается непустой интервал.

Различные варианты `BETWEEN` реализуются посредством обычных операторов сравнения, и поэтому они будут работать с любыми типами данных, которые можно сравнивать.

Примечание

Использование `AND` в конструкции `BETWEEN` создаёт неоднозначность с использованием `AND` в качестве логического оператора. Для её устранения в качестве второго аргумента предложения `BETWEEN` принимается только ограниченный набор типов выражений. Если вам нужно записать более сложное подвыражение в `BETWEEN`, заключите это подвыражение в скобки.

Обычные операторы сравнения выдают `NULL` (что означает «неопределённость»), а не `true` или `false`, когда любое из сравниваемых значений `NULL`. Например, `7 = NULL` выдаёт `NULL`, так же, как и `7 <> NULL`. Когда это поведение нежелательно, можно использовать предикаты `IS [ NOT ] DISTINCT FROM`:

`a IS DISTINCT FROM b`

`a IS NOT DISTINCT FROM b`

Для значений не `NULL` условие `IS DISTINCT FROM` работает так же, как оператор `<>`. Однако если оба сравниваемых значения `NULL`, результат будет `false`, и только если одно из значений `NULL`, возвращается `true`. Аналогично, условие `IS NOT DISTINCT FROM` равносильно `=` для значений не `NULL`, но возвращает `true`, если оба сравниваемых значения `NULL`, и `false` в противном случае. Таким образом, эти предикаты по сути работают с `NULL`, как с обычным значением, а не с «неопределённостью».

Для проверки, содержит ли значение `NULL` или нет, используются предикаты:

`выражение IS NULL`

`выражение IS NOT NULL`

или равнозначные (но нестандартные) предикаты:

`выражение ISNULL`

`выражение NOTNULL`

Заметьте, что проверка `выражение = NULL` не будет работать, так как `NULL` считается не «равным» `NULL`. (Значение `NULL` представляет неопределённость, и равны ли две неопределённости, тоже не определено.)

Подсказка

Некоторые приложения могут ожидать, что *выражение* = NULL вернёт true, если результатом *выражения* является NULL. Такие приложения настоятельно рекомендуется исправить и привести в соответствие со стандартом SQL. Однако в случаях, когда это невозможно, это поведение можно изменить с помощью параметра конфигурации [transform_null_equals](#). Когда этот параметр включён, Postgres Pro преобразует условие `x = NULL` в `x IS NULL`.

Если *выражение* возвращает табличную строку, тогда `IS NULL` будет истинным, когда само выражение — NULL или все поля строки — NULL, а `IS NOT NULL` будет истинным, когда само выражение не NULL, и все поля строки так же не NULL. Вследствие такого определения, `IS NULL` и `IS NOT NULL` не всегда будут возвращать взаимодополняющие результаты для таких выражений; в частности такие выражения со строками, одни поля которых NULL, а другие не NULL, будут ложными одновременно. В некоторых случаях имеет смысл написать *строка* `IS DISTINCT FROM NULL` или *строка* `IS NOT DISTINCT FROM NULL`, чтобы просто проверить, равно ли NULL всё значение строки, без каких-либо дополнительных проверок полей строки.

Логические значения можно также проверить с помощью предикатов

```
логическое_выражение IS TRUE
логическое_выражение IS NOT TRUE
логическое_выражение IS FALSE
логическое_выражение IS NOT FALSE
логическое_выражение IS UNKNOWN
логическое_выражение IS NOT UNKNOWN
```

Они всегда возвращают true или false и никогда NULL, даже если какой-либо операнд — NULL. Они интерпретируют значение NULL как «неопределённость». Заметьте, что `IS UNKNOWN` и `IS NOT UNKNOWN` по сути равнозначны `IS NULL` и `IS NOT NULL`, соответственно, за исключением того, что выражение может быть только булевого типа.

Также имеется несколько связанных со сравнениями функций; они перечислены в [Таблице 9.3](#).

Таблица 9.3. Функции сравнения

Функция	Описание	Пример(ы)
<code>num_nonnulls</code>	<code>(VARIADIC "any") → integer</code> Возвращает число аргументов, отличных от NULL.	<code>num_nonnulls(1, NULL, 2) → 2</code>
<code>num_nulls</code>	<code>(VARIADIC "any") → integer</code> Возвращает число аргументов NULL.	<code>num_nulls(1, NULL, 2) → 1</code>

9.3. Математические функции и операторы

Математические операторы определены для множества типов Postgres Pro. Как работают эти операции с типами, для которых нет стандартных соглашений о математических действиях (например, с типами даты/времени), мы опишем в последующих разделах.

В [Таблице 9.4](#) показаны математические операторы, реализованные для стандартных числовых типов. Если не отмечено обратное, операторы, принимающие *числовой_тип*, работают с типами `smallint`, `integer`, `bigint`, `numeric`, `real` и `double precision`. Операторы, принимающие *целочисленный_тип*, работают с типами `smallint`, `integer` и `bigint`. За исключением явно отмеченных случаев, все разновидности операторов возвращают такой же тип данных, который имеет их аргумент (или их аргументы). Вызовы с неодинаковыми типами аргументов, например `integer + numeric`, разрешаются в пользу типа, оказавшегося последним в определённых выше списках.

Таблица 9.4. Математические операторы

Оператор Описание Пример(ы)
<code>числовой_тип + числовой_тип → числовой_тип</code> Сложение $2 + 3 \rightarrow 5$
<code>+ числовой_тип → числовой_тип</code> Унарный плюс (нет операции) $+ 3.5 \rightarrow 3.5$
<code>числовой_тип - числовой_тип → числовой_тип</code> Вычитание $2 - 3 \rightarrow -1$
<code>- числовой_тип → числовой_тип</code> Смена знака $- (-4) \rightarrow 4$
<code>числовой_тип * числовой_тип → числовой_тип</code> Умножение $2 * 3 \rightarrow 6$
<code>числовой_тип / числовой_тип → числовой_тип</code> Деление (при делении целочисленных типов результат округляется в направлении нуля) $5.0 / 2 \rightarrow 2.5000000000000000$ $5 / 2 \rightarrow 2$ $(-5) / 2 \rightarrow -2$
<code>числовой_тип % числовой_тип → числовой_тип</code> Остаток от деления; имеется для типов smallint, integer, bigint и numeric $5 \% 4 \rightarrow 1$
<code>numeric ^ numeric → numeric</code> <code>double precision ^ double precision → double precision</code> Возведение в степень $2 ^ 3 \rightarrow 8$ В отличие от принятого в математике порядка, ^ по умолчанию вычисляется слева направо $2 ^ 3 ^ 3 \rightarrow 512$ $2 ^ (3 ^ 3) \rightarrow 134217728$
<code> / double precision → double precision</code> Квадратный корень $ / 25.0 \rightarrow 5$
<code> / double precision → double precision</code> Кубический корень $ / 64.0 \rightarrow 4$
<code>@ числовой_тип → числовой_тип</code> Абсолютное значение $@ -5.0 \rightarrow 5.0$
<code>целочисленный_тип & целочисленный_тип → целочисленный_тип</code> Битовое И

Оператор Описание Пример(ы)
$91 \ \& \ 15 \rightarrow 11$
<i>целочисленный_тип</i> <i>целочисленный_тип</i> $\rightarrow$ <i>целочисленный_тип</i> Битовое ИЛИ $32 \ \ 3 \rightarrow 35$
<i>целочисленный_тип</i> # <i>целочисленный_тип</i> $\rightarrow$ <i>целочисленный_тип</i> Битовое исключающее ИЛИ $17 \ \# \ 5 \rightarrow 20$
$\sim$ <i>целочисленный_тип</i> $\rightarrow$ <i>целочисленный_тип</i> Битовое НЕ $\sim 1 \rightarrow -2$
<i>целочисленный_тип</i> << <i>integer</i> $\rightarrow$ <i>целочисленный_тип</i> Битовый сдвиг влево $1 \ \<\< \ 4 \rightarrow 16$
<i>целочисленный_тип</i> >> <i>integer</i> $\rightarrow$ <i>целочисленный_тип</i> Битовый сдвиг вправо $8 \ \>\> \ 2 \rightarrow 2$

В [Таблице 9.5](#) перечислены все существующие математические функции. Многие из этих функций имеют несколько форм с разными типами аргументов. За исключением случаев, где это указано явно, все разновидности функций возвращают тот же тип данных, который имеет их аргумент (или аргументы); вызовы с неодинаковыми типами разрешаются по тому же принципу, что был описан выше для операторов. Функции, работающие с данными `double precision`, в массе своей используют реализации из системных библиотек сервера, поэтому точность и поведение в граничных случаях может зависеть от системы сервера.

Таблица 9.5. Математические функции

Функция Описание Пример(ы)
<code>abs (<i>числовой_тип</i>)</code> $\rightarrow$ <i>числовой_тип</i> Абсолютное значение <code>abs (-17.4)</code> $\rightarrow 17.4$
<code>cbrt (<i>double precision</i>)</code> $\rightarrow$ <i>double precision</i> Кубический корень <code>cbrt (64.0)</code> $\rightarrow 4$
<code>ceil (<i>numeric</i>)</code> $\rightarrow$ <i>numeric</i> <code>ceil (<i>double precision</i>)</code> $\rightarrow$ <i>double precision</i> Ближайшее целое, большее или равное аргументу <code>ceil (42.2)</code> $\rightarrow 43$ <code>ceil (-42.8)</code> $\rightarrow -42$
<code>ceiling (<i>numeric</i>)</code> $\rightarrow$ <i>numeric</i> <code>ceiling (<i>double precision</i>)</code> $\rightarrow$ <i>double precision</i> Ближайшее целое, большее или равное аргументу (равнозначно ceil) <code>ceiling (95.3)</code> $\rightarrow 96$
<code>degrees (<i>double precision</i>)</code> $\rightarrow$ <i>double precision</i>

Функция	Описание	Пример(ы)
	Преобразование радианов в градусы	<code>degrees(0.5) → 28.64788975654116</code>
	<code>div (y numeric, x numeric) → numeric</code> Целочисленный результат y/x (округлённый в направлении нуля)	<code>div(9, 4) → 2</code>
	<code>exp (numeric) → numeric</code> <code>exp (double precision) → double precision</code> Экспонента (е возводится в заданную степень)	<code>exp(1.0) → 2.7182818284590452</code>
	<code>factorial (bigint) → numeric</code> Факториал	<code>factorial(5) → 120</code>
	<code>floor (numeric) → numeric</code> <code>floor (double precision) → double precision</code> Ближайшее целое, меньшее или равное аргументу	<code>floor(42.8) → 42</code> <code>floor(-42.8) → -43</code>
	<code>gcd (числовой_тип , числовой_тип) → числовой_тип</code> Наибольший общий делитель (наибольшее положительное целое, на которое оба аргумента делятся без остатка); 0, если оба аргумента нулевые; имеется для типов <code>integer</code> , <code>bigint</code> и <code>numeric</code>	<code>gcd(1071, 462) → 21</code>
	<code>lcm (числовой_тип , числовой_тип) → числовой_тип</code> Наименьшее общее кратное (наименьшее строго положительное число, которое делится нацело на оба аргумента); 0, если оба аргумента нулевые; имеется для типов <code>integer</code> , <code>bigint</code> и <code>numeric</code>	<code>lcm(1071, 462) → 23562</code>
	<code>ln (numeric) → numeric</code> <code>ln (double precision) → double precision</code> Натуральный логарифм	<code>ln(2.0) → 0.6931471805599453</code>
	<code>log (numeric) → numeric</code> <code>log (double precision) → double precision</code> Логарифм по основанию 10	<code>log(100) → 2</code>
	<code>log10 (numeric) → numeric</code> <code>log10 (double precision) → double precision</code> Логарифм по основанию 10 (то же, что и <code>log</code>)	<code>log10(1000) → 3</code>
	<code>log (b numeric, x numeric) → numeric</code> Логарифм x по основанию b	<code>log(2.0, 64.0) → 6.0000000000000000</code>
	<code>min_scale (numeric) → integer</code>	

Функция	Описание	Пример(ы)
	Минимальный масштаб (число цифр в дробной части), необходимый для точного представления заданного значения	<code>min_scale(8.4100) → 2</code>
	<code>mod (у числовой_тип , х числовой_тип) → числовой_тип</code> Остаток от деления y/x ; имеется для типов <code>smallint</code> , <code>integer</code> , <code>bigint</code> и <code>numeric</code>	<code>mod(9, 4) → 1</code>
	<code>pi () → double precision</code> Приближённое значение π	<code>pi () → 3.141592653589793</code>
	<code>power (a numeric, b numeric) → numeric</code> <code>power (a double precision, b double precision) → double precision</code> a возводится в степень b	<code>power(9, 3) → 729</code>
	<code>radians (double precision) → double precision</code> Преобразование градусов в радианы	<code>radians(45.0) → 0.7853981633974483</code>
	<code>round (numeric) → numeric</code> <code>round (double precision) → double precision</code> Округляет до ближайшего целого числа. Для <code>numeric</code> половина (.5) округляется до одного по модулю. Для <code>double precision</code> результат округления половины зависит от платформы (обычно выполняется «округление до ближайшего чётного»).	<code>round(42.4) → 42</code>
	<code>round (v numeric, s integer) → numeric</code> Округление v до s десятичных знаков. Половина (.5) округляется до 1 по модулю.	<code>round(42.4382, 2) → 42.44</code> <code>round(1234.56, -1) → 1230</code>
	<code>scale (numeric) → integer</code> Масштаб аргумента (число десятичных цифр в дробной части)	<code>scale(8.4100) → 4</code>
	<code>sign (numeric) → numeric</code> <code>sign (double precision) → double precision</code> Знак аргумента (-1, 0 или +1)	<code>sign(-8.4) → -1</code>
	<code>sqrt (numeric) → numeric</code> <code>sqrt (double precision) → double precision</code> Квадратный корень	<code>sqrt(2) → 1.4142135623730951</code>
	<code>trim_scale (numeric) → numeric</code> Сокращает масштаб значения (число цифр в дробной части), убирая конечные нули	<code>trim_scale(8.4100) → 8.41</code>
	<code>trunc (numeric) → numeric</code> <code>trunc (double precision) → double precision</code> Усечение до целого (в направлении нуля)	

Функция	Описание	Пример(ы)
		<code>trunc(42.8) → 42</code> <code>trunc(-42.8) → -42</code>
	<code>trunc (v numeric, s integer) → numeric</code> Усечение <i>v</i> до <i>s</i> десятичных знаков	<code>trunc(42.4382, 2) → 42.43</code>
	<code>width_bucket (operand numeric, low numeric, high numeric, count integer) → integer</code> <code>width_bucket (operand double precision, low double precision, high double precision, count integer) → integer</code> Возвращает номер группы, в которую попадёт <i>operand</i> в гистограмме с числом групп <i>count</i> равного размера, в диапазоне от <i>low</i> до <i>high</i>. Группы определяются нижними границами (включая значения на границе) и верхними границами (без значений на границе). Результатом будет 0, если операнд лежит левее значения <i>low</i>, и <i>count</i>+1, если значение больше значения <i>high</i> или равно ему. При сценарии <i>low</i> > <i>high</i> ситуация будет обратная, и группа 1 теперь будет ниже значения <i>low</i>, и включаться будут пограничные значения начиная с верхних.	<code>width_bucket(5.35, 0.024, 10.06, 5) → 3</code> <code>width_bucket(9, 10, 0, 10) → 2</code>
	<code>width_bucket (operand anycompatible, thresholds anycompatiblearray) → integer</code> Возвращает номер группы, в которую попадёт <i>operand</i> (группы определяются нижними границами, включающими пограничные значения, передаваемыми в <i>thresholds</i>). Результатом будет 0, если операнд оказывается левее нижней границы. Аргумент <i>operand</i> и элементы массива могут быть любого типа, для которого определены стандартные операторы сравнения. Массив <i>thresholds</i> должен быть отсортирован по возрастанию, иначе будут получены неожиданные результаты.	<code>width_bucket(now(), array['yesterday', 'today', 'tomorrow']::timestampz[]) → 2</code>

В Таблице 9.6 перечислены все функции для генерации случайных чисел.

Таблица 9.6. Случайные функции

Функция	Описание	Пример(ы)
	<code>random() → double precision</code> Возвращает случайное число в диапазоне 0.0 ≤ x < 1.0	<code>random() → 0.897124072839091</code>
	<code>setseed (double precision) → void</code> Задаёт отправную точку для последующих вызовов <code>random()</code> ; аргументом может быть число от -1.0 до 1.0, включая границы	<code>setseed(0.12345)</code>

Функция `random()` использует детерминированный генератор псевдослучайных чисел. Она работает быстро, но не подходит для криптографических приложений; более безопасная альтернатива имеется в модуле [pgcrypto](#). Если воспользоваться функцией `setseed()` и вызывать её с одним и тем же аргументом, в текущем сеансе можно получать повторяющиеся последовательности результатов `random()`. Если функция `setseed()` до этого не вызывалась в текущем сеансе, при первом вызове `random()` загрузка будет получена из платформозависимого источника случайных чисел.

В [Таблице 9.7](#) перечислены все имеющиеся тригонометрические функции. У каждой функции имеются две вариации: одна измеряет углы в радианах, а вторая — в градусах.

Таблица 9.7. Тригонометрические функции

Функция Описание Пример(ы)
<code>acos (double precision) → double precision</code> Арккосинус в радианах <code>acos(1) → 0</code>
<code>acosd (double precision) → double precision</code> Арккосинус в градусах <code>acosd(0.5) → 60</code>
<code>asin (double precision) → double precision</code> Арксинус в радианах <code>asin(1) → 1.5707963267948966</code>
<code>asind (double precision) → double precision</code> Арксинус в градусах <code>asind(0.5) → 30</code>
<code>atan (double precision) → double precision</code> Арктангенс в радианах <code>atan(1) → 0.7853981633974483</code>
<code>atand (double precision) → double precision</code> Арктангенс в градусах <code>atand(1) → 45</code>
<code>atan2 (y double precision, x double precision) → double precision</code> Арктангенс отношения y/x в радианах <code>atan2(1, 0) → 1.5707963267948966</code>
<code>atan2d (y double precision, x double precision) → double precision</code> Арктангенс отношения y/x в градусах <code>atan2d(1, 0) → 90</code>
<code>cos (double precision) → double precision</code> Косинус угла в радианах <code>cos(0) → 1</code>
<code>cosd (double precision) → double precision</code> Косинус угла в градусах <code>cosd(60) → 0.5</code>
<code>cot (double precision) → double precision</code> Котангенс угла в радианах <code>cot(0.5) → 1.830487721712452</code>
<code>cotd (double precision) → double precision</code> Котангенс угла в градусах <code>cotd(45) → 1</code>
<code>sin (double precision) → double precision</code> Синус угла в радианах <code>sin(1) → 0.8414709848078965</code>

Функция Описание Пример(ы)
<code>sind (double precision) → double precision</code> Синус угла в градусах <code>sind(30) → 0.5</code>
<code>tan (double precision) → double precision</code> Тангенс угла в радианах <code>tan(1) → 1.5574077246549023</code>
<code>tand (double precision) → double precision</code> Тангенс угла в градусах <code>tand(45) → 1</code>

Примечание

Также можно работать с углами в градусах, применяя вышеупомянутые функции преобразования единиц `radians()` и `degrees()`. Однако предпочтительнее использовать тригонометрические функции с градусами, так как это позволяет избежать ошибок округления в особых случаях, например, при вычислении `sind(30)`.

В [Таблице 9.8](#) перечислены все имеющиеся гиперболические операторы.

Таблица 9.8. Гиперболические функции

Функция Описание Пример(ы)
<code>sinh (double precision) → double precision</code> Гиперболический синус <code>sinh(1) → 1.1752011936438014</code>
<code>cosh (double precision) → double precision</code> Гиперболический косинус <code>cosh(0) → 1</code>
<code>tanh (double precision) → double precision</code> Гиперболический тангенс <code>tanh(1) → 0.7615941559557649</code>
<code>asinh (double precision) → double precision</code> Обратный гиперболический синус <code>asinh(1) → 0.881373587019543</code>
<code>acosh (double precision) → double precision</code> Обратный гиперболический косинус <code>acosh(1) → 0</code>
<code>atanh (double precision) → double precision</code> Обратный гиперболический тангенс <code>atanh(0.5) → 0.5493061443340548</code>

9.4. Строковые функции и операторы

В этом разделе описаны функции и операторы для работы с текстовыми строками. Под строками в данном контексте подразумеваются значения типов `character`, `character varying` и `text`. Если

не отмечено обратное, эти функции и операторы принимают и возвращают тип `text`. С тем же успехом в аргументах может передаваться тип `character varying`. Аргументы же типа `character` до вызова оператора или функции приводятся к типу `text`, вследствие чего завершающие пробелы в значениях `character` будут обрезаться.

В SQL определены несколько строковых функций, в которых аргументы разделяются не запятыми, а ключевыми словами. Они перечислены в [Таблице 9.9](#). Postgres Pro также предоставляет варианты этих функций с синтаксисом, обычным для функций (см. [Таблицу 9.10](#)).

Примечание

Оператор конкатенации строк (`||`) сможет принять нестроковый аргумент, если хотя бы один аргумент имеет строковый тип, как показано в [Таблице 9.9](#). В других случаях можно использовать явное преобразование в `text`, чтобы оператор принял нестроковый аргумент.

Таблица 9.9. Строковые функции и операторы языка SQL

Функция/оператор Описание Пример(ы)
<code>text text → text</code> Соединяет две строки. <code>'Post' 'greSQL' → PostgreSQL</code>
<code>text anynonarray → text</code> <code>anynonarray text → text</code> Преобразует нестроковый аргумент в текст, а затем соединяет две строки. (Нестроковый аргумент не должен быть массивом, иначе возникает неоднозначность с операторами массивов <code> </code> . Если вы хотите соединить строку с текстовой формой массива, явно приведите его к типу <code>text</code> .) <code>'Value: ' 42 → Value: 42</code>
<code>text IS [NOT] [form] NORMALIZED → boolean</code> Проверяет, соответствует ли строка определённой форме нормализации Юникода. Форма указывается в необязательном ключевом слове <i>form</i> : NFC (по умолчанию), NFD, NFKC или NFKD. Это выражение можно использовать, только если кодировка сервера — UTF8. Заметьте, что проверить нормализацию с помощью этого выражения, как правило, будет быстрее, чем нормализовать уже, возможно, нормализованные строки. <code>U&'0061\0308bc' IS NFD NORMALIZED → t</code>
<code>bit_length (text) → integer</code> Возвращает число бит в строке (это число в 8 раз больше <code>octet_length</code>). <code>bit_length('jose') → 32</code>
<code>char_length (text) → integer</code> <code>character_length (text) → integer</code> Возвращает число символов в строке. <code>char_length('josé') → 4</code>
<code>lower (text) → text</code> Переводит символы строки в нижний регистр в соответствии с правилами локали базы данных. <code>lower('TOM') → tom</code>
<code>normalize (text [, form]) → text</code>

Функция/оператор	Описание	Пример(ы)
	Переводит строку в заданную форму нормализации Unicode. Форма указывается в необязательном ключевом слове <i>form</i> : NFC (по умолчанию), NFD, NFKC или NFKD. Эту функцию можно использовать, только если кодировка сервера — UTF8.	<code>normalize(U&'00610308bc', NFC) → U&'00E4bc'</code>
<code>octet_length (text) → integer</code>	Возвращает число байт в строке.	<code>octet_length('josé') → 5</code> (если серверная кодировка — UTF8)
<code>octet_length (character) → integer</code>	Возвращает число байт в строке. Так как эта вариация функции принимает непосредственно тип <i>character</i> , завершающие строку пробелы не обрезаются.	<code>octet_length('abc '::character(4)) → 4</code>
<code>overlay (string text PLACING newsubstring text FROM start integer [FOR count integer]) → text</code>	Заменяет подстроку в <i>string</i> , начиная с символа с номером <i>start</i> , длиной <i>count</i> символов, на подстроку <i>newsubstring</i> . В отсутствие параметра <i>count</i> количество заменяемых символов определяется длиной <i>newsubstring</i> .	<code>overlay('Txxxxas' placing 'hom' from 2 for 4) → Thomas</code>
<code>position (substring text IN string text) → integer</code>	Возвращает начальную позицию первого вхождения <i>substring</i> в строке <i>string</i> либо 0, если такого вхождения нет.	<code>position('om' in 'Thomas') → 3</code>
<code>substring (string text [FROM start integer] [FOR count integer]) → text</code>	Извлекает из <i>string</i> подстроку, начиная с позиции <i>start</i> (если она указана), длиной до <i>count</i> символов (если она указана). Параметры <i>start</i> и <i>count</i> могут опускаться, но не оба сразу.	<code>substring('Thomas' from 2 for 3) → hom</code> <code>substring('Thomas' from 3) → omas</code> <code>substring('Thomas' for 2) → Th</code>
<code>substring (string text FROM pattern text) → text</code>	Извлекает первую подстроку, соответствующую регулярному выражению в стиле POSIX; см. Подраздел 9.7.3 .	<code>substring('Thomas' from '...\$') → mas</code>
<code>substring (string text SIMILAR pattern text ESCAPE escape text) → text</code> <code>substring (string text FROM pattern text FOR escape text) → text</code>	Извлекает первую подстроку, соответствующую регулярному выражению в стиле SQL; см. Подраздел 9.7.2 . Первая форма определена в стандарте, начиная с SQL:2003, а вторая существовала только в стандарте SQL:1999 и её следует считать устаревшей.	<code>substring('Thomas' similar '%#"o_a#"_"' escape '#') → oma</code>
<code>trim ([LEADING TRAILING BOTH] [characters text] FROM string text) → text</code>	Удаляет наибольшую подстроку, содержащую только символы <i>characters</i> (по умолчанию пробелы), с начала, с конца или с обеих сторон (BOTH, по умолчанию) строки <i>string</i> .	<code>trim(both 'xyz' from 'yxTomxx') → Tom</code>
<code>trim ([LEADING TRAILING BOTH] [FROM] string text [, characters text]) → text</code>	Это нестандартный синтаксис вызова <code>trim()</code> .	<code>trim(both from 'yxTomxx', 'xyz') → Tom</code>

Функция/оператор	Описание	Пример(ы)
<code>upper (text)</code>	→ text	Переводит символы строки в верхний регистр, в соответствии с правилами локали базы данных.
		<code>upper('tom') → TOM</code>

Кроме этого, в PostgreSQL есть и другие функции для работы со строками, перечисленные в [Таблице 9.10](#). (Некоторые из них используются в качестве внутренней реализации стандартных строковых функций SQL, приведённых в [Таблице 9.9](#).) Также имеются операторы сопоставления шаблонов, описанные в [Разделе 9.7](#), и операторы полнотекстового поиска, описанные в [Главе 12](#).

Таблица 9.10. Другие строковые функции и операторы

Функция/оператор	Описание	Пример(ы)
<code>text ^@ text</code>	→ boolean	Возвращает true, если первая строка начинается со второй строки (равнозначно функции <code>starts_with()</code>).
		<code>'alphabet' ^@ 'alph' → t</code>
<code>ascii (text)</code>	→ integer	Возвращает числовой код первого символа аргумента. Для UTF8 возвращает код символа в Unicode. Для других многобайтных кодировок аргумент должен быть ASCII-символом.
		<code>ascii('x') → 120</code>
<code>btrim (string text [, characters text])</code>	→ text	Удаляет наибольшую подстроку, содержащую только символы <i>characters</i> (по умолчанию пробел), с начала и с конца строки <i>string</i> .
		<code>btrim('yxtrimyyx', 'xyz') → trim</code>
<code>chr (integer)</code>	→ text	Возвращает символ с данным кодом. Для UTF8 аргумент воспринимается как код символа Unicode, а для других кодировок он должен указывать на ASCII-символ. Эта функция не может выдать <code>chr(0)</code> , так как данный символ нельзя сохранить в текстовых типах данных.
		<code>chr(65) → A</code>
<code>concat (val1 "any" [, val2 "any" [, ...]])</code>	→ text	Соединяет текстовые представления всех аргументов, игнорируя NULL.
		<code>concat('abcde', 2, NULL, 22) → abcde222</code>
<code>concat_ws (sept text, val1 "any" [, val2 "any" [, ...]])</code>	→ text	Соединяет вместе все аргументы, кроме первого, через разделитель. Разделитель задаётся в первом аргументе и должен быть отличен от NULL. В других аргументах значение NULL игнорируется.
		<code>concat_ws(',', 'abcde', 2, NULL, 22) → abcde,2,22</code>
<code>format (formatstr text [, formatarg "any" [, ...]])</code>	→ text	Форматирует аргументы в соответствии со строкой формата; см. Подраздел 9.4.1 . Эта функция работает подобно <code>sprintf</code> в языке C.
		<code>format('Hello %s, %1\$s', 'World') → Hello World, World</code>
<code>initcap (text)</code>	→ text	

Функция/оператор	Описание	Пример(ы)
	Переводит первую букву каждого слова в строке в верхний регистр, а остальные — в нижний. Словами считаются последовательности алфавитно-цифровых символов, разделённые любыми другими символами.	<code>initcap('hi THOMAS') → Hi Thomas</code>
	<code>left (string text, n integer) → text</code> Возвращает первые <i>n</i> символов в строке. Когда <i>n</i> меньше нуля, возвращаются все символы слева, кроме последних $ n $.	<code>left('abcde', 2) → ab</code>
	<code>length (text) → integer</code> Возвращает число символов в строке.	<code>length('jose') → 4</code>
	<code>lpad (string text, length integer [, fill text]) → text</code> Дополняет строку <i>string</i> слева до длины <i>length</i> символами <i>fill</i> (по умолчанию пробелами). Если длина строки уже больше заданной, она обрезается справа.	<code>lpad('hi', 5, 'xy') → xyxhi</code>
	<code>ltrim (string text [, characters text]) → text</code> Удаляет наибольшую подстроку, содержащую только символы <i>characters</i> (по умолчанию пробелы), с начала строки <i>string</i> .	<code>ltrim('zzzytest', 'xyz') → test</code>
	<code>md5 (text) → text</code> Вычисляет MD5-хеш аргумента и выдаёт результат в шестнадцатеричном виде.	<code>md5('abc') → 900150983cd24fb0d6963f7d28e17f72</code>
	<code>parse_ident (qualified_identifier text [, strict_mode boolean DEFAULT true]) → text[]</code> Раскладывает полный идентификатор, задаваемый параметром <i>qualified_identifier</i> , на массив идентификаторов, удаляя кавычки, обрамляющие отдельные идентификаторы. По умолчанию лишние символы после последнего идентификатора вызывают ошибку, но если отключить строгий режим (передать во втором параметре <i>false</i>), такие символы игнорируются. (Это поведение полезно для разбора имён таких объектов, как функции.) Заметьте, что эта функция не усекает чрезмерно длинные идентификаторы. Если вы хотите получить усечённые имена, можно привести результат к <code>name[]</code> .	<code>parse_ident('"SomeSchema".someTable') → {SomeSchema,sometable}</code>
	<code>pg_client_encoding () → name</code> Возвращает имя текущей клиентской кодировки.	<code>pg_client_encoding() → UTF8</code>
	<code>quote_ident (text) → text</code> Преобразует аргумент в строку, подходящую для использования в качестве идентификатора в SQL-операторе. При необходимости идентификатор заключается в кавычки (например, если он содержит символы, недопустимые в открытом виде, или буквы в разных регистрах). Если переданная строка содержит кавычки, они дублируются. См. также Пример 42.1 .	<code>quote_ident('Foo bar') → "Foo bar"</code>
	<code>quote_literal (text) → text</code> Преобразует аргумент в строку, подходящую для использования в качестве текстовой константы в SQL-операторе. Внутренние символы апостроф и обратная косая черта при этом дублируются. Заметьте, что <code>quote_literal</code> возвращает NULL, когда на вход ей пе-	

Функция/оператор	Описание	Пример(ы)
	редаётся строка NULL; если же нужно получить представление и такого аргумента, лучше использовать <code>quote_nullable</code> . См. также Пример 42.1 .	<code>quote_literal(E'О\''Reilly')</code> → <code>'О\''Reilly'</code>
<code>quote_literal (anyelement)</code>	→ text Переводит данное значение в текстовый вид и заключает в апострофы как текстовую строку. Символы апостроф и обратная косая черта при этом дублируются.	<code>quote_literal(42.5)</code> → <code>'42.5'</code>
<code>quote_nullable (text)</code>	→ text Преобразует аргумент в строку, подходящую для использования в качестве текстовой константы в SQL-операторе; при этом для аргумента NULL возвращается строка NULL. Символы апостроф и обратная косая черта дублируются должным образом. См. также Пример 42.1 .	<code>quote_nullable(NULL)</code> → NULL
<code>quote_nullable (anyelement)</code>	→ text Переводит данное значение в текстовый вид и заключает в апострофы как текстовую строку, при этом для аргумента NULL возвращается строка NULL. Символы апостроф и обратная косая черта дублируются должным образом.	<code>quote_nullable(42.5)</code> → <code>'42.5'</code>
<code>regexp_count (string text, pattern text [, start integer [, flags text]])</code>	→ integer Возвращает количество вхождений регулярного выражения POSIX <i>pattern</i> в строке <i>string</i> ; см. Подраздел 9.7.3 .	<code>regexp_count('123456789012', '\d\d\d', 2)</code> → 3
<code>regexp_instr (string text, pattern text [, start integer [, N integer [, endoption integer [, flags text [, subexpr integer]]]]])</code>	→ integer Возвращает позицию в строке <i>string</i> <i>N</i> -го вхождения шаблона <i>pattern</i> , содержащего регулярное выражение POSIX, или ноль, если такого вхождения нет; см. Подраздел 9.7.3 .	<code>regexp_instr('ABCDEF', 'c(.)(..)', 1, 1, 0, 'i')</code> → 3 <code>regexp_instr('ABCDEF', 'c(.)(..)', 1, 1, 0, 'i', 2)</code> → 5
<code>regexp_like (string text, pattern text [, flags text])</code>	→ boolean Проверяет, есть ли в строке <i>string</i> вхождение регулярного выражения POSIX <i>pattern</i> ; см. Подраздел 9.7.3 .	<code>regexp_like('Hello World', 'world\$', 'i')</code> → t
<code>regexp_match (string text, pattern text [, flags text])</code>	→ text [] Возвращает подстроки из первого вхождения регулярного выражения POSIX <i>pattern</i> в строке <i>string</i> ; см. Подраздел 9.7.3 .	<code>regexp_match('foobarbequebaz', '(bar) (beque)')</code> → {bar, beque}
<code>regexp_matches (string text, pattern text [, flags text])</code>	→ set of text [] Возвращает подстроки из первого вхождения регулярного выражения POSIX <i>pattern</i> в строке <i>string</i> или подстроки из всех таких вхождений, если используется флаг <i>g</i> ; см. Подраздел 9.7.3 .	<code>regexp_matches('foobarbequebaz', 'ba.', 'g')</code> → {bar} {baz}
<code>regexp_replace (string text, pattern text, replacement text [, start integer [, flags text]])</code>	→ text	

Функция/оператор	Описание	Пример(ы)
	Заменяет подстроку первого вхождения регулярного выражения POSIX <i>pattern</i> или подстроки всех таких вхождений, если использован флаг <i>g</i> ; см. Подраздел 9.7.3 .	<code>regex_replace('Thomas', '[mN]a.', 'M')</code> → <code>ThM</code>
<code>regex_replace (string text, pattern text, replacement text, start integer, N integer [, flags text])</code>	→ <code>text</code> Заменяет подстроку <i>N</i> -го вхождения регулярного выражения POSIX <i>pattern</i> в строке <i>string</i> или подстроки всех таких вхождений, если <i>N</i> равен нулю; см. Подраздел 9.7.3 .	<code>regex_replace('Thomas', '.', 'X', 3, 2)</code> → <code>ThoXas</code>
<code>regex_split_to_array (string text, pattern text [, flags text])</code>	→ <code>text[]</code> Разделяет содержимое <i>string</i> на элементы, используя в качестве разделителя регулярное выражение POSIX, и выдаёт массив результатов; см. Подраздел 9.7.3 .	<code>regex_split_to_array('hello world', '\s+')</code> → <code>{hello, world}</code>
<code>regex_split_to_table (string text, pattern text [, flags text])</code>	→ <code>setof text</code> Разделяет содержимое <i>string</i> на элементы, используя в качестве разделителя регулярное выражение POSIX, и выдаёт набор результатов; см. Подраздел 9.7.3 .	<code>regex_split_to_table('hello world', '\s+')</code> → <code>hello</code> <code>world</code>
<code>regex_substr (string text, pattern text [, start integer [, N integer [, flags text [, subexpr integer]]]])</code>	→ <code>text</code> Возвращает подстроку <i>N</i> -го вхождения регулярного выражения POSIX <i>pattern</i> в строке <i>string</i> или NULL, если таких вхождений не нашлось; см. Подраздел 9.7.3 .	<code>regex_substr('ABCDEF', 'c(.)(..)', 1, 1, 'i')</code> → <code>CDEF</code> <code>regex_substr('ABCDEF', 'c(.)(..)', 1, 1, 'i', 2)</code> → <code>EF</code>
<code>repeat (string text, number integer)</code>	→ <code>text</code> Повторяет содержимое <i>string</i> указанное число (<i>number</i>) раз.	<code>repeat('Pg', 4)</code> → <code>PgPgPgPg</code>
<code>replace (string text, from text, to text)</code>	→ <code>text</code> Заменяет все вхождения в <i>string</i> подстроки <i>from</i> подстрокой <i>to</i> .	<code>replace('abcdefabcdef', 'cd', 'XX')</code> → <code>abXXefabXXef</code>
<code>reverse (text)</code>	→ <code>text</code> Переставляет символы в строке в обратном порядке.	<code>reverse('abcde')</code> → <code>edcba</code>
<code>right (string text, n integer)</code>	→ <code>text</code> Возвращает последние <i>n</i> символов в строке. Когда <i>n</i> меньше нуля, возвращаются все символы справа, кроме первых <i> n </i> .	<code>right('abcde', 2)</code> → <code>de</code>
<code>rpadd (string text, length integer [, fill text])</code>	→ <code>text</code> Дополняет строку <i>string</i> справа до длины <i>length</i> символами <i>fill</i> (по умолчанию пробелами). Если длина строки уже больше заданной, она обрезается.	<code>rpadd('hi', 5, 'xy')</code> → <code>hixyx</code>
<code>rtrim (string text [, characters text])</code>	→ <code>text</code> Удаляет наибольшую подстроку, содержащую только символы <i>characters</i> (по умолчанию пробелы), с конца строки <i>string</i> .	

Функция/оператор	Описание	Пример(ы)
		<code>rtrim('testxxxz', 'xyz') → test</code>
	<code>split_part (string text, delimiter text, n integer) → text</code> Разделяет строку <i>string</i> по символу <i>delimiter</i> и выдаёт <i>n</i> -й элемент (считая с 1) или <i>n</i> -й с конца, если <i>n</i> отрицательный.	<code>split_part('abc~@~def~@~ghi', '~@~', 2) → def</code> <code>split_part('abc,def,ghi,jkl', ',', -2) → ghi</code>
	<code>starts_with (string text, prefix text) → boolean</code> Возвращает true, если строка <i>string</i> начинается с подстроки <i>prefix</i> .	<code>starts_with('alphabet', 'alph') → t</code>
	<code>string_to_array (string text, delimiter text [, null_string text]) → text[]</code> Разделяет заданную параметром <i>string</i> строку на поля по разделителю <i>delimiter</i> и формирует из полученных полей массив значений <i>text</i> . Если в качестве <i>delimiter</i> передаётся NULL, каждый символ <i>string</i> становится отдельным элементом массива. Если в <i>delimiter</i> передаётся пустая строка, вся строка <i>string</i> воспринимается как одно поле и помещается в один элемент массива. Если в аргументе <i>null_string</i> передана отличная от NULL строка, поля, совпадающие с этой строкой, заменяются значениями NULL. См. также array_to_string .	<code>string_to_array('xx~yy~zz', '~', 'yy') → {xx,NULL,zz}</code>
	<code>string_to_table (string text, delimiter text [, null_string text]) → setof text</code> Разделяет заданную параметром <i>string</i> строку на поля по разделителю <i>delimiter</i> и возвращает полученные поля как набор строк типа <i>text</i> . Если в качестве <i>delimiter</i> передаётся NULL, каждый символ <i>string</i> становится отдельной строкой результата. Если в <i>delimiter</i> передаётся пустая строка, вся строка <i>string</i> воспринимается как одно поле. Если в аргументе <i>null_string</i> передана отличная от NULL строка, совпадающие с ней поля заменяются значениями NULL.	<code>string_to_table('xx~^yy~^zz', '~^', 'yy') →</code> xx NULL zz
	<code>strpos (string text, substring text) → integer</code> Возвращает начальную позицию первого вхождения <i>substring</i> в строке <i>string</i> либо 0, если такого вхождения нет. (Ей подобна функция <code>position(substring in string)</code> , но обратите внимание на другой порядок аргументов.)	<code>strpos('high', 'ig') → 2</code>
	<code>substr (string text, start integer [, count integer]) → text</code> Извлекает из <i>string</i> подстроку, начиная с позиции <i>start</i> , длиной до <i>count</i> символов (если это значение указано). (Ей равнозначна функция <code>substring(string from start for count)</code> .)	<code>substr('alphabet', 3) → phabet</code> <code>substr('alphabet', 3, 2) → ph</code>
	<code>to_ascii (string text) → text</code> <code>to_ascii (string text, encoding name) → text</code> <code>to_ascii (string text, encoding integer) → text</code> Преобразует <i>string</i> в ASCII из другой кодировки, задаваемой по имени или номеру. В отсутствие указания <i>encoding</i> подразумевается кодировка базы данных (и на практике это единственный полезный вариант использования). В основном суть преобразования	

Функция/оператор	Описание	Пример(ы)
	сводится к отбрасыванию диакритических знаков. Это преобразование поддерживается только для кодировок <code>LATIN1</code> , <code>LATIN2</code> , <code>LATIN9</code> и <code>WIN1250</code> . (Другое, более гибкое решение реализовано в модуле <code>unaccent</code>).	<code>to_ascii('Karél')</code> → <code>Karel</code>
<code>to_hex (integer)</code>	→ <code>text</code>	
<code>to_hex (bigint)</code>	→ <code>text</code>	
	Преобразует число в шестнадцатеричное представление.	<code>to_hex(2147483647)</code> → <code>7fffffff</code>
<code>translate (string text, from text, to text)</code>	→ <code>text</code>	
	Заменяет каждый символ в <i>string</i> , входящий в множество <i>from</i> , на соответствующий символ в множестве <i>to</i> . Если строка <i>from</i> длиннее <i>to</i> , найденные в исходной строке лишние символы <i>from</i> удаляются.	<code>translate('12345', '143', 'ax')</code> → <code>a2x5</code>
<code>unistr (text)</code>	→ <code>text</code>	
	Преобразует спецпоследовательности Unicode в аргументе. Символы Unicode могут быть заданы как <code>\xxxx</code> (4 шестнадцатеричных цифры), <code>\+xxxxxx</code> (6 шестнадцатеричных цифр), <code>\uxxxx</code> (4 шестнадцатеричных цифры) или <code>\Uxxxxxxxx</code> (8 шестнадцатеричных цифр). Чтобы указать обратную косую черту, напишите две обратные косые черты. Все остальные символы берутся буквально. Когда кодировка сервера — не UTF-8, символ с кодом, заданным этой спецпоследовательностью, преобразуется в фактическую кодировку сервера; если такое преобразование невозможно, выдаётся ошибка. Эта функция предоставляет (нестандартную) альтернативу строковым константам со спецпоследовательностями Unicode (см. Подраздел 4.1.2.3).	<code>unistr('d\0061t\+000061')</code> → <code>data</code> <code>unistr('d\u0061t\U00000061')</code> → <code>data</code>

Функции `concat`, `concat_ws` и `format` принимают переменное число аргументов, так что им для объединения или форматирования можно передавать значения в виде массива, помеченного ключевым словом `VARIADIC` (см. [Подраздел 37.5.6](#)). Элементы такого массива обрабатываются, как если бы они были обычными аргументами функции. Если вместо массива в соответствующем аргументе передаётся `NULL`, функции `concat` и `concat_ws` возвращают `NULL`, а `format` воспринимает `NULL` как массив нулевого размера.

Также обратите внимание на агрегатную функцию `string_agg` в [Разделе 9.21](#) и функции для преобразования текста в `bytea` и наоборот в [Таблице 9.13](#).

9.4.1. format

Функция `format` выдаёт текст, отформатированный в соответствии со строкой формата, подобно функции `sprintf` в C.

```
format(formatstr text [, formatarg "any" [, ...] ])
```

formatstr — строка, определяющая, как будет форматироваться результат. Обычный текст в строке формата непосредственно копируется в результат, за исключением *спецификаторов формата*. Спецификаторы формата представляют собой местозаполнители, определяющие, как должны форматироваться и выводиться в результате аргументы функции. Каждый аргумент *formatarg* преобразуется в текст по правилам вывода своего типа данных, а затем форматруется и вставляется в результирующую строку согласно спецификаторам формата.

Спецификаторы формата предваряются символом `%` и имеют форму

```
%[position][flags][width]type
```

Здесь:

position (необязательный)

Строка вида *n\$*, где *n* — индекс выводимого аргумента. Индекс, равный 1, выбирает первый аргумент после *formatstr*. Если *position* опускается, по умолчанию используется следующий аргумент по порядку.

flags (необязательный)

Дополнительные флаги, управляющие форматированием данного спецификатора. В настоящее время поддерживается только знак минус (-), который выравнивает результат спецификатора по левому краю. Этот флаг работает, только если также определено поле *width*.

width (необязательный)

Задаёт минимальное число символов, которое будет занимать результат данного спецификатора. Выводимое значение выравнивается по правой или левой стороне (в зависимости от флага -) с дополнением необходимым числом пробелов. Если ширина слишком мала, она просто игнорируется, т. е. результат не усекается. Ширину можно обозначить положительным целым, звёздочкой (*), тогда ширина будет получена из следующего аргумента функции, или строкой вида **n\$*, тогда ширина будет задаваться в *n*-м аргументе функции.

Если ширина передаётся в аргументе функции, этот аргумент выбирается до аргумента, используемого для спецификатора. Если аргумент ширины отрицательный, результат выравнивается по левой стороне (как если бы был указан флаг -) в рамках поля длины *abs(width)*.

type (обязательный)

Тип спецификатора определяет преобразование соответствующего выводимого значения. Поддерживаются следующие типы:

- *s* форматирует значение аргумента как простую строку. Значение NULL представляется пустой строкой.
- *I* обрабатывает значение аргумента как SQL-идентификатор, при необходимости заключая его в кавычки. Значение NULL для такого преобразования считается ошибочным (так же, как и для *quote_ident*).
- *L* заключает значение аргумента в апострофы, как строку SQL. Значение NULL выводится буквально, как NULL, без кавычек (так же, как и с *quote_nullable*).

В дополнение к спецификаторам, описанным выше, можно использовать спецпоследовательность *%%*, которая просто выведет символ *%*.

Несколько примеров простых преобразований формата:

```
SELECT format('Hello %s', 'World');
Результат: Hello World
```

```
SELECT format('Testing %s, %s, %s, %%', 'one', 'two', 'three');
Результат: Testing one, two, three, %
```

```
SELECT format('INSERT INTO %I VALUES(%L)', 'Foo bar', E'O\'Reilly');
Результат: INSERT INTO "Foo bar" VALUES('O\'Reilly')
```

```
SELECT format('INSERT INTO %I VALUES(%L)', 'locations', 'C:\Program Files');
Результат: INSERT INTO locations VALUES('C:\Program Files')
```

Следующие примеры иллюстрируют использование поля *width* и флага -:

```
SELECT format('|%10s|', 'foo');
Результат: |          foo|
```

```
SELECT format('|%-10s|', 'foo');
Результат: |foo          |
```

```
SELECT format('|%*s|', 10, 'foo');
```

```
Результат: |          foo|
```

```
SELECT format('|%*s|', -10, 'foo');
```

```
Результат: |foo          |
```

```
SELECT format('|%-*s|', 10, 'foo');
```

```
Результат: |foo          |
```

```
SELECT format('|%-*s|', -10, 'foo');
```

```
Результат: |foo          |
```

Эти примеры показывают применение полей *position*:

```
SELECT format('Testing %3$s, %2$s, %1$s', 'one', 'two', 'three');
```

```
Результат: Testing three, two, one
```

```
SELECT format('|%*2$s|', 'foo', 10, 'bar');
```

```
Результат: |          bar|
```

```
SELECT format('|%1$*2$s|', 'foo', 10, 'bar');
```

```
Результат: |          foo|
```

В отличие от стандартной функции C `sprintf`, функция `format` в Postgres Pro позволяет комбинировать в одной строке спецификаторы с полями *position* и без них. Спецификатор формата без поля *position* всегда использует следующий аргумент после последнего выбранного. Кроме того, функция `format` не требует, чтобы в строке формата использовались все аргументы функции. Пример этого поведения:

```
SELECT format('Testing %3$s, %2$s, %s', 'one', 'two', 'three');
```

```
Результат: Testing three, two, three
```

Спецификаторы формата `%I` и `%L` особенно полезны для безопасного составления динамических операторов SQL. См. [Пример 42.1](#).

9.5. Функции и операторы двоичных строк

В этом разделе описываются функции и операторы для работы с двоичными строками, то есть со значениями типа `bytea`. Многие из них идентичны по функциональности и синтаксису описанным в предыдущем разделе функциям, предназначенным для текстовых строк.

В SQL определены несколько строковых функций, в которых аргументы разделяются не запятыми, а ключевыми словами. Подробнее это описано в [Таблице 9.11](#). Postgres Pro также предоставляет варианты этих функций с синтаксисом, обычным для функций (см. [Таблицу 9.12](#)).

Таблица 9.11. SQL-функции и операторы для работы с двоичными строками

Функция/оператор	Описание	Пример(ы)
<code>bytea bytea</code>	<code>→ bytea</code> Соединяет две двоичные строки.	<code>'\x123456'::bytea '\x789a00bcde'::bytea</code> <code>→ \x123456789a00bcde</code>
<code>bit_length (bytea)</code>	<code>→ integer</code> Возвращает число бит в двоичной строке (это число в 8 раз больше <code>octet_length</code>).	<code>bit_length('\x123456'::bytea)</code> <code>→ 24</code>
<code>octet_length (bytea)</code>	<code>→ integer</code> Возвращает число байт в двоичной строке.	

Функция/оператор	Описание	Пример(ы)
	<code>octet_length('x123456'::bytea)</code>	<code>→ 3</code>
	<code>overlay (bytes bytea PLACING newsubstring bytea FROM start integer [FOR count integer]) → bytea</code> Заменяет подстроку в <i>bytes</i> , начиная с байта с номером <i>start</i> , длиной <i>count</i> байт, на подстроку <i>newsubstring</i> . В отсутствие параметра <i>count</i> количество заменяемых байтов определяется длиной <i>newsubstring</i> .	<code>overlay('x1234567890'::bytea placing '\002\003'::bytea from 2 for 3) → x12020390</code>
	<code>position (substring bytea IN bytes bytea) → integer</code> Возвращает начальную позицию первого вхождения <i>substring</i> в <i>bytes</i> либо 0, если такого вхождения нет.	<code>position('x5678'::bytea in 'x1234567890'::bytea) → 3</code>
	<code>substring (bytes bytea [FROM start integer] [FOR count integer]) → bytea</code> Извлекает из <i>bytes</i> подстроку, начиная с позиции <i>start</i> (если она указана), длиной до <i>count</i> символов (если она указана). Параметры <i>start</i> и <i>count</i> могут опускаться, но не оба сразу.	<code>substring('x1234567890'::bytea from 3 for 2) → x5678</code>
	<code>trim ([LEADING TRAILING BOTH] bytesremoved bytea FROM bytes bytea) → bytea</code> Удаляет наибольшую строку, содержащую только байты, заданные в параметре <i>bytesremoved</i> , с начала, с конца или с обеих сторон (по умолчанию подразумевается указание BOTH) строки <i>bytes</i> .	<code>trim('x9012'::bytea from 'x1234567890'::bytea) → x345678</code>
	<code>trim ([LEADING TRAILING BOTH] [FROM] bytes bytea, bytesremoved bytea) → bytea</code> Это нестандартный синтаксис вызова <code>trim()</code> .	<code>trim(both from 'x1234567890'::bytea, 'x9012'::bytea) → x345678</code>

В PostgreSQL есть и другие функции для работы с двоичными строками, перечисленные в [Таблице 9.12](#). Некоторые из них используются в качестве внутренней реализации стандартных функций SQL, приведённых в [Таблице 9.11](#).

Таблица 9.12. Другие функции для работы с двоичными строками

Функция	Описание	Пример(ы)
<code>bit_count (bytes bytea) → bigint</code>	Возвращает число бит в двоичной строке (эту операцию также называют «popcount»).	<code>bit_count('x1234567890'::bytea) → 15</code>
<code>btrim (bytes bytea, bytesremoved bytea) → bytea</code>	Удаляет наибольшую строку, содержащую только байты, заданные в параметре <i>bytesremoved</i> , с начала и с конца строки <i>bytes</i> .	<code>btrim('x1234567890'::bytea, 'x9012'::bytea) → x345678</code>
<code>get_bit (bytes bytea, n bigint) → integer</code>	Извлекает из двоичной строки бит с номером <i>n</i> .	<code>get_bit('x1234567890'::bytea, 30) → 1</code>
<code>get_byte (bytes bytea, n integer) → integer</code>	Извлекает из двоичной строки байт с номером <i>n</i> .	

Функция	Описание	Пример(ы)
		<code>get_byte('\x1234567890'::bytea, 4) → 144</code>
	<code>length (bytea) → integer</code> Выдаёт число байт в двоичной строке.	<code>length('\x1234567890'::bytea) → 5</code>
	<code>length (bytes bytea, encoding name) → integer</code> Выдаёт число символов в двоичной строке, в предположении, что она содержит текст в кодировке <i>encoding</i> .	<code>length('jose'::bytea, 'UTF8') → 4</code>
	<code>ltrim (bytes bytea, bytesremoved bytea) → bytea</code> Удаляет наибольшую строку, содержащую только байты, заданные в параметре <i>bytesremoved</i> , с начала строки <i>bytes</i> .	<code>ltrim('\x1234567890'::bytea, '\x9012'::bytea) → \x34567890</code>
	<code>md5 (bytea) → text</code> Вычисляет MD5-хеш двоичной строки и выдаёт результат в шестнадцатеричном виде.	<code>md5('Th\000omas'::bytea) → 8ab2d3c9689aaf18b4958c334c82d8b1</code>
	<code>rtrim (bytes bytea, bytesremoved bytea) → bytea</code> Удаляет наибольшую строку, содержащую только байты, заданные в параметре <i>bytesremoved</i> , с конца строки <i>bytes</i> .	<code>rtrim('\x1234567890'::bytea, '\x9012'::bytea) → \x12345678</code>
	<code>set_bit (bytes bytea, n bigint, newvalue integer) → bytea</code> Устанавливает в двоичной строке для бита с номером <i>n</i> значение <i>newvalue</i> .	<code>set_bit('\x1234567890'::bytea, 30, 0) → \x1234563890</code>
	<code>set_byte (bytes bytea, n integer, newvalue integer) → bytea</code> Устанавливает в двоичной строке для байта с номером <i>n</i> значение <i>newvalue</i> .	<code>set_byte('\x1234567890'::bytea, 4, 64) → \x1234567840</code>
	<code>sha224 (bytea) → bytea</code> Вычисляет хеш SHA-224 для двоичной строки.	<code>sha224('abc'::bytea) → \x23097d223405d8228642a477bda2 55b32aadbce4bda0b3f7e36c9da7</code>
	<code>sha256 (bytea) → bytea</code> Вычисляет хеш SHA-256 для двоичной строки.	<code>sha256('abc'::bytea) → \xba7816bf8f01cfea414140de5dae2223 b00361a396177a9cb410ff61f20015ad</code>
	<code>sha384 (bytea) → bytea</code> Вычисляет хеш SHA-384 для двоичной строки.	<code>sha384('abc'::bytea) → \xcb00753f45a35e8bb5a03d699ac65007 272c32ab0eded1631a8b605a43ff5bed8086072ba1e7cc2358baeca134c825a7</code>
	<code>sha512 (bytea) → bytea</code> Вычисляет хеш SHA-512 для двоичной строки.	<code>sha512('abc'::bytea) → \xddaf35a193617abacc417349ae204131 12e6fa4e89a97ea20a9eeee64b55d39a2192992a274fc1a836ba3c23a3feebbd 454d4423643ce80e2a9ac94fa54ca49f</code>
	<code>substr (bytes bytea, start integer [, count integer]) → bytea</code>	

Функция	Описание	Пример(ы)
	Извлекает из <i>bytes</i> подстроку, начиная с позиции <i>start</i> , длиной до <i>count</i> байт (если это значение указано). (Ей равнозначна функция <code>substring(bytes from start for count)</code> .)	<code>substr('\x1234567890'::bytea, 3, 2) → \x5678</code>

Для функций `get_byte` и `set_byte` байты нумеруются с 0. Функции `get_bit` и `set_bit` нумеруют биты справа налево; например, бит 0 будет меньшим значащим битом первого байта, а бит 15 — большим значащим битом второго байта.

По историческим причинам функция `md5` возвращает значение в шестнадцатеричном виде в типе `text`, тогда как функции `SHA-2` возвращают тип `bytea`. Для преобразования значения из одного представления в другое используйте функции `encode` и `decode`. Например, вызвав `encode(sha256('abc'), 'hex')`, вы получите значение в шестнадцатеричном виде в текстовой строке, а `decode(md5('abc'), 'hex')` выдаст значение `bytea`.

В [Таблице 9.13](#) показаны функции для перекодирования текста из одного набора символов (кодировки) в другой и для представления произвольных двоичных данных в текстовом виде. Для всех этих функций аргумент или результат типа `text` содержит текст в текущей кодировке базы данных, тогда как аргументы или результаты типа `bytea` содержат текст в кодировке, заданной соответствующим аргументом.

Таблица 9.13. Функции для преобразования текстовых/двоичных строк

Функция	Описание	Пример(ы)
<code>convert (bytes bytea, src_encoding name, dest_encoding name) → bytea</code>	Преобразует двоичную строку, содержащую текст в кодировке <i>src_encoding</i> , в двоичную строку с текстом в кодировке <i>dest_encoding</i> (возможные варианты преобразований описаны в Подразделе 22.3.4).	<code>convert('text_in_utf8', 'UTF8', 'LATIN1') → \x746578745f696e5f75746638</code>
<code>convert_from (bytes bytea, src_encoding name) → text</code>	Преобразует двоичную строку, содержащую текст в кодировке <i>src_encoding</i> , в строку типа <code>text</code> в кодировке базы данных (возможные варианты преобразований описаны в Подразделе 22.3.4).	<code>convert_from('text_in_utf8', 'UTF8') → text_in_utf8</code>
<code>convert_to (string text, dest_encoding name) → bytea</code>	Преобразует строку типа <code>text</code> в кодировке базы данных в двоичную строку с текстом в кодировке <i>dest_encoding</i> (возможные варианты преобразований описаны в Подразделе 22.3.4).	<code>convert_to('некоторый_текст', 'UTF8') → \x73666d655f74657874</code>
<code>encode (bytes bytea, format text) → text</code>	Переводит двоичные данные в текстовое представление; поддерживаются следующие значения <i>format</i> : <code>base64</code> , <code>escape</code> , <code>hex</code> .	<code>encode('123\000\001', 'base64') → MTIzAAE=</code>
<code>decode (string text, format text) → bytea</code>	Переводит двоичные данные из текстового представления; поддерживает те же значения <i>format</i> , что и функция <code>encode</code> .	<code>decode('MTIzAAE=', 'base64') → \x3132330001</code>

Функции `encode` и `decode` поддерживают следующие текстовые форматы:

base64

Формат `base64` описан в [RFC 2045, Разделе 6.8](#). Согласно этому RFC, закодированные строки разбиваются по 76 символов. Однако завершаются строки не символами CRLF (как требуется в соответствии с MIME), а одним символом конца строки. Функция `decode`, с другой стороны, игнорирует символы перевода каретки, новой строки, пробелы и табуляции. Если на вход `decode` поступают некорректные данные `base64`, возникает ошибка — в том числе, если оказывается некорректным завершающее выравнивание.

escape

В формате `escape` нулевые байты и байты с установленным старшим битом переводятся в восьмеричные спецпоследовательности (`\nnn`), а обратная косая черта дублируется. Другие байтовые значения представляются в буквальном виде. Функция `decode` выдаст ошибку, встретив обратную косую черту, за которой не следует ещё одна обратная косая или три восьмеричных цифры; другие значения байта она принимает без изменений.

hex

В формате `hex` каждые 4 бита данных представляются одной шестнадцатеричной цифрой, от 0 до f, при этом первой идёт цифра, представляющая старшие биты. Шестнадцатеричные цифры a-f функция `encode` выводит в нижнем регистре. Так как наименьшая единица данных — байт (8 бит), функция `encode` всегда возвращает чётное количество символов. Функция `decode`, с другой стороны, принимает символы a-f в любом регистре. Если на вход функции `decode` поступают некорректные данные, возникает ошибка — в том числе, если число символов оказывается нечётным.

См. также агрегатную функцию `string_agg` в [Разделе 9.21](#) и функции для работы с большими объектами в [Разделе 34.4](#).

9.6. Функции и операторы для работы с битовыми строками

В этом разделе описываются функции и операторы, предназначенные для работы с битовыми строками, то есть с данными типов `bit` и `bit varying`. (Хотя в этих таблицах упоминается только тип `bit`, с ним полностью взаимозаменяем тип `bit varying`.) Для битовых строк поддерживаются обычные операторы сравнения, показанные в [Таблице 9.1](#), а также операторы, приведённые в [Таблице 9.14](#).

Таблица 9.14. Операторы для работы с битовыми строками

Оператор Описание Пример(ы)
<code>bit bit → bit</code> Конкатенация <code>B'10001' B'011' → 10001011</code>
<code>bit & bit → bit</code> Битовое И (операнды должны быть одинаковой длины) <code>B'10001' & B'01101' → 00001</code>
<code>bit bit → bit</code> Битовое ИЛИ (операнды должны быть одинаковой длины) <code>B'10001' B'01101' → 11101</code>
<code>bit # bit → bit</code> Битовое исключающее ИЛИ (операнды должны быть одинаковой длины) <code>B'10001' # B'01101' → 11100</code>

Оператор Описание Пример(ы)
<code>~ bit → bit</code> Битовое НЕ <code>~ B'10001' → 01110</code>
<code>bit << integer → bit</code> Битовый сдвиг влево (с сохранением длины строки) <code>B'10001' << 3 → 01000</code>
<code>bit >> integer → bit</code> Битовый сдвиг вправо (с сохранением длины строки) <code>B'10001' >> 2 → 00100</code>

Некоторые функции, работающие с двоичными строками, также работают и с битовыми строками, как показано в [Таблице 9.15](#).

Таблица 9.15. Функции для работы с битовыми строками

Функция Описание Пример(ы)
<code>bit_count (bit) → bigint</code> Возвращает число бит в битовой строке (эту операцию так же называют «popcount»). <code>bit_count(B'10111') → 4</code>
<code>bit_length (bit) → integer</code> Возвращает число бит в битовой строке. <code>bit_length(B'10111') → 5</code>
<code>length (bit) → integer</code> Возвращает число бит в битовой строке. <code>length(B'10111') → 5</code>
<code>octet_length (bit) → integer</code> Возвращает число байт в битовой строке. <code>octet_length(B'1011111011') → 2</code>
<code>overlay (bits bit PLACING newsubstring bit FROM start integer [FOR count integer]) → bit</code> Заменяет подстроку в <i>bits</i> , начиная с бита с номером <i>start</i> , длиной <i>count</i> бит, на подстроку <i>newsubstring</i> . В отсутствие параметра <i>count</i> количество заменяемых бит определяется длиной <i>newsubstring</i> . <code>overlay(B'01010101010101010' placing B'11111' from 2 for 3) → 01111101010101010</code>
<code>position (substring bit IN bits bit) → integer</code> Возвращает начальную позицию первого вхождения <i>substring</i> в <i>bits</i> либо 0, если такого вхождения нет. <code>position(B'010' in B'000001101011') → 8</code>
<code>substring (bits bit [FROM start integer] [FOR count integer]) → bit</code> Извлекает из <i>bits</i> подстроку, начиная с позиции <i>start</i> (если она указана), длиной до <i>count</i> бит (если она указана). Параметры <i>start</i> и <i>count</i> могут опускаться, но не оба сразу. <code>substring(B'110010111111' from 3 for 2) → 00</code>
<code>get_bit (bits bit, n integer) → integer</code>

Функция	Описание	Пример(ы)
	Извлекает из битовой строки бит с номером <i>n</i> ; первый (самый левый) бит имеет номер 0.	<code>get_bit (B'101010101010101010', 6) → 1</code>
	Устанавливает для бита с номером <i>n</i> в битовой строке значение <i>newvalue</i> ; первый (самый левый) бит имеет номер 0.	<code>set_bit (bits bit, n integer, newvalue integer) → bit</code> <code>set_bit (B'101010101010101010', 6, 0) → 101010001010101010</code>

Кроме того, целые значения можно преобразовать в тип `bit` и обратно. Приведение целого к типу `bit (n)` заключается в копировании в это значение правых *n* бит. Когда целое приводится к битовой строке, имеющей длину больше, чем размер этого целого, результат дополняется слева знаком. Некоторые примеры:

```
44::bit(10)          0000101100
44::bit(3)           100
cast(-44 as bit(12)) 111111010100
'1110'::bit(4)::integer 14
```

Заметьте, что приведение к типу «`bit`» без длины будет означать приведение к `bit (1)` и в результате будет получен только один самый младший бит числа.

9.7. Поиск по шаблону

Postgres Pro предлагает три разных способа поиска текста по шаблону: традиционный оператор `LIKE` языка SQL, более современный `SIMILAR TO` (добавленный в SQL:1999) и регулярные выражения в стиле POSIX. Помимо простых операторов, отвечающих на вопрос «соответствует ли строка этому шаблону?», в Postgres Pro есть функции для извлечения или замены соответствующих подстрок и для разделения строки по заданному шаблону.

Подсказка

Если этих встроенных возможностей оказывается недостаточно, вы можете написать собственные функции на языке Perl или Tcl.

Внимание

Хотя чаще всего поиск по регулярному выражению бывает очень быстрым, регулярные выражения бывают и настолько сложными, что их обработка может занять приличное время и объём памяти. Поэтому опасайтесь шаблонов регулярных выражений, поступающих из недоверенных источников. Если у вас нет другого выхода, рекомендуется ввести тайм-аут для операторов.

Поиск с шаблонами `SIMILAR TO` несёт те же риски безопасности, так как конструкция `SIMILAR TO` предоставляет во многом те же возможности, что и регулярные выражения в стиле POSIX.

Поиск с `LIKE` гораздо проще, чем два другие варианта, поэтому его безопаснее использовать с недоверенными источниками шаблонов поиска.

Все три вида операторов поиска по шаблону не поддерживают недетерминированные правила сортировки. В случае необходимости это ограничение можно обойти, применив к выражению другое правило сортировки.

9.7.1. LIKE

строка LIKE шаблон [ESCAPE спецсимвол]
строка NOT LIKE шаблон [ESCAPE спецсимвол]

Выражение `LIKE` возвращает `true`, если строка соответствует заданному шаблону. (Как можно было ожидать, выражение `NOT LIKE` возвращает `false`, когда `LIKE` возвращает `true`, и наоборот. Этому выражению равносильно выражение `NOT (строка LIKE шаблон)`.)

Если шаблон не содержит знаков процента и подчёркиваний, тогда шаблон представляет в точности строку и `LIKE` работает как оператор сравнения. Подчёркивание (`_`) в шаблоне подменяет (вместо него подходит) любой символ; а знак процента (`%`) подменяет любую (в том числе и пустую) последовательность символов.

Несколько примеров:

```
'abc' LIKE 'abc'      true
'abc' LIKE 'a%'       true
'abc' LIKE '_b_'       true
'abc' LIKE 'c'         false
```

При проверке по шаблону `LIKE` всегда рассматривается вся строка. Поэтому, если нужно найти последовательность символов где-то в середине строки, шаблон должен начинаться и заканчиваться знаками процента.

Чтобы найти в строке буквальное вхождение знака процента или подчёркивания, перед соответствующим символом в шаблоне нужно добавить спецсимвол. По умолчанию в качестве спецсимвола выбрана обратная косая черта, но с помощью предложения `ESCAPE` можно выбрать и другой. Чтобы включить спецсимвол в шаблон поиска, продублируйте его.

Примечание

Если параметр `standard_conforming_strings` выключен, каждый символ обратной косой черты, записываемый в текстовой константе, нужно дублировать. Подробнее это описано в [Подразделе 4.1.2.1](#).

Также можно отказаться от спецсимвола, написав `ESCAPE ''`. При этом механизм спецпоследовательностей фактически отключается и использовать знаки процента и подчёркивания буквально в шаблоне нельзя.

Согласно стандарту SQL, отсутствие указания `ESCAPE` означает, что спецсимвол не определён (то есть спецсимволом не будет обратная косая черта), а пустое значение в `ESCAPE` не допускается. Таким образом, в этом поведение Postgres Pro несколько отличается от оговорённого в стандарте.

Вместо `LIKE` можно использовать ключевое слово `ILIKE`, чтобы поиск был регистр-независимым с учётом текущей языковой среды. Этот оператор не описан в стандарте SQL; это расширение Postgres Pro.

Кроме того, в Postgres Pro есть оператор `~~`, равнозначный `LIKE`, и `~~*`, соответствующий `ILIKE`. Есть также два оператора `!~~` и `!~~*`, представляющие `NOT LIKE` и `NOT ILIKE`, соответственно. Все эти операторы относятся к особенностям Postgres Pro. Вы можете увидеть их, например, в выводе команды `EXPLAIN`, так как при разборе запроса проверка `LIKE` и подобные заменяются ими.

Фразы `LIKE`, `ILIKE`, `NOT LIKE` и `NOT ILIKE` в синтаксисе Postgres Pro обычно обрабатываются как операторы; например, их можно использовать в конструкциях *выражение оператор ANY (подвыражение)*, хотя предложение `ESCAPE` здесь добавить нельзя. В некоторых особых случаях всё же может потребоваться использовать вместо них нижележащие операторы.

Также обратите внимание на оператор проверки начала строки `^@` и соответствующую функцию `starts_with()`, которые полезны в случаях, когда нужно произвести сопоставление только с началом строки.

9.7.2. Регулярные выражения `SIMILAR TO`

строка `SIMILAR TO` *шаблон* [`ESCAPE` *спецсимвол*]
строка `NOT SIMILAR TO` *шаблон* [`ESCAPE` *спецсимвол*]

Оператор `SIMILAR TO` возвращает `true` или `false` в зависимости от того, соответствует ли данная строка шаблону или нет. Он работает подобно оператору `LIKE`, только его шаблоны соответствуют определению регулярных выражений в стандарте SQL. Регулярные выражения SQL представляют собой любопытный гибрид синтаксиса `LIKE` с синтаксисом обычных регулярных выражений (POSIX).

Как и `LIKE`, условие `SIMILAR TO` истинно, только если шаблон соответствует всей строке; это отличается от условий с регулярными выражениями, в которых шаблон может соответствовать любой части строки. Также подобно `LIKE`, `SIMILAR TO` воспринимает символы `_` и `%` как знаки подстановки, подменяющие любой один символ или любую подстроку, соответственно (в регулярных выражениях POSIX им аналогичны символы `.` и `*`).

Помимо средств описания шаблонов, позаимствованных от `LIKE`, `SIMILAR TO` поддерживает следующие метасимволы, унаследованные от регулярных выражений POSIX:

- `|` означает выбор (одного из двух вариантов).
- `*` означает повторение предыдущего элемента 0 и более раз.
- `+` означает повторение предыдущего элемента 1 и более раз.
- `?` означает вхождение предыдущего элемента 0 или 1 раз.
- `{m}` означает повторяет предыдущего элемента ровно *m* раз.
- `{m,}` означает повторение предыдущего элемента *m* или более раз.
- `{m,n}` означает повторение предыдущего элемента не менее чем *m* и не более чем *n* раз.
- Скобки `()` объединяют несколько элементов в одну логическую группу.
- Квадратные скобки `[...]` обозначают класс символов так же, как и в регулярных выражениях POSIX.

Обратите внимание, точка `.` не является метасимволом для оператора `SIMILAR TO`.

Как и с `LIKE`, обратная косая черта отменяет специальное значение любого из этих метасимволов. Предложение `ESCAPE` позволяет выбрать другой спецсимвол, а запись `ESCAPE ''` — отказаться от использования спецсимвола.

Согласно стандарту SQL, отсутствие указания `ESCAPE` означает, что спецсимвол не определён (то есть спецсимволом не будет обратная косая черта), а пустое значение в `ESCAPE` не допускается. Таким образом, в этом поведении Postgres Pro несколько отличается от оговорённого в стандарте.

Ещё одно отклонение от стандарта состоит в том, что следующая за спецсимволом буква или цифра открывает возможности спецпоследовательностей, определённых для регулярных выражений POSIX; см. [Таблицу 9.20](#), [Таблицу 9.21](#) и [Таблицу 9.22](#).

Несколько примеров:

```
'abc' SIMILAR TO 'abc'           true
'abc' SIMILAR TO 'a'             false
'abc' SIMILAR TO '%(b|d)%'       true
'abc' SIMILAR TO '(b|c)%'        false
'-abc-' SIMILAR TO '%\mabc\M%'   true
```

```
'xabcy' SIMILAR TO '%\mabc\M%' false
```

Функция `substring` с тремя параметрами производит извлечение подстроки, соответствующей шаблону регулярного выражения SQL. Эту функцию можно вызвать в стандартном синтаксисе SQL:

```
substring(строка similar шаблон escape спецсимвол)
```

или используя синтаксис уже устаревшего стандарта SQL:1999:

```
substring(строка from шаблон for спецсимвол)
```

или в виде обычной функции с тремя аргументами:

```
substring(строка, шаблон, спецсимвол)
```

Как и с `SIMILAR TO`, указанному шаблону должна соответствовать вся строка; в противном случае функция не найдёт ничего и вернёт `NULL`. Для выделения в шаблоне границ подстроки, которая представляет интерес в соответствующей этому шаблону входной строке, шаблон может содержать два спецсимвола с кавычками (") после каждого. В случае успешного обнаружения шаблона эта функция возвращает часть строки, заключённую между этими разделителями.

Разделители «спецсимвол+кавычки» фактически разделяют шаблон `substring` на три независимых регулярных выражения, так что, например, вертикальная черта (|) в любой из этих трёх частей действует только в рамках этой части. Кроме того, в случае неоднозначности в выборе подстрок, соответствующих этим частям, первое и третье регулярные выражения считаются захватывающими наименьшие возможные подстроки. (На языке POSIX можно сказать, что первое и третье регулярные выражения — «нежадные».)

В качестве расширения стандарта SQL в Postgres Pro допускается указание только одного разделителя, в этом случае третье регулярное выражение считается пустым; также разделители могут отсутствовать вовсе, в этом случае пустыми считаются первое и третье регулярные выражения.

Несколько примеров с маркерами `#`, выделяющими возвращаемую строку:

```
substring('foobar' similar '%#"o_b#"%' escape '#') oob
substring('foobar' similar '##"o_b#"%' escape '#') NULL
```

9.7.3. Регулярные выражения POSIX

В [Таблице 9.16](#) перечислены все существующие операторы для проверки строк регулярными выражениями POSIX.

Таблица 9.16. Операторы регулярных выражений

Оператор Описание Пример(ы)
<pre>text ~ text → boolean</pre> Проверка соответствия строки регулярному выражению с учётом регистра <pre>'thomas' ~ 't.*ma' → t</pre>
<pre>text ~* text → boolean</pre> Проверка соответствия строки регулярному выражению без учёта регистра <pre>'thomas' ~* 'T.*ma' → t</pre>
<pre>text !~ text → boolean</pre> Проверка несоответствия строки регулярному выражению с учётом регистра <pre>'thomas' !~ 't.*ma' → t</pre>
<pre>text !~* text → boolean</pre> Проверка несоответствия строки регулярному выражению без учёта регистра

Оператор	Описание	Пример(ы)
		'thomas' !~* 'T.*ma' → f

Регулярные выражения POSIX предоставляют более мощные средства поиска по шаблонам, чем операторы LIKE и SIMILAR TO. Во многих командах Unix, таких как egrep, sed и awk используется язык шаблонов, похожий на описанный здесь.

Регулярное выражение — это последовательность символов, представляющая собой краткое определение набора строк (*регулярное множество*). Строка считается соответствующей регулярному выражению, если она является членом регулярного множества, описываемого регулярным выражением. Как и для LIKE, символы шаблона непосредственно соответствуют символам строки, за исключением специальных символов языка регулярных выражений. При этом спецсимволы регулярных выражений отличается от спецсимволов LIKE. В отличие от шаблонов LIKE, регулярное выражение может совпадать с любой частью строки, если только оно не привязано явно к началу и/или концу строки.

Несколько примеров:

```
'abcd' ~ 'bc'           true
'abcd' ~ 'a.c'          true — точке соответствует любой символ
'abcd' ~ 'a.*d'         true — * обозначает повторение предыдущего элемента шаблона
'abcd' ~ '(b|x)'        true — | означает ИЛИ для группы в скобках
'abcd' ~ '^a'           true — ^ привязывает шаблон к началу строки
'abcd' ~ '^ (b|c)'      false — совпадение не найдено по причине привязки к началу
```

Более подробно язык шаблонов в стиле POSIX описан ниже.

Функция `substring` с двумя параметрами, `substring(строка from шаблон)`, извлекает подстроку, соответствующую шаблону регулярного выражения POSIX. Она возвращает первый фрагмент текста, подходящий шаблону, если таковой находится в строке, либо NULL в противном случае. Но если шаблон содержит скобки, она возвращает первое подвыражение, заключённое в скобки (то, которое начинается с самой первой открывающей скобки). Если вы хотите использовать скобки, но не в таком особом режиме, можно просто заключить в них всё выражение. Если же вам нужно включить скобки в шаблон до подвыражения, которое вы хотите извлечь, это можно сделать, используя группы без захвата, которые будут описаны ниже.

Несколько примеров:

```
substring('foobar' from 'o.b')      oob
substring('foobar' from 'o(.)b')    o
```

Функция `regexp_count` подсчитывает количество мест, где шаблон регулярного выражения POSIX соответствует строке. Она имеет синтаксис `regexp_count(строка, шаблон [, начало [, флаги]])`. Шаблон ищется в строке обычно с начала строки, но если указан параметр *начало*, то начиная с этого индекса. В параметре *флаги* может передаваться текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Например, буква *i* во *флагах* выбирает регистронезависимый режим поиска. Поддерживаемые флаги описаны в [Таблице 9.24](#).

Несколько примеров:

```
regexp_count('ABCABCAxYaxy', 'A.')      3
regexp_count('ABCABCAxYaxy', 'A.', 1, 'i') 4
```

Функция `regexp_instr` возвращает начальную или конечную позицию *N*-го вхождения шаблона регулярного выражения POSIX в строке или ноль, если такого вхождения нет. Она имеет синтаксис `regexp_instr(строка, шаблон [, начало [, N [, выбор_конца [, флаги [, подвыражение]]]])`. Шаблон ищется в строке обычно с начала строки, но если указан параметр *начало*, то начиная с этого индекса. Если указано *N*, то ищется *N*-е вхождение шаблона, иначе ищется первое вхождение. Если пара-

метр *выбор_конца* опущен или равен нулю, функция возвращает позицию первого символа вхождения. Иначе *выбор_конца* должен быть равен одному и функция возвращает позицию символа, следующего за вхождением. В параметре *флаги* может передаваться текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Например, буква *i* во *флагах* выбирает регистронезависимый режим поиска. Поддерживаемые флаги описаны в [Таблице 9.24](#). Для шаблона, содержащего подвыражения в скобках, *подвыражение* — целое число, которое выбирает представляющее интерес подвыражение: в результате выдаётся позиция подстроки, соответствующей этому подвыражению. Подвыражения нумеруются по порядку открывающих их скобок. Когда *подвыражение* опущено или равно нулю, в результате выдаётся позиция всего вхождения независимо от подвыражений в скобках.

Несколько примеров:

```
regexp_instr('number of your street, town zip, FR', '[^,]+', 1, 2)
                                23
regexp_instr('ABCDEFGHII', '(c..)(...)', 1, 1, 0, 'i', 2)
                                6
```

Функция `regexp_like` проверяет, встречается ли в строке вхождение шаблона регулярного выражения POSIX, возвращая логическое значение `true` или `false`. Она имеет синтаксис `regexp_like(строка, шаблон [, флаги])`. В параметре *флаги* передаётся необязательная текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Поддерживаемые флаги описаны в [Таблица 9.24](#). Если флаги не указаны, эта функция выдаёт те же результаты, что и оператор `~`. Если указан только флаг *i*, она выдаёт те же результаты, что и оператор `~*`.

Несколько примеров:

```
regexp_like('Hello World', 'world')           false
regexp_like('Hello World', 'world', 'i')       true
```

Функция `regexp_match` возвращает текстовый массив с совпавшими подстроками из первого вхождения шаблона регулярного выражения POSIX в строке. Она имеет синтаксис `regexp_match(строка, шаблон [, флаги])`. Если вхождение не находится, результатом будет `NULL`. Если вхождение находится и *шаблон* не содержит подвыражений в скобках, результатом будет текстовый массив с одним элементом, содержащим подстроку, которая соответствует всему шаблону. Если вхождение находится и *шаблон* содержит подвыражения в скобках, результатом будет текстовый массив, в котором *n*-м элементом будет *n*-е заключённое в скобки подвыражение *шаблона* (не считая «незахватывающих» скобок; подробнее см. ниже). В параметре *флаги* передаётся необязательная текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Допустимые флаги описаны в [Таблице 9.24](#).

Несколько примеров:

```
SELECT regexp_match('foobarbequebaz', 'bar.*que');
       regexp_match
-----
 {barbeque}
(1 row)

SELECT regexp_match('foobarbequebaz', '(bar) (beque)');
       regexp_match
-----
 {bar,beque}
(1 row)
```

Подсказка

В общем случае, когда нужно просто получить всю найденную подстроку или `NULL` при отсутствии вхождений, лучше использовать `regexp_substr()`. Однако функция `regexp_substr()`

есть только в PostgreSQL версии 15 и выше. В предыдущих версиях для этого можно извлечь первый элемент результата `regexp_match()`, например:

```
SELECT (regexp_match('foobarbequebaz', 'bar.*que'))[1];
 regexp_match
-----
 barbeque
(1 row)
```

Функция `regexp_matches` возвращает набор текстовых массивов с совпавшими подстроками из вхождений шаблона регулярного выражения POSIX в строке. Она имеет тот же синтаксис, что и `regexp_match`. Эта функция не возвращает никаких строк, если вхождений нет; возвращает одну строку, если найдено одно вхождение и не передан флаг `g`, или N строк, если найдено N вхождений и передан флаг `g`. Каждая возвращаемая строка представляет собой текстовый массив, содержащий всю найденную подстроку или подстроки, соответствующие заключённым в скобки подвыражениям шаблона, как и описанный выше результат `regexp_match`. Функция `regexp_matches` принимает все флаги, показанные в Таблице 9.24, а также флаг `g`, указывающий ей выдать все вхождения, а не только первое.

Несколько примеров:

```
SELECT regexp_matches('foo', 'not there');
 regexp_matches
-----
(0 rows)

SELECT regexp_matches('foobarbequebazilbarfbonk', '(b[^b]+)(b[^b]+)', 'g');
 regexp_matches
-----
 {bar,beque}
 {bazil,barf}
(2 rows)
```

Подсказка

В большинстве случаев `regexp_matches()` должна применяться с флагом `g`, так как если вас интересует только первое вхождение, проще и эффективнее использовать функцию `regexp_match()`. Однако `regexp_match()` существует только в Postgres Pro версии 10 и выше. В старых версиях обычно помещали вызов `regexp_matches()` во вложенный SELECT, например, так:

```
SELECT col1, (SELECT regexp_matches(col2, '(bar)(beque)')) FROM tab;
```

В результате выдаётся текстовый массив, если вхождение найдено, или NULL в противном случае, так же как с `regexp_match()`. Без вложенного SELECT этот запрос не возвращает никакие строки, если соответствие не находится, а это обычно не то, что нужно.

Функция `regexp_replace` подставляет другой текст вместо подстрок, соответствующих шаблонам регулярных выражений POSIX. Она имеет синтаксис `regexp_replace(исходная_строка, шаблон, замена [, начало [, N]] [, флаги])`. (Заметьте, что если не задано *начало*, указывать N нельзя, но флаги можно указывать в любом случае.) Если *исходная_строка* не содержит фрагмента, подходящего под шаблон, она возвращается неизменной. Если же соответствие находится, возвращается *исходная_строка*, в которой вместо соответствующего фрагмента подставляется *замена*. Строка *замена* может содержать `\n`, где n — число от 1 до 9, указывающее на исходную подстроку, соответствующую n -му подвыражению в скобках, и может содержать обозначение `&`, указывающее, что будет вставлена подстрока, соответствующая всему шаблону. Если же в текст замены нужно включить обратную косую черту буквально, следует написать `\\`. Шаблон ищется в строке обычно с начала строки, но если указан параметр *начало*, то начиная с этого индекса. По умолчанию заменяется

только первое вхождение шаблона. Если *N* указано и больше нуля, заменяется *N*-е вхождение шаблона. Если передан флаг *g* или заданное *N* равно нулю, заменяются все вхождения с позиции *начало* и далее. (Флаг *g* игнорируется, когда задано *N*.) В необязательном параметре *флаги* передаётся текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Допустимые флаги (кроме *g*) описаны в [Таблице 9.24](#).

Несколько примеров:

```
regexp_replace('foobarbaz', 'b..', 'X')
      fooXbaz
regexp_replace('foobarbaz', 'b..', 'X', 'g')
      fooXX
regexp_replace('foobarbaz', 'b(..)', 'X\1Y', 'g')
      fooXarYXazY
regexp_replace('A PostgreSQL function', 'a|e|i|o|u', 'X', 1, 0, 'i')
      X PXstgrXSQL fXnctXXn
regexp_replace('A PostgreSQL function', 'a|e|i|o|u', 'X', 1, 3, 'i')
      A PostgrXSQL function
```

Функция `regexp_split_to_table` разделяет строку, используя в качестве разделителя шаблон регулярного выражения POSIX. Она имеет синтаксис `regexp_split_to_table(строка, шаблон [, флаги])`. Если *шаблон* не находится в переданной строке, возвращается вся *строка* целиком. Если находится минимум одно вхождение, для каждого такого вхождения возвращается текст от конца предыдущего вхождения (или начала строки) до начала вхождения. После последнего найденного вхождения возвращается фрагмент от его конца до конца строки. В необязательном параметре *флаги* передаётся текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Флаги, которые поддерживает `regexp_split_to_table`, описаны в [Таблице 9.24](#).

Функция `regexp_split_to_array` ведёт себя подобно `regexp_split_to_table`, за исключением того, что `regexp_split_to_array` возвращает результат в массиве элементов типа `text`. Она имеет синтаксис `regexp_split_to_array(строка, шаблон [, флаги])`. Параметры у этой функции те же, что и у `regexp_split_to_table`.

Несколько примеров:

```
SELECT foo FROM regexp_split_to_table('the quick brown fox jumps over the lazy dog',
'\s+') AS foo;
foo
-----
the
quick
brown
fox
jumps
over
the
lazy
dog
(9 rows)

SELECT regexp_split_to_array('the quick brown fox jumps over the lazy dog', '\s+');
      regexp_split_to_array
-----
{the,quick,brown,fox,jumps,over,the,lazy,dog}
(1 row)

SELECT foo FROM regexp_split_to_table('the quick brown fox', '\s*') AS foo;
foo
```



```
-----
t
h
e
q
u
i
c
k
b
r
o
w
n
f
o
x
(16 rows)
```

Как показывает последний пример, функции разделения по регулярным выражениям игнорируют вхождения нулевой длины, идущие в начале и в конце строки, а также непосредственно за предыдущим вхождением. Это поведение противоречит строгому определению поиска по регулярным выражениям, который реализуют другие функции, но обычно более удобно на практике. Подобное поведение наблюдается и в других программных средах, например в Perl.

Функция `regexp_substr` возвращает подстроку, соответствующую шаблону регулярного выражения POSIX, или NULL, если такого соответствия нет. Она имеет синтаксис `regexp_substr(строка, шаблон [, начало [, N [, флаги [, подвыражение]]]])`. Шаблон ищется в строке обычно с начала строки, но если указан параметр *начало*, то начиная с этого индекса. Если указано *N*, возвращается *N*-е вхождение шаблона, в противном случае возвращается первое вхождение. В параметре *флаги* передаётся необязательная текстовая строка, содержащая ноль или более однобуквенных флагов, меняющих поведение функции. Поддерживаемые флаги описаны в [Таблица 9.24](#). Для шаблона, содержащего подвыражения в скобках, *подвыражение* — целое число, которое выбирает представляющее интерес подвыражение: в результате выдаётся подстрока, соответствующая этому подвыражению. Подвыражения нумеруются по порядку открывающих их скобок. Когда *подвыражение* опущено или равно нулю, в результате выдаётся всё вхождение независимо от подвыражений в скобках.

Несколько примеров:

```
regexp_substr('number of your street, town zip, FR', '[^,]+', 1, 2)
                                town zip
regexp_substr('ABCDEFGHFI', '(c..)(...)', 1, 1, 'i', 2)
                                FGH
```

9.7.3.1. Подробное описание регулярных выражений

Регулярные выражения в Postgres Pro реализованы с использованием программного пакета, который разработал Генри Спенсер (Henry Spencer). Практически всё следующее описание регулярных выражений дословно скопировано из его руководства.

Регулярное выражение (Regular expression, RE), согласно определению в POSIX 1003.2, может иметь две формы: *расширенное* RE или ERE (грубо говоря, это выражения, которые понимает `egrep`) и *простое* RE или BRE (грубо говоря, это выражения для `ed`). Postgres Pro поддерживает обе формы, а кроме того реализует некоторые расширения, не предусмотренные стандартом POSIX, но широко используемые вследствие их доступности в некоторых языках программирования, например в Perl и Tcl. Регулярные выражения, использующие эти несовместимые с POSIX расширения, здесь называются *усовершенствованными* RE или ARE. ARE практически представляют собой надмножество ERE, тогда как BRE отличаются некоторой несовместимостью в записи (помимо того, что они гораздо более ограничены). Сначала мы опишем формы ARE и ERE, отметив особенности, присущие только ARE, а затем расскажем, чем от них отличаются BRE.

Примечание

Postgres Pro изначально всегда предполагает, что регулярное выражение следует правилам ARE. Однако можно переключиться на более ограниченные правила ERE или BRE, добавив в шаблон RE *встроенный параметр*, как описано в [Подразделе 9.7.3.4](#). Это может быть полезно для совместимости с приложениями, ожидающими от СУБД строгого следования правилам POSIX 1003.2.

Регулярное выражение определяется как одна или более *ветвей*, разделённых символами `|`. Оно считается соответствующим всему, что соответствует одной из этих ветвей.

Ветвь — это ноль или несколько *количественных атомов* или *ограничений*, соединённых вместе. Соответствие ветви в целом образуется из соответствия первой части, за которым следует соответствие второй части и т. д.; пустой ветви соответствует пустая строка.

Количественный атом — это *атом*, за которым может следовать *определитель количества*. Без этого определителя ему соответствует одно вхождение атома. С определителем количества ему может соответствовать некоторое число вхождений этого атома. Все возможные *атомы* перечислены в [Таблице 9.17](#). Варианты определителей количества и их значения перечислены в [Таблице 9.18](#).

Ограничению соответствует пустая строка, но это соответствие возможно только при выполнении определённых условий. Ограничения могут использоваться там же, где и атомы, за исключением того, что их нельзя дополнять определителями количества. Простые ограничения показаны в [Таблице 9.19](#); некоторые дополнительные ограничения описаны ниже.

Таблица 9.17. Атомы регулярных выражений

Атом	Описание
<code>(re)</code>	(где <i>re</i> — любое регулярное выражение) описывает соответствие <i>re</i> , при этом данное соответствие захватывается для последующей обработки
<code>(?: re)</code>	подобно предыдущему, но соответствие не захватывается (т. е. это набор скобок «без захвата») (применимо только к ARE)
<code>.</code>	соответствует любому символу
<code>[СИМВОЛЫ]</code>	<i>выражение в квадратных скобках</i> , соответствует любому из <i>СИМВОЛОВ</i> (подробнее это описано в Подразделе 9.7.3.2)
<code>\k</code>	(где <i>k</i> — не алфавитно-цифровой символ) соответствует обычному символу буквально, т. е. <code>\\</code> соответствует обратной косой черте
<code>\c</code>	где <i>c</i> — алфавитно-цифровой символ (за которым могут следовать другие символы), это <i>спецсимвол</i> , см. Подраздел 9.7.3.3 (применим только к ARE; в ERE и BRE этому атому соответствует <i>c</i>)
<code>{</code>	когда за этим символом следует любой символ, кроме цифры, этот атом соответствует левой фигурной скобке (<code>{</code>), если же за ним следует цифра, это обозначает начало <i>границы</i> (см. ниже)
<code>x</code>	(где <i>x</i> — один символ, не имеющий специального значения) соответствует этому символу

Выражение RE не может заканчиваться обратной косой чертой (\).

Примечание

Если параметр `standard_conforming_strings` выключен, каждый символ обратной косой черты, записываемый в текстовой константе, нужно дублировать. Подробнее это описано в [Подразделе 4.1.2.1](#).

Таблица 9.18. Определители количества в регулярных выражениях

Определитель	Соответствует
*	0 или более вхождений атома
+	1 или более вхождений атома
?	0 или 1 вхождение атома
{ <i>m</i> }	ровно <i>m</i> вхождений атома
{ <i>m</i> , }	<i>m</i> или более вхождений атома
{ <i>m</i> , <i>n</i> }	от <i>m</i> до <i>n</i> (включая границы) вхождений атома; <i>m</i> не может быть больше <i>n</i>
*?	нежадная версия *
+?	нежадная версия +
??	нежадная версия ?
{ <i>m</i> }?	нежадная версия { <i>m</i> }
{ <i>m</i> , }?	нежадная версия { <i>m</i> , }
{ <i>m</i> , <i>n</i> }?	нежадная версия { <i>m</i> , <i>n</i> }

В формах с { . . . } числа *m* и *n* определяют так называемые *границы* количества. Эти числа должны быть беззнаковыми десятичными целыми в диапазоне от 0 до 255 включительно.

Не жадные определители (допустимые только в ARE) описывают те же возможные соответствия, что и аналогичные им обычные («жадные»), но предпочитают выбирать наименьшее, а не наибольшее количество вхождений. Подробнее это описано в [Подразделе 9.7.3.5](#).

Примечание

Определители количества не могут следовать один за другим, например запись ** будет ошибочной. Кроме того, определители не могут стоять в начале выражения или подвыражения и идти сразу после ^ или |.

Таблица 9.19. Ограничения в регулярных выражениях

Ограничение	Описание
^	соответствует началу строки
\$	соответствует концу строки
(?= <i>re</i>)	<i>позитивный просмотр вперёд</i> находит соответствие там, где начинается подстрока, соответствующая <i>re</i> (только для ARE)
(?! <i>re</i>)	<i>негативный просмотр вперёд</i> находит соответствие там, где не начинается подстрока, соответствующая <i>re</i> (только для ARE)

Ограничение	Описание
(?<= re)	<i>позитивный просмотр назад</i> находит соответствие там, где заканчивается подстрока, соответствующая <i>re</i> (только для ARE)
(?<! re)	<i>негативный просмотр назад</i> находит соответствие там, где не заканчивается подстрока, соответствующая <i>re</i> (только для ARE)

Ограничения просмотра вперёд и назад не могут содержать *ссылки назад* (см. [Подраздел 9.7.3.3](#)), и все скобки в них считаются «скобками без захвата».

9.7.3.2. Выражения в квадратных скобках

Выражение в квадратных скобках содержит список символов, заключённый в []. Обычно ему соответствует любой символ из списка (об исключении написано ниже). Если список начинается с ^, ему соответствует любой символ, который *не* перечисляется далее в этом списке. Если два символа в списке разделяются знаком –, это воспринимается как краткая запись полного интервала символов между двумя заданными (и включая их) в порядке сортировки; например выражению [0–9] в ASCII соответствует любая десятичная цифра. Два интервала не могут разделять одну границу, т. е. выражение a–c–e недопустимо. Интервалы зависят от порядка сортировки, который может меняться, поэтому в переносимых программах их лучше не использовать.

Чтобы включить в список], этот символ нужно написать первым (сразу за ^, если он присутствует). Чтобы включить в список символ –, его нужно написать первым или последним, либо как вторую границу интервала. Указать – в качестве первой границы интервал можно, заключив его между [. и .], чтобы он стал элементом сортировки (см. ниже). За исключением этих символов, некоторых комбинаций с [(см. следующие абзацы) и спецсимволов (в ARE), все остальные специальные символы в квадратных скобках теряют своё особое значение. В частности, символ \ по правилам ERE или BRE воспринимается как обычный, хотя в ARE он экранирует символ, следующий за ним.

Выражения в квадратных скобках могут содержать элемент сортировки (символ или последовательность символов или имя такой последовательности), определение которого заключается между [. и .]. Определяющая его последовательность воспринимается в выражении в скобках как один элемент. Это позволяет включать в такие выражения элементы, соответствующие последовательности нескольких символов. Например, с элементом сортировки ch в квадратных скобках регулярному выражению [[.ch.]]*c будут соответствовать первые пять символов строки chchcc.

Примечание

В настоящее время Postgres Pro не поддерживает элементы сортировки, состоящие из нескольких символов. Эта информация относится к возможному в будущем поведению.

В квадратных скобках могут содержаться элементы сортировки, заключённые между [= и =], обозначающие *классы эквивалентности*, т. е. последовательности символов из всех элементов сортировки, эквивалентных указанному, включая его самого. (Если для этого символа нет эквивалентных, он обрабатывается как заключённый между [. и .].) Например, если e и ë — члены одного класса эквивалентности, выражения [[=e=]], [[=ë=]] и [eë] будут равнозначными. Класс эквивалентности нельзя указать в качестве границы интервала.

В квадратных скобках может также задаваться имя класса символов, заключённое между [: и :], которое обозначает множество всех символов, принадлежащих этому классу. Класс символов не может представлять границу интервала. В стандарте POSIX определены следующие имена классов: alnum (буквы и цифры), alpha (буквы), blank (пробел и табуляция), cntrl (управляющие символы), digit (десятичные цифры), graph (печатаемые символы, кроме пробела), lower (буквы в нижнем регистре), print (печатаемые символы, включая пробел), punct (знаки пунктуации), space

(любые пробельные символы), `upper` (буквы в верхнем регистре) и `xdigit` (шестнадцатеричные цифры). На согласованное поведение этих стандартных классов символов на разных платформах можно рассчитывать в рамках 7-битного набора ASCII. Будет ли определённый не ASCII-символ считаться принадлежащим одному из этих классов, зависит от правила сортировки, используемого оператором или функцией регулярных выражений (см. [Раздел 22.2](#)). По умолчанию это правило определяется свойством `LC_COLLATE` базы данных (см. [Раздел 22.1](#)). Классификация не ASCII-символов может меняться от платформы к платформе даже при выборе локалей с похожими названиями. (Но локаль `C` никогда не относит не ASCII-символы ни к одному из этих классов.) Помимо этих стандартных классов символов, в Postgres Pro определён класс символов `word`, отличающийся от класса `alnum` лишь тем, что включает ещё символ подчёркивания (`_`), и класс символов `ascii`, который содержит все 7-битные символы ASCII, но не какие-либо другие.

Есть два особых вида выражений в квадратных скобках: выражения `[[:<:]]` и `[[:>:]]`, представляющие собой ограничения, соответствующие пустым строкам в начале и конце слова. Слово в данном контексте определяется как последовательность словосоставляющих символов, перед или после которой нет словосоставляющих символов. Словосоставляющий символ — это любой символ, принадлежащий классу `word`, то есть любая буква, цифра или подчёркивание. Это расширение совместимо со стандартом POSIX 1003.2, но не описано в нём, и поэтому его следует использовать с осторожностью там, где важна совместимость с другими системами. Обычно лучше использовать ограничивающие спецсимволы, описанные ниже; они также не совсем стандартны, но набрать их легче.

9.7.3.3. Спецсимволы регулярных выражений

Спецсимволы — это специальные команды, состоящие из `\` и последующего алфавитно-цифрового символа. Можно выделить следующие категории спецсимволов: обозначения символов, коды классов, ограничения и ссылки назад. Символ `\`, за которым идёт алфавитно-цифровой символ, не образующий допустимый спецсимвол, считается ошибочным в ARE. В ERE спецсимволов нет: вне квадратных скобок пара из `\` и последующего алфавитно-цифрового символа, воспринимается просто как данный символ, а в квадратных скобках и сам символ `\` воспринимается просто как обратная косая черта. (Последнее на самом деле нарушает совместимость между ERE и ARE.)

Спецобозначения символов введены для того, чтобы облегчить ввод в RE непечатаемых и других неудобных символов. Они приведены в [Таблице 9.20](#).

Коды классов представляют собой краткий способ записи имён некоторых распространённых классов символов. Они перечислены в [Таблице 9.21](#).

Спецсимволы ограничений обозначают ограничения, которым при совпадении определённых условий соответствует пустая строка. Они перечислены в [Таблице 9.22](#).

Ссылка назад (`\n`) соответствует той же строке, какой соответствовало предыдущее подвыражение в скобках под номером `n` (см. [Таблицу 9.23](#)). Например, `([bc])\1` соответствует `bb` или `cc`, но не `bc` или `cb`. Это подвыражение должно полностью предшествовать ссылке назад в RE. Нумеруются подвыражения в порядке следования их открывающих скобок. При этом скобки без захвата исключаются из рассмотрения. Ссылка назад учитывает только строковые символы, совпадающие с указанным подвыражением, но не содержащиеся в нём ограничения. Например, выражению `(^ \d)\1` будет соответствовать `22`).

Таблица 9.20. Спецобозначения символов в регулярных выражениях

Спецсимвол	Описание
<code>\a</code>	символ звонка, как в C
<code>\b</code>	символ «забой», как в C
<code>\B</code>	синоним для обратной косой черты (<code>\</code>), сокращающий потребность в дублировании этого символа

Спецсимвол	Описание
<code>\cX</code>	(где <i>X</i> — любой символ) символ, младшие 5 бит которого те же, что и у <i>X</i> , а остальные равны 0
<code>\e</code>	символ, определённый в последовательности сортировки с именем <code>ESC</code> , либо, если таковой не определён, символ с восьмеричным значением 033
<code>\f</code>	подача формы, как в C
<code>\n</code>	новая строка, как в C
<code>\r</code>	возврат каретки, как в C
<code>\t</code>	горизонтальная табуляция, как в C
<code>\uwxyz</code>	(где <i>wxyz</i> ровно четыре шестнадцатеричные цифры) символ с шестнадцатеричным кодом 0 <i>wxyz</i>
<code>\Ustuvwxyz</code>	(где <i>stuvwxyz</i> ровно восемь шестнадцатеричных цифр) символ с шестнадцатеричным кодом 0 <i>xstuvwxyz</i>
<code>\v</code>	вертикальная табуляция, как в C
<code>\xhhh</code>	(где <i>hhh</i> — несколько шестнадцатеричных цифр) символ с шестнадцатеричным кодом 0 <i>xhhh</i> (символ всегда один вне зависимости от числа шестнадцатеричных цифр)
<code>\0</code>	символ с кодом 0 (нулевой байт)
<code>\xy</code>	(где <i>xy</i> — ровно две восьмеричных цифры, не ссылка назад) символ с восьмеричным кодом 0 <i>xy</i>
<code>\xyz</code>	(где <i>xyz</i> — ровно три восьмеричных цифры, не ссылка назад) символ с восьмеричным кодом 0 <i>xyz</i>

Шестнадцатеричные цифры записываются символами 0-9 и a-f или A-F. Восьмеричные цифры — цифры от 0 до 7.

Спецпоследовательности с числовыми кодами, задающими значения вне диапазона ASCII (0-127), воспринимаются по-разному в зависимости от кодировки базы данных. Когда база данных имеет кодировку UTF-8, спецкод равнозначен позиции символа в Unicode, например, `\u1234` обозначает символ U+1234. Для других многобайтных кодировок спецпоследовательности обычно просто задают серию байт, определяющих символ. Если в кодировке базы данных отсутствует символ, заданный спецпоследовательностью, ошибки не будет, но и никакие данные не будут ей соответствовать.

Символы, переданные спецобозначением, всегда воспринимаются как обычные символы. Например, `\135` кодирует] в ASCII, но спецпоследовательность `\135` не будет закрывать выражение в квадратных скобках.

Таблица 9.21. Спецкоды классов в регулярных выражениях

Спецсимвол	Описание
<code>\d</code>	соответствует любой цифре; равнозначно <code>[[:digit:]]</code>
<code>\s</code>	соответствует любому пробельному символу; равнозначно <code>[[:space:]]</code>

Спецсимвол	Описание
\w	соответствует любому буквенному символу; равнозначно <code>[[:word:]]</code>
\D	соответствует любому нецифровому символу; равнозначно <code>^[[:digit:]]</code>
\S	соответствует любому непробельному символу; равнозначно <code>^[[:space:]]</code>
\W	соответствует любому небуквенному символу; равнозначно <code>^[[:word:]]</code>

Коды классов также работают в выражениях в квадратных скобках, хотя определения, показанные выше, не совсем синтаксически верны в данном контексте. Например, выражение `[a-c\d]` равнозначно `[a-c[:digit:]]`.

Таблица 9.22. Спецсимволы ограничений в регулярных выражений

Спецсимвол	Описание
\A	соответствует только началу строки (чем это отличается от <code>^</code> , описано в Подразделе 9.7.3.5)
\b	соответствует только началу слова
\B	соответствует только концу слова
\b	соответствует только началу или концу слова
\B	соответствует только положению не в начале и не в конце слова
\Z	соответствует только концу строки (чем это отличается от <code>\$</code> , описано в Подразделе 9.7.3.5)

Определением слова здесь служит то же, что было приведено выше в описании `[[:<:]]` и `[[:>:]]`. В квадратных скобках спецсимволы ограничений не допускаются.

Таблица 9.23. Ссылки назад в регулярных выражениях

Спецсимвол	Описание
\m	(где <i>m</i> — цифра, отличная от 0) — ссылка назад на подвыражение под номером <i>m</i>
\mnn	(где <i>m</i> — цифра, отличная от 0, а <i>nn</i> — ещё несколько цифр с десятичным значением <i>mnn</i> , не превышающим число закрытых до этого скобок с захватом) ссылка назад на подвыражение под номером <i>mnn</i>

Примечание

Регулярным выражениям присуща неоднозначность между восьмеричными кодами символов и ссылками назад, которая разрешается следующим образом (это упоминалось выше). Ведущий ноль всегда считается признаком восьмеричной последовательности. Единственная цифра, отличная от 0, за которой не следует ещё одна цифра, всегда воспринимается как ссылка назад. Последовательность из нескольких цифр, которая начинается не с 0, воспринимается как ссылка назад, если она идёт за подходящим подвыражением (т. е. число оказывается в диапазоне, допустимом для ссылки назад), в противном случае она воспринимается как восьмеричное число.

9.7.3.4. Метасинтаксис регулярных выражений

В дополнение к основному синтаксису, описанному выше, можно использовать также несколько особых форм и разнообразные синтаксические удобства.

Регулярное выражение может начинаться с одного из двух специальных префиксов режима. Если RE начинается с `***:`, его продолжение рассматривается как ARE. (В Postgres Pro это обычно не имеет значения, так как регулярные выражения воспринимаются как ARE по умолчанию; но это может быть полезно, когда параметр *флаги* функций `regex` включает режим ERE или BRE.) Если RE начинается с `***=`, его продолжение воспринимается как обычная текстовая строка, все его символы воспринимаются буквально.

ARE может начинаться со *встроенных параметров*: последовательности `(?xyz)` (где *xyz* — один или несколько алфавитно-цифровых символов), определяющих параметры остального регулярного выражения. Эти параметры переопределяют любые ранее определённые параметры, в частности они могут переопределить режим чувствительности к регистру, подразумеваемый для оператора `regex`, или параметр *флаги* функции `regex`. Допустимые буквы параметров показаны в [Таблице 9.24](#). Заметьте, что те же буквы используются в параметре *флаги* функций `regex`.

Таблица 9.24. Буквы встроенных параметров ARE

Параметр	Описание
b	продолжение регулярного выражения — BRE
c	поиск соответствий с учётом регистра (переопределяет тип оператора)
e	продолжение RE — ERE
i	поиск соответствий без учёта регистра (см. Подраздел 9.7.3.5) (переопределяет тип оператора)
m	исторически сложившийся синоним n
n	поиск соответствий с учётом перевода строк (см. Подраздел 9.7.3.5)
p	переводы строк учитываются частично (см. Подраздел 9.7.3.5)
q	продолжение регулярного выражения — обычная строка («в кавычках»), содержимое которой воспринимается буквально
s	поиск соответствий без учёта перевода строк (по умолчанию)
t	компактный синтаксис (по умолчанию; см. ниже)
w	переводы строк учитываются частично, но в другом, «странном» режиме (см. Подраздел 9.7.3.5)
x	развёрнутый синтаксис (см. ниже)

Внедрённые параметры начинают действовать сразу после скобки `)`, завершающей их последовательность. Они могут находиться только в начале ARE (после указания `***:`, если оно присутствует).

Помимо обычного (*компактного*) синтаксиса RE, в котором имеют значение все символы, поддерживается также *развёрнутый* синтаксис, включить который можно с помощью встроенного параметра `x`. В развёрнутом синтаксисе игнорируются пробельные символы, а также все символы от `#` до конца строки (или конца RE). Это позволяет разделять RE на строки и добавлять в него комментарии. Но есть три исключения:

- пробельный символ или #, за которым следует \, сохраняется
- пробельный символ или # внутри выражения в квадратных скобках сохраняется
- пробельные символы и комментарии не могут присутствовать в составных символах, например, в (? :

В данном контексте пробельными символами считаются пробел, табуляция, перевод строки и любой другой символ, относящийся к классу символов *space*.

И наконец, в ARE последовательность (?#*ttt*) (где *ttt* — любой текст, не содержащий)) вне квадратных скобок также считается комментарием и полностью игнорируется. При этом она так же не может находиться внутри составных символов, таких как (? :. Эти комментарии в большей степени историческое наследие, чем полезное средство; они считаются устаревшими, а вместо них рекомендуется использовать развёрнутый синтаксис.

Ни одно из этих расширений метасинтаксиса не будет работать, если выражение начинается с префикса ****=*, после которого строка воспринимается буквально, а не как RE.

9.7.3.5. Правила соответствия регулярным выражениям

В случае, когда RE может соответствовать более чем одной подстроке в заданной строке, соответствующей RE считается подстрока, которая начинается в ней первой. Если к данной позиции подобных соответствующих подстрок оказывается несколько, из них выбирается либо самая длинная, либо самая короткая из возможных, в зависимости от того, какой режим выбран в RE: *жадный* или *не жадный*.

Где жадный или не жадный характер RE определяется по следующим правилам:

- Большинство атомов и все ограничения не имеют признака жадности (так как они всё равно не могут соответствовать подстрокам разного состава).
- Скобки, окружающие RE, не влияют на его «жадность».
- Атом с определителем фиксированного количества (*{m}* или *{m}?*) имеет ту же характеристику жадности (или может не иметь её), как и сам атом.
- Атом с другими обычными определителями количества (включая *{m, n}*, где *m* равняется *n*) считается жадным (предпочитает соответствие максимальной длины).
- Атом с не жадным определителем количества (включая *{m, n}?*, где *m* равно *n*) считается не жадным (предпочитает соответствие минимальной длины).
- Ветвь (RE без оператора | на верхнем уровне) имеет ту же характеристику жадности, что и первый количественный атом в нём, имеющий атрибут жадности.
- RE, образованное из двух или более ветвей, соединённых оператором |, всегда считается жадным.

Эти правила связывают характеристики жадности не только с отдельными количественными атомами, но и с ветвями и целыми RE, содержащими количественные атомы. Это означает, что при сопоставлении ветвь или целое RE может соответствовать максимально длинной или короткой подстроке *в целом*. Когда определена длина всего соответствия, часть его, соответствующая конкретному подвыражению, определяется с учётом характеристики жадности для этого подвыражения, при этом подвыражения, начинающиеся в RE раньше, имеют больший приоритет, чем следующие за ними.

Это иллюстрирует следующий пример:

```
SELECT SUBSTRING('XY1234Z', 'Y*([0-9]{1,3})');
Результат: 123
SELECT SUBSTRING('XY1234Z', 'Y*?([0-9]{1,3})');
Результат: 1
```

В первом случае RE в целом жадное, так как жадным является атом Y^* . Соответствие ему начинается с буквы Y и оно включает подстроку максимальной длины с этого места, т. е. подстроку Y123. Результат выражения — её часть, соответствующая подвыражению в скобках, т. е. 123. Во втором случае, RE в целом наследует не жадный характер от атома $\text{Y}^*?$. Соответствие ему так же начинается с Y, но включает оно подстроку минимальной длины с этого места, т. е. Y1. И хотя подвыражение $[0-9]\{1, 3\}$ имеет жадный характер, оно не может повлиять на выбор длины соответствия в целом, поэтому ему остаётся только подстрока 1.

Другими словами, когда RE содержит и жадные, и не жадные подвыражения, всё соответствие будет максимально длинным или коротким в зависимости от характеристики всего RE. Характеристики, связанные с подвыражениями, влияют только на то, какую часть подстроки может «поглотить» одно подвыражение относительно другого.

Чтобы явно придать характеристику «жадности» или «нежадности» подвыражению или всему RE, можно использовать определители $\{1, 1\}$ и $\{1, 1\}?$, соответственно. Это полезно, когда вам нужно, чтобы общая характеристика жадности RE отличалась от той, что вытекает из его элементов. Например, предположим, что вы пытаетесь выделить из строки, содержащей несколько цифр, эти цифры и части до и после них. Можно попытаться сделать это так:

```
SELECT regexp_match('abc01234xyz', '(.*) (\d+) (.*)');
```

Результат: {abc0123, 4, xyz}

Но это не будет работать: первая группа $.^*$ — «жадная», она «съест» всё, что сможет, оставляя для соответствия d^+ только последнюю возможность, то есть последнюю цифру. Можно попытаться сделать запрос «нежадным»:

```
SELECT regexp_match('abc01234xyz', '(.*)? (\d+) (.*)');
```

Результат: {abc, 0, ""}

И это не будет работать, так теперь весь RE в целом стал нежадным, и все соответствия завершаются как можно раньше. Но мы можем получить нужный результат, явно сделав жадным всё RE:

```
SELECT regexp_match('abc01234xyz', '(?: (.*)? (\d+) (.*))\{1, 1\}');
```

Результат: {abc, 01234, xyz}

Управление общей характеристикой «жадности» RE независимо от «жадности» его компонентов даёт большую гибкость в описании шаблонов переменной длины.

При определении более длинного или более короткого соответствия длины соответствий определяются в символах, а не в элементах сортировки. Пустая строка считается длиннее, чем отсутствие соответствия. Например, выражению bb^* соответствуют три символа в середине строки `abbbbc`, выражению $(\text{week}|\text{wee})(\text{night}|\text{knight})$ — все десять символов `weeknights`; когда выражение $(. ^*) .^*$ сопоставляется со строкой `abc`, подвыражению в скобках соответствуют все три символа; а когда $(a ^*) ^*$ сопоставляется со строкой `bc`, то и RE в целом, и подстроке в скобках соответствует пустая строка.

Игнорирование регистра символов даёт практически тот же эффект, как если бы в алфавите исчезли различия прописных и строчных букв. Если буква, существующая и в верхнем, и в нижнем регистре, фигурирует вне квадратных скобок как обычный символ, она по сути преобразуется в выражение в квадратных скобках, содержащее оба варианта, например `x` становится `[xX]`. Если же она фигурирует в выражении в квадратных скобках, в это выражение добавляются все её варианты, например `[x]` становится `[xX]`, а `^[x]` — `^[xX]`.

Когда включён многострочный режим поиска, атом `.` и выражения в квадратных скобках с `^` никогда не будут соответствовать символам конца строки (так что соответствия никогда не будут пересекать границы строк, если в RE нет явных указаний на эти символы), а `^` и `$` будут соответствовать пустой подстроке не только в начале и конце всего текста, но и в начале и конце каждой отдельной его строки. Однако спецсимволы ARE `\A` и `\Z` по-прежнему будут соответствовать *только* началу и концу всего текста. Кроме того, коды классов символов `\D` и `\W` будут соответствовать концам строк независимо от этого режима. (До PostgreSQL Pro 14 они не соответствовали концам

строк в многострочном режиме. Для использования старого поведения напишите `[^[:digit:]]` или `[^[:word:]]`.)

В режиме, когда переводы строк учитываются частично, особый смысл перевод строк имеет для атома `.` и выражений в квадратных скобках, но не для `^` и `$`.

В обратном частичном режиме, перевод строк имеет особый смысл для `^` и `$`, как и в режиме с учётом перевода строк, но не для `.` и выражений в квадратных скобках. Данный режим не очень полезен, но существует для симметрии.

9.7.3.6. Пределы и совместимость

В текущей реализации отсутствует какой-либо явно заданный предел длины RE. Однако, разрабатывая программы высокой степени переносимости, не следует применять RE длиннее 256 байт, так как другая POSIX-совместимая реализация может отказаться обрабатывать такие регулярные выражения.

Единственная особенность ARE, действительно несовместимая с ERE стандарта POSIX проявляется в том, что в ARE знак `\` не теряет своё специальное значение в квадратных скобках. Все другие расширения ARE используют синтаксические возможности, которые не определены, не допустимы или не поддерживаются в ERE; синтаксис переключения режимов (`***`) также выходит за рамки синтаксиса POSIX как для BRE, так и для ERE.

Многие расширения ARE заимствованы из языка Perl, но некоторые были изменены, оптимизированы, а несколько расширений Perl были исключены. В результате имеют место следующие несовместимости: атомы `\b` и `\B`, отсутствие специальной обработки завершающего перевода строки, добавление исключений в квадратных скобках в число случаев, когда учитывается перевод строк, особые условия для скобок и ссылок назад в ограничениях просмотра вперёд/назад и семантика «наиболее длинное/короткое соответствие» (вместо «первое соответствие»).

9.7.3.7. Простые регулярные выражения

BRE имеют ряд отличий от ERE. В BRE знаки `|`, `+` и `?` теряют специальное значение, а замены им нет. Границы количества окружаются символами `\{` и `\}`, тогда как `{` и `}` рассматриваются как обычные символы. Вложенные подвыражения помещаются между `\(` и `\)`, а `(` и `)` представляют обычные символы. Символ `^` воспринимается как обычный, если только он не находится в начале RE или подвыражения в скобках, `$` — тоже обычный символ, если он находится не в конце RE или в конце подвыражения в скобках, и `*` — обычный символ, когда он находится в начале RE или подвыражения в скобках (возможно, после начального `^`). И, наконец, в BRE работают ссылки назад с одной цифрой, `\<` и `\>` — синонимы для `[:<:]` и `[:>:]`, соответственно; никакие другие спецсимволы в BRE не поддерживаются.

9.7.3.8. Отличия от XQuery и стандарта SQL

Начиная с SQL:2008, в стандарт SQL входят операторы и функции регулярных выражений, которые выполняют сопоставление с шаблоном в соответствии со стандартом регулярных выражений XQuery:

- `LIKE_REGEX`
- `OCCURRENCES_REGEX`
- `POSITION_REGEX`
- `SUBSTRING_REGEX`
- `TRANSLATE_REGEX`

В Postgres Pro эти операторы и функции на данный момент не реализованы. Но вы можете в каждом случае получить примерно тот же результат, как показано в [Таблице 9.25](#). (В этой таблице опущены различные необязательные предложения с обеих сторон.)

Таблица 9.25. Аналоги функций регулярных выражений

Стандарт SQL	PostgreSQL
<i>строка</i> LIKE_REGEX <i>шаблон</i>	regex_like(<i>строка</i> , <i>шаблон</i>) или <i>строка</i> ~ <i>шаблон</i>
OCCURRENCES_REGEX(<i>шаблон</i> IN <i>строка</i>)	regex_count(<i>строка</i> , <i>шаблон</i>)
POSITION_REGEX(<i>шаблон</i> IN <i>строка</i>)	regex_instr(<i>строка</i> , <i>шаблон</i>)
SUBSTRING_REGEX(<i>шаблон</i> IN <i>строка</i>)	regex_substr(<i>строка</i> , <i>шаблон</i>)
TRANSLATE_REGEX(<i>шаблон</i> IN <i>строка</i> WITH <i>замена</i>)	regex_replace(<i>строка</i> , <i>шаблон</i> , <i>замена</i>)

Функции регулярных выражений, подобные существующим в PostgreSQL, также имеются в ряде других реализаций SQL, в то время как функции стандарта SQL распространены не так широко. Некоторые детали синтаксиса регулярных выражений, скорее всего, будут различаться в каждой реализации.

В функциях и операторах стандарта SQL используются регулярные выражения XQuery, синтаксис которых похож на ARE, описанный выше. Регулярные выражения на базе POSIX в существующей реализации и регулярные выражения XQuery имеют ряд заметных отличий, в том числе:

- Вычитание классов символов XQuery не поддерживается. Например, это вычитание позволяет извлекать только английские согласные так: `[a-z-[aeiou]]`.
- Коды классов символов XQuery `\c`, `\C`, `\i` и `\I` не поддерживаются.
- Элементы классов символов XQuery с обозначением `\p{UnicodeProperty}` или обратным, `\P{UnicodeProperty}`, не поддерживаются.
- В POSIX классы символов, такие как `\w` (см. [Таблицу 9.21](#)), интерпретируются согласно текущей локали (и вы можете управлять этим, добавив предложение `COLLATE` к оператору или вызову функции). В XQuery эти классы определяются по свойствам символов Unicode, поэтому одинаковое поведение возможно только с локалями, соответствующими требованиям Unicode.
- Синтаксис стандарта SQL (не сам язык XQuery) стремится воспринять больше вариантов «перевода строки», чем синтаксис POSIX. Для описанных выше вариантов сопоставления с учётом перевода строк переводом строки считается только символ ASCII NL (`\n`), тогда как SQL считает переводом строки также CR (`\r`), CRLF (`\r\n`) (перевод строки в стиле Windows) и некоторые присущие только Unicode символы, например, LINE SEPARATOR (U+2028). Стоит заметить, что согласно SQL коды шаблона `.` и `\s` должны считать последовательность `\r\n` одним символом, а не двумя.
- Из спецкодов, определяющих символы, описанных в [Таблице 9.20](#), XQuery поддерживает только `\n`, `\r` и `\t`.
- XQuery не поддерживает синтаксис `[ :имя: ]` для указания класса символов в квадратных скобках.
- В XQuery отсутствуют условия с просмотром вперёд и назад, а также не поддерживаются спецсимволы ограничений, описанные в [Таблице 9.22](#).
- Формы метасинтаксиса, описанные в [Подразделе 9.7.3.4](#), в XQuery не существуют.
- Буквы флагов регулярных выражений, определённые в XQuery, имеют общее с буквами флагов в POSIX (см. [Таблицу 9.24](#)), но не равнозначны им. Одинаково ведут себя только флаги `i` и `q`, а все остальные различаются:
 - Флаги XQuery `s` (допускающий сопоставление точки с переводом строки) и `m` (допускающий сопоставление `^` и `$` с переводами строк) позволяют получить то же поведение, что и флаги `n`, `p` и `w` в POSIX, но они не равнозначны флагам POSIX `s` и `m`. В частности, заметьте, что по умолчанию точка соответствует переводу строки в POSIX, но не в XQuery.
 - Флаг `x` (игнорировать пробельные символы в шаблоне) в XQuery значительно отличается от флага расширенного режима POSIX. В POSIX флаг `x` дополнительно позволяет написать

комментарий в шаблоне (начиная с символа #), а пробельный символ после обратной косой черты не игнорируется.

9.8. Функции форматирования данных

Функции форматирования в Postgres Pro предоставляют богатый набор инструментов для преобразования самых разных типов данных (дата/время, целое, числа с плавающей и фиксированной точкой) в форматированные строки и обратно. Все они перечислены в [Таблице 9.26](#). Все эти функции следует одному соглашению: в первом аргументе передаётся значение, которое нужно отформатировать, а во втором — шаблон, определяющий формат ввода или вывода.

Таблица 9.26. Функции форматирования

Функция	Описание	Пример(ы)
<code>to_char (timestamp, text)</code>	Преобразует время в строку согласно заданному формату.	<code>to_char(timestamp '2002-04-20 17:31:12.66', 'HH12:MI:SS') → 05:31:12</code>
<code>to_char (timestamp with time zone, text)</code>	Преобразует интервал в строку согласно заданному формату.	<code>to_char(interval '15h 2m 12s', 'HH24:MI:SS') → 15:02:12</code>
<code>to_char (numeric_type , text)</code>	Преобразует число в строку согласно заданному формату; поддерживаются типы <code>integer</code> , <code>bigint</code> , <code>numeric</code> , <code>real</code> , <code>double precision</code> .	<code>to_char(125, '999') → 125</code> <code>to_char(125.8::real, '999D9') → 125.8</code> <code>to_char(-125.8, '999D99S') → 125.80-</code>
<code>to_date (text, text)</code>	Преобразует строку в дату согласно заданному формату.	<code>to_date('05 Dec 2000', 'DD Mon YYYY') → 2000-12-05</code>
<code>to_number (text, text)</code>	Преобразует строку в число согласно заданному формату.	<code>to_number('12,454.8-', '99G999D9S') → -12454.8</code>
<code>to_timestamp (text, text)</code>	Преобразует строку в значение времени согласно заданному формату. (См. также <code>to_timestamp(double precision)</code> в Таблице 9.33 .)	<code>to_timestamp('05 Dec 2000', 'DD Mon YYYY') → 2000-12-05 00:00:00-05</code>

Подсказка

Функции `to_timestamp` и `to_date` предназначены для работы с входными форматами, которые нельзя преобразовать простым приведением. Для большинства стандартных форматов даты/времени работает простое приведение исходной строки к требуемому типу и использовать его гораздо легче. Так же и функцию `to_number` нет необходимости использовать для стандартных представлений чисел.

Шаблон вывода `to_char` может содержать ряд кодов, которые распознаются при форматировании и заменяются соответствующими данными в зависимости от заданного значения. Любой текст, который не является кодом, просто копируется в неизменном виде. Подобным образом в строке шаблона ввода (для других функций) шаблонные коды определяют, какие значения должны посту-

пать из входной строки. Если в строке шаблона есть символы, не относящиеся к шаблонным кодам, соответствующие символы во входной строке просто пропускаются (вне зависимости от того, совпадают ли они с символами в строке шаблона).

Все коды форматирования даты и времени перечислены в [Таблице 9.27](#).

Таблица 9.27. Коды форматирования даты/времени

Код	Описание
HH	час дня (01-12)
HH12	час дня (01-12)
HH24	час дня (00-23)
MI	минута (00-59)
SS	секунда (00-59)
MS	миллисекунда (000-999)
US	микросекунда (000000-999999)
FF1	десятая доля секунды (0-9)
FF2	сотая доля секунды (00-99)
FF3	миллисекунда (000-999)
FF4	десятитысячная доля секунды (0000-9999)
FF5	стотысячная доля секунды (00000-99999)
FF6	микросекунда (000000-999999)
SSSS, SSSSS	секунды после полуночи (0-86399)
AM, am, PM или pm	обозначение времени до/после полудня (без точек)
A.M., a.m., P.M. или p.m.	обозначение времени до/после полудня (с точками)
Y, YYYY	год (4 или более цифр) с разделителем
YYYY	год (4 или более цифр)
YYY	последние 3 цифры года
YY	последние 2 цифры года
Y	последняя цифра года
IYYY	недельный год по ISO 8601 (4 или более цифр)
IYY	последние 3 цифры недельного года по ISO 8601
IY	последние 2 цифры недельного года по ISO 8601
I	последняя цифра недельного года по ISO 8601
BC, bc, AD или ad	обозначение эры (без точек)
B.C., b.c., A.D. или a.d.	обозначение эры (с точками)
MONTH	полное название месяца в верхнем регистре (дополненное пробелами до 9 символов)
Month	полное название месяца с большой буквы (дополненное пробелами до 9 символов)
month	полное название месяца в нижнем регистре (дополненное пробелами до 9 символов)

Код	Описание
MON	сокращённое название месяца в верхнем регистре (3 буквы в английском; в других языках длина может меняться)
Mon	сокращённое название месяца с большой буквы (3 буквы в английском; в других языках длина может меняться)
mon	сокращённое название месяца в нижнем регистре (3 буквы в английском; в других языках длина может меняться)
MM	номер месяца (01–12)
DAY	полное название дня недели в верхнем регистре (дополненное пробелами до 9 символов)
Day	полное название дня недели с большой буквы (дополненное пробелами до 9 символов)
day	полное название дня недели в нижнем регистре (дополненное пробелами до 9 символов)
DY	сокращённое название дня недели в верхнем регистре (3 буквы в английском; в других языках может меняться)
Dy	сокращённое название дня недели с большой буквы (3 буквы в английском; в других языках длина может меняться)
dy	сокращённое название дня недели в нижнем регистре (3 буквы в английском; в других языках длина может меняться)
DDD	номер дня в году (001–366)
IDDD	номер дня в году по ISO 8601 (001–371; первый день года — понедельник первой недели по ISO)
DD	день месяца (01–31)
D	номер дня недели, считая с воскресенья (1) до субботы (7)
ID	номер дня недели по ISO 8601, считая с понедельника (1) до воскресенья (7)
W	неделя месяца (1–5) (первая неделя начинается в первый день месяца)
WW	номер недели в году (1–53) (первая неделя начинается в первый день года)
IW	номер недели в году по ISO 8601 (01–53; первый четверг года относится к неделе 1)
CC	век (2 цифры) (двадцать первый век начался 2001-01-01)
J	юлианская дата (целое число дней от 24 ноября 4714 г. до н. э. 00:00 по местному времени; см. Раздел В.7)
Q	квартал
RM	номер месяца римскими цифрами в верхнем регистре (I–XII; I=январь)

Код	Описание
rm	номер месяца римскими цифрами в нижнем регистре (i-xii; i=январь)
TZ	сокращённое название часового пояса в верхнем регистре (поддерживается только в to_char)
tz	сокращённое название часового пояса в нижнем регистре (поддерживается только в to_char)
TZH	часы часового пояса
TZM	минуты часового пояса
OF	смещение часового пояса от UTC (поддерживается только в to_char)

К любым кодам форматирования можно добавить модификаторы, изменяющие их поведение. Например, шаблон форматирования FMMonth включает код Month с модификатором FM. Модификаторы, предназначенные для форматирования даты/времени, перечислены в [Таблице 9.28](#).

Таблица 9.28. Модификаторы кодов для форматирования даты/времени

Модификатор	Описание	Пример
Приставка FM	режим заполнения (подавляет ведущие нули и дополнение пробелами)	FMMonth
Окончание TH	окончание порядкового числительного в верхнем регистре	DDTH, например 12TH
Окончание th	окончание порядкового числительного в нижнем регистре	DDth, например 12th
Приставка FX	глобальный параметр фиксированного формата (см. замечания)	FX Month DD Day
Приставка TM	режим перевода (используются локализованные названия дней и месяцев, исходя из lc_time)	TMMonth
Окончание SP	режим числа прописью (не реализован)	DDSP

Замечания по использованию форматов даты/времени:

- FM подавляет дополняющие пробелы и нули справа, которые в противном случае будут добавлены, чтобы результат имел фиксированную ширину. В Postgres Pro модификатор FM действует только на следующий код, тогда как в Oracle FM её действие распространяется на все последующие коды, пока не будет отключено последующим модификатором FM.
- TM подавляет замыкающие пробелы вне зависимости от указания FM.
- Функции to_timestamp и to_date игнорируют регистр букв во входной строке; поэтому, например, для шаблонов MON, Mon и mon подойдут одни и те же строки. Если используется приставка TM, смена регистра производится в соответствии с правилом сортировки, установленным для входной строки (см. [Раздел 22.2](#)).
- to_timestamp и to_date пропускают повторяющиеся пробелы в начале входной строки и вокруг значений даты и времени, если только не используется приставка FX. Например, to_timestamp(' 2000 JUN', 'YYYY MON') и to_timestamp('2000 - JUN', 'YYYY-MON') будут работать, но to_timestamp('2000 JUN', 'FXYYYY MON') выдаст ошибку, так как to_timestamp ожидает только один пробел. Приставка FX должна быть первой в шаблоне.

- Разделитель (пробел или отличный от цифры/буквы символ) в строке шаблона функций `to_timestamp` и `to_date` соответствует любому разделителю во входной строке или пропускается, если только не добавлена приставка `FX`. Например, `to_timestamp('2000JUN', 'YYYY///MON')` и `to_timestamp('2000/JUN', 'YYYY MON')` будут работать, но `to_timestamp('2000//JUN', 'YYYY/MON')` выдаст ошибку, так как количество разделителей во входной строке превышает количество разделителей в шаблоне.

Если добавляется приставка `FX`, разделитель в строке шаблона соответствует ровно одному символу во входной строке. Но заметьте, что символ во входной строке не обязательно должен совпадать с символом разделителя в шаблоне. Например, `to_timestamp('2000/JUN', 'FXYYYY MON')` будет работать, а `to_timestamp('2000/JUN', 'FXYYYY MON')` выдаст ошибку, потому что второй пробел в строке шаблона забирает букву `J` из входной строки.

- Коду шаблона `TZH` может соответствовать число со знаком. Без приставки `FX` знаки минуса могут быть неоднозначными и восприниматься как разделители. Эта неоднозначность разрешается следующим образом: если число разделителей перед `TZH` в строке шаблона меньше числа разделителей перед знаком минуса во входной строке, знак минус воспринимается как относящийся к `TZH`. В противном случае знак минуса воспринимается как разделитель значений. Например, в `to_timestamp('2000 -10', 'YYYY TZH')` в поле `TZH` попадает `-10`, а в `to_timestamp('2000 -10', 'YYYY TZH')` в `TZH` попадает значение `10`.
- Шаблоны для `to_char` могут содержать обычный текст; он будет выведен в неизменном виде. Чтобы принудительно вывести текст буквально, даже если он содержит шаблонные коды, подстроку с ним можно заключить в кавычки. Например, в строке `"Hello Year "YYYY"`, код `YYYY` будет заменён годом, а буква `Y` в слове `Year` останется неизменной. В функциях `to_date`, `to_number` и `to_timestamp` при обработке подстрок в кавычках и буквального текста некоторой длины пропускается такое же число символов во входной строке; например, при обработке подстроки `"XX"` будут пропущены два символа (любые, не обязательно `XX`).

Подсказка

До Postgres Pro 12 во входной строке можно было пропускать произвольный текст, используя в шаблоне символы, отличные от цифр и букв. Например, раньше работало `to_timestamp('2000y6m1d', 'yyyy-MM-DD')`. Теперь для этой цели можно использовать только буквы. Например, шаблоны `to_timestamp('2000y6m1d', 'yyyymMMtDDt')` и `to_timestamp('2000y6m1d', 'yyyy"y"MM"m"DD"d")` пропускают `y`, `m` и `d`.

- Если вам нужно получить на выходе двойные кавычки, вы должны предварить их символом обратной косой черты, например: `"\"YYYY Month\""`. В остальном этот символ вне кавычек воспринимается как обычный. Внутри строки в кавычках он указывает, что следующий символ должен восприниматься буквально, каким бы он ни был (но это имеет смысл, только если следующий символ — кавычки или обратная косая черта).
- Если в функциях `to_timestamp` и `to_date` формат года определяется менее, чем 4 цифрами, например, как `YYY`, и в переданном значении года тоже меньше 4 цифр, год пересчитывается в максимально близкий к году 2020, т. е. 95 воспринимается как 1995.
- Функции `to_timestamp` и `to_date` воспринимают отрицательные значения годов как относящиеся к годам до н. э. Если же указать отрицательное значение и добавить явный признак `BC` (до н. э.), год будет относиться к н. э. Нулевое значение года воспринимается как 1 год до н. э.
- В функциях `to_timestamp` и `to_date` с преобразованием `YYYY` связано ограничение, когда обрабатываемый год записывается более чем 4 цифрами. После `YYYY` необходимо будет добавить нецифровой символ или соответствующий код, иначе год всегда будет восприниматься как 4 цифры. Например, в `to_date('200001130', 'YYYYMMDD')` (с годом 20000) год будет интерпретирован как состоящий из 4 цифр; чтобы исправить ситуацию, нужно добавить нецифровой разделитель после года, как в `to_date('20000-1130', 'YYYY-MMDD')`, или код как в `to_date('20000Nov30', 'YYYYMonDD')`.

- Функции `to_timestamp` и `to_date` принимают поле `CC` (век), но игнорируют его, если в шаблоне есть поле `YYY`, `YYYY` или `Y`, `YY`. Если `CC` используется с `YY` или `Y`, результатом будет год в данном столетии. Если присутствует только код столетия, без года, подразумевается первый год этого века.
- Функции `to_timestamp` и `to_date` принимают названия и номера дней недели (`DAY`, `D` и связанные типы полей), но игнорируют их при вычислении результата. То же самое происходит с полями квартала (`Q`).
- Функциям `to_timestamp` и `to_date` можно передать даты по недельному календарю ISO 8601 (отличающиеся от григорианских) одним из двух способов:
 - Год, номер недели и дня недели: например, `to_date('2006-42-4', 'IYYY-IW-ID')` возвращает дату 2006-10-19. Если день недели опускается, он считается равным 1 (понедельнику).
 - Год и день года: например, `to_date('2006-291', 'IYYY-IDDD')` также возвращает 2006-10-19.

Попытка ввести дату из смеси полей григорианского и недельного календаря ISO 8601 бессмысленна, поэтому это будет считаться ошибкой. В контексте ISO 8601 понятия «номер месяца» и «день месяца» не существуют, а в григорианском календаре нет понятия номера недели по ISO.

Внимание

Тогда как `to_date` не примет смесь полей григорианского и недельного календаря ISO, `to_char` способна на это, так как форматы вроде `YYYY-MM-DD` (`IYYY-IDDD`) могут быть полезны. Но избегайте форматов типа `IYYY-MM-DD`; в противном случае с датами в начале года возможны сюрпризы. (За дополнительными сведениями обратитесь к [Подразделу 9.9.1.](#))

- Функция `to_timestamp` воспринимает поля миллисекунд (`MS`) или микросекунд (`US`) как дробную часть число секунд. Например, `to_timestamp('12.3', 'SS.MS')` — это не 3 миллисекунды, а 300, так как это значение воспринимается как $12 + 0.3$ секунды. Это значит, что для формата `SS.MS` входные значения 12.3, 12.30 и 12.300 задают одно и то же число миллисекунд. Чтобы получить три миллисекунды, время нужно записать в виде 12.003, тогда оно будет воспринято как $12 + 0.003 = 12.003$ сек.
- Ещё более сложный пример: `to_timestamp('15:12:02.020.001230', 'HH24:MI:SS.MS.US')` будет преобразовано в 15 часов, 12 минут и 2 секунды + 20 миллисекунд + 1230 микросекунд = 2.021230 seconds.
- Нумерация дней недели в `to_char(..., 'ID')` соответствует функции `extract(isodow from ...)`, но нумерация `to_char(..., 'D')` не соответствует нумерации, принятой в `extract(dow from ...)`.
 - Функция `to_char(interval)` обрабатывает форматы `HH` и `HH12` в рамках 12 часов, то есть 0 и 36 часов будут выводиться как 12, тогда как `HH24` выводит число часов полностью, и для значений `interval` результат может превышать 23.

Коды форматирования числовых значений перечислены в [Таблице 9.29.](#)

Таблица 9.29. Коды форматирования чисел

Код	Описание
9	позиция цифры (может отсутствовать, если цифра незначащая)
0	позиция цифры (присутствует всегда, даже если цифра незначащая)

Код	Описание
.	десятичная точка
,	разделитель групп (тысяч)
PR	отрицательное значение в угловых скобках
S	знак, добавляемый к числу (с учётом локали)
L	символ денежной единицы (с учётом локали)
D	разделитель целой и дробной части числа (с учётом локали)
G	разделитель групп (с учётом локали)
MI	знак минус в заданной позиции (если число < 0)
PL	знак плюс в заданной позиции (если число > 0)
SG	знак плюс или минус в заданной позиции
RN	число римскими цифрами (в диапазоне от 1 до 3999)
TH или th	окончание порядкового числительного
V	сдвиг на заданное количество цифр (см. замечания)
EEEE	экспоненциальная запись числа

Замечания по использованию форматов чисел:

- 0 обозначает позицию цифры, которая будет выводиться всегда, даже если это незначащий ноль слева или справа. 9 также обозначает позицию цифры, но если это незначащий ноль слева, он заменяется пробелом, а если справа и задан режим заполнения, он удаляется. (Для функции `to_number()` эти два символа равнозначны.)
- Если в формате числа предусмотрено меньше знаков после запятой, чем в форматируемом числе, функция `to_char()` округлит число до указанного количества знаков.
- Символы шаблона S, L, D и G представляют знак, символ денежной единицы, десятичную точку и разделитель тысяч, как их определяет текущая локаль (см. [lc_monetary](#) и [lc_numeric](#)). Символы точка и запятая представляют те же символы, обозначающие десятичную точку и разделитель тысяч, но не зависят от локали.
- Если в шаблоне `to_char()` отсутствует явное указание положения знака, для него резервируется одна позиция рядом с числом (слева от него). Если левее нескольких 9 помещён S, знак также будет приписан слева к числу.
- Знак числа, полученный кодами SG, PL или MI, не присоединяется к числу; например, `to_char(-12, 'MI9999')` выдаёт '- 12', тогда как `to_char(-12, 'S9999')` — '-12'. (В Oracle MI не может идти перед 9, наоборот 9 нужно указать перед MI.)
- TH не преобразует значения меньше 0 и не поддерживает дробные числа.
- PL, SG и TH — расширения Postgres Pro.
- В `to_number` при использовании шаблонных кодов, не обозначающих данные, таких как L и TH, пропускается соответствующее количество входных символов. При этом не имеет значения, совпадают ли они с символами шаблона, если только это не символы данных (то есть цифры, знак числа, десятичная точка или запятая). Например, для подстроки TH будут пропущены два символа, не представляющие данные.
- V с `to_char` умножает вводимое значение на 10^n , где n — число цифр, следующих за V. V с `to_number` подобным образом делит значение. Функции `to_char` и `to_number` не поддерживают V с дробными числами (например, 99.9V99 не допускается).

- Код EEEE (научная запись) не может сочетаться с любыми другими вариантами форматирования или модификаторами, за исключением цифр и десятичной точки, и должен располагаться в конце строки шаблона (например, 9.99EEEE — допустимый шаблон).

Для изменения поведения кодов к ним могут быть применены определённые модификаторы. Например, FM99.99 обрабатывается как код 99.99 с модификатором FM. Все модификаторы для форматирования чисел перечислены в [Таблице 9.30](#).

Таблица 9.30. Модификаторы шаблонов для форматирования чисел

Модификатор	Описание	Пример
Приставка FM	режим заполнения (подавляет завершающие нули и дополнение пробелами)	FM99.99
Окончание TH	окончание порядкового числительного в верхнем регистре	999TH
Окончание th	окончание порядкового числительного в нижнем регистре	999th

В [Таблице 9.31](#) приведены некоторые примеры использования функции to_char.

Таблица 9.31. Примеры to_char

Выражение	Результат
to_char(current_timestamp, 'Day, DD HH12:MI:SS')	'Tuesday , 06 05:39:18'
to_char(current_timestamp, 'FMDay, FMDD HH12:MI:SS')	'Tuesday, 6 05:39:18'
to_char(-0.1, '99.99')	' -.10'
to_char(-0.1, 'FM9.99')	'-.1'
to_char(-0.1, 'FM90.99')	'-0.1'
to_char(0.1, '0.9')	' 0.1'
to_char(12, '9990999.9')	' 0012.0'
to_char(12, 'FM9990999.9')	'0012.'
to_char(485, '999')	' 485'
to_char(-485, '999')	'-485'
to_char(485, '9 9 9')	' 4 8 5'
to_char(1485, '9,999')	' 1,485'
to_char(1485, '9G999')	' 1 485'
to_char(148.5, '999.999')	' 148.500'
to_char(148.5, 'FM999.999')	'148.5'
to_char(148.5, 'FM999.990')	'148.500'
to_char(148.5, '999D999')	' 148,500'
to_char(3148.5, '9G999D999')	' 3 148,500'
to_char(-485, '999S')	'485-'
to_char(-485, '999MI')	'485-'
to_char(485, '999MI')	'485 '
to_char(485, 'FM999MI')	'485'
to_char(485, 'PL999')	'+485'

Выражение	Результат
<code>to_char(485, 'SG999')</code>	<code>'+485'</code>
<code>to_char(-485, 'SG999')</code>	<code>'-485'</code>
<code>to_char(-485, '9SG99')</code>	<code>'4-85'</code>
<code>to_char(-485, '999PR')</code>	<code>'<485>'</code>
<code>to_char(485, 'L999')</code>	<code>'DM 485'</code>
<code>to_char(485, 'RN')</code>	<code>'CDLXXXV'</code>
<code>to_char(485, 'FMRN')</code>	<code>'CDLXXXV'</code>
<code>to_char(5.2, 'FMRN')</code>	<code>'V'</code>
<code>to_char(482, '999th')</code>	<code>' 482nd'</code>
<code>to_char(485, '"Good number:"999')</code>	<code>'Good number: 485'</code>
<code>to_char(485.8, '"Pre:"999" Post:" .999')</code>	<code>'Pre: 485 Post: .800'</code>
<code>to_char(12, '99V999')</code>	<code>' 12000'</code>
<code>to_char(12.4, '99V999')</code>	<code>' 12400'</code>
<code>to_char(12.45, '99V9')</code>	<code>' 125'</code>
<code>to_char(0.0004859, '9.99EEEE')</code>	<code>' 4.86e-04'</code>

9.9. Операторы и функции даты/времени

Все существующие функции для обработки даты/времени перечислены в [Таблице 9.33](#), а подробнее они описаны в следующих подразделах. Поведение основных арифметических операторов (+, * и т. д.) описано в [Таблице 9.32](#). Функции форматирования этих типов данных были перечислены в [Разделе 9.8](#). Общую информацию об этих типах вы получили (или можете получить) в [Разделе 8.5](#).

Помимо этого, для типов даты/времени имеются обычные операторы сравнения, показанные в [Таблице 9.1](#). Значения даты и даты со временем (с часовым поясом или без него) можно сравнивать как угодно, тогда как значения только времени (с часовым поясом или без него) и интервалы допустимо сравнивать, только если их типы совпадают. При сравнении даты со временем без часового пояса и даты со временем с часовым поясом предполагается, что первое значение задано в часовом поясе, установленном параметром [TimeZone](#), и оно пересчитывается в UTC для сравнения со вторым значением (внутри уже представленным в UTC). Аналогичным образом, при сравнении значений даты и даты со времени первое считается соответствующим полночи в часовом поясе `TimeZone`.

Все описанные ниже функции и операторы, принимающие аргументы `time` или `timestamp`, фактически представлены в двух вариациях: одна принимает тип `time with time zone` или `timestamp with time zone`, а вторая — `time without time zone` или `timestamp without time zone`. Для краткости эти вариации здесь не разделяются. Кроме того, операторы + и * определяются парами, наделяющими их переместительным свойством (например, `date + integer` и `integer + date`); здесь приводится только один вариант для каждой пары.

Таблица 9.32. Операторы даты/времени

Оператор Описание Пример(ы)
<code>date + integer → date</code> Добавляет к дате заданное число дней <code>date '2001-09-28' + 7 → 2001-10-05</code>
<code>date + interval → timestamp</code>

Оператор	Описание	Пример(ы)
	Добавляет к дате интервал	<code>date '2001-09-28' + interval '1 hour' → 2001-09-28 01:00:00</code>
<code>date + time → timestamp</code>	Добавляет к дате время	<code>date '2001-09-28' + time '03:00' → 2001-09-28 03:00:00</code>
<code>interval + interval → interval</code>	Складывает интервалы	<code>interval '1 day' + interval '1 hour' → 1 day 01:00:00</code>
<code>timestamp + interval → timestamp</code>	Добавляет к отметке времени интервал	<code>timestamp '2001-09-28 01:00' + interval '23 hours' → 2001-09-29 00:00:00</code>
<code>time + interval → time</code>	Добавляет к времени интервал	<code>time '01:00' + interval '3 hours' → 04:00:00</code>
<code>- interval → interval</code>	Меняет направление интервала	<code>- interval '23 hours' → -23:00:00</code>
<code>date - date → integer</code>	Вычитает даты, выдавая разницу в днях	<code>date '2001-10-01' - date '2001-09-28' → 3</code>
<code>date - integer → date</code>	Вычитает из даты заданное число дней	<code>date '2001-10-01' - 7 → 2001-09-24</code>
<code>date - interval → timestamp</code>	Вычитает из даты интервал	<code>date '2001-09-28' - interval '1 hour' → 2001-09-27 23:00:00</code>
<code>time - time → interval</code>	Вычитает из одного времени другое	<code>time '05:00' - time '03:00' → 02:00:00</code>
<code>time - interval → time</code>	Вычитает из времени интервал	<code>time '05:00' - interval '2 hours' → 03:00:00</code>
<code>timestamp - interval → timestamp</code>	Вычитает из отметки времени интервал	<code>timestamp '2001-09-28 23:00' - interval '23 hours' → 2001-09-28 00:00:00</code>
<code>interval - interval → interval</code>	Вычитает из одного интервала другой	<code>interval '1 day' - interval '1 hour' → 1 day -01:00:00</code>
<code>timestamp - timestamp → interval</code>	Вычитает из одной отметки времени другую (преобразуя 24-часовые интервалы в дни подобно <code>justify_hours()</code>)	<code>timestamp '2001-09-29 03:00' - timestamp '2001-07-27 12:00' → 63 days 15:00:00</code>

Оператор
Описание
Пример(ы)
<code>interval * double precision → interval</code> Умножает интервал на скалярное значение <code>interval '1 second' * 900 → 00:15:00</code> <code>interval '1 day' * 21 → 21 days</code> <code>interval '1 hour' * 3.5 → 03:30:00</code>
<code>interval / double precision → interval</code> Делит интервал на скалярное значение <code>interval '1 hour' / 1.5 → 00:40:00</code>

Таблица 9.33. Функции даты/времени

Функция
Описание
Пример(ы)
<code>age (timestamp, timestamp) → interval</code> Вычитает аргументы и выдаёт «символический» результат с годами и месяцами, а не просто днями <code>age(timestamp '2001-04-10', timestamp '1957-06-13') → 43 years 9 mons 27 days</code> (43 года 9 месяцев 27 дней)
<code>age (timestamp) → interval</code> Вычитает аргумент из current_date (полночь текущего дня) <code>age(timestamp '1957-06-13') → 62 years 6 mons 10 days</code> (62 года 6 месяцев 10 дней)
<code>clock_timestamp () → timestamp with time zone</code> Текущая дата и время (меняется в процессе выполнения операторов); см. Подраздел 9.9.5 <code>clock_timestamp () → 2019-12-23 14:39:53.662522-05</code>
<code>current_date → date</code> Текущая дата; см. Подраздел 9.9.5 <code>current_date → 2019-12-23</code>
<code>current_time → time with time zone</code> Текущее время суток; см. Подраздел 9.9.5 <code>current_time → 14:39:53.662522-05</code>
<code>current_time (integer) → time with time zone</code> Текущее время суток (с ограниченной точностью); см. Подраздел 9.9.5 <code>current_time(2) → 14:39:53.66-05</code>
<code>current_timestamp → timestamp with time zone</code> Текущая дата и время (на момент начала транзакции); см. Подраздел 9.9.5 <code>current_timestamp → 2019-12-23 14:39:53.662522-05</code>
<code>current_timestamp (integer) → timestamp with time zone</code> Текущие дата и время (на момент начала транзакции; с ограниченной точностью); см. Подраздел 9.9.5 <code>current_timestamp(0) → 2019-12-23 14:39:53-05</code>
<code>date_bin (interval, timestamp, timestamp) → timestamp</code>

Функция	Описание	Пример(ы)
	Подгоняет заданное значение под интервал, отсчитывая от указанного начального момента; см. Подраздел 9.9.3	<code>date_bin('15 minutes', timestamp '2001-02-16 20:38:40', timestamp '2001-02-16 20:05:00') → 2001-02-16 20:35:00</code>
<code>date_part (text, timestamp) → double precision</code>	Возвращает поле даты/времени (равнозначно <code>extract</code>); см. Подраздел 9.9.1	<code>date_part('hour', timestamp '2001-02-16 20:38:40') → 20</code>
<code>date_part (text, interval) → double precision</code>	Возвращает поле интервала (равнозначно <code>extract</code>); см. Подраздел 9.9.1	<code>date_part('month', interval '2 years 3 months') → 3</code>
<code>date_trunc (text, timestamp) → timestamp</code>	Отсекает компоненты даты до заданной точности; см. Подраздел 9.9.2	<code>date_trunc('hour', timestamp '2001-02-16 20:38:40') → 2001-02-16 20:00:00</code>
<code>date_trunc (text, timestamp with time zone, text) → timestamp with time zone</code>	Отсекает компоненты даты до заданной точности в указанном часовом поясе; см. Подраздел 9.9.2	<code>date_trunc('day', timestamptz '2001-02-16 20:38:40+00', 'Australia/Sydney') → 2001-02-16 13:00:00+00</code>
<code>date_trunc (text, interval) → interval</code>	Отсекает компоненты даты до заданной точности; см. Подраздел 9.9.2	<code>date_trunc('hour', interval '2 days 3 hours 40 minutes') → 2 days 03:00:00</code>
<code>extract (field from timestamp) → numeric</code>	Возвращает поле даты/времени; см. Подраздел 9.9.1	<code>extract(hour from timestamp '2001-02-16 20:38:40') → 20</code>
<code>extract (field from interval) → numeric</code>	Возвращает поле интервала; см. Подраздел 9.9.1	<code>extract(month from interval '2 years 3 months') → 3</code>
<code>isfinite (date) → boolean</code>	Проверяет конечность даты (её отличие от +/-бесконечности)	<code>isfinite(date '2001-02-16') → true</code>
<code>isfinite (timestamp) → boolean</code>	Проверяет конечность времени (его отличие от +/-бесконечности)	<code>isfinite(timestamp 'infinity') → false</code>
<code>isfinite (interval) → boolean</code>	Проверяет конечность интервала (в настоящее время все интервалы конечны)	<code>isfinite(interval '4 hours') → true</code>
<code>justify_days (interval) → interval</code>	Преобразует интервал так, что каждый 30-дневный период считается одним месяцем	<code>justify_days(interval '1 year 65 days') → 1 year 2 mons 5 days</code>
<code>justify_hours (interval) → interval</code>	Преобразует интервал так, что каждый 24-часовой период считается одним днём	<code>justify_hours(interval '50 hours 10 minutes') → 2 days 02:10:00</code>
<code>justify_interval (interval) → interval</code>		

Функция	Описание	Пример(ы)
	Преобразует интервал с применением <code>justify_days</code> и <code>justify_hours</code> и дополнительно корректирует знаки	<code>justify_interval(interval '1 mon -1 hour')</code> → 29 days 23:00:00 (29 дней 23:00:00)
<code>localtime</code>	→ <code>time</code> Текущее время суток; см. Подраздел 9.9.5	<code>localtime</code> → 14:39:53.662522
<code>localtime (integer)</code>	→ <code>time</code> Текущее время суток (с ограниченной точностью); см. Подраздел 9.9.5	<code>localtime(0)</code> → 14:39:53
<code>localtimestamp</code>	→ <code>timestamp</code> Текущая дата и время (на момент начала транзакции); см. Подраздел 9.9.5	<code>localtimestamp</code> → 2019-12-23 14:39:53.662522
<code>localtimestamp (integer)</code>	→ <code>timestamp</code> Текущие дата и время (на момент начала транзакции; с ограниченной точностью); см. Подраздел 9.9.5	<code>localtimestamp(2)</code> → 2019-12-23 14:39:53.66
<code>make_date (year int, month int, day int)</code>	→ <code>date</code> Образует дату из полей: <code>year</code> (год), <code>month</code> (месяц) и <code>day</code> (день) (отрицательное значение поля <code>year</code> означает год до н. э.)	<code>make_date(2013, 7, 15)</code> → 2013-07-15
<code>make_interval ([years int [, months int [, weeks int [, days int [, hours int [, mins int [, secs double precision]]]]]])</code>	→ <code>interval</code> Образует интервал из полей: <code>years</code> (годы), <code>months</code> (месяцы), <code>weeks</code> (недели), <code>days</code> (дни), <code>hours</code> (часы), <code>minutes</code> (минуты) и <code>secs</code> (секунды), каждое из которых по умолчанию считается равным нулю.	<code>make_interval(days => 10)</code> → 10 days
<code>make_time (hour int, min int, sec double precision)</code>	→ <code>time</code> Образует время из полей: <code>hour</code> (час), <code>minute</code> (минута) и <code>sec</code> (секунда)	<code>make_time(8, 15, 23.5)</code> → 08:15:23.5
<code>make_timestamp (year int, month int, day int, hour int, min int, sec double precision)</code>	→ <code>timestamp</code> Образует момент времени из полей: <code>year</code> (год), <code>month</code> (месяц), <code>day</code> (день), <code>hour</code> (час), <code>minute</code> (минута) и <code>sec</code> (секунда) (отрицательное значение поля <code>year</code> означает год до н. э.)	<code>make_timestamp(2013, 7, 15, 8, 15, 23.5)</code> → 2013-07-15 08:15:23.5
<code>make_timestamptz (year int, month int, day int, hour int, min int, sec double precision [, timezone text])</code>	→ <code>timestamp with time zone</code> Образует дату и время с часовым поясом из полей: <code>year</code> (год), <code>month</code> (месяц), <code>day</code> (день), <code>hour</code> (час), <code>minute</code> (минута) и <code>sec</code> (секунда) (отрицательное значение поля <code>year</code> означает год до н. э.). Если параметр <code>timezone</code> (часовой пояс) не указан, используется текущий часовой пояс; в примерах предполагается часовой пояс Europe/London (Европа/Лондон).	<code>make_timestamptz(2013, 7, 15, 8, 15, 23.5)</code> → 2013-07-15 08:15:23.5+01 <code>make_timestamptz(2013, 7, 15, 8, 15, 23.5, 'America/New_York')</code> → 2013-07-15 13:15:23.5+01
<code>now ()</code>	→ <code>timestamp with time zone</code>	

Функция	Описание	Пример(ы)
	Текущая дата и время (на момент начала транзакции); см. Подраздел 9.9.5	<code>now()</code> → 2019-12-23 14:39:53.662522-05
	<code>statement_timestamp ()</code> → timestamp with time zone Текущая дата и время (на момент начала текущего оператора); см. Подраздел 9.9.5	<code>statement_timestamp()</code> → 2019-12-23 14:39:53.662522-05
	<code>timeofday ()</code> → text Текущая дата и время (как <code>clock_timestamp</code> , но в виде строки типа text); см. Подраздел 9.9.5	<code>timeofday()</code> → Mon Dec 23 14:39:53.662522 2019 EST
	<code>transaction_timestamp ()</code> → timestamp with time zone Текущая дата и время (на момент начала транзакции); см. Подраздел 9.9.5	<code>transaction_timestamp()</code> → 2019-12-23 14:39:53.662522-05
	<code>to_timestamp (double precision)</code> → timestamp with time zone Преобразует время эпохи Unix (число секунд с 1970-01-01 00:00:00+00) в дату/время с часовым поясом	<code>to_timestamp(1284352323)</code> → 2010-09-13 04:32:03+00

В дополнение к этим функциям поддерживается SQL-оператор `OVERLAPS`:

```
(начало1, конец1) OVERLAPS (начало2, конец2)
(начало1, длительность1) OVERLAPS (начало2, длительность2)
```

Его результатом будет `true`, когда два периода времени (определённые своими границами) пересекаются, и `false` в противном случае. Границы периода можно задать либо в виде пары дат, времени или дат со временем, либо как дату, время (или дату со временем) с интервалом. Когда указывается пара значений, первым может быть и начало, и конец периода: `OVERLAPS` автоматически считает началом периода меньшее значение. Периоды времени считаются наполовину открытыми, т. е. `начало ≤ время < конец`, если только `начало` и `конец` не равны — в этом случае период представляет один момент времени. Это означает, например, что два периода, имеющие только общую границу, не будут считаться пересекающимися.

```
SELECT (DATE '2001-02-16', DATE '2001-12-21') OVERLAPS
      (DATE '2001-10-30', DATE '2002-10-30');
```

Результат: true

```
SELECT (DATE '2001-02-16', INTERVAL '100 days') OVERLAPS
      (DATE '2001-10-30', DATE '2002-10-30');
```

Результат: false

```
SELECT (DATE '2001-10-29', DATE '2001-10-30') OVERLAPS
      (DATE '2001-10-30', DATE '2001-10-31');
```

Результат: false

```
SELECT (DATE '2001-10-30', DATE '2001-10-30') OVERLAPS
      (DATE '2001-10-30', DATE '2001-10-31');
```

Результат: true

При добавлении к метке типа `timestamp with time zone` значения `interval` (или при вычитании из него `interval`), поле дней в этой дате увеличивается (или уменьшается) на указанное число суток, а время суток остаётся неизменным. При пересечении границы перехода на летнее время (если в часовом поясе текущего сеанса производится этот переход) это означает, что `interval '1 day'` и `interval '24 hours'` не обязательно будут равны. Например, в часовом поясе `America/Denver`:

```
SELECT timestamp with time zone '2005-04-02 12:00:00-07' + interval '1 day';
```

Результат: 2005-04-03 12:00:00-06

```
SELECT timestamp with time zone '2005-04-02 12:00:00-07' + interval '24 hours';
```

Результат: 2005-04-03 13:00:00-06

Эта разница объясняется тем, что 2005-04-03 02:00 в часовом поясе America/Denver произошёл переход на летнее время.

Обратите внимание на возможную неоднозначность в поле `months` в результате функции `age`, вызванную тем, что число дней в разных месяцах неодинаково. Вычисляя оставшиеся дни месяца, Postgres Pro рассматривает месяц меньшей из двух дат. Например, результатом `age('2004-06-01', '2004-04-30')` будет `1 mon 1 day`, так как в апреле 30 дней, а то же выражение с датой 30 мая выдаст `1 mon 2 days`, так как в мае 31 день.

Вычитание дат и дат со временем также может быть нетривиальной операцией. Один принципиально простой способ выполнить такое вычисление — преобразовать каждое значение в количество секунд, используя `EXTRACT(EPOCH FROM ...)`, а затем найти разницу результатов; при этом будет получено число *секунд* между двумя датами. При этом будет учтено неодинаковое число дней в месяцах, изменения часовых поясов и переходы на летнее время. При вычитании дат или дат со временем с помощью оператора «-» выдаётся число дней (по 24 часа) и часов/минут/секунд между данными значениями, с учётом тех же факторов. Функция `age` возвращает число лет, месяцев, дней и часов/минут/секунд, выполняя вычитание по полям, а затем пересчитывая отрицательные значения. Различие этих подходов иллюстрируют следующие запросы. Показанные результаты были получены для часового пояса 'US/Eastern'; между двумя заданными датами произошёл переход на летнее время:

```
SELECT EXTRACT(EPOCH FROM timestampz '2013-07-01 12:00:00') -
       EXTRACT(EPOCH FROM timestampz '2013-03-01 12:00:00');
Результат:10537200.000000
SELECT (EXTRACT(EPOCH FROM timestampz '2013-07-01 12:00:00') -
       EXTRACT(EPOCH FROM timestampz '2013-03-01 12:00:00'))
       / 60 / 60 / 24;
Результат:121.95833333333333
SELECT timestampz '2013-07-01 12:00:00' - timestampz '2013-03-01 12:00:00';
Результат:121 days 23:00:00
SELECT age(timestampz '2013-07-01 12:00:00', timestampz '2013-03-01 12:00:00');
Результат:4 mons
```

9.9.1. EXTRACT, date_part

`EXTRACT(поле FROM источник)`

Функция `extract` извлекает поля, такие как год или час, из значений даты и времени. *Источником* должно быть заданное выражением значение типа `timestamp`, `date`, `time` или `interval`. (Значения `timestamp` и `time` могут быть как с указанием часового пояса, так и без.) *Поле* — это идентификатор или строка, с помощью которой выбирается поле для извлечения из значения источника. Для разных типов входных данных допустимы разные поля: например, поля меньше дней не могут быть извлечены из `date`, а поля дней и более не могут быть извлечены из `time`. Функция `extract` возвращает значения типа `numeric`.

Допустимы следующие имена полей:

`century`

Век; для значений `interval` — значение поля года, разделённое на сто

```
SELECT EXTRACT(CENTURY FROM TIMESTAMP '2000-12-16 12:21:13');
Результат:20
SELECT EXTRACT(CENTURY FROM TIMESTAMP '2001-02-16 20:38:40');
Результат:21
SELECT EXTRACT(CENTURY FROM DATE '0001-01-01 AD');
Результат:1
SELECT EXTRACT(CENTURY FROM DATE '0001-12-31 BC');
Результат:-1
SELECT EXTRACT(CENTURY FROM INTERVAL '2001 years');
Результат:20
```

day

День месяца (1-31); для значений `interval` — количество дней

```
SELECT EXTRACT(DAY FROM TIMESTAMP '2001-02-16 20:38:40');
Результат:16
SELECT EXTRACT(DAY FROM INTERVAL '40 days 1 minute');
Результат:40
```

decade

Год, делённый на 10

```
SELECT EXTRACT(DECADE FROM TIMESTAMP '2001-02-16 20:38:40');
Результат:200
```

dow

День недели, считая с воскресенья (0) до субботы (6)

```
SELECT EXTRACT(DOW FROM TIMESTAMP '2001-02-16 20:38:40');
Результат:5
```

Заметьте, что в `extract` дни недели нумеруются не так, как в функции `to_char(..., 'D')`.

doym

День года (1-365/366)

```
SELECT EXTRACT(DOY FROM TIMESTAMP '2001-02-16 20:38:40');
Результат:47
```

epoch

Для значений `timestamp with time zone` это число секунд с 1970-01-01 00:00:00 UTC (отрицательное для предшествующего времени); для значений `date` и `timestamp` — номинальное число секунд с 1970-01-01 00:00:00 без учёта часового пояса, переходов на летнее время и т. п.; для значений `interval` — общее количество секунд в интервале

```
SELECT EXTRACT(EPOCH FROM TIMESTAMP WITH TIME ZONE '2001-02-16 20:38:40.12-08');
Результат:982384720.120000
SELECT EXTRACT(EPOCH FROM TIMESTAMP '2001-02-16 20:38:40.12');
Результат:982355920.120000
SELECT EXTRACT(EPOCH FROM INTERVAL '5 days 3 hours');
Результат:442800.000000
```

Преобразовать время эпохи назад, в значение `timestamp with time zone`, с помощью `to_timestamp` можно так:

```
SELECT to_timestamp(982384720.12);
Результат:2001-02-17 04:38:40.12+00
```

Имейте в виду, что применяя `to_timestamp` к времени эпохи, извлечённому из значения `date` или `timestamp`, можно получить не вполне ожидаемый результат: эта функция подразумевает, что изначальное значение задано в часовом поясе UTC, но это может быть не так.

hour

Поле часов (0-23 в метках времени, в интервалах — без ограничений)

```
SELECT EXTRACT(HOUR FROM TIMESTAMP '2001-02-16 20:38:40');
Результат:20
```

isodow

День недели, считая с понедельника (1) до воскресенья (7)

```
SELECT EXTRACT(ISODOW FROM TIMESTAMP '2001-02-18 20:38:40');
Результат:7
```

Результат отличается от `dow` только для воскресенья. Такая нумерация соответствует ISO 8601.

`isoyear`

Год с нумерацией недель согласно ISO 8601, на который приходится дата

```
SELECT EXTRACT(ISOYEAR FROM DATE '2006-01-01');
```

*Результат:*2005

```
SELECT EXTRACT(ISOYEAR FROM DATE '2006-01-02');
```

*Результат:*2006

Год по недельному календарю ISO начинается с понедельника недели, в которой оказывается 4 января, так что в начале января или в конце декабря год по ISO может отличаться от года по григорианскому календарю. Подробнее об этом рассказывается в описании поля `week`.

`julian`

Юлианская дата, соответствующая дате или дате/времени. Значение будет дробным, если заданное время отличается от начала суток по местному времени. За дополнительной информацией обратитесь к [Разделу В.7](#).

```
SELECT EXTRACT(JULIAN FROM DATE '2006-01-01');
```

*Результат:*2453737

```
SELECT EXTRACT(JULIAN FROM TIMESTAMP '2006-01-01 12:00');
```

*Результат:*2453737.500000000000000000000000

`microseconds`

Значение секунд с дробной частью, умноженное на 1 000 000; заметьте, что оно включает и целые секунды

```
SELECT EXTRACT(MICROSECONDS FROM TIME '17:12:28.5');
```

*Результат:*28500000

`millennium`

Тысячелетие; для значений `interval` — значение поля года, разделённое на тысячу

```
SELECT EXTRACT(MILLENNIUM FROM TIMESTAMP '2001-02-16 20:38:40');
```

*Результат:*3

```
SELECT EXTRACT(MILLENNIUM FROM INTERVAL '2001 years');
```

*Результат:*2

Годы 20 века относятся ко второму тысячелетию. Третье тысячелетие началось 1 января 2001 г.

`milliseconds`

Значение секунд с дробной частью, умноженное на 1 000; заметьте, что оно включает и целые секунды.

```
SELECT EXTRACT(MILLISECONDS FROM TIME '17:12:28.5');
```

*Результат:*28500.000

`minute`

Минуты (0–59)

```
SELECT EXTRACT(MINUTE FROM TIMESTAMP '2001-02-16 20:38:40');
```

*Результат:*38

`month`

Номер месяца в году (1–12), а для `interval` — остаток от деления числа месяцев на 12 (0–11)

```
SELECT EXTRACT(MONTH FROM TIMESTAMP '2001-02-16 20:38:40');
```

*Результат:*2

```
SELECT EXTRACT(MONTH FROM INTERVAL '2 years 3 months');
```

*Результат:*3

```
SELECT EXTRACT(MONTH FROM INTERVAL '2 years 13 months');
```

Результат:1

quarter

Квартал (1–4), к которому относится дата

```
SELECT EXTRACT(QUARTER FROM TIMESTAMP '2001-02-16 20:38:40');
```

Результат:1

second

Секунды, включая дробную часть

```
SELECT EXTRACT(SECOND FROM TIMESTAMP '2001-02-16 20:38:40');
```

Результат:40.000000

```
SELECT EXTRACT(SECOND FROM TIME '17:12:28.5');
```

Результат:28.500000

timezone

Смещение часового пояса от UTC, представленное в секундах. Положительные значения соответствуют часовым поясам к востоку от UTC, а отрицательные — к западу. (Строго говоря, в Postgres Pro используется не UTC, так как секунды координации не учитываются.)

timezone_hour

Поле часов в смещении часового пояса

timezone_minute

Поле минут в смещении часового пояса

week

Номер недели в году по недельному календарю ISO 8601. По определению, недели ISO 8601 начинаются с понедельника, а первая неделя года включает 4 января этого года. Другими словами, первый четверг года всегда оказывается в 1 неделе этого года.

В системе нумерации недель ISO первые числа января могут относиться к 52-й или 53-й неделе предыдущего года, а последние числа декабря — к первой неделе следующего года. Например, 2005-01-01 относится к 53-й неделе 2004 г., а 2006-01-01 — к 52-й неделе 2005 г., тогда как 2012-12-31 включается в первую неделю 2013 г. Поэтому для получения согласованных результатов рекомендуется использовать поле `isoyear` в паре с `week`.

```
SELECT EXTRACT(WEEK FROM TIMESTAMP '2001-02-16 20:38:40');
```

Результат:7

year

Поле года. Учтите, что года 0 не было, и это следует иметь в виду, вычитая из годов нашей эры годы до нашей эры.

```
SELECT EXTRACT(YEAR FROM TIMESTAMP '2001-02-16 20:38:40');
```

Результат:2001

При обработке значения `interval` функция `extract` создаёт значения полей, соответствующие интерпретации, которая используется функцией вывода интервала. При этом, если начать с ненормализованного представления интервала, могут быть получены неожиданные результаты, например:

```
SELECT INTERVAL '80 minutes';
```

Результат: 01:20:00

```
SELECT EXTRACT(MINUTES FROM INTERVAL '80 minutes');
```

Результат: 20

Примечание

С аргументом `+/-бесконечность` `extract` возвращает `+/-бесконечность` для монотонно увеличивающихся полей (`epoch`, `julian`, `year`, `isoyear`, `decade`, `century` и `millennium`). Для других

полей возвращается NULL. До версии 9.6 Postgres Pro возвращал ноль для всех случаев с бесконечными аргументами.

Функция `extract` в основном предназначена для вычислительных целей. Функции форматирования даты/времени описаны в [Разделе 9.8](#).

Функция `date_part` эмулирует традиционный для Ingres эквивалент стандартной SQL-функции `extract`:

```
date_part('поле', источник)
```

Заметьте, что здесь параметр *поле* должен быть строковым значением, а не именем. Функция `date_part` воспринимает те же поля, что и `extract`. По историческим причинам функция `date_part` возвращает значения типа `double precision`. В некоторых случаях это может привести к потере точности. Поэтому вместо неё рекомендуется использовать функцию `extract`.

```
SELECT date_part('day', TIMESTAMP '2001-02-16 20:38:40');
```

*Результат:*16

```
SELECT date_part('hour', INTERVAL '4 hours 3 minutes');
```

*Результат:*4

9.9.2. date_trunc

Функция `date_trunc` работает подобно `trunc` для чисел.

```
date_trunc(поле, значение [, часовой_пояс ])
```

Здесь *значение* — выражение типа `timestamp`, `timestamp with time zone` или `interval`. (Значения типов `date` и `time` автоматически приводятся к типам `timestamp` и `interval`, соответственно.) Параметр *поле* определяет, до какой точности обрезать переданное значение. В возвращаемом значении, имеющем также тип `timestamp`, `timestamp with time zone` или `interval`, все поля, менее значимые, чем заданное, будут равны нулю (или одному, если это номер дня или месяца).

Параметр *поле* может принимать следующие значения:

```
microseconds
milliseconds
second
minute
hour
day
week
month
quarter
year
decade
century
millennium
```

Когда входное значение имеет тип `timestamp with time zone`, оно обрезается с учётом заданного часового пояса; например, если обрезать значение до поля `day` (день), в результате будет получена полночь в этом часовом поясе. По умолчанию входное значение обрезается с учётом параметра [TimeZone](#), но дополнительный аргумент *часовой_пояс* позволяет выбрать и другой пояс. Название часового пояса может задаваться любым из способов, описанных в [Подразделе 8.5.3](#).

Часовой пояс нельзя задать для значений типа `timestamp without time zone` или `interval`. Такие значения всегда воспринимаются как есть.

Несколько примеров (выбран часовой пояс `America/New_York`):

```
SELECT date_trunc('hour', TIMESTAMP '2001-02-16 20:38:40');
```

Результат: 2001-02-16 20:00:00

```
SELECT date_trunc('year', TIMESTAMP '2001-02-16 20:38:40');
Результат: 2001-01-01 00:00:00
SELECT date_trunc('day', TIMESTAMP WITH TIME ZONE '2001-02-16 20:38:40+00');
Результат: 2001-02-16 00:00:00-05
SELECT date_trunc('day', TIMESTAMP WITH TIME ZONE '2001-02-16 20:38:40+00', 'Australia/
Sydney');
Результат: 2001-02-16 08:00:00-05
SELECT date_trunc('hour', INTERVAL '3 days 02:47:33');
Результат: 3 days 02:00:00
```

9.9.3. date_bin

Функция `date_bin` «подгоняет» заданное значение под интервал (*шаг*), отсчитывая от указанного начального момента.

`date_bin(шаг, значение, источник)`

Здесь *значение* — выражение типа `timestamp`, `timestamp with time zone`. (Значения типов `date` автоматически приводятся к типу `timestamp`.) В качестве *шага* передаётся выражение типа `interval`. Результат этой функции, также имеющий тип `timestamp`, `timestamp with time zone` или `interval`, — начало интервала, под который подгоняется *значение*.

Примеры:

```
SELECT date_bin('15 minutes', TIMESTAMP '2020-02-11 15:44:17', TIMESTAMP '2001-01-01');
Результат: 2020-02-11 15:30:00
SELECT date_bin('15 minutes', TIMESTAMP '2020-02-11 15:44:17', TIMESTAMP '2001-01-01
00:02:30');
Результат: 2020-02-11 15:32:30
```

В случае целых единиц измерения (1 минута, 1 час и т. д.), она даёт тот же результат, что и аналогичный вызов функции `date_trunc`, с той лишь разницей, что `date_bin` может округлить дату до произвольного интервала.

Параметр *шаг* интервала должен быть больше нуля и задаваться в единицах измерения меньше месяца.

9.9.4. AT TIME ZONE

Оператор `AT TIME ZONE` преобразует дату/время без часового пояса в дату/время с часовым поясом и обратно, а также пересчитывает значения `time with time zone` для различных часовых поясов. Его вариации показаны в [Таблице 9.34](#).

Таблица 9.34. Разновидности AT TIME ZONE

Оператор	Описание	Пример(ы)
<code>timestamp without time zone AT TIME ZONE часовой_пояс</code>	<code>→ timestamp with time zone</code> Переводит значение даты/времени без часового пояса в дату/время с часовым поясом, в предположении, что входное значение задано в указанном поясе.	<code>timestamp '2001-02-16 20:38:40' at time zone 'America/Denver' → 2001-02-17 03:38:40+00</code>
<code>timestamp with time zone AT TIME ZONE часовой_пояс</code>	<code>→ timestamp without time zone</code> Переводит значение даты/времени с часовым поясом в дату/время без часового пояса, которое соответствует входному значению в указанном поясе.	<code>timestamp with time zone '2001-02-16 20:38:40-05' at time zone 'America/Denver' → 2001-02-16 18:38:40</code>
<code>time with time zone AT TIME ZONE часовой_пояс</code>	<code>→ time with time zone</code>	

Оператор	Описание	Пример(ы)
	Переводит значение времени с часовым поясом в другой часовой пояс. Так как время задётся без даты, в расчёте используется действующее в данный момент смещение указанного часового пояса от UTC.	time with time zone '05:34:17-05' at time zone 'UTC' → 10:34:17+00

В этих выражениях желаемый *часовой_пояс* можно задать либо в виде текстовой строки (например, 'America/Los_Angeles'), либо как интервал (например, INTERVAL '-08:00'). В первом случае название часового пояса можно указать любым из способов, описанных в [Подразделе 8.5.3](#). Вариант с интервалом полезен, только если для часового пояса смещение от UTC всегда постоянно, что на практике встречается нечасто.

Примеры (параметр [TimeZone](#) имеет значение America/Los_Angeles):

```
SELECT TIMESTAMP '2001-02-16 20:38:40' AT TIME ZONE 'America/Denver';
Результат: 2001-02-16 19:38:40-08
SELECT TIMESTAMP WITH TIME ZONE '2001-02-16 20:38:40-05' AT TIME ZONE 'America/Denver';
Результат: 2001-02-16 18:38:40
SELECT TIMESTAMP '2001-02-16 20:38:40' AT TIME ZONE 'Asia/Tokyo' AT TIME ZONE 'America/Chicago';
Результат: 2001-02-16 05:38:40
```

В первом примере для значения, заданного без часового пояса, указывается часовой пояс и полученное время выводится в текущем часовом поясе (заданном параметром *TimeZone*). Во втором примере значение времени смещается в заданный часовой пояс и выдаётся без указания часового пояса. Это позволяет хранить и выводить значения с часовым поясом, отличным от текущего. В третьем примере время в часовом поясе Токио преобразуется для часового пояса Чикаго.

Функция `timezone(часовой_пояс, время)` равнозначна SQL-совместимой конструкции `время AT TIME ZONE часовой_пояс`.

9.9.5. Текущая дата/время

Postgres Pro предоставляет набор функций, результат которых зависит от текущей даты и времени. Все следующие функции соответствуют стандарту SQL и возвращают значения, отражающие время начала текущей транзакции:

```
CURRENT_DATE
CURRENT_TIME
CURRENT_TIMESTAMP
CURRENT_TIME(точность)
CURRENT_TIMESTAMP(точность)
LOCALTIME
LOCALTIMESTAMP
LOCALTIME(точность)
LOCALTIMESTAMP(точность)
```

`CURRENT_TIME` и `CURRENT_TIMESTAMP` возвращают время с часовым поясом. В результатах `LOCALTIME` и `LOCALTIMESTAMP` нет информации о часовом поясе.

`CURRENT_TIME`, `CURRENT_TIMESTAMP`, `LOCALTIME` и `LOCALTIMESTAMP` могут принимать необязательный параметр точности, определяющий, до какого знака после запятой следует округлять поле секунд. Если этот параметр отсутствует, результат будет иметь максимально возможную точность.

Несколько примеров:

```
SELECT CURRENT_TIME;
Результат: 14:39:53.662522-05
SELECT CURRENT_DATE;
Результат: 2019-12-23
```

```
SELECT CURRENT_TIMESTAMP;
Результат: 2019-12-23 14:39:53.662522-05
SELECT CURRENT_TIMESTAMP(2);
Результат: 2019-12-23 14:39:53.66-05
SELECT LOCALTIMESTAMP;
Результат: 2019-12-23 14:39:53.662522
```

Так как эти функции возвращают время начала текущей транзакции, во время транзакции эти значения не меняются. Это считается не ошибкой, а особенностью реализации: цель такого поведения в том, чтобы в одной транзакции «текущее» время было одинаковым и для разных изменений в одной транзакции записывалась одна отметка времени.

Примечание

В других СУБД эти значения могут изменяться чаще.

В Postgres Pro есть также функции, возвращающие время начала текущего оператора, а также текущее время в момент вызова функции. Таким образом, в Postgres Pro есть следующие функции, не описанные в стандарте SQL:

```
transaction_timestamp()
statement_timestamp()
clock_timestamp()
timeofday()
now()
```

Функция `transaction_timestamp()` равнозначна конструкции `CURRENT_TIMESTAMP`, но в её названии явно отражено, что она возвращает. Функция `statement_timestamp()` возвращает время начала текущего оператора (более точно, время получения последнего командного сообщения от клиента). Функции `statement_timestamp()` и `transaction_timestamp()` возвращают одно и то же значение в первом операторе транзакции, но в последующих операторах их показания будут расходиться. Функция `clock_timestamp()` возвращает фактическое текущее время, так что её значение меняется в рамках одного оператора SQL. Функция `timeofday()` существует в Postgres Pro по историческим причинам и, подобно `clock_timestamp()`, она возвращает фактическое текущее время, но представленное в виде форматированной строки типа `text`, а не значения `timestamp with time zone`. Функция `now()` — традиционный для Postgres Pro эквивалент функции `transaction_timestamp()`.

Все типы даты/времени также принимают специальное буквальное значение `now`, подразумевающее текущую дату и время (тоже на момент начала транзакции). Таким образом, результат следующих трёх операторов будет одинаковым:

```
SELECT CURRENT_TIMESTAMP;
SELECT now();
SELECT TIMESTAMP 'now'; -- см. замечание ниже
```

Подсказка

Не используйте третью форму для указания значения, которое будет вычисляться позднее, например, в предложении `DEFAULT` для столбца таблицы. Система преобразует `now` в значение `timestamp` в момент разбора константы, поэтому когда будет вставляться такое значение по умолчанию, в соответствующем столбце окажется время создания таблицы! Первые две формы будут вычисляться, только когда значение по умолчанию потребуется, так как это вызовы функции. Поэтому они дадут желаемый результат при добавлении строки в таблицу. (См. также [Подраздел 8.5.1.4.](#))

9.9.6. Задержка выполнения

В случае необходимости выполнение серверного процесса можно приостановить, используя следующие функции:

```
pg_sleep( double precision )
pg_sleep_for( interval )
pg_sleep_until( timestamp with time zone )
```

Функция `pg_sleep` переводит процесс текущего сеанса в спящее состояние на указанное число секунд (это число может быть дробным). В дополнение к ней для удобства добавлены две функции: `pg_sleep_for`, принимающая время задержки в типе `interval`, и `pg_sleep_until`, позволяющая задать определённое время выхода из спящего состояния. Например:

```
SELECT pg_sleep(1.5);
SELECT pg_sleep_for('5 minutes');
SELECT pg_sleep_until('tomorrow 03:00');
```

Примечание

Действительное разрешение интервала задержки зависит от платформы; обычно это 0.01. Фактическая длительность задержки не будет меньше указанного времени, но может быть больше, в зависимости, например от нагрузки на сервер. В частности, не гарантируется, что `pg_sleep_until` проснётся именно в указанное время, но она точно не проснётся раньше.

Предупреждение

Прежде чем вызывать `pg_sleep` или её вариации, убедитесь в том, что в текущем сеансе нет ненужных блокировок. В противном случае в состоянии ожидания могут перейти и другие сеансы, так что это отразится на системе в целом.

9.10. Функции для перечислений

Для типов перечислений (описанных в [Разделе 8.7](#)) предусмотрено несколько функций, которые позволяют сделать код чище, не «зашивая» в нём конкретные значения перечисления. Эти функции перечислены в [Таблице 9.35](#). В этих примерах подразумевается, что перечисление создано так:

```
CREATE TYPE rainbow AS ENUM ('red', 'orange', 'yellow', 'green', 'blue', 'purple');
```

Таблица 9.35. Функции для перечислений

Функция	Описание	Пример(ы)
<code>enum_first (anyenum) → anyenum</code>	Возвращает первое значение заданного перечисления.	<code>enum_first(null::rainbow) → red</code>
<code>enum_last (anyenum) → anyenum</code>	Возвращает последнее значение заданного перечисления.	<code>enum_last(null::rainbow) → purple</code>
<code>enum_range (anyenum) → anyarray</code>	Возвращает все значения заданного перечисления в упорядоченном массиве.	<code>enum_range(null::rainbow) → {red, orange, yellow, green, blue, purple}</code>
<code>enum_range (anyenum, anyenum) → anyarray</code>	Возвращает набор значений перечисления, лежащих между двумя заданными, в виде упорядоченного массива. Значения в параметрах должны принадлежать одному перечислению. Если в первом параметре передаётся NULL, результат функции начинается с первого значения перечисления, а если во втором — заканчивается последним.	

Функция	Описание	Пример(ы)
	<code>enum_range('orange'::rainbow, 'green'::rainbow)</code>	<code>→ {orange, yellow, green}</code>
	<code>enum_range(NULL, 'green'::rainbow)</code>	<code>→ {red, orange, yellow, green}</code>
	<code>enum_range('orange'::rainbow, NULL)</code>	<code>→ {orange, yellow, green, blue, purple}</code>

Заметьте, что за исключением варианта `enum_range` с двумя аргументами, эти функции не обращают внимание на конкретное переданное им значение; их интересует только объявленный тип. Они возвращают один и тот же результат, когда им передаётся `NULL` или любое другое значение типа. Обычно эти функции применяются к столбцам таблицы или аргументам внешних функций, а не к предопределённым типам, как в этих примерах.

9.11. Геометрические функции и операторы

Для геометрических типов `point`, `box`, `lseg`, `line`, `path`, `polygon` и `circle` разработан большой набор встроенных функций и операторов, представленный в [Таблице 9.36](#), [Таблице 9.37](#) и [Таблице 9.38](#).

Таблица 9.36. Геометрические операторы

Оператор	Описание	Пример(ы)
<code>геометрический_тип + point</code>	<code>→ геометрический_тип</code> Добавляет координаты второго аргумента типа <code>point</code> к каждой точке первого аргумента, осуществляя таким образом перенос объекта. Имеется для типов <code>point</code> , <code>box</code> , <code>path</code> и <code>circle</code> .	<code>box '(1,1), (0,0)' + point '(2,0)'</code> <code>→ (3,1), (2,0)</code>
<code>path + path</code>	<code>→ path</code> Соединяет два открытых пути (если один из путей замкнутый, возвращает <code>NULL</code>).	<code>path '[(0,0), (1,1)]' + path '[(2,2), (3,3), (4,4)]'</code> <code>→ [(0,0), (1,1), (2,2), (3,3), (4,4)]</code>
<code>геометрический_тип - point</code>	<code>→ геометрический_тип</code> Вычитает координаты второго аргумента типа <code>point</code> из каждой точки первого аргумента, осуществляя таким образом перенос объекта. Имеется для типов <code>point</code> , <code>box</code> , <code>path</code> и <code>circle</code> .	<code>box '(1,1), (0,0)' - point '(2,0)'</code> <code>→ (-1,1), (-2,0)</code>
<code>геометрический_тип * point</code>	<code>→ геометрический_тип</code> Умножает координаты каждой точки первого аргумента на координаты второго аргумента типа <code>point</code> (координаты точек воспринимаются как комплексные числа с вещественной и мнимой частью, результатом становится их обычное комплексное произведение). Если же рассматривать вторую точку как вектор, данная операция равнозначна умножению размера объекта и расстояния от начала координат на длину вектора с поворотом против часовой стрелки относительно начала координат на угол, равный углу между вектором и осью <code>x</code> . Имеется для типов <code>point</code> , <code>box</code> , ^a <code>path</code> , <code>circle</code> .	<code>path '((0,0), (1,0), (1,1))' * point '(3.0,0)'</code> <code>→ ((0,0), (3,0), (3,3))</code> <code>path '((0,0), (1,0), (1,1))' * point (cosd(45), sind(45))</code> <code>→ ((0,0), (0.7071067811865475, 0.7071067811865475), (0, 1.414213562373095))</code>
<code>геометрический_тип / point</code>	<code>→ геометрический_тип</code> Делит координаты каждой точки первого аргумента на координаты второго аргумента типа <code>point</code> (координаты точек воспринимаются как комплексные числа с вещественной и мнимой частью, результатом становится их обычное комплексное частное). Если же	

Оператор	Описание	Пример(ы)
	рассматривать вторую точку как вектор, данная операция равнозначна делению размера объекта и расстояния от начала координат на длину вектора с поворотом по часовой стрелке относительно начала координат на угол, равный углу между вектором и осью x. Имеется для типов point, box, ^a path, circle.	<pre>path '((0,0),(1,0),(1,1))' / point '(2.0,0)' → ((0,0),(0.5,0),(0.5,0.5)) path '((0,0),(1,0),(1,1))' / point(cosd(45), sind(45)) → ((0,0),(0.7071067811865476,-0.7071067811865476),(1.4142135623730951,0))</pre>
@-@	геометрический_тип → double precision Вычисляет общую длину. Имеется для типов lseg, path.	@-@ path '[(0,0),(1,0),(1,1)]' → 2
@@	геометрический_тип → point Вычисляет центральную точку. Имеется для типов box, lseg, polygon, circle.	@@ box '(2,2),(0,0)' → (1,1)
#	геометрический_тип → integer Возвращает количество точек. Имеется для типов path, polygon.	# path '((1,0),(0,1),(-1,0))' → 3
геометрический_тип # геометрический_тип	→ point Вычисляет точку пересечения, а если пересечения нет, возвращает NULL. Имеется для типов lseg, line.	lseg '[(0,0),(1,1)]' # lseg '[(1,0),(0,1)]' → (0.5,0.5)
box # box	→ box Вычисляет пересечение двух прямоугольников, а если пересечения нет, возвращает NULL.	box '(2,2),(-1,-1)' # box '(1,1),(-2,-2)' → (1,1),(-1,-1)
геометрический_тип ## геометрический_тип	→ point Вычисляет ближайшую к первому объекту точку, принадлежащую второму объекту. Имеется для следующих пар типов: (point, box), (point, lseg), (point, line), (lseg, box), (lseg, lseg), (line, lseg).	point '(0,0)' ## lseg '[(2,0),(0,2)]' → (1,1)
геометрический_тип <-> геометрический_тип	→ double precision Вычисляет расстояние между объектами. Имеется для всех семи геометрических типов, для всех сочетаний типа point с другим геометрическим типом, а также для следующих пар типов: (box, lseg), (lseg, line), (polygon, circle) (и пар с обратным порядком).	circle '<(0,0),1>' <-> circle '<(5,0),1>' → 3
геометрический_тип @> геометрический_тип	→ boolean Первый объект содержит второй? Имеется для следующих пар типов: (box, point), (box, box), (path, point), (polygon, point), (polygon, polygon), (circle, point), (circle, circle).	circle '<(0,0),2>' @> point '(1,1)' → t
геометрический_тип <@ геометрический_тип	→ boolean Первый объект содержится во втором? Имеется для следующих пар типов: (point, box), (point, lseg), (point, line), (point, path), (point, polygon), (point, circle), (box, box), (lseg, box), (lseg, line), (polygon, polygon), (circle, circle).	point '(1,1)' <@ circle '<(0,0),2>' → t
геометрический_тип && геометрический_тип	→ boolean	

Оператор	Описание	Пример(ы)
	Объекты пересекаются? (Для выполнения этого условия достаточно одной общей точки.) Имеется для типов box, polygon, circle.	box '(1,1), (0,0)' && box '(2,2), (0,0)' → t
<i>геометрический_тип</i> << <i>геометрический_тип</i> → boolean	Первый объект строго слева от второго? Имеется для типов point, box, polygon, circle.	circle '<(0,0),1>' << circle '<(5,0),1>' → t
<i>геометрический_тип</i> >> <i>геометрический_тип</i> → boolean	Первый объект строго справа от второго? Имеется для типов point, box, polygon, circle.	circle '<(5,0),1>' >> circle '<(0,0),1>' → t
<i>геометрический_тип</i> &< <i>геометрический_тип</i> → boolean	Первый объект не простирается правее второго? Имеется для типов box, polygon, circle.	box '(1,1), (0,0)' &< box '(2,2), (0,0)' → t
<i>геометрический_тип</i> &> <i>геометрический_тип</i> → boolean	Первый объект не простирается левее второго? Имеется для типов box, polygon, circle.	box '(3,3), (0,0)' &> box '(2,2), (0,0)' → t
<i>геометрический_тип</i> << <i>геометрический_тип</i> → boolean	Первый объект строго ниже второго? Имеется для типов point, box, polygon, circle.	box '(3,3), (0,0)' << box '(5,5), (3,4)' → t
<i>геометрический_тип</i> >> <i>геометрический_тип</i> → boolean	Первый объект строго выше второго? Имеется для типов point, box, polygon, circle.	box '(5,5), (3,4)' >> box '(3,3), (0,0)' → t
<i>геометрический_тип</i> &< <i>геометрический_тип</i> → boolean	Первый объект не простирается выше второго? Имеется для типов box, polygon, circle.	box '(1,1), (0,0)' &< box '(2,2), (0,0)' → t
<i>геометрический_тип</i> &> <i>геометрический_тип</i> → boolean	Первый объект не простирается ниже второго? Имеется для типов box, polygon, circle.	box '(3,3), (0,0)' &> box '(2,2), (0,0)' → t
box <^ box → boolean	Первый объект ниже (или касается снизу) второго?	box '((1,1), (0,0))' <^ box '((2,2), (1,1))' → t
box >^ box → boolean	Первый объект выше (или касается сверху) второго?	box '((2,2), (1,1))' >^ box '((1,1), (0,0))' → t
<i>геометрический_тип</i> ?# <i>геометрический_тип</i> → boolean	Объекты пересекаются? Имеется для следующих пар типов: (box, box), (lseg, box), (lseg, lseg), (lseg, line), (line, box), (line, line), (path, path).	lseg '[(−1,0), (1,0)]' ?# box '(2,2), (−2,−2)' → t
?- line → boolean ?- lseg → boolean	Линия является горизонтальной?	?- lseg '[(−1,0), (1,0)]' → t

Оператор Описание Пример(ы)
<code>point ?- point → boolean</code> Точки выровнены по горизонтали (имеют одинаковую координату y)? <code>point '(1,0)' ?- point '(0,0)' → t</code>
<code>? line → boolean</code> <code>? lseg → boolean</code> Линия является вертикальной? <code>? lseg ' [(-1,0), (1,0)] ' → f</code>
<code>point ? point → boolean</code> Точки выровнены по вертикали (имеют одинаковую координату x)? <code>point '(0,1)' ? point '(0,0)' → t</code>
<code>line ?- line → boolean</code> <code>lseg ?- lseg → boolean</code> Линии перпендикулярны? <code>lseg ' [(0,0), (0,1)] ' ?- lseg ' [(0,0), (1,0)] ' → t</code>
<code>line ? line → boolean</code> <code>lseg ? lseg → boolean</code> Линии параллельны? <code>lseg ' [(-1,0), (1,0)] ' ? lseg ' [(-1,2), (1,2)] ' → t</code>
<code>геометрический_тип ~= геометрический_тип → boolean</code> Объекты совпадают? Имеется для типов <code>point</code> , <code>box</code> , <code>polygon</code> , <code>circle</code> . <code>polygon ' ((0,0), (1,1)) ' ~= polygon ' ((1,1), (0,0)) ' → t</code>

^aПри «повороте» прямоугольника эти операторы только перемещают его угловые точки: стороны прямоугольника считаются всегда параллельными осям. Таким образом, эта операция, в отличие от настоящего поворота, изменяет размер прямоугольника.

Внимание

Заметьте, что оператор «идентичности», `~=`, представляет собой обычную проверку равенства значений `point`, `box`, `polygon` и `circle`. Для некоторых геометрических типов определён также оператор `=`, но `=` проверяет только равенство *площадей*. Другие скалярные операторы сравнения (`<=` и т. д.) для тех типов, для которых они реализованы, тоже сравнивают площади.

Примечание

До Postgres Pro 14 применявшиеся к точкам операторы `point <<| point` (строго ниже) и `point |>> point` (строго выше) назывались `<^` и `>^` соответственно. Эти имена по-прежнему доступны, но считаются устаревшими и в конце концов будут удалены.

Таблица 9.37. Геометрические функции

Функция Описание Пример(ы)
<code>area (геометрический_тип) → double precision</code>

Функция	Описание	Пример(ы)
	Вычисляет площадь. Имеется для типов <code>box</code> , <code>path</code> , <code>circle</code> . Путь, переданный в аргументе типа <code>path</code> , должен быть замкнутым; в противном случае возвращается <code>NULL</code> . Если же путь является самопересекающимся, результат может не иметь смысла.	<code>area(box '(2,2),(0,0)')</code> → 4
	Вычисляет центральную точку. Имеется для типов <code>box</code> , <code>circle</code> .	<code>center(geometric_type) → point</code> <code>center(box '(1,2),(0,0)')</code> → (0.5,1)
	Вычисляет диагональ прямоугольника в виде отрезка (аналогично <code>lseg(box)</code>).	<code>diagonal(box '(1,2),(0,0)')</code> → [(1,2),(0,0)]
	Вычисляет диаметр круга.	<code>diameter(circle '<(0,0),2>')</code> → 4
	Вычисляет вертикальный размер прямоугольника.	<code>height(box '(1,2),(0,0)')</code> → 2
	Путь является замкнутым?	<code>isclosed(path '((0,0),(1,1),(2,0))')</code> → t
	Путь является открытым?	<code>isopen(path '[(0,0),(1,1),(2,0)]')</code> → t
	Вычисляет общую длину. Имеется для типов <code>lseg</code> , <code>path</code> .	<code>length(геометрический_тип) → double precision</code> <code>length(path '((-1,0),(1,0))')</code> → 4
	Возвращает количество точек. Имеется для типов <code>path</code> , <code>polygon</code> .	<code>npoints(геометрический_тип) → integer</code> <code>npoints(path '[(0,0),(1,1),(2,0)]')</code> → 3
	Преобразует путь в замкнутый.	<code>pclose(path '[(0,0),(1,1),(2,0)]')</code> → ((0,0),(1,1),(2,0))
	Преобразует путь в открытый.	<code>popen(path '((0,0),(1,1),(2,0))')</code> → [(0,0),(1,1),(2,0)]
	Вычисляет радиус окружности.	<code>radius(circle '<(0,0),2>')</code> → 2
	Вычисляет наклон линии, проведённой через две точки.	<code>slope(point '(0,0)', point '(2,1)')</code> → 0.5
	Вычисляет горизонтальный размер прямоугольника.	<code>width(box) → double precision</code>

Функция	Описание	Пример(ы)
<code>width</code>	<code>box</code>	<code>'(1,2), (0,0)'</code> → 1

Таблица 9.38. Функции преобразования геометрических типов

Функция	Описание	Пример(ы)
<code>box (circle)</code>	→ <code>box</code> Вычисляет прямоугольник, вписанный в окружность.	<code>box(circle '(0,0),2>')</code> → (1.414213562373095, 1.414213562373095), (-1.414213562373095, -1.414213562373095)
<code>box (point)</code>	→ <code>box</code> Преобразует точку в пустой прямоугольник.	<code>box(point '(1,0)')</code> → (1,0), (1,0)
<code>box (point, point)</code>	→ <code>box</code> Преобразует две угловые точки в прямоугольник.	<code>box(point '(0,1)', point '(1,0)')</code> → (1,1), (0,0)
<code>box (polygon)</code>	→ <code>box</code> Вычисляет окружающий прямоугольник для многоугольника.	<code>box(polygon '((0,0), (1,1), (2,0))')</code> → (2,1), (0,0)
<code>bound_box (box, box)</code>	→ <code>box</code> Вычисляет окружающий прямоугольник для двух прямоугольников.	<code>bound_box(box '(1,1), (0,0)', box '(4,4), (3,3)')</code> → (4,4), (0,0)
<code>circle (box)</code>	→ <code>circle</code> Вычисляет минимальную окружность, описанную около прямоугольника.	<code>circle(box '(1,1), (0,0)')</code> → <(0.5,0.5), 0.7071067811865476>
<code>circle (point, double precision)</code>	→ <code>circle</code> Формирует окружность по точке, задающей центр, и радиусу.	<code>circle(point '(0,0)', 2.0)</code> → <(0,0), 2>
<code>circle (polygon)</code>	→ <code>circle</code> Преобразует многоугольник в окружность. Центром окружности будет средняя точка для всех точек многоугольника, а радиусом среднее расстояние от этого центра до точек многоугольника.	<code>circle(polygon '((0,0), (1,3), (2,0))')</code> → <(1,1), 1.6094757082487299>
<code>line (point, point)</code>	→ <code>line</code> Преобразует две точки в проходящую через них линию.	<code>line(point '(-1,0)', point '(1,0)')</code> → {0,-1,0}
<code>lseg (box)</code>	→ <code>lseg</code> Вычисляет диагональ прямоугольника в виде отрезка.	<code>lseg(box '(1,0), (-1,0)')</code> → [(1,0), (-1,0)]
<code>lseg (point, point)</code>	→ <code>lseg</code> Формирует отрезок по двум точкам.	<code>lseg(point '(-1,0)', point '(1,0)')</code> → [(-1,0), (1,0)]
<code>path (polygon)</code>	→ <code>path</code>	

Функция	Описание	Пример(ы)
	Преобразует многоугольник в замкнутый путь с тем же набором точек.	<code>path(polygon '((0,0),(1,1),(2,0))')</code> → <code>((0,0),(1,1),(2,0))</code>
<code>point (double precision, double precision)</code>	Формирует точку по заданным координатам.	<code>point(23.4, -44.5)</code> → <code>(23.4,-44.5)</code>
<code>point (box)</code>	Вычисляет центр прямоугольника.	<code>point(box '(1,0),(-1,0)')</code> → <code>(0,0)</code>
<code>point (circle)</code>	Вычисляет центр окружности.	<code>point(circle '<(0,0),2>')</code> → <code>(0,0)</code>
<code>point (lseg)</code>	Вычисляет середину отрезка.	<code>point(lseg '[(−1,0),(1,0)]')</code> → <code>(0,0)</code>
<code>point (polygon)</code>	Вычисляет центр многоугольника (среднее арифметическое по координатам его точек).	<code>point(polygon '((0,0),(1,1),(2,0))')</code> → <code>(1,0.3333333333333333)</code>
<code>polygon (box)</code>	Преобразует прямоугольник в многоугольник с 4 вершинами.	<code>polygon(box '(1,1),(0,0)')</code> → <code>((0,0),(0,1),(1,1),(1,0))</code>
<code>polygon (circle)</code>	Преобразует круг в многоугольник с 12 вершинами.	<code>polygon(circle '<(0,0),2>')</code> → <code>((−2,0),(−1.7320508075688774,0.9999999999999999),(−1.0000000000000002,1.7320508075688772),(−1.2246063538223773e−16,2),(0.9999999999999996,1.7320508075688774),(1.732050807568877,1.0000000000000007),(2,2.4492127076447545e−16),(1.7320508075688776,−0.9999999999999994),(1.0000000000000009,−1.7320508075688767),(3.673819061467132e−16,−2),(−0.9999999999999987,−1.732050807568878),(−1.7320508075688767,−1.0000000000000009))</code>
<code>polygon (integer, circle)</code>	Преобразует окружность в многоугольник с <i>n</i> вершинами.	<code>polygon(4, circle '<(3,0),1>')</code> → <code>((2,0),(3,1),(4,1.2246063538223773e−16),(3,−1))</code>
<code>polygon (path)</code>	Преобразует замкнутый путь в многоугольник с тем же набором точек.	<code>polygon(path '((0,0),(1,1),(2,0))')</code> → <code>((0,0),(1,1),(2,0))</code>

К двум компонентам типа `point` (точка) можно обратиться, как к элементам массива с индексами 0 и 1. Например, если `t.p` — столбец типа `point`, `SELECT p[0] FROM t` вернёт координату X, а `UPDATE t SET p[1] = ...` изменит координату Y. Таким же образом, значение типа `box` или `lseg` можно воспринимать как массив двух значений типа `point`.

9.12. Функции и операторы для работы с сетевыми адресами

Типы `cidr` и `inet`, предназначенные для сетевых IP-адресов, поддерживают обычные операторы сравнения, показанные в [Таблице 9.1](#), а также специализированные операторы и функции, показанные в [Таблице 9.39](#) и [Таблице 9.40](#).

Любое значение `cidr` можно привести к типу `inet` неявно, поэтому все функции, показанные выше с типом `inet`, также будут работать со значениями `cidr`. (То, что некоторые из функций описаны для типов `inet` и `cidr` в отдельности, объясняется тем, что их поведение с разными типами различается.) Кроме того, значение `inet` тоже можно привести к типу `cidr`. При этом все биты справа от сетевой маски просто обнуляются, чтобы значение стало допустимым для типа `cidr`.

Таблица 9.39. Операторы для работы с IP-адресами

Оператор	Описание	Пример(ы)
<code>inet << inet → boolean</code>	Первая подсеть содержится во второй и не равна ей? Этот оператор и следующие четыре проверяют вхождение одной сети в другую или их равенство, при этом рассматривая в адресах только компонент сети (биты справа от сетевой маски игнорируются).	<pre>inet '192.168.1.5' << inet '192.168.1/24' → t inet '192.168.0.5' << inet '192.168.1/24' → f inet '192.168.1/24' << inet '192.168.1/24' → f</pre>
<code>inet <= inet → boolean</code>	Первая подсеть содержится во второй или равна ей?	<pre>inet '192.168.1/24' <= inet '192.168.1/24' → t</pre>
<code>inet >> inet → boolean</code>	Первая подсеть содержит вторую подсеть и не равна ей?	<pre>inet '192.168.1/24' >> inet '192.168.1.5' → t</pre>
<code>inet >= inet → boolean</code>	Первая подсеть содержит вторую подсеть или равна ей?	<pre>inet '192.168.1/24' >= inet '192.168.1/24' → t</pre>
<code>inet && inet → boolean</code>	Одна из двух подсетей содержит другую или равна ей?	<pre>inet '192.168.1/24' && inet '192.168.1.80/28' → t inet '192.168.1/24' && inet '192.168.2.0/28' → f</pre>
<code>~ inet → inet</code>	Вычисляет результат побитового НЕ.	<pre>~ inet '192.168.1.6' → 63.87.254.249</pre>
<code>inet & inet → inet</code>	Вычисляет результат побитового И.	<pre>inet '192.168.1.6' & inet '0.0.0.255' → 0.0.0.6</pre>
<code>inet inet → inet</code>	Вычисляет результат побитового ИЛИ.	<pre>inet '192.168.1.6' inet '0.0.0.255' → 192.168.1.255</pre>
<code>inet + bigint → inet</code>	Добавляет смещение к адресу.	

Оператор Описание Пример(ы)
<code>inet '192.168.1.6' + 25 → 192.168.1.31</code>
<code>bigint + inet → inet</code> Добавляет смещение к адресу. <code>200 + inet '::ffff:fff0:1' → ::ffff:255.240.0.201</code>
<code>inet - bigint → inet</code> Вычитает смещение из адреса. <code>inet '192.168.1.43' - 36 → 192.168.1.7</code>
<code>inet - inet → bigint</code> Вычисляет разность двух адресов. <code>inet '192.168.1.43' - inet '192.168.1.19' → 24</code> <code>inet ':::1' - inet ':::ffff:1' → -4294901760</code>

Таблица 9.40. Функции для работы с IP-адресами

Функция Описание Пример(ы)
<code>abbrev (inet) → text</code> Выводит адрес в сокращённом текстовом виде. (Результат не отличается от того, что даёт функция вывода <code>inet</code> ; «сокращённым» он является только в сравнении с явным приведением к типу <code>text</code> , которое по историческим причинам не убирает компонент маски сети). <code>abbrev(inet '10.1.0.0/32') → 10.1.0.0</code>
<code>abbrev (cidr) → text</code> Выводит адрес в сокращённом текстовом виде. (Сокращение заключается в отбрасывании полностью нулевых октетов в конце сетевой маски; другие примеры приведены в Таблице 8.22.) <code>abbrev(cidr '10.1.0.0/16') → 10.1/16</code>
<code>broadcast (inet) → inet</code> Вычисляет широковещательный адрес для сети. <code>broadcast(inet '192.168.1.5/24') → 192.168.1.255/24</code>
<code>family (inet) → integer</code> Выдаёт семейство адреса: 4 для адресов IPv4, 6 для адресов IPv6. <code>family(inet ':::1') → 6</code>
<code>host (inet) → text</code> Выдаёт IP-адрес в текстовом виде, опуская маску сети. <code>host(inet '192.168.1.0/24') → 192.168.1.0</code>
<code>hostmask (inet) → inet</code> Вычисляет маску узла для сети в заданном адресе. <code>hostmask(inet '192.168.23.20/30') → 0.0.0.3</code>
<code>inet_merge (inet, inet) → cidr</code> Вычисляет наименьшую сеть, содержащую обе заданные сети. <code>inet_merge(inet '192.168.1.5/24', inet '192.168.2.5/24') → 192.168.0.0/22</code>
<code>inet_same_family (inet, inet) → boolean</code> Проверяет, относятся ли адреса к одному семейству IP?

Функция	Описание	Пример(ы)
		<code>inet_same_family(inet '192.168.1.5/24', inet ':::1')</code> → <code>f</code>
<code>masklen (inet)</code>	→ integer Выдаёт длину маски сети в битах.	<code>masklen(inet '192.168.1.5/24')</code> → 24
<code>netmask (inet)</code>	→ inet Вычисляет маску сети для заданного адреса.	<code>netmask(inet '192.168.1.5/24')</code> → 255.255.255.0
<code>network (inet)</code>	→ cidr Выдаёт компонент сети для заданного адреса, обнуляя все биты справа от маски сети. (Это равнозначно приведению значения к типу <code>cidr</code> .)	<code>network(inet '192.168.1.5/24')</code> → 192.168.1.0/24
<code>set_masklen (inet, integer)</code>	→ inet Задаёт размер маски сети для значения <code>inet</code> . Компонент адреса при этом не меняется.	<code>set_masklen(inet '192.168.1.5/24', 16)</code> → 192.168.1.5/16
<code>set_masklen (cidr, integer)</code>	→ cidr Задаёт размер маски сети для значения <code>cidr</code> . Биты адреса справа от новой маски при этом обнуляются.	<code>set_masklen(cidr '192.168.1.0/24', 16)</code> → 192.168.0.0/16
<code>text (inet)</code>	→ text Выдаёт несокращённый IP-адрес и размер маски в виде текста. (Такой же результат получается при явном приведении к типу <code>text</code> .)	<code>text(inet '192.168.1.5')</code> → 192.168.1.5/32

Подсказка

Функции `abbrev`, `host` и `text` предназначены в основном для вывода IP-адресов в альтернативных текстовых форматах.

Типы MAC-адресов `macaddr` и `macaddr8` поддерживают обычные операторы сравнения, показанные в [Таблице 9.1](#), а также специализированные функции, показанные в [Таблице 9.41](#). Кроме того, они поддерживают битовые логические операторы: `~`, `&` и `|` (НЕ, И, ИЛИ), показанные выше для IP-адресов.

Таблица 9.41. Функции для работы с MAC-адресами

Функция	Описание	Пример(ы)
<code>trunc (macaddr)</code>	→ macaddr Обнуляет три последних байта адреса. Оставшийся префикс можно сопоставить с конкретным производителем сетевой карты (в Postgres Pro необходимой для этого информации нет).	<code>trunc(macaddr '12:34:56:78:90:ab')</code> → 12:34:56:00:00:00
<code>trunc (macaddr8)</code>	→ macaddr8 Обнуляет 5 последних байт адреса. Оставшийся префикс можно сопоставить с конкретным производителем сетевой карты (в Postgres Pro необходимой для этого информации нет).	

Функция
Описание
Пример(ы)
<code>trunc(macaddr8 '12:34:56:78:90:ab:cd:ef') → 12:34:56:00:00:00:00:00</code>
<code>macaddr8_set7bit (macaddr8) → macaddr8</code> Устанавливает в 7 бите адреса единицу, получая тем самым так называемый модифицированный адрес EUI-64 для включения в адрес IPv6. <code>macaddr8_set7bit(macaddr8 '00:34:56:ab:cd:ef') → 02:34:56:ff:fe:ab:cd:ef</code>

9.13. Функции и операторы текстового поиска

В [Таблице 9.42](#), [Таблице 9.43](#) и [Таблице 9.44](#) собраны все существующие функции и операторы, предназначенные для полнотекстового поиска. Во всех деталях возможности полнотекстового поиска в Postgres Pro описаны в [Главе 12](#).

Таблица 9.42. Операторы текстового поиска

Оператор
Описание
Пример(ы)
<code>tsvector @@ tsquery → boolean</code> <code>tsquery @@ tsvector → boolean</code> Аргумент <code>tsvector</code> соответствует аргументу <code>tsquery</code> ? (Аргументы могут передаваться в любом порядке.) <code>to_tsvector('fat cats ate rats') @@ to_tsquery('cat & rat') → t</code>
<code>text @@ tsquery → boolean</code> Переданная текстовая строка, неявно преобразованная функцией <code>to_tsvector()</code> , соответствует <code>tsquery</code> ? <code>'fat cats ate rats' @@ to_tsquery('cat & rat') → t</code>
<code>tsvector @@@ tsquery → boolean</code> <code>tsquery @@@ tsvector → boolean</code> Это устаревший синоним для <code>@@</code> . <code>to_tsvector('fat cats ate rats') @@@ to_tsquery('cat & rat') → t</code>
<code>tsvector tsvector → tsvector</code> Соединяет два значения <code>tsvector</code> . Если в обоих значениях содержатся позиции лексем, позиции во втором при совмещении корректируются. <code>'a:1 b:2'::tsvector 'c:1 d:2 b:3'::tsvector → 'a':1 'b':2,5 'c':3 'd':4</code>
<code>tsquery && tsquery → tsquery</code> Вычисляет конъюнкцию двух значений <code>tsquery</code> , получая тем самым запрос, которому будут соответствовать документы, соответствующие обоим входным запросам. <code>'fat rat'::tsquery && 'cat'::tsquery → ('fat' 'rat') & 'cat'</code>
<code>tsquery tsquery → tsquery</code> Вычисляет дизъюнкцию двух значений <code>tsquery</code> , получая тем самым запрос, которому будут соответствовать документы, соответствующие любому из входных запросов. <code>'fat rat'::tsquery 'cat'::tsquery → 'fat' 'rat' 'cat'</code>
<code>!! tsquery → tsquery</code> Вычисляет отрицание <code>tsquery</code> , получая тем самым запрос, которому будут соответствовать документы, не соответствующие входному запросу. <code>!! 'cat'::tsquery → !'cat'</code>
<code>tsquery <-> tsquery → tsquery</code>

Оператор	Описание	Пример(ы)
	Конструирует фразовый запрос, которому будут соответствовать документы, содержащие подряд идущие лексемы, удовлетворяющие двум входным запросам.	<code>to_tsquery('fat') <-> to_tsquery('rat') → 'fat' <-> 'rat'</code>
	<code>tsquery @> tsquery → boolean</code> Первый запрос типа <code>tsquery</code> содержит второй? (При этом учитывается только факт нахождения всех лексем из одного запроса в другом; операторы их сочетания игнорируются.)	<code>'cat'::tsquery @> 'cat & rat'::tsquery → f</code>
	<code>tsquery <@ tsquery → boolean</code> Первый запрос типа <code>tsquery</code> содержится во втором? (При этом учитывается только факт нахождения всех лексем из одного запроса в другом; операторы их сочетания игнорируются.)	<code>'cat'::tsquery <@ 'cat & rat'::tsquery → t</code> <code>'cat'::tsquery <@ '!cat & rat'::tsquery → t</code>

Помимо этих специализированных операторов, для типов `tsvector` и `tsquery` имеются обычные операторы сравнения, показанные в [Таблице 9.1](#). Они не очень полезны для поиска, но позволяют, в частности, создавать индексы по столбцам этих типов.

Таблица 9.43. Функции текстового поиска

Функция	Описание	Пример(ы)
<code>array_to_tsvector (text[]) → tsvector</code>	Преобразует массив текстовых строк в <code>tsvector</code> . Переданные строки используются в виде лексем как есть, без дополнительной обработки. Элементы массива не должны быть пустыми строками или <code>NULL</code> .	<code>array_to_tsvector('{fat,cat,rat'}::text[]) → 'cat' 'fat' 'rat'</code>
<code>get_current_ts_config () → regconfig</code>	Возвращает OID текущей конфигурации текстового поиска по умолчанию (задаваемой параметром default_text_search_config).	<code>get_current_ts_config() → english</code>
<code>length (tsvector) → integer</code>	Возвращает число лексем в значении <code>tsvector</code> .	<code>length('fat:2,4 cat:3 rat:5A'::tsvector) → 3</code>
<code>numnode (tsquery) → integer</code>	Возвращает число лексем и операторов в запросе <code>tsquery</code> .	<code>numnode('(fat & rat) cat'::tsquery) → 5</code>
<code>plainto_tsquery ([config regconfig,] query text) → tsquery</code>	Преобразует текст в <code>tsquery</code> , выполняя нормализацию слов согласно конфигурации по умолчанию или указанной явно. Знаки пунктуации во входной строке при этом игнорируются (в данном случае она не определяет операторы запроса). Результирующему запросу будут соответствовать документы, содержащие все слова этого текста, кроме стоп-слов.	<code>plainto_tsquery('english', 'The Fat Rats') → 'fat' & 'rat'</code>
<code>phraseto_tsquery ([config regconfig,] query text) → tsquery</code>		

Функция	Описание	Пример(ы)
	Преобразует текст в <code>tsquery</code> , выполняя нормализацию слов согласно конфигурации по умолчанию или указанной явно. Знаки пунктуации во входной строке при этом игнорируются (в данном случае она не определяет операторы запроса). Результирующему запросу будут соответствовать фразы, содержащие последовательность слов этого текста, кроме стоп-слов.	<pre>phraseto_tsquery('english', 'The Fat Rats') → 'fat' <-> 'rat'</pre> <pre>phraseto_tsquery('english', 'The Cat and Rats') → 'cat' <2> 'rat'</pre>
<code>websearch_to_tsquery ([config regconfig,] query text) → tsquery</code>	Преобразует текст в <code>tsquery</code> , выполняя нормализацию слов согласно конфигурации по умолчанию или указанной явно. Последовательности слов в кавычках преобразуются в проверки фраз. Слово «or» воспринимается как оператор OR (ИЛИ), а символ минуса преобразуется в оператор NOT (НЕ); другие знаки пунктуации игнорируются. Это примерно соответствует поведению ряда распространённых средств поиска в вебе.	<pre>websearch_to_tsquery('english', '"fat rat" or cat dog') → 'fat' <-> 'rat' 'cat' & 'dog'</pre>
<code>querytree (tsquery) → text</code>	Формирует представление индексируемой части <code>tsquery</code> . Если возвращается пустой результат или просто <code>T</code> , это означает, что запрос не индексируемый.	<pre>querytree('foo & ! bar'::tsquery) → 'foo'</pre>
<code>setweight (vector tsvector, weight "char") → tsvector</code>	Назначает вес, указанный в аргументе <code>weight</code> , каждому элементу аргумента <code>vector</code> .	<pre>setweight('fat:2,4 cat:3 rat:5B'::tsvector, 'A') → 'cat':3A 'fat':2A,4A 'rat':5A</pre>
<code>setweight (vector tsvector, weight "char", lexemes text[]) → tsvector</code>	Назначает вес, указанный в аргументе <code>weight</code> , элементам аргумента <code>vector</code> , перечисленным в аргументе <code>lexemes</code> . Строки в <code>lexemes</code> принимаются в виде лексем как есть, без дальнейшей обработки. Строки, не соответствующие ни одной лексеме в <code>vector</code> , игнорируются.	<pre>setweight('fat:2,4 cat:3 rat:5,6B'::tsvector, 'A', '{cat,rat}') → 'cat':3A 'fat':2,4 'rat':5A,6A</pre>
<code>strip (tsvector) → tsvector</code>	Убирает позиции и веса из значения <code>tsvector</code> .	<pre>strip('fat:2,4 cat:3 rat:5A'::tsvector) → 'cat' 'fat' 'rat'</pre>
<code>to_tsquery ([config regconfig,] query text) → tsquery</code>	Преобразует текст в <code>tsquery</code> , выполняя нормализацию слов согласно конфигурации по умолчанию или указанной явно. Слова должны разделяться операторами <code>tsquery</code> .	<pre>to_tsquery('english', 'The & Fat & Rats') → 'fat' & 'rat'</pre>
<code>to_tsvector ([config regconfig,] document text) → tsvector</code>	Преобразует текст в <code>tsvector</code> , выполняя нормализацию слов согласно конфигурации по умолчанию или указанной явно. Результат будет включать информацию о позициях слов.	<pre>to_tsvector('english', 'The Fat Rats') → 'fat':2 'rat':3</pre>
<code>to_tsvector ([config regconfig,] document json) → tsvector</code> <code>to_tsvector ([config regconfig,] document jsonb) → tsvector</code>	Преобразует каждое строковое значение в документе JSON в значение <code>tsvector</code> , выполняя нормализацию слов согласно конфигурации по умолчанию или указанной явно. В	

Функция	Описание	Пример(ы)
	результате выдаются полученные значения, соединённые вместе в порядке следования в документе. При вычислении выдаваемых позиций слов считается, что между каждой парой строковых значений находится одно стоп-слово. (Учтите, что в случае с <code>jsonb</code> «порядок следования в документе» полей объекта JSON зависит от реализации; это наглядно показано в примере.)	<pre>to_tsvector('english', '{"aa": "The Fat Rats", "b": "dog"}'::json) → 'dog':5 'fat':2 'rat':3 to_tsvector('english', '{"aa": "The Fat Rats", "b": "dog"}'::jsonb) → 'dog':1 'fat':4 'rat':5</pre>
<pre>json_to_tsvector ([config regconfig,] document json, filter jsonb) → tsvector jsonb_to_tsvector ([config regconfig,] document jsonb, filter jsonb) → tsvector</pre>	Выбирает из JSON-документа все элементы, соответствующие фильтру <i>filter</i> , и преобразует каждый в значение <i>tsvector</i> , нормализуя их согласно конфигурации по умолчанию или указанной явно. В результате выдаются полученные значения, соединённые вместе в порядке следования в документе. При вычислении выдаваемых позиций слов считается, что между каждой парой строковых значений находится одно стоп-слово. (Учтите, что в случае с <code>jsonb</code> «порядок следования в документе» полей объекта JSON зависит от реализации.) В параметре <i>filter</i> должен передаваться массив <code>jsonb</code> , содержащий ноль или более следующих ключевых слов: "string" (включить все строковые значения), "numeric" (все числовые значения), "boolean" (все логические значения), "key" (все ключи) или "all" (включить всё вышеперечисленное). В качестве особого значения <i>filter</i> принимается простое JSON-значение, содержащее одно из этих ключевых слов.	<pre>json_to_tsvector('english', '{"a": "The Fat Rats", "b": 123}'::json, ['string', 'numeric']) → '123':5 'fat':2 'rat':3 json_to_tsvector('english', '{"cat": "The Fat Rats", "dog": 123}'::json, 'all') → '123':9 'cat':1 'dog':7 'fat':4 'rat':5</pre>
<pre>ts_delete (vector tsvector, lexeme text) → tsvector</pre>	Удаляет все вхождения лексемы, задаваемой аргументом <i>lexeme</i> , из значения <i>vector</i> . Строка <i>lexeme</i> принимается в виде лексемы как есть, без дальнейшей обработки.	<pre>ts_delete('fat:2,4 cat:3 rat:5A'::tsvector, 'fat') → 'cat':3 'rat':5A</pre>
<pre>ts_delete (vector tsvector, lexemes text[]) → tsvector</pre>	Удаляет все вхождения лексем, переданных в параметре <i>lexemes</i> , из параметра <i>vector</i> . Строки в <i>lexemes</i> принимаются в виде лексем как есть, без дальнейшей обработки. Строки, которые не соответствуют никаким лексемам в <i>vector</i> , игнорируются.	<pre>ts_delete('fat:2,4 cat:3 rat:5A'::tsvector, ARRAY['fat', 'rat']) → 'cat':3</pre>
<pre>ts_filter (vector tsvector, weights "char"[]) → tsvector</pre>	Выбирает из значения <i>vector</i> только элементы с весами, перечисленными в массиве <i>weights</i> .	<pre>ts_filter('fat:2,4 cat:3b,7c rat:5A'::tsvector, '{a,b}') → 'cat':3B 'rat':5A</pre>
<pre>ts_headline ([config regconfig,] document text, query tsquery [, options text]) → text</pre>	Выводит в виде выдержек соответствующие поисковому запросу <i>query</i> фрагменты содержимого <i>document</i> , которое должно быть просто текстом, а не значением <i>tsvector</i> . Слова в документе перед поиском нормализуются согласно конфигурации по умолчанию или указанной явно. Использование этой функции рассматривается в Подразделе 12.3.4 ; также там описываются возможные значения <i>options</i> .	<pre>ts_headline('The fat cat ate the rat.', 'cat') → The fat cat ate the rat.</pre>

Функция	Описание	Пример(ы)
ts_headline	([config regconfig,] document json, query tsquery [, options text]) → text	
ts_headline	([config regconfig,] document jsonb, query tsquery [, options text]) → text	Выводит в виде выдержек соответствующие поисковому запросу query вхождения, найденные в строковых значениях внутри JSON-документа document. За подробностями обратитесь к Подразделу 12.3.4.
ts_headline	('{"cat": "raining cats and dogs"}'::jsonb, 'cat') → {"cat": "raining cats and dogs"}	
ts_rank	([weights real[],] vector tsvector, query tsquery [, normalization integer]) → real	Вычисляет оценку, показывающую, в какой степени значение vector соответствует запросу query. За подробностями обратитесь к Подразделу 12.3.3.
ts_rank	(to_tsvector('raining cats and dogs'), 'cat') → 0.06079271	
ts_rank_cd	([weights real[],] vector tsvector, query tsquery [, normalization integer]) → real	Вычисляет по алгоритму расчёта плотности покрытия оценку, показывающую, в какой степени значение vector соответствует запросу query За подробностями обратитесь к Подразделу 12.3.3.
ts_rank_cd	(to_tsvector('raining cats and dogs'), 'cat') → 0.1	
ts_rewrite	(query tsquery, target tsquery, substitute tsquery) → tsquery	Заменяет в аргументе query вхождения target значением substitute. За подробностями обратитесь к Подразделу 12.4.2.1.
ts_rewrite	('a & b'::tsquery, 'a'::tsquery, 'foo bar'::tsquery) → 'b' & ('foo' 'bar')	
ts_rewrite	(query tsquery, select text) → tsquery	Заменяет фрагменты запроса query, извлекая искомое вхождение и замену для него с помощью команды SELECT. За подробностями обратитесь к Подразделу 12.4.2.1.
ts_rewrite	(SELECT ts_rewrite('a & b'::tsquery, 'SELECT t,s FROM aliases') → 'b' & ('foo' 'bar')	
tsquery_phrase	(query1 tsquery, query2 tsquery) → tsquery	Конструирует фразовый запрос, который будет находить подряд идущие лексемы, удовлетворяющие запросам query1 и query2 (как делает оператор <->).
tsquery_phrase	(to_tsquery('fat'), to_tsquery('cat')) → 'fat' <-> 'cat'	
tsquery_phrase	(query1 tsquery, query2 tsquery, distance integer) → tsquery	Конструирует фразовый запрос, который будет находить вхождения, удовлетворяющие запросам query1 и query2, на расстоянии ровно в distance лексем друг от друга.
tsquery_phrase	(to_tsquery('fat'), to_tsquery('cat'), 10) → 'fat' <10> 'cat'	
tsvector_to_array	(tsvector) → text[]	Преобразует tsvector в массив лексем.
tsvector_to_array	('fat:2,4 cat:3 rat:5A'::tsvector) → {cat,fat,rat}	
unnest	(tsvector) → setof record (lexeme text, positions smallint[], weights text)	Разворачивает tsvector в набор строк, по одной лексеме в строке.
unnest	('cat:3 fat:2,4 rat:5A'::tsvector) →	
	lexeme positions weights	
	-----+-----+-----	
	cat {3} {D}	

Функция	Описание	Пример(ы)
fat	{2, 4}	{D, D}
rat	{5}	{A}

Примечание

Все функции текстового поиска, принимающие необязательный аргумент `regconfig`, будут использовать конфигурацию, указанную в параметре `default_text_search_config`, когда этот аргумент опущен.

Функции в [Таблице 9.44](#) перечислены отдельно, так как они не очень полезны в традиционных операциях поиска. Они предназначены в основном для разработки и отладки новых конфигураций текстового поиска.

Таблица 9.44. Функции отладки текстового поиска

Функция	Описание	Пример(ы)
<code>ts_debug</code>	([<i>config</i> <i>regconfig</i> ,] <i>document</i> <i>text</i>) → <i>setof</i> <i>record</i> (<i>alias</i> <i>text</i> , <i>description</i> <i>text</i> , <i>token</i> <i>text</i> , <i>dictionaries</i> <i>regdictionary</i> [], <i>dictionary</i> <i>regdictionary</i> , <i>lexemes</i> <i>text</i> []) Извлекает из текста <i>document</i> фрагменты, нормализуя их согласно конфигурации текстового поиска по умолчанию или указанной явно, и выдаёт информацию о том, как был обработан каждый фрагмент. За подробностями обратитесь к Подразделу 12.8.1 .	<code>ts_debug('english', 'The Brightest supernovaes') → (asciiword, "Word, all ASCII", The, {english_stem}, english_stem, {}) ...</code>
<code>ts_lexize</code>	(<i>dict</i> <i>regdictionary</i> , <i>token</i> <i>text</i>) → <i>text</i> [] Возвращает массив заменяющих лексем, если слово, заданное аргументом <i>token</i> , есть в словаре, пустой массив, если это стоп-слово, которое есть в словаре, либо NULL, если такого слова нет. За подробностями обратитесь к Подразделу 12.8.3 .	<code>ts_lexize('english_stem', 'stars') → {star}</code>
<code>ts_parse</code>	(<i>parser_name</i> <i>text</i> , <i>document</i> <i>text</i>) → <i>setof</i> <i>record</i> (<i>tokid</i> <i>integer</i> , <i>token</i> <i>text</i>) Извлекает фрагменты из текста <i>document</i> , применяя анализатор с указанным именем. За подробностями обратитесь к Подразделу 12.8.2 .	<code>ts_parse('default', 'foo - bar') → (1,foo) ...</code>
<code>ts_parse</code>	(<i>parser_oid</i> <i>oid</i> , <i>document</i> <i>text</i>) → <i>setof</i> <i>record</i> (<i>tokid</i> <i>integer</i> , <i>token</i> <i>text</i>) Извлекает фрагменты из текста <i>document</i> , применяя анализатор с указанным OID. За подробностями обратитесь к Подразделу 12.8.2 .	<code>ts_parse(3722, 'foo - bar') → (1,foo) ...</code>
<code>ts_token_type</code>	(<i>parser_name</i> <i>text</i>) → <i>setof</i> <i>record</i> (<i>tokid</i> <i>integer</i> , <i>alias</i> <i>text</i> , <i>description</i> <i>text</i>) Возвращает таблицу, описывающую все типы фрагментов, которые может распознать анализатор с указанным именем. За подробностями обратитесь к Подразделу 12.8.2 .	<code>ts_token_type('default') → (1,asciiword, "Word, all ASCII") ...</code>
<code>ts_token_type</code>	(<i>parser_oid</i> <i>oid</i>) → <i>setof</i> <i>record</i> (<i>tokid</i> <i>integer</i> , <i>alias</i> <i>text</i> , <i>description</i> <i>text</i>) Возвращает таблицу, описывающую все типы фрагментов, которые может распознать анализатор с указанным OID. За подробностями обратитесь к Подразделу 12.8.2 .	

Функция	Описание	Пример(ы)
		<code>ts_token_type(3722) → (1,asciiword,"Word, all ASCII") ...</code>
	Выполняет запрос <i>sqlquery</i> , который должен возвращать единственный столбец <i>tsvector</i> , и выдаёт статистику по каждой отдельной лексеме, содержащейся в данных. За подробностями обратитесь к Подразделу 12.4.4 .	<code>ts_stat (sqlquery text [, weights text]) → setof record (word text, ndoc integer, nentry integer)</code> <code>ts_stat('SELECT vector FROM apod') → (foo,10,15) ...</code>

9.14. Функции генерирования UUID

В Postgres Pro имеется функция для генерирования UUID:

```
gen_random_uuid () → uuid
```

Она возвращает случайный UUID версии 4. Это наиболее популярный тип UUID, подходящий для большинства приложений.

Модуль [uuid-oss](#) предоставляет дополнительные функции, реализующие другие стандартные алгоритмы генерирования UUID.

В Postgres Pro также реализованы показанные в [Таблице 9.1](#) операторы для сравнения значений UUID.

9.15. XML-функции

Функции и подобные им выражения, описанные в этом разделе, работают со значениями типа `xml`. Информацию о типе `xml` вы можете найти в [Разделе 8.13](#). Подобные функциям выражения `xmlparse` и `xmlserialize`, преобразующие значения `xml` в текст и обратно, здесь повторно не рассматриваются.

Для использования этих функций Postgres Pro нужно задействовать соответствующую библиотеку при сборке: `configure --with-libxml`.

9.15.1. Создание XML-контента

Для получения XML-контента из данных SQL существует целый набор функций и функциональных выражений, особенно полезных для выдачи клиентским приложениям результатов запроса в виде XML-документов.

9.15.1.1. `xmlcomment`

```
xmlcomment ( text ) → xml
```

Функция `xmlcomment` создаёт XML-значение, содержащее XML-комментарий с заданным текстом. Этот текст не должен содержать «--» или заканчиваться знаком «-», в противном случае результат не будет допустимым XML-комментарием. Если аргумент этой функции NULL, результатом её тоже будет NULL.

Пример:

```
SELECT xmlcomment('hello');
```

```
xmlcomment
-----
<!--hello-->
```

9.15.1.2. `xmlconcat`

```
xmlconcat ( xml [, ...] ) → xml
```

Функция `xmlconcat` объединяет несколько XML-значений и выдаёт в результате один фрагмент XML-контента. Значения NULL отбрасываются, так что результат будет равен NULL, только если все аргументы равны NULL.

Пример:

```
SELECT xmlconcat ('<abc/>', '<bar>foo</bar>');
```

```
      xmlconcat
-----
<abc/><bar>foo</bar>
```

XML-объявления, если они присутствуют, обрабатываются следующим образом. Если во всех аргументах содержатся объявления одной версии XML, эта версия будет выдана в результате; в противном случае версии не будет. Если во всех аргументах определён атрибут `standalone` со значением «yes», это же значение будет выдано в результате. Если во всех аргументах есть объявление `standalone`, но минимум в одном со значением «no», в результате будет это значение. В противном случае в результате не будет объявления `standalone`. Если же окажется, что в результате должно присутствовать объявление `standalone`, а версия не определена, тогда в результате будет выведена версия 1.0, так как XML-объявление не будет допустимым без указания версии. Указания кодировки игнорируются и будут удалены в любых случаях.

Пример:

```
SELECT xmlconcat ('<?xml version="1.1"?><foo/>', '<?xml version="1.1" standalone="no"?><bar/>');
```

```
      xmlconcat
-----
<?xml version="1.1"?><foo/><bar/>
```

9.15.1.3. xmlelement

```
xmlelement ( NAME имя [, XMLATTRIBUTES ( значение_атрибута [AS атрибут] [, ...] )]
[, содержимое [, ...]] ) → xml
```

Выражение `xmlelement` создаёт XML-элемент с заданным именем, атрибутами и содержимым. Аргументы *имя* и *значение_атрибута*, показанные в синтаксисе, обозначают простые идентификаторы, а не определённые значения. Аргументы *значение_атрибута* и *содержимое* являются выражениями, которые могут выдавать любой тип данных Postgres Pro. Аргументы внутри `XMLATTRIBUTES` генерируют атрибуты XML-элемента, к которому также добавляются значения *содержимое*, в итоге формируя его содержимое.

Примеры:

```
SELECT xmlelement(name foo);
```

```
      xmlelement
-----
<foo/>
```

```
SELECT xmlelement(name foo, xmlattributes('xyz' as bar));
```

```
      xmlelement
-----
<foo bar="xyz"/>
```

```
SELECT xmlelement(name foo, xmlattributes(current_date as bar), 'cont', 'ent');
```

```
      xmlelement
```

```
-----
<foo bar="2007-01-26">content</foo>
```

Если имена элементов и атрибутов содержат символы, недопустимые в XML, эти символы заменяются последовательностями `_xHHHH_`, где `HHHH` — шестнадцатеричный код символа в Unicode. Например:

```
SELECT xmlelement(name "foo$bar", xmlattributes('xyz' as "a&b"));

          xmlelement
-----
<foo_x0024_bar a_x0026_b="xyz"/>
```

Если в качестве значения атрибута используется столбец таблицы, имя атрибута можно не указывать явно, этим именем станет имя столбца. Во всех остальных случаях имя атрибута должно быть определено явно. Таким образом, это выражение допустимо:

```
CREATE TABLE test (a xml, b xml);
SELECT xmlelement(name test, xmlattributes(a, b)) FROM test;
```

А следующие варианты — нет:

```
SELECT xmlelement(name test, xmlattributes('constant'), a, b) FROM test;
SELECT xmlelement(name test, xmlattributes(func(a, b))) FROM test;
```

Содержимое элемента, если оно задано, будет форматировано согласно его типу данных. Когда оно само имеет тип `xml`, из него можно конструировать сложные XML-документы. Например:

```
SELECT xmlelement(name foo, xmlattributes('xyz' as bar),
                  xmlelement(name abc),
                  xmlcomment('test'),
                  xmlelement(name xyz));

          xmlelement
-----
<foo bar="xyz"><abc/><!--test--><xyz/></foo>
```

Содержимое других типов будет оформлено в виде допустимых символьных данных XML. Это, в частности, означает, что символы `<`, `>` и `&` будут преобразованы в сущности XML. Двоичные данные (данные типа `bytea`) представляются в кодировке `base64` или в шестнадцатеричном виде, в зависимости от значения параметра `xmlbinary`. Следует ожидать, что конкретные представления отдельных типов данных могут быть изменены для приведения преобразований Postgres Pro в соответствие со стандартом SQL:2006 и новее, как описано в [Подразделе D.3.1.3](#).

9.15.1.4. xmlforest

```
xmlforest ( содержимое [ AS имя ] [, ...] ) → xml
```

Выражение `xmlforest` создаёт последовательность XML-элементов с заданными именами и содержанием. Как и в функции `xmlelement`, каждый аргумент *имя* должен быть простым идентификатором, а выражения *содержимое* могут иметь любой тип данных.

Примеры:

```
SELECT xmlforest('abc' AS foo, 123 AS bar);

          xmlforest
-----
<foo>abc</foo><bar>123</bar>

SELECT xmlforest(table_name, column_name)
FROM information_schema.columns
WHERE table_schema = 'pg_catalog';
```

xmlforest

```
-----
<table_name>pg_authid</table_name><column_name>rolname</column_name>
<table_name>pg_authid</table_name><column_name>rolsuper</column_name>
...
```

Как показано во втором примере, имя элемента можно опустить, если источником содержимого служит столбец (в этом случае именем элемента по умолчанию будет имя столбца). В противном случае это имя необходимо указывать.

Имена элементов с символами, недопустимыми для XML, преобразуются так же, как и для `xmlelement`. Данные содержимого тоже приводятся к виду, допустимому для XML (кроме данных, которые уже имеют тип `xml`).

Заметьте, что такие XML-последовательности не являются допустимыми XML-документами, если они содержат больше одного элемента на верхнем уровне, поэтому может иметь смысл вложить выражения `xmlforest` в `xmlelement`.

9.15.1.5. xmlpi

`xmlpi ( NAME имя [, содержимое] ) → xml`

Выражение `xmlpi` создаёт инструкцию обработки XML. Как и в функции `xmlelement`, аргумент *имя* должен быть простым идентификатором, а выражения *содержимое* могут иметь любой тип данных. Содержимое, если оно задано, не должно содержать последовательность символов `?>`.

Пример:

```
SELECT xmlpi(name php, 'echo "hello world";');
```

xmlpi

```
-----
<?php echo "hello world";?>
```

9.15.1.6. xmlroot

`xmlroot ( xml, VERSION {text|NO VALUE} [, STANDALONE {YES|NO|NO VALUE}] ) → xml`

Выражение `xmlroot` изменяет свойства корневого узла XML-значения. Если в нём указывается версия, она заменяет значение в объявлении версии корневого узла; также в корневой узел переносится значение свойства `standalone`.

```
SELECT xmlroot(xmlparse(document '<?xml version="1.1"?><content>abc</content>'),
               version '1.0', standalone yes);
```

xmlroot

```
-----
<?xml version="1.0" standalone="yes"?>
<content>abc</content>
```

9.15.1.7. xmlagg

`xmlagg ( xml ) → xml`

Функция `xmlagg`, в отличие от других описанных здесь функций, является агрегатной. Она соединяет значения, поступающие на вход агрегатной функции, подобно функции `xmlconcat`, но делает это, обрабатывая множество строк, а не несколько выражений в одной строке. Дополнительно агрегатные функции описаны в [Разделе 9.21](#).

Пример:

```
CREATE TABLE test (y int, x xml);
```

```
INSERT INTO test VALUES (1, '<foo>abc</foo>');
INSERT INTO test VALUES (2, '<bar/>');
SELECT xmlagg(x) FROM test;
      xmlagg
```

```
-----
<foo>abc</foo><bar/>
```

Чтобы задать порядок сложения элементов, в агрегатный вызов можно добавить предложение ORDER BY, описанное в [Подразделе 4.2.7](#). Например:

```
SELECT xmlagg(x ORDER BY y DESC) FROM test;
      xmlagg
```

```
-----
<bar/><foo>abc</foo>
```

Следующий нестандартный подход рекомендовался в предыдущих версиях и может быть по-прежнему полезен в некоторых случаях:

```
SELECT xmlagg(x) FROM (SELECT * FROM test ORDER BY y DESC) AS tab;
      xmlagg
```

```
-----
<bar/><foo>abc</foo>
```

9.15.2. Условия с XML

Описанные в этом разделе выражения проверяют свойства значений xml.

9.15.2.1. IS DOCUMENT

```
xml IS DOCUMENT → boolean
```

Выражение IS DOCUMENT возвращает true, если аргумент представляет собой правильный XML-документ, false в противном случае (т. е. если это фрагмент содержимого) и NULL, если его аргумент также NULL. Чем документы отличаются от фрагментов содержимого, вы можете узнать в [Разделе 8.13](#).

9.15.2.2. IS NOT DOCUMENT

```
xml IS NOT DOCUMENT → boolean
```

Выражение IS NOT DOCUMENT возвращает false, если аргумент представляет собой правильный XML-документ, true в противном случае (т. е. если это фрагмент содержимого) и NULL, если его аргумент — NULL.

9.15.2.3. XMLEXISTS

```
XMLEXISTS ( text PASSING [BY {REF|VALUE}] xml [BY {REF|VALUE}] ) → boolean
```

Функция xmlexists вычисляет выражение XPath 1.0 (первый аргумент), используя в качестве элемента контекста переданное XML-значение. Эта функция возвращает false, если в результате этого вычисления выдаётся пустое множество узлов, или true, если выдаётся любое другое значение. Если один из аргументов равен NULL, результатом также будет NULL. Отличный от NULL аргумент, передающий элемент контекста, должен представлять XML-документ, а не фрагмент содержимого или какое-либо значение, недопустимое в XML.

Пример:

```
SELECT xmlexists('//*[town[text() = ''Toronto'']]' PASSING BY VALUE
  '<towns><town>Toronto</town><town>Ottawa</town></towns>');
```

```
      xmlexists
```

```
-----
t
(1 row)
```


Предложения BY REF и BY VALUE в Postgres Pro принимаются, но игнорируются, о чём рассказывается в [Подразделе D.3.2](#).

Согласно стандарту SQL, функция `xml_exists` должна вычислять выражение, используя средства языка XML Query, но Postgres Pro воспринимает только выражения XPath 1.0, что освещается в [Подразделе D.3.1](#).

9.15.2.4. `xml_is_well_formed`

```
xml_is_well_formed ( text ) →boolean
xml_is_well_formed_document ( text ) →boolean
xml_is_well_formed_content ( text ) →boolean
```

Эти функции проверяют, представляет ли текст правильно оформленный XML, и возвращают соответствующее логическое значение. Функция `xml_is_well_formed_document` проверяет аргумент как правильно оформленный документ, а `xml_is_well_formed_content` — правильно оформленное содержимое. Функция `xml_is_well_formed` может делать первое или второе, в зависимости от значения параметра конфигурации `xmloption` (DOCUMENT или CONTENT, соответственно). Это значит, что `xml_is_well_formed` помогает понять, будет ли успешным простое приведение к типу `xml`, тогда как две другие функции проверяют, будут ли успешны соответствующие варианты `XMLPARSE`.

Примеры:

```
SET xmloption TO DOCUMENT;
SELECT xml_is_well_formed('<>');
xml_is_well_formed
-----
f
(1 row)

SELECT xml_is_well_formed('<abc/>');
xml_is_well_formed
-----
t
(1 row)

SET xmloption TO CONTENT;
SELECT xml_is_well_formed('abc');
xml_is_well_formed
-----
t
(1 row)

SELECT xml_is_well_formed_document('<pg:foo xmlns:pg="http://postgresql.org/stuff">bar</pg:foo>');
xml_is_well_formed_document
-----
t
(1 row)

SELECT xml_is_well_formed_document('<pg:foo xmlns:pg="http://postgresql.org/stuff">bar</my:foo>');
xml_is_well_formed_document
-----
f
(1 row)
```

Последний пример показывает, что при проверке также учитываются сопоставления пространств имён.

9.15.3. Обработка XML

Для обработки значений типа `xml` в Postgres Pro представлены функции `xpath` и `xpath_exists`, вычисляющие выражения XPath 1.0, а также табличная функция `XMLTABLE`.

9.15.3.1. xpath

```
xpath ( xpath text, xml xml [, nsarray text[] ] ) → xml[]
```

Функция `xpath` вычисляет выражение XPath 1.0 в аргументе `xpath` (заданное в виде текста) для заданного XML-значения `xml`. Она возвращает массив XML-значений, соответствующих набору узлов, полученному при вычислении выражения XPath. Если выражение XPath выдаёт не набор узлов, а скалярное значение, возвращается массив с одним элементом.

Вторым аргументом должен быть правильно оформленный XML-документ. В частности, в нём должен быть единственный корневой элемент.

В необязательном третьем аргументе функции передаются сопоставления пространств имён. Эти сопоставления должны определяться в двумерном массиве типа `text`, во второй размерности которого 2 элемента (т. е. это должен быть массив массивов, состоящих из 2 элементов). В первом элементе каждого массива определяется псевдоним (префикс) пространства имён, а во втором — его URI. Псевдонимы, определённые в этом массиве, не обязательно должны совпадать с префиксами пространств имён в самом XML-документе (другими словами, для XML-документа и функции `xpath` псевдонимы имеют *локальный* характер).

Пример:

```
SELECT xpath('/my:a/text()', '<my:a xmlns:my="http://example.com">test</my:a>',
            ARRAY[ARRAY['my', 'http://example.com']]);
```

```
xpath
-----
{test}
(1 row)
```

Для пространства имён по умолчанию (анонимного) это выражение можно записать так:

```
SELECT xpath('//mydefns:b/text()', '<a xmlns="http://example.com"><b>test</b></a>',
            ARRAY[ARRAY['mydefns', 'http://example.com']]);
```

```
xpath
-----
{test}
(1 row)
```

9.15.3.2. xpath_exists

```
xpath_exists ( xpath text, xml xml [, nsarray text[] ] ) → boolean
```

Функция `xpath_exists` представляет собой специализированную форму функции `xpath`. Она возвращает не отдельные XML-значения, удовлетворяющие выражению XPath 1.0, а только один логический признак, показывающий, имеются ли такие значения (то есть выдаёт ли это выражение что-либо, отличное от пустого множества узлов). Эта функция равнозначна стандартному условию `XMLEXISTS`, за исключением того, что она также поддерживает сопоставления пространств имён.

Пример:

```
SELECT xpath_exists('/my:a/text()', '<my:a xmlns:my="http://example.com">test</my:a>',
                    ARRAY[ARRAY['my', 'http://example.com']]);
```

```
xpath_exists
-----
```

```
t
(1 row)
```

9.15.3.3. xmltable

```
XMLTABLE (
    [XMLNAMESPACES ( uri_пространства_имён AS имя_пространства_имён [, ...] ),]
    выражение_строки PASSING [BY {REF|VALUE}] выражение_документа [BY {REF|VALUE}]
    COLUMNS имя { тип [PATH выражение_столбца] [DEFAULT выражение_по_умолчанию] [NOT
    NULL | NULL]
                | FOR ORDINALITY }
    [, ...]
) → setof record
```

Выражение `xmltable` строит таблицу из XML-значения, применяя фильтр XPath для извлечения строк и формируя столбцы с заданным определением. Хотя по синтаксису оно подобно функции, оно может применяться только в качестве таблицы в предложении `FROM`.

Необязательное предложение `XMLNAMESPACES` задаёт разделённый запятыми список определений пространств имён, в котором `uri_пространства_имён` — выражение типа `text`, а `имя_пространства_имён` — простой идентификатор. В нём определяются пространства имён XML, используемые в документе, и их псевдонимы. Определение пространства по умолчанию в настоящее время не поддерживается.

В обязательном аргументе `выражение_строки` передаётся выражение XPath 1.0 (в виде значения `text`), которое будет вычисляться для XML-значения `выражение_документа`, служащего элементом контекста, и выдавать набор узлов XML. Эти узлы `xmltable` преобразует в выходные строки. Если `выражение_документа` — `NULL`, а также если `выражение_строки` выдаёт пустой набор узлов или любое значение, отличное от набора узлов, выходные строки не выдаются.

`Выражение_документа` передаёт элемент контекста для `выражения_строки`. Таким элементом должен быть правильно оформленный XML-документ; фрагменты/наборы деревьев не допускаются. Предложения `BY REF` и `BY VALUE` принимаются, но игнорируются, о чём рассказывается в [Подразделе D.3.2](#).

Согласно стандарту SQL, функция `xmltable` должна вычислять выражения, используя средства языка XML Query, но Postgres Pro воспринимает только выражения XPath 1.0; подробнее об этом говорится в [Подразделе D.3.1](#).

В обязательном предложении `COLUMNS` задаётся список столбцов для выходной таблицы. Формат этого предложения показан выше в описании синтаксиса. Для каждого столбца должно задаваться имя и тип данных (если только не указано `FOR ORDINALITY`, что подразумевает тип `integer`). Путь, значение по умолчанию и признак допустимости `NULL` могут опускаться.

Столбец с признаком `FOR ORDINALITY` будет заполняться номерами строк, начиная с 1, в том порядке, в котором эти строки идут в наборе узлов, представляющем результат `выражения_строки`. Признак `FOR ORDINALITY` может быть не больше чем у одного столбца.

Примечание

В XPath 1.0 не определён порядок узлов в наборе, поэтому код, рассчитывающий на определённый порядок результатов, будет зависеть от конкретной реализации. Подробнее об этом рассказывается в [Подразделе D.3.1.2](#).

В `выражении_столбца` задаётся выражение XPath 1.0, которое вычисляется для каждой строки применительно к текущему узлу, полученному в результате вычисления `выражения_строки` и служащему элементом контекста, и выдаёт значение столбца. Если `выражение_столбца` отсутствует, в качестве неявного пути используется имя столбца.

Если выражение XPath возвращает не XML (это может быть строка, логическое или числовое значение в XPath 1.0) и столбец имеет тип Postgres Pro, отличный от `xml`, значение присваивается столбцу так же, как типу Postgres Pro присваивается строковое представление значения. (Если это значение логическое, его строковым представлением будет 1 или 0, если тип столбца относится к числовой категории, и `true` или `false` в противном случае.)

Если заданное для столбца выражение XPath возвращает непустой набор узлов XML и этот столбец в Postgres Pro имеет тип `xml`, значение ему будет присвоено как есть, когда оно имеет форму документа или содержимого.¹

Когда выходному столбцу `xml` присваивается не XML-содержимое, оно представляется в виде одного текстового узла, содержащего строковое значение результата. XML-результат, присваиваемый столбцу любого другого типа, должен состоять не более чем из одного узла, иначе выдаётся ошибка. При наличии же ровно одного узла столбцу, с учётом его типа Postgres Pro, присваивается строковое представление этого узла (получаемое по правилам функции `string` в XPath 1.0).

Строковым значением XML-элемента является конкатенация всех текстовых узлов, содержащихся в нём и во вложенных в него элементах, в порядке следования этих узлов в документе. Строковым значением элемента без внутренних текстовых узлов является пустая строка (не `NULL`). Любые атрибуты со свойствами `xsi:nil` игнорируются. Заметьте, что состоящий только из пробельных символов узел `text()` между двумя нетекстовыми элементами при этом сохраняется, и начальные пробелы в узле `text()` не убираются. Правила, определяющие строковые представления для других типов XML-узлов и не XML-значений, можно найти в описании функции `string` языка XPath 1.0.

Представленные здесь правила преобразования не соответствуют в точности тем, что определены в стандарте SQL, о чём рассказывается в [Подразделе D.3.1.3](#).

Когда выражение пути возвращает для данной строки пустой набор узлов (обычно когда нет соответствия этому пути), столбец получает значение `NULL`, если только не задано *выражение_по_умолчанию*. Если же оно задано, столбец получает результат данного выражения.

Указанное *выражение_по_умолчанию* вычисляется не единожды для вызова функции `xmltable`, а каждый раз, когда для столбца требуется очередное значение по умолчанию. Если же это выражение оказывается стабильным или постоянным, повторное вычисление может не выполняться. Это означает, что в *выражении_по_умолчанию* вы можете с пользой применять изменчивые функции, такие как `nextval`.

Столбцы могут иметь признак `NOT NULL`. Если *выражение_столбца* для столбца с признаком `NOT NULL` не соответствует ничему и при этом отсутствует указание `DEFAULT` или *выражение_по_умолчанию* также выдаёт `NULL`, происходит ошибка.

Примеры:

```
CREATE TABLE xmldata AS SELECT
xml $$
<ROWS>
  <ROW id="1">
    <COUNTRY_ID>AU</COUNTRY_ID>
    <COUNTRY_NAME>Australia</COUNTRY_NAME>
  </ROW>
  <ROW id="5">
    <COUNTRY_ID>JP</COUNTRY_ID>
    <COUNTRY_NAME>Japan</COUNTRY_NAME>
    <PREMIER_NAME>Shinzo Abe</PREMIER_NAME>
    <SIZE unit="sq_mi">145935</SIZE>
```

¹Примером формы содержимого является результат, содержащий больше одного элемента-узла на верхнем уровне или какой-либо не пробельный текст вне единственного элемента. Результат выражения XPath может иметь и другую форму, например, если оно выбирает узел атрибута из содержащего этот атрибут элемента. Такой результат будет приведён в форму содержимого, в которой каждый подобный недопустимый узел заменяется строковым представлением, получаемым по правилам функции `string` в XPath 1.0.

```

</ROW>
<ROW id="6">
  <COUNTRY_ID>SG</COUNTRY_ID>
  <COUNTRY_NAME>Singapore</COUNTRY_NAME>
  <SIZE unit="sq_km">697</SIZE>
</ROW>
</ROWS>
$$ AS data;

SELECT xmltable.*
  FROM xmldata,
       XMLTABLE('/ROWS/ROW'
                PASSING data
                COLUMNS id int PATH '@id',
                        ordinality FOR ORDINALITY,
                        "COUNTRY_NAME" text,
                        country_id text PATH 'COUNTRY_ID',
                        size_sq_km float PATH 'SIZE[@unit = "sq_km"]',
                        size_other text PATH
                          'concat(SIZE[@unit!="sq_km"], " ", SIZE[@unit!="sq_km"]/'
@unit)',
                        premier_name text PATH 'PREMIER_NAME' DEFAULT 'not specified');

 id | ordinality | COUNTRY_NAME | country_id | size_sq_km | size_other | premier_name
----+-----+-----+-----+-----+-----+-----
1 | 1 | Australia | AU |  |  | not
specified
5 | 2 | Japan | JP |  | 145935 sq_mi | Shinzo Abe
6 | 3 | Singapore | SG | 697 |  | not
specified

```

Следующий пример иллюстрирует сложение нескольких узлов text(), использование имени столбца в качестве фильтра XPath и обработку пробельных символов, XML-комментариев и инструкций обработки:

```

CREATE TABLE xmlelements AS SELECT
xml $$
  <root>
    <element> Hello<!-- xyxxz -->2a2<?aaaaa?> <!--x--> bbb<x>xxx</x>CC </element>
  </root>
$$ AS data;

SELECT xmltable.*
  FROM xmlelements, XMLTABLE('/root' PASSING data COLUMNS element text);
 element
-----
Hello2a2 bbbxxxCC

```

Следующий пример показывает, как с помощью предложения XMLNAMESPACES можно задать список пространств имён, используемых в XML-документе и в выражениях XPath:

```

WITH xmldata(data) AS (VALUES ('
<example xmlns="http://example.com/myns" xmlns:B="http://example.com/b">
  <item foo="1" B:bar="2"/>
  <item foo="3" B:bar="4"/>
  <item foo="4" B:bar="5"/>
</example>'::xml)
)
SELECT xmltable.*

```

```
FROM XMLTABLE (XMLNAMESPACES ('http://example.com/myns' AS x,
                              'http://example.com/b' AS "B"),
              '/x:example/x:item'
              PASSING (SELECT data FROM xmldata)
              COLUMNS foo int PATH '@foo',
                      bar int PATH '@B:bar');
```

```
foo | bar
-----+-----
  1 |    2
  3 |    4
  4 |    5
(3 rows)
```

9.15.4. Отображение таблиц в XML

Следующие функции отображают содержимое реляционных таблиц в значения XML. Их можно рассматривать как средства экспорта в XML:

```
table_to_xml ( table regclass, nulls boolean,
              tableforest boolean, targetns text ) → xml
query_to_xml ( query text, nulls boolean,
              tableforest boolean, targetns text ) → xml
cursor_to_xml ( cursor refcursor, count integer, nulls boolean,
              tableforest boolean, targetns text ) → xml
```

`table_to_xml` отображает в xml содержимое таблицы, имя которой задаётся в параметре `table`. Тип `regclass` принимает идентификаторы строк в обычной записи, которые могут содержать указание схемы и кавычки (за подробностями обратитесь к [Разделу 8.19](#)). Функция `query_to_xml` выполняет запрос, текст которого передаётся в параметре `query`, и отображает в xml результирующий набор. Последняя функция, `cursor_to_xml` выбирает указанное число строк из курсора, переданного в параметре `cursor`. Этот вариант рекомендуется использовать с большими таблицами, так как все эти функции создают результирующий xml в памяти.

Если параметр `tableforest` имеет значение `false`, результирующий XML-документ выглядит так:

```
<имя_таблицы>
  <row>
    <имя_столбца1> данные </имя_столбца1>
    <имя_столбца2> данные </имя_столбца2>
  </row>

  <row>
    ...
  </row>

  ...
</имя_таблицы>
```

А если `tableforest` равен `true`, в результате будет выведен следующий фрагмент XML:

```
<имя_таблицы>
  <имя_столбца1> данные </имя_столбца1>
  <имя_столбца2> данные </имя_столбца2>
</имя_таблицы>

<имя_таблицы>
  ...
</имя_таблицы>

...
```

Если имя таблицы неизвестно, например, при отображении результатов запроса или курсора, вместо него в первом случае вставляется `table`, а во втором — `row`.

Выбор между этими форматами остаётся за пользователем. Первый вариант позволяет создать готовый XML-документ, что может быть полезно для многих приложений, а второй удобно применять с функцией `cursor_to_xml`, если её результаты будут собираться в документ позже. Полученный результат можно изменить по вкусу с помощью рассмотренных выше функций создания XML-содержимого, в частности `xmlelement`.

Значения данных эти функции отображают так же, как и ранее описанная функция `xmlelement`.

Параметр `nulls` определяет, нужно ли включать в результат значения NULL. Если он установлен, значения NULL в столбцах представляются так:

```
<имя_столбца xsi:nil="true"/>
```

Здесь `xsi` — префикс пространства имён XML Schema Instance. При этом в результирующий XML будет добавлено соответствующее объявление пространства имён. Если же данный параметр равен `false`, столбцы со значениями NULL просто не будут выводиться.

Параметр `targetns` определяет целевое пространство имён для результирующего XML. Если пространство имён не нужно, значением этого параметра должна быть пустая строка.

Следующие функции выдают документы XML Schema, которые содержат схемы отображений, выполняемых соответствующими ранее рассмотренными функциями:

```
table_to_xmlschema ( table regclass, nulls boolean,
                    tableforest boolean, targetns text ) → xml
query_to_xmlschema ( query text, nulls boolean,
                    tableforest boolean, targetns text ) → xml
cursor_to_xmlschema ( cursor refcursor, nulls boolean,
                    tableforest boolean, targetns text ) → xml
```

Чтобы результаты отображения данных в XML соответствовали XML-схемам, важно, чтобы соответствующим функциям передавались одинаковые параметры.

Следующие функции выдают отображение данных в XML и соответствующую XML-схему в одном документе (или фрагменте), объединяя их вместе. Это может быть полезно там, где желательно получить самодостаточные результаты с описанием:

```
table_to_xml_and_xmlschema ( table regclass, nulls boolean,
                             tableforest boolean, targetns text ) → xml
query_to_xml_and_xmlschema ( query text, nulls boolean,
                             tableforest boolean, targetns text ) → xml
```

В дополнение к ним есть следующие функции, способные выдать аналогичные представления для целых схем в базе данных или даже для всей текущей базы:

```
schema_to_xml ( schema name, nulls boolean,
               tableforest boolean, targetns text ) → xml
schema_to_xmlschema ( schema name, nulls boolean,
                    tableforest boolean, targetns text ) → xml
schema_to_xml_and_xmlschema ( schema name, nulls boolean,
                             tableforest boolean, targetns text ) → xml

database_to_xml ( nulls boolean,
                tableforest boolean, targetns text ) → xml
database_to_xmlschema ( nulls boolean,
                      tableforest boolean, targetns text ) → xml
database_to_xml_and_xmlschema ( nulls boolean,
                               tableforest boolean, targetns text ) → xml
```

Эти функции пропускают таблицы, которые не может читать текущий пользователь. Функции уровня базы также пропускают схемы, для которых текущий пользователь не имеет права использования (USAGE).

Заметьте, что объём таких данных может быть очень большим, а XML должен сформироваться в памяти. Поэтому, вместо того чтобы пытаться отобразить в XML сразу всё содержимое больших схем или баз данных, лучше делать это по таблицам, возможно, даже используя курсор.

Результат отображения содержимого схемы будет выглядеть так:

```
<имя_схемы>

отображение-таблицы1

отображение-таблицы2

...

</имя_схемы>
```

Формат отображения таблицы определяется параметром *tableforest*, описанным выше.

Результат отображения содержимого базы данных будет таким:

```
<имя_БД>

<имя_схемы1>
...
</имя_схемы1>

<имя_схемы2>
...
</имя_схемы2>

...

</имя_БД>
```

Здесь отображение схемы имеет вид, показанный выше.

В качестве примера, иллюстрирующего использование результата этих функций, на [Примере 9.1](#) показано XSLT-преобразование, которое переводит результат функции *table_to_xml_and_xmlschema* в HTML-документ, содержащий таблицу с данными. Подобным образом результаты этих функций можно преобразовать и в другие форматы на базе XML.

Пример 9.1. XSLT-преобразование, переводящее результат SQL/XML в формат HTML

```
<?xml version="1.0"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://www.w3.org/1999/xhtml"
>

  <xsl:output method="xml"
    doctype-system="http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd"
    doctype-public="-//W3C/DTD XHTML 1.0 Strict//EN"
    indent="yes"/>

  <xsl:template match="/*">
    <xsl:variable name="schema" select="//xsd:schema"/>
    <xsl:variable name="tabletypename"
```



```

        select="$schema/xsd:element[@name=name(current())]/@type"/>
<xsl:variable name="rowtypename"
        select="$schema/xsd:complexType[@name=$tablename]/xsd:sequence/
xsd:element[@name='row']/@type"/>

<html>
  <head>
    <title><xsl:value-of select="name(current())"/></title>
  </head>
  <body>
    <table>
      <tr>
        <xsl:for-each select="$schema/xsd:complexType[@name=$rowtypename]/
xsd:sequence/xsd:element[@name=">
          <th><xsl:value-of select="."/></th>
        </xsl:for-each>
      </tr>

      <xsl:for-each select="row">
        <tr>
          <xsl:for-each select="*>
            <td><xsl:value-of select="."/></td>
          </xsl:for-each>
        </tr>
      </xsl:for-each>
    </table>
  </body>
</html>
</xsl:template>

</xsl:stylesheet>

```

9.16. Функции и операторы JSON

В этом разделе описываются:

- функции и операторы, предназначенные для работы с данными JSON
- язык путей SQL/JSON

Чтобы узнать больше о стандарте SQL/JSON, обратитесь к [sqltr-19075-6](#). Типы JSON, поддерживаемые в Postgres Pro, описаны в [Разделе 8.14](#).

9.16.1. Обработка и создание данных JSON

Примечание

Функции, работающие с JSONB, не принимают символы '\u0000'. Чтобы избежать ошибок и заменять их на лету, необходимо указать символ Unicode в параметре конфигурации [unicode_nul_character_replacement_in_jsonb](#).

В [Таблице 9.45](#) показаны имеющиеся операторы для работы с данными JSON (см. [Раздел 8.14](#)). Кроме них для типа jsonb, но не для json, определены обычные операторы сравнения, показанные в [Таблице 9.1](#). Они следуют правилам упорядочивания для операций В-дерева, описанным в [Подразделе 8.14.4](#). В [Разделе 9.21](#) вы также можете узнать об агрегатной функции json_agg, которая агрегирует значения записи в виде JSON, и агрегатной функции json_object_agg, агрегирующей пары значений в объект JSON, а также их аналогах для jsonb, функциях jsonb_agg и jsonb_object_agg.

Таблица 9.45. Операторы для типов json и jsonb

Оператор Описание Пример(ы)
<pre>json -> integer → json jsonb -> integer → jsonb</pre> Извлекает n-й элемент JSON-массива (элементы массива нумеруются с 0, а отрицательные числа задают позиции с конца). <pre>'[{"a":"foo"}, {"b":"bar"}, {"c":"baz"}]':::json -> 2 → {"c":"baz"} '[{"a":"foo"}, {"b":"bar"}, {"c":"baz"}]':::json -> -3 → {"a":"foo"}</pre>
<pre>json -> text → json jsonb -> text → jsonb</pre> Извлекает поле JSON-объекта по заданному ключу. <pre>'{"a": {"b":"foo"}}':::json -> 'a' → {"b":"foo"}</pre>
<pre>json ->> integer → text jsonb ->> integer → text</pre> Извлекает n-й элемент из JSON-массива, в виде значения text. <pre>'[1,2,3]':::json ->> 2 → 3</pre>
<pre>json ->> text → text jsonb ->> text → text</pre> Извлекает поле JSON-объекта по заданному ключу, в виде значения text. <pre>'{"a":1, "b":2}':::json ->> 'b' → 2</pre>
<pre>json #> text[] → json jsonb #> text[] → jsonb</pre> Извлекает внутренний JSON-объект по заданному пути, элементами которого могут быть индексы массивов или ключи. <pre>'{"a": {"b": ["foo", "bar"]}}':::json #> '{a,b,1}' → "bar"</pre>
<pre>json #>> text[] → text jsonb #>> text[] → text</pre> Извлекает внутренний JSON-объект по заданному пути в виде значения text. <pre>'{"a": {"b": ["foo", "bar"]}}':::json #>> '{a,b,1}' → bar</pre>

Примечание

Если структура входного JSON не соответствует запросу, например указанный ключ или элемент массива отсутствует, операторы извлечения поля/элемента/пути не выдают ошибку, а возвращают NULL.

Некоторые из следующих операторов существуют только для jsonb, как показано в [Таблице 9.46](#). В [Подразделе 8.14.4](#) описано, как эти операторы могут использоваться для эффективного поиска в индексированных данных jsonb.

Таблица 9.46. Дополнительные операторы jsonb

Оператор Описание Пример(ы)
<pre>jsonb @> jsonb → boolean</pre>

Оператор	Описание	Пример(ы)
	Первое значение JSON содержит второе? (Что означает «содержит», подробно описывается в Подразделе 8.14.3.)	<code>'{"a":1, "b":2}':::jsonb @> '{"b":2}':::jsonb → t</code>
<code>jsonb <@ jsonb → boolean</code>	Первое значение JSON содержится во втором?	<code>'{"b":2}':::jsonb <@ '{"a":1, "b":2}':::jsonb → t</code>
<code>jsonb ? text → boolean</code>	Текстовая строка присутствует в значении JSON в качестве ключа верхнего уровня или элемента массива?	<code>'{"a":1, "b":2}':::jsonb ? 'b' → t</code> <code>'[a, b, c]':::jsonb ? 'b' → t</code>
<code>jsonb ? text[] → boolean</code>	Какие-либо текстовые строки из массива присутствуют в качестве ключей верхнего уровня или элементов массива?	<code>'{"a":1, "b":2, "c":3}':::jsonb ? array['b', 'd'] → t</code>
<code>jsonb ?& text[] → boolean</code>	Все текстовые строки из массива присутствуют в качестве ключей верхнего уровня или элементов массива?	<code>'[a, b, c]':::jsonb ?& array['a', 'b'] → t</code>
<code>jsonb jsonb → jsonb</code>	Соединяет два значения jsonb. При соединении двух массивов получается массив, содержащий все их элементы. При соединении двух объектов получается объект с объединённым набором ключей и значений, при этом в случае совпадения ключей выбирается значение из второго объекта. Все другие варианты соединения реализуются путём преобразования аргументов, отличных от массивов, в массивы с одним элементом, которые затем как массивы и соединяются. Эта операция не рекурсивна — объединение производится только на верхнем уровне структуры объекта или массива.	<code>'[a, b]':::jsonb '[a, d]':::jsonb → [a, b, a, d]</code> <code>'{a: b}':::jsonb '{c: d}':::jsonb → {a: b, c: d}</code> <code>'[1, 2]':::jsonb '3':::jsonb → [1, 2, 3]</code> <code>'{a: b}':::jsonb '42':::jsonb → [{a: b}, 42]</code> Чтобы вставить один массив в другой в качестве массива, поместите его в дополнительный массив, например: <code>'[1, 2]':::jsonb jsonb_build_array('[3, 4]':::jsonb) → [1, 2, [3, 4]]</code>
<code>jsonb - text → jsonb</code>	Удаляет ключ (и его значение) из JSON-объекта или соответствующие строковые значения из JSON-массива.	<code>'{a: b, c: d}':::jsonb - 'a' → {c: d}</code> <code>'[a, b, c, b]':::jsonb - 'b' → [a, c]</code>
<code>jsonb - text[] → jsonb</code>	Удаляет из левого операнда все перечисленные ключи или элементы массива.	<code>'{a: b, c: d}':::jsonb - '{a, c}':::text[] → {}</code>
<code>jsonb - integer → jsonb</code>	Удаляет из массива элемент в заданной позиции (отрицательные номера позиций отсчитываются от конца). Выдаёт ошибку, если переданное значение JSON — не массив.	

Оператор
Описание
Пример(ы)
<code>'["a", "b"]'::jsonb - 1 → ["a"]</code>
<code>jsonb #- text[] → jsonb</code> Удаляет поле или элемент массива с заданным путём, в составе которого могут быть индексы массивов или ключи. <code>'["a", {"b":1}]'::jsonb #- '{1,b}' → ["a", {}]</code>
<code>jsonb @? jsonpath → boolean</code> Выдаёт ли путь JSON какой-либо элемент для заданного значения JSON? <code>'{"a":[1,2,3,4,5]}'::jsonb @? '\$.a[*] ? (@ > 2)' → t</code>
<code>jsonb @@ jsonpath → boolean</code> Возвращает результат проверки предиката пути JSON для заданного значения JSON. При этом учитывается только первый элемент результата. Если результат не является логическим, возвращается NULL. <code>'{"a":[1,2,3,4,5]}'::jsonb @@ '\$.a[*] > 2' → t</code>

Примечание

Операторы `jsonpath @?` и `@@` подавляют следующие ошибки: отсутствие поля объекта или элемента массива, несовпадение типа элемента JSON и ошибки в числах и дате/времени. Описанные ниже функции, связанные с `jsonpath`, тоже могут подавлять ошибки такого рода. Это может быть полезно, когда нужно произвести поиск по набору документов JSON, имеющих различную структуру.

В [Таблице 9.47](#) показаны функции, предназначенные для создания значений `json` и `jsonb`.

Таблица 9.47. Функции для создания JSON

Функция
Описание
Пример(ы)
<code>to_json (anyelement) → json</code> <code>to_jsonb (anyelement) → jsonb</code> Преобразует произвольное SQL-значение в <code>json</code> или <code>jsonb</code>. Массивы и составные структуры преобразуются рекурсивно в массивы и объекты (многомерные массивы становятся в JSON массивами массивов). Для других типов, для которых определено приведение к <code>json</code>, применяется эта функция приведения;^a для всех остальных выдаётся скалярное значение. Значения всех скалярных типов, кроме числового, логического и NULL, представляются в текстовом виде; при этом может добавляться экранирование символов, необходимое для получения допустимых строковых значений <code>json</code> или <code>jsonb</code>. <code>to_json('Fred said "Hi."'::text) → "Fred said \"Hi.\""</code> <code>to_jsonb(row(42, 'Fred said "Hi."'::text)) → {"f1": 42, "f2": "Fred said \"Hi.\""}</code>
<code>array_to_json (anyarray [, boolean]) → json</code> Преобразует массив SQL в JSON-массив. Эта функция работает так же, как <code>to_json</code>, но если в необязательном логическом параметре передаётся <code>true</code>, между элементами массива верхнего уровня дополнительно добавляются переводы строк. <code>array_to_json('{{1,5},{99,100}}'::int[]) → [[1,5],[99,100]]</code>
<code>row_to_json (record [, boolean]) → json</code>

Функция	Описание	Пример(ы)
	Преобразует составное значение SQL в JSON-объект. Эта функция работает так же, как <code>to_json</code> , но если в необязательном логическом параметре передаётся <code>true</code> , между элементами верхнего уровня дополнительно добавляются переводы строк.	<code>row_to_json(row(1, 'foo'))</code> → <code>{"f1":1, "f2":"foo"}</code>
<code>json_build_array</code> (VARIADIC "any") → json <code>jsonb_build_array</code> (VARIADIC "any") → jsonb	Формирует JSON-массив (возможно, разнородный) из переменного списка аргументов. Каждый аргумент преобразуется методом <code>to_json</code> или <code>to_jsonb</code> .	<code>json_build_array(1, 2, 'foo', 4, 5)</code> → <code>[1, 2, "foo", 4, 5]</code>
<code>json_build_object</code> (VARIADIC "any") → json <code>jsonb_build_object</code> (VARIADIC "any") → jsonb	Формирует JSON-объект из переменного списка аргументов. По соглашению в этом списке перечисляются по очереди ключи и значения. Аргументы, задающие ключи, приводятся к текстовому типу, а аргументы-значения преобразуются методом <code>to_json</code> или <code>to_jsonb</code> .	<code>json_build_object('foo', 1, 2, row(3, 'bar'))</code> → <code>{"foo" : 1, "2" : {"f1":3, "f2":"bar"}}</code>
<code>json_object</code> (text[]) → json <code>jsonb_object</code> (text[]) → jsonb	Формирует объект JSON из текстового массива. Этот массив должен иметь либо одну размерность с чётным числом элементов (в этом случае они воспринимаются как чередующиеся ключи/значения), либо две размерности и при этом каждый внутренний массив содержит ровно два элемента, которые воспринимаются как пара ключ/значение. Все значения преобразуются в строки JSON.	<code>json_object('{a, 1, b, "def", c, 3.5}')</code> → <code>{"a" : "1", "b" : "def", "c" : "3.5"}</code> <code>json_object('{{a, 1}, {b, "def"}, {c, 3.5}}')</code> → <code>{"a" : "1", "b" : "def", "c" : "3.5"}</code>
<code>json_object</code> (keys text[], values text[]) → json <code>jsonb_object</code> (keys text[], values text[]) → jsonb	Эта форма <code>json_object</code> принимает ключи и значения по парам из двух отдельных текстовых массивов. В остальных отношениях она не отличается от вариации с одним аргументом.	<code>json_object('{a,b}', '{1,2}')</code> → <code>{"a": "1", "b": "2"}</code>

^aНапример, в расширении [hstore](#) определено преобразование из `hstore` в `json`, так что значения `hstore`, преобразуемые функциями создания JSON, будут представлены в виде объектов JSON, а не как примитивные строковые значения.

В [Таблице 9.48](#) показаны функции, предназначенные для работы со значениями `json` и `jsonb`.

Таблица 9.48. Функции для обработки JSON

Функция	Описание	Пример(ы)
<code>json_array_elements</code> (json) → setof json <code>jsonb_array_elements</code> (jsonb) → setof jsonb	Разворачивает JSON-массив верхнего уровня в набор значений JSON.	<code>select * from json_array_elements('[1,true, [2,false]]')</code> → value

Функция Описание Пример(ы)
<pre> ----- 1 true [2,false]</pre>
<pre> json_array_elements_text (json) → setof text jsonb_array_elements_text (jsonb) → setof text Разворачивает JSON-массив верхнего уровня в набор значений text. select * from json_array_elements_text('["foo", "bar"]') → value ----- foo bar</pre>
<pre> json_array_length (json) → integer jsonb_array_length (jsonb) → integer Возвращает число элементов во внешнем JSON-массиве верхнего уровня. json_array_length(' [1,2,3,{ "f1":1, "f2":[5,6] },4] ') → 5 jsonb_array_length('[]') → 0</pre>
<pre> json_each (json) → setof record (key text, value json) jsonb_each (jsonb) → setof record (key text, value jsonb) Разворачивает JSON-объект верхнего уровня в набор пар ключ/значение (key/value). select * from json_each('{ "a": "foo", "b": "bar" }') → key value -----+----- a "foo" b "bar"</pre>
<pre> json_each_text (json) → setof record (key text, value text) jsonb_each_text (jsonb) → setof record (key text, value text) Разворачивает JSON-объект верхнего уровня в набор пар ключ/значение (key/value). Воз- вращаемые значения будут иметь тип text. select * from json_each_text('{ "a": "foo", "b": "bar" }') → key value -----+----- a foo b bar</pre>
<pre> json_extract_path (from_json json, VARIADIC path_elems text[]) → json jsonb_extract_path (from_json jsonb, VARIADIC path_elems text[]) → jsonb Извлекает внутренний JSON-объект по заданному пути. (То же самое делает оператор #>, но в некоторых случаях может быть удобнее записать путь в виде списка отдельных аргументов.) json_extract_path('{ "f2":{ "f3":1 }, "f4":{ "f5":99, "f6": "foo" } }', 'f4', 'f6') → "foo"</pre>
<pre> json_extract_path_text (from_json json, VARIADIC path_elems text[]) → text jsonb_extract_path_text (from_json jsonb, VARIADIC path_elems text[]) → text Извлекает внутренний JSON-объект по заданному пути в виде значения text. (То же са- мое делает оператор #>>.)</pre>

Функция	Описание	Пример(ы)
		<pre>json_extract_path_text('{"f2":{"f3":1},"f4":{"f5":99,"f6":"foo"}}', 'f4', 'f6') → foo</pre>
		<pre>json_object_keys (json) → setof text jsonb_object_keys (jsonb) → setof text</pre> Выдаёт множество ключей в JSON-объекте верхнего уровня. <pre>select * from json_object_keys('{"f1":"abc","f2":{"f3":"a", "f4":"b"}}') → json_object_keys ----- f1 f2</pre>
		<pre>json_populate_record (base anyelement, from_json json) → anyelement jsonb_populate_record (base anyelement, from_json jsonb) → anyelement</pre> Разворачивает JSON-объект верхнего уровня в строку, имеющую составной тип аргумента <i>base</i>. В JSON-объекте просматриваются поля, имена которых соответствуют именам столбцов выходного типа, и их значения вставляются в эти столбцы результата. (Поля, не соответствующие именам никаких выходных столбцов, пропускаются.) Обычно в <i>base</i> просто передаётся NULL, что означает, что выходные столбцы, которым не нашлось соответствие в объекте, получают значения NULL. Однако если в аргументе <i>base</i> передаётся не NULL, то для таких столбцов будут использованы значения из этого аргумента. Для преобразования значения JSON в SQL-тип выходного столбца последовательно применяются следующие правила: • Значение NULL в JSON всегда преобразуется в SQL NULL. • Если выходной столбец имеет тип <code>json</code> или <code>jsonb</code>, значение JSON воспроизводится без изменений. • Если выходной столбец имеет составной тип (тип кортежа) и значение JSON является объектом JSON, поля этого объекта преобразуются в столбцы типа выходного кортежа в результате рекурсивного применения этих правил. • Подобным образом, если выходной столбец имеет тип-массив и значение JSON представляет массив JSON, элементы данного массива преобразуются в элементы выходного массива в результате рекурсивного применения этих правил. • Если же значение JSON является строкой, содержимое этой строки передаётся входной функции преобразования для типа данных целевого столбца. • В противном случае функции преобразования для типа данных целевого столбца передаётся обычное текстовое представление значения JSON. В следующем примере значение JSON фиксировано, но обычно такая функция обращается с использованием <code>LATERAL</code> к столбцу <code>json</code> или <code>jsonb</code> из другой таблицы, фигурирующей в предложении <code>FROM</code>. Функция <code>json_populate_record</code> в предложении <code>FROM</code> будет работать эффективно, так как все извлечённые столбцы можно использовать, не выполняя повторные вызовы функции. <pre>create type subrowtype as (d int, e text); create type myrowtype as (a int, b text[], c subrowtype); select * from json_populate_record(null::myrowtype, '{"a": 1, "b": ["2", "a b"], "c": {"d": 4, "e": "a b c"}, "x": "foo"}') →</pre> <pre> a b c ---+-----+----- 1 {2,"a b"} (4,"a b c")</pre>
		<pre>json_populate_recordset (base anyelement, from_json json) → setof anyelement jsonb_populate_recordset (base anyelement, from_json jsonb) → setof anyelement</pre>

Функция

Описание

Пример(ы)

Разворачивает JSON-массив верхнего уровня с объектами в набор строк, имеющих составной тип аргумента *base*. Каждый элемент JSON-массива обрабатывается так же, как было описано выше для `json[b]_populate_record`.

```
create type twoints as (a int, b int);
select * from json_populate_recordset(null::twoints, ' [{"a":1,"b":2},
{"a":3,"b":4}] ') →
```

```
a | b
---+---
1 | 2
3 | 4
```

`json_to_record ( json ) → record`

`jsonb_to_record ( jsonb ) → record`

Разворачивает JSON-объект верхнего уровня в строку, имеющую составной тип, определённый в предложении AS. (Как и со всеми функциями, возвращающими значение `record`, вызывающий запрос должен явно определять структуру записи в AS.) Выходная запись заполняется полями JSON-объекта так же, как было описано выше для `json[b]_populate_record`. Так как этой функции не передаётся запись, столбцы, для которых не находится соответствие, всегда получают значения NULL.

```
create type myrowtype as (a int, b text);
select * from json_to_record('{"a":1,"b":[1,2,3],"c":[1,2,3],
"e":"bar","r":{"a": 123, "b": "a b c"}}') as x(a int, b text, c int[],
d text, r myrowtype) →
```

```
a |      b      |      c      | d |      r
---+-----+-----+---+-----
1 | [1,2,3] | {1,2,3} |   | (123,"a b c")
```

`json_to_recordset ( json ) → setof record`

`jsonb_to_recordset ( jsonb ) → setof record`

Разворачивает JSON-массив верхнего уровня с объектами в набор строк, имеющих составной тип, определённый в предложении AS. (Как и со всеми функциями, возвращающими значение `record`, вызывающий запрос должен явно определять структуру записи в AS.) Каждый элемент JSON-массива обрабатывается так же, как было описано выше для `json[b]_populate_record`.

```
select * from json_to_recordset(' [{"a":1,"b":"foo"}, {"a":"2",
"b":"bar"}] ') as x(a int, b text) →
```

```
a | b
---+---
1 | foo
2 |
```

`jsonb_set ( target jsonb, path text[], new_value jsonb[, create_if_missing boolean] ) → jsonb`

Возвращает объект *target*, в котором элемент, на который указывает путь *path*, заменяется значением *new_value* либо значение *new_value* добавляется, когда параметр *create_if_missing* равен true (это значение по умолчанию) и элемент, на который указывает *path*, не существует. Чтобы это изменение произошло, все предыдущие элементы, на которые указывает путь, должны существовать. В противном случае *target* возвращается без изменений. Как и с операторами, принимающими пути, отрицательные целые числа, фигурирующие в *path*, отсчитывают элементы с конца JSON-массива. Если на последнем шаге пути указывается индекс, выходящий за границы массива, и па-

Функция	Описание	Пример(ы)
	параметр <i>create_if_missing</i> равен <code>true</code>, новое значение добавляется в начало массива, когда индекс отрицательный, или в конец, когда он положительный. <code>jsonb_set('{"f1":1,"f2":null},2,null,3)', '{0,f1}', '[2,3,4]', false) → [{"f1": [2, 3, 4], "f2": null}, 2, null, 3]</code> <code>jsonb_set('{"f1":1,"f2":null},2)', '{0,f3}', '[2,3,4]') → [{"f1": 1, "f2": null, "f3": [2, 3, 4]}, 2]</code>	
<code>jsonb_set_lax</code>	(<i>target jsonb, path text[], new_value jsonb[, create_if_missing boolean[, null_value_treatment text]]</i>) → <i>jsonb</i> Если значение <i>new_value</i> отлично от <code>NULL</code>, эта функция действует так же, как и <code>jsonb_set</code>. В противном случае она действует согласно значению <i>null_value_treatment</i>, которое может принимать значение <code>'raise_exception'</code>, <code>'use_json_null'</code>, <code>'delete_key'</code> или <code>'return_target'</code>. Значение по умолчанию: <code>'use_json_null'</code>. <code>jsonb_set_lax('{"f1":1,"f2":null},2,null,3)', '{0,f1}', null) → [{"f1": null, "f2": null}, 2, null, 3]</code> <code>jsonb_set_lax('{"f1":99,"f2":null},2)', '{0,f3}', null, true, 'return_target') → [{"f1": 99, "f2": null}, 2]</code>	
<code>jsonb_insert</code>	(<i>target jsonb, path text[], new_value jsonb[, insert_after boolean]</i>) → <i>jsonb</i> Возвращает объект <i>target</i> с вставленным в него значением <i>new_value</i>. Когда элемент, на который указывает путь <i>path</i>, является элементом массива, <i>new_value</i> вставляется перед этим элементом, если параметр <i>insert_after</i> равен <code>false</code> (по умолчанию), либо после него, если <i>insert_after</i> равен <code>true</code>. Когда элемент, на который указывает <i>path</i>, является полем объекта, <i>new_value</i> будет вставлено только если у объекта ещё нет такого ключа. Чтобы это изменение произошло, все предыдущие элементы, на которые указывает путь, должны существовать. В противном случае <i>target</i> возвращается без изменений. Как и с операторами, принимающими пути, отрицательные целые числа, фигурирующие в <i>path</i>, отсчитывают элементы с конца JSON-массива. Если на последнем шаге пути указывается индекс, выходящий за границы массива, новое значение добавляется в начало массива, когда индекс отрицательный, или в конец, когда он положительный. <code>jsonb_insert('{"a": [0,1,2]}', '{a, 1}', '"new_value"') → {"a": [0, "new_value", 1, 2]}</code> <code>jsonb_insert('{"a": [0,1,2]}', '{a, 1}', '"new_value"', true) → {"a": [0, 1, "new_value", 2]}</code>	
<code>json_strip_nulls</code>	(<i>json</i>) → <i>json</i> <code>jsonb_strip_nulls</code> (<i>jsonb</i>) → <i>jsonb</i> Удаляет из данного значения JSON все поля объектов, имеющие значения <code>null</code>, на всех уровнях вложенности. Значения <code>null</code>, не относящиеся к полям объектов, сохраняются без изменений. <code>json_strip_nulls('{"f1":1, "f2":null}, 2, null, 3)') → [{"f1":1},2, null,3]</code>	
<code>jsonb_path_exists</code>	(<i>target jsonb, path jsonpath[, vars jsonb[, silent boolean]]</i>) → <i>boolean</i> Проверяет, есть ли в заданном значении JSON какой-либо элемент, соответствующий пути JSON. В случае присутствия аргумента <i>vars</i>, он должен содержать JSON-объект, поля которого будут подставляться в выражение <i>jsonpath</i> под их именами. Если передаётся аргумент <i>silent</i> и он равен <code>true</code>, функция подавляет те же ошибки, что и операторы <code>@?</code> и <code>@@</code>.	

Функция	Описание	Пример(ы)
		<pre>jsonb_path_exists('{ "a": [1,2,3,4,5] }', '\$.a[*] ? (@ >= \$min && @ <= \$max)', '{ "min":2, "max":4 }') → t</pre>
	<code>jsonb_path_match (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → boolean</code> Возвращает результат проверки предиката пути JSON для заданного значения JSON. При этом учитывается только первый элемент результата. Если результат не является логическим, возвращается NULL. Дополнительные аргументы <i>vars</i> и <i>silent</i> действуют так же, как и для <code>jsonb_path_exists</code> .	<pre>jsonb_path_match('{ "a": [1,2,3,4,5] }', 'exists(\$.a[*] ? (@ >= \$min && @ <= \$max))', '{ "min":2, "max":4 }') → t</pre>
	<code>jsonb_path_query (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → setof jsonb</code> Возвращает все элементы JSON, полученные по указанному пути для заданного значения JSON. Дополнительные аргументы <i>vars</i> и <i>silent</i> действуют так же, как и для <code>jsonb_path_exists</code> .	<pre>select * from jsonb_path_query('{ "a": [1,2,3,4,5] }', '\$.a[*] ? (@ >= \$min && @ <= \$max)', '{ "min":2, "max":4 }') →</pre> <pre> jsonb_path_query ----- 2 3 4</pre>
	<code>jsonb_path_query_array (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → jsonb</code> Возвращает все элементы JSON, полученные по указанному пути для заданного значения JSON, в виде JSON-массива. Дополнительные аргументы <i>vars</i> и <i>silent</i> действуют так же, как и для <code>jsonb_path_exists</code> .	<pre>jsonb_path_query_array('{ "a": [1,2,3,4,5] }', '\$.a[*] ? (@ >= \$min && @ <= \$max)', '{ "min":2, "max":4 }') → [2, 3, 4]</pre>
	<code>jsonb_path_query_first (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → jsonb</code> Возвращает первый элемент JSON, полученный по указанному пути для заданного значения JSON, либо NULL, если этому пути не соответствуют никакие элементы. Дополнительные аргументы <i>vars</i> и <i>silent</i> действуют так же, как и для <code>jsonb_path_exists</code> .	<pre>jsonb_path_query_first('{ "a": [1,2,3,4,5] }', '\$.a[*] ? (@ >= \$min && @ <= \$max)', '{ "min":2, "max":4 }') → 2</pre>
	<code>jsonb_path_exists_tz (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → boolean</code> <code>jsonb_path_match_tz (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → boolean</code> <code>jsonb_path_query_tz (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → setof jsonb</code> <code>jsonb_path_query_array_tz (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → jsonb</code> <code>jsonb_path_query_first_tz (target jsonb, path jsonpath [, vars jsonb [, silent boolean]]) → jsonb</code>	Эти функции работают подобно их двойникам без суффикса <code>_tz</code> , описанным выше, за исключением того, что данные функции поддерживают сравнение значений с датой/временем, для которых должны учитываться часовые пояса. В следующем примере дата без

Функция	Описание	Пример(ы)
	указания времени 2015-08-02 должна преобразоваться в дату/время с часовым поясом, поэтому результат будет зависеть от текущего значения TimeZone . Вследствие этой зависимости данные функции помечены как стабильные (не постоянные), и поэтому их нельзя использовать в индексах. Их двойники являются постоянными и могут использоваться в индексах, но при попытке выполнить такое сравнение они будут выдавать ошибку.	<pre>jsonb_path_exists_tz('["2015-08-01 12:00:00 -05"]', '\$[*] ? (@.datetime() < "2015-08-02".datetime())') → t</pre>
	Преобразует данное значение JSON в визуально улучшенное текстовое представление с отступами.	<pre>jsonb_pretty (jsonb) → text jsonb_pretty('[{ "f1":1, "f2":null}, 2]') →</pre> <pre>[{ "f1": 1, "f2": null }, 2]</pre>
	Возвращает тип значения на верхнем уровне JSON в виде текстовой строки. Возможные типы: object, array, string, number, boolean и null. (Не следует путать эту строку null с SQL-значением NULL; см. примеры.)	<pre>json_typeof (json) → text jsonb_typeof (jsonb) → text json_typeof('-123.4') → number json_typeof('null'::json) → null json_typeof(NULL::json) IS NULL → t</pre>

9.16.2. Язык путей SQL/JSON

Выражения путей SQL/JSON определяют элементы, извлекаемые из данных JSON, подобно тому, как выражения XPath позволяют обращаться из SQL к XML. В Postgres Pro выражения путей представляются в виде типа данных `jsonpath` и могут использовать любые элементы, описанные в [Подразделе 8.14.7](#).

Операторы и функции запросов к JSON передают поступившее им выражение *обработчику путей* для вычисления. Если выражению соответствуют фигурирующие в запросе данные JSON, в результате выдаётся соответствующий элемент JSON или набор элементов. Выражения путей записываются на языке путей SQL/JSON и могут включать сложные арифметические выражения и функции.

Выражение пути состоит из последовательности элементов, допустимых для типа `jsonpath`. Обычно оно вычисляется слева направо, но при необходимости порядок операций можно изменить, добавив скобки. В случае успешного вычисления выдаётся последовательность элементов JSON, и результат вычисления возвращается в функцию JSON-запроса, которая завершает обработку выражения.

Для обращения к поступившему в запрос значению JSON (*элементу контекста*) в выражении пути используется переменная `$`. Затем могут следовать один или более [операторов обращения](#), которые, опускаясь в структуре JSON с одного уровня на другой, извлекают элементы, вложенные в текущий элемент контекста. При этом каждый последующий оператор имеет дело с результатом вычисления, полученным на предыдущем шаге.

Например, предположим, что у вас есть данные JSON, полученные от GPS-трекера, которые вы хотели бы проанализировать:

```
{
  "track": {
    "segments": [
      {
        "location": [ 47.763, 13.4034 ],
        "start time": "2018-10-14 10:05:14",
        "HR": 73
      },
      {
        "location": [ 47.706, 13.2635 ],
        "start time": "2018-10-14 10:39:21",
        "HR": 135
      }
    ]
  }
}
```

Чтобы получить имеющиеся сегменты треков, воспользуйтесь оператором обращения *.ключ*, который позволяет погрузиться внутрь JSON-объектов:

```
$.track.segments
```

Для получения содержимого массива обычно используется оператор *[*]*. Например, следующий путь выдаст координаты концов всех имеющихся сегментов треков:

```
$.track.segments[*].location
```

Чтобы получить координаты только первого сегмента, можно задать соответствующий индекс в операторе обращения *[]*. Заметьте, что индексы в JSON-массивах отсчитываются с 0:

```
$.track.segments[0].location
```

Результат каждого шага вычисления выражения может быть обработан операторами и методами `jsonpath`, перечисленными в [Подразделе 9.16.2.2](#). Перед именем метода должна стоять точка. Например, так можно получить размер массива:

```
$.track.segments.size()
```

Другие примеры использования операторов и методов `jsonpath` в выражениях пути приведены ниже в [Подразделе 9.16.2.2](#).

Определяя путь, также можно использовать *выражения фильтра*, работающие подобно предложению `WHERE` в SQL. Выражение фильтра начинается со знака вопроса и содержит условие в скобках:

```
? (условие)
```

Выражения фильтра указываются сразу после шага вычисления пути, к которому они должны применяться. Результаты шага проходят через фильтр, и на выходе остаются только те элементы, которые удовлетворяют заданному условию. В SQL/JSON действует троичная логика, то есть результатом выражения может быть `true`, `false` или `unknown` (неизвестность). Значение `unknown` играет ту же роль, что и `NULL` в SQL, и может быть проверено предикатом `is unknown`. На последующих шагах вычисления пути будут обрабатываться только те элементы, для которых выражение фильтра выдало `true`.

Функции и операторы, которые можно использовать в выражениях фильтра, перечислены в [Таблице 9.50](#). Переменная `@` в выражении фильтра представляет фильтруемое значение (результат предыдущего шага в пути). Для получения внутренних элементов этого значения вы можете добавить после `@` операторы обращения.

Например, предположим, что вы хотите получить все показатели пульса, превышающие 130. Это можно сделать с помощью следующего выражения:

```
$.track.segments[*].HR ? (@ > 130)
```

Чтобы получить в результате время начала соответствующих сегментов, вы должны отфильтровать ненужные сегменты, а затем выбрать время, так что фильтр будет применяться к предыдущему шагу и путь окажется другим:

```
$.track.segments[*] ? (@.HR > 130). "start time"
```

Можно также использовать несколько выражений фильтра по очереди, когда это требуется. Например, следующее выражение выбирает время начала всех сегментов с определёнными координатами и высоким показателем пульса:

```
$.track.segments[*] ? (@.location[1] < 13.4) ? (@.HR > 130). "start time"
```

Также возможно использовать фильтры на разных уровнях вложенности. В следующем примере сначала сегменты фильтруются по координатам, а затем для подходящих сегментов, если они находятся, выбираются значения высокого пульса:

```
$.track.segments[*] ? (@.location[1] < 13.4).HR ? (@ > 130)
```

Можно также вкладывать выражения фильтра одно в другое:

```
$.track ? (exists(@.segments[*] ? (@.HR > 130))).segments.size()
```

Данное выражение возвращает количество сегментов в треке, если он содержит сегменты с высокими показателями пульса, или пустую последовательность, если таких сегментов нет.

Реализация языка путей SQL/JSON в Postgres Pro имеет следующие отличия от стандарта SQL/JSON:

- Выражение пути может быть булевым предикатом, хотя стандарт SQL/JSON допускает предикаты только в фильтрах. Это необходимо для реализации оператора @@. Например, следующее выражение jsonpath допускается в Postgres Pro:

```
$.track.segments[*].HR < 70
```

- Есть небольшие различия в интерпретации шаблонов регулярных выражений, используемых в фильтрах like_regex; имеющиеся особенности описаны в [Подразделе 9.16.2.4](#).
- Если поместить описание пути в квадратные скобки [], то результат вычисления пути автоматически обёртывается в массив.

9.16.2.1. Строгий и нестрогий режимы

Когда вы обращаетесь к данным JSON, выражение пути может не соответствовать фактической структуре данных JSON. Попытка обратиться к несуществующему члену объекта или элементу массива приводит к ошибке структурного типа. Для обработки такого рода ошибок в выражениях путей SQL/JSON предусмотрены два режима:

- lax (по умолчанию) — нестрогий режим, в котором обработчик путей неявно адаптирует обрабатываемые данные к указанному пути. Любые возникающие структурные ошибки подавляются и заменяются пустыми последовательностями SQL/JSON.
- strict — строгий режим, в котором структурные ошибки выдаются как есть.

Нестрогий режим упрощает сопоставление структуры документа JSON с выражением пути в случаях, когда данные JSON не соответствуют ожидаемой схеме. Если операнд не удовлетворяет требованиям определённой операции, он может перед выполнением этой операции автоматически оборачиваться в массив SQL/JSON или наоборот, разворачиваться так, чтобы его элементы образовали последовательность SQL/JSON. Помимо этого, в нестрогом режиме операторы сравнения автоматически разворачивают свои операнды, что позволяет легко сравнивать массивы SQL/JSON. Массив с одним элементом в таком режиме считается равным своему элементу. Автоматическое разворачивание не выполняется только в следующих случаях:

- В выражении пути фигурируют методы size() и type(), возвращающие соответственно число элементов в массиве и тип.
- Обрабатываемые данные JSON содержат вложенные массивы. В этом случае разворачивается только массив верхнего уровня, а внутренние массивы остаются без изменений. Таким обра-

зом, неявное разворачивание может опускаться на каждом шаге вычисления пути только на один уровень.

Например, обрабатывая данные GPS, показанные выше, в нестрогом режиме можно не обращать внимание на то, что в них содержится массив сегментов:

```
lax $.track.segments.location
```

В строгом режиме указанный путь должен в точности соответствовать структуре обрабатываемого документа JSON и выдавать элемент SQL/JSON, поэтому использование такого выражения пути приведёт к ошибке. Чтобы получить такой же результат, как в нестрогом режиме, необходимо явно развернуть массив `segments`:

```
strict $.track.segments[*].location
```

Оператор обращения `.**` в нестрогом режиме может выдавать несколько неожиданные результаты. Например, следующий запрос выберет каждое значение `HR` дважды:

```
lax $.**.HR
```

Это происходит потому, что оператор `.**` выбирает и массив `segments`, и каждый из его элементов, а обращение `.HR` в нестрогом режиме автоматически разворачивает массивы. Во избежание подобных сюрпризов мы рекомендуем использовать оператор обращения `.**` только в строгом режиме. Следующий запрос выбирает каждое значение `HR` в единственном экземпляре:

```
strict $.**.HR
```

9.16.2.2. Операторы и методы SQL/JSON

В [Таблице 9.49](#) показаны операторы и методы, поддерживаемые в значениях `jsonpath`. Обратите внимание, что унарные операторы и методы могут применяться к множеству значений, полученных на предыдущем шаге пути, тогда как бинарные операторы (сложение и т. п.) применяются только к отдельным значениям.

Таблица 9.49. Операторы и методы `jsonpath`

Оператор/Метод Описание Пример(ы)
<i>число + число → число</i> Сложение <code>jsonb_path_query('[2]', '\$[0] + 3')</code> → 5
<i>+ число → число</i> Унарный плюс (нет операции); в отличие от сложения, он может итерационно применяться к множеству значений <code>jsonb_path_query_array('{ "x": [2,3,4] }', '+ \$.x')</code> → [2, 3, 4]
<i>число - число → число</i> Вычитание <code>jsonb_path_query('[2]', '7 - \$[0]')</code> → 5
<i>- число → число</i> Смена знака; в отличие от вычитания, этот оператор может итерационно применяться к множеству значений <code>jsonb_path_query_array('{ "x": [2,3,4] }', '- \$.x')</code> → [-2, -3, -4]
<i>число * число → число</i> Умножение <code>jsonb_path_query('[4]', '2 * \$[0]')</code> → 8
<i>число / число → число</i>

Оператор/Метод Описание Пример(ы)
Деление <code>jsonb_path_query('[8.5]', '\$[0] / 2')</code> → 4.2500000000000000
<i>число % число → число</i> Остаток от деления <code>jsonb_path_query('[32]', '\$[0] % 10')</code> → 2
<i>значение.type() → строка</i> Тип элемента JSON (см. json_typeof) <code>jsonb_path_query_array('[1, "2", {}]', '\$[*].type()')</code> → ["number", "string", "object"]
<i>значение.size() → число</i> Размер элемента JSON (число элементов в массиве либо 1, если это не массив) <code>jsonb_path_query('{ "m": [11, 15] }', '\$.m.size()')</code> → 2
<i>значение.double() → число</i> Приблизительное число с плавающей точкой, преобразованное из строки или числа JSON <code>jsonb_path_query('{ "len": "1.9" }', '\$.len.double() * 2')</code> → 3.8
<i>число.ceil() → число</i> Ближайшее целое, большее или равное заданному числу <code>jsonb_path_query('{ "h": 1.3 }', '\$.h.ceil()')</code> → 2
<i>число.floor() → число</i> Ближайшее целое, меньшее или равное заданному числу <code>jsonb_path_query('{ "h": 1.7 }', '\$.h.floor()')</code> → 1
<i>число.abs() → число</i> Модуль заданного числа (абсолютное значение) <code>jsonb_path_query('{ "z": -0.3 }', '\$.z.abs()')</code> → 0.3
<i>строка.datetime() → тип_даты_времени (см. примечание)</i> Значение даты/времени, полученное из строки <code>jsonb_path_query('["2015-8-1", "2015-08-12"]', '\$[*] ? (@.datetime() < "2015-08-2".datetime())')</code> → "2015-8-1"
<i>string.datetime(шаблон) → тип_даты_времени (см. примечание)</i> Значение даты/времени, преобразованное из строки по шаблону to_timestamp <code>jsonb_path_query_array('["12:30", "18:40"]', '\$[*].datetime("HH24:MI")')</code> → ["12:30:00", "18:40:00"]
<i>object.keyvalue() → массив</i> Пары ключ-значение, представленные в виде массива объектов со следующими тремя полями: "key", "value" и "id"; в "id" содержится уникальный идентификатор объекта, к которому относится данная пара ключ-значение <code>jsonb_path_query_array('{ "x": "20", "y": 32 }', '\$.keyvalue()')</code> → [{"id": 0, "key": "x", "value": "20"}, {"id": 0, "key": "y", "value": 32}]

Примечание

Результирующим типом методов `datetime()` и `datetime(шаблон)` может быть `date`, `timetz`, `time`, `timestampz` или `timestamp`. Эти два метода определяют тип своего результата автоматически.

Метод `datetime()` пытается последовательно сопоставить поступившую на вход строку с ISO-форматами типов `date`, `timetz`, `time`, `timestamptz` и `timestamp`. Встретив первый подходящий формат, он останавливается и возвращает соответствующий тип данных.

Метод `datetime(шаблон)` определяет результирующий тип в соответствии с полями заданного шаблона.

Методы `datetime()` и `datetime(шаблон)` применяют те же правила разбора строки, что и SQL-функция `to_timestamp` (см. [Раздел 9.8](#)), но с тремя исключениями. Во-первых, эти методы не позволяют использовать в шаблоне поля, которым не находится соответствие. Во-вторых, в шаблоне допускаются только следующие разделители: знак минуса, точка, косая черта, запятая, апостроф, точка с запятой, запятая и пробел. В-третьих, разделители в шаблоне должны в точности соответствовать входной строке.

Если требуется сравнить значения разных типов даты/времени, применяется неявное приведение типа. Значение `date` может быть приведено к типу `timestamp` или `timestamptz`; `timestamp` — к типу `timestamptz`, а `time` — к `timetz`. Однако все эти приведения, кроме первого, зависят от текущего значения [TimeZone](#) и поэтому не могут выполняться в функциях `jsonpath`, не учитывающих часовой пояс.

В [Таблице 9.50](#) перечислены допустимые элементы выражения фильтра.

Таблица 9.50. Элементы выражения фильтра `jsonpath`

Предикат/значение	Описание	Пример(ы)
<code>значение == значение → boolean</code>	Проверка равенства (все операторы сравнения, включая этот, работают с любыми скалярными значениями JSON)	<code>jsonb_path_query_array('[1, "a", 1, 3]', '\$[*] ? (@ == 1)')</code> → [1, 1] <code>jsonb_path_query_array('[1, "a", 1, 3]', '\$[*] ? (@ == "a")')</code> → ["a"]
<code>значение != значение → boolean</code> <code>значение <> значение → boolean</code>	Проверка неравенства	<code>jsonb_path_query_array('[1, 2, 1, 3]', '\$[*] ? (@ != 1)')</code> → [2, 3] <code>jsonb_path_query_array('["a", "b", "c"]', '\$[*] ? (@ <> "b")')</code> → ["a", "c"]
<code>значение < значение → boolean</code>	Проверка «меньше»	<code>jsonb_path_query_array('[1, 2, 3]', '\$[*] ? (@ < 2)')</code> → [1]
<code>значение <= значение → boolean</code>	Проверка «меньше или равно»	<code>jsonb_path_query_array('["a", "b", "c"]', '\$[*] ? (@ <= "b")')</code> → ["a", "b"]
<code>значение > значение → boolean</code>	Проверка «больше»	<code>jsonb_path_query_array('[1, 2, 3]', '\$[*] ? (@ > 2)')</code> → [3]
<code>значение >= значение → boolean</code>	Проверка «больше или равно»	<code>jsonb_path_query_array('[1, 2, 3]', '\$[*] ? (@ >= 2)')</code> → [2, 3]
<code>true → boolean</code>		

Предикат/значение	Описание	Пример(ы)
	JSON-константа true	<code>jsonb_path_query('{{"name": "John", "parent": false}, {"name": "Chris", "parent": true}}', '\$[*] ? (@.parent == true)') → {"name": "Chris", "parent": true}</code>
	false → boolean JSON-константа false	<code>jsonb_path_query('{{"name": "John", "parent": false}, {"name": "Chris", "parent": true}}', '\$[*] ? (@.parent == false)') → {"name": "John", "parent": false}</code>
	null → значение JSON-константа null (заметьте, что в отличие от SQL сравнение с null работает традиционным образом)	<code>jsonb_path_query('{{"name": "Mary", "job": null}, {"name": "Michael", "job": "driver"}}', '\$[*] ? (@.job == null) .name') → "Mary"</code>
	логическое_значение && логическое_значение → boolean Логическое И	<code>jsonb_path_query('[1, 3, 7]', '\$[*] ? (@ > 1 && @ < 5)') → 3</code>
	логическое_значение логическое_значение → boolean Логическое ИЛИ	<code>jsonb_path_query('[1, 3, 7]', '\$[*] ? (@ < 1 @ > 5)') → 7</code>
	! логическое_значение → boolean Логическое НЕ	<code>jsonb_path_query('[1, 3, 7]', '\$[*] ? (!(@ < 5))') → 7</code>
	логическое_значение is unknown → boolean Проверяет, является ли unknown результатом логического условия.	<code>jsonb_path_query('[-1, 2, 7, "foo"]', '\$[*] ? ((@ > 0) is unknown)') → "foo"</code>
	строка like_regex строка [flag строка] → boolean Проверяет, соответствует ли первый операнд регулярному выражению, которое задаёт второй операнд с необязательным аргументом flag, влияющим на поведение выражения (см. Подраздел 9.16.2.4).	<code>jsonb_path_query_array('["abc", "abd", "aBdC", "abdacb", "babc"]', '\$[*] ? (@ like_regex "^ab.*c")') → ["abc", "abdacb"]</code> <code>jsonb_path_query_array('["abc", "abd", "aBdC", "abdacb", "babc"]', '\$[*] ? (@ like_regex "^ab.*c" flag "i")') → ["abc", "aBdC", "abdacb"]</code>
	строка starts with строка → boolean Проверяет, является ли второй операнд начальной подстрокой первого.	<code>jsonb_path_query('["John Smith", "Mary Stone", "Bob Johnson"]', '\$[*] ? (@ starts with "John")') → "John Smith"</code>
	exists (выражение_пути) → boolean Проверяет, соответствует ли выражению пути минимум один элемент SQL/JSON. Возвращает unknown, если вычисление выражения пути могло привести к ошибке; это используется во втором примере для недопущения ошибки «ключ не найден» в строгом режиме.	<code>jsonb_path_query('{ "x": [1, 2], "y": [2, 4] }', 'strict \$.* ? (exists (@ ? (@[*] > 2)))') → [2, 4]</code>

Предикат/значение Описание Пример(ы)
<code>jsonb_path_query_array('{ "value": 41}', 'strict \$? (exists (@.name)) .name') → []</code>

9.16.2.3. Расширения

В Postgres Pro есть некоторые расширения стандарта пути SQL/JSON. Эти расширения синтаксиса можно включить, указав дополнительный модификатор `pgpro` перед флагами `strict/lax`. Примеры использования расширений показаны в [Таблице 9.51](#).

Таблица 9.51. Расширения синтаксиса jsonpath

Имя	Описание	Пример JSON	Примеры пути JSON	Результат
Конструктор последовательностей	Создаёт последовательность JSON из списка выражений, разделённых запятыми	<code>[1, 2, 3]</code>	<code>pgpro \$[*], 4, 5</code>	<code>1, 2, 3, 4, 5</code>
Конструктор массивов	Создаёт массив JSON путём перечисления его элементов, заключённых в скобки	<code>[1, 2, 3]</code>	<code>pgpro [\$[*], 4, 5]</code>	<code>[1, 2, 3, 4, 5]</code>
Конструктор объектов	Создаёт объект JSON путём перечисления его полей, заключённых в скобки	<code>{"x": "y"}</code>	<code>pgpro {a: 1, "b c": \$.x}</code>	<code>{"a": 1, "b c": "y"}</code>
Индексация объектов	Извлекает поле объекта JSON, используя указанное выражение в качестве ключа	<code>{"a": 1, "b": "a"}</code>	<code>pgpro \$[\$.b]</code>	<code>1</code>

9.16.2.4. Регулярные выражения SQL/JSON

Выражения путей SQL/JSON могут содержать фильтры `like_regex`, позволяющие сопоставлять текст с регулярным выражением. Например, следующий запрос пути SQL/JSON выберет все строки в массиве, которые начинаются с английской гласной в любом регистре:

```
$[*] ? (@ like_regex "^[aeiou]" flag "i")
```

Необязательная строка `flag` может содержать один или несколько следующих символов: `i`, делающий поиск регистронезависимым, `m`, допускающий сопоставление `^` и `$` с переводами строк, `s`, допускающий сопоставление `.` с символом новой строки, и `a`, берущий в кавычки весь шаблон (в результате производится простой поиск подстроки).

Стандарт SQL/JSON заимствует определение регулярных выражений от оператора `LIKE_REGEX`, который, в свою очередь, реализуется по стандарту XQuery. Однако в Postgres Pro оператор `LIKE_REGEX` в настоящее время отсутствует. Поэтому фильтр `like_regex` реализован с использованием механизма регулярных выражений POSIX, который описан в [Подразделе 9.7.3](#). Вследствие этого наблюдается ряд небольших отклонений от описанного в стандарте поведения SQL/JSON, о которых рассказывается в [Подразделе 9.7.3.8](#). Заметьте однако, что описанная там несовместимость букв флагов не проявляется на уровне SQL/JSON, так как заданные в SQL/JSON флаги XQuery переводятся во флаги, воспринимаемые механизмом POSIX.

Помните, что аргумент, задающий шаблон для `like_regex`, является строкой пути JSON и записывается по правилам, описанным в [Подразделе 8.14.7](#). Это в частности означает, что каждую косую черту в регулярном выражении надо дублировать. Например, чтобы отобрать строковые значения, находящиеся на уровне корня документа и содержащие только цифры, нужно написать:

```
$.* ? (@ like_regex "^\\d+$")
```

9.16.3. Функции и выражения SQL/JSON

Postgres Pro реализует *модель данных SQL/JSON*, обеспечивая встроенную поддержку типов данных JSON в среде SQL. В этой модели данные представляются последовательностями элементов. Каждый элемент может содержать скалярные значения SQL, дополнительно определённое в SQL/JSON значение `null` и составные структуры данных, образуемые объектами и массивами JSON. Данная модель по сути формализует модель данных, описанную в спецификации JSON [RFC 7159](#).

Поддержка SQL/JSON позволяет обрабатывать данные JSON наряду с обычными данными SQL, используя при этом транзакции, например:

- Загружать данные JSON в базу и сохранять их в обычных столбцах SQL в виде символьных или двоичных строк.
- Создавать объекты и массивы JSON из реляционных данных.
- Обращаться к данным JSON, используя функции запросов SQL/JSON и выражения языка путей SQL/JSON.

Существуют две группы функций SQL/JSON. [Функции-конструкторы](#) генерируют данные JSON из значений типов SQL. [Функции запросов](#) вычисляют выражения языка путей SQL/JSON по значениям JSON и создают значения типов SQL/JSON, которые преобразуются в типы SQL.

Многие функции SQL/JSON для соответствия стандарту SQL принимают необязательное предложение `FORMAT`, но оно не влияет на результат, если не указано иное.

В [Таблице 9.52](#) перечислены функции-конструкторы SQL/JSON. Каждая функция принимает предложение `RETURNING`, указывающее возвращаемый тип данных. Для функций `json` и `json_scalar` в этом предложении должно указываться либо `json`, либо `jsonb`. Для других функций-конструкторов — либо один из типов `json`, `jsonb`, `bytea`, либо тип символьных строк (`text`, `char`, `varchar` или `nchar`), либо тип, для которого существует приведение из `json`. По умолчанию возвращается тип `json`.

Примечание

Многие результаты, которые можно получить посредством функций-конструкторов SQL/JSON, также можно получить, вызвав собственные функции PostgreSQL, подробно описанные в [Таблице 9.47](#) и [Таблице 9.62](#).

Таблица 9.52. Функции-конструкторы SQL/JSON

Сигнатура функции
Описание
Пример(ы)
<pre>json (выражение [FORMAT JSON [ENCODING UTF8]] [{ WITH WITHOUT } UNIQUE [KEYS]] [RETURNING тип_данных_json])</pre> Аргумент <i>выражение</i> может быть любого текстового типа или типа <code>bytea</code> в кодировке UTF-8. Если в <i>выражении</i> передаётся <code>NULL</code>, возвращается значение SQL <code>NULL</code>. С указанием <code>WITH UNIQUE</code> в <i>выражении</i> не должно быть объектов с дублирующимися ключами. <pre>json('{"a":123, "b":[true,"foo"], "a":"bar"}') → {"a":123, "b":[true, "foo"], "a":"bar"}</pre> <pre>json('{"a":123,"b":[true,"foo"],"a":"bar"}' returning jsonb) → {"a": "bar", "b": [true, "foo"]}</pre>

Сигнатура функции	Описание	Пример(ы)
<code>json_scalar</code>	<i>(выражение [RETURNING тип_данных_json])</i> Возвращает скалярное значение JSON, представляющее <i>выражение</i> . Если передаётся NULL, возвращается SQL NULL. Если передаётся число или логическое значение, возвращается соответствующее числовое или логическое значение JSON. Для любого другого значения возвращается строка JSON.	<code>json_scalar(123.45) → 123.45</code> <code>json_scalar(CURRENT_TIMESTAMP) → "2022-05-10T10:51:04.62128-04:00"</code>
<code>json_object</code>	<i>([{ выражение_ключа { VALUE ':' } выражение_значения [FORMAT JSON [ENCODING UTF8]] }, ...] [{ NULL ABSENT } ON NULL] [{ WITH WITHOUT } UNIQUE [KEYS]] [RETURNING тип_данных [FORMAT JSON [ENCODING UTF8]]])</i> Создаёт объект JSON из всех заданных пар ключ/значение или пустой объект, если ни одна пара не задана. В аргументе <i>выражение_ключа</i> передаётся скалярное выражение, определяющее ключ JSON, который преобразуется в тип <code>text</code> . Этот параметр не может быть NULL и не должен иметь тип, приводимый к <code>json</code> . С указанием <code>WITH UNIQUE</code> в <i>выражении_ключа</i> не должно быть дублирующихся значений. С указанием <code>ABSENT ON NULL</code> вся пара пропускается, если <i>выражение_значения</i> равно NULL.	<code>json_object('code' VALUE 'P123', 'title': 'Jaws') → {"code" : "P123", "title" : "Jaws"}</code>
<code>json_objectagg</code>	<i>([{ выражение_ключа { VALUE ':' } выражение_значения }] [{ NULL ABSENT } ON NULL] [{ WITH WITHOUT } UNIQUE [KEYS]] [RETURNING тип_данных [FORMAT JSON [ENCODING UTF8]]])</i> Ведёт себя так же, как описанная выше функция <code>json_object</code> , но является агрегатной, поэтому принимает только один параметр <i>выражение_ключа</i> и один параметр <i>выражение_значения</i> .	<code>SELECT json_objectagg(k:v) FROM (VALUES ('a'::text,current_date), ('b', current_date + 1)) AS t(k,v) → { "a" : "2022-05-10", "b" : "2022-05-11" }</code>
<code>json_array</code>	<i>([{ выражение_значения [FORMAT JSON] } [, ...]] [{ NULL ABSENT } ON NULL] [RETURNING тип_данных [FORMAT JSON [ENCODING UTF8]]])</i> <i>json_array ([выражение_запроса] [RETURNING тип_данных [FORMAT JSON [ENCODING UTF8]]])</i> Создаёт массив JSON либо из набора параметров <i>выражения_значения</i> , либо из результатов <i>выражения_запроса</i> , которое должно быть запросом SELECT, возвращающим один столбец. С указанием <code>ABSENT ON NULL</code> значения NULL игнорируются. Такие значения всегда игнорируются, если используется <i>выражение_запроса</i> .	<code>json_array(1,true,json '{"a":null}')</code> → <code>[1, true, {"a":null}]</code> <code>json_array(SELECT * FROM (VALUES(1),(2)) t)</code> → <code>[1, 2]</code>
<code>json_arrayagg</code>	<i>([выражение_значения] [ORDER BY выражение_сортировки] [{ NULL ABSENT } ON NULL] [RETURNING тип_данных [FORMAT JSON [ENCODING UTF8]]])</i> Ведёт себя так же, как функция <code>json_array</code> , но агрегатная, поэтому принимает только один параметр <i>выражение_значения</i> . С указанием <code>ABSENT ON NULL</code> любые значения NULL пропускаются. С указанием <code>ORDER BY</code> элементы добавляются в массив в заданном порядке, а не в том порядке, в котором они вводятся.	<code>SELECT json_arrayagg(v) FROM (VALUES(2),(1)) t(v) → [2, 1]</code>

В [Таблице 9.53](#) описаны средства SQL/JSON для проверки и сериализации JSON.

Таблица 9.53. Функции проверки и сериализации SQL/JSON

Сигнатура функции	Описание	Пример(ы)
<code>выражение IS [NOT] JSON</code>	<code>[{ VALUE SCALAR ARRAY OBJECT }] [{ WITH WITHOUT } UNIQUE [KEYS]]</code>	

Сигнатура функции																														
Описание																														
Пример(ы)																														
Этот предикат проверяет, может ли аргумент <i>выражение</i> представлять собой JSON (возможно, указанного типа). Если указано SCALAR, ARRAY или OBJECT, проверяется, относится ли JSON к этому конкретному типу. С указанием WITH UNIQUE все объекты в <i>выражении</i> также проверяются на наличие дубликатов ключей. <pre>SELECT js, js IS JSON "json?", js IS JSON SCALAR "scalar?", js IS JSON OBJECT "object?", js IS JSON ARRAY "array?" FROM (VALUES ('123'), ('"abc"'), ('{"a": "b"}'), ('[1,2]'), ('abc')) foo(js);</pre> <table><tr><th>js</th><th>json?</th><th>scalar?</th><th>object?</th><th>array?</th></tr><tr><td>123</td><td>t</td><td>t</td><td>f</td><td>f</td></tr><tr><td>"abc"</td><td>t</td><td>t</td><td>f</td><td>f</td></tr><tr><td>{"a": "b"}</td><td>t</td><td>f</td><td>t</td><td>f</td></tr><tr><td>[1,2]</td><td>t</td><td>f</td><td>f</td><td>t</td></tr><tr><td>abc</td><td>f</td><td>f</td><td>f</td><td>f</td></tr></table>	js	json?	scalar?	object?	array?	123	t	t	f	f	"abc"	t	t	f	f	{"a": "b"}	t	f	t	f	[1,2]	t	f	f	t	abc	f	f	f	f
js	json?	scalar?	object?	array?																										
123	t	t	f	f																										
"abc"	t	t	f	f																										
{"a": "b"}	t	f	t	f																										
[1,2]	t	f	f	t																										
abc	f	f	f	f																										
<pre>json_serialize (<i>выражение</i> [FORMAT JSON [ENCODING UTF8]] [RETURNING <i>тип_данных</i> [FORMAT JSON [ENCODING UTF8]]])</pre> Преобразует значение SQL/JSON в символьную или двоичную строку. Аргумент <i>выражение</i> может быть любого типа JSON, любого типа символьных строк или bytea в кодировке UTF-8. Возвращаемый тип может быть любым типом символьных строк или bytea, по умолчанию — text. <pre>json_serialize('{ "a" : 1 } ' RETURNING bytea) → \x7b20226122203a2031207d20</pre>																														

В [Таблице 9.54](#) описаны функции SQL/JSON (кроме json_table), которые можно использовать для обращения к данным JSON.

Примечание

Пути SQL/JSON можно применять только к типу jsonb, поэтому аргумент *элемент_контекста* этих функций, возможно, потребуется привести к типу jsonb.

Таблица 9.54. Функции запросов SQL/JSON

Сигнатура функции
Описание
Пример(ы)
<pre>json_exists (<i>элемент_контекста</i> , <i>выражение_пути</i> [PASSING { <i>значение</i> AS <i>имя_переменной</i> } [, ...]] [RETURNING <i>тип_данных</i>] [{ TRUE FALSE UNKNOWN ERROR } ON ERROR])</pre> Возвращает true, если результатом применения <i>выражения_пути</i> к <i>элементу_контекста</i> с использованием <i>значения</i> оказывается непустое множество элементов. Предложение ON ERROR определяет, что выдаётся в случае возникновения ошибки. Обратите внимание, что если в <i>выражении_пути</i> передаётся strict, в случае отсутствия элементов возникает ошибка. Значение по умолчанию — UNKNOWN, то есть при ошибке выдаётся NULL. <pre>json_exists(jsonb '{"key1": [1,2,3]}', 'strict \$.key1[*] ? (@ > 2)') → t json_exists(jsonb '{"a": [1,2,3]}', 'lax \$.a[5]' ERROR ON ERROR) → f</pre>

Сигнатура функции Описание Пример(ы)	
<code>json_exists(jsonb '{"a": [1,2,3]}', 'strict \$.a[5]' ERROR ON ERROR)</code> → ERROR: jsonpath array subscript is out of bounds (ОШИБКА: индекс массива jsonpath вне диапазона)	
<code>json_value (элемент_контекста , выражение_пути [PASSING { значение AS имя_переменной } [, ...]] [RETURNING тип_данных] [{ ERROR NULL DEFAULT выражение } ON EMPTY] [{ ERROR NULL DEFAULT выражение } ON ERROR])</code> Возвращает результат применения <i>выражения_пути</i> к элементу_контекста с использованием <i>значения</i>. Извлечённое значение должно быть одним скалярным элементом SQL/JSON. Для результатов, которые являются объектами или массивами, вместо этого используйте <code>json_query</code>. Возвращаемый <i>тип_данных</i> имеет ту же семантику, что и для функций-конструкторов, таких как <code>json_objectagg</code>. По умолчанию возвращается тип <code>text</code>. Предложение <code>ON EMPTY</code> определяет поведение, если <i>выражение_пути</i> не выдаёт никакого значения. Предложение <code>ON ERROR</code> определяет поведение в случае ошибки, возникшей либо при вычислении <code>jsonpath</code> (включая приведение к выходному типу), либо при вычислении предложения <code>ON EMPTY</code> (вызванного пустым результатом вычисления <code>jsonpath</code>). <code>json_value(jsonb '"123.45"', '\$' RETURNING float)</code> → 123.45 <code>json_value(jsonb '"03:04 2015-02-01"', '\$.datetime("HH24:MI YYYY-MM-DD")' RETURNING date)</code> → 2015-02-01 <code>json_value(jsonb '[1,2]', 'strict \$[*]' DEFAULT 9 ON ERROR)</code> → 9	
<code>json_query (элемент_контекста , выражение_пути [PASSING { значение AS имя_переменной } [, ...]] [RETURNING тип_данных [FORMAT JSON [ENCODING UTF8]]] [{ WITHOUT WITH { CONDITIONAL [UNCONDITIONAL] } } [ARRAY] WRAPPER] [{ KEEP OMIT } QUOTES [ON SCALAR STRING]] [{ ERROR NULL EMPTY { [ARRAY] OBJECT } DEFAULT выражение } ON EMPTY] [{ ERROR NULL EMPTY { [ARRAY] OBJECT } DEFAULT выражение } ON ERROR])</code> Возвращает результат применения <i>выражения_пути</i> к элементу_контекста с использованием <i>значения</i>. Эта функция должна возвращать строку JSON, поэтому если выражение пути возвращает несколько элементов SQL/JSON, результат следует обернуть, применив предложение <code>WITH WRAPPER</code>. Если выбрана обёртка <code>UNCONDITIONAL</code>, всегда будет применяться обёртка массива, даже если возвращаемое значение уже является одиночным объектом JSON или массивом, но если выбрана обёртка <code>CONDITIONAL</code>, она не будет применяться к одиночному массиву или объекту. По умолчанию используется обёртка <code>UNCONDITIONAL</code>. Если результатом является скалярная строка, возвращаемое значение по умолчанию заключается в кавычки, что делает его допустимым значением JSON. Однако если указано <code>OMIT QUOTES</code>, это поведение меняется (кавычки не добавляются). Предложения <code>ON ERROR</code> и <code>ON EMPTY</code> имеют семантику, аналогичную семантике этих предложений для <code>json_value</code>. Возвращаемый <i>тип_данных</i> имеет ту же семантику, что и для функций-конструкторов, таких как <code>json_objectagg</code>. По умолчанию возвращается тип <code>text</code>. <code>json_query(jsonb '[1,[2,3],null]', 'lax \$[*][1]' WITH CONDITIONAL WRAPPER)</code> → [3]	

9.16.4. JSON_TABLE

`json_table` — это функция SQL/JSON, которая обрабатывает данные JSON и выдаёт результаты в виде реляционного представления, к которому можно обращаться как к обычной таблице SQL. Использовать `json_table` можно только внутри предложения `FROM` оператора `SELECT`.

Принимая данные JSON, функция `json_table` обрабатывает выражение пути и извлекает часть представленных данных, которая будет использоваться в качестве *шаблона строк* для создаваемого представления. Каждый элемент SQL/JSON на верхнем уровне шаблона строк служит источником для отдельной строки в создаваемом реляционном представлении.

Для разделения шаблона строк на столбцы в функции `json_table` применяется предложение `COLUMNS`, определяющее схему создаваемого представления. В этом предложении для каждого создаваемого столбца задаётся отдельное выражение пути, обрабатывающее шаблон строк, извлекающее элемент JSON и возвращающее его в виде отдельного значения SQL для данного столбца. Если требуемое значение находится на вложенном уровне шаблона строк, его можно извлечь, используя вложенное предложение `NESTED PATH`. При объединении столбцов, возвращаемых `NESTED PATH`, в создаваемом представлении могут добавиться несколько новых строк. Такие строки называются *дочерними строками*, а строка, которая их создаёт, — *родительской строкой*.

Строки, формируемые функцией `JSON_TABLE`, соединяются как последующие (`LATERAL`) со строкой, из которой они сформированы, поэтому нет необходимости явно соединять создаваемое представление с исходной таблицей, содержащей данные JSON. При этом, используя предложение `PLAN`, можно определить, как соединять столбцы, которые возвращает `NESTED PATH`.

Каждое предложение `NESTED PATH` может создать один или несколько столбцов. Столбцы, созданные `NESTED PATH` на одном уровне, считаются *соседними*; при этом столбцы, созданные вложенным выражением `NESTED PATH`, считаются потомками столбца, сформированного другим выражением `NESTED PATH` или выражением строки на более высоком уровне. При формировании результата сначала вместе составляются соседние столбцы, а после этого полученные строки соединяются с родительской строкой.

элемент_контекста, *выражение_пути* [`AS имя_пути_json`] [`PASSING { значение AS имя_переменной } [, ...]`]

Входные данные для запроса, выражение пути JSON, определяющее запрос, и необязательное предложение `PASSING`, которое может предоставлять значения данных для *выражения_пути*. Результат обработки входных данных называется *шаблоном строк*. Шаблон строк используется в качестве источника для значений строк в создаваемом представлении.

`COLUMNS( столбец_таблицы_json [, ...] )`

Предложение `COLUMNS`, определяющее схему создаваемого представления. В этом предложении должны указываться все столбцы, в которые будут помещаться элементы SQL/JSON. Выражение *столбец_таблицы_json* имеет следующие варианты синтаксиса:

имя тип [`PATH описание_пути_json`]

Вставляет один элемент SQL/JSON во все строки с указанным столбцом.

Если задаётся выражение `PATH`, *описание_пути_json* в нём определяет шаблон строк и столбец заполняется сформированными элементами SQL/JSON (по одному в строке). Если выражение `PATH` опускается, функция `JSON_TABLE` вычисляет выражение пути `$.имя`, где *имя* — указанное имя столбца. В этом случае имя столбца должно соответствовать одному из ключей в элементе SQL/JSON, созданном шаблоном строк.

Также можно добавить предложения `ON EMPTY` и `ON ERROR`, чтобы определить, как обрабатывать отсутствующие значения или структурные ошибки. Предложения `WRAPPER` и `QUOTES` можно использовать только с типами JSON, массивами и составными типами. Эти предложения имеют тот же синтаксис и семантику, что и в `json_value` и `json_query`.

имя тип `FORMAT представление_json` [`PATH описание_пути_json`]

Создаёт столбец и вставляет составной элемент SQL/JSON во все строки с указанным столбцом.

Если задаётся выражение `PATH`, *описание_пути_json* в нём определяет шаблон строк и столбец заполняется сформированными элементами SQL/JSON (по одному в строке). Если выражение `PATH` опускается, функция `JSON_TABLE` вычисляет выражение пути `$.имя`, где *имя* — указанное имя столбца. В этом случае имя столбца должно соответствовать одному из ключей в элементе SQL/JSON, созданном шаблоном строк.

Также можно добавить предложения `WRAPPER`, `QUOTES`, `ON EMPTY` и `ON ERROR`, чтобы определить дополнительные параметры для возвращаемых элементов SQL/JSON. Эти предложения имеют тот же синтаксис и семантику, что и в `json_query`.

имя `тип EXISTS [ PATH описание_пути_json ]`

Создаёт столбец и вставляет логический элемент во все строки с указанным столбцом.

Если задаётся выражение `PATH`, `описание_пути_json` в нём определяет шаблон строк и столбец заполняется логическими значениями (по одному в каждой строке), показывающими, были ли получены соответствующие элементы SQL/JSON. Для заданного *типа* должно существовать приведение из `boolean`. Если выражение `PATH` опускается, функция `JSON_TABLE` вычисляет выражение пути `$.имя`, где *имя* — указанное имя столбца.

Также можно добавить предложение `ON ERROR`, чтобы определить поведение при ошибке. Это предложение имеет тот же синтаксис и семантику, что и в `json_exists`.

`NESTED PATH описание_пути_json [AS имя_пути_json] COLUMNS ( столбец_таблицы_json [, ...] )`

Извлекает элементы SQL/JSON из вложенных уровней шаблона строк, создаёт один или несколько столбцов, как определено во вложенном предложении `COLUMNS`, и вставляет извлечённые элементы SQL/JSON во все строки с этими столбцами. Выражение `столбец_таблицы_json` во вложенном предложении `COLUMNS` имеет тот же синтаксис, что и в родительском предложении `COLUMNS`.

Синтаксис `NESTED PATH` является рекурсивным, поэтому вкладывая одно предложение `NESTED PATH` в другое, можно опускаться ниже от уровня к уровню. Это позволяет развернуть иерархию объектов и массивов JSON в одном вызове функции, а не связывать несколько выражений `JSON_TABLE` в операторе SQL.

Используя предложение `PLAN`, можно определить, как объединять столбцы, возвращаемые предложениями `NESTED PATH`.

имя `FOR ORDINALITY`

Добавляет столбец, обеспечивающий последовательную нумерацию строк. В таблице может быть только один столбец нумерации. Нумерация строк начинается с единицы. Для дочерних строк, сформированных предложениями `NESTED PATH`, повторяется номер родительской строки.

`AS имя_пути_json`

Необязательный параметр `имя_пути_json` служит идентификатором заданного параметра `описание_пути_json`. Имя пути должно быть уникальным и отличаться от имён столбцов. Когда применяется предложение `PLAN`, необходимо задать имена для всех путей, включая шаблон строк. Имена путей в предложении `PLAN` не могут повторяться.

`PLAN ( план_таблицы_json )`

Определяет, как соединять данные, возвращаемые предложениями `NESTED PATH`, с создаваемым представлением.

Соединять столбцы, реализуя отношения родитель/потомок, можно следующими способами:

`INNER`

Используйте `INNER JOIN`, чтобы родительская строка была исключена из вывода, если для неё не нашлось дочерних строк при объединении с данными, возвращёнными `NESTED PATH`.

`OUTER`

Используйте `LEFT OUTER JOIN`, чтобы родительская строка всегда включалась в вывод, даже если для неё не нашлось дочерних строк при объединении с данными, возвращёнными `NESTED PATH`. Если соответствующие значения отсутствуют, в дочерние столбцы будут вставлены значения `NULL`.

Соединять соседние столбцы можно следующими способами:

Сгенерировать одну строку для каждого значения отдельного соседнего столбца. Соседи данного столбца при этом заполняются значениями NULL.

CROSS

PLAN DEFAULT (OUTER / INNER [, UNION / CROSS])

Использовать `PLAN DEFAULT` проще, чем указывать полный `PLAN`, и зачастую этого достаточно для получения желаемого результата.

В этих примерах будет использоваться следующая небольшая таблица, содержащая данные JSON:

```
INSERT INTO my_films VALUES (
'{ "favorites" : [
  { "kind" : "comedy", "films" : [
    { "title" : "Bananas",
      "director" : "Woody Allen"},
    { "title" : "The Dinner Game",
      "director" : "Francis Veber" } ] },
  { "kind" : "horror", "films" : [
    { "title" : "Psycho",
      "director" : "Alfred Hitchcock" } ] },
  { "kind" : "thriller", "films" : [
    { "title" : "Vertigo",
      "director" : "Alfred Hitchcock" } ] },
  { "kind" : "drama", "films" : [
    { "title" : "Yojimbo",
      "director" : "Akira Kurosawa" } ] }
] }'):
```

```
SELECT jt.* FROM
  my_films,
  JSON_TABLE ( js, '$.favorites[*]' COLUMNS (
    id FOR ORDINALITY,
    kind text PATH '$.kind',
    NESTED PATH '$.films[*]' COLUMNS (
      title text PATH '$.title',
      director text PATH '$.director' ))) AS jt;
```

id	kind	title	director
1	comedy	Bananas	Woody Allen
1	comedy	The Dinner Game	Francis Veber
2	horror	Psycho	Alfred Hitchcock
3	thriller	Vertigo	Alfred Hitchcock
4	drama	Yojimbo	Akira Kurosawa

(5 rows)

Найдите режиссёра, который снимал фильмы в двух разных жанрах:

```
SELECT
  director1 AS director, title1, kind1, title2, kind2
FROM
  my_films,
  JSON_TABLE ( js, '$.favorites' AS favs COLUMNS (
    NESTED PATH '$[*]' AS films1 COLUMNS (
      kind1 text PATH '$.kind',
      NESTED PATH '$.films[*]' AS film1 COLUMNS (
        title1 text PATH '$.title',
        director1 text PATH '$.director')
    ),
    NESTED PATH '$[*]' AS films2 COLUMNS (
      kind2 text PATH '$.kind',
      NESTED PATH '$.films[*]' AS film2 COLUMNS (
        title2 text PATH '$.title',
        director2 text PATH '$.director'
      )
    )
  )
  PLAN (favs OUTER ((films1 INNER film1) CROSS (films2 INNER film2)))
) AS jt
WHERE kind1 > kind2 AND director1 = director2;
```

director	title1	kind1	title2	kind2
Alfred Hitchcock	Vertigo	thriller	Psycho	horror

(1 row)

9.17. Функции для работы с последовательностями

В этом разделе описаны функции для работы с объектами, представляющими *последовательности*. Такие объекты (также называемыми генераторами последовательностей или просто последовательностями) являются специальными таблицами из одной строки и создаются командой **CREATE SEQUENCE**. Используются они обычно для получения уникальных идентификаторов строк таблицы. Функции, перечисленные в [Таблице 9.55](#), предоставляют простые и безопасные для параллельного использования методы получения очередных значений таких последовательностей.

Таблица 9.55. Функции для работы с последовательностями

Функция	Описание
<code>nextval (regclass) → bigint</code>	Продвигает объект последовательности к следующему значению и возвращает это значение. Это действие атомарно: даже если вызвать <code>nextval</code> параллельно в нескольких сеансах, в каждом вызове будет гарантированно получено неповторяющееся значение последовательности. Если объект последовательности создаётся с параметрами по умолчанию, при каждом очередном вызове <code>nextval</code> будет выдаваться следующее по порядку

Функция	Описание
	значение, начиная с 1. Изменить это поведение можно, установив соответствующие параметры команды CREATE SEQUENCE. Этой функции требуется право USAGE или UPDATE для последовательности.
<code>setval (regclass, bigint [, boolean]) → bigint</code>	Устанавливает для объекта последовательности текущее значение и может также установить флаг <code>is_called</code>. В форме с двумя параметрами устанавливает для последовательности заданное значение поля <code>last_value</code> и значение <code>true</code> для флага <code>is_called</code>, показывающего, что при следующем вызове <code>nextval</code> последовательность должна сначала продвинуться к очередному значению, которое будет возвращено. При этом <code>currval</code> также возвратит заданное значение. В форме с тремя параметрами флагу <code>is_called</code> тоже можно присвоить <code>true</code> или <code>false</code>. Со значением <code>true</code> она действует так же, как и форма с двумя параметрами. Если же присвоить этому флагу значение <code>false</code>, первый вызов <code>nextval</code> после этого вернёт именно заданное значение, а продвижение последовательности произойдёт при последующем вызове <code>nextval</code>. Кроме того, значение, возвращаемое <code>currval</code> в этом случае, не меняется. Например: <pre>SELECT setval('myseq', 42); При следующем вызове nextval выдаст 43 SELECT setval('myseq', 42, true); То же самое SELECT setval('myseq', 42, false); При следующем вызове nextval выдаст 42</pre> Результатом <code>setval</code> будет просто значение её второго аргумента. Этой функции требуется право UPDATE для последовательности.
<code>currval (regclass) → bigint</code>	Возвращает значение, выданное при последнем вызове <code>nextval</code> для этой последовательности в текущем сеансе. (Если в данном сеансе <code>nextval</code> ни разу не вызывалась для данной последовательности, возвращается ошибка.) Так как это значение ограничено рамками сеанса, эта функция выдаёт предсказуемый результат вне зависимости от того, вызывалась ли впоследствии <code>nextval</code> в других сеансах или нет. Этой функции требуется право USAGE или SELECT для последовательности.
<code>lastval () → bigint</code>	Возвращает значение, выданное при последнем вызове <code>nextval</code> в текущем сеансе. Эта функция подобна <code>currval</code>, но она не принимает в параметрах имя последовательности, а обращается к той последовательности, для которой вызывалась <code>nextval</code> в последний раз в текущем сеансе. Если в текущем сеансе функция <code>nextval</code> ещё не вызывалась, при вызове <code>lastval</code> произойдёт ошибка. Этой функции требуется право USAGE или SELECT для последней использованной последовательности.

Внимание

Во избежание блокирования параллельных транзакций, пытающихся получить значения одной последовательности, значение, выданное функцией `nextval`, не высвобождается для повторного использования, если вызывающая транзакция впоследствии прерывается. Это означает, что в случае сбоев базы данных или прерывания транзакций в последовательности заданных значений могут появляться пропуски. Пропуски могут появиться и без прерывания транзакции. Например, команда `INSERT` с предложением `ON CONFLICT` вычислит кортеж, претендующий на добавление, произведя все требуемые вызовы `nextval`, прежде чем выявит конфликты, которые могут привести к отработке правил `ON CONFLICT` вместо добавления. Таким образом, объекты последовательностей Postgres Pro не годятся для получения непрерывных последовательностей.

В том же ключе изменения состояния последовательности, произведённые функцией `setval`, сразу же становятся видимыми для других транзакций и не отменяются при откате транзакции, вызвавшей эту функцию.

Если кластер базы прерывает работу до фиксации транзакции, содержащей вызов функции `nextval` или `setval`, изменение состояния последовательности может не попасть в постоянное хранилище, поэтому неизвестно, какое состояние будет иметь последовательность после перезапуска кластера: исходное или изменённое. Это не создаёт проблем при использовании последовательности внутри базы данных, так как другие изменения незафиксированных транзакций также не будут видны. Однако если значение последовательности будет сохраняться и использоваться вне базы данных, следует предварительно убедиться в том, что вызов функции `nextval` успешно зафиксирован.

Последовательность, с которой будет работать любая из вышеописанных функций, определяется аргументом типа `regclass`, который представляет собой просто OID последовательности в системном каталоге `pg_class`. Однако вам не нужно вручную вычислять OID, так как процедура ввода значения `regclass` может сделать это за вас. За подробностями обратитесь к [Разделу 8.19](#).

9.18. Условные выражения

В этом разделе описаны SQL-совместимые условные выражения, которые поддерживаются в Postgres Pro.

Подсказка

Если возможностей этих условных выражений оказывается недостаточно, вероятно, имеет смысл перейти к написанию серверных функций на более мощном языке программирования.

Примечание

Хотя конструкции `COALESCE`, `GREATEST` и `LEAST` синтаксически похожи на функции, они не являются обычными функциями, и поэтому им нельзя передать в аргументах явно описанный массив `VARIADIC`.

9.18.1. CASE

Выражение `CASE` в SQL представляет собой общее условное выражение, напоминающее операторы `if/else` в других языках программирования:

```
CASE WHEN условие THEN результат
      [WHEN ...]
      [ELSE результат]
END
```

Предложения `CASE` можно использовать везде, где допускаются выражения. Каждое *условие* в нём представляет собой выражение, возвращающее результат типа `boolean`. Если результатом выражения оказывается `true`, значением выражения `CASE` становится *результат*, следующий за условием, а остальная часть выражения `CASE` не вычисляется. Если же условие не выполняется, за ним таким же образом проверяются все последующие предложения `WHEN`. Если не выполняется ни одно из *условий* `WHEN`, значением `CASE` становится *результат*, записанный в предложении `ELSE`. Если при этом предложение `ELSE` отсутствует, результатом выражения будет `NULL`.

Пример:

```
SELECT * FROM test;

a
---
1
2
3
```

```
SELECT a,
       CASE WHEN a=1 THEN 'one'
            WHEN a=2 THEN 'two'
            ELSE 'other'
       END
FROM test;
```

```
a | case
---+-----
1 | one
2 | two
3 | other
```

Типы данных всех выражений *результатов* должны приводиться к одному выходному типу. Подробнее это описано в [Разделе 10.5](#).

Существует также «простая» форма выражения CASE, разновидность вышеприведённой общей формы:

```
CASE выражение
     WHEN значение THEN результат
     [WHEN ...]
     [ELSE результат]
END
```

В такой форме сначала вычисляется первое *выражение*, а затем его результат сравнивается с *выражениями значений* в предложениях WHEN, пока не будет найдено равное ему. Если такого значения не находится, возвращается *результат* предложения ELSE (или NULL). Эта форма больше похожа на оператор switch, существующий в языке C.

Показанный ранее пример можно записать по-другому, используя простую форму CASE:

```
SELECT a,
       CASE a WHEN 1 THEN 'one'
            WHEN 2 THEN 'two'
            ELSE 'other'
       END
FROM test;
```

```
a | case
---+-----
1 | one
2 | two
3 | other
```

В выражении CASE вычисляются только те подвыражения, которые необходимы для получения результата. Например, так можно избежать ошибки деления на ноль:

```
SELECT ... WHERE CASE WHEN x <> 0 THEN y/x > 1.5 ELSE false END;
```

Примечание

Как было описано в [Подразделе 4.2.14](#), всё же возможны ситуации, когда подвыражения вычисляются на разных этапах, так что железной гарантии, что в «CASE вычисляются только необходимые подвыражения», в принципе нет. Например, константное подвыражение 1/0 обычно вызывает ошибку деления на ноль на этапе планирования, хотя эта ветвь CASE может вовсе не вычисляться во время выполнения.

9.18.2. COALESCE

`COALESCE(значение [, ...])`

Функция `COALESCE` возвращает первый попавшийся аргумент, отличный от `NULL`. Если же все аргументы равны `NULL`, результатом тоже будет `NULL`. Это часто используется при отображении данных для подстановки некоторого значения по умолчанию вместо значений `NULL`:

```
SELECT COALESCE(description, short_description, '(none)') ...
```

Этот запрос вернёт значение `description`, если оно не равно `NULL`, либо `short_description`, если оно не `NULL`, и строку `(none)`, если оба эти значения равны `NULL`.

Аргументы должны быть приводимыми к одному общему типу, который и будет типом результата (подробнее об этом говорится в [Разделе 10.5](#)).

Как и выражение `CASE`, `COALESCE` вычисляет только те аргументы, которые необходимы для получения результата; то есть, аргументы правее первого отличного от `NULL` аргумента не вычисляются. Эта функция соответствует стандарту SQL, а в некоторых других СУБД её аналоги называются `NVL` и `IFNULL`.

9.18.3. NULLIF

`NULLIF(значение1, значение2)`

Функция `NULLIF` выдаёт значение `NULL`, если `значение1` равно `значение2`; в противном случае она возвращает `значение1`. Это может быть полезно для реализации обратной операции к `COALESCE`. В частности, для примера, показанного выше:

```
SELECT NULLIF(value, '(none)') ...
```

В данном примере если `value` равно `(none)`, выдаётся `null`, а иначе возвращается значение `value`.

Два её аргумента должны быть сравнимых типов. Если говорить точнее, они сравниваются точно так же, как сравнивались бы в записи `значение1 = значение2`, так что для этих типов должен существовать подходящий оператор `=`.

Результат будет иметь тот же тип, что и первый аргумент, но есть одна тонкость. Эта функция фактически возвращает первый аргумент подразумеваемого оператора `=`, который в некоторых случаях преобразуется к типу второго аргумента. Например, `NULLIF(1, 2.2)` возвращает `numeric`, так как оператор `integer = numeric` не существует, существует только оператор `numeric = numeric`.

9.18.4. GREATEST и LEAST

`GREATEST(значение [, ...])`

`LEAST(значение [, ...])`

Функции `GREATEST` и `LEAST` выбирают наибольшее или наименьшее значение из списка выражений. Все эти выражения должны приводиться к общему типу данных, который станет типом результата (подробнее об этом в [Разделе 10.5](#)). Значения `NULL` в этом списке игнорируются, так что результат выражения будет равен `NULL`, только если все его аргументы равны `NULL`.

Заметьте, что функции `GREATEST` и `LEAST` не описаны в стандарте SQL, но часто реализуются в СУБД как расширения. В некоторых других СУБД они могут возвращать `NULL`, когда не все, а любой из аргументов равен `NULL`.

9.19. Функции и операторы для работы с массивами

В [Таблице 9.56](#) показаны имеющиеся специальные операторы для типов-массивов. Кроме них для массивов определены обычные операторы сравнения, показанные в [Таблице 9.1](#). Эти операторы сравнения сопоставляют содержимое массивов по элементам, используя при этом функцию сравнения для B-дерева, определённую для типа данного элемента по умолчанию, и упорядочивают их по первому различию. В многомерных массивах элементы просматриваются по строкам (индекс последней размерности меняется в первую очередь). Если содержимое двух массивов совпадает,

а размерности отличаются, результат их сравнения будет определяться первым отличием в размерностях.

Таблица 9.56. Операторы для работы с массивами

Оператор	Описание	Пример(ы)
<code>anyarray @> anyarray</code>	<code>→ boolean</code> Первый массив содержит второй (имеется ли для каждого элемента второго массива равный ему в первом)? (Повторяющиеся элементы рассматриваются на общих основаниях, поэтому массивы <code>ARRAY[1]</code> и <code>ARRAY[1, 1]</code> считаются содержащими друг друга.)	<code>ARRAY[1, 4, 3] @> ARRAY[3, 1, 3] → t</code>
<code>anyarray <@ anyarray</code>	<code>→ boolean</code> Первый массив содержится во втором?	<code>ARRAY[2, 2, 7] <@ ARRAY[1, 7, 4, 2, 6] → t</code>
<code>anyarray && anyarray</code>	<code>→ boolean</code> Массивы пересекаются (у них есть общие элементы)?	<code>ARRAY[1, 4, 3] && ARRAY[2, 1] → t</code>
<code>anycompatiblearray anycompatiblearray</code>	<code>→ anycompatiblearray</code> Соединяет два массива. Если один из операндов — <code>NULL</code> или пустой массив, оператор никак не действует; в противном случае число размерностей массивов должно быть одинаковым (этот случай показан в первом примере) или могут отличаться на один (это иллюстрирует второй пример). Если это массивы не одинаковых типов элементов, они будут приведены к общему типу (см. Раздел 10.5).	<code>ARRAY[1, 2, 3] ARRAY[4, 5, 6, 7] → {1, 2, 3, 4, 5, 6, 7}</code> <code>ARRAY[1, 2, 3] ARRAY[[4, 5, 6], [7, 8, 9.9]] → {{1, 2, 3}, {4, 5, 6}, {7, 8, 9.9}}</code>
<code>anycompatible anycompatiblearray</code>	<code>→ anycompatiblearray</code> Вставляет элемент в начало массива (массив должен быть пустым или одномерным).	<code>3 ARRAY[4, 5, 6] → {3, 4, 5, 6}</code>
<code>anycompatiblearray anycompatible</code>	<code>→ anycompatiblearray</code> Вставляет элемент в конец массива (массив должен быть пустым или одномерным).	<code>ARRAY[4, 5, 6] 7 → {4, 5, 6, 7}</code>

Подробнее поведение операторов с массивами описано в [Разделе 8.15](#). За дополнительными сведениями об операторах, поддерживающих индексы, обратитесь к [Разделу 11.2](#).

В [Таблице 9.57](#) перечислены функции, предназначенные для работы с массивами. Дополнительная информация о них и примеры использования приведены в [Разделе 8.15](#).

Таблица 9.57. Функции для работы с массивами

Функция	Описание	Пример(ы)
<code>array_append (anycompatiblearray, anycompatible)</code>	<code>→ anycompatiblearray</code> Добавляет элемент в конец массива (так же, как оператор <code>anycompatiblearray anycompatible</code>).	<code>array_append(ARRAY[1, 2], 3) → {1, 2, 3}</code>
<code>array_cat (anycompatiblearray, anycompatiblearray)</code>	<code>→ anycompatiblearray</code> Соединяет два массива (так же, как оператор <code>anycompatiblearray anycompatiblearray</code>).	

Функция	Описание	Пример(ы)
		<code>array_cat (ARRAY[1,2,3], ARRAY[4,5])</code> → {1,2,3,4,5}
<code>array_dims (anyarray)</code>	→ text Возвращает текстовое представление размерностей массива.	<code>array_dims (ARRAY[[1,2,3], [4,5,6]])</code> → [1:2][1:3]
<code>array_fill (anyelement, integer[] [, integer[]])</code>	→ anyarray Возвращает массив, заполненный заданным значением и имеющий размерности, указанные во втором аргументе. В необязательном третьем аргументе могут быть заданы нижние границы для каждой размерности (по умолчанию 1).	<code>array_fill(11, ARRAY[2,3])</code> → {{11,11,11},{11,11,11}} <code>array_fill(7, ARRAY[3], ARRAY[2])</code> → [2:4]={7,7,7}
<code>array_length (anyarray, integer)</code>	→ integer Возвращает длину указанной размерности массива. (Для пустых или несуществующих размерностей массива выдаёт не 0, а NULL.)	<code>array_length(array[1,2,3], 1)</code> → 3 <code>array_length(array[]::int[], 1)</code> → NULL <code>array_length(array['text'], 2)</code> → NULL
<code>array_lower (anyarray, integer)</code>	→ integer Возвращает нижнюю границу указанной размерности массива.	<code>array_lower(' [0:2]={1,2,3}'::integer[], 1)</code> → 0
<code>array_ndims (anyarray)</code>	→ integer Возвращает число размерностей массива.	<code>array_ndims (ARRAY[[1,2,3], [4,5,6]])</code> → 2
<code>array_position (anycompatiblearray, anycompatible [, integer])</code>	→ integer Возвращает позицию первого вхождения второго аргумента в массиве либо NULL в случае отсутствия соответствующего элемента. Если задан третий аргумент, поиск начинается с заданной позиции. Массив должен быть одномерным. Эта функция определяет равенство как IS NOT DISTINCT FROM, что позволяет искать и значения NULL.	<code>array_position (ARRAY['sun', 'mon', 'tue', 'wed', 'thu', 'fri', 'sat'], 'mon')</code> → 2
<code>array_positions (anycompatiblearray, anycompatible)</code>	→ integer[] Возвращает массив позиций всех вхождений второго аргумента в одномерном массиве, заданном первым аргументом. Эта функция определяет равенство как IS NOT DISTINCT FROM, что позволяет искать и значения NULL. Результат NULL возвращается, только если в качестве массива передаётся NULL; в случае же отсутствия искомого значения в заданном массиве возвращается пустой массив.	<code>array_positions (ARRAY['A', 'A', 'B', 'A'], 'A')</code> → {1,2,4}
<code>array_prepend (anycompatible, anycompatiblearray)</code>	→ anycompatiblearray Вставляет элемент в начало массива (так же, как оператор <code>anycompatibleelement anycompatiblearray</code>).	<code>array_prepend(1, ARRAY[2,3])</code> → {1,2,3}
<code>array_remove (anycompatiblearray, anycompatible)</code>	→ anycompatiblearray Удаляет из массива все элементы, равные заданному значению. Массив должен быть одномерным. Эта функция определяет равенство как IS NOT DISTINCT FROM, что позволяет удалять и элементы NULL.	<code>array_remove (ARRAY[1,2,3,2], 2)</code> → {1,3}

Функция	Описание	Пример(ы)
<code>array_replace</code>	<code>(anycompatiblearray, anycompatible, anycompatible) → anycompatiblearray</code> Заменяет каждый элемент массива, равный второму аргументу, значением третьего аргумента.	<code>array_replace(ARRAY[1,2,5,4], 5, 3) → {1,2,3,4}</code>
<code>array_to_string</code>	<code>(array anyarray, delimiter text [, null_string text]) → text</code> Представляет все элементы массива в виде текстовых строк и объединяет эти строки через разделитель, заданный параметром <i>delimiter</i> . Если в параметре <i>null_string</i> передана отличная от NULL строка, эта строка будет представлять содержащиеся в массиве элементы NULL, в противном случае такие элементы опускаются. См. также string_to_array .	<code>array_to_string(ARRAY[1, 2, 3, NULL, 5], ',', '*') → 1,2,3,*,5</code>
<code>array_upper</code>	<code>(anyarray, integer) → integer</code> Возвращает верхнюю границу указанной размерности массива.	<code>array_upper(ARRAY[1,8,3,7], 1) → 4</code>
<code>cardinality</code>	<code>(anyarray) → integer</code> Возвращает общее число элементов в массиве (0, если массив пуст).	<code>cardinality(ARRAY[[1,2],[3,4]]) → 4</code>
<code>trim_array</code>	<code>(array anyarray, n integer) → anyarray</code> Сокращает массив, удаляя последние <i>n</i> элементов. Если массив многомерный, сокращается только первая размерность.	<code>trim_array(ARRAY[1,2,3,4,5,6], 2) → {1,2,3,4}</code>
<code>unnest</code>	<code>(anyarray) → setof anyelement</code> Разворачивает массив в набор строк. Элементы массива прочитываются в порядке хранения.	<code>unnest(ARRAY[1,2]) →</code> 1 2 <code>unnest(ARRAY[['foo','bar'],['baz','quux']]) →</code> foo bar baz quux
<code>unnest</code>	<code>(anyarray, anyarray [, ...]) → setof anyelement, anyelement [, ...]</code> Разворачивает массивы (возможно разных типов) в набор кортежей. Если массивы имеют разную длину, кортежи дополняются до большей длины значениями NULL. Эта форма допускается только в предложении FROM; см. Подраздел 7.2.1.4 .	<code>select * from unnest(ARRAY[1,2], ARRAY['foo','bar','baz']) as x(a,b) →</code> a b ---+----- 1 foo 2 bar baz

Вы также можете узнать об агрегатной функции, работающей с массивами, `array_agg` в [Разделе 9.21](#).

9.20. Диапазонные/мультидиапазонные функции и операторы

Диапазонные типы данных рассматриваются в [Разделе 8.17](#).

В [Таблице 9.58](#) показаны имеющиеся специальные операторы для диапазонных типов, а в [Таблице 9.59](#) — для мультидиапазонных типов. Кроме них для диапазонов и мультидиапазонов определены обычные операторы сравнения, показанные в [Таблице 9.1](#). Операторы сравнения сначала сравнивают нижние границы диапазонов, и только если они равны, сравнивают верхние границы. Операторы сравнения мультидиапазонов сравнивают каждый диапазон, пока не найдут неравенство. Такие операторы сравнения обычно не помогают получить полезный в целом вариант сортировки, но позволяют строить по диапазонам уникальные индексы.

Таблица 9.58. Диапазонные операторы

Оператор Описание Пример(ы)
<code>anyrange @> anyrange → boolean</code> Первый диапазон содержит второй? <code>int4range(2,4) @> int4range(2,3) → t</code>
<code>anyrange @> anyelement → boolean</code> Диапазон содержит заданный элемент? <code>'[2011-01-01,2011-03-01]':::tsrange @> '2011-01-10':::timestamp → t</code>
<code>anyrange <@ anyrange → boolean</code> Первый диапазон содержится во втором? <code>int4range(2,4) <@ int4range(1,7) → t</code>
<code>anyelement <@ anyrange → boolean</code> Заданный элемент содержится в диапазоне? <code>42 <@ int4range(1,7) → f</code>
<code>anyrange && anyrange → boolean</code> Диапазоны пересекаются (у них есть общие элементы)? <code>int8range(3,7) && int8range(4,12) → t</code>
<code>anyrange << anyrange → boolean</code> Первый диапазон располагается строго слева от второго? <code>int8range(1,10) << int8range(100,110) → t</code>
<code>anyrange >> anyrange → boolean</code> Первый диапазон располагается строго справа от второго? <code>int8range(50,60) >> int8range(20,30) → t</code>
<code>anyrange &< anyrange → boolean</code> Первый диапазон не простирается правее второго? <code>int8range(1,20) &< int8range(18,20) → t</code>
<code>anyrange &> anyrange → boolean</code> Первый диапазон не простирается левее второго? <code>int8range(7,20) &> int8range(5,10) → t</code>
<code>anyrange - - anyrange → boolean</code> Диапазоны примыкают друг к другу? <code>numrange(1.1,2.2) - - numrange(2.2,3.3) → t</code>
<code>anyrange + anyrange → anyrange</code>

Оператор	Описание	Пример(ы)
	Вычисляет объединение диапазонов. Диапазоны должны пересекаться или касаться друг друга, чтобы их объединением был один диапазон (но см. <code>range_merge()</code>).	<code>numrange(5,15) + numrange(10,20) → [5,20)</code>
<code>anyrange * anyrange → anyrange</code>	Вычисляет пересечение диапазонов.	<code>int8range(5,15) * int8range(10,20) → [10,15)</code>
<code>anyrange - anyrange → anyrange</code>	Вычисляет разность диапазонов. При этом второй диапазон, если он содержится в первом, должен располагаться так, чтобы в результате его вычитания получался один диапазон.	<code>int8range(5,15) - int8range(10,20) → [5,10)</code>

Таблица 9.59. Мультидиапазонные операторы

Оператор	Описание	Пример(ы)
<code>anymultirange @> anymultirange → boolean</code>	Первый мультидиапазон содержит второй?	<code>'{[2,4)}'::int4multirange @> '[2,3)}'::int4multirange → t</code>
<code>anymultirange @> anyrange → boolean</code>	Мультидиапазон содержит заданный диапазон?	<code>'{[2,4)}'::int4multirange @> int4range(2,3) → t</code>
<code>anymultirange @> anyelement → boolean</code>	Мультидиапазон содержит заданный элемент?	<code>'{[2011-01-01,2011-03-01)}'::tsmultirange @> '2011-01-10'::timestamp → t</code>
<code>anyrange @> anymultirange → boolean</code>	Диапазон содержит заданный мультидиапазон?	<code>'[2,4)'::int4range @> '[2,3)}'::int4multirange → t</code>
<code>anymultirange <@ anymultirange → boolean</code>	Первый мультидиапазон содержится во втором?	<code>'{[2,4)}'::int4multirange <@ '[1,7)}'::int4multirange → t</code>
<code>anymultirange <@ anyrange → boolean</code>	Мультидиапазон содержится в диапазоне?	<code>'{[2,4)}'::int4multirange <@ int4range(1,7) → t</code>
<code>anyrange <@ anymultirange → boolean</code>	Диапазон содержится в мультидиапазоне?	<code>int4range(2,4) <@ '[1,7)}'::int4multirange → t</code>
<code>anyelement <@ anymultirange → boolean</code>	Заданный элемент содержится в мультидиапазоне?	<code>4 <@ '[1,7)}'::int4multirange → t</code>
<code>anymultirange && anymultirange → boolean</code>	Мультидиапазоны пересекаются (у них есть общие элементы)?	<code>'{[3,7)}'::int8multirange && '[4,12)}'::int8multirange → t</code>
<code>anymultirange && anyrange → boolean</code>		

Оператор Описание Пример(ы)
Мультидиапазон пересекает заданный диапазон? <pre>'{[3,7)}'::int8multirange && int8range(4,12) → t</pre>
<pre>anyrange && anymultirange → boolean</pre> Диапазон пересекает заданный мультидиапазон? <pre>int8range(3,7) && '{[4,12)}'::int8multirange → t</pre>
<pre>anymultirange << anymultirange → boolean</pre> Первый мультидиапазон располагается строго слева от второго? <pre>'{[1,10)}'::int8multirange << '{[100,110)}'::int8multirange → t</pre>
<pre>anymultirange << anyrange → boolean</pre> Мультидиапазон располагается строго слева от заданного диапазона? <pre>'{[1,10)}'::int8multirange << int8range(100,110) → t</pre>
<pre>anyrange << anymultirange → boolean</pre> Диапазон располагается строго слева от заданного мультидиапазона? <pre>int8range(1,10) << '{[100,110)}'::int8multirange → t</pre>
<pre>anymultirange >> anymultirange → boolean</pre> Первый мультидиапазон располагается строго справа от второго? <pre>'{[50,60)}'::int8multirange >> '{[20,30)}'::int8multirange → t</pre>
<pre>anymultirange >> anyrange → boolean</pre> Мультидиапазон располагается строго справа от заданного диапазона? <pre>'{[50,60)}'::int8multirange >> int8range(20,30) → t</pre>
<pre>anyrange >> anymultirange → boolean</pre> Диапазон располагается строго справа от заданного мультидиапазона? <pre>int8range(50,60) >> '{[20,30)}'::int8multirange → t</pre>
<pre>anymultirange &< anymultirange → boolean</pre> Первый мультидиапазон не простирается правее второго? <pre>'{[1,20)}'::int8multirange &< '{[18,20)}'::int8multirange → t</pre>
<pre>anymultirange &< anyrange → boolean</pre> Мультидиапазон не простирается правее заданного диапазона? <pre>'{[1,20)}'::int8multirange &< int8range(18,20) → t</pre>
<pre>anyrange &< anymultirange → boolean</pre> Диапазон не простирается правее заданного мультидиапазона? <pre>int8range(1,20) &< '{[18,20)}'::int8multirange → t</pre>
<pre>anymultirange &> anymultirange → boolean</pre> Первый мультидиапазон не простирается левее второго? <pre>'{[7,20)}'::int8multirange &> '{[5,10)}'::int8multirange → t</pre>
<pre>anymultirange &> anyrange → boolean</pre> Мультидиапазон не простирается левее заданного диапазона? <pre>'{[7,20)}'::int8multirange &> int8range(5,10) → t</pre>
<pre>anyrange &> anymultirange → boolean</pre> Диапазон не простирается левее заданного мультидиапазона? <pre>int8range(7,20) &> '{[5,10)}'::int8multirange → t</pre>
<pre>anymultirange - - anymultirange → boolean</pre>

Оператор
Описание
Пример(ы)
Мультидиапазоны примыкают друг к другу? <code>'{[1.1,2.2)}'::nummultirange - - '{[2.2,3.3)}'::nummultirange → t</code>
<code>anymultirange - - anyrange → boolean</code> Мультидиапазон примыкает к заданному диапазону? <code>'{[1.1,2.2)}'::nummultirange - - numrange(2.2,3.3) → t</code>
<code>anyrange - - anymultirange → boolean</code> Диапазон примыкает к заданному мультидиапазону? <code>numrange(1.1,2.2) - - '{[2.2,3.3)}'::nummultirange → t</code>
<code>anymultirange + anymultirange → anymultirange</code> Вычисляет объединение мультидиапазонов. Мультидиапазоны могут не пересекаться и не примыкать друг к другу. <code>'{[5,10)}'::nummultirange + '{[15,20)}'::nummultirange → {[5,10), [15,20)}</code>
<code>anymultirange * anymultirange → anymultirange</code> Вычисляет пересечение мультидиапазонов. <code>'{[5,15)}'::int8multirange * '{[10,20)}'::int8multirange → {[10,15)}</code>
<code>anymultirange - anymultirange → anymultirange</code> Вычисляет разность мультидиапазонов. <code>'{[5,20)}'::int8multirange - '{[10,15)}'::int8multirange → {[5,10), [15,20)}</code>

Операторы слева/справа/примыкает всегда возвращают `false`, если один из диапазонов или мультидиапазонов пуст; то есть, считается, что пустой диапазон находится не слева и не справа от какого-либо другого диапазона.

В других случаях пустые диапазоны и мультидиапазоны рассматриваются как нулевой элемент: всё, что объединяется с пустым значением, остаётся неизменным. Все, от чего отнимается пустое значение, остаётся неизменным. Пустой мультидиапазон обладает теми же свойствами, что и пустой диапазон. Каждый диапазон содержит пустой диапазон. Каждый мультидиапазон содержит сколько угодно пустых диапазонов.

При вычислении операторов объединения и вычитания диапазонов возникнет ошибка, если в результате получатся два непересекающихся поддиапазона, поскольку такой результат нельзя представить в виде диапазона. Существуют отдельные операторы для объединения и вычитания, которые принимают мультидиапазонные параметры и возвращают мультидиапазон, и они не вызовут ошибку, даже если их аргументы не пересекаются. Поэтому, если вы хотите избежать ошибок при объединении или вычитании диапазонов, когда могут получиться непересекающиеся поддиапазоны, сначала преобразуйте входные диапазоны в мультидиапазоны.

В [Таблице 9.57](#) перечислены функции, предназначенные для работы с диапазонными типами. Функции, предназначенные для использования с мультидиапазонными типами, приведены в [Таблице 9.61](#).

Таблица 9.60. Диапазонные функции

Функция
Описание
Пример(ы)
<code>lower (anyrange) → anyelement</code> Выдаёт нижнюю границу диапазона (<code>NULL</code> , если диапазон пуст или нижняя граница не является конечной). <code>lower(numrange(1.1, 2.2)) → 1.1</code>

Функция Описание Пример(ы)
<code>upper (anyrange) → anyelement</code> Выдаёт верхнюю границу диапазона (NULL, если диапазон пуст или верхняя граница не является конечной). <code>upper(numrange(1.1, 2.2)) → 2.2</code>
<code>isempty (anyrange) → boolean</code> Диапазон пуст? <code>isempty(numrange(1.1, 2.2)) → f</code>
<code>lower_inc (anyrange) → boolean</code> Нижняя граница диапазона включается в него? <code>lower_inc(numrange(1.1, 2.2)) → t</code>
<code>upper_inc (anyrange) → boolean</code> Верхняя граница диапазона включается в него? <code>upper_inc(numrange(1.1, 2.2)) → f</code>
<code>lower_inf (anyrange) → boolean</code> Нижняя граница диапазона бесконечна? <code>lower_inf('(',') '::daterange) → t</code>
<code>upper_inf (anyrange) → boolean</code> Верхняя граница диапазона бесконечна? <code>upper_inf('(',') '::daterange) → t</code>
<code>range_merge (anyrange, anyrange) → anyrange</code> Вычисляет наименьший диапазон, включающий оба заданных диапазона. <code>range_merge('[1,2] '::int4range, '[3,4] '::int4range) → [1,4)</code>

Таблица 9.61. Мультидиапазонные функции

Функция Описание Пример(ы)
<code>lower (anymultirange) → anyelement</code> Выдаёт нижнюю границу мультидиапазона (NULL, если мультидиапазон пуст или нижняя граница не является конечной). <code>lower('{[1.1,2.2]}'::nummultirange) → 1.1</code>
<code>upper (anymultirange) → anyelement</code> Выдаёт верхнюю границу мультидиапазона (NULL, если мультидиапазон пуст или верхняя граница не является конечной). <code>upper('{[1.1,2.2]}'::nummultirange) → 2.2</code>
<code>isempty (anymultirange) → boolean</code> Мультидиапазон пуст? <code>isempty('{[1.1,2.2]}'::nummultirange) → f</code>
<code>lower_inc (anymultirange) → boolean</code> Нижняя граница мультидиапазона включается в него? <code>lower_inc('{[1.1,2.2]}'::nummultirange) → t</code>
<code>upper_inc (anymultirange) → boolean</code> Верхняя граница мультидиапазона включается в него? <code>upper_inc('{[1.1,2.2]}'::nummultirange) → f</code>

Функция Описание Пример(ы)
<code>lower_inf (anymultirange) → boolean</code> Нижняя граница мультидиапазона бесконечна? <code>lower_inf('{(,)}'::datemultirange) → t</code>
<code>upper_inf (anymultirange) → boolean</code> Верхняя граница мультидиапазона бесконечна? <code>upper_inf('{(,)}'::datemultirange) → t</code>
<code>range_merge (anymultirange) → anyrange</code> Вычисляет наименьший диапазон, включающий весь заданный мультидиапазон. <code>range_merge('{[1,2), [3,4)}'::int4multirange) → [1,4)</code>
<code>multirange (anyrange) → anymultirange</code> Возвращает мультидиапазон, содержащий только заданный диапазон. <code>multirange('[1,2)'::int4range) → {[1,2)}</code>
<code>unnest (anymultirange) → set of anyrange</code> Разворачивает мультидиапазон в набор диапазонов. Диапазоны прочитываются в порядке хранения (восходящем). <code>unnest('{[1,2), [3,4)}'::int4multirange) →</code> <code>[1,2)</code> <code>[3,4)</code>

Если функциям `lower_inc`, `upper_inc`, `lower_inf`, `upper_inf` передаётся пустой диапазон или мультидиапазон, они возвращают `false`.

9.21. Агрегатные функции

Агрегатные функции получают единственный результат из набора входных значений. Встроенные агрегатные функции общего назначения перечислены в [Таблице 9.62](#), а статистические агрегатные функции — в [Таблице 9.63](#). Встроенные внутригрупповые сортирующие агрегатные функции перечислены в [Таблице 9.64](#), встроенные внутригрупповые гипотезирующие — в [Таблице 9.65](#). Группирующие операторы, тесно связанные с агрегатными функциями, перечислены в [Таблице 9.66](#). Особенности синтаксиса агрегатных функций разъясняются в [Подразделе 4.2.7](#). За дополнительной вводной информацией обратитесь к [Разделу 2.7](#).

Агрегатные функции, поддерживающие *частичный режим*, являются кандидатами на участие в различных оптимизациях, например, в параллельном агрегировании.

Таблица 9.62. Агрегатные функции общего назначения

Функция Описание	Частичный режим
<code>array_agg (anynonarray) → anyarray</code> Собирает все входные значения, включая NULL, в массив.	Нет
<code>array_agg (anyarray) → anyarray</code> Собирает все входные массивы в массив с размерностью больше на один. (Входные массивы должны иметь одинаковую размерность и не могут быть пустыми или равны NULL).	Нет
<code>avg (smallint) → numeric</code> <code>avg (integer) → numeric</code>	Да

Функция Описание	Ча- стич- ный режим
<code>avg (bigint) → numeric</code> <code>avg (numeric) → numeric</code> <code>avg (real) → double precision</code> <code>avg (double precision) → double precision</code> <code>avg (interval) → interval</code> Вычисляет арифметическое среднее для всех входных значений, отличных от NULL.	
<code>bit_and (smallint) → smallint</code> <code>bit_and (integer) → integer</code> <code>bit_and (bigint) → bigint</code> <code>bit_and (bit) → bit</code> Вычисляет побитовое И для всех входных значений, отличных от NULL.	Да
<code>bit_or (smallint) → smallint</code> <code>bit_or (integer) → integer</code> <code>bit_or (bigint) → bigint</code> <code>bit_or (bit) → bit</code> Вычисляет побитовое ИЛИ для всех входных значений, отличных от NULL.	Да
<code>bit_xor (smallint) → smallint</code> <code>bit_xor (integer) → integer</code> <code>bit_xor (bigint) → bigint</code> <code>bit_xor (bit) → bit</code> Вычисляет побитовое исключающее ИЛИ для всех входных значений, отличных от NULL. Может быть полезна для вычисления контрольной суммы неупорядоченного набора значений.	Да
<code>bool_and (boolean) → boolean</code> Возвращает true, если все входные значения, отличные от NULL, равны true, и false в противном случае.	Да
<code>bool_or (boolean) → boolean</code> Возвращает true, если хотя бы одно входное значение, отличное от NULL, равно true, и false в противном случае.	Да
<code>count (*) → bigint</code> Выдаёт количество входных строк.	Да
<code>count ("any") → bigint</code> Выдаёт количество входных строк, в которых входное значение отлично от NULL.	Да
<code>every (boolean) → boolean</code> Это соответствующий стандарту SQL аналог <code>bool_and</code> .	Да
<code>json_agg (anyelement) → json</code> <code>jsonb_agg (anyelement) → jsonb</code> Собирает все входные значения, включая NULL, в JSON-массив. Значения преобразуются в JSON методом <code>to_json</code> или <code>to_jsonb</code> .	Нет
<code>json_agg_strict (anyelement) → json</code> <code>jsonb_agg_strict (anyelement) → jsonb</code>	Нет

Функция Описание	Ча- стич- ный режим
Собирает все входные значения, исключая NULL, в JSON-массив. Значения преобразуются в JSON методом <code>to_json</code> или <code>to_jsonb</code> .	
<code>json_object_agg (key "any", value "any") → json</code> <code>jsonb_object_agg (key "any", value "any") → jsonb</code> Собирает все пары ключ/значение в JSON-объект. Аргументы-ключи приводятся к текстовому типу, а значения преобразуются методом <code>to_json</code> или <code>to_jsonb</code> . В качестве значений, но не ключей, могут передаваться null.	Нет
<code>json_object_agg_strict (key "any", value "any") → json</code> <code>jsonb_object_agg_strict (key "any", value "any") → jsonb</code> Собирает все пары ключ/значение в JSON-объект. Аргументы-ключи приводятся к текстовому типу, а значения преобразуются методом <code>to_json</code> или <code>to_jsonb</code> . Аргумент <code>key</code> должен быть отличен от NULL. Если <code>value</code> равно NULL, запись пропускается.	Нет
<code>json_object_agg_unique (key "any", value "any") → json</code> <code>jsonb_object_agg_unique (key "any", value "any") → jsonb</code> Собирает все пары ключ/значение в JSON-объект. Аргументы-ключи приводятся к текстовому типу, а значения преобразуются методом <code>to_json</code> или <code>to_jsonb</code> . В качестве значений, но не ключей, могут передаваться null. При наличии повторяющихся ключей выдаётся ошибка.	Нет
<code>json_object_agg_unique_strict (key "any", value "any") → json</code> <code>jsonb_object_agg_unique_strict (key "any", value "any") → jsonb</code> Собирает все пары ключ/значение в JSON-объект. Аргументы-ключи приводятся к текстовому типу, а значения преобразуются методом <code>to_json</code> или <code>to_jsonb</code> . Аргумент <code>key</code> должен быть отличен от null. Если <code>value</code> равно null, запись пропускается. При наличии повторяющихся ключей выдаётся ошибка.	Нет
<code>max (см. описание) → тот же тип, что на входе</code> Вычисляет максимальное из всех значений, отличных от NULL. Имеется для всех числовых и строковых типов, типов-перечислений и даты/времени, а также типов <code>inet</code> , <code>interval</code> , <code>money</code> , <code>oid</code> , <code>pg_lsn</code> , <code>tid</code> , <code>xid8</code> и массивов с элементами этих типов.	Да
<code>min (см. описание) → тот же тип, что на входе</code> Вычисляет минимальное из всех значений, отличных от NULL. Имеется для всех числовых и строковых типов, типов-перечислений и даты/времени, а также типов <code>inet</code> , <code>interval</code> , <code>money</code> , <code>oid</code> , <code>pg_lsn</code> , <code>tid</code> , <code>xid8</code> и массивов с элементами этих типов.	Да
<code>range_agg (value anyrange) → anymultirange</code> <code>range_agg (value anymultirange) → anymultirange</code> Вычисляет объединение всех входных значений, отличных от NULL.	Нет
<code>range_intersect_agg (value anyrange) → anyrange</code> <code>range_intersect_agg (value anymultirange) → anymultirange</code> Вычисляет пересечение всех входных значений, отличных от NULL.	Нет
<code>string_agg (value text, delimiter text) → text</code> <code>string_agg (value bytea, delimiter bytea) → bytea</code>	Нет

Функция Описание	Частичный режим
Соединяет все входные значения, отличные от NULL, в строку. Перед каждым значением, кроме первого, добавляется соответствующий разделитель, заданный параметром <i>delimiter</i> (если он отличен от NULL).	
<code>sum (smallint) → bigint</code> <code>sum (integer) → bigint</code> <code>sum (bigint) → numeric</code> <code>sum (numeric) → numeric</code> <code>sum (real) → real</code> <code>sum (double precision) → double precision</code> <code>sum (interval) → interval</code> <code>sum (money) → money</code> Вычисляет сумму всех входных значений, отличных от NULL.	Да
<code>xmllagg (xml) → xml</code> Соединяет вместе входные XML-значения, отличные от NULL (см. также Подраздел 9.15.1.7).	Нет

Следует заметить, что за исключением `count`, все эти функции возвращают NULL, если для них не была выбрана ни одна строка. В частности, функция `sum`, не получив строк, возвращает NULL, а не 0, как можно было бы ожидать, и `array_agg` в этом случае возвращает NULL, а не пустой массив. Если необходимо, подставить в результат 0 или пустой массив вместо NULL можно с помощью функции `coalesce`.

Агрегатные функции `array_agg`, `json_agg`, `jsonb_agg`, `json_agg_strict`, `jsonb_agg_strict`, `json_object_agg`, `jsonb_object_agg`, `json_object_agg_strict`, `jsonb_object_agg_strict`, `json_object_agg_unique`, `jsonb_object_agg_unique`, `json_object_agg_unique_strict`, `jsonb_object_agg_unique_strict`, `string_agg` и `xmllagg` так же, как и подобные пользовательские агрегатные функции, выдают разные по содержанию результаты в зависимости от порядка входных значений. По умолчанию порядок не определён, но его можно задать, дополнив вызов агрегатной функции предложением `ORDER BY`, как описано в [Подразделе 4.2.7](#). Обычно нужного результата также можно добиться, передав для агрегирования результат подзапроса с сортировкой. Например:

```
SELECT xmllagg(x) FROM (SELECT x FROM test ORDER BY y DESC) AS tab;
```

Но учтите, что этот подход может не работать, если на внешнем уровне запроса выполняется дополнительная обработка, например соединение, так как при этом результат подзапроса может быть переупорядочен перед вычислением агрегатной функции.

Примечание

Помимо агрегатных функций JSON, показанных здесь, см. функции-конструкторы `json_objectagg` и `json_arrayagg` в [Подразделе 9.16.3](#).

Примечание

Логические агрегатные функции `bool_and` и `bool_or` равнозначны описанным в стандарте SQL агрегатам `every` и `any` или `some`. Postgres Pro поддерживает `every`, но не `any/some`, так как синтаксис, описанный в стандарте, допускает неоднозначность:

```
SELECT b1 = ANY((SELECT b2 FROM t2 ...)) FROM t1 ...;
```

Здесь ANY можно рассматривать и как объявление подзапроса, и как агрегатную функцию, если этот подзапрос возвращает одну строку с булевым значением. Таким образом, этим агрегатным функциям нельзя было дать стандартные имена.

Примечание

Пользователи с опытом использования других СУБД SQL могут быть недовольны скоростью агрегатной функции `count`, когда она применяется ко всей таблице. Подобный запрос:

```
SELECT count(*) FROM sometable;
```

потребуется затрат в количестве, пропорциональном размеру таблицы: Postgres Pro придётся полностью просканировать либо всю таблицу, либо один из индексов, включающий все её строки.

В Таблице 9.63 перечислены агрегатные функции, обычно применяемые в статистическом анализе. (Они выделены просто для того, чтобы не загромождать список наиболее популярных агрегатных функций.) Функции, показанные как принимающие *числовой_тип*, существуют для типов `smallint`, `integer`, `bigint`, `numeric`, `real`, и `double precision`. В их описании под *N* подразумевается число входных строк, для которых все входные выражения отличны от NULL. Во всех случаях, когда вычисление не имеет смысла, например, когда *N* равно нулю, возвращается `null`.

Таблица 9.63. Агрегатные функции для статистических вычислений

Функция Описание	Ча- стич- ный режим
<code>corr (Y double precision, X double precision) → double precision</code> Вычисляет коэффициент корреляции.	Да
<code>covar_pop (Y double precision, X double precision) → double precision</code> Вычисляет ковариацию совокупности.	Да
<code>covar_samp (Y double precision, X double precision) → double precision</code> Вычисляет ковариацию выборки.	Да
<code>regr_avgx (Y double precision, X double precision) → double precision</code> Вычисляет среднее для независимой переменной, $\text{sum}(X)/N$.	Да
<code>regr_avgy (Y double precision, X double precision) → double precision</code> Вычисляет среднее для зависимой переменной, $\text{sum}(Y)/N$.	Да
<code>regr_count (Y double precision, X double precision) → bigint</code> Вычисляет число входных строк, в которых оба выражения отличны от NULL.	Да
<code>regr_intercept (Y double precision, X double precision) → double precision</code> Вычисляет пересечение с осью ОУ линии, полученной методом наименьших квадратов по парам (X, Y).	Да
<code>regr_r2 (Y double precision, X double precision) → double precision</code> Вычисляет квадрат коэффициента корреляции.	Да
<code>regr_slope (Y double precision, X double precision) → double precision</code> Вычисляет наклон линии, полученной методом наименьших квадратов по парам (X, Y).	Да
<code>regr_sxx (Y double precision, X double precision) → double precision</code> Вычисляет «сумму квадратов» независимой переменной, $\text{sum}(X^2) - \text{sum}(X)^2/N$.	Да

Функция Описание	Частичный режим
<code>regr_sxy (Y double precision, X double precision) → double precision</code> Вычисляет «сумму произведений» независимой и зависимой переменных, $\text{sum}(X*Y) - \text{sum}(X) * \text{sum}(Y)/N$.	Да
<code>regr_syy (Y double precision, X double precision) → double precision</code> Вычисляет «сумму квадратов» зависимой переменной, $\text{sum}(Y^2) - \text{sum}(Y)^2/N$.	Да
<code>stddev (числовой_тип) → double precision</code> для real или double precision, иначе numeric Сохранившийся синоним <code>stddev_samp</code> .	Да
<code>stddev_pop (numeric_type) → double precision</code> для real или double precision, иначе numeric Вычисляет стандартное отклонение по генеральной совокупности входных значений.	Да
<code>stddev_samp (числовой_тип) → double precision</code> для real или double precision, иначе numeric Вычисляет стандартное отклонение по выборке входных значений.	Да
<code>variance (numeric_type) → double precision</code> для real или double precision, иначе numeric Сохранившийся синоним <code>var_samp</code> .	Да
<code>var_pop (numeric_type) → double precision</code> для real или double precision, иначе numeric Вычисляет дисперсию для генеральной совокупности входных значений (квадрат стандартного отклонения).	Да
<code>var_samp (числовой_тип) → double precision</code> для real или double precision, иначе numeric Вычисляет дисперсию по выборке для входных значений (квадрат отклонения по выборке).	Да

В Таблица 9.64 показаны некоторые агрегатные функции, использующие синтаксис *сортирующих агрегатных функций*. Эти функции иногда называются функциями «обратного распределения». Их агрегированные входные данные формируются указанием `ORDER BY`, а кроме того они могут принимать не агрегируемый *непосредственный аргумент*, который вычисляется только один раз. Все эти функции игнорируют значения NULL в агрегируемых данных. Для тех функций, которые принимают параметр *fraction*, его значение должно быть между 0 и 1; в противном случае выдаётся ошибка. Однако если значение *fraction* — NULL, они выдают NULL в результате.

Таблица 9.64. Сортирующие агрегатные функции

Функция Описание	Частичный режим
<code>mode () WITHIN GROUP (ORDER BY anyelement) → anyelement</code> Вычисляет <i>моду</i> — наиболее часто встречающееся в агрегируемом аргументе значение (если одинаково часто встречаются несколько значений, произвольно выбирается первое из них). Агрегируемый аргумент должен быть сортируемого типа.	Нет

Функция Описание	Частичный режим
<pre>percentile_cont (fraction double precision) WITHIN GROUP (ORDER BY double precision) → double precision</pre> <pre>percentile_cont (fraction double precision) WITHIN GROUP (ORDER BY interval) → interval</pre> Вычисляет <i>непрерывный процентиль</i> — значение, соответствующее дроби, заданной параметром <i>fraction</i>, в отсортированном множестве значений агрегатного аргумента. При этом в случае необходимости соседние входные значения будут интерполироваться.	Нет
<pre>percentile_cont (fractions double precision[]) WITHIN GROUP (ORDER BY double precision) → double precision[]</pre> <pre>percentile_cont (fractions double precision[]) WITHIN GROUP (ORDER BY interval) → interval[]</pre> Вычисляет множественные непрерывные процентиля. Возвращает массив той же размерности, что имеет параметр <i>fractions</i>, в котором каждый отличный от NULL элемент заменяется соответствующим данному перцентилю значением (возможно интерполированным).	Нет
<pre>percentile_disc (fraction double precision) WITHIN GROUP (ORDER BY anyelement) → anyelement</pre> Вычисляет <i>дискретный процентиль</i> — первое значение в отсортированном множестве значений агрегатного аргумента, позиция которого в этом множестве равна или больше значения <i>fraction</i>. Агрегируемый аргумент должен быть сортируемого типа.	Нет
<pre>percentile_disc (fractions double precision[]) WITHIN GROUP (ORDER BY anyelement) → anyarray</pre> Вычисляет множественные дискретные процентиля. Возвращает массив той же размерности, что имеет параметр <i>fractions</i>, в котором каждый отличный от NULL элемент заменяется соответствующим данному перцентилю значением. Агрегируемый аргумент должен быть сортируемого типа.	Нет

Все «гипотезирующие» агрегатные функции, перечисленные в [Таблице 9.65](#), связаны с одноимёнными оконными функциями, определёнными в [Разделе 9.22](#). В каждом случае их результат — значение, которое бы вернула связанная оконная функция для «гипотетической» строки, полученной из *аргументов*, если бы такая строка была добавлена в сортированную группу строк, которую образуют *сортирующие_аргументы*. Для всех этих функций список непосредственных аргументов, переданный в качестве *аргументов*, по числу и типу элементов должен соответствовать списку, передаваемому в качестве *сортирующих_аргументов*. В отличие от большинства встроенных агрегатов, данные агрегаты не являются строгими, то есть они не игнорируют строки, содержащие NULL. Значения NULL сортируются по правилу, заданному в предложении `ORDER BY`.

Таблица 9.65. Гипотезирующие агрегатные функции

Функция Описание	Частичный режим
<pre>rank (аргументы) WITHIN GROUP (ORDER BY сортирующие_аргументы) → bigint</pre> Вычисляет ранг гипотетической строки с пропусками, то есть номер первой родственной ей строки.	Нет
<pre>dense_rank (args) WITHIN GROUP (ORDER BY сортируемые_аргументы) → bigint</pre>	Нет

Функция Описание	Частичный режим
Вычисляет ранг гипотетической строки без пропусков; по сути эта функция считает группы родственных строк.	
percent_rank (аргументы) WITHIN GROUP (ORDER BY сортируемые_аргументы) → double precision Вычисляет относительный ранг гипотетической строки, то есть (rank - 1) / (общее число строк - 1). Таким образом, результат лежит в интервале от 0 до 1, включая границы.	Нет
cume_dist (аргументы) WITHIN GROUP (ORDER BY сортируемые_аргументы) → double precision Вычисляет кумулятивное распределение, то есть (число строк, предшествующих или родственных гипотетической строке) / (общее число строк). Результат лежит в интервале от 1/N до 1.	Нет

Таблица 9.66. Операции группировки

Функция Описание
GROUPING (выражения_group_by) → integer Выдаёт битовую маску, показывающую, какие выражения GROUP BY не вошли в текущий набор группирования. Биты назначаются справа налево, то есть самому правому аргументу соответствует самый младший бит; бит равен 0, если соответствующее выражение вошло в критерий группировки набора группирования, для которого сформирована текущая строка результата, или 1 в противном случае.

Операции группировки, показанные в [Таблице 9.66](#), применяются в сочетании с наборами группирования (см. [Подраздел 7.2.4](#)) для различения результирующих строк. Аргументы функции GROUPING на самом деле не вычисляются, но они должны в точности соответствовать выражениям, заданным в предложении GROUP BY на их уровне запроса. Например:

```
=> SELECT * FROM items_sold;
```

```
make | model | sales
-----+-----+-----
Foo   | GT    | 10
Foo   | Tour  | 20
Bar   | City  | 15
Bar   | Sport | 5
(4 rows)
```

```
=> SELECT make, model, GROUPING(make,model), sum(sales) FROM items_sold GROUP BY
ROLLUP (make,model);
```

```
make | model | grouping | sum
-----+-----+-----+-----
Foo   | GT    |          0 | 10
Foo   | Tour  |          0 | 20
Bar   | City  |          0 | 15
Bar   | Sport |          0 | 5
Foo   |       |          1 | 30
Bar   |       |          1 | 20
      |       |          3 | 50
(7 rows)
```

Здесь значение grouping, равное 0 в первых четырёх строках, показывает, что эти строки были сгруппированы обычным образом по обоим группирующим столбцам. Значение 1 в двух предпоследних строках показывает, что столбец model не был группирующим, а значение 3 в последней

строке показывает, что при группировании не учитывались ни `make`, ни `model`, то есть агрегирование выполнялось по всем входным строкам.

9.22. Оконные функции

Оконные функции дают возможность выполнять вычисления с набором строк, каким-либо образом связанным с текущей строкой запроса. Вводную информацию об этом можно получить в [Разделе 3.5](#), а подробнее узнать о синтаксисе можно в [Подразделе 4.2.8](#).

Встроенные оконные функции перечислены в [Таблице 9.67](#). Заметьте, что эти функции *должны* вызываться именно как оконные, т. е. при вызове необходимо использовать предложение `OVER`.

В дополнение к этим функциям в качестве оконных можно использовать любые встроенные или обычные пользовательские агрегатные функции (но не сортирующие и не гипотезирующие); список встроенных агрегатных функций приведён в [Разделе 9.21](#). Агрегатные функции работают как оконные, только когда за их вызовом следует предложение `OVER`; в противном случае они работают как обычные функции и выдают для всего набора единственную строку.

Таблица 9.67. Оконные функции общего назначения

Функция	Описание
<code>row_number () → bigint</code>	Возвращает номер текущей строки в её разделе, начиная с 1.
<code>rank () → bigint</code>	Возвращает ранг текущей строки с пропусками; то же, что и <code>row_number</code> для первой родственной ей строки.
<code>dense_rank () → bigint</code>	Возвращает ранг текущей строки без пропусков; по сути эта функция считает группы родственных строк.
<code>percent_rank () → double precision</code>	Вычисляет относительный ранг текущей строки, то есть $(rank - 1) / (\text{общее число строк раздела} - 1)$. Таким образом, результат лежит в интервале от 0 до 1, включительно.
<code>cume_dist () → double precision</code>	Возвращает кумулятивное распределение, то есть $(\text{число строк раздела, предшествующих или родственных текущей строке}) / (\text{общее число строк раздела})$. Таким образом, результат лежит в интервале от $1/N$ до 1.
<code>ntile (num_buckets integer) → integer</code>	Возвращает целое от 1 до значения аргумента для разбиения раздела на части максимально близких размеров.
<code>lag (value anycompatible [, offset integer [, default anycompatible]]) → anycompatible</code>	Возвращает значение <code>value</code> , вычисленное для строки, сдвинутой на <code>offset</code> строк от текущей к началу раздела; если такой строки нет, возвращается значение <code>default</code> (оно должно быть совместимого с <code>value</code> типа). Оба аргумента, <code>offset</code> и <code>default</code> , вычисляются для текущей строки. Если они не указываются, <code>offset</code> считается равным 1, а <code>default</code> — <code>NULL</code> .
<code>lead (value anycompatible [, offset integer [, default anycompatible]]) → anycompatible</code>	Возвращает значение <code>value</code> , вычисленное для строки, сдвинутой на <code>offset</code> строк от текущей к концу раздела; если такой строки нет, возвращается значение <code>default</code> (оно должно быть совместимого с <code>value</code> типа). Оба аргумента, <code>offset</code> и <code>default</code> , вычисляются для текущей строки. Если они не указываются, <code>offset</code> считается равным 1, а <code>default</code> — <code>NULL</code> .
<code>first_value (value anyelement) → anyelement</code>	

Функция	Описание
	Возвращает значение (<i>value</i>), вычисленное для первой строки в рамке окна.
<code>last_value (value anyelement) → anyelement</code>	Возвращает значение (<i>value</i>), вычисленное для последней строки в рамке окна.
<code>nth_value (value anyelement, n integer) → anyelement</code>	Возвращает значение (<i>value</i>), вычисленное в <i>n</i> -й строке в рамке окна (считая с 1), или NULL, если такой строки нет.

Результат всех функций, перечисленных в [Таблице 9.67](#), зависит от порядка сортировки, заданного предложением `ORDER BY` в определении соответствующего окна. Строки, которые являются одинаковыми при рассмотрении только столбцов `ORDER BY`, считаются *родственными*. Четыре функции, вычисляющие ранг (включая `cume_dist`), реализованы так, что их результат будет одинаковым для всех родственных строк.

Заметьте, что функции `first_value`, `last_value` и `nth_value` рассматривают только строки в «рамке окна», которая по умолчанию содержит строки от начала раздела до последней родственной строки для текущей. Поэтому результаты `last_value` и иногда `nth_value` могут быть не очень полезны. В таких случаях можно переопределить рамку, добавив в предложение `OVER` подходящее указание рамки (`RANGE`, `ROWS` или `GROUPS`). Подробнее эти указания описаны в [Подразделе 4.2.8](#).

Когда в качестве оконной функции используется агрегатная, она обрабатывает строки в рамке текущей строки. Агрегатная функция с `ORDER BY` и определением рамки окна по умолчанию будет вычисляться как «бегущая сумма», что может не соответствовать желаемому результату. Чтобы агрегатная функция работала со всем разделом, следует опустить `ORDER BY` или использовать `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`. Используя другие указания в определении рамки, можно получить и другие эффекты.

Примечание

В стандарте SQL определены параметры `RESPECT NULLS` или `IGNORE NULLS` для функций `lead`, `lag`, `first_value`, `last_value` и `nth_value`. В Postgres Pro такие параметры не реализованы: эти функции ведут себя так, как положено в стандарте по умолчанию (или с подразумеваемым параметром `RESPECT NULLS`). Также функция `nth_value` не поддерживает предусмотренные стандартом параметры `FROM FIRST` и `FROM LAST`: реализовано только поведение по умолчанию (с подразумеваемым параметром `FROM FIRST`). (Получить эффект параметра `FROM LAST` можно, изменив порядок `ORDER BY` на обратный.)

9.23. Выражения подзапросов

В этом разделе описаны выражения подзапросов, которые реализованы в Postgres Pro в соответствии со стандартом SQL. Все рассмотренные здесь формы выражений возвращает булевы значения (`true/false`).

9.23.1. EXISTS

`EXISTS (подзапрос)`

Аргументом `EXISTS` является обычный оператор `SELECT`, т. е. *подзапрос*. Выполнив запрос, система проверяет, возвращает ли он строки в результате. Если он возвращает минимум одну строку, результатом `EXISTS` будет «`true`», а если не возвращает ни одной — «`false`».

Подзапрос может обращаться к переменным внешнего запроса, которые в рамках одного вычисления подзапроса считаются константами.

Вообще говоря, подзапрос может выполняться не полностью, а завершаться, как только будет возвращена хотя бы одна строка. Поэтому в подзапросах следует избегать побочных эффектов (на-

пример, обращений к генераторам последовательностей); проявление побочного эффекта может быть непредсказуемым.

Так как результат этого выражения зависит только от того, возвращаются строки или нет, но не от их содержимого, список выходных значений подзапроса обычно не имеет значения. Как следствие, широко распространена практика, когда проверки `EXISTS` записываются в форме `EXISTS (SELECT 1 WHERE ...)`. Однако из этого правила есть и исключения, например с подзапросами с предложением `INTERSECT`.

Этот простой пример похож на внутреннее соединение по столбцу `col2`, но он выдаёт максимум одну строку для каждой строки в `tab1`, даже если в `tab2` ей соответствуют несколько строк:

```
SELECT col1
FROM tab1
WHERE EXISTS (SELECT 1 FROM tab2 WHERE col2 = tab1.col2);
```

9.23.2. IN

выражение IN (подзапрос)

В правой стороне этого выражения в скобках задаётся подзапрос, который должен возвращать ровно один столбец. Вычисленное значение левого выражения сравнивается со значениями во всех строках, возвращённых подзапросом. Результатом всего выражения `IN` будет «true», если строка с таким значением находится, и «false» в противном случае (в том числе, когда подзапрос вообще не возвращает строк).

Заметьте, что если результатом выражения слева оказывается `NULL` или равных значений справа не находится, а хотя бы одно из значений справа равно `NULL`, конструкция `IN` возвращает `NULL`, а не `false`. Это соответствует принятым в SQL правилам сравнения переменных со значениями `NULL`.

Так же, как и с `EXISTS`, здесь не следует рассчитывать на то, что подзапрос будет всегда выполняться полностью.

конструктор_строки IN (подзапрос)

В левой части этой формы `IN` записывается конструктор строки (подробнее они рассматриваются в [Подразделе 4.2.13](#)). Справа в скобках записывается подзапрос, который должен вернуть ровно столько столбцов, сколько содержит строка в выражении слева. Вычисленные значения левого выражения сравниваются построчно со значениями во всех строках, возвращённых подзапросом. Результатом всего выражения `IN` будет «true», если строка с такими значениями находится, и «false» в противном случае (в том числе, когда подзапрос вообще не возвращает строк).

Как обычно, значения `NULL` в строках обрабатываются при этом по принятым в SQL правилам сравнения. Две строки считаются равными, если все их соответствующие элементы не равны `NULL`, но равны между собой; неравными они считаются, когда в них находятся элементы, не равные `NULL`, и не равные друг другу; в противном случае результат сравнения строк не определён (равен `NULL`). Если в результатах сравнения строк нет ни одного положительного, но есть хотя бы один `NULL`, результатом `IN` будет `NULL`.

9.23.3. NOT IN

выражение NOT IN (подзапрос)

Справа в скобках записывается подзапрос, который должен возвращать ровно один столбец. Вычисленное значение левого выражения сравнивается со значением во всех строках, возвращённых подзапросом. Результатом всего выражения `NOT IN` будет «true», если находятся только несовпадающие строки (в том числе, когда подзапрос вообще не возвращает строк). Если же находится хотя бы одна подходящая строка, результатом будет «false».

Заметьте, что если результатом выражения слева оказывается `NULL` или равных значений справа не находится, а хотя бы одно из значений справа равно `NULL`, конструкция `NOT IN` возвращает `NULL`, а не `true`. Это соответствует принятым в SQL правилам сравнения переменных со значениями `NULL`.

Так же, как и с `EXISTS`, здесь не следует рассчитывать на то, что подзапрос будет всегда выполняться полностью.

конструктор_строки NOT IN (подзапрос)

В левой части этой формы `NOT IN` записывается конструктор строки (подробнее они описываются в [Подразделе 4.2.13](#)). Справа в скобках записывается подзапрос, который должен вернуть ровно столько столбцов, сколько содержит строка в выражении слева. Вычисленные значения левого выражения сравниваются построчно со значениями во всех строках, возвращённых подзапросом. Результатом всего выражения `NOT IN` будет «true», если равных строк не найдётся (в том числе, и когда подзапрос не возвращает строк), и «false», если такие строки есть.

Как обычно, значения `NULL` в строках обрабатываются при этом по принятым в SQL правилам сравнения. Две строки считаются равными, если все их соответствующие элементы не равны `NULL`, но равны между собой; неравными они считаются, когда в них находятся элементы, не равные `NULL`, и не равные друг другу; в противном случае результат сравнения строк не определён (равен `NULL`). Если в результатах сравнения строк нет ни одного положительного, но есть хотя бы один `NULL`, результатом `NOT IN` будет `NULL`.

9.23.4. ANY/SOME

выражение оператор ANY (подзапрос)

выражение оператор SOME (подзапрос)

В правой части конструкции в скобках записывается подзапрос, который должен возвращать ровно один столбец. Вычисленное значение левого выражения сравнивается со значением в каждой строке результата подзапроса с помощью заданного *оператора* условия, который должен выдавать логическое значение. Результатом `ANY` будет «true», если хотя бы для одной строки условие истинно, и «false» в противном случае (в том числе, и когда подзапрос не возвращает строк).

Ключевое слово `SOME` является синонимом `ANY`. Конструкцию `IN` можно также записать как `= ANY`.

Заметьте, что если условие не выполняется ни для одной из строк, а хотя бы для одной строки условный оператор выдаёт `NULL`, конструкция `ANY` возвращает `NULL`, а не `false`. Это соответствует принятым в SQL правилам сравнения переменных со значениями `NULL`.

Так же, как и с `EXISTS`, здесь не следует рассчитывать на то, что подзапрос будет всегда выполняться полностью.

конструктор_строки оператор ANY (подзапрос)

конструктор_строки оператор SOME (подзапрос)

В левой части этой формы `ANY` записывается конструктор строки (подробнее они описываются в [Подразделе 4.2.13](#)). Справа в скобках записывается подзапрос, который должен возвращать ровно столько столбцов, сколько содержит строка в выражении слева. Вычисленные значения левого выражения сравниваются построчно со значениями во всех строках, возвращённых подзапросом, с применением заданного *оператора*. Результатом всего выражения `ANY` будет «true», если для какой-либо из строк подзапроса результатом сравнения будет `true`, или «false», если для всех строк результатом сравнения оказывается `false` (в том числе, и когда подзапрос не возвращает строк). Результатом выражения будет `NULL`, если ни для одной из строк подзапроса результат сравнения не равен `true`, а минимум для одной равен `NULL`.

Подробнее логика сравнения конструкторов строк описана в [Подразделе 9.24.5](#).

9.23.5. ALL

выражение оператор ALL (подзапрос)

В правой части конструкции в скобках записывается подзапрос, который должен возвращать ровно один столбец. Вычисленное значение левого выражения сравнивается со значением в каждой строке результата подзапроса с помощью заданного *оператора* условия, который должен выдавать логическое значение. Результатом `ALL` будет «true», если условие истинно для всех строк (и когда подзапрос не возвращает строк), или «false», если находятся строки, для которых оно ложно. Ре-

результатом выражения будет NULL, если ни для одной из строк подзапроса результат сравнения не равен true, а минимум для одной равен NULL.

Конструкция `NOT IN` равнозначна `<> ALL`.

Так же, как и с `EXISTS`, здесь не следует рассчитывать на то, что подзапрос будет всегда выполняться полностью.

конструктор_строки оператор ALL (подзапрос)

В левой части этой формы `ALL` записывается конструктор строки (подробнее они описываются в [Подразделе 4.2.13](#)). Справа в скобках записывается подзапрос, который должен возвращать ровно столько столбцов, сколько содержит строка в выражении слева. Вычисленные значения левого выражения сравниваются построчно со значениями во всех строках, возвращённых подзапросом, с применением заданного *оператора*. Результатом всего выражения `ALL` будет «true», если для всех строк подзапроса результатом сравнения будет true (или если подзапрос не возвращает строк), либо «false», если результат сравнения равен false для любой из строк подзапроса. Результатом выражения будет NULL, если ни для одной из строк подзапроса результат сравнения не равен true, а минимум для одной равен NULL.

Подробнее логика сравнения конструкторов строк описана в [Подразделе 9.24.5](#).

9.23.6. Сравнение единичных строк

конструктор_строки оператор (подзапрос)

В левой части конструкции записывается конструктор строки (подробнее они описываются в [Подразделе 4.2.13](#)). Справа в скобках записывается подзапрос, который должен возвращать ровно столько столбцов, сколько содержит строка в выражении слева. Более того, подзапрос может вернуть максимум одну строку. (Если он не вернёт строку, результатом будет NULL.) Конструкция возвращает результат сравнения строки слева с этой одной строкой результата подзапроса.

Подробнее логика сравнения конструкторов строк описана в [Подразделе 9.24.5](#).

9.24. Сравнение табличных строк и массивов

В этом разделе описываются несколько специальных конструкций, позволяющих сравнивать группы значений. Синтаксис этих конструкций связан с формами выражений с подзапросами, описанными в предыдущем разделе, а отличаются они отсутствием подзапросов. Конструкции, в которых в качестве подвыражений используются массивы, являются расширениями Postgres Pro; все остальные формы соответствуют стандарту SQL. Все описанные здесь выражения возвращают логические значения (true/false).

9.24.1. IN

выражение IN (значение [, ...])

Справа в скобках записывается список выражений. Результатом будет «true», если значение левого выражения равняется одному из значений выражений в правой части. Эту конструкцию можно считать краткой записью условия

```
выражение = значение1
OR
выражение = значение2
OR
...
```

Заметьте, что если результатом выражения слева оказывается NULL или равных значений справа не находится, а хотя бы одно из значений справа равно NULL, конструкция `IN` возвращает NULL, а не false. Это соответствует принятым в SQL правилам сравнения переменных со значениями NULL.

9.24.2. NOT IN

выражение NOT IN (значение [, ...])

Справа в скобках записывается список выражений. Результатом будет «true», если значение левого выражения не равно ни одному из значений выражений в правой части. Эту конструкцию можно считать краткой записью условия

```
выражение <> значение1
AND
выражение <> значение2
AND
...
```

Заметьте, что если результатом выражения слева оказывается NULL или равных значений справа не находится, а хотя бы одно из значений справа равно NULL, конструкция `NOT IN` возвращает NULL, а не true, как можно было бы наивно полагать. Это соответствует принятым в SQL правилам сравнения переменных со значениями NULL.

Подсказка

Выражения `x NOT IN y` и `NOT (x IN y)` полностью равнозначны. Учитывая, что значения NULL могут ввести в заблуждение начинающих скорее в конструкции `NOT IN`, чем в `IN`, лучше формулировать условия так, чтобы в них было как можно меньше отрицаний.

9.24.3. ANY/SOME (с массивом)

```
выражение оператор ANY (выражение массива)
выражение оператор SOME (выражение массива)
```

Справа в скобках записывается выражение, результатом которого является массив. Вычисленное значение левого выражения сравнивается с каждым элементом этого массива с применением заданного оператора условия, который должен выдавать логическое значение. Результатом `ANY` будет «true», если для какого-либо элемента условие истинно, и «false» в противном случае (в том числе, и когда массив оказывается пустым).

Если значением массива оказывается NULL, результатом `ANY` также будет NULL. Если NULL получен в левой части, результатом `ANY` обычно тоже будет NULL (хотя оператор нестрогого сравнения может выдать другой результат). Кроме того, если массив в правой части содержит элементы NULL и ни с одним из элементов условие не выполняется, результатом `ANY` будет NULL, а не false (опять же, если используется оператор строгого сравнения). Это соответствует принятым в SQL правилам сравнения переменных со значениями NULL.

Ключевое слово `SOME` является синонимом `ANY`.

9.24.4. ALL (с массивом)

```
выражение оператор ALL (выражение массива)
```

Справа в скобках записывается выражение, результатом которого является массив. Вычисленное значение левого выражения сравнивается с каждым элементом этого массива с применением заданного оператора условия, который должен выдавать логическое значение. Результатом `ALL` будет «true», если для всех элементов условие истинно (или массив не содержит элементов), и «false», если находятся строки, для которых оно ложно.

Если значением массива оказывается NULL, результатом `ALL` также будет NULL. Если NULL получен в левой части, результатом `ALL` обычно тоже будет NULL (хотя оператор нестрогого сравнения может выдать другой результат). Кроме того, если массив в правой части содержит элементы NULL и при этом нет элементов, с которыми условие не выполняется, результатом `ALL` будет NULL, а не true (опять же, если используется оператор строгого сравнения). Это соответствует принятым в SQL правилам сравнения переменных со значениями NULL.

9.24.5. Сравнение конструкторов строк

```
конструктор_строки оператор конструктор_строки
```

С обеих сторон представлены конструкторы строк (они описываются в [Подразделе 4.2.13](#)). При этом данные конструкторы должны содержать одинаковое число полей. Заданный *оператор* применяется к каждой паре соответствующих полей. (Поскольку поля могут быть разных типов, для каждой пары может быть выбран отдельный конкретный оператор.) Все выбранные операторы должны быть членами некоторого класса операторов В-дерева или быть обратными члену класса операторов В-дерева =. Сравнения конструкторов строк возможны с оператором =, <>, <, <=, > или >=, или имеющим аналогичную семантику.

Сравнения = и <> несколько отличаются от других. С этими операторами две строки считаются равными, если все их соответствующие поля не равны NULL и равны между собой, и неравными, если какие-либо соответствующие их поля не NULL и не равны между собой. В противном случае результатом сравнения будет неопределённость (NULL).

С операторами <, <=, > и >= элементы строк сравниваются слева направо до тех пор, пока не будет найдена пара неравных элементов или значений NULL. Если любым из элементов пары оказывается NULL, результатом сравнения будет неопределённость (NULL), в противном случае результат всего выражения определяется результатом сравнения этих двух элементов. Например, результатом ROW(1,2,NULL) < ROW(1,3,0) будет true, а не NULL, так как третья пара элементов не принимается в рассмотрение.

Примечание

До версии 8.2 PostgreSQL обрабатывал условия <, <=, > и >= не так, как это описано в стандарте SQL. Сравнение ROW(a,b) < ROW(c,d) выполнялось как a < c AND b < d, тогда как по стандарту должно быть a < c OR (a = c AND b < d).

конструктор_строки IS DISTINCT FROM конструктор_строки

Эта конструкция похожа на сравнение строк с оператором <>, но со значениями NULL она выдаёт не NULL. Любое значение NULL для неё считается неравным (отличным от) любому значению не NULL, а два NULL считаются равными (не различными). Таким образом, результатом такого выражения будет true или false, но не NULL.

конструктор_строки IS NOT DISTINCT FROM конструктор_строки

Эта конструкция похожа на сравнение строк с оператором =, но со значениями NULL она выдаёт не NULL. Любое значение NULL для неё считается неравным (отличным от) любому значению не NULL, а два NULL считаются равными (не различными). Таким образом, результатом такого выражения всегда будет true или false, но не NULL.

9.24.6. Сравнение составных типов

запись оператор запись

Стандарт SQL требует, чтобы при сравнении строк возвращался NULL, если результат зависит от сравнения двух значений NULL или значения NULL и не NULL. Postgres Pro выполняет это требование только при сравнении строк, созданных конструкторами (как описано в [Подразделе 9.24.5](#)), или строки, созданной конструктором, со строкой результата подзапроса (как было описано в [Разделе 9.23](#)). В других контекстах при сравнении полей составных типов два значения NULL считаются равными, а любое значение не NULL полагается меньшим NULL. Это отклонение от правила необходимо для полноценной реализации сортировки и индексирования составных типов.

После вычисления каждой стороны они сравниваются по строкам. Сравнения составных типов возможны с оператором =, <>, <, <=, > или >=, либо другим подобным. (Точнее, оператором сравнения строк может быть любой оператор, входящий в класс операторов В-дерева, либо обратный к оператору =, входящему в класс операторов В-дерева.) По умолчанию вышеперечисленные операторы действуют так же, как выражение IS [NOT] DISTINCT FROM для конструкторов строк (см. [Подраздел 9.24.5](#)).

Для поддержки сравнения строк с элементами, для которых не определён класс операторов В-дерева по умолчанию, введены следующие операторы: *=, *<>, *<, *<=, *> и *>=. Эти операторы

сравнивают внутреннее двоичное представление двух строк. Учтите, что две строки могут иметь различное двоичное представление, даже когда при сравнении оператором равенства считаются равными. Порядок строк с такими операторами детерминирован, но не несёт смысловой нагрузки. Данные операторы не предназначены для обычных запросов; они применяются внутри системы для материализованных представлений и могут быть полезны для других специальных целей, например для репликации или исключения дубликатов в В-дереве (см. [Подраздел 64.4.3](#)).

9.25. Функции, возвращающие множества

В этом разделе описаны функции, которые могут возвращать не одну, а множество строк. Чаще всего из их числа используются функции, генерирующие ряды значений, которые перечислены в [Таблице 9.68](#) и [Таблице 9.69](#). Другие, более специализированные функции множеств описаны в других разделах этой документации. Варианты комбинирования нескольких функций, возвращающих множества строк, описаны в [Подразделе 7.2.1.4](#).

Таблица 9.68. Функции, генерирующие ряды значений

Функция	Описание
<code>generate_series (start integer, stop integer [, step integer])</code>	<code>→ setof integer</code>
<code>generate_series (start bigint, stop bigint [, step bigint])</code>	<code>→ setof bigint</code>
<code>generate_series (start numeric, stop numeric [, step numeric])</code>	<code>→ setof numeric</code>
	Выдаёт ряд значений от <i>start</i> до <i>stop</i> с заданным шагом (<i>step</i>), по умолчанию равным 1.
<code>generate_series (start timestamp, stop timestamp, step interval)</code>	<code>→ setof timestamp</code>
<code>generate_series (start timestamp with time zone, stop timestamp with time zone, step interval)</code>	<code>→ setof timestamp with time zone</code>
	Выдаёт ряд значений от <i>start</i> до <i>stop</i> с заданным шагом (<i>step</i>).

Если заданный шаг (*step*) положительный, а *start* оказывается больше *stop*, эти функции возвращают 0 строк. Тот же результат будет, если *step* меньше 0, а *start* меньше *stop*, или если в каком-либо аргументе передаётся NULL. Если же *step* будет равен 0, произойдёт ошибка. Несколько примеров:

```
SELECT * FROM generate_series(2,4);
```

```
generate_series
-----
                2
                3
                4
(3 rows)
```

```
SELECT * FROM generate_series(5,1,-2);
```

```
generate_series
-----
                5
                3
                1
(3 rows)
```

```
SELECT * FROM generate_series(4,3);
```

```
generate_series
-----
(0 rows)
```

```
SELECT generate_series(1.1, 4, 1.3);
```

```
generate_series
-----
```

```

1.1
2.4
3.7
(3 rows)

-- в этом примере задействуется оператор прибавления к дате целого числа:
SELECT current_date + s.a AS dates FROM generate_series(0,14,7) AS s(a);
      dates
-----
2004-02-05
2004-02-12
2004-02-19
(3 rows)

SELECT * FROM generate_series('2008-03-01 00:00'::timestamp,
                              '2008-03-04 12:00', '10 hours');
      generate_series
-----
2008-03-01 00:00:00
2008-03-01 10:00:00
2008-03-01 20:00:00
2008-03-02 06:00:00
2008-03-02 16:00:00
2008-03-03 02:00:00
2008-03-03 12:00:00
2008-03-03 22:00:00
2008-03-04 08:00:00
(9 rows)

```

Таблица 9.69. Функции, генерирующие индексы массивов

Функция	Описание
<code>generate_subscripts (array anyarray, dim integer) → setof integer</code>	Выдаёт ряд значений, содержащий все допустимые индексы в размерности <i>dim</i> данного массива.
<code>generate_subscripts (array anyarray, dim integer, reverse boolean) → setof integer</code>	Выдаёт ряд значений, содержащий все допустимые индексы в размерности <i>dim</i> данного массива. Если параметр <i>reverse</i> равен true, значения выдаются от большего к меньшему.

Функция `generate_subscripts` упрощает задачу получения всех допустимых индексов для указанной размерности данного массива. Она выдаёт 0 строк, если в массиве нет указанной размерности или любой из аргументов — NULL. Взгляните на следующие примеры:

```

-- простой пример использования:
SELECT generate_subscripts('{NULL,1,NULL,2}'::int[], 1) AS s;
      s
----
1
2
3
4
(4 rows)

-- для показанного массива получение индекса и обращение
-- к элементу по индексу выполняется с помощью подзапроса:
SELECT * FROM arrays;

```

```

a
-----
{-1,-2}
{100,200,300}
(2 rows)

SELECT a AS array, s AS subscript, a[s] AS value
FROM (SELECT generate_subscripts(a, 1) AS s, a FROM arrays) foo;
array      | subscript | value
-----+-----+-----
{-1,-2}    |          | -1
{-1,-2}    |          | -2
{100,200,300}|         | 100
{100,200,300}|         | 200
{100,200,300}|         | 300
(5 rows)

-- разворачивание двумерного массива:
CREATE OR REPLACE FUNCTION unnest2(anyarray)
RETURNS SETOF anyelement AS $$
select $1[i][j]
    from generate_subscripts($1,1) g1(i),
         generate_subscripts($1,2) g2(j);
$$ LANGUAGE sql IMMUTABLE;
CREATE FUNCTION
SELECT * FROM unnest2(ARRAY[[1,2],[3,4]]);
unnest2
-----
1
2
3
4
(4 rows)

```

Когда для функции в предложении FROM указывается WITH ORDINALITY, к столбцам результата функции добавляется столбец типа bigint, числа в котором начинаются с 1 и увеличиваются на 1 для каждой строки, выданной функцией. В первую очередь это полезно для функций, возвращающих множества, например, unnest().

```

-- функция, возвращающая множество, с нумерацией
SELECT * FROM pg_ls_dir('.') WITH ORDINALITY AS t(ls,n);
ls      | n
-----+---
pg_serial    | 1
pg_twophase  | 2
postmaster.opts | 3
pg_notify    | 4
postgresql.conf | 5
pg_tblspc    | 6
logfile      | 7
base         | 8
postmaster.pid | 9
pg_ident.conf | 10
global       | 11
pg_xact      | 12
pg_snapshots | 13
pg_multixact | 14
PG_VERSION   | 15

```



```
pg_wal          | 16
pg_hba.conf     | 17
pg_stat_tmp     | 18
pg_subtrans     | 19
(19 rows)
```

9.26. Системные информационные функции и операторы

В [Таблице 9.70](#) перечислен ряд функций, предназначенных для получения информации о текущем сеансе и системе.

В дополнение к перечисленным здесь функциям существуют также функции, связанные с подсистемой статистики, которые тоже предоставляют системную информацию. Подробнее они рассматриваются в [Подразделе 26.2.2](#).

Таблица 9.70. Функции получения информации о сеансе

Функция	Описание
<code>current_catalog</code> $\rightarrow$ name <code>current_database</code> $() \rightarrow$ name	Выдаёт имя текущей базы данных. (В стандарте SQL базы данных называются «каталогами», поэтому стандарту соответствует написание <code>current_catalog</code> .)
<code>current_query</code> $() \rightarrow$ text	Выдаёт текст запроса, выполняемого в данный момент, в том виде, в каком его передал клиент (может состоять из нескольких операторов).
<code>current_role</code> $\rightarrow$ name	Аналог <code>current_user</code> .
<code>current_schema</code> $\rightarrow$ name <code>current_schema</code> $() \rightarrow$ name	Выдаёт имя первой схемы в пути поиска (или значение NULL, если путь поиска пустой). В эту схему будут добавляться таблицы и другие объекты, при создании которых схема не указывается явно.
<code>current_schemas</code> $(include_implicit \text{ boolean}) \rightarrow$ name[]	Выдаёт массив имён всех схем, в настоящее время представленных в действующем пути поиска, по порядку их приоритета. (Элементы текущего значения search_path , которым не соответствуют существующие схемы, опускаются). Если логический аргумент равен true, в результат включаются системные схемы, неявно просматриваемые при поиске, например, <code>pg_catalog</code> .
<code>current_user</code> $\rightarrow$ name	Выдаёт имя пользователя в текущем контексте выполнения.
<code>inet_client_addr</code> $() \rightarrow$ inet	Выдаёт IP-адрес текущего клиента либо NULL, если текущее соединение установлено через Unix-сокеты.
<code>inet_client_port</code> $() \rightarrow$ integer	Выдаёт номер TCP-порта текущего клиента либо NULL, если текущее подключение установлено через Unix-сокеты.
<code>inet_server_addr</code> $() \rightarrow$ inet	Выдаёт IP-адрес, через который сервер принял текущее подключение, либо NULL, если текущее подключение установлено через Unix-сокеты.
<code>inet_server_port</code> $() \rightarrow$ integer	

Функция	Описание
	Выдаёт номер TCP-порта, через который сервер принял текущее подключение, либо NULL, если текущее подключение установлено через Unix-сокеты.
<code>pg_backend_pid () → integer</code>	Выдаёт идентификатор серверного процесса, обслуживающего текущий сеанс.
<code>pg_blocking_pids (integer) → integer[]</code>	Выдаёт массив с идентификаторами процессов сеансов, которые блокируют серверный процесс с указанным идентификатором, либо пустой массив, если указанный серверный процесс не найден или не заблокирован. Один серверный процесс блокирует другой, если он либо удерживает блокировку, конфликтующую с блокировкой, запрашиваемой вторым (жёсткая блокировка), либо ожидает блокировку, которая вызвала бы конфликт с запросом блокировки заблокированного процесса и находится перед ней в очереди ожидания (мягкая блокировка). При параллелизации запросов эта функция всегда выдаёт видимые клиентом идентификаторы процессов (то есть, результаты <code>pg_backend_pid</code>), даже если фактическая блокировка удерживается или ожидается дочерним рабочим процессом. Вследствие этого в результатах могут оказаться дублирующие PID. Также заметьте, что когда конфликтующую блокировку удерживает подготовленная транзакция, в выводе этой функции она будет представлена нулевым ID процесса. Частые вызовы этой функции могут отразиться на производительности базы данных, так как ей нужен монопольный доступ к общему состоянию менеджера блокировок, пусть и на короткое время.
<code>pg_conf_load_time () → timestamp with time zone</code>	Выдаёт время, когда в последний раз сервер загружал файлы конфигурации. (Если текущий сеанс начался раньше, она возвращает время, когда эти файлы были перезагружены для данного сеанса, так что в разных сеансах это значение может немного различаться. В противном случае это будет время, когда файлы конфигурации считал главный процесс.)
<code>pg_current_logfile ([text]) → text</code>	Выдаёт путь к файлу журнала, в настоящее время используемому сборщиком сообщений. Этот путь состоит из пути каталога log_directory и имени конкретного файла. Если сборщик сообщений отключён, выдаётся значение NULL. Если ведутся несколько журналов в разных форматах, при вызове функции <code>pg_current_logfile</code> без аргументов возвращается путь первого существующего файла следующего формата (в порядке приоритета): <code>stderr</code> , <code>csvlog</code> , <code>jsonlog</code> . Если файл журнала в этих форматах отсутствует, выдаётся NULL. Чтобы запросить информацию о файле в определённом формате, передайте <code>csvlog</code> , <code>jsonlog</code> или <code>stderr</code> в качестве значения необязательного параметра типа <code>text</code> . Если запрошенный формат не включён в log_destination , результатом будет NULL. Результат функции отражает содержимое файла <code>current_logfiles</code> .
<code>pg_my_temp_schema () → oid</code>	Выдаёт OID временной схемы текущего сеанса или 0, если такой схемы нет (в рамках сеанса не создавались временные таблицы).
<code>pg_is_other_temp_schema (oid) → boolean</code>	Выдаёт true, если заданный OID относится к временной схеме другого сеанса. (Это может быть полезно, например для исключения временных таблиц других сеансов из общего списка при просмотре таблиц базы данных.)
<code>pg_jit_available () → boolean</code>	Выдаёт true, если в данном сеансе доступна JIT-компиляция (см. Главу 30) и параметр конфигурации <code>jit</code> включён.
<code>pg_listening_channels () → setof text</code>	

Функция	Описание
	Выдаёт список имён каналов асинхронных уведомлений, по которым текущий сеанс принимает сигналы.
<code>pg_notification_queue_usage () → double precision</code>	Выдаёт долю (0–1) максимального размера очереди асинхронных уведомлений, в настоящее время занятую уведомлениями, ожидающими обработки. За дополнительными сведениями обратитесь к LISTEN и NOTIFY .
<code>pg_postmaster_start_time () → timestamp with time zone</code>	Выдаёт время, когда был запущен сервер.
<code>pg_safe_snapshot_blocking_pids (integer) → integer[]</code>	Выдаёт массив с идентификаторами процессов сеансов, которые блокируют серверный процесс с указанным идентификатором, не давая ему получить безопасный снимок, либо выдаёт пустой массив, если такой серверный процесс не найден или он не заблокирован. Сеанс, выполняющий транзакцию уровня <code>SERIALIZABLE</code> , блокирует транзакцию <code>SERIALIZABLE READ ONLY DEFERRABLE</code> , не давая ей получить снимок, пока она не определит, что можно безопасно избежать установления предикатных блокировок. За дополнительными сведениями о сериализуемых и откладываемых транзакциях обратитесь к Подразделу 13.2.3 . Частые вызовы этой функции могут отразиться на производительности сервера, так как ей нужен доступ к общему состоянию менеджера предикатных блокировок, пусть и на короткое время.
<code>pg_trigger_depth () → integer</code>	Выдаёт текущий уровень вложенности в триггерах Postgres Pro (0, если эта функция вызывается не из тела триггера, непосредственно или косвенно).
<code>session_user → name</code>	Выдаёт имя пользователя сеанса.
<code>user → name</code>	Аналог <code>current_user</code> .
<code>version () → text</code>	Выдаёт текстовую строку, описывающую версию сервера PostgreSQL. Эту информацию также можно получить из переменной server_version или, в более машинно-ориентированном формате, из переменной server_version_num . При разработке программ следует использовать функцию <code>server_version_num</code> (появившуюся в версии 8.2) или PQserverVersion , а не разбирать текстовую версию.
<code>pgpro_version () → text</code>	Выдаёт текстовую строку, описывающую версию сервера Postgres Pro.
<code>pgpro_edition () → text</code>	Выдаёт текстовую строку, описывающую редакцию Postgres Pro, например <code>standard</code> или <code>enterprise</code> .
<code>pgpro_build () → text</code>	Выдаёт идентификатор состояния исходного кода, из которого скомпилирован Postgres Pro.

Примечание

Функции `current_catalog`, `current_role`, `current_schema`, `current_user`, `session_user` и `user` имеют особый синтаксический статус в SQL: они должны вызываться без скобок после

имени. Postgres Pro позволяет добавить скобки в вызове `current_schema`, но не других функций.

Функция `session_user` обычно возвращает имя пользователя, установившего текущее соединение с базой данных, но суперпользователи могут изменить это имя, выполнив команду [SET SESSION AUTHORIZATION](#). Функция `current_user` возвращает идентификатор пользователя, по которому будут проверяться его права. Обычно это тот же пользователь, что и пользователь сеанса, но его можно сменить с помощью [SET ROLE](#). Этот идентификатор также меняется при выполнении функций с атрибутом `SECURITY DEFINER`. На языке Unix пользователь сеанса называется «реальным», а текущий — «эффективным». Имена `current_role` и `user` являются синонимами `current_user`. (В стандарте SQL `current_role` и `current_user` имеют разное значение, но в Postgres Pro они не различаются, так как пользователи и роли объединены в единую сущность.)

В [Таблице 9.71](#) показаны функции, позволяющие программно проверить права доступа к объектам. (Подробнее о правах рассказывается в [Разделе 5.7](#).) Этим функциям в качестве идентификатора пользователя, для которого запрашиваются права, можно передать его имя или OID (`pg_authid.oid`), а также можно передать `public` (это будет указывать на псевдороль PUBLIC). Кроме того, аргумент `user` можно полностью опустить, тогда будет подразумеваться `current_user`. Объект, права для доступа к которому запрашиваются, также можно задать по имени или по OID. Когда указывается имя объекта, его можно дополнить именем схемы, если это уместно. Запрашиваемое право записывается текстовой строкой, которая должна задавать одно из прав доступа, соответствующих типу объекта (например, `SELECT`). Дополнительно к названию права можно добавить `WITH GRANT OPTION` и проверить, разрешено ли пользователю передавать это право другим. Кроме того, в одном параметре можно перечислить несколько прав через запятую, и тогда функция возвратит `true`, если пользователь имеет какие-либо из них. (Регистр в названии прав не имеет значения, а между ними (но не внутри) разрешены пробельные символы.) Пара примеров:

```
SELECT has_table_privilege('myschema.mytable', 'select');
SELECT has_table_privilege('joe', 'mytable', 'INSERT, SELECT WITH GRANT OPTION');
```

Таблица 9.71. Функции для проверки прав доступа

Функция	Описание
<code>has_any_column_privilege</code>	<code>([user name или oid,] table text или oid, privilege text) → boolean</code> Имеет ли пользователь указанное право для какого-либо столбца таблицы? Ответ положительный, когда он имеет это право для всей таблицы или ему дано соответствующее право на уровне столбцов хотя бы для одного столбца. Возможные права: <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> и <code>REFERENCES</code> .
<code>has_column_privilege</code>	<code>([user name или oid,] table text или oid, column text или smallint, privilege text) → boolean</code> Имеет ли пользователь указанное право для заданного столбца таблицы? Ответ положительный, когда он имеет это право для всей таблицы или ему дано соответствующее право на уровне столбца. Столбец можно задать по имени или номеру атрибута (<code>pg_attribute.attnum</code>). Возможные права: <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> и <code>REFERENCES</code> .
<code>has_database_privilege</code>	<code>([user name или oid,] database text или oid, privilege text) → boolean</code> Имеет ли пользователь указанное право в базе данных? Возможные права: <code>CREATE</code> , <code>CONNECT</code> , <code>TEMPORARY</code> и <code>TEMP</code> (синоним <code>TEMPORARY</code>).
<code>has_foreign_data_wrapper_privilege</code>	<code>([user name или oid,] fdw text или oid, privilege text) → boolean</code> Имеет ли пользователь право для обёртки сторонних данных? На данный момент возможно только одно право: <code>USAGE</code> .

Функция	Описание
<code>has_function_privilege</code>	<code>([user name или oid,] function text или oid, privilege text) → boolean</code> Имеет ли пользователь право для функции? Возможно единственное право: EXECUTE. При указании функции по имени, а не по OID, допускаются те же входные значения, что и для типа regprocedure (см. Раздел 8.19). Например: <code>SELECT has_function_privilege('joeuser', 'myfunc(int, text)', 'execute');</code>
<code>has_language_privilege</code>	<code>([user name или oid,] language text или oid, privilege text) → boolean</code> Имеет ли пользователь право для языка? Возможно единственное право: USAGE.
<code>has_parameter_privilege</code>	<code>([user name или oid,] parameter text, privilege text) → boolean</code> Имеет ли пользователь право для параметра конфигурации? Регистр в имени параметра игнорируется. Возможные права: SET и ALTER SYSTEM.
<code>has_schema_privilege</code>	<code>([user name или oid,] schema text или oid, privilege text) → boolean</code> Имеет ли пользователь право для схемы? Возможные права: CREATE и USAGE.
<code>has_sequence_privilege</code>	<code>([user name или oid,] sequence text или oid, privilege text) → boolean</code> Имеет ли пользователь право для последовательности? Возможные права: USAGE, SELECT и UPDATE.
<code>has_server_privilege</code>	<code>([user name или oid,] server text или oid, privilege text) → boolean</code> Имеет ли пользователь право для стороннего сервера? На данный момент возможно только одно право: USAGE.
<code>has_table_privilege</code>	<code>([user name или oid,] table text или oid, privilege text) → boolean</code> Имеет ли пользователь право для таблицы? Возможные права: SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES и TRIGGER.
<code>has_tablespace_privilege</code>	<code>([user name или oid,] tablespace text или oid, privilege text) → boolean</code> Имеет ли пользователь право для табличного пространства? Возможно единственное право: CREATE.
<code>has_type_privilege</code>	<code>([user name или oid,] type text или oid, privilege text) → boolean</code> Имеет ли пользователь право для типа данных? Возможно единственное право: USAGE. При указании типа по имени, а не по OID, допускаются те же входные значения, что и для типа данных regtype (см. Раздел 8.19).
<code>pg_has_role</code>	<code>([user name или oid,] role text или oid, privilege text) → boolean</code> Имеет ли пользователь право для роли? Возможные права: MEMBER и USAGE. Под MEMBER подразумевается непосредственное или косвенное членство в роли (то есть право выполнить команду SET ROLE), тогда как USAGE означает, что пользователь получает права этой роли автоматически, не выполняя SET ROLE. К любому из этих типов прав можно добавить WITH ADMIN OPTION или WITH GRANT OPTION, чтобы проверить наличие права ADMIN (все четыре варианта написания проверяют одно и то же). Эта функция не поддерживает особый случай установки значения public для user, так как псевдороль PUBLIC не может быть членом обычных ролей.
<code>row_security_active</code>	<code>(table text или oid) → boolean</code> Действует ли защита на уровне строк для заданной таблицы в контексте текущего пользователя в текущем окружении?

В [Таблице 9.72](#) показаны операторы, предназначенные для работы с типом aclitem, который представляет в системном каталоге права доступа. О содержании значений этого типа рассказывается в [Разделе 5.7](#).

Таблица 9.72. Операторы для типа aclitem

Оператор Описание Пример(ы)
<code>aclitem = aclitem → boolean</code> Значения <code>aclitem</code> равны? (Заметьте, что для типа <code>aclitem</code> не определён обычный набор операторов сравнения, поддерживается только проверка равенства. Массивы <code>aclitem</code> , в свою очередь, также могут сравниваться только на равенство.) <code>'calvin=r*w/hobbes'::aclitem = 'calvin=r*w*/hobbes'::aclitem → f</code>
<code>aclitem[] @> aclitem → boolean</code> Содержит ли массив заданное право? (Ответ положительный, если в массиве есть запись, соответствующая правообладателю и праводателю из <code>aclitem</code> и содержащая как минимум указанный набор прав.) <code>'{calvin=r*w/hobbes,hobbes=r*w*/postgres}'::aclitem[] @> 'calvin=r*/hobbes'::aclitem → t</code>
<code>aclitem[] ~ aclitem → boolean</code> Это устаревший синоним для <code>@></code> . <code>'{calvin=r*w/hobbes,hobbes=r*w*/postgres}'::aclitem[] ~ 'calvin=r*/hobbes'::aclitem → t</code>

В [Таблице 9.73](#) приведены дополнительные функции, предназначенные для работы с типом `aclitem`.

Таблица 9.73. Функции для типа aclitem

Функция Описание
<code>acldefault (type "char", ownerId oid) → aclitem[]</code> Выдаёт массив с элементами <code>aclitem</code> , содержащий действующий по умолчанию набор прав доступа для объектов типа <code>type</code> , принадлежащих роли <code>ownerId</code> . Результат показывает, какие права доступа подразумеваются, когда список ACL определённого объекта пуст. (Права доступа, действующие по умолчанию, описываются в Разделе 5.7 .) Параметр <code>type</code> может принимать одно из следующих значений: 'c' — столбец (COLUMN), 'r' — таблица или подобный таблице объект (TABLE), 's' — последовательность (SEQUENCE), 'd' — база данных (DATABASE), 'f' — функция (FUNCTION) или процедура (PROCEDURE), 'l' — язык (LANGUAGE), 'L' — большой объект (LARGE OBJECT), 'n' — схема (SCHEMA), 'p' — параметр (PARAMETER), 't' — табличное пространство (TABLESPACE), 'F' — обёртка сторонних данных (FOREIGN DATA WRAPPER), 'S' — сторонний сервер (FOREIGN SERVER) и 'T' — тип (TYPE) или домен (DOMAIN).
<code>aclexplode (aclitem[]) → setof record (grantor oid, grantee oid, privilege_type text, is_grantable boolean)</code> Возвращает массив <code>aclitem</code> в виде набора табличных строк. Если праводателем является псевдороль PUBLIC, она представляется нулём в столбце <code>grantee</code> . Назначенные права представляются ключевыми словами SELECT, INSERT и т. д. Заметьте, что каждое право выводится в отдельной строке, так что в столбце <code>privilege_type</code> содержится только одно ключевое слово.
<code>makeaclitem (grantee oid, grantor oid, privileges text, is_grantable boolean) → aclitem</code> Конструирует значение <code>aclitem</code> с заданными свойствами.

В [Таблице 9.74](#) показаны функции, определяющие *видимость* объекта с текущим путём поиска схем. К примеру, таблица считается видимой, если содержащая её схема включена в путь поиска и нет другой таблицы с тем же именем, которая была бы найдена в этом пути раньше. Другими словами, к этой таблице можно будет обратиться просто по её имени, без явного указания схемы. Просмотреть список всех видимых таблиц можно так:


```
SELECT relname FROM pg_class WHERE pg_table_is_visible(oid);
```

Объект, представляющий функцию и оператор, считаются видимым в пути поиска, если при просмотре пути не находится предшествующий ему другой объект с тем же именем *и типами аргументов*. Для семейств и классов операторов помимо имени объекта во внимание принимается также связанный метод доступа к индексу.

Таблица 9.74. Функции для определения видимости

Функция Описание
<code>pg_collation_is_visible (collation oid) → boolean</code> Видимо ли правило сортировки в пути поиска?
<code>pg_conversion_is_visible (conversion oid) → boolean</code> Видимо ли преобразование в пути поиска?
<code>pg_function_is_visible (function oid) → boolean</code> Видима ли функция в пути поиска? (Также работает для процедур и агрегатных функций.)
<code>pg_opclass_is_visible (opclass oid) → boolean</code> Виден ли класс операторов в пути поиска?
<code>pg_operator_is_visible (operator oid) → boolean</code> Видим ли оператор в пути поиска?
<code>pg_opfamily_is_visible (opclass oid) → boolean</code> Видимо ли семейство операторов в пути поиска?
<code>pg_statistics_obj_is_visible (stat oid) → boolean</code> Видим ли объект статистики в пути поиска?
<code>pg_table_is_visible (table oid) → boolean</code> Видима ли таблица в пути поиска? (Эта проверка работает со всеми типами отношений, включая представления, матпредставления, индексы, последовательности и сторонние таблицы.)
<code>pg_ts_config_is_visible (config oid) → boolean</code> Видима ли конфигурация текстового поиска?
<code>pg_ts_dict_is_visible (dict oid) → boolean</code> Видим ли словарь текстового поиска?
<code>pg_ts_parser_is_visible (parser oid) → boolean</code> Видим ли анализатор текстового поиска?
<code>pg_ts_template_is_visible (template oid) → boolean</code> Видим ли шаблон текстового поиска?
<code>pg_type_is_visible (type oid) → boolean</code> Видим ли тип (или домен) в пути поиска?

Всем этим функциям должен передаваться OID проверяемого объекта. Если вы хотите проверить объект по имени, удобнее использовать типы-псевдонимы OID (`regclass`, `regtype`, `regprocedure`, `regoperator`, `regconfig` или `regdictionary`), например:

```
SELECT pg_type_is_visible('myschema.widget'::regtype);
```

Заметьте, что проверять таким способом имена без указания схемы не имеет большого смысла — если имя удастся распознать, значит и объект будет видимым.

В [Таблице 9.75](#) перечислены функции, извлекающие информацию из системных каталогов.

Таблица 9.75. Функции для обращения к системным каталогам

Функция	Описание
<code>format_type (type oid, typemod integer) → text</code>	Выдаёт в формате SQL имя типа данных, определяемого по OID и, возможно, модификатору типа. Если модификатор неизвестен, вместо него можно передать NULL.
<code>pg_char_to_encoding (encoding name) → integer</code>	Преобразует предоставленное имя кодировки в целое число, представляющее внутренний идентификатор, используемый в некоторых таблицах системного каталога. Возвращает -1, если указано неизвестное имя кодировки.
<code>pg_encoding_to_char (encoding integer) → name</code>	Преобразует целое число, используемое в качестве внутреннего идентификатора кодировки в некоторых таблицах системного каталога, в понятное имя. Возвращает пустую строку, если указан неправильный номер кодировки.
<code>pg_get_catalog_foreign_keys () → setof record (fktable regclass, fkcols text [], pktable regclass, pkcols text [], is_array boolean, is_opt boolean)</code>	Возвращает набор записей, описывающих отношения внешних ключей, существующие в системных каталогах Postgres Pro. Столбец <i>fktable</i> содержит имя ссылающегося каталога, а столбец <i>fkcols</i> содержит имена ссылающихся столбцов. Точно так же столбец <i>pktable</i> содержит имя целевого каталога, а столбец <i>pkcols</i> содержит имена целевых столбцов. Если столбец <i>is_array</i> содержит значение true, последний ссылающийся столбец является массивом, каждый из элементов которого должен соответствовать некоторой записи в целевом каталоге. Если столбец <i>is_opt</i> содержит значение true, ссылающиеся столбцы могут содержать нули вместо действительной ссылки.
<code>pg_get_constraintdef (constraint oid [, pretty boolean]) → text</code>	Восстанавливает команду, создающую ограничение. (Текст команды не является изначальным, он восстанавливается путём декомпиляции.)
<code>pg_get_expr (expr pg_node_tree , relation oid [, pretty boolean]) → text</code>	Декомпилирует внутреннюю форму выражений, находящихся в системных каталогах, например, выражений, задающих значения по умолчанию. Если выражение может содержать Vars (переменные), передайте во втором параметре OID отношения, к которому обращается данное выражение; в противном случае достаточно передать 0.
<code>pg_get_functiondef (func oid) → text</code>	Восстанавливает команду, создающую функцию или процедуру. (Текст команды не является изначальным, он восстанавливается путём декомпиляции.) В результате выдаётся полноценный оператор CREATE OR REPLACE FUNCTION или CREATE OR REPLACE PROCEDURE.
<code>pg_get_function_arguments (func oid) → text</code>	Восстанавливает список аргументов для функции или процедуры в том виде, в каком он должен задаваться в команде CREATE FUNCTION (включая значения по умолчанию).
<code>pg_get_function_identity_arguments (func oid) → text</code>	Восстанавливает список аргументов, необходимый для однозначной идентификации функции или процедуры, в том виде, в каком он должен задаваться, например в ALTER FUNCTION. В этом виде значения по умолчанию не указываются.
<code>pg_get_function_result (func oid) → text</code>	Восстанавливает для функции предложение RETURNS в том виде, в котором оно должно указываться в команде CREATE FUNCTION. Для процедуры возвращается NULL.
<code>pg_get_indexdef (index oid [, column integer, pretty boolean]) → text</code>	Восстанавливает команду, создающую индекс. (Текст команды не является изначальным, он восстанавливается путём декомпиляции.) Если передаётся значение <i>column</i> и оно отлично от нуля, восстанавливается только определение этого столбца.

Функция	Описание
<code>pg_get_keywords () → setof record (word text, catcode "char", barelabel boolean, catdesc text, baredesc text)</code>	Выдаёт набор записей, описывающих ключевые слова, которые воспринимает сервер. Столбец <i>word</i> содержит слово, а столбец <i>catcode</i> — код категории: U — не зарезервировано, C — слово, которое может быть именем столбца, T — слово, которое может быть именем типа или функции, R — полностью зарезервированное слово. Столбец <i>barelabel</i> содержит true, если ключевое слово можно использовать в качестве «открытой» метки столбца в списках SELECT, или false, если его можно использовать только после AS. Столбец <i>catdesc</i> содержит описание категории слова, возможно локализованное. Столбец <i>baredesc</i> содержит возможно локализованную строку, описывающую статус метки столбца, содержащего слово.
<code>pg_get_ruledef (rule oid [, pretty boolean]) → text</code>	Восстанавливает команду, создающую правило. (Текст команды не является изначально, он восстанавливается путём декомпиляции.)
<code>pg_get_serial_sequence (table text, column text) → text</code>	Выдаёт имя последовательности, связанной со столбцом, либо NULL, если такой последовательности нет. Для столбца идентификации это будет последовательность, созданная для него неявно внутри. Для столбцов, имеющих один из последовательных типов (<i>serial</i> , <i>smallserial</i> , <i>bigserial</i>), это последовательность, созданная объявлением данного столбца. В последнем случае эту связь можно изменить или разорвать, воспользовавшись командой ALTER SEQUENCE OWNED BY. (Возможно, эту функцию стоило назвать <code>pg_get_owned_sequence</code> ; её существующее имя отражает тот факт, что изначально она использовалась со столбцами последовательных типов.) В первом параметре функции указывается имя таблицы (возможно, дополненное схемой), а во втором — имя столбца. Так как первый параметр может содержать имя схемы и таблицы, он воспринимается не как идентификатор в кавычках и поэтому по умолчанию приводится к нижнему регистру, тогда как имя столбца во втором параметре воспринимается как заключённое в кавычки и в нём регистр символов сохраняется. Эта функция возвращает имя в виде, пригодном для передачи функциям, работающим с последовательностями (см. Раздел 9.17). Обычно эта функция применяется для получения текущего значения последовательности для столбца идентификации или последовательного столбца, например так: <code>SELECT currval(pg_get_serial_sequence('sometable', 'id'));</code>
<code>pg_get_statisticsobjdef (statobj oid) → text</code>	Восстанавливает команду, создающую объект расширенной статистики. (Текст команды не является изначально, он восстанавливается путём декомпиляции.)
<code>pg_get_triggerdef (trigger oid [, pretty boolean]) → text</code>	Восстанавливает команду, создающую триггер. (Текст команды не является изначально, он восстанавливается путём декомпиляции.)
<code>pg_get_userbyid (role oid) → name</code>	Выдаёт имя роли по заданному OID.
<code>pg_get_viewdef (view oid [, pretty boolean]) → text</code>	Восстанавливает команду SELECT, определяющую представление или матпредставление. (Текст команды не является изначально, он восстанавливается путём декомпиляции.)
<code>pg_get_viewdef (view oid, wrap_column integer) → text</code>	Восстанавливает команду SELECT, определяющую представление или матпредставление. (Текст команды не является изначально, он восстанавливается путём декомпиляции.) В этой вариации функции всегда применяется визуальное улучшение и длинные строки переносятся, чтобы текст умещался по ширине в заданное число столбцов.
<code>pg_get_viewdef (view text [, pretty boolean]) → text</code>	

Функция	Описание
	Восстанавливает команду <code>SELECT</code> , определяющую представление или матпредставление, заданное по имени, а не по OID. (Это устаревшая вариация, используйте вместо неё вариацию с OID).
<code>pg_index_column_has_property</code> (<i>index regclass, column integer, property text</i>) → <i>boolean</i>	Проверяет, имеет ли столбец индекса заданное свойство. Типичные свойства столбцов описаны в Таблице 9.76 . (Заметьте, что расширенные методы доступа могут определять для своих индексов дополнительные свойства.) Если заданное имя свойства не известно или не применимо к конкретному объекту, либо если OID или номер столбца не указывают на существующий объект, эта функция возвращает <code>NULL</code> .
<code>pg_index_has_property</code> (<i>index regclass, property text</i>) → <i>boolean</i>	Проверяет, имеет ли индекс заданное свойство. Типичные свойства индексов описаны в Таблице 9.76 . (Заметьте, что расширенные методы доступа могут определять для своих индексов дополнительные свойства.) Если заданное имя свойства не известно или не применимо к конкретному объекту, либо если OID не указывает на существующий объект, эта функция возвращает <code>NULL</code> .
<code>pg_indexam_has_property</code> (<i>am oid, property text</i>) → <i>boolean</i>	Проверяет, имеет ли метод доступа индекса заданное свойство. Свойства методов доступа описаны в Таблице 9.78 . Если заданное имя свойства неизвестно или не применимо к конкретному объекту, либо если OID не указывает на существующий объект, эта функция возвращает <code>NULL</code> .
<code>pg_options_to_table</code> (<i>options_array text[], option_name text, option_value text</i>) → <i>setof record</i>	Выдаёт набор параметров хранилища, представленных значением из <code>pg_class .reloptions</code> или <code>pg_attribute .attoptions</code> .
<code>pg_settings_get_flags</code> (<i>guc text</i>) → <i>text[]</i>	Возвращает массив флагов, связанных с данным параметром GUC, или <code>NULL</code> , если указанный параметр не существует. Если же он существует, но у него нет флагов для отображения, возвращается пустой массив. Показываются только самые полезные флаги, перечисленные в Таблице 9.79 .
<code>pg_tablespace_databases</code> (<i>tablespace oid</i>) → <i>setof oid</i>	Выдаёт набор OID баз данных, объекты которых размещены в заданном табличном пространстве. Если эта функция возвращает строки, это означает, что табличное пространство не пустое и удалить его нельзя. Какие именно объекты находятся в табличном пространстве, можно определить, подключаясь к базам данных, OID которых сообщила <code>pg_tablespace_databases</code> , и анализируя их каталоги <code>pg_class</code> .
<code>pg_tablespace_location</code> (<i>tablespace oid</i>) → <i>text</i>	Выдаёт путь в файловой системе к местоположению заданного табличного пространства.
<code>pg_typeof</code> (<i>"any"</i>) → <i>regtype</i>	Выдаёт OID типа данных для переданного значения. Это может быть полезно для разрешения проблем или динамического создания SQL-запросов. Эта функция объявлена как возвращающая тип <code>regtype</code> , который является псевдонимом типа OID (см. Раздел 8.19); это означает, что значение этого типа можно сравнивать как OID, но выводится оно как название типа. Пример: <pre>SELECT pg_typeof(33); pg_typeof -----</pre>

Функция	Описание
<code>integer</code>	<pre>SELECT typplen FROM pg_type WHERE oid = pg_typeof(33); typplen ----- 4</pre>
<code>COLLATION FOR ("any") → text</code>	Выдаёт имя правила сортировки для переданного значения. Это значение может быть заключено в кавычки и дополнено схемой. Если с выражением аргумента не связано правило сортировки, возвращается NULL. Если же правила сортировки не применимы для типа аргумента, происходит ошибка. Пример: <pre>SELECT collation for (description) FROM pg_description LIMIT 1; pg_collation_for ----- "default" SELECT collation for ('foo' COLLATE "de_DE"); pg_collation_for ----- "de_DE"</pre>
<code>to_regclass (text) → regclass</code>	Переводит в OID текстовое имя отношения. Подобный результат можно получить, приведя строку с именем к типу <code>regclass</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит NULL, а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regcollation (text) → regcollation</code>	Переводит в OID текстовое имя правила сортировки. Подобный результат можно получить, приведя строку с именем к типу <code>regcollation</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит NULL, а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regnamespace (text) → regnamespace</code>	Переводит в OID текстовое имя схемы. Подобный результат можно получить, приведя строку с именем к типу <code>regnamespace</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит NULL, а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regoper (text) → regoper</code>	Переводит в OID текстовое имя оператора. Подобный результат можно получить, приведя строку с именем к типу <code>regoper</code> (см. Раздел 8.19); однако если имя не будет распознано или окажется неоднозначным, эта функция возвратит NULL, а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regoperator (text) → regoperator</code>	Переводит в OID текстовое имя оператора с указанием типов параметров. Подобный результат можно получить, приведя строку с именем к типу <code>regoperator</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит NULL, а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regproc (text) → regproc</code>	

Функция	Описание
	Переводит в OID текстовое имя функции или процедуры. Подобный результат можно получить, приведя строку с именем к типу <code>regoperator</code> (см. Раздел 8.19); однако если имя не будет распознано или окажется неоднозначным, эта функция возвратит <code>NULL</code> , а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regprocedure (text) → regprocedure</code>	Переводит в OID текстовое имя функции или процедуры с указанием типов аргументов. Подобный результат можно получить, приведя строку с именем к типу <code>regprocedure</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит <code>NULL</code> , а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regprofile (text) → regprofile</code>	Переводит в OID текстовое имя профиля. Подобный результат можно получить, приведя строку с именем к типу <code>regprofile</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит <code>NULL</code> , а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regrole (text) → regrole</code>	Переводит в OID текстовое имя роли. Подобный результат можно получить, приведя строку с именем к типу <code>regrole</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит <code>NULL</code> , а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.
<code>to_regtype (text) → regtype</code>	Переводит в OID текстовое имя типа. Подобный результат можно получить, приведя строку с именем к типу <code>regtype</code> (см. Раздел 8.19); однако если имя не будет распознано, эта функция возвратит <code>NULL</code> , а не выдаст ошибку. Также, в отличие от приведения, данная функция не принимает в качестве входных данных числовой OID.

Многие из функций, восстанавливающих (декомпилирующих) объекты БД, принимают дополнительный флаг `pretty`, в котором можно передать `true` для визуального улучшения вывода. При этом будут убраны лишние скобки и добавлены пробельные символы, чтобы текст был более разборчивым. Преобразованный вывод легче читается, но формат по умолчанию с большей вероятностью будет успешно восприниматься будущими версиями PostgreSQL, поэтому для загрузки содержимого БД рекомендуется избегать использования визуально улучшенного формата. Если параметр `pretty` равен `false`, это равнозначно его отсутствию.

Таблица 9.76. Свойства столбца индекса

Имя	Описание
<code>asc</code>	Сортируется ли столбец по возрастанию при сканировании вперёд?
<code>desc</code>	Сортируется ли столбец по убыванию при сканировании вперёд?
<code>nulls_first</code>	Выдаются ли <code>NULL</code> в начале при сканировании вперёд?
<code>nulls_last</code>	Выдаются ли <code>NULL</code> в конце при сканировании вперёд?
<code>orderable</code>	Связан ли со столбцом некоторый порядок сортировки?
<code>distance_orderable</code>	Может ли столбец сканироваться по порядку оператором «расстояния», например, <code>ORDER BY столбец <-> константа</code> ?

Имя	Описание
returnable	Может ли значение столбца быть получено при сканировании только индекса?
search_array	Поддерживает ли столбец внутренними средствами поиск <code>столбец = ANY(массив)</code> ?
search_nulls	Поддерживает ли столбец поиск <code>IS NULL</code> и <code>IS NOT NULL</code> ?

Таблица 9.77. Свойства индекса

Имя	Описание
clusterable	Может ли индекс использоваться в команде <code>CLUSTER</code> ?
index_scan	Поддерживает ли индекс простое сканирование (не по битовой карте)?
bitmap_scan	Поддерживает ли индекс сканирование по битовой карте?
backward_scan	Может ли в процессе сканирования меняться направление (для поддержки перемещения курсора <code>FETCH BACKWARD</code> без необходимости материализации)?

Таблица 9.78. Свойства метода доступа индекса

Имя	Описание
can_order	Поддерживает ли метод доступа <code>ASC</code> , <code>DESC</code> и связанные ключевые слова в <code>CREATE INDEX</code> ?
can_unique	Поддерживает ли метод доступа уникальные индексы?
can_multi_col	Поддерживает ли метод доступа индексы по нескольким столбцам?
can_exclude	Поддерживает ли метод доступа ограничения-исключения?
can_include	Поддерживает ли метод доступа предложение <code>INCLUDE</code> команды <code>CREATE INDEX</code> ?

Таблица 9.79. Флаги параметров GUC

Флаг	Описание
EXPLAIN	Параметры с этим флагом включаются в команды <code>EXPLAIN (SETTINGS)</code> .
NO_SHOW_ALL	Параметры с этим флагом исключаются из команд <code>SHOW ALL</code> .
NO_RESET_ALL	Параметры с этим флагом исключаются из команд <code>RESET ALL</code> .
NOT_IN_SAMPLE	Параметры с этим флагом не включаются в <code>postgresql.conf</code> по умолчанию.
RUNTIME_COMPUTED	Параметры с этим флагом вычисляются во время выполнения.

В [Таблице 9.80](#) перечислены функции, связанные с идентификацией и адресацией объектов баз данных.

Таблица 9.80. Функции получения информации и адресации объектов

Функция	Описание
<code>pg_describe_object</code>	<code>(classid oid, objid oid, objsubid integer) → text</code> Выдаёт текстовое описание объекта БД, идентифицируемого по OID каталога, OID объекта и ID подобъекта (например, номер столбца в таблице; ID подобъекта равен нулю для объекта в целом). Это описание предназначено для человека и может переводиться, в зависимости от конфигурации сервера. Данная функция в особенности полезна, когда требуется узнать, что за объект хранится в каталоге <code>pg_depend</code> . Эта функция возвращает значение <code>NULL</code> для неопределённых объектов.
<code>pg_identify_object</code>	<code>(classid oid, objid oid, objsubid integer) → record (type text, schema text, name text, identity text)</code> Выдаёт запись, содержащую достаточно информации для однозначной идентификации объекта БД по OID каталога, OID объекта и ID подобъекта. Эта информация предназначена для машины и поэтому никогда не переводится. Столбец <code>type</code> содержит тип объекта БД; <code>schema</code> — имя схемы, к которой относится объект (либо <code>NULL</code> для объектов, не относящихся к схемам); <code>name</code> — имя объекта, при необходимости в кавычках, которое присутствует, только если оно (возможно, вместе со схемой) однозначно идентифицирует объект (в противном случае <code>NULL</code>); <code>identity</code> — полный идентификатор объекта, точный формат которого зависит от типа объекта, а каждая его часть дополняется схемой и заключается в кавычки, если требуется. Для неопределённых объектов эти столбцы содержат значение <code>NULL</code> .
<code>pg_identify_object_as_address</code>	<code>(classid oid, objid oid, objsubid integer) → record (type text, object_names text[], object_args text[])</code> Выдаёт запись, содержащую достаточно информации для однозначной идентификации объекта БД по OID каталога, OID объекта и ID подобъекта. Выдаваемая информация не зависит от текущего сервера, то есть по ней можно идентифицировать одноимённый объект на другом сервере. Поле <code>type</code> содержит тип объекта БД, а <code>object_names</code> и <code>object_args</code> — текстовые массивы, в совокупности формирующие ссылку на объект. Эти три значения можно передать функции <code>pg_get_object_address</code> , чтобы получить внутренний адрес объекта.
<code>pg_get_object_address</code>	<code>(type text, object_names text[], object_args text[]) → record (classid oid, objid oid, objsubid integer)</code> Выдаёт запись, содержащую достаточно информации для уникальной идентификации объекта БД по коду типа и массивам имён и аргументов. В ней возвращаются значения, которые используются в системных каталогах, например <code>pg_depend</code> , и могут передаваться в другие системные функции, например <code>pg_describe_object</code> или <code>pg_identify_object</code> . Поле <code>classid</code> содержит OID системного каталога, к которому относится объект; <code>objid</code> — OID самого объекта, а <code>objsubid</code> — идентификатор подобъекта или 0 в случае его отсутствия. Эта функция является обратной к <code>pg_identify_object_as_address</code> . Для неопределённых объектов эти поля содержат значение <code>NULL</code> .

Функции, перечисленные в [Таблице 9.81](#), извлекают комментарии, заданные для объектов с помощью команды `COMMENT`. Если найти комментарий для заданных параметров не удаётся, они возвращают `NULL`.

Таблица 9.81. Функции получения комментариев

Функция	Описание
<code>col_description</code>	<code>(table oid, column integer) → text</code> Выдаёт комментарий для столбца с заданным номером в таблице с указанным OID. (<code>obj_description</code> нельзя использовать для столбцов таблицы, так столбцы не имеют собственных OID.)

Функция	Описание
<code>obj_description (object oid, catalog name) → text</code>	Выдаёт комментарий для объекта, имеющего заданный OID и находящегося в указанном системном каталоге. Например, в результате вызова <code>obj_description(123456, 'pg_class')</code> будет получен комментарий для таблицы с OID 123456.
<code>obj_description (object oid) → text</code>	Выдаёт комментарий для объекта, заданного только по OID. Эта вариация считается <i>устаревшей</i> , так как нет гарантии, что OID будет уникальным в разных системных каталогах, и поэтому может быть получен неправильный комментарий.
<code>shobj_description (object oid, catalog name) → text</code>	Выдаёт комментарий для общего объекта баз данных, имеющего заданный OID и находящегося в указанном системном каталоге. Эта функция аналогична <code>obj_description</code> , за исключением того, что она извлекает комментарий для общих объектов (то есть, баз данных, ролей и табличных пространств). Некоторые системные каталоги являются глобальными для всех баз данных в кластере и описания объектов в них также хранятся глобально.

Функции, показанные в [Таблице 9.82](#), выдают информацию о транзакциях сервера в виде, подходящем для экспорта. В основном эти функции используются, чтобы определить, какие транзакции были зафиксированы между двумя снимками состояния.

Таблица 9.82. Функции получения информации об идентификаторах транзакций и снимках состояния

Функция	Описание
<code>age (xid) → integer</code>	Возвращает количество транзакций между указанным идентификатором транзакции и текущим значением счётчика транзакций.
<code>mxid_age (xid) → integer</code>	Возвращает количество идентификаторов мультитранзакций между указанным идентификатором мультитранзакций и текущим значением счётчика мультитранзакций.
<code>pg_current_xact_id () → xid8</code>	Выдаёт идентификатор текущей транзакции. Если у текущей транзакции ещё нет идентификатора (она не успела выполнить какие-либо изменения в базе), он будет ей назначен.
<code>pg_current_xact_id_if_assigned () → xid8</code>	Выдаёт идентификатор текущей транзакции или NULL, если она ещё не имеет идентификатора. (Если транзакция может оставаться только читающей, лучше использовать эту вариацию, чтобы избежать излишнего потребления XID.)
<code>pg_xact_status (xid8) → text</code>	Выдаёт состояние фиксации транзакции. Результатом будет одно из значений: <code>in progress</code> (выполняется), <code>committed</code> (зафиксирована) и <code>aborted</code> (прервана), если только транзакция не настолько давняя, чтобы система удалила информацию о её состоянии. Если же упоминаний о транзакции в системе не осталось и информация о её фиксации потеряна, эта функция возвращает NULL. Приложения могут использовать эту функцию, например, чтобы определить, была ли транзакция зафиксирована или прервана, после потери подключения к серверу в процессе выполнения COMMIT. Заметьте, что для подготовленных транзакций возвращается состояние <code>in progress</code> ; приложения должны проверять <code>pg_prepared_xacts</code> , если им нужно определить, является ли транзакция с заданным ID подготовленной.

Функция	Описание
<code>pg_current_snapshot ()</code> $\rightarrow$ <code>pg_snapshot</code>	Возвращает текущий <i>снимок состояния</i> — структуру данных, показывающую, какие транзакции выполняются в момент снимка.
<code>pg_snapshot_xip (pg_snapshot)</code> $\rightarrow$ <code>setof xid8</code>	Выдаёт набор идентификаторов выполняющихся транзакций, содержащихся в снимке.
<code>pg_snapshot_xmax (pg_snapshot)</code> $\rightarrow$ <code>xid8</code>	Выдаёт значение <code>xmax</code> для заданного снимка.
<code>pg_snapshot_xmin (pg_snapshot)</code> $\rightarrow$ <code>xid8</code>	Выдаёт <code>xmin</code> для заданного снимка.
<code>pg_visible_in_snapshot (xid8, pg_snapshot)</code> $\rightarrow$ <code>boolean</code>	Видна ли транзакция с указанным идентификатором в данном снимке (то есть, была ли она завершена до момента получения снимка)? Заметьте, что эта функция не выдаст правильный ответ, если ей передать идентификатор подтранзакции.
<code>pg_get_multixact_members (multixid xid)</code> $\rightarrow$ <code>setof record (xid xid, mode text)</code>	Выдаёт идентификатор транзакции и режима блокировки для каждого члена с заданным идентификатором мультитранзакции. Режимы блокировок <code>forupd</code> , <code>fornokeyupd</code> , <code>sh</code> и <code>keysh</code> соответствуют блокировкам уровня строк <code>FOR UPDATE</code> , <code>FOR NO KEY UPDATE</code> , <code>FOR SHARE</code> и <code>FOR KEY SHARE</code> соответственно, более подробная информация предоставлена в Подразделе 13.3.2 . Ещё два режима относятся только к мультитранзакциям: <code>nokeyupd</code> , используется для обновлений, не меняющий ключевые столбцы, и <code>upd</code> , используется для обновлений и удалений, которые затрагивают ключевые столбцы.

Внутренний тип идентификаторов транзакций, `xid`, имеет размер 32 бита и значения в нём повторяются через каждые 4 миллиарда транзакций. Однако функции, показанные в [Таблице 9.82](#), за исключением `age`, `mxid_age` и `pg_get_multixact_members`, используют 64-битный тип `xid8`, значения которого не повторяются на протяжении всей жизни сервера и могут быть приведены к типу `xid`, если требуется. Тип данных `pg_snapshot` содержит информацию о видимости транзакций в определённый момент времени. Его компоненты описаны в [Таблице 9.83](#). В текстовом виде `pg_snapshot` представляется как `xmin:xmax:список_xip`. Например, запись `10:20:10,14,15` означает `xmin=10`, `xmax=20`, `xip_list=10, 14, 15`.

Таблица 9.83. Состав информации о снимке

Имя	Описание
<code>xmin</code>	Минимальный идентификатор транзакции среди всех активных. Все транзакции, идентификаторы которых меньше <code>xmin</code> , уже либо зафиксированы и видимы, либо отменены и «мертвы».
<code>xmax</code>	Идентификатор на один больше идентификатора последней завершённой транзакции. Все транзакции, идентификаторы которых больше или равны <code>xmax</code> , на момент получения снимка ещё не завершены и не являются видимыми.
<code>xip_list</code>	Транзакции, выполняющиеся в момент получения снимка. Транзакции с такими идентификаторами, для которых <code>xmin</code> $\leq$ <i>ид</i> $<$ <code>xmax</code> , не попавшие в этот список, были уже завершены на момент получения снимка, и поэтому либо видимы, либо «мертвы», в зависимости от варианта завершения. Идентификаторы подтранзакций в этот список не включаются.

До версии 13 в Postgres Pro отсутствовал тип `xid8`, поэтому предлагались вариации этих функций, в которых 64-битные XID представлялись в типе `bigint`, а для информации о снимке использовался отдельный тип `txid_snapshot`. В названиях этих старых функций фигурирует `txid`. Они по-прежнему поддерживаются для обратной совместимости, но могут быть удалены в будущем. См. [Таблицу 9.84](#).

Таблица 9.84. Устаревшие функции получения информации об идентификаторах транзакций и снимках состояния

Функция	Описание
<code>txid_current ()</code>	<code>→ bigint</code> См. <code>pg_current_xact_id()</code> .
<code>txid_current_if_assigned ()</code>	<code>→ bigint</code> См. <code>pg_current_xact_id_if_assigned()</code> .
<code>txid_current_snapshot ()</code>	<code>→ txid_snapshot</code> См. <code>pg_current_snapshot()</code> .
<code>txid_snapshot_xip (txid_snapshot)</code>	<code>→ setof bigint</code> См. <code>pg_snapshot_xip()</code> .
<code>txid_snapshot_xmax (txid_snapshot)</code>	<code>→ bigint</code> См. <code>pg_snapshot_xmax()</code> .
<code>txid_snapshot_xmin (txid_snapshot)</code>	<code>→ bigint</code> См. <code>pg_snapshot_xmin()</code> .
<code>txid_visible_in_snapshot (bigint, txid_snapshot)</code>	<code>→ boolean</code> См. <code>pg_visible_in_snapshot()</code> .
<code>txid_status (bigint)</code>	<code>→ text</code> См. <code>pg_xact_status()</code> .

Функции, показанные в [Таблице 9.85](#), выдают информацию о времени фиксирования уже завершённых транзакций. Полезным результатом этих функций будет, только когда включён параметр конфигурации [track_commit_timestamp](#) и только для транзакций, зафиксированных после его включения.

Таблица 9.85. Функции получения информации о фиксации транзакций

Функция	Описание
<code>pg_xact_commit_timestamp (xid)</code>	<code>→ timestamp with time zone</code> Выдаёт время фиксации транзакции.
<code>pg_xact_commit_timestamp_origin (xid)</code>	<code>→ record (timestamp timestamp with time zone, <i>roident oid</i>)</code> Выдаёт время фиксации и источник репликации транзакции.
<code>pg_last_committed_xact ()</code>	<code>→ record (xidxid, timestamp timestamp with time zone, <i>roident oid</i>)</code> Выдаёт идентификатор, время фиксации и источник репликации транзакции, зафиксированной последней.

Функции, показанные в [Таблице 9.86](#), выдают информацию, записываемую во время `initdb`, например, версию каталога. Они также выводят сведения о журнале предзаписи и контрольных точках. Эта информация относится ко всему кластеру, а не к отдельной базе. Данные функции выдают практически ту же информацию, и из того же источника, что и [pg_controldata](#).

Таблица 9.86. Функции управления данными

Функция	Описание
<code>pg_control_checkpoint</code> () → record	Выдаёт информацию о текущем состоянии контрольных точек, показанную в Таблице 9.87 .
<code>pg_control_system</code> () → record	Выдаёт информацию о текущем состоянии управляющего файла, показанную в Таблице 9.88 .
<code>pg_control_init</code> () → record	Выдаёт информацию о состоянии инициализации кластера, показанную в Таблице 9.89 .
<code>pg_control_recovery</code> () → record	Выдаёт информацию о состоянии восстановления, как показано в Таблице 9.90 .

Таблица 9.87. Столбцы результата `pg_control_checkpoint`

Имя столбца	Тип данных
<code>checkpoint_lsn</code>	<code>pg_lsn</code>
<code>redo_lsn</code>	<code>pg_lsn</code>
<code>redo_wal_file</code>	<code>text</code>
<code>timeline_id</code>	<code>integer</code>
<code>prev_timeline_id</code>	<code>integer</code>
<code>full_page_writes</code>	<code>boolean</code>
<code>next_xid</code>	<code>text</code>
<code>next_oid</code>	<code>oid</code>
<code>next_multixact_id</code>	<code>xid</code>
<code>next_multi_offset</code>	<code>xid</code>
<code>oldest_xid</code>	<code>xid</code>
<code>oldest_xid_dbid</code>	<code>oid</code>
<code>oldest_active_xid</code>	<code>xid</code>
<code>oldest_multi_xid</code>	<code>xid</code>
<code>oldest_multi_dbid</code>	<code>oid</code>
<code>oldest_commit_ts_xid</code>	<code>xid</code>
<code>newest_commit_ts_xid</code>	<code>xid</code>
<code>checkpoint_time</code>	<code>timestamp with time zone</code>

Таблица 9.88. Столбцы результата `pg_control_system`

Имя столбца	Тип данных
<code>pg_control_version</code>	<code>integer</code>
<code>catalog_version_no</code>	<code>integer</code>
<code>system_identifier</code>	<code>bigint</code>
<code>pg_control_last_modified</code>	<code>timestamp with time zone</code>
<code>pg_control_edition</code>	<code>text</code>

Таблица 9.89. Столбцы результата pg_control_init

Имя столбца	Тип данных
max_data_alignment	integer
database_block_size	integer
blocks_per_segment	integer
wal_block_size	integer
bytes_per_wal_segment	integer
max_identifier_length	integer
max_index_columns	integer
max_toast_chunk_size	integer
large_object_chunk_size	integer
float8_pass_by_value	boolean
data_page_checksum_version	integer

Таблица 9.90. Столбцы результата pg_control_recovery

Имя столбца	Тип данных
min_recovery_end_lsn	pg_lsn
min_recovery_end_timeline	integer
backup_start_lsn	pg_lsn
backup_end_lsn	pg_lsn
end_of_backup_record_required	boolean

9.27. Функции для системного администрирования

Функции, описанные в этом разделе, предназначены для контроля и управления сервером PostgreSQL Pro.

9.27.1. Функции для управления конфигурацией

В [Таблице 9.91](#) показаны функции, позволяющие получить и изменить значения параметров конфигурации выполнения.

Таблица 9.91. Функции для управления конфигурацией

Функция Описание Пример(ы)
<code>current_setting (setting_name text [, missing_ok boolean]) → text</code> Выдаёт текущее значение параметра <code>setting_name</code> . Если такого параметра нет, <code>current_setting</code> выдаёт ошибку, если только дополнительно не передан параметр <code>missing_ok</code> со значением <code>true</code> (в этом случае выдаётся NULL). Эта функция соответствует SQL-команде SHOW. <code>current_setting('datestyle') → ISO, MDY</code>
<code>set_config (setting_name text, new_value text, is_local boolean) → text</code> Устанавливает для параметра <code>setting_name</code> значение <code>new_value</code> и возвращает это значение. Если параметр <code>is_local</code> равен <code>true</code>, новое значение будет действовать только в рамках текущей транзакции. Чтобы это значение действовало на протяжении текущего сеанса, присвойте этому параметру <code>false</code>. Эта функция соответствует SQL-команде SET. <code>set_config('log_statement_stats', 'off', false) → off</code>

9.27.2. Функции для передачи сигналов серверу

Функции, перечисленные в [Таблице 9.92](#), позволяют передавать управляющие сигналы другим серверным процессам. Вызывать эти функции по умолчанию разрешено только суперпользователям, но доступ к ним можно дать и другим пользователям командой `GRANT`, кроме явно отмеченных исключений.

Все эти функции возвращают `true`, если сигнал успешно отправлен, и `false`, если отправить сигнал не удалось.

Таблица 9.92. Функции для передачи сигналов серверу

Функция	Описание
<code>pg_cancel_backend (pid integer) → boolean</code>	Отменяет текущий запрос в сеансе, который обслуживается процессом с заданным PID. Это действие разрешается и ролям, являющимся членами роли, запрос которой отменяется, и ролям с правами роли <code>pg_signal_backend</code> ; однако только суперпользователям разрешено отменять запросы других суперпользователей.
<code>pg_log_backend_memory_contexts (pid integer) → boolean</code>	Запрашивает вывод в журнал информации о контекстах памяти обслуживающего процесса с указанным PID. Эта функция может отправлять запрос обслуживающим и вспомогательным процессам, кроме процесса протоколирования. Запрошенная информация будет выведена в сообщениях уровня <code>LOG</code> , которые появятся в журнале сервера в зависимости от заданной конфигурации журнала (за дополнительными сведениями обратитесь к Разделу 18.8), но не будут передаваться клиенту независимо от <code>client_min_messages</code> .
<code>pg_reload_conf () → boolean</code>	Даёт всем процессам сервера Postgres Pro команду перезагрузить файлы конфигурации. (Для этого посылается сигнал <code>SIGHUP</code> главному процессу, который, в свой очередь, посылает <code>SIGHUP</code> всем своим дочерним процессам.) Вы можете прочитать представления <code>pg_file_settings</code> , <code>pg_hba_file_rules</code> и <code>pg_ident_file_mappings</code> , чтобы проверить файлы конфигурации на предмет возможных ошибок перед перезагрузкой.
<code>pg_rotate_logfile () → boolean</code>	Указывает менеджеру журнала сообщений немедленно переключиться на новый файл. Это имеет смысл, только когда работает встроенный сборщик сообщений, так как без него подпроцесс менеджера журнала не запускается.
<code>pg_terminate_backend (pid integer, timeout bigint DEFAULT 0) → boolean</code>	Завершает сеанс, который обслуживается процессом с заданным PID. Это действие разрешается и ролям, являющимся членами роли, процесс которой прерывается, и ролям с правами роли <code>pg_signal_backend</code> ; однако только суперпользователям разрешено прерывать обслуживающие процессы других суперпользователей. Если параметр <code>timeout</code> не задан или равен нулю, данная функция возвращает <code>true</code> независимо от того, действительно ли завершается процесс; то есть результат показывает только, что сигнал был отправлен успешно. Если параметр <code>timeout</code> задан (в миллисекундах) и больше нуля, функция ожидает, пока процесс не будет фактически завершён или пока не пройдёт заданное время. Если процесс завершён, функция возвращает <code>true</code> . В случае тайм-аута выдаётся предупреждение и возвращается <code>false</code> .

`pg_cancel_backend` и `pg_terminate_backend` передают сигналы (`SIGINT` и `SIGTERM`, соответственно) серверному процессу с заданным кодом PID. Код активного процесса можно получить из столбца `pid` представления `pg_stat_activity` или просмотрев на сервере процессы с именем `postgres` (используя `ps` в Unix или Диспетчер задач в Windows). Роль пользователя активного процесса можно узнать в столбце `username` представления `pg_stat_activity`.

`pg_log_backend_memory_contexts` может использоваться для получения в журнале сервера информации о контекстах памяти. Например:

```
postgres=# SELECT pg_log_backend_memory_contexts(pg_backend_pid());
pg_log_backend_memory_contexts
-----
 t
(1 row)
```

Для каждого контекста памяти будет выводиться одно сообщение. Например:

```
LOG:  logging memory contexts of PID 10377
STATEMENT:  SELECT pg_log_backend_memory_contexts(pg_backend_pid());
LOG:  level: 0; TopMemoryContext: 80800 total in 6 blocks; 14432 free (5 chunks); 66368
      used
LOG:  level: 1; pgstat TabStatusArray lookup hash table: 8192 total in 1 blocks; 1408
      free (0 chunks); 6784 used
LOG:  level: 1; TopTransactionContext: 8192 total in 1 blocks; 7720 free (1 chunks);
      472 used
LOG:  level: 1; RowDescriptionContext: 8192 total in 1 blocks; 6880 free (0 chunks);
      1312 used
LOG:  level: 1; MessageContext: 16384 total in 2 blocks; 5152 free (0 chunks); 11232
      used
LOG:  level: 1; Operator class cache: 8192 total in 1 blocks; 512 free (0 chunks); 7680
      used
LOG:  level: 1; smgr relation table: 16384 total in 2 blocks; 4544 free (3 chunks);
      11840 used
LOG:  level: 1; TransactionAbortContext: 32768 total in 1 blocks; 32504 free (0
      chunks); 264 used
...
LOG:  level: 1; ErrorContext: 8192 total in 1 blocks; 7928 free (3 chunks); 264 used
LOG:  Grand total: 1651920 bytes in 201 blocks; 622360 free (88 chunks); 1029560 used
```

Если у одного родителя более 100 дочерних контекстов, в журнале описываются первые 100 дочерних контекстов, а по остальным выдаётся краткая сводка. Обратите внимание, что частые вызовы этой функции могут создавать значительную нагрузку, так как она может записывать в журнал массу сообщений.

9.27.3. Функции управления резервным копированием

Функции, перечисленные в [Таблице 9.93](#), предназначены для выполнения резервного копирования «на ходу». Эти функции нельзя выполнять во время восстановления (за исключением `pg_backup_start`, `pg_backup_stop` и `pg_wal_lsn_diff`).

Подробнее практическое применение этих функций описывается в [Разделе 24.3](#).

Таблица 9.93. Функции управления резервным копированием

Функция	Описание
<code>pg_create_restore_point (name text) → pg_lsn</code>	Создаёт в журнале предзаписи именованный маркер, который можно использовать как цель при восстановлении, и возвращает соответствующую ему позицию в журнале. Полученное имя затем можно указать в параметре <code>recovery_target_name</code> , определив тем самым точку, до которой будет выполняться восстановление. Учтите, что если будет создано несколько точек восстановления с одним именем, восстановление остановится на первой точке с этим именем. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_current_wal_flush_lsn () → pg_lsn</code>	Выдаёт текущую позицию сброса данных в журнале предзаписи (см. примечания ниже).
<code>pg_current_wal_insert_lsn () → pg_lsn</code>	

Функция	Описание
	Выдаёт текущую позицию добавления в журнале предзаписи (см. примечания ниже).
<code>pg_current_wal_lsn</code> () → <code>pg_lsn</code>	Выдаёт текущую позицию записи в журнале предзаписи (см. примечания ниже).
<code>pg_backup_start</code> (<i>label text</i> [, <i>fast boolean</i>]) → <code>pg_lsn</code>	Подготавливает сервер к резервному копированию «на лету». Единственный обязательный параметр задаёт произвольную пользовательскую метку резервной копии. (Обычно это имя, которое получит файл резервной копии.) Если необязательный второй параметр задан и имеет значение <code>true</code>, функция <code>pg_backup_start</code> должна выполняться максимально быстро. Это означает, что принудительно будет выполнена контрольная точка, вследствие чего кратковременно увеличится нагрузка на ввод-вывод и параллельно выполняемые запросы могут замедлиться. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_backup_stop</code> ([<i>wait_for_archive boolean</i>]) → <code>record (lsn pg_lsn , label file text , spcmap file text)</code>	Завершает выполнение резервного копирования. В результате возвращается ожидаемое содержимое файла метки резервной копии и файла карты табличных пространств, которое нужно сохранить в файлы в архиве (не в каталоге данных). Эти файлы нельзя записывать в каталог работающей базы данных (в противном случае PostgreSQL не сможет перезапуститься, если произойдёт сбой). У этой функции есть также необязательный параметр типа <code>boolean</code>. Если он равен <code>false</code>, <code>pg_backup_stop</code> завершится сразу после окончания резервного копирования, не дожидаясь архивации WAL. Это поведение полезно только для программ резервного копирования, которые осуществляют архивацию WAL независимо. Если же WAL не будет заархивирован вовсе, резервная копия может оказаться неполной, и, как следствие, непригодной для восстановления. Когда он равен <code>true</code> (по умолчанию), <code>pg_backup_stop</code> будет ждать выполнения архивации WAL, если архивирование включено. Для резервного сервера это означает, что ожидание возможно только при условии <code>archive_mode = always</code>. Если активность записи на ведущем сервере невысока, может иметь смысл выполнить на нём <code>pg_switch_wal</code> для немедленного переключения сегмента. При выполнении на ведущем эта функция также создаёт файл истории резервного копирования в области архива журнала предзаписи. В этом файле для данной резервной копии сохраняется метка, заданная при вызове <code>pg_backup_start</code> , начальная и конечная позиция в журнале предзаписи, а также время начала и окончания копирования. После записи конечной позиции текущая позиция автоматически перемещается к следующему файлу журнала предзаписи, чтобы файл конечной позиции можно было немедленно архивировать для завершения резервного копирования. В результате эта функция выдаёт единственную запись. Столбец <code>lsn</code> в ней содержит позицию завершения копирования в журнале предзаписи (её также можно игнорировать). Во втором столбце передаётся содержимое файла метки резервной копии, а в третьем — содержимое файла карты табличных пространств. Эту информацию нужно сохранить в составе резервной копии, она потребуется в процессе восстановления. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_switch_wal</code> () → <code>pg_lsn</code>	Производит принудительное переключение журнала предзаписи на новый файл, что позволяет архивировать текущий (в предположении, что выполняется непрерывная архивация). Результат функции — конечная позиция в только что законченном файле журнала предзаписи + 1. Если с момента последнего переключения файлов не было активности, отражающейся в журнале предзаписи, <code>pg_switch_wal</code> не делает ничего и возвращает начальную позицию в файле журнала предзаписи, используемом в данный момент.

Функция	Описание
<code>pg_walfile_name (lsn pg_lsn) → text</code>	По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_walfile_name_offset (lsn pg_lsn) → record (file_name text, file_offset integer)</code>	Выдаёт для заданной позиции в журнале предзаписи имя соответствующего файла WAL.
<code>pg_wal_lsn_diff (lsn1 pg_lsn , lsn2 pg_lsn) → numeric</code>	Выдаёт для заданной позиции в журнале предзаписи имя соответствующего файла и байтовое смещение в нём.
<code>pg_wal_lsn_diff (lsn1 pg_lsn , lsn2 pg_lsn) → numeric</code>	Вычисляет разницу в байтах ($lsn1 - lsn2$) между двумя позициями в журнале предзаписи. Полученный результат можно использовать с <code>pg_stat_replication</code> или с некоторыми функциями, перечисленными в Таблица 9.93 , для определения задержки репликации.

`pg_current_wal_lsn` выводит текущую позицию записи в журнале предзаписи в том же формате, что и вышеописанные функции. `pg_current_wal_insert_lsn` подобным образом выводит текущую позицию добавления, а `pg_current_wal_flush_lsn` — позицию сброса данных журнала. Позицией добавления называется «логический» конец журнала предзаписи в любой момент времени, тогда как позиция записи указывает на конец данных, фактически вынесённых из внутренних буферов сервера, а позиция сброса показывает, до какого места данные считаются сохранёнными в надёжном хранилище. Позиция записи отмечает конец данных, которые может видеть снаружи внешний процесс, и именно она представляет интерес при копировании частично заполненных файлов журнала. Позиция добавления и позиция сброса выводятся в основном для отладки серверной части. Все эти функции выполняются в режиме «только чтение» и не требуют прав суперпользователя.

Используя функцию `pg_walfile_name_offset`, из значения `pg_lsn` можно получить имя соответствующего ему файла журнала предзаписи и байтовое смещение в этом файле. Например:

```
postgres=# SELECT * FROM pg_walfile_name_offset((pg_backup_stop()).lsn);
      file_name      | file_offset
-----+-----
000000010000000000000000D |      4039624
(1 row)
```

Родственная ей функция `pg_walfile_name` извлекает только имя файла журнала предзаписи. Когда заданная позиция в журнале предзаписи находится ровно на границе файлов, обе эти функции возвращают имя предыдущего файла. Обычно это поведение предпочтительно при архивировании журнала предзаписи, так как именно предыдущий файл является последним подлежащим архивации.

9.27.4. Функции управления восстановлением

Функции, показанные в [Таблице 9.94](#), предоставляют сведения о текущем состоянии ведомого сервера. Эти функции могут выполняться как во время восстановления, так и в обычном режиме работы.

Таблица 9.94. Функции для получения информации о восстановлении

Функция	Описание
<code>pg_is_in_recovery () → boolean</code>	Возвращает true, если в данный момент выполняется процедура восстановления.
<code>pg_last_wal_receive_lsn () → pg_lsn</code>	Выдаёт позицию последней записи в журнале предзаписи, которая была получена и записана на диск в процессе потоковой репликации. Пока выполняется потоковая репликация, эта позиция постоянно увеличивается. По окончании восстановления она остаётся

Функция	Описание
	ся на записи WAL, полученной и записанной на диск последней. Если потоковая репликация отключена или ещё не запускалась, функция возвращает NULL.
<code>pg_last_wal_replay_lsn</code> <code>() → pg_lsn</code>	Выдаёт позицию последней записи в журнале предзаписи, которая была воспроизведена при восстановлении. В процессе восстановления эта позиция постоянно увеличивается. По окончании этого процесса она остаётся на записи WAL, которая была восстановлена последней. Если сервер при запуске не выполнял процедуру восстановления, эта функция выдаёт NULL.
<code>pg_last_xact_replay_timestamp</code> <code>() → timestamp with time zone</code>	Выдаёт отметку времени последней транзакции, воспроизведённой при восстановлении. Это время, когда на главном сервере произошла фиксация или откат записи WAL для этой транзакции. Если в процессе восстановления не была воспроизведена ни одна транзакция, эта функция выдаёт NULL. В противном случае возвращаемое значение постоянно увеличивается в процессе восстановления. По окончании восстановления в нём остаётся время транзакции, которая была восстановлена последней. Если сервер при запуске не выполнял процедуру восстановления, эта функция выдаёт NULL.
<code>pg_get_wal_resource_managers</code> <code>() → setof record (<i>rm_id</i> integer, <i>rm_name</i> text, <i>rm_builtin</i> boolean)</code>	Выдаёт загруженные на данный момент менеджеры ресурсов WAL в системе. Столбец <i>rm_builtin</i> показывает, является ли данный менеджер ресурсов встроенным или пользовательским, то есть создаваемым расширением.

Функции, перечисленные в [Таблице 9.95](#) управляют процессом восстановления. Вызывать их в другое время нельзя.

Таблица 9.95. Функции управления восстановлением

Функция	Описание
<code>pg_is_wal_replay_paused</code> <code>() → boolean</code>	Возвращает true, если запрошена приостановка восстановления.
<code>pg_get_wal_replay_pause_state</code> <code>() → text</code>	Возвращает состояние приостановки восстановления. Возвращаемые значения: <code>not paused</code> — приостановка не запрашивалась, <code>pause requested</code> — получен запрос на приостановку, но восстановление ещё не приостановлено, и <code>paused</code> , если восстановление действительно приостановлено.
<code>pg_promote</code> (<i>wait</i> boolean DEFAULT true, <i>wait_seconds</i> integer DEFAULT 60) <code>→ boolean</code>	Повышает статус ведомого сервера до ведущего. Если параметр <i>wait</i> равен true (по умолчанию), эта функция дожидается завершения операции повышения в течение <i>wait_seconds</i> секунд и возвращает true в случае успешного повышения или false в противном случае. Если параметр <i>wait</i> равен false, функция возвращает true сразу после передачи процессу postmaster сигнала SIGUSR1, инициирующего повышение. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_wal_replay_pause</code> <code>() → void</code>	Запрашивает приостановку восстановления. Запрос приостановки может быть выполнен не сразу. Если вы хотите убедиться, что восстановление действительно приостановлено, проверьте состояние приостановки восстановления с помощью функции <code>pg_get_wal_replay_pause_state()</code> . Заметьте, что <code>pg_get_wal_replay_paused</code> показывает только был ли сделан запрос. Когда восстановление приостановлено, запись изменений в базу не производится. В режиме «горячего резерва» все последующие запросы

Функция	Описание
	будут видеть один согласованный снимок базы данных, что исключает конфликты запросов до продолжения восстановления. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_wal_replay_resume</code> () → void	Возобновляет восстановление, если оно было приостановлено. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.

Функции `pg_wal_replay_pause` и `pg_wal_replay_resume` нельзя выполнять в процессе повышения. Если повышение запрашивается, когда восстановление приостановлено, сервер выходит из состояния паузы и продолжает процедуру повышения.

Когда потоковая репликация отключена, пауза при восстановлении может длиться сколь угодно долго без каких-либо проблем. Если же потоковая репликация осуществляется, новые записи WAL продолжают поступать и заполняют весь диск рано или поздно, в зависимости от длительности паузы, интенсивности записи в WAL и объема свободного пространства.

9.27.5. Функции синхронизации снимков

Postgres Pro позволяет синхронизировать снимки состояния между сеансами баз данных. *Снимок состояния* определяет, какие данные видны транзакции, работающей с этим снимком. Синхронизация снимков необходима, когда в двух или более сеансах нужно видеть одно и то же содержимое базы данных. Если в двух сеансах транзакции запускаются независимо, всегда есть вероятность, что некая третья транзакция будет зафиксирована между командами `START TRANSACTION` для первых двух, и в результате в одном сеансе будет виден результат третьей, а в другом — нет.

Для решения этой проблемы Postgres Pro позволяет транзакции *экспортировать* снимок состояния, с которым она работает. Пока экспортирующая этот снимок транзакция выполняется, другие транзакции могут *импортировать* его и, таким образом, увидеть абсолютно то же состояние базы данных, что видит первая транзакция. Но учтите, что любые изменения, произведённые этими транзакциями, будут не видны для других, как это и должно быть с изменениями в незафиксированных транзакциях. Таким образом, транзакции синхронизируют только начальное состояние данных, а последующие производимые в них изменения изолируются как обычно.

Снимки состояния экспортируются с помощью функции `pg_export_snapshot`, показанной в [Таблице 9.96](#), и импортируются командой [SET TRANSACTION](#).

Таблица 9.96. Функции синхронизации снимков

Функция	Описание
<code>pg_export_snapshot</code> () → text	Сохраняет текущий снимок состояния транзакции и возвращает строку типа <code>text</code> с идентификатором этого снимка. Эта строка должна передаваться внешними средствами клиентам, которые будут импортировать этот снимок. Снимок может быть импортирован только при условии, что экспортировавшая его транзакция ещё не завершена. Если необходимо, транзакция может экспортировать несколько снимков. Заметьте, что это имеет смысл только для транзакций уровня <code>READ COMMITTED</code> , так как транзакции уровня изоляции <code>REPEATABLE READ</code> и выше работают с одним снимком состояния на протяжении всего своего существования. После того как транзакция экспортировала снимок, её нельзя сделать подготовленной с помощью PREPARE TRANSACTION .

9.27.6. Функции управления репликацией

В [Таблице 9.97](#) показаны функции, предназначенные для управления механизмом репликации и взаимодействия с ним. Чтобы изучить этот механизм подробнее, обратитесь к [Подразделу 25.2.5](#),

[Подразделу 25.2.6](#) и [Главе 49](#). По умолчанию только суперпользователи имеют разрешение выполнять функции с источниками репликации, но его можно дать и другим, воспользовавшись командой `GRANT`. Использование данных функций для слотов репликации разрешено только суперпользователям и пользователям, имеющим право `REPLICATION`.

Многие из этих функций соответствуют командам в протоколе репликации; см. [Раздел 54.4](#).

Функции, описанные в [Подразделе 9.27.3](#), [Подразделе 9.27.4](#) и [Подразделе 9.27.5](#), также имеют отношение к репликации.

Таблица 9.97. Функции управления репликацией

Функция	Описание
<code>pg_create_physical_replication_slot</code>	<code>(slot_name name [, immediately_reserve boolean, temporary boolean]) → record (slot_name name, lsn pg_lsn)</code> Создаёт новый физический слот репликации с именем <code>slot_name</code> . Необязательный второй параметр, когда он равен <code>true</code> , указывает, что LSN для этого слота репликации должен быть зарезервирован немедленно; в противном случае LSN резервируется при первом подключении клиента потоковой репликации. Передача изменений из физического слота возможна только по протоколу потоковой репликации — см. Раздел 54.4 . Необязательный третий параметр, <code>temporary</code> , когда он равен <code>true</code> , указывает, что этот слот не должен постоянно храниться на диске, так как он предназначен только для текущего сеанса. Временные слоты также освобождаются при любой ошибке. Эта функция соответствует команде протокола репликации <code>CREATE_REPLICATION_SLOT ... PHYSICAL</code> .
<code>pg_drop_replication_slot</code>	<code>(slot_name name) → void</code> Удаляет физический или логический слот репликации с именем <code>slot_name</code> . Соответствует команде протокола репликации <code>DROP_REPLICATION_SLOT</code> .
<code>pg_create_logical_replication_slot</code>	<code>(slot_name name, plugin name [, temporary boolean, twophase boolean]) → record (slot_name name, lsn pg_lsn)</code> Создаёт новый логический (декодирующий) слот репликации с именем <code>slot_name</code> , используя модуль вывода <code>plugin</code> . Необязательный третий параметр, <code>temporary</code> , когда равен <code>true</code> , указывает, что этот слот не должен постоянно храниться на диске, так как предназначен только для текущего сеанса. Временные слоты также освобождаются при любой ошибке. Необязательный четвёртый параметр, <code>twophase</code> , когда равен <code>true</code> , указывает, что для данного слота включается декодирование подготовленных транзакций. Вызов этой функции равнозначен выполнению команды протокола репликации <code>CREATE_REPLICATION_SLOT ... LOGICAL</code> .
<code>pg_copy_physical_replication_slot</code>	<code>(src_slot_name name, dst_slot_name name [, temporary boolean]) → record (slot_name name, lsn pg_lsn)</code> Копирует существующий слот физической репликации с именем <code>src_slot_name</code> в слот с именем <code>dst_slot_name</code> . Скопированный слот физической репликации начинает резервировать WAL с того же последовательного номера LSN, что и исходный слот. Параметр <code>temporary</code> является необязательным и позволяет указать, будет ли слот временным. Если параметр <code>temporary</code> опущен, сохраняется то же свойство, что имеет исходный слот.
<code>pg_copy_logical_replication_slot</code>	<code>(src_slot_name name, dst_slot_name name [, temporary boolean [, plugin name]]) → record (slot_name name, lsn pg_lsn)</code> Копирует существующий слот логической репликации с именем <code>src_slot_name</code> в слот с именем <code>dst_slot_name</code> , с возможностью смены модуля вывода и свойства временности. Скопированный логический слот начинает передачу с того же последовательного номера LSN, что и исходный слот. Параметры <code>temporary</code> и <code>plugin</code> являются необязательными; если они опущены, сохраняются свойства исходного слота.
<code>pg_logical_slot_get_changes</code>	<code>(slot_name name, upto_lsn pg_lsn, upto_nchanges integer, VARIADIC options text[]) → setof record (lsn pg_lsn, xid xid, data text)</code>

Функция	Описание
	Возвращает изменения в слоте <i>slot_name</i> с позиции, до которой ранее были получены изменения. Если параметры <i>upto_lsn</i> и <i>upto_nchanges</i> равны NULL, логическое декодирование продолжится до конца журнала транзакций. Если <i>upto_lsn</i> не NULL, декодироваться будут только транзакции, зафиксированные до заданного LSN. Если <i>upto_nchanges</i> не NULL, декодирование остановится, когда число строк, полученных при декодировании, превысит заданное значение. Обратите внимание, что фактическое число возвращённых строк может быть больше, так как это ограничение проверяется только после добавления строк, декодированных для очередной транзакции.
<code>pg_logical_slot_peek_changes (slot_name name, upto_lsn pg_lsn , upto_nchanges integer, VARIADIC options text[]) → setof record (lsn pg_lsn , xid xid, data text)</code>	Работает так же, как функция <code>pg_logical_slot_get_changes()</code> , но не забирает изменения; то есть, они будут получены снова при следующих вызовах.
<code>pg_logical_slot_get_binary_changes (slot_name name, upto_lsn pg_lsn , upto_nchanges integer, VARIADIC options text[]) → setof record (lsn pg_lsn , xid xid, data bytea)</code>	Работает так же, как функция <code>pg_logical_slot_get_changes()</code> , но выдаёт изменения в типе <code>bytea</code> .
<code>pg_logical_slot_peek_binary_changes (slot_name name, upto_lsn pg_lsn , upto_nchanges integer, VARIADIC options text[]) → setof record (lsn pg_lsn , xid xid, data bytea)</code>	Работает аналогично функции <code>pg_logical_slot_peek_changes()</code> , но выдаёт изменения в значении <code>bytea</code> .
<code>pg_replication_slot_advance (slot_name name, upto_lsn pg_lsn) → record (slot_name name, end_lsn pg_lsn)</code>	Продвигает текущую подтверждённую позицию слота репликации с именем <i>slot_name</i> . Слот нельзя переместить назад, а также вперёд за текущую позицию добавления. Возвращает имя слота и позицию, до которой он фактически продвинулся. Информация об изменившемся положении слота, если он продвинулся, сохраняется только при ближайшей контрольной точке. Поэтому в случае прерывания работы слот может оказаться в предыдущем положении.
<code>pg_replication_origin_create (node_name text) → oid</code>	Создаёт источник репликации с заданным внешним именем и возвращает назначенный ему внутренний идентификатор.
<code>pg_replication_origin_drop (node_name text) → void</code>	Удаляет ранее созданный источник репликации, в том числе связанную информацию о воспроизведении.
<code>pg_replication_origin_oid (node_name text) → oid</code>	Ищет источник репликации по имени и возвращает внутренний идентификатор. Если такой источник не находится, возвращает NULL.
<code>pg_replication_origin_session_setup (node_name text) → void</code>	Помечает текущий сеанс как воспроизводящий изменения из указанного источника, что позволяет отслеживать процесс воспроизведения. Может применяться, только если в текущий момент источник ещё не выбран. Для отмены этого действия вызовите <code>pg_replication_origin_session_reset</code> .
<code>pg_replication_origin_session_reset () → void</code>	Отменяет действие <code>pg_replication_origin_session_setup()</code> .
<code>pg_replication_origin_session_is_setup () → boolean</code>	Возвращает true, если в текущем сеансе выбран источник репликации.

Функция	Описание
<code>pg_replication_origin_session_progress (flush boolean) → pg_lsn</code>	Возвращает позицию воспроизведения для источника репликации, выбранного в текущем сеансе. Параметр <i>flush</i> определяет, будет ли гарантироваться сохранение локальной транзакции на диске.
<code>pg_replication_origin_xact_setup (origin_lsn pg_lsn , origin_timestamp timestamp with time zone) → void</code>	Помечает текущую транзакцию как воспроизводящую транзакцию, зафиксированную с указанным LSN и временем. Может вызываться только после того, как источник репликации был выбран в результате вызова <code>pg_replication_origin_session_setup()</code> .
<code>pg_replication_origin_xact_reset () → void</code>	Отменяет действие <code>pg_replication_origin_xact_setup()</code> .
<code>pg_replication_origin_advance (node_name text, lsn pg_lsn) → void</code>	Устанавливает положение репликации для заданного узла в указанную позицию. Это в основном полезно для установки начальной позиции или новой позиции после изменения конфигурации и подобных действий. Но учтите, что неосторожное использование этой функции может привести к несогласованности реплицированных данных.
<code>pg_replication_origin_progress (node_name text, flush boolean) → pg_lsn</code>	Выдаёт позицию воспроизведения для заданного источника репликации. Параметр <i>flush</i> определяет, будет ли гарантироваться сохранение локальной транзакции на диске.
<code>pg_logical_emit_message (transactional boolean, prefix text, content text) → pg_lsn</code> <code>pg_logical_emit_message (transactional boolean, prefix text, content bytea) → pg_lsn</code>	Генерирует сообщение логического декодирования. Её можно использовать для передачи через WAL произвольных сообщений модулям логического декодирования. Параметр <i>transactional</i> определяет, должно ли сообщение относиться к текущей транзакции или оно должно записываться немедленно и декодироваться сразу, как только эта запись будет прочитана при логическом декодировании. Параметр <i>prefix</i> задаёт текстовый префикс, по которому модуль логического декодирования может легко распознать интересующие именно его сообщения. В параметре <i>content</i> передаётся содержимое сообщения, в текстовом или двоичном виде.

9.27.7. Функции управления объектами баз данных

Функции, показанные в [Таблице 9.98](#), вычисляют объём, который занимают на диске объекты базы данных, и помогают воспринять и проанализировать информацию об используемом объёме. Результаты типа `bigint` представляют значения в байтах. Если им передаётся OID, не соответствующий существующему объекту, эти функции возвращают `NULL`.

Таблица 9.98. Функции получения размера объектов БД

Функция	Описание
<code>pg_column_size ("any") → integer</code>	Показывает, сколько байт занимает при хранении отдельное значение. Если эта функция применяется непосредственно к значению в столбце таблицы, в результате отражается эффект возможного сжатия.
<code>pg_column_compression ("any") → text</code>	Показывает алгоритм сжатия, который использовался для сжатия отдельного значения переменной длины. Возвращает <code>NULL</code> , если значение не сжато.
<code>pg_database_size (name) → bigint</code>	

Функция	Описание
<code>pg_database_size (oid) → bigint</code>	Вычисляет общий объём, который занимает на диске база данных с указанным именем или OID. Для использования этой функции необходимо иметь право <code>CONNECT</code> для заданной базы (оно даётся по умолчанию) или иметь права роли <code>pg_read_all_stats</code> .
<code>pg_indexes_size (regclass) → bigint</code>	Вычисляет общий объём, который занимают на диске индексы, связанные с указанной таблицей.
<code>pg_relation_size (relation regclass [, fork text]) → bigint</code>	Вычисляет объём, который занимает на диске один «слой» заданного отношения. (Заметьте, что в большинстве случаев удобнее использовать более высокоуровневые функции <code>pg_total_relation_size</code> и <code>pg_table_size</code> , которые суммируют размер всех слоёв.) С одним аргументом она выдаёт размер основного слоя с данными отношения. Название другого интересующего слоя можно передать во втором аргументе: • <code>main</code> выдаёт размер основного слоя данных заданного отношения. • <code>fsm</code> выдаёт размер карты свободного места (см. Раздел 70.3), связанной с заданным отношением. • <code>vm</code> выдаёт размер карты видимости (см. Раздел 70.4), связанной с заданным отношением. • <code>init</code> выдаёт размер слоя инициализации для заданного отношения, если он имеется.
<code>pg_size_bytes (text) → bigint</code>	Преобразует размер в человеко-ориентированном формате (который выдаёт <code>pg_size_pretty</code>) в число байт.
<code>pg_size_pretty (bigint) → text</code> <code>pg_size_pretty (numeric) → text</code>	Преобразует размер в байтах в более понятный человеку формат с единицами измерения размера (bytes, kB, MB, GB, TB или PB, в зависимости от значения). Заметьте, что эти единицы определяются как степени 2, а не 10, так что 1kB — это 1024 байта, 1MB — $1024^2 = 1048576$ байт и т. д.
<code>pg_table_size (regclass) → bigint</code>	Вычисляет объём, который занимает на диске данная таблица, за исключением индексов (но включая её TOAST-таблицу (если она есть), карту свободного места и карту видимости).
<code>pg_tablespace_size (name) → bigint</code> <code>pg_tablespace_size (oid) → bigint</code>	Вычисляет общий объём, который занимает на диске табличное пространство с заданным именем или OID. Для использования этой функции требуется иметь право <code>CREATE</code> для указанного табличного пространства или иметь права роли <code>pg_read_all_stats</code> , если только это не табличное пространство по умолчанию для текущей базы данных.
<code>pg_total_relation_size (regclass) → bigint</code>	Вычисляет общий объём, который занимает на диске заданная таблица, включая все её индексы и данные TOAST. Результат этой функции равен значению <code>pg_table_size</code> + <code>pg_indexes_size</code> .

Вышеописанные функции, работающие с таблицами или индексами, принимают аргумент типа `regclass`, который представляет собой просто OID таблицы или индекса в системном каталоге `pg_class`. Однако вам не нужно вручную вычислять OID, так как процедура ввода значения `regclass` может сделать это за вас. За подробностями обратитесь к [Разделу 8.19](#).

Функции, перечисленные в [Таблице 9.99](#), помогают определить, в каких файлах на диске хранятся объекты базы данных.

Таблица 9.99. Функции определения расположения объектов

Функция	Описание
<code>pg_relation_filenode (relation regclass) → oid</code>	Выдаёт номер «файлового узла», связанного с заданным объектом. Файловым узлом называется основной компонент имени файла, используемого для хранения данных (подробнее это описано в Разделе 70.1). Для большинства отношений этот номер совпадает со значением <code>pg_class .relfilenode</code> , но для некоторых системных каталогов <code>relfilenode</code> равен 0, и нужно использовать эту функцию, чтобы узнать действительное значение. Если указанное отношение не хранится на диске, как, например представление, данная функция возвращает NULL.
<code>pg_relation_filepath (relation regclass) → text</code>	Выдаёт полный путь к файлу отношения (относительно каталога данных PGDATA).
<code>pg_filenode_relation (tablespace oid, filenode oid) → regclass</code>	Выдаёт OID отношения, которое хранится в табличном пространстве, заданном по OID, в указанном файловом узле. По сути эта функция является обратной к <code>pg_relation_filepath</code> . Для табличного пространства по умолчанию можно передать нулевое значение OID. Если эта функция не находит в базе данных отношение по заданным значениям или если отношение является временным, она выдаёт NULL.

В [Таблице 9.100](#) перечислены функции, предназначенные для управления правилами сортировки.

Таблица 9.100. Функции управления правилами сортировки

Функция	Описание
<code>pg_collation_actual_version (oid) → text</code>	Возвращает действующую версию объекта правила сортировки, которая в настоящее время установлена в операционной системе. Если она отличается от значения в <code>pg_collation .collversion</code> , может потребоваться перестроить объекты, зависящие от данного правила сортировки. См. также ALTER COLLATION .
<code>pg_database_collation_actual_version (oid) → text</code>	Возвращает действующую версию правила сортировки БД, установленную на данный момент в операционной системе. Если она отличается от значения в <code>pg_database .datcollversion</code> , может потребоваться перестроить объекты, зависящие от данного правила сортировки. См. также ALTER DATABASE .
<code>pg_import_system_collations (schema regnamespace) → integer</code>	Добавляет правила сортировки в системный каталог <code>pg_collation</code> , анализируя все локали, которые она находит в операционной системе. Эту информацию использует <code>initdb</code> ; за подробностями обратитесь к Подразделу 22.2.2 . Если позднее в систему устанавливаются дополнительные локали, эту функцию можно запустить снова, чтобы добавились правила сортировки для новых локалей. Локали, для которых обнаруживаются существующие записи в <code>pg_collation</code> , будут пропущены. (Объекты правил сортировки для локалей, которые перестают существовать в операционной системе, никогда не удаляются этой функцией.) В параметре <code>schema</code> обычно передаётся <code>pg_catalog</code> , но это не обязательно; правила сортировки могут быть установлены и в другую схему. Эта функция возвращает число созданных ей объектов правил сортировки; вызывать её разрешено только суперпользователям.

В [Таблице 9.101](#) перечислены функции, предоставляющие информацию о структуре секционированных таблиц.

Таблица 9.101. Функции получения информации о секционировании

Функция	Описание
<code>pg_partition_tree (regclass) → setof record (relid regclass, parentrelid regclass, isleaf boolean, level integer)</code>	Выводит информацию о таблицах и индексах в дереве секционирования для заданной секционированной таблицы или секционированного индекса, отражая каждую секцию в отдельной строке. В этой информации представляется OID секции, OID её непосредственного родителя, логический признак того, что секция является конечной, и целое число, показывающее её уровень в иерархии. На уровне 0 будет находиться указанная таблица или индекс, на уровне 1 непосредственные потомки-секции, на уровне 2 секции последних и т. д. Если заданное отношение не существует или не является секцией или секционированным отношением, эта функция выдаёт пустой результат.
<code>pg_partition_ancestors (regclass) → setof regclass</code>	Выводит список вышестоящих отношений для заданной секции, включая её саму. Если заданное отношение не существует или не является секцией или секционированным отношением, эта функция выдаёт пустой результат.
<code>pg_partition_root (regclass) → regclass</code>	Выдаёт самое верхнее отношение в дереве секционирования, к которому относится заданное отношение. Если заданное отношение не существует или не является секцией или секционированным отношением, эта функция выдаёт NULL.

Например, чтобы определить общий размер данных, содержащихся в секционированной таблице `measurement`, можно использовать следующий запрос:

```
SELECT pg_size_pretty(sum(pg_relation_size(relid))) AS total_size
FROM pg_partition_tree('measurement');
```

9.27.8. Функции обслуживания индексов

В [Таблице 9.102](#) перечислены функции, предназначенные для обслуживания индексов. (Заметьте, что задачи обслуживания обычно выполняются автоматически в ходе автоочистки; пользоваться данными функциями требуется только в особых случаях.) Выполнять эти функции во время восстановления нельзя. Использовать их разрешено только суперпользователям и владельцу определённого индекса.

Таблица 9.102. Функции обслуживания индексов

Функция	Описание
<code>brin_summarize_new_values (index regclass) → integer</code>	Сканирует указанный BRIN-индекс в поисках зон страниц в базовой таблице, ещё не обобщённых в индексе; для каждой такой зоны в результате сканирования этих страниц создаётся новый обобщающий кортеж в индексе. Возвращает эта функция число вставленных в индекс обобщающих записей о зонах страниц.
<code>brin_summarize_range (index regclass, blockNumber bigint) → integer</code>	Обобщает зону страниц, охватывающую данный блок, если она ещё не была обобщена. Эта функция подобна <code>brin_summarize_new_values</code> , но обрабатывает только одну выбранную зону страниц.
<code>brin_desummarize_range (index regclass, blockNumber bigint) → void</code>	Удаляет из BRIN-индекса кортеж, который обобщает зону страниц, охватывающую данный блок таблицы, если такой кортеж имеется.
<code>gin_clean_pending_list (index regclass) → bigint</code>	Очищает очередь указанного GIN-индекса, массово перемещая элементы из неё в основную структуру данных GIN, и возвращает число страниц, убранных из очереди. Ес-

Функция	Описание
	ли для обработки ей передаётся индекс GIN, построенный с отключённым параметром <code>fastupdate</code> , очистка не производится и возвращается 0, так как у такого индекса нет очереди записей. Подробнее об очереди записей и параметре <code>fastupdate</code> рассказывается в Подразделе 67.4.1 и Разделе 67.5 .

9.27.9. Функции для работы с обычными файлами

Функции, перечисленные в [Таблице 9.103](#), предоставляют прямой доступ к файлам, находящимся на сервере. Обычным пользователям, не включённым в роль `pg_read_server_files`, они позволяют обращаться только к файлам в каталоге кластера баз данных и в каталоге `log_directory`. Для файлов в каталоге кластера этим функциям передаётся относительный путь, а для файлов журнала — путь, соответствующий значению параметра `log_directory`.

Заметьте, что пользователи, обладающие правом `EXECUTE` для `pg_read_file()` или связанных функций, имеют возможность читать любой файл на сервере, который может прочитать серверный процесс; на эти функции не действуют никакие ограничения доступа внутри базы данных. В частности это означает, что пользователь с такими правами может прочитать содержимое таблицы `pg_authid`, в которой хранятся данные аутентификации, равно как и любой другой файл в базе данных. Таким образом, разрешать доступ к этим функциям следует с большой осторожностью.

Некоторые из этих функций принимают необязательный параметр `missing_ok`, который определяет их поведение в случае отсутствия файла или каталога. Если он равен `true`, функция возвращает `NULL` или пустой набор данных. Если же он равен `false`, в указанном случае выдаётся ошибка. Значение по умолчанию — `false`.

Таблица 9.103. Функции для работы с обычными файлами

Функция	Описание
<code>pg_ls_dir</code>	<code>(dirname text [, missing_ok boolean, include_dot_dirs boolean]) → setof text</code> Выдаёт имена всех файлов (а также каталогов и других специальных файлов) в заданном каталоге. Параметр <code>include_dot_dirs</code> определяет, будут ли в результирующий набор включаться каталоги «.» и «..». По умолчанию они не включаются, но их можно включить, чтобы с параметром <code>missing_ok</code> равным <code>true</code> , пустой каталог можно было отличить от несуществующего. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (<code>EXECUTE</code>) можно дать и другим пользователям.
<code>pg_ls_logdir</code>	<code>() → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (<code>mtime</code>) всех обычных файлов в каталоге журналов сервера. Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи или роли с правами роли <code>pg_monitor</code> , но право на её выполнение (<code>EXECUTE</code>) можно дать и другим пользователям.
<code>pg_ls_waldir</code>	<code>() → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (<code>mtime</code>) всех обычных файлов в каталоге журнала предзаписи (WAL) сервера. Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи или роли с правами роли <code>pg_monitor</code> , но право на её выполнение (<code>EXECUTE</code>) можно дать и другим пользователям.

Функция	Описание
<code>pg_ls_logicalmapdir</code>	<code>() → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (mtime) всех обычных файлов в каталоге сервера <code>pg_logical/mappings</code> . Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи и члены группы <code>pg_monitor</code> , но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_ls_logicalsnapdir</code>	<code>() → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (mtime) всех обычных файлов в каталоге сервера <code>pg_logical/snapshots</code> . Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи и члены группы <code>pg_monitor</code> , но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_ls_replslotdir</code>	<code>(slot_name text) → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (mtime) всех обычных файлов в каталоге сервера <code>pg_replslot/slot_name</code> , где <code>slot_name</code> — имя слота репликации, переданное функции. Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи и члены группы <code>pg_monitor</code> , но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_ls_archive_statusdir</code>	<code>() → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (mtime) всех обычных файлов в каталоге состояния архива WAL (<code>pg_wal/archive_status</code>). Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи и члены группы <code>pg_monitor</code> , но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_ls_tmpdir</code>	<code>([tablespace oid]) → setof record (name text, size bigint, modification timestamp with time zone)</code> Выводит имя, размер и время последнего изменения (mtime) всех обычных файлов во временном каталоге для табличного пространства <code>tablespace</code>. Если параметр <code>tablespace</code> не задан, подразумевается табличное пространство <code>pg_default</code> . Файлы с именами, начинающимися с точки, каталоги и другие специальные файлы исключаются из рассмотрения. По умолчанию доступ к этой функции имеют только суперпользователи и члены группы <code>pg_monitor</code> , но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_read_file</code>	<code>(filename text [, offset bigint, length bigint [, missing_ok boolean]]) → text</code> Читает из текстового файла всё содержимое или фрагмент с заданного смещения (<code>offset</code>), размером не больше <code>length</code> байт (размер может быть меньше, если файл кончится раньше). Если смещение <code>offset</code> отрицательно, оно отсчитывается от конца файла. Если параметры <code>offset</code> и <code>length</code> опущены, возвращается всё содержимое файла. Прочитанные из файла байты обрабатываются как символы в кодировке базы данных; если они оказываются недопустимыми для этой кодировки, возникает ошибка.

Функция	Описание
	По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.
<code>pg_read_binary_file (filename text [, offset bigint, length bigint [, missing_ok boolean]]) → bytea</code>	Читает из текстового файла всё содержимое или заданный фрагмент. Эта функция подобна <code>pg_read_file</code> , но может читать произвольные двоичные данные и возвращает результат в значении типа <code>bytea</code>, а не <code>text</code>; как следствие, никакие проверки кодировок не производятся. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям. В сочетании с <code>convert_from</code> эту функцию можно применять для чтения текста в указанной кодировке и преобразования в кодировку базы данных: <pre>SELECT convert_from(pg_read_binary_file('file_in_utf8.txt'), 'UTF8');</pre>
<code>pg_stat_file (filename text [, missing_ok boolean]) → record (size bigint, access timestamp with time zone, modification timestamp with time zone, change timestamp with time zone, creation timestamp with time zone, isdir boolean)</code>	Выдаёт запись, содержащую размер файла, время последнего обращения и последнего изменения, а также время последнего изменения состояния (только в Unix-системах), время создания (только в Windows) и флаг, показывающий, что это каталог. По умолчанию доступ к этой функции имеют только суперпользователи, но право на её выполнение (EXECUTE) можно дать и другим пользователям.

9.27.10. Функции управления рекомендательными блокировками

Функции, перечисленные в [Таблице 9.104](#), предназначены для управления рекомендательными блокировками. Подробнее об их использовании можно узнать в [Подразделе 13.3.5](#).

Все эти функции предназначены для оперирования блокировками ресурсов, определяемых приложением и задаваемых одним 64-битным или двумя 32-битными ключами (заметьте, что пространства этих ключей не пересекаются). Если конфликтующую блокировку с тем же идентификатором уже удерживает другой сеанс, эти функции либо дожидаются освобождения ресурса, либо выдают в результате `false`, в зависимости от вида функции. Блокировки могут быть как исключительными, так и разделяемыми — разделяемая блокировка не конфликтует с другими разделяемыми блокировками того же ресурса, но конфликтует с исключительными. Блокировки могут устанавливаться на сеансовом уровне (тогда они удерживаются до момента освобождения или до завершения сеанса) и на транзакционном (тогда они удерживаются до конца текущей транзакции, освободить их вручную нет возможности). Когда поступает несколько запросов на блокировку сеансового уровня, они накапливаются, так что если один идентификатор ресурса был заблокирован три раза, должны поступить три запроса на освобождение блокировки, чтобы ресурс был разблокирован до завершения сеанса.

Таблица 9.104. Функции управления рекомендательными блокировками

Функция	Описание
<code>pg_advisory_lock (key bigint) → void</code> <code>pg_advisory_lock (key1 integer, key2 integer) → void</code>	Получает исключительную рекомендательную блокировку сеансового уровня, ожидая её, если это необходимо.
<code>pg_advisory_lock_shared (key bigint) → void</code> <code>pg_advisory_lock_shared (key1 integer, key2 integer) → void</code>	Получает разделяемую рекомендательную блокировку сеансового уровня, ожидая её, если это необходимо.

Функция	Описание
<code>pg_advisory_unlock (key bigint) → boolean</code> <code>pg_advisory_unlock (key1 integer, key2 integer) → boolean</code>	Освобождает ранее полученную исключительную рекомендательную блокировку сеансового уровня. Если блокировка освобождена успешно, эта функция возвращает <code>true</code> , а если сеанс не владел ей — <code>false</code> , при этом сервер выдаёт предупреждение SQL.
<code>pg_advisory_unlock_all () → void</code>	Освобождает все закреплённые за текущим сеансом рекомендательные блокировки сеансового уровня. (Эта функция неявно вызывается в конце любого сеанса, даже при нештатном отключении клиента.)
<code>pg_advisory_unlock_shared (key bigint) → boolean</code> <code>pg_advisory_unlock_shared (key1 integer, key2 integer) → boolean</code>	Освобождает ранее полученную разделяемую рекомендательную блокировку сеансового уровня. Если она освобождена успешно, эта функция возвращает <code>true</code> , а если сеанс не владел этой блокировкой — <code>false</code> , при этом сервер выдаёт предупреждение SQL.
<code>pg_advisory_xact_lock (key bigint) → void</code> <code>pg_advisory_xact_lock (key1 integer, key2 integer) → void</code>	Получает исключительную рекомендательную блокировку транзакционного уровня, ожидая её, если это необходимо.
<code>pg_advisory_xact_lock_shared (key bigint) → void</code> <code>pg_advisory_xact_lock_shared (key1 integer, key2 integer) → void</code>	Получает разделяемую рекомендательную блокировку транзакционного уровня, ожидая её, если это необходимо.
<code>pg_try_advisory_lock (key bigint) → boolean</code> <code>pg_try_advisory_lock (key1 integer, key2 integer) → boolean</code>	Получает исключительную рекомендательную блокировку сеансового уровня, если она доступна. То есть эта функция либо немедленно получает блокировку и возвращает <code>true</code> , либо сразу возвращает <code>false</code> , если получить её нельзя.
<code>pg_try_advisory_lock_shared (key bigint) → boolean</code> <code>pg_try_advisory_lock_shared (key1 integer, key2 integer) → boolean</code>	Получает разделяемую рекомендательную блокировку сеансового уровня, если она доступна. То есть эта функция либо немедленно получает блокировку и возвращает <code>true</code> , либо сразу возвращает <code>false</code> , если получить её нельзя.
<code>pg_try_advisory_xact_lock (key bigint) → boolean</code> <code>pg_try_advisory_xact_lock (key1 integer, key2 integer) → boolean</code>	Получает исключительную рекомендательную блокировку транзакционного уровня, если она доступна. То есть эта функция либо немедленно получает блокировку и возвращает <code>true</code> , либо сразу возвращает <code>false</code> , если получить её нельзя.
<code>pg_try_advisory_xact_lock_shared (key bigint) → boolean</code> <code>pg_try_advisory_xact_lock_shared (key1 integer, key2 integer) → boolean</code>	Получает разделяемую рекомендательную блокировку транзакционного уровня, если она доступна. То есть эта функция либо немедленно получает блокировку и возвращает <code>true</code> , либо сразу возвращает <code>false</code> , если получить её нельзя.

9.27.11. Опорные функции журнала операций

В журнале операций хранится информация о критически важных системных событиях, таких как обновление, выполнение [pg_resetwal](#) и т. п. Эта информация не представляет особого интереса для обычного пользователя, но важна для осуществления технической поддержки со стороны по-

ставщика. Запись в журнал операций производится только на системном уровне (без каких-либо действий со стороны пользователя), а для его чтения используются функции SQL. Функция, показанная в Таблица 9.105, читает журнал операций.

Таблица 9.105. Опорные функции журнала операций

Функция	Описание
<code>pg_operation_log</code>	<code>() → setof record (event text, edition text, version text, lsn pg_lsn , last timestampz, count int4)</code> Здесь: <code>event</code> — тип события (операции). Может быть <code>bootstrap</code>, <code>startup</code>, <code>pg_resetwal</code>, <code>pg_rewind</code>, <code>pg_upgrade</code> или <code>promoted</code>. <code>edition</code> — редакция Postgres Pro. Может быть <code>vanilla</code>, <code>1c</code>, <code>std</code>, <code>ent</code> или <code>unknown</code>. <code>version</code> — версия Postgres Pro. <code>lsn</code> — положение последней контрольной точки, возвращённое утилитой <code>pg_controldata</code> на момент возникновения события. <code>last</code> — дата/время возникновения последнего события этого типа, если события накапливаются, или дата/время возникновения события в противном случае. <code>count</code> — количество событий этого типа, если события накапливаются, или 1 в противном случае. Эта системная функция создаётся автоматически для новых баз данных, а для существующих баз данных должна создаваться следующим образом: <pre>CREATE OR REPLACE FUNCTION pg_operation_log(OUT event text, OUT edition text, OUT version text, OUT lsn pg_lsn, OUT last timestampz, OUT count int4) RETURNS SETOF record LANGUAGE INTERNAL STABLE STRICT PARALLEL SAFE AS 'pg_operation_log';</pre>

Ниже представлен пример вывода для функции `pg_operation_log`:

```
select * from pg_operation_log();
```

event	edition	version	lsn	last	count
startup	vanilla	10.22.0	0/8000028	2022-10-27 23:06:27+03	1
pg_upgrade	1c	15.0.0	0/8000028	2022-10-27 23:06:27+03	1
startup	1c	15.0.0	0/80001F8	2022-10-27 23:06:53+03	2

(3 rows)

Примечание

Если журнал операций пуст, то при обновлении перед записью `pg_upgrade` сначала записывается событие `startup` с предыдущим значением `version`, имеющим следующие особенности: вместо версии патча (третья цифра в номере версии) всегда устанавливается ноль, поскольку в `pg_upgrade` нет информации о версии патча ни для одной редакции (`1c`, `std`, `ent`).

При обновлении с 1c или vanilla невозможно выяснить, к какой из этих редакций относится база данных до обновления, поэтому для записи startup используется редакция vanilla.

9.27.12. Отладочные функции

Функция, показанная в [Таблице 9.106](#), может быть полезна для низкоуровневых операций, например в целях отладки или исследования испорченных баз данных Postgres Pro.

Таблица 9.106. Функции синхронизации снимков

Имя	Тип результата	Описание
<code>pg_snapshot_any()</code>	<code>void</code>	Отключает действие правил MVCC в рамках текущей транзакции, что позволяет видеть в ней все версии данных.

Применяйте `pg_snapshot_any` крайне осторожно. Выполнять её следует в транзакции с уровнем изоляции `REPEATABLE READ` или выше, в противном случае следующий же запрос заменит выбранный снимок новым. Выполнять её могут только суперпользователи.

Примечание

Если ваш кластер БД создавался сервером версии, в которой этой функции ещё не было, определите её следующей командой:

```
CREATE FUNCTION pg_snapshot_any() RETURNS void AS 'pg_snapshot_any' LANGUAGE internal;
```

9.28. Триггерные функции

Тогда как в большинстве случаев использование триггеров подразумевает написание пользовательских триггерных функций, в Postgres Pro имеется несколько встроенных триггерных функций, которые можно использовать непосредственно в пользовательских триггерах. Эти функции показаны в [Таблице 9.107](#). (Существуют и другие встроенные триггерные функции, реализующие ограничения внешнего ключа и отложенные ограничения по индексам. Однако они не документированы, так как пользователям не нужно использовать их непосредственно.)

Подробнее о создании триггеров можно узнать в описании [CREATE TRIGGER](#).

Таблица 9.107. Встроенные триггерные функции

Функция	Описание	Пример использования
<code>suppress_redundant_updates_trigger</code>	<code>() → trigger</code>	Предотвращает изменения, не меняющие данные. Подробнее об этом рассказывается ниже. <code>CREATE TRIGGER ... suppress_redundant_updates_trigger()</code>
<code>tsvector_update_trigger</code>	<code>() → trigger</code>	Автоматически обновляет содержимое столбца <code>tsvector</code> из связанных столбцов с обычным текстовым содержимым. Конфигурация текстового поиска, которая будет использоваться, задаётся по имени в аргументе триггера. За подробностями обратитесь к Подразделу 12.4.3 . <code>CREATE TRIGGER ... tsvector_update_trigger(tsvcol, 'pg_catalog.swedish', title, body)</code>

Функция
Описание
Пример использования
<pre>tsvector_update_trigger_column () → trigger</pre> Автоматически обновляет содержимое столбца <code>tsvector</code> из связанных столбцов с обычным текстовым содержимым. Конфигурация текстового поиска, которая будет использоваться, определяется содержимым столбца <code>regconfig</code> целевой таблицы. За подробностями обратитесь к Подразделу 12.4.3. <pre>CREATE TRIGGER ... tsvector_update_trigger_column(tsvcol, tsconfigcol, title, body)</pre>

Функция `suppress_redundant_updates_trigger`, применяемая в качестве триггера `BEFORE UPDATE` на уровне строк, предотвратит внесение изменений, при которых данные в строке фактически не меняются. Тем самым переопределяется обычное поведение, когда изменение физической строки происходит вне зависимости от того, были ли изменены данные. (Обычное поведение не предполагает сравнения данных, поэтому операции изменения выполняются быстрее, и в ряде случаев именно это поведение желательно.)

В идеале следует избегать операций изменения, которые фактически не меняют данные в записях. Подобные ненужные изменения могут обходиться дорого, особенно когда требуется обновлять множество индексов, к тому же впоследствии базу данных придётся очищать от «мёртвых» строк. Однако выявить такие изменения в клиентском коде бывает сложно, если вообще возможно, а при составлении соответствующих проверочных выражений легко допустить ошибку. В качестве альтернативного решения можно использовать функцию `suppress_redundant_updates_trigger`, которая опускает изменения, не меняющие данные. Однако использовать её следует с осторожностью. Данный триггер выполняется для каждой записи довольно быстро, но всё же не мгновенно, так что если большинство затронутых записей фактически изменяется, с этим триггером операция изменения в среднем замедлится.

Функцию `suppress_redundant_updates_trigger` можно привязать к таблице так:

```
CREATE TRIGGER z_min_update
BEFORE UPDATE ON tablename
FOR EACH ROW EXECUTE FUNCTION suppress_redundant_updates_trigger();
```

В большинстве случаев этот триггер должен вызываться для каждой строки последним, чтобы он не перекрыл другие триггеры, которым может понадобится изменить строку. С учётом того, что триггеры вызываются по порядку сортировки их имён, имя для него нужно выбирать таким, чтобы оно было последним среди имён всех триггеров, которые могут быть в таблице. (Из этого соображения выбран префикс «z» в данном примере.)

9.29. Функции событийных триггеров

Postgres Pro предоставляет следующие вспомогательные функции для получения информации в событийных триггерах.

Подробнее о событийных триггерах можно узнать в [Главе 39](#).

9.29.1. Получение изменений в конце команды

```
pg_event_trigger_ddl_commands ( ) → setof record
```

Функция `pg_event_trigger_ddl_commands` возвращает список команд DDL, выполняемых в результате действия пользователя. Вызывать её можно только в функции, реализующей событийный триггер `ddl_command_end`. При попытке вызвать её в любом другом контексте возникнет ошибка. Функция `pg_event_trigger_ddl_commands` возвращает одну строку для каждой базовой команды; для некоторых команд, записываемых в виде одного предложения SQL, может возвращаться несколько строк. Эта функция возвращает следующие столбцы:

Имя	Тип	Описание
classid	oid	OID каталога, к которому относится объект
objid	oid	OID самого объекта
objsubid	integer	Идентификатор подобъекта (например, номер для столбца)
command_tag	text	Тег команды
object_type	text	Тип объекта
schema_name	text	Имя схемы, к которой относится объект; если объект не относится ни к какой схеме — NULL. В кавычки имя не заключается.
object_identity	text	Текстовое представление идентификатора объекта, включающее схему. При необходимости компоненты этого идентификатора заключаются в кавычки.
in_extension	boolean	True, если команда является частью скрипта расширения
command	pg_ddl_command	Полное представление команды, во внутреннем формате. Его нельзя вывести непосредственно, но можно передать другим функциям, чтобы получить различные сведения о команде.

9.29.2. Обработка объектов, удалённых командой DDL

`pg_event_trigger_dropped_objects ()` → setof record

Функция `pg_event_trigger_dropped_objects` выдаёт список всех объектов, удалённых командой, для которых вызывалось событие `sql_drop`. При вызове в любом другом контексте происходит ошибка. Эта функция выдаёт следующие столбцы:

Имя	Тип	Описание
classid	oid	OID каталога, к которому относился объект
objid	oid	OID самого объекта
objsubid	integer	Идентификатор подобъекта (например, номер для столбца)
original	boolean	True, если это один из корневых удаляемых объектов
normal	boolean	True, если к этому объекту в графе зависимостей привело отношение обычной зависимости
is_temporary	boolean	True, если объект был временным
object_type	text	Тип объекта

Имя	Тип	Описание
schema_name	text	Имя схемы, к которой относился объект; если объект не относился ни к какой схеме — NULL. В кавычки имя не заключается.
object_name	text	Имя объекта, если сочетание схемы и имени позволяет уникально идентифицировать объект; в противном случае — NULL. Имя не заключается в кавычки и не дополняется именем схемы.
object_identity	text	Текстовое представление идентификатора объекта, включающее схему. При необходимости компоненты этого идентификатора заключаются в кавычки.
address_names	text[]	Массив, который в сочетании с object_type и массивом address_args можно передать функции pg_get_object_address, чтобы воссоздать адрес объекта на удалённом сервере, содержащем одноимённый объект того же рода.
address_args	text[]	Дополнение к массиву address_names

Функцию pg_event_trigger_dropped_objects можно использовать в событийном триггере так:

```
CREATE FUNCTION test_event_trigger_for_drops()
    RETURNS event_trigger LANGUAGE plpgsql AS $$
DECLARE
    obj record;
BEGIN
    FOR obj IN SELECT * FROM pg_event_trigger_dropped_objects()
    LOOP
        RAISE NOTICE '% dropped object: % %.% %',
            tg_tag,
            obj.object_type,
            obj.schema_name,
            obj.object_name,
            obj.object_identity;
    END LOOP;
END;
$$;
CREATE EVENT TRIGGER test_event_trigger_for_drops
    ON sql_drop
    EXECUTE FUNCTION test_event_trigger_for_drops();
```

9.29.3. Обработка события перезаписи таблицы

В [Таблице 9.108](#) показаны функции, выдающие информацию о таблице, для которой произошло событие перезаписи таблицы (table_rewrite). При попытке вызвать их в другом контексте возникнет ошибка.

Таблица 9.108. Функции получения информации о перезаписи таблицы

Функция	Описание
<code>pg_event_trigger_table_rewrite_oid</code> <code>() → oid</code>	Выдаёт OID таблицы, которая будет перезаписана.
<code>pg_event_trigger_table_rewrite_reason</code> <code>() → integer</code>	Возвращает код с объяснением причины(причин) перезаписи в виде битовой карты, построенной из следующих значений: 1 (в таблице изменилась характеристика хранения отношений), 2 (в столбце изменилось значение по умолчанию), 4 (у столбца изменился тип данных) и 8 (изменился табличный метод доступа).

Эти функции можно использовать в событийном триггере так:

```
CREATE FUNCTION test_event_trigger_table_rewrite_oid()
RETURNS event_trigger
LANGUAGE plpgsql AS
$$
BEGIN
    RAISE NOTICE 'rewriting table % for reason %',
        pg_event_trigger_table_rewrite_oid()::regclass,
        pg_event_trigger_table_rewrite_reason();
END;
$$;

CREATE EVENT TRIGGER test_table_rewrite_oid
ON table_rewrite
EXECUTE FUNCTION test_event_trigger_table_rewrite_oid();
```

9.30. Системные информационные функции

Postgres Pro предоставляет функцию для исследования сложной статистики, создаваемой командой `CREATE STATISTICS`.

9.30.1. Исследование списков MCV

`pg_mcv_list_items` (`pg_mcv_list`) `→ setof record`

Функция `pg_mcv_list_items` возвращает набор записей, описывающих все элементы в многоколоном списке MCV. Эти записи имеют следующие столбцы:

Имя	Тип	Описание
<code>index</code>	<code>integer</code>	индекс элемента в списке MCV
<code>values</code>	<code>text[]</code>	значения, сохранённые в элементе списка MCV
<code>nulls</code>	<code>boolean[]</code>	флаги, помечающие значения NULL
<code>frequency</code>	<code>double precision</code>	частота вхождения этого элемента MCV
<code>base_frequency</code>	<code>double precision</code>	базовая частота вхождения этого элемента MCV

Использовать функцию `pg_mcv_list_items` можно следующим образом:

```
SELECT m.* FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid),
pg_mcv_list_items(stxmcv) m WHERE stxname = 'stts';
```

Значения типа `pg_mcv_list` можно получить только из столбца `pg_statistic_ext_data.stxdmcv`.

Глава 10. Преобразование типов

SQL-операторы, намеренно или нет, требуют совмещать данные разных типов в одном выражении. Для вычисления подобных выражений со смешанными типами Postgres Pro предоставляет широкий набор возможностей.

Очень часто пользователю не нужно понимать все тонкости механизма преобразования. Однако следует учитывать, что неявные преобразования, производимые Postgres Pro, могут влиять на результат запроса. Поэтому при необходимости нужные результаты можно получить, применив *явное* преобразование типов.

В этой главе описываются общие механизмы преобразования типов и соглашения, принятые в Postgres Pro. За дополнительной информацией о конкретных типах данных и разрешённых для них функциях и операторах обратитесь к соответствующим разделам в [Главе 8](#) и [Главе 9](#).

10.1. Обзор

SQL — язык со строгой типизацией. То есть каждый элемент данных в нём имеет некоторый тип, определяющий его поведение и допустимое использование. Postgres Pro наделён расширяемой системой типов, более универсальной и гибкой по сравнению с другими реализациями SQL. При этом преобразования типов в Postgres Pro в основном подчиняются определённым общим правилам, для их понимания не нужен эвристический анализ. Благодаря этому в выражениях со смешанными типами можно использовать даже типы, определённые пользователями.

Анализатор выражений Postgres Pro разделяет их лексические элементы на пять основных категорий: целые числа, другие числовые значения, текстовые строки, идентификаторы и ключевые слова. Константы большинства не числовых типов сначала классифицируются как строки. В определении языка SQL допускается указывать имена типов в строках и это можно использовать в Postgres Pro, чтобы направить анализатор по верному пути. Например, запрос:

```
SELECT text 'Origin' AS "label", point '(0,0)' AS "value";
```

```
label | value
-----+-----
Origin | (0,0)
(1 row)
```

содержит две строковых константы, типа `text` и типа `point`. Если для такой константы не указан тип, для неё первоначально предполагается тип `unknown`, который затем может быть уточнён, как описано ниже.

В SQL есть четыре фундаментальных фактора, определяющих правила преобразования типов для анализатора выражений Postgres Pro:

Вызовы функций

Система типов Postgres Pro во многом построена как дополнение к богатым возможностям функций. Функции могут иметь один или несколько аргументов, и при этом Postgres Pro разрешает перегружать имена функций, так что имя функции само по себе не идентифицирует вызываемую функцию; анализатор выбирает правильную функцию в зависимости от типов переданных аргументов.

Операторы

Postgres Pro позволяет использовать в выражениях префиксные операторы (с одним аргументом), а также инфиксные операторы (с двумя аргументами). Как и функции, операторы можно перегружать, так что и с ними существует проблема выбора правильного оператора.

Сохранение значений

SQL-операторы `INSERT` и `UPDATE` помещают результаты выражений в таблицы. При этом получаемые значения должны соответствовать типам целевых столбцов или, возможно, приводиться к ним.

UNION, CASE и связанные конструкции

Так как все результаты запроса объединяющего оператора `SELECT` должны оказаться в одном наборе столбцов, результаты каждого подзапроса `SELECT` должны приводиться к одному набору типов. Подобным образом, результирующие выражения конструкции `CASE` должны приводиться к общему типу, так как выражение `CASE` в целом должно иметь определённый выходной тип. Подобное определение общего типа для значений нескольких подвыражений требуется и для некоторых других конструкций, например `ARRAY [ ]`, а также для функций `GREATEST` и `LEAST`.

Информация о существующих преобразованиях или *приведениях* типов, для каких типов они определены и как их выполнять, хранится в системных каталогах. Пользователь также может добавить дополнительные преобразования с помощью команды [CREATE CAST](#). (Обычно это делается, когда определяются новые типы данных. Набор приведений для встроенных типов достаточно хорошо проработан, так что его лучше не менять.)

Дополнительная логика анализа помогает выбрать оптимальное приведение в группах типов, допускающих неявные преобразования. Для этого типы данных разделяются на несколько базовых *категорий*, которые включают: `boolean`, `numeric`, `string`, `bitstring`, `datetime`, `timespan`, `geometric`, `network` и пользовательские типы. (Полный список категорий приведён в [Таблице 52.67](#); хотя его тоже можно расширить, определив свои категории.) В каждой категории могут быть выбраны один или несколько *предпочитаемых типов*, которые будут считаться наиболее подходящими при рассмотрении нескольких вариантов. Аккуратно выбирая предпочитаемые типы и допустимые неявные преобразования, можно добиться того, что выражения с неоднозначностями (в которых возможны разные решения задачи преобразования) будут разрешаться наилучшим образом.

Все правила преобразования типов разработаны с учётом следующих принципов:

- Результат неявных преобразований всегда должен быть предсказуемым и понятным.
- Если в неявном преобразовании нет нужды, анализатор и исполнитель запроса не должны тратить лишнее время на это. То есть, если запрос хорошо сформулирован и типы значений совпадают, он должен выполняться без дополнительной обработки в анализаторе и без лишних вызовов неявных преобразований.
- Кроме того, если запрос изначально требовал неявного преобразования для функции, а пользователь определил новую функцию с точно совпадающими типами аргументов, анализатор должен переключиться на новую функцию и больше не выполнять преобразование для вызова старой.

10.2. Операторы

При выборе конкретного оператора, задействованного в выражении, PostgreSQL следует описанному ниже алгоритму. Заметьте, что на этот выбор могут неявно влиять приоритеты остальных операторов в данном выражении, так как они определяют, какие подвыражения будут аргументами операторов. Подробнее об этом рассказывается в [Подразделе 4.1.6](#).

Выбор оператора по типу

1. Выбрать операторы для рассмотрения из системного каталога `pg_operator`. Если имя оператора не дополнено именем схемы (обычно это так), будут рассматриваться все операторы с подходящим именем и числом аргументов, видимые в текущем пути поиска (см. [Подраздел 5.9.3](#)). Если имя оператора определено полностью, в рассмотрение принимаются только операторы из указанной схемы.
 - (Optional) Если в пути поиска оказывается несколько операторов с одинаковыми типами аргументов, учитываются только те из них, которые находятся в пути раньше. Операторы с разными типами аргументов рассматриваются на равных правах вне зависимости от их положения в пути поиска.

2. Проверить, нет ли среди них оператора с точно совпадающими типами аргументов. Если такой оператор есть (он может быть только одним в отобранном ранее наборе), использовать его. Отсутствие точного совпадения создаёт угрозу вызова с указанием полного имени¹ (нетипичным) любого оператора, который может оказаться в схеме, где могут создавать объекты недоверенные пользователи. В таких ситуациях приведите типы аргументов для получения точного совпадения.
 - a. (Optional) Если один аргумент при вызове бинарного оператора имеет тип `unknown`, для данной проверки предполагается, что он имеет тот же тип, что и второй его аргумент. При вызове бинарного оператора с двумя аргументами `unknown` или префиксного с одним `unknown` оператор не будет выбран на этом шаге.
 - b. (Optional) Если один аргумент при вызове бинарного оператора имеет тип `unknown`, а другой — домен, проверить, есть ли оператор, принимающий базовый тип домена с обеих сторон; если таковой находится, использовать его.
3. Найти самый подходящий.
 - a. Отбросить кандидатов, для которых входные типы не совпадают и не могут быть преобразованы (неявным образом) так, чтобы они совпали. В данном случае считается, что константы типа `unknown` можно преобразовать во что угодно. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
 - b. Если один из аргументов имеет тип домен, далее считать его типом базовый тип домена. Благодаря этому при поиске неоднозначно заданного оператора домены будут подобны своим базовым типам.
 - c. Просмотреть всех кандидатов и оставить только тех, для которых точно совпадают как можно больше типов аргументов. Оставить всех кандидатов, если точных совпадений нет. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
 - d. Просмотреть всех кандидатов и оставить только тех, которые принимают предпочитаемые типы (из категории типов входных значений) в наибольшем числе позиций, где требуется преобразование типов. Оставить всех кандидатов, если ни один не принимает предпочитаемые типы. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
 - e. Если какие-либо значения имеют тип `unknown`, проверить категории типов, принимаемых в данных позициях аргументов оставшимися кандидатами. Для каждой позиции выбрать категорию `string`, если какой-либо кандидат принимает эту категорию. (Эта склонность к строкам объясняется тем, что константа типа `unknown` выглядит как строка.) Если эта категория не подходит, но все оставшиеся кандидаты принимают одну категорию, выбрать её; в противном случае констатировать неудачу — сделать правильный выбор без дополнительных подсказок нельзя. Затем отбросить кандидатов, которые не принимают типы выбранной категории. Далее, если какой-либо кандидат принимает предпочитаемый тип из этой категории, отбросить кандидатов, принимающих другие, не предпочитаемые типы для данного аргумента. Оставить всех кандидатов, если эти проверки не прошёл ни один. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
 - f. Если в списке аргументов есть аргументы и типа `unknown`, и известного типа, и этот известный тип один для всех аргументов, предположить, что аргументы типа `unknown` также имеют этот тип, и проверить, какие кандидаты могут принимать этот тип в позиции аргумента `unknown`. Если остаётся только один кандидат, использовать его, в противном случае констатировать неудачу.

Ниже это проиллюстрировано на примерах.

¹Эта угроза неактуальна для имён без схемы, так как путь поиска, содержащий схемы, в которых недоверенные пользователи могут создавать объекты, не соответствует [шаблону безопасного использования схем](#).

Пример 10.1. Разрешение типа для оператора квадратного корня

В стандартном каталоге определён только один оператор квадратного корня (префиксный `|/`) и он принимает аргумент типа `double precision`. При просмотре следующего выражения его аргументу изначально назначается тип `integer`:

```
SELECT |/ 40 AS "square root of 40";
square root of 40
-----
6.324555320336759
(1 row)
```

Поэтому анализатор преобразует тип этого операнда и запрос становится равносильным такому:

```
SELECT |/ CAST(40 AS double precision) AS "square root of 40";
```

Пример 10.2. Разрешение оператора конкатенации строк

Синтаксис текстовых строк используется как для записи строковых типов, так и для сложных типов расширений. Если тип не указан явно, такие строки сопоставляются по тому же алгоритму с наиболее подходящими операторами.

Пример с одним неопределённым аргументом:

```
SELECT text 'abc' || 'def' AS "text and unknown";

text and unknown
-----
abcdef
(1 row)
```

В этом случае анализатор смотрит, есть ли оператор, у которого оба аргумента имеют тип `text`. Такой оператор находится, поэтому предполагается, что второй аргумент следует воспринимать как аргумент типа `text`.

Конкатенация двух значений неопределённых типов:

```
SELECT 'abc' || 'def' AS "unspecified";

unspecified
-----
abcdef
(1 row)
```

В данном случае нет подсказки для выбора типа, так как в данном запросе никакие типы не указаны. Поэтому анализатор просматривает все возможные операторы и находит в них кандидатов, принимающих аргументы категорий `string` и `bit-string`. Так как категория `string` является предпочтительной, выбирается она, а затем для разрешения типа не типизированной константы выбирается предпочтительный тип этой категории, `text`.

Пример 10.3. Разрешение оператора абсолютного значения и отрицания

В каталоге операторов Postgres Pro для префиксного оператора `@` есть несколько записей, описывающих операции получения абсолютного значения для различных числовых типов данных. Одна из записей соответствует типу `float8`, предпочтительного в категории числовых типов. Таким образом, столкнувшись со значением типа `unknown`, Postgres Pro выберет эту запись:

```
SELECT @ '-4.5' AS "abs";
abs
----
4.5
(1 row)
```

Здесь система неявно привела константу неизвестного типа к типу `float8`, прежде чем применять выбранный оператор. Можно убедиться в том, что выбран именно тип `float8`, а не какой-то другой:

```
SELECT @ '-4.5e500' AS "abs";
```

ОШИБКА: "-4.5e500" вне диапазона для типа double precision

С другой стороны, префиксный оператор ~ (побитовое отрицание) определён только для целочисленных типов данных, но не для float8. Поэтому, если попытаться выполнить похожий запрос с ~, мы получаем:

```
SELECT ~ '20' AS "negation";
```

ОШИБКА: оператор не уникален: ~ "unknown"

ПОДСКАЗКА: Не удалось выбрать лучшую кандидатуру оператора. Возможно, вам следует добавить явные преобразования типов.

Это происходит оттого, что система не может решить, какой оператор предпочесть из нескольких возможных вариантов ~. Мы можем облегчить её задачу, добавив явное преобразование:

```
SELECT ~ CAST('20' AS int8) AS "negation";
```

```
negation
-----
      -21
(1 row)
```

Пример 10.4. Разрешение оператора включения в массив

Ещё один пример разрешения оператора с одним аргументом известного типа и другим неизвестного:

```
SELECT array[1,2] <@ '{1,2,3}' as "is subset";
```

```
is subset
-----
t
(1 row)
```

В каталоге операторов Postgres Pro есть несколько записей для инфиксного оператора <@, но только два из них могут принять целочисленный массива слева: оператор включения массива (anyarray<@anyarray) и оператор включения диапазона (anyelement<@anyrange). Так как ни один из этих полиморфных псевдотипов (см. [Раздел 8.21](#)) не считается предпочтительным, анализатор не может избавиться от неоднозначности на данном этапе. Однако в [Шар 3.f](#) говорится, что константа неизвестного типа должна рассматриваться как значение типа другого аргумента, в данном случае это целочисленный массив. После этого подходящим считается только один из двух операторов, так что выбирается оператор с целочисленными массивами. (Если бы был выбран оператор включения диапазона, мы получили бы ошибку, так как значение в строке не соответствует формату значений диапазона.)

Пример 10.5. Нестандартный оператор с доменом

Иногда пользователи пытаются ввести операторы, применимые только к определённому домену. Это возможно, но вовсе не так полезно, как может показаться, ведь правила разрешения операторов применяются к базовому типу домена. Взгляните на этот пример:

```
CREATE DOMAIN mytext AS text CHECK(...);
CREATE FUNCTION mytext_eq_text (mytext, text) RETURNS boolean AS ...;
CREATE OPERATOR = (procedure=mytext_eq_text, leftarg=mytext, rightarg=text);
CREATE TABLE mytable (val mytext);
```

```
SELECT * FROM mytable WHERE val = 'foo';
```

В этом запросе не будет использоваться нововведённый оператор. При разборе запроса сначала будет проверено, есть ли оператор mytext = mytext (см. [Шар 2.a](#)), но это не так; затем будет рассмотрен базовый тип домена (text) и проверено наличие оператора text = text (см. [Шар 2.b](#)), и таковой действительно есть; в итоге строковое значение типа unknown будет воспринято как text

и будет применён оператор `text = text`. Единственный вариант задействовать нововведённый оператор — добавить явное приведение:

```
SELECT * FROM mytable WHERE val = text 'foo';
```

так, чтобы оператор `mytext = text` был найден сразу, согласно правилу точного совпадения. Если дело доходит до правил наибольшего соответствия, они активно дискредитируют операторы доменных типов. Если бы они этого не делали, с таким оператором возникало бы слишком много ошибок разрешения операторов, потому что правила приведения всегда считают домен приводимым к базовому типу и наоборот, так что доменный оператор применялся бы во всех случаях, где применяется одноимённый оператор с базовым типом.

10.2.1. Предотвращение ошибок разрешения типов операторов

В PostgreSQL есть известная проблема разрешения типов операторов. Если операторы, определённые в схеме, отличной от `pg_catalog`, используют нестандартные типы, восстановление базы данных с помощью [pg_restore](#) или запуск [pg_upgrade](#) может завершиться ошибкой. Проблема возникает из-за того, что `pg_restore` выполняется с пустым значением [search_path](#), при котором по умолчанию используется схема `pg_catalog`, и поэтому операторы, определённые в другой схеме, могут оказаться недоступными.

Чтобы предотвратить проявление этой проблемы, везде, где это возможно, пишите имена операторов, использующих нестандартные типы, в полной форме:

```
OPERATOR(schema.operator)
```

. За подробностями обратитесь к [Подразделу 5.9.3](#).

Проблема также затрагивает SQL-конструкции, в которых оператор сравнения не указан явно, и поэтому их нельзя записать с именем схемы. К ним, например, относятся конструкции `IS DISTINCT FROM`, `IS NOT DISTINCT FROM`, `IN/NOT IN`, `NULLIF`, `JOIN ... USING`, `NATURAL JOIN` и `CASE a WHEN b`. В таких случаях предотвратить ошибки можно только на уровне кода:

- Явно указывайте оператор сравнения. Например, замените простой `CASE` на `CASE` с перебором условий:

```
CASE WHEN a OPERATOR(schema.=) b THEN ...;
```

За подробностями обратитесь к [Подразделу 42.6.4](#).

- Если нестандартный тип часто используется в неявных сравнениях, попробуйте определить соответствующие операторы в схеме `pg_catalog`.
- При большом количестве подобных конструкций проверяйте их поведение после обновления или восстановления и при необходимости вносите правки в код вручную.

10.3. Функции

При выборе конкретной функции, задействованной в выражении, PostgreSQL следует описанному ниже алгоритму.

Разрешение функции по типу

1. Выбрать функции для рассмотрения из системного каталога `pg_proc`. Если имя функции не дополнено именем схемы, будут рассматриваться все функции с подходящим именем и числом аргументов, видимые в текущем пути поиска (см. [Подраздел 5.9.3](#)). Если имя функции определено полностью, в рассмотрение принимаются только функции из указанной схемы.
 - a. (Optional) Если в пути поиска оказывается несколько функций с одинаковыми типами аргументов, учитываются только те из них, которые находятся в пути раньше. Функции с разными типами аргументов рассматриваются на равных правах вне зависимости от их положения в пути поиска.
 - b. (Optional) Если в числе параметров функции есть массив `VARIADIC` и при вызове не указывается ключевое слово `VARIADIC`, функция обрабатывается, как если бы этот параметр был

заменён одним или несколькими параметрами типа элементов массива, по числу аргументов при вызове. После такого расширения по фактическим типам аргументов она может совпасть с некоторой функцией с постоянным числом аргументов. В этом случае используется функция, которая находится в пути раньше, а если они оказываются в одной схеме, предпочитается вариант с постоянными аргументами.

Это создаёт угрозу безопасности при вызове с полным именем ² функции с переменным числом аргументов, которая может оказаться в схеме, где могут создавать объекты недоверенные пользователи. Злонамеренный пользователь может перехватывать управление и выполнять произвольные SQL-функции, как будто их выполняете вы. Запись вызова с ключевым словом `VARIADIC` устраняет эту угрозу. Однако для вызовов с передачей параметров `VARIADIC "any"` часто не существует необходимой формулировки с ключом `VARIADIC`. Чтобы такие вызовы были безопасными, создание объектов в схеме функции должно разрешаться только доверенным пользователям.

- c. (Optional) Функции, для которых определены значения параметров по умолчанию, считаются совпадающими с вызовом, в котором опущено ноль или более параметров в соответствующих позициях. Если для вызова подходят несколько функций, используется та, что обнаруживается в пути поиска раньше. Если в одной схеме оказываются несколько функций с одинаковыми типами в позициях обязательных параметров (что возможно, если в них определены разные наборы пропускаемых параметров), система не сможет выбрать оптимальную, и выдаст ошибку «неоднозначный вызов функции», если лучшее соответствие для вызова не будет найдено.

Это создаёт угрозу при вызове с полным именем ² любой функции, которая может оказаться в схеме, где могут создавать объекты недоверенные пользователи. Злонамеренный пользователь может создать функцию с именем уже существующей, продублировав параметры исходной и добавив дополнительные со значениями по умолчанию. В результате при последующих вызовах будет выполняться не исходная функция. Для ликвидации этой угрозы помещайте функции в схемы, в которых создавать объекты могут только доверенные объекты.

2. Проверить, нет ли функции, принимающей в точности типы входных аргументов. Если такая функция есть (она может быть только одной в отобранном ранее наборе), использовать её. Отсутствие точного совпадения создаёт угрозу вызова с полным именем ² функции в схеме, где могут создавать объекты недоверенные пользователи. В таких ситуациях приведите типы аргументов для получения точного соответствия. (В случаях с `unknown` совпадения на этом этапе не будет никогда.)
3. Если точное совпадение не найдено, проверить, не похож ли вызов функции на особую форму преобразования типов. Это имеет место, когда при вызове функции передаётся всего один аргумент и имя функции совпадает с именем (внутренним) некоторого типа данных. Более того, аргументом функции должна быть либо строка неопределённого типа, либо значение типа, двоично-совместимого с указанным или приводимого к нему с помощью функций ввода-вывода типа (то есть, преобразований в стандартный строковый тип и обратно). Если эти условия выполняются, вызов функции воспринимается как особая форма конструкции `CAST`.³
4. Найти самый подходящий.
 - a. Отбросить кандидатов, для которых входные типы не совпадают и не могут быть преобразованы (неявным образом) так, чтобы они совпали. В данном случае считается, что константы типа `unknown` можно преобразовать во что угодно. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
 - b. Если один из аргументов имеет тип домен, далее считать его типом базовый тип домена. Благодаря этому при поиске неоднозначно заданной функции домены будут подобны своим базовым типам.

²Эта угроза неактуальна для имён без схемы, так как путь поиска, содержащий схемы, в которых недоверенные пользователи могут создавать объекты, не соответствует шаблону безопасного использования схем.

³Этот шаг нужен для поддержки приведений типов в стиле вызова функции, когда на самом деле соответствующей функции приведения нет. Если такая функция приведения есть, она обычно называется именем выходного типа и необходимости в особом подходе нет. За дополнительными комментариями обратитесь к [CREATE CAST](#).

- c. Просмотреть всех кандидатов и оставить только тех, для которых точно совпадают как можно больше типов аргументов. Оставить всех кандидатов, если точных совпадений нет. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
- d. Просмотреть всех кандидатов и оставить только тех, которые принимают предпочитаемые типы (из категории типов входных значений) в наибольшем числе позиций, где требуется преобразование типов. Оставить всех кандидатов, если ни один не принимает предпочитаемые типы. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
- e. Если какие-либо значения имеют тип `unknown`, проверить категории типов, принимаемых в данных позициях аргументов оставшимися кандидатами. Для каждой позиции выбрать категорию `string`, если какой-либо кандидат принимает эту категорию. (Эта склонность к строкам объясняется тем, что константа типа `unknown` выглядит как строка.) Если эта категория не подходит, но все оставшиеся кандидаты принимают одну категорию, выбрать её; в противном случае констатировать неудачу — сделать правильный выбор без дополнительных подсказок нельзя. Затем отбросить кандидатов, которые не принимают типы выбранной категории. Далее, если какой-либо кандидат принимает предпочитаемый тип из этой категории, отбросить кандидатов, принимающих другие, не предпочитаемые типы для данного аргумента. Оставить всех кандидатов, если эти проверки не прошёл ни один. Если остаётся только один кандидат, использовать его, в противном случае перейти к следующему шагу.
- f. Если в списке аргументов есть аргументы и типа `unknown`, и известного типа, и этот известный тип один для всех аргументов, предположить, что аргументы типа `unknown` также имеют этот тип, и проверить, какие кандидаты могут принимать этот тип в позиции аргумента `unknown`. Если остаётся только один кандидат, использовать его, в противном случае констатировать неудачу.

Заметьте, что для функций действуют те же правила «оптимального соответствия», что и для операторов. Они проиллюстрированы следующими примерами.

Пример 10.6. Разрешение функции округления по типам аргументов

В PostgreSQL есть только одна функция `round`, принимающая два аргумента: первый типа `numeric`, а второй — `integer`. Поэтому в следующем запросе первый аргумент `integer` автоматически приводится к типу `numeric`:

```
SELECT round(4, 4);
```

```
round
-----
4.0000
(1 row)
```

Таким образом, анализатор преобразует этот запрос в:

```
SELECT round(CAST (4 AS numeric), 4);
```

Так как числовые константы с десятичными точками изначально относятся к типу `numeric`, для следующего запроса преобразование типов не потребуется, так что он немного эффективнее:

```
SELECT round(4.0, 4);
```

Пример 10.7. Разрешение функций с переменными параметрами

```
CREATE FUNCTION public.variadic_example(VARIADIC numeric[]) RETURNS int
    LANGUAGE sql AS 'SELECT 1';
CREATE FUNCTION
```

Эта функция принимает в аргументах ключевое слово `VARIADIC`, но может вызываться и без него. Ей можно передавать и целочисленные, и любые числовые аргументы:

```
SELECT public.variadic_example(0),
```

```

    public.variadic_example(0.0),
    public.variadic_example(VARIADIC array[0.0]);
variadic_example | variadic_example | variadic_example
-----+-----+-----
          1 |              1 |              1
(1 row)

```

Однако для первого и второго вызова предпочтительнее окажутся специализированные функции, если таковые есть:

```

CREATE FUNCTION public.variadic_example(numeric) RETURNS int
  LANGUAGE sql AS 'SELECT 2';
CREATE FUNCTION

```

```

CREATE FUNCTION public.variadic_example(int) RETURNS int
  LANGUAGE sql AS 'SELECT 3';
CREATE FUNCTION

```

```

SELECT public.variadic_example(0),
       public.variadic_example(0.0),
       public.variadic_example(VARIADIC array[0.0]);
variadic_example | variadic_example | variadic_example
-----+-----+-----
          3 |              2 |              1
(1 row)

```

Если используется конфигурация по умолчанию и существует только первая функция, первый и второй вызовы будут небезопасными. Любой пользователь может перехватить их, создав вторую или третью функцию. Безопасным будет третий вызов, в котором тип аргумента соответствует в точности и используется ключевое слово `VARIADIC`.

Пример 10.8. Разрешение функции извлечения подстроки

В PostgreSQL есть несколько вариантов функции `substr`, и один из них принимает аргументы типов `text` и `integer`. Если эта функция вызывается со строковой константой неопределённого типа, система выбирает функцию, принимающую аргумент предпочитаемой категории `string` (а конкретнее, типа `text`).

```
SELECT substr('1234', 3);
```

```

substr
-----
      34
(1 row)

```

Если текстовая строка имеет тип `varchar`, например когда данные поступают из таблицы, анализатор попытается привести её к типу `text`:

```
SELECT substr (varchar '1234', 3);
```

```

substr
-----
      34
(1 row)

```

Этот запрос анализатор фактически преобразует в:

```
SELECT substr(CAST (varchar '1234' AS text), 3);
```

Примечание

Анализатор узнаёт из каталога `pg_cast`, что типы `text` и `varchar` двоично-совместимы, что означает, что один тип можно передать функции, принимающей другой, не выполняя физи-

ческого преобразования. Таким образом, в данном случае операция преобразования на самом не добавляется.

И если функция вызывается с аргументом типа `integer`, анализатор попытается преобразовать его в тип `text`:

```
SELECT substr(1234, 3);
```

ОШИБКА: функция `substr(integer, integer)` не существует

ПОДСКАЗКА: Функция с данными именем и типами аргументов не найдена. Возможно, вам следует добавить явные преобразования типов.

Этот вариант не работает, так как `integer` нельзя неявно преобразовать в `text`. Однако с явным преобразованием запрос выполняется:

```
SELECT substr(CAST (1234 AS text), 3);
```

```
substr
-----
      34
(1 row)
```

10.4. Хранимое значение

Значения, вставляемые в таблицу, преобразуются в тип данных целевого столбца по следующему алгоритму.

Преобразование по типу хранения

1. Проверить точное совпадение с целевым типом.
2. Если типы не совпадают, попытаться привести тип к целевому. Это возможно, если в каталоге `pg_cast` (см. [CREATE CAST](#)) зарегистрировано *приведение присваивания* между двумя типами. Если же результат выражения — строка неизвестного типа, содержимое этой строки будет подано на вход процедуре ввода целевого типа.
3. Проверить, не требуется ли приведение размера для целевого типа. Приведение размера — это преобразование типа к такому же. Если это приведение описано в каталоге `pg_cast`, применить к его к результату выражения, прежде чем сохранить в целевом столбце. Функция, реализующая такое приведение, всегда принимает дополнительный параметр типа `integer`, в котором передаётся значение `atttypmod` для целевого столбца (обычно это её объявленный размер, хотя интерпретироваться значение `atttypmod` для разных типов данных может по-разному), и третий параметр типа `boolean`, передающий признак явное/неявное преобразование. Функция приведения отвечает за все операции с длиной, включая её проверку и усечение данных.

Пример 10.9. Преобразование для типа хранения `character`

Следующие запросы показывают, что сохраняемое значение подгоняется под размер целевого столбца, объявленного как `character(20)`:

```
CREATE TABLE vv (v character(20));
INSERT INTO vv SELECT 'abc' || 'def';
SELECT v, octet_length(v) FROM vv;
```

```
      v                | octet_length
-----+-----
 abcdef                |             20
(1 row)
```

Суть происходящего здесь в том, что две константы неизвестного типа по умолчанию воспринимаются как значения `text`, что позволяет применить к ним оператор `||` как оператор конкатенации значений `text`. Затем результат оператора, имеющий тип `text`, приводится к типу `bpchar` («blank-

padded char» (символы, дополненные пробелами), внутреннее имя типа `character`) в соответствии с типом целевого столбца. (Так как типы `text` и `bpchar` двоично-совместимы, при этом преобразовании реальный вызов функции не добавляется.) Наконец, в системном каталоге находится функция изменения размера `bpchar(bpchar, integer, boolean)` и применяется для результата оператора и длины столбца. Эта связанная с типом функция проверяет длину данных и добавляет недостающие пробелы.

10.5. UNION, CASE и связанные конструкции

SQL-конструкция `UNION` взаимодействует с системой типов, так как ей приходится объединять значения возможно различных типов в единый результирующий набор. Алгоритм разрешения типов при этом применяется независимо к каждому выходному столбцу запроса. Конструкции `INTERSECT` и `EXCEPT` сопоставляют различные типы подобно `UNION`. По такому же алгоритму сопоставляют типы выражений и определяют тип своего результата некоторые другие конструкции, включая `CASE`, `ARRAY`, `VALUES` и функции `GREATEST` и `LEAST`.

Разрешение типов для UNION, CASE и связанных конструкций

1. Если все данные одного типа и это не тип `unknown`, выбрать его.
2. Если тип данных — домен, далее считать их типом базовый тип домена.⁴
3. Если все данные типа `unknown`, выбрать для результата тип `text` (предпочитаемый для категории `string`). В противном случае значения `unknown` для остальных правил игнорируются.
4. Если известные типы входных данных оказываются не из одной категории, констатировать неудачу.
5. Выбрать первый известный тип данных в качестве типа-кандидата, затем рассмотреть все остальные известные типы данных, слева направо.⁵ Если ранее выбранный тип может быть неявно преобразован к другому типу, но преобразовать второй в первый нельзя, выбрать второй тип в качестве нового кандидата. Затем продолжать рассмотрение последующих данных. Если на любом этапе этого процесса выбирается предпочитаемый тип, следующие данные больше не рассматриваются.
6. Привести все данные к окончательно выбранному типу. Констатировать неудачу, если неявное преобразование из типа входных данных в выбранный тип невозможно.

Ниже это проиллюстрировано на примерах.

Пример 10.10. Разрешение типов с частичным определением в Union

```
SELECT text 'a' AS "text" UNION SELECT 'b';
```

```
text
-----
a
b
(2 rows)
```

В данном случае константа `'b'` неизвестного типа будет преобразована в тип `text`.

Пример 10.11. Разрешение типов в простом объединении

```
SELECT 1.2 AS "numeric" UNION SELECT 1;
```

```
numeric
-----
1
```

⁴Так же, как домены воспринимаются при выборе операторов и функций, доменные типы могут сохраняться в конструкции `UNION` или подобной, если пользователь позаботится о том, чтобы все входные данные приводились к этому типу явно или неявно. В противном случае будет использоваться базовый тип домена.

⁵По историческим причинам в конструкции `CASE` выражение в предложении `ELSE` (если оно есть) обрабатывается как «первое», а предложения `THEN` рассматриваются после. Во всех остальных случаях, «слева направо» означает порядок, в котором выражения действительно идут в тексте запроса.

```
1.2
(2 rows)
```

Константа 1.2 имеет тип `numeric` и целочисленное значение 1 может быть неявно приведено к типу `numeric`, так что используется этот тип.

Пример 10.12. Разрешение типов в противоположном объединении

```
SELECT 1 AS "real" UNION SELECT CAST('2.2' AS REAL);
```

```
real
-----
1
2.2
(2 rows)
```

Здесь значение типа `real` нельзя неявно привести к `integer`, но `integer` можно неявно привести к `real`, поэтому типом результата объединения будет `real`.

Пример 10.13. Разрешение типов во вложенном объединении

```
SELECT NULL UNION SELECT NULL UNION SELECT 1;
```

```
ERROR:  UNION types text and integer cannot be matched
```

Эта ошибка возникает из-за того, что Postgres Pro воспринимает множественные `UNION` как пары с вложенными операциями, то есть как запись

```
(SELECT NULL UNION SELECT NULL) UNION SELECT 1;
```

Внутренний `UNION` разрешается как выдающий тип `text`, согласно правилам, приведённым выше. Затем внешний `UNION` получает на вход типы `text` и `integer`, что и приводит к показанной ошибке. Эту проблему можно устранить, сделав так, чтобы у самого левого `UNION` минимум с одной стороны были данные желаемого типа результата.

Операции `INTERSECT` и `EXCEPT` также разрешаются по парам. Однако остальные конструкции, описанные в этом разделе, рассматривают все входные данные сразу.

10.6. Выходные столбцы SELECT

Правила, описанные в предыдущих разделах, распространяются на присваивание типов данных, кроме `unknown`, во всех выражениях в запросах SQL, за исключением бестиповых буквальных значений, принимающих вид простых выходных столбцов команды `SELECT`. Например, в запросе

```
SELECT 'Hello World';
```

ничто не говорит о том, какой тип должно принимать строковое буквальное значение. В этой ситуации Postgres Pro разрешит тип такого значения как `text`.

Когда `SELECT` является одной из ветвей конструкции `UNION` (или `INTERSECT/EXCEPT`) или когда он находится внутри `INSERT ... SELECT`, это правило не действует, так как более высокий приоритет имеют правила, описанные в предыдущих разделах. В первом случае тип бестипового буквального значения может быть получен из другой ветви `UNION`, а во втором — из целевого столбца.

Списки `RETURNING` в данном контексте воспринимаются так же, как выходные списки `SELECT`.

Примечание

До Postgres Pro 10 этого правила не было и бестиповые буквальное значения в выходном списке `SELECT` оставались с типом `unknown`. Это имело различные негативные последствия, так что было решено это изменить.

Глава 11. Индексы

Индексы — это традиционное средство увеличения производительности БД. Используя индекс, сервер баз данных может находить и извлекать нужные строки гораздо быстрее, чем без него. Однако с индексами связана дополнительная нагрузка на СУБД в целом, поэтому применять их следует обдуманно.

11.1. Введение

Предположим, что у нас есть такая таблица:

```
CREATE TABLE test1 (  
    id integer,  
    content varchar  
);
```

и приложение выполняет много подобных запросов:

```
SELECT content FROM test1 WHERE id = константа;
```

Если система не будет заранее подготовлена, ей придётся сканировать всю таблицу `test1`, строку за строкой, чтобы найти все подходящие записи. Когда таблица `test1` содержит большое количество записей, а этот запрос должен вернуть всего несколько (возможно, одну или ноль), такое сканирование, очевидно, неэффективно. Но если создать в системе индекс по полю `id`, она сможет находить строки гораздо быстрее. Возможно, для этого ей понадобится опуститься всего на несколько уровней в дереве поиска.

Подобный подход часто используется в технической литературе: термины и понятия, которые могут представлять интерес, собираются в алфавитном указателе в конце книги. Читатель может просмотреть этот указатель довольно быстро и затем перейти сразу к соответствующей странице, вместо того, чтобы пролистывать всю книгу в поисках нужного материала. Так же, как задача автора предугадать, что именно будут искать в книге читатели, задача программиста баз данных — заранее определить, какие индексы будут полезны.

Создать индекс для столбца `id` рассмотренной ранее таблицы можно с помощью следующей команды:

```
CREATE INDEX test1_id_index ON test1 (id);
```

Имя индекса `test1_id_index` может быть произвольным, главное, чтобы оно позволяло понять, для чего этот индекс.

Для удаления индекса используется команда `DROP INDEX`. Добавлять и удалять индексы можно в любое время.

Когда индекс создан, никакие дополнительные действия не требуются: система сама будет обновлять его при изменении данных в таблице и сама будет использовать его в запросах, где, по её мнению, это будет эффективнее, чем сканирование всей таблицы. Вам, возможно, придётся только периодически запускать команду `ANALYZE` для обновления статистических данных, на основе которых планировщик запросов принимает решения. В [Главе 14](#) вы можете узнать, как определить, используется ли определённый индекс и при каких условиях планировщик может решить не использовать его.

Индексы могут быть полезны также при выполнении команд `UPDATE` и `DELETE` с условиями поиска. Кроме того, они могут применяться в поиске с соединением. То есть, индекс, определённый для столбца, участвующего в условии соединения, может значительно ускорить запросы с `JOIN`.

В целом индексы PostgreSQL можно использовать для оптимизации запросов, содержащих одно или несколько предложений `WHERE` или `JOIN` в таком виде:

индексированный-столбец индексировемый-оператор значение-сравнения

Здесь *индексированный-столбец* — это любой столбец или выражение, для которого был определён индекс, *индексируемый-оператор* — это оператор, который является членом класса *операторов* индекса для индексированного столбца (более подробную информацию об этом см. ниже), а *значение-сравнения* может быть любым выражением, которое не является изменчивым и не ссылается на таблицу с этим индексом.

В некоторых случаях планировщик запросов может извлечь индексируемое предложение этого вида из другой конструкции SQL. Например, если исходное предложение выглядело так:

значение сравнения оператор индексированный столбец

, то его можно преобразовать в индексируемую форму, если исходный *оператор* имеет коммутирующий оператор, который является членом класса операторов индекса.

Создание индекса для большой таблицы может занимать много времени. По умолчанию Postgres Pro позволяет параллельно с созданием индекса выполнять чтение (операторы `SELECT`) таблицы, но операции записи (`INSERT`, `UPDATE` и `DELETE`) блокируются до окончания построения индекса. Для производственной среды это ограничение часто бывает неприемлемым. Хотя есть возможность разрешить запись параллельно с созданием индексов, при этом нужно учитывать ряд оговорок — они описаны в подразделе [Неблокирующее построение индексов](#).

После создания индекса система должна поддерживать его в состоянии, соответствующем данным таблицы. С этим связаны неизбежные накладные расходы при изменении данных. Наличие индексов также может препятствовать созданию [кортежей только в куче](#). Таким образом, индексы, которые используются в запросах редко или вообще никогда, должны быть удалены.

11.2. Типы индексов

Postgres Pro поддерживает несколько типов индексов: B-дерево, хеш, GiST, SP-GiST, GIN, BRIN и расширение [bloom](#). Для разных типов индексов применяются разные алгоритмы, ориентированные на определённые типы индексируемых предложений. По умолчанию команда `CREATE INDEX` создаёт индексы-B-деревья, эффективные в большинстве случаев. Выбрать другой тип можно, написав название типа индекса после ключевого слова `USING`. Например, создать хеш-индекс можно так:

```
CREATE INDEX имя ON таблица USING HASH (столбец);
```

11.2.1. B-дерево

B-деревья могут работать в условиях на равенство и в проверках диапазонов с данными, которые можно отсортировать в некотором порядке. Точнее, планировщик запросов Postgres Pro может задействовать индекс-B-дерева, когда индексируемый столбец участвует в сравнении с одним из следующих операторов:

```
<    <=    =    >=    >
```

При обработке конструкций, представимых как сочетание этих операторов, например `BETWEEN` и `IN`, так же может выполняться поиск по индексу-B-дерева. Кроме того, такие индексы могут использоваться и в условиях `IS NULL` и `IS NOT NULL` по индексированным столбцам.

Также оптимизатор может использовать эти индексы в запросах с операторами сравнения по шаблону `LIKE` и `~`, если этот шаблон определяется константой и он привязан к началу строки — например, `col LIKE 'foo%'` или `col ~ '^foo'`, но не `col LIKE '%bar'`. Но если ваша база данных использует не локаль C, для поддержки индексирования запросов с шаблонами вам потребуется создать индекс со специальным классом операторов; см. [Раздел 11.10](#). Индексы-B-деревья можно использовать и для `ILIKE` и `~*`, но только если шаблон начинается не с алфавитных символов, то есть символов, не подверженных преобразованию регистра.

B-деревья могут также применяться для получения данных, отсортированных по порядку. Это не всегда быстрее простого сканирования и сортировки, но иногда бывает полезно.

11.2.2. Хеш

Хеш-индексы хранят 32-битный хеш-код, полученный из значения индексируемого столбца, поэтому хеш-индексы работают только с простыми условиями равенства. Планировщик запросов может применить хеш-индекс, только если индексируемый столбец участвует в сравнении с оператором `=`. Создать такой индекс можно следующей командой:

```
CREATE INDEX имя ON таблица USING HASH (столбец);
```

11.2.3. GiST

GiST-индексы представляют собой не просто разновидность индексов, а инфраструктуру, позволяющую реализовать много разных стратегий индексирования. Как следствие, GiST-индексы могут применяться с разными операторами, в зависимости от стратегии индексирования (*класса операторов*). Например, стандартный дистрибутив Postgres Pro включает классы операторов GiST для нескольких двумерных типов геометрических данных, что позволяет применять индексы в запросах с операторами:

```
<<    &<    &>    >>    <<|    &<|    |&>    |>>    @>    <@    ~=    &&
```

(Эти операторы описаны в [Разделе 9.11.](#)) Классы операторов GiST, включённые в стандартный дистрибутив, описаны в [Таблице 65.1](#). В коллекции `contrib` можно найти и другие классы операторов GiST, реализованные как отдельные проекты. За дополнительными сведениями обратитесь к [Главе 65](#).

GiST-индексы также могут оптимизировать поиск «ближайшего соседа», например такой:

```
SELECT * FROM places ORDER BY location <-> point '(101,456)' LIMIT 10;
```

который возвращает десять расположений, ближайших к заданной точке. Возможность такого применения индекса опять же зависит от класса используемого оператора. Операторы, которые можно использовать таким образом, перечислены в [Таблице 65.1](#), в столбце «Операторы сортировки».

11.2.4. SP-GiST

Индексы SP-GiST, как и GiST, предоставляют инфраструктуру, поддерживающую различные типы поиска. SP-GiST позволяет организовывать на диске самые разные несбалансированные структуры данных, такие как деревья квадрантов, k-мерные и префиксные деревья. Например, стандартный дистрибутив Postgres Pro включает классы операторов SP-GiST для точек в двумерном пространстве, что позволяет применять индексы в запросах с операторами:

```
<<    >>    ~=    <@    <<|    |>>
```

(Эти операторы описаны в [Разделе 9.11.](#)) Классы операторов SP-GiST, включённые в стандартный дистрибутив, описаны в [Таблице 66.1](#). За дополнительными сведениями обратитесь к [Главе 66](#).

Индексы SP-GiST, как и GiST, поддерживают поиск ближайших соседей. Для классов операторов SP-GiST, поддерживающих упорядочивание по расстоянию, соответствующий оператор указан в столбце «Операторы упорядочивания» в [Таблице 66.1](#).

11.2.5. GIN

GIN-индексы представляют собой «инвертированные индексы», в которых могут содержаться значения с несколькими ключами, например массивы. Инвертированный индекс содержит отдельный элемент для значения каждого компонента, и может эффективно работать в запросах, проверяющих присутствие определённых значений компонентов.

Подобно GiST и SP-GiST, индексы GIN могут поддерживать различные определённые пользователем стратегии и в зависимости от них могут применяться с разными операторами. Например, стандартный дистрибутив Postgres Pro включает класс операторов GIN для массивов, что позволяет применять индексы в запросах с операторами:

<@ @> = &&

(Эти операторы описаны в [Разделе 9.19.](#)) Классы операторов GIN, включённые в стандартный дистрибутив, описаны в [Таблице 67.1](#). В коллекции contrib и в отдельных проектах можно найти и много других классов операторов GIN. За дополнительными сведениями обратитесь к [Главе 67](#).

11.2.6. BRIN

BRIN-индексы (сокращение от Block Range INdexes, Индексы зон блоков) хранят обобщённые сведения о значениях, находящихся в физически последовательно расположенных блоках таблицы. Поэтому такие индексы наиболее эффективны для столбцов, значения в которых хорошо коррелируют с физическим порядком столбцов таблицы. Подобно GiST, SP-GiST и GIN, индексы BRIN могут поддерживать определённые пользователем стратегии и в зависимости от них применяться с разными операторами. Для типов данных, имеющих линейный порядок сортировки, записям в индексе соответствуют минимальные и максимальные значения данных в столбце для каждой зоны блоков. Это позволяет поддерживать запросы со следующими операторами:

< <= = >= >

Классы операторов BRIN, включённые в стандартный дистрибутив, описаны в [Таблице 68.1](#). За дополнительными сведениями обратитесь к [Главе 68](#).

11.3. Составные индексы

Индексы можно создавать и по нескольким столбцам таблицы. Например, если у вас есть таблица:

```
CREATE TABLE test2 (
    major int,
    minor int,
    name varchar
);
```

(предположим, что вы поместили в неё содержимое каталога /dev) и вы часто выполняете запросы вида:

```
SELECT name FROM test2 WHERE major = константа AND minor = константа;
```

тогда имеет смысл определить индекс, покрывающий оба столбца major и minor. Например:

```
CREATE INDEX test2_mm_idx ON test2 (major, minor);
```

В настоящее время составными могут быть только индексы типов В-дерево, GiST, GIN и BRIN. Возможность построения индекса по нескольким ключевым столбцам не зависит от возможности добавления в индекс неключевых столбцов (INCLUDE). Число столбцов в индексе ограничивается 32, включая столбцы INCLUDE. (Этот предел можно изменить при компиляции Postgres Pro.)

Составной индекс-В-дерево может применяться в условиях с любым подмножеством столбцов индекса, но наиболее эффективен он при ограничениях по ведущим (левым) столбцам. Точное правило состоит в том, что сканируемая область индекса определяется условиями равенства с ведущими столбцами и условиями неравенства с первым столбцом, не участвующим в условии равенства. Ограничения столбцов правее них также проверяются по индексу, так что обращение к таблице откладывается, но на размер сканируемой области индекса это уже не влияет. Например, если есть индекс по столбцам (a, b, c) и условие WHERE a = 5 AND b >= 42 AND c < 77, индекс будет сканироваться от первой записи a = 5 и b = 42 до последней с a = 5. Записи индекса, в которых c >= 77, не будут учитываться, но тем не менее будут просканированы. Этот индекс в принципе может использоваться в запросах с ограничениями по b и/или c, без ограничений столбца a, но при этом будет просканирован весь индекс, так что в большинстве случаев планировщик предпочтёт использованию индекса полное сканирование таблицы.

Составной индекс GiST может применяться в условиях с любым подмножеством столбцов индекса. Условия с дополнительными столбцами ограничивают записи, возвращаемые индексом, но в первую очередь сканируемая область индекса определяется ограничением первого столбца. GiST-

индекс будет относительно малоэффективен, когда первый его столбец содержит только несколько различающихся значений, даже если дополнительные столбцы дают множество различных значений.

Составной индекс GIN может применяться в условиях с любым подмножеством столбцов индекса. В отличие от индексов GiST или B-деревьев, эффективность поиска по нему не меняется в зависимости от того, какие из его столбцов используются в условиях запроса.

Составной индекс BRIN может применяться в условиях запроса с любым подмножеством столбцов индекса. Подобно индексу GIN и в отличие от B-деревьев или GiST, эффективность поиска по нему не меняется в зависимости от того, какие из его столбцов используются в условиях запроса. Единственное, зачем в одной таблице могут потребоваться несколько индексов BRIN вместо одного составного индекса — это затем, чтобы применялись разные параметры хранения `pages_per_range`.

При этом, разумеется, каждый столбец должен использоваться с операторами, соответствующими типу индекса; ограничения с другими операторами рассматриваться не будут.

Составные индексы следует использовать обдуманно. В большинстве случаев индекс по одному столбцу будет работать достаточно хорошо и сэкономит время и место. Индексы по более чем трём столбцам вряд ли будут полезными, если только таблица не используется крайне однообразно. Описание достоинств различных конфигураций индексов можно найти в [Разделе 11.5](#) и [Разделе 11.9](#).

11.4. Индексы и предложения ORDER BY

Помимо простого поиска строк для выдачи в результате запроса, индексы также могут применяться для сортировки строк в определённом порядке. Это позволяет учесть предложение `ORDER BY` в запросе, не выполняя сортировку дополнительно. Из всех типов индексов, которые поддерживает Postgres Pro, сортировать данные могут только B-деревья — индексы других типов возвращают строки в неопределённом, зависящем от реализации порядке.

Планировщик может выполнить указание `ORDER BY`, либо просканировав существующий индекс, подходящий этому указанию, либо просканировав таблицу в физическом порядке и выполнив сортировку явно. Для запроса, требующего сканирования большей части таблицы, явная сортировка скорее всего будет быстрее, чем применение индекса, так как при последовательном чтении она потребует меньше операций ввода-вывода. Важный особый случай представляет `ORDER BY` в сочетании с `LIMIT n`: при явной сортировке системе потребуется обработать все данные, чтобы выбрать первые n строк, но при наличии индекса, соответствующего столбцам в `ORDER BY`, первые n строк можно получить сразу, не просматривая остальные вовсе.

По умолчанию элементы B-дерева хранятся в порядке возрастания, при этом значения `NULL` идут в конце (для упорядочивания равных записей используется табличный столбец `TID`). Это означает, что при прямом сканировании индекса по столбцу x порядок оказывается соответствующим указанию `ORDER BY x` (или точнее, `ORDER BY x ASC NULLS LAST`). Индекс также может сканироваться в обратную сторону, и тогда порядок соответствует указанию `ORDER BY x DESC` (или точнее, `ORDER BY x DESC NULLS FIRST`, так как для `ORDER BY DESC` подразумевается `NULLS FIRST`).

Вы можете изменить порядок сортировки элементов B-дерева, добавив уточнения `ASC`, `DESC`, `NULLS FIRST` и/или `NULLS LAST` при создании индекса; например:

```
CREATE INDEX test2_info_nulls_low ON test2 (info NULLS FIRST);
CREATE INDEX test3_desc_index ON test3 (id DESC NULLS LAST);
```

Индекс, в котором элементы хранятся в порядке возрастания и значения `NULL` идут первыми, может удовлетворять указанию `ORDER BY x ASC NULLS FIRST` или `ORDER BY x DESC NULLS LAST`, в зависимости от направления просмотра.

У вас может возникнуть вопрос, зачем нужны все четыре варианта при создании индексов, когда и два варианта с учётом обратного просмотра покрывают все виды `ORDER BY`. Для индексов по одному столбцу это и в самом деле излишне, но для индексов по многим столбцам это может быть

полезно. Рассмотрим индекс по двум столбцам (x , y): он может удовлетворять указанию `ORDER BY x`, y при прямом сканировании или `ORDER BY x DESC`, y `DESC` при обратном. Но вполне возможно, что приложение будет часто выполнять `ORDER BY x ASC`, y `DESC`. В этом случае получить такую сортировку от простого индекса нельзя, но можно получить подходящий индекс, определив его как (x `ASC`, y `DESC`) или (x `DESC`, y `ASC`).

Очевидно, что индексы с нестандартными правилами сортировки весьма специфичны, но иногда они могут кардинально ускорить определённые запросы. Стоит ли вводить такие индексы, зависит от того, как часто выполняются запросы с необычным порядком сортировки.

11.5. Объединение нескольких индексов

При простом сканировании индекса могут обрабатываться только те предложения в запросе, в которых применяются операторы его класса и объединяет их `AND`. Например, для индекса (a , b) условие запроса `WHERE a = 5 AND b = 6` сможет использовать этот индекс, а запрос `WHERE a = 5 OR b = 6` — нет.

К счастью, `Postgres Pro` способен соединять несколько индексов (и в том числе многократно применять один индекс) и охватывать также случаи, когда сканирование одного индекса недостаточно. Система может сформировать условия `AND` и `OR` за несколько проходов индекса. Например, запрос `WHERE x = 42 OR x = 47 OR x = 53 OR x = 99` можно разбить на четыре сканирования индекса по x , по сканированию для каждой части условия. Затем результаты этих сканирований будут логически сложены (`OR`) вместе и дадут конечный результат. Другой пример — если у нас есть отдельные индексы по x и y , запрос `WHERE x = 5 AND y = 6` можно выполнить, применив индексы для соответствующих частей запроса, а затем вычислив логическое произведение (`AND`) для найденных строк, которое и станет конечным результатом.

Выполняя объединение нескольких индексов, система сканирует все необходимые индексы и создаёт в памяти *битовую карту* расположения строк таблицы, которые удовлетворяют условиям каждого индекса. Затем битовые карты объединяются операциями `AND` и `OR`, как того требуют условия в запросе. Наконец система обращается к соответствующим отмеченным строкам таблицы и возвращает их данные. Строки таблицы просматриваются в физическом порядке, как они представлены в битовой карте; это означает, что порядок сортировки индексов при этом теряется и в запросах с предложением `ORDER BY` сортировка будет выполняться отдельно. По этой причине, а также потому, что каждое сканирование индекса занимает дополнительное время, планировщик иногда выбирает простое сканирование индекса, несмотря на то, что можно было бы подключить и дополнительные индексы.

В большинстве приложений (кроме самых простых) полезными могут оказаться различные комбинации индексов, поэтому разработчик баз данных, определяя набор индексов, должен искать компромиссное решение. Иногда оказываются хороши составные индексы, а иногда лучше создать отдельные индексы и положиться на возможности объединения индексов. Например, если типичную нагрузку составляют запросы иногда с условием только по столбцу x , иногда только по y , а иногда по обоим столбцам, вы можете ограничиться двумя отдельными индексами по x и y , рассчитывая на то, что при обработке условий с обоими столбцами эти индексы будут объединяться. С другой стороны, вы можете создать один составной индекс по (x , y). Этот индекс скорее всего будет работать эффективнее, чем объединение индексов, в запросах с двумя столбцами, но как говорилось в [Разделе 11.3](#), он будет практически бесполезен для запросов с ограничениями только по y , так что одного этого индекса будет недостаточно. Выигрышным в этом случае может быть сочетание составного индекса с отдельным индексом по y . В запросах, где задействуется только x , может применяться составной индекс, хотя он будет больше и, следовательно, медленнее индекса по одному x . Наконец, можно создать все три индекса, но это будет оправдано, только если данные в таблице изменяются гораздо реже, чем выполняется поиск в таблице, при этом частота запросов этих трёх типов примерно одинакова. Если запросы какого-то одного типа выполняются гораздо реже других, возможно лучше будет оставить только два индекса, соответствующих наиболее частым запросам.

11.6. Уникальные индексы

Индексы также могут обеспечивать уникальность значения в столбце или уникальность сочетания значений в нескольких столбцах.

```
CREATE UNIQUE INDEX имя ON таблица (столбец [, ...]) [ NULLS [ NOT ] DISTINCT ];
```

В настоящее время обеспечение уникальности поддерживают только индексы-B-деревья.

Если индекс создаётся как уникальный, в таблицу нельзя будет добавить несколько строк с одинаковыми значениями ключа индекса. По умолчанию значения NULL в уникальном столбце считаются не равными друг другу, так что в таком столбце может быть несколько значений NULL. Параметр `NULLS NOT DISTINCT` изменяет это правило, так что значения NULL в индексе считаются равными друг другу. Составной уникальный индекс не принимает только те строки, в которых все индексируемые столбцы содержат одинаковые значения.

Когда для таблицы определяется ограничение уникальности или первичный ключ, Postgres Pro автоматически создаёт уникальный индекс по всем столбцам, составляющим это ограничение или первичный ключ (индекс может быть составным). Такой индекс и является механизмом, который обеспечивает выполнение ограничения.

Примечание

Для уникальных столбцов не нужно вручную создавать отдельные индексы — они просто продублируют индексы, созданные автоматически.

11.7. Индексы по выражениям

Индекс можно создать не только по столбцу нижележащей таблицы, но и по функции или скалярному выражению с одним или несколькими столбцами таблицы. Это позволяет быстро находить данные в таблице по результатам вычислений.

Например, для сравнений без учёта регистра символов часто используется функция `lower`:

```
SELECT * FROM test1 WHERE lower(col1) = 'value';
```

Этот запрос сможет использовать индекс, определённый для результата функции `lower(col1)` так:

```
CREATE INDEX test1_lower_col1_idx ON test1 (lower(col1));
```

Если мы объявим этот индекс уникальным (`UNIQUE`), он не даст добавить строки, в которых значения `col1` различаются только регистром, как и те, в которых значения `col1` действительно одинаковые. Таким образом, индексы по выражениям можно использовать ещё и для обеспечения ограничений, которые нельзя записать как простые ограничения уникальности.

Если же часто выполняются запросы вида:

```
SELECT * FROM people WHERE (first_name || ' ' || last_name) = 'John Smith';
```

тогда, возможно, стоит создать такой индекс:

```
CREATE INDEX people_names ON people ((first_name || ' ' || last_name));
```

Синтаксис команды `CREATE INDEX` обычно требует заключать индексные выражения в скобки, как показано во втором примере. Если же выражение представляет собой просто вызов функции, как в первом примере, дополнительные скобки можно опустить.

Поддержка индексируемых выражений обходится довольно дорого, так как эти выражения должны вычисляться при каждом добавлении строки и при каждом изменении без оптимизации [HOT](#). Однако при поиске по индексу индексируемое выражение *не* вычисляется повторно, так как его результат уже сохранён в индексе. В рассмотренных выше случаях система видит запрос как `WHERE столбец_индекса = 'константа'`, и поэтому поиск выполняется так же быстро, как и с простым индексом. Таким образом, индексы по выражениям могут быть полезны, когда скорость извлечения данных гораздо важнее скорости добавления и изменения.

11.8. Частичные индексы

Частичный индекс — это индекс, который строится по подмножеству строк таблицы, определяемому условным выражением (оно называется *предикатом* частичного индекса). Такой индекс содержит записи только для строк, удовлетворяющих предикату. Частичные индексы довольно специфичны, но в ряде ситуаций они могут быть очень полезны.

Частичные индексы могут быть полезны, во-первых, тем, что позволяют избежать индексирования распространённых значений. Так как при поиске распространённого значения (такого, которое содержится в значительном проценте всех строк) индекс всё равно не будет использоваться, хранить эти строки в индексе нет смысла. Исключив их из индекса, можно уменьшить его размер, а значит и ускорить запросы, использующие этот индекс. Это также может ускорить операции изменения данных в таблице, так как индекс будет обновляться не всегда. Возможное применение этой идеи проиллюстрировано в [Примере 11.1](#).

Пример 11.1. Настройка частичного индекса, исключаящего распространённые значения

Предположим, что вы храните в базе данных журнал обращений к корпоративному сайту. Большая часть обращений будет происходить из диапазона IP-адресов вашей компании, а остальные могут быть откуда угодно (например, к нему могут подключаться внешние сотрудники с динамическими IP). Если при поиске по IP вас обычно интересуют внешние подключения, IP-диапазон внутренней сети компании можно не включать в индекс.

Пусть у вас есть такая таблица:

```
CREATE TABLE access_log (  
    url varchar,  
    client_ip inet,  
    ...  
);
```

Создать частичный индекс для нашего примера можно так:

```
CREATE INDEX access_log_client_ip_ix ON access_log (client_ip)  
WHERE NOT (client_ip > inet '192.168.100.0' AND  
          client_ip < inet '192.168.100.255');
```

Так будет выглядеть типичный запрос, использующий этот индекс:

```
SELECT *  
FROM access_log  
WHERE url = '/index.html' AND client_ip = inet '212.78.10.32';
```

В нём фигурирует IP-адрес, попадающий в частичный индекс. Следующий запрос не может использовать частичный индекс, так как в нём IP-адрес не попадает в диапазон индекса:

```
SELECT *  
FROM access_log  
WHERE url = '/index.html' AND client_ip = inet '192.168.100.23';
```

Заметьте, что при таком определении частичного индекса необходимо, чтобы распространённые значения были известны заранее, так что такие индексы лучше использовать, когда распределение данных не меняется. Хотя такие индексы можно пересоздавать время от времени, подстраиваясь под новое распределение, это значительно усложняет поддержку.

Во-вторых, частичные индексы могут быть полезны тем, что позволяют исключить из индекса значения, которые обычно не представляют интереса; это проиллюстрировано в [Примере 11.2](#). При этом вы получаете те же преимущества, что и в предыдущем случае, но система не сможет извлечь «неинтересные» значения по этому индексу, даже если сканирование индекса может быть эффективным. Очевидно, настройка частичных индексов в таких случаях требует тщательного анализа и тестирования.

Пример 11.2. Настройка частичного индекса, исключающего неинтересные значения

Если у вас есть таблица, в которой хранятся и оплаченные, и неоплаченные счета, и при этом неоплаченные счета составляют только небольшую часть всей таблицы, но представляют наибольший интерес, производительность запросов можно увеличить, создав индекс только по неоплаченным счетам. Сделать это можно следующей командой:

```
CREATE INDEX orders_unbilled_index ON orders (order_nr)
WHERE billed is not true;
```

Этот индекс будет применяться, например в таком запросе:

```
SELECT * FROM orders WHERE billed is not true AND order_nr < 10000;
```

Однако он также может применяться в запросах, где `order_nr` вообще не используется, например:

```
SELECT * FROM orders WHERE billed is not true AND amount > 5000.00;
```

Конечно, он будет не так эффективен, как мог бы быть частичный индекс по столбцу `amount`, так как системе придётся сканировать его целиком. Тем не менее, если неоплаченных счетов сравнительно мало, выиграть при поиске неоплаченного счёта можно и с таким частичным индексом.

Заметьте, что в таком запросе этот индекс не будет использоваться:

```
SELECT * FROM orders WHERE order_nr = 3501;
```

Счёт с номером 3501 может оказаться, как в числе неоплаченных, так и оплаченных.

Пример 11.2 также показывает, что индексируемый столбец не обязательно должен совпадать со столбцом, используемым в предикате. Postgres Pro поддерживает частичные индексы с произвольными предикатами — главное, чтобы в них фигурировали только столбцы индексируемой таблицы. Однако не забывайте, что предикат должен соответствовать условиям запросов, для оптимизации которых предназначается данный индекс. Точнее, частичный индекс будет применяться в запросе, только если система сможет понять, что условие `WHERE` данного запроса математически сводится к предикату индекса. Но учтите, что Postgres Pro не умеет доказывать математические утверждения об эквивалентности выражений, записанных в разных формах. (Составить программу для таких доказательств крайне сложно, и если даже это удастся, скорость её будет неприемлема для применения на практике.) Система может выявить только самые простые следствия с неравенствами; например, понять, что из « $x < 1$ » следует « $x < 2$ »; во всех остальных случаях условие предиката должно точно совпадать с условием в предложении `WHERE`, иначе индекс будет считаться неподходящим. Сопоставление условий происходит во время планирования запросов, а не во время выполнения. Как следствие, запросы с параметрами не будут работать с частичными индексами. Например, условие с параметром « $x < ?$ » в подготовленном запросе никогда не будет сведено к « $x < 2$ » при всех возможных значениях параметра.

Третье возможное применение частичных индексов вообще не связано с использованием индекса в запросах. Идея заключается в том, чтобы создать уникальный индекс по подмножеству строк таблицы, как в **Примере 11.3**. Это обеспечит уникальность среди строк, удовлетворяющих условию предиката, но никак не будет ограничивать остальные.

Пример 11.3. Настройка частичного уникального индекса

Предположим, что у нас есть таблица с результатами теста. Мы хотим, чтобы для каждого сочетания предмета и целевой темы была только одна запись об успешном результате, а неудачных попыток могло быть много. Вот как можно этого добиться:

```
CREATE TABLE tests (
    subject text,
    target text,
    success boolean,
    ...
);

CREATE UNIQUE INDEX tests_success_constraint ON tests (subject, target)
```

```
WHERE success;
```

Это подход будет особенно эффективным, когда неудачных попыток будет намного больше, чем удачных. Также можно потребовать, чтобы в столбце допускался только один NULL, создав уникальный частичный индекс с ограничением `IS NULL`.

Наконец, с помощью частичных индексов можно также переопределять выбираемый системой план запроса. Возможно, что для данных с неудачным распределением система решит использовать индекс, тогда как на самом деле это неэффективно. В этом случае индекс можно настроить так, чтобы в подобных запросах он не работал. Обычно Postgres Pro принимает разумные решения относительно применения индексов (т. е. старается не использовать их для получения распространённых значений, так что частичный индекс в вышеприведённом примере помог только уменьшить размер индекса, для отказа от использования индекса он не требовался), поэтому крайне неэффективный план может быть поводом для сообщения об ошибке.

Помните, что настраивая частичный индекс, вы тем самым заявляете, что знаете о данных гораздо больше, чем планировщик запросов. В частности, вы знаете, когда такой индекс может быть полезен. Это знание обязательно должно подкрепляться опытом и пониманием того, как работают индексы в Postgres Pro. В большинстве случаев преимущества частичных индексов по сравнению с обычными будут минимальными. Однако в ряде случаев эти индексы могут быть даже вредны, о чём говорится в [Примере 11.4](#).

Пример 11.4. Не применяйте частичные индексы в качестве замены секционирования

У вас может возникнуть желание создать множество неперекрывающихся частичных индексов, например:

```
CREATE INDEX mytable_cat_1 ON mytable (data) WHERE category = 1;
CREATE INDEX mytable_cat_2 ON mytable (data) WHERE category = 2;
CREATE INDEX mytable_cat_3 ON mytable (data) WHERE category = 3;
...
CREATE INDEX mytable_cat_N ON mytable (data) WHERE category = N;
```

Но так делать не следует! Почти наверняка вам лучше использовать один не частичный индекс, объявленный так:

```
CREATE INDEX mytable_cat_data ON mytable (category, data);
```

(Поставьте первым столбец категорий, по причинам описанным в [Разделе 11.3](#).) При поиске в большем индексе может потребоваться опуститься на несколько уровней ниже, чем при поиске в меньшем частичном, но это почти гарантированно будет дешевле, чем выбрать при планировании из всех частичных индексов подходящий. Сложность с выбором индекса объясняется тем, что система не знает, как взаимосвязаны частичные индексы, и ей придётся проверять каждый из них, чтобы понять, соответствует ли он текущему запросу.

Если ваша таблица настолько велика, что создавать один индекс кажется действительно плохой идеей, рассмотрите возможность использования секционирования (см. [Раздел 5.11](#)). Когда применяется этот механизм, система понимает, что таблицы и индексы не перекрываются, и может выполнять запросы гораздо эффективнее.

Узнать о частичных индексах больше можно в следующих источниках: [ston89b](#), [olson93](#) и [seshadri95](#).

11.9. Сканирование только индекса и покрывающие индексы

Все индексы в Postgres Pro являются *вторичными*, что значит, что каждый индекс хранится вне области основных данных таблицы (которая в терминологии Postgres Pro называется *кучей* таблицы). Это значит, что при обычном сканировании индекса для извлечения каждой строки необходимо прочитать данные и из индекса, и из кучи. Более того, тогда как элементы индекса, соответствующие заданному условию `WHERE`, обычно находятся в индексе рядом, строки таблицы могут

располагаться в куче произвольным образом. Таким образом, обращение к куче при поиске по индексу влечёт множество операций произвольного чтения кучи, которые могут обойтись недёшево, особенно на традиционных вращающихся носителях. (Как описано в [Разделе 11.5](#), сканирование по битовой карте пытается снизить стоимость этих операций, упорядочивая доступ к куче, но не более того.)

Чтобы решить эту проблему с производительностью, Postgres Pro поддерживает *сканирование только индекса*, при котором результат запроса может быть получен из самого индекса, без обращения к куче. Основная идея такого сканирования в том, чтобы выдавать значения непосредственно из элемента индекса, и не обращаться к соответствующей записи в куче. Для применения этого метода есть два фундаментальных ограничения:

1. Тип индекса должен поддерживать сканирование только индекса. Индексы-B-деревья поддерживают его всегда. Индексы GiST и SP-GiST могут поддерживать его с одними классами операторов и не поддерживать с другими. Другие индексы такое сканирование не поддерживают. Суть нижележащего требования в том, что индекс должен физически хранить или каким-то образом восстанавливать исходное значение данных для каждого элемента индекса. В качестве контрпримера, индексы GIN неспособны поддерживать сканирование только индекса, так как в элементах индекса обычно хранится только часть исходного значения данных.
2. Запрос должен обращаться только к столбцам, сохранённым в индексе. Например, если в таблице построен индекс по столбцам *x* и *y*, и в ней есть также столбец *z*, такие запросы будут использовать сканирование только индекса:

```
SELECT x, y FROM tab WHERE x = 'key';
SELECT x FROM tab WHERE x = 'key' AND y < 42;
```

А эти запросы не будут:

```
SELECT x, z FROM tab WHERE x = 'key';
SELECT x FROM tab WHERE x = 'key' AND z < 42;
```

(Индексы по выражениям и частичные индексы усложняют это правило, как описано ниже.)

Если два этих фундаментальных ограничения выполняются, то все данные, требуемые для выполнения запроса, содержатся в индексе, так что сканирование только по индексу физически возможно. Но в Postgres Pro существует и ещё одно требование для сканирования таблицы: необходимо убедиться, что все возвращаемые строки «видны» в снимке MVCC запроса, как описано в [Главе 13](#). Информация о видимости хранится не в элементах индекса, а только в куче; поэтому на первый взгляд может показаться, что для получения данных каждой строки всё равно необходимо обращаться к куче. И это в самом деле так, если в таблице недавно произошли изменения. Однако для редко меняющихся данных есть возможность обойти эту проблему. Postgres Pro отслеживает для каждой страницы в куче таблицы, являются ли все строки в этой странице достаточно старыми, чтобы их видели все текущие и будущие транзакции. Это отражается в битах в *карте видимости* таблицы. Процедура сканирования только индекса, найдя потенциально подходящую запись в индексе, проверяет бит в карте видимости для соответствующей страницы в куче. Если он установлен, значит эта строка видна, и данные могут быть возвращены сразу. В противном случае придётся посетить запись строки в куче и проверить, видима ли она, так что никакого выигрыша по сравнению с обычным сканированием индекса не будет. Однако в благоприятном случае обращение к куче заменяется обращением к карте видимости; но так как карта видимости на четыре порядка меньше соответствующей ей области кучи, для работы с ней требуется много меньше операций физического ввода-вывода. В большинстве ситуаций карта видимости просто всё время находится в памяти.

Таким образом, тогда как сканирование только по индексу возможно лишь при выполнении двух фундаментальных требований, оно даст выигрыш, только если для значительной части страниц в куче таблицы установлены биты полной видимости. Но таблицы, в которых меняется лишь небольшая часть строк, встречаются достаточно часто, чтобы этот тип сканирования был весьма полезен на практике.

Чтобы эффективно использовать возможность сканирования только индекса, вы можете создавать *покрывающие индексы*. Такие индексы специально предназначены для включения столбцов, ко-

которые требуются в определённых часто выполняемых запросах. Так как в запросах обычно нужно получить не только столбцы, по которым выполняется поиск, Postgres Pro позволяет создать индекс, в котором некоторые столбцы будут просто «дополнительной нагрузкой», но не войдут в поисковый ключ. Это реализуется предложением `INCLUDE`, в котором перечисляются дополнительные столбцы. Например, если часто выполняется запрос вида

```
SELECT y FROM tab WHERE x = 'key';
```

при традиционном подходе его можно ускорить, создав индекс только по x . Однако такой индекс:

```
CREATE INDEX tab_x_y ON tab(x) INCLUDE (y);
```

может удовлетворить такие запросы при сканировании только индекса, так как значение y можно получить из индекса, не обращаясь к данным в куче.

Так как столбец y не является частью поискового ключа, он не обязательно должен иметь тип данных, воспринимаемый данным индексом; он просто сохраняется внутри индекса и никак не обрабатывается механизмом индекса. Кроме того, в случае с уникальным индексом, например:

```
CREATE UNIQUE INDEX tab_x_y ON tab(x) INCLUDE (y);
```

условие уникальности распространяется только на столбец x , а не на x и y в совокупности. (Предложение `INCLUDE` можно также добавить в ограничения `UNIQUE` и `PRIMARY KEY`, что позволяет определить такой индекс альтернативным образом.)

Добавлять в индекс неключевые дополнительные столбцы следует обдуманно, особенно когда это большие столбцы. Если размер кортежа в индексе превысит максимально допустимый размер для типа индексов, при добавлении данных возникнет ошибка. В любом случае в неключевых столбцах дублируются данные из самой таблицы, что приводит к раздуванию индекса, а следствием этого может быть замедление запросов. И помните, что практический смысл включать дополнительные столбцы в индекс есть только тогда, когда таблица меняется достаточно медленно, и при сканировании только индекса не приходится обращаться к куче. Если кортеж в любом случае придётся прочитывать из кучи, получить значение столбца из него ничего не стоит. Покрывающие индексы имеют и другие ограничения: в настоящее время в качестве неключевых столбцов нельзя задать выражения, и поддерживаются такие индексы только трёх типов: В-деревья, GiST и SP-GIST.

До появления в Postgres Pro покрывающих индексов (`INCLUDE`) пользователям иногда приходилось задействовать дополнительные столбцы как обычные столбцы индекса, то есть писать

```
CREATE INDEX tab_x_y ON tab(x, y);
```

даже не намереваясь когда-либо использовать y в предложении `WHERE`. Это работает, когда дополнительные столбцы добавляются в конец; делать их начальными неразумно по причинам, описанным в [Разделе 11.3](#). Однако этот подход не годится для случая, когда вам нужно обеспечить уникальность ключевого столбца (столбцов).

В процессе *усечения суффикса* с верхних уровней В-дерева всегда удаляются неключевые столбцы. Так как они содержат дополнительную нагрузку, они никогда не управляют поиском по индексу. В этом процессе также удаляется один или несколько замыкающих столбцов, если остающегося префикса ключевых столбцов оказывается достаточно, чтобы описать кортежи на нижележащем уровне В-дерева. На практике и в покрывающих индексах без предложения `INCLUDE` часто удаётся избежать хранения столбцов, которые вверху по сути являются допнагрузкой. Однако явное обозначение дополнительных столбцов неключевыми *гарантирует* минимальный размер кортежей на верхних уровнях.

В принципе сканирование только индекса может применяться и с индексами по выражениям. Например, при наличии индекса по $f(x)$, где x — столбец таблицы, должно быть возможно выполнить

```
SELECT f(x) FROM tab WHERE f(x) < 1;
```

как сканирование только индекса; и это очень заманчиво, если $f()$ — сложная для вычисления функция. Однако планировщик Postgres Pro в настоящее время может вести себя не очень разумно. Он считает, что запрос может выполняться со сканированием только индекса, лишь когда из индекса могут быть получены все *столбцы*, требующиеся для запроса. В этом примере x фигурирует

только в контексте $f(x)$, но планировщик не замечает этого и решает, что сканирование только по индексу невозможно. Если сканирование только индекса заслуживает того, эту проблему можно обойти, добавив x как неключевой столбец, например:

```
CREATE INDEX tab_f_x ON tab (f(x)) INCLUDE (x);
```

Если это делается ради предотвращения многократных вычислений $f(x)$, следует также учесть, что планировщик не обязательно свяжет упоминания $f(x)$, фигурирующие вне индексируемых предложений `WHERE`, со столбцом индекса. Обычно он делает это правильно в простых запросах, вроде показанного выше, но не в запросах с соединениями. Эти недостатки могут быть устранены в будущих версиях Postgres Pro.

С использованием частичных индексов при сканировании только по индексу тоже связаны интересные особенности. Предположим, что у нас есть частичный индекс, показанный в [Примере 11.3](#):

```
CREATE UNIQUE INDEX tests_success_constraint ON tests (subject, target)
WHERE success;
```

В принципе с ним мы можем произвести сканирование только по индексу при выполнении запроса

```
SELECT target FROM tests WHERE subject = 'some-subject' AND success;
```

Но есть одна проблема: предложение `WHERE` обращается к столбцу `success`, который отсутствует в результирующих столбцах индекса. Тем не менее сканирование только индекса возможно, так как плану не нужно перепроверять эту часть предложения `WHERE` во время выполнения: у всех записей, найденных в индексе, значение `success = true`, так что в плане его не нужно проверять явно. Postgres Pro версий 9.6 и новее распознает такую ситуацию и сможет произвести сканирование только по индексу, но старые версии неспособны на это.

11.10. Семейства и классы операторов

В определении индекса можно указать *класс операторов* для каждого столбца индекса.

```
CREATE INDEX имя ON таблица (столбец класс_оп [ ( параметры_класса_оп ) ] [параметры
сортировки] [, ...]);
```

Класс операторов определяет, какие операторы будет использовать индекс для этого столбца. Например, индекс-В-дерево по столбцу `int4` будет использовать класс `int4_ops`; этот класс операторов включает операции со значениями типа `int4`. На практике часто достаточно принять класс операторов, назначенный для типа столбца классом по умолчанию. Однако для некоторых типов данных могут иметь смысл несколько разных вариантов индексирования и реализовать их как раз позволяют разные классы операторов. Например, комплексные числа можно сортировать как по вещественной части, так и по модулю. Получить два варианта индексов для них можно, определив два класса операторов для данного типа и выбрав соответствующий класс при создании индекса. Выбранный класс операторов задаст основной порядок сортировки данных (его можно уточнить, добавив параметры `COLLATE`, `ASC/DESC` и/или `NULLS FIRST/NULLS LAST`).

Помимо классов операторов по умолчанию есть ещё несколько встроенных:

- Классы операторов `text_pattern_ops`, `varchar_pattern_ops` и `bpchar_pattern_ops` поддерживают индексы-В-дерева для типов `text`, `varchar` и `char`, соответственно. От стандартных классов операторов они отличаются тем, что сравнивают значения по символам, не применяя правила сортировки, определённые локалью. Благодаря этому они подходят для запросов с поиском по шаблону (с `LIKE` и регулярными выражениями POSIX), когда локаль базы данных не стандартная «C». Например, вы можете проиндексировать столбец `varchar` так:

```
CREATE INDEX test_index ON test_table (col varchar_pattern_ops);
```

Заметьте, что при этом также следует создать индекс с классом операторов по умолчанию, если вы хотите ускорить запросы с обычными сравнениями `<`, `<=`, `>` и `>=` за счёт применения индексов. Классы операторов `xxx_pattern_ops` не подходят для таких сравнений. (Однако для проверки равенств эти классы операторов вполне пригодны.) В подобных случаях для одного столбца можно создать несколько индексов с разными классами операторов. Если же вы ис-

пользуете локаль C, классы операторов `xxx_pattern_ops` вам не нужны, так как для поиска по шаблону в локали C будет достаточно индексов с классом операторов по умолчанию.

Следующий запрос выводит список всех существующих классов операторов:

```
SELECT am.amname AS index_method,
       opc.opcname AS opclass_name,
       opc.opcintype::regtype AS indexed_type,
       opc.opcdefault AS is_default
FROM pg_am am, pg_opclass opc
WHERE opc.opcmethod = am.oid
ORDER BY index_method, opclass_name;
```

Класс операторов на самом деле является всего лишь подмножеством большой структуры, называемой *семейством операторов*. В случаях, когда несколько типов данных ведут себя одинаково, часто имеет смысл определить операторы так, чтобы они могли использоваться с индексами сразу нескольких типов. Сделать это можно, сгруппировав классы операторов для этих типов в одном семействе операторов. Такие многоцелевые операторы, являясь членами семейства, не будут связаны с каким-либо одним его классом.

Расширенная версия предыдущего запроса показывает семейство операторов, к которому принадлежит каждый класс операторов:

```
SELECT am.amname AS index_method,
       opc.opcname AS opclass_name,
       opf.opfname AS opfamily_name,
       opc.opcintype::regtype AS indexed_type,
       opc.opcdefault AS is_default
FROM pg_am am, pg_opclass opc, pg_opfamily opf
WHERE opc.opcmethod = am.oid AND
       opc.opcfamily = opf.oid
ORDER BY index_method, opclass_name;
```

Этот запрос выводит все существующие семейства операторов и все операторы, включённые в эти семейства:

```
SELECT am.amname AS index_method,
       opf.opfname AS opfamily_name,
       amop.amopopr::regoperator AS opfamily_operator
FROM pg_am am, pg_opfamily opf, pg_amop amop
WHERE opf.opfmethod = am.oid AND
       amop.amopfamily = opf.oid
ORDER BY index_method, opfamily_name, opfamily_operator;
```

Подсказка

В **psql** есть команды `\dAc`, `\dAf` и `\dAo`, которые выводят более усовершенствованные версии этих запросов.

11.11. Индексы и правила сортировки

Один индекс может поддерживать только одно правило сортировки для индексируемого столбца. Поэтому при необходимости применять разные правила сортировки могут потребоваться несколько индексов.

Рассмотрим следующие операторы:

```
CREATE TABLE test1c (
  id integer,
```

```
content varchar COLLATE "x"
);
```

```
CREATE INDEX test1c_content_index ON test1c (content);
```

Этот индекс автоматически использует правило сортировки нижележащего столбца. И запрос вида

```
SELECT * FROM test1c WHERE content > константа;
```

сможет использовать этот индекс, так как при сравнении по умолчанию будет действовать правило сортировки столбца. Однако этот индекс не поможет ускорить запросы с каким-либо другим правилом сортировки. Поэтому, если интерес представляют также и запросы вроде

```
SELECT * FROM test1c WHERE content > константа COLLATE "y";
```

для них можно создать дополнительный индекс, поддерживающий правило сортировки "y", примерно так:

```
CREATE INDEX test1c_content_y_index ON test1c (content COLLATE "y");
```

11.12. Контроль использования индексов

Хотя индексы в Postgres Pro не требуют какого-либо обслуживания или настройки, это не избавляет от необходимости проверять, как и какие индексы используются на самом деле в реальных условиях. Узнать, как отдельный запрос использует индексы, можно с помощью команды [EXPLAIN](#); её применение для этих целей описывается в [Разделе 14.1](#). Также возможно собрать общую статистику об использовании индексов на работающем сервере, как описано в [Разделе 26.2](#).

Вывести универсальную формулу, определяющую, какие индексы нужно создавать, довольно сложно, если вообще возможно. В предыдущих разделах рассматривались некоторые типовые ситуации, иллюстрирующие подходы к этому вопросу. Часто найти ответ на него помогают эксперименты. Ниже приведены ещё несколько советов:

- Всегда начинайте исследование с [ANALYZE](#). Эта команда собирает статистические данные о распределении значений в таблице, которые необходимы для оценивания числа строк, возвращаемых запросов. А это число, в свою очередь, нужно планировщику, чтобы оценить реальные затраты для всевозможных планов выполнения запроса. Не имея реальной статистики, планировщик будет вынужден принять некоторые значения по умолчанию, которые почти наверняка не будут соответствовать действительности. Поэтому понять, как индекс используется приложением без предварительного запуска [ANALYZE](#), практически невозможно. Подробнее это рассматривается в [Подразделе 23.1.3](#) и [Подразделе 23.1.6](#).
- Используйте в экспериментах реальные данные. Анализируя работу системы с тестовыми данными, вы поймёте, какие индексы нужны для тестовых данных, но не более того.

Особенно сильно искажают картину очень маленькие наборы тестовых данных. Тогда как для извлечения 1000 строк из 100000 может быть применён индекс, для выбора 1 из 100 он вряд ли потребуется, так как 100 строк скорее всего уместятся в одну страницу данных на диске и никакой другой план не будет лучше обычного сканирования 1 страницы.

Тем не менее, пока приложение не эксплуатируется, создавать какие-то тестовые данные всё равно нужно, и это нужно делать обдуманно. Если вы наполняете базу данных очень близкими, или наоборот, случайными значениями, либо добавляете строки в отсортированном порядке, вы получите совсем не ту статистику распределения, что дадут реальные данные.

- Когда индексы не используются, ради тестирования может быть полезно подключить их принудительно. Для этого можно воспользоваться параметрами выполнения, позволяющими включать различные типы планов (см. [Подраздел 18.7.1](#)). Например, выключив наиболее простые планы: последовательное сканирование (`enable_seqscan`) и соединения с вложенными циклами (`enable_nestloop`), вы сможете заставить систему выбрать другой план. Если же система продолжает выполнять сканирование или соединение с вложенными циклами, вероятно, у неё есть более серьёзная причина не использовать индекс; например, индекс может не

соответствовать условию запроса. (Какие индексы работают в запросах разных типов, обсуждалось в предыдущих разделах.)

- Если система начинает использовать индекс только под принуждением, тому может быть две причины: либо система права и применять индекс в самом деле неэффективно, либо оценка стоимости применения индекса не соответствует действительности. В этом случае вам следует замерить время выполнения запроса с индексами и без них. В анализе этой ситуации может быть полезна команда `EXPLAIN ANALYZE`.
- Если выясняется, что оценка стоимости неверна, это может иметь тоже два объяснения. Общая стоимость вычисляется как произведение цены каждого узла плана для одной строки и оценки избирательности узла плана. Цены узлов при необходимости можно изменить параметрами выполнения (описанными в [Подразделе 18.7.2](#)). С другой стороны, оценка избирательности может быть неточной из-за некачественной статистики. Улучшить её можно, настроив параметры сбора статистики (см. [ALTER TABLE](#)).

Если ваши попытки скорректировать стоимость планов не увенчаются успехом, возможно вам останется только явно заставить систему использовать нужный индекс. Вероятно, имеет смысл также связаться с разработчиками Postgres Pro, чтобы прояснить ситуацию.

Глава 12. Полнотекстовый поиск

12.1. Введение

Полнотекстовый поиск (или просто *поиск текста*) — это возможность находить *документы* на естественном языке, соответствующие *запросу*, и, возможно, дополнительно сортировать их по релевантности для этого запроса. Наиболее распространённая задача — найти все документы, содержащие *слова запроса*, и выдать их отсортированными по степени *соответствия* запросу. Понятия запроса и соответствия довольно расплывчаты и зависят от конкретного приложения. В самом простом случае запросом считается набор слов, а соответствие определяется частотой слов в документе.

Операторы текстового поиска существуют в СУБД уже многие годы. В Postgres Pro для текстовых типов данных есть операторы `~`, `~*`, `LIKE` и `ILIKE`, но им не хватает очень важных вещей, которые требуются сегодня от информационных систем:

- Нет поддержки лингвистической функциональности, даже для английского языка. Возможности регулярных выражений ограничены — они не рассчитаны на работу со словоформами, например, подходят и подходят. С ними вы можете пропустить документы, которые содержат подходят, но, вероятно, и они представляют интерес при поиске по ключевому слову подходит. Конечно, можно попытаться перечислить в регулярном выражении все варианты слова, но это будет очень трудоёмко и чревато ошибками (некоторые слова могут иметь десятки словоформ).
- Они не позволяют упорядочивать результаты поиска (по релевантности), а без этого поиск неэффективен, когда находятся сотни подходящих документов.
- Они обычно выполняются медленно из-за отсутствия индексов, так как при каждом поиске приходится просматривать все документы.

Полнотекстовая индексация заключается в *предварительной обработке* документов и сохранении индекса для последующего быстрого поиска. Предварительная обработка включает следующие операции:

Разбор документов на фрагменты. При этом полезно выделить различные классы фрагментов, например, числа, слова, словосочетания, почтовые адреса и т. д., которые будут обрабатываться по-разному. В принципе классы фрагментов могут зависеть от приложения, но для большинства применений вполне подойдёт предопределённый набор классов. Эту операцию в Postgres Pro выполняет *анализатор* (parser). Вы можете использовать как стандартный анализатор, так и создавать свои, узкоспециализированные.

Преобразование фрагментов в лексемы. Лексема — это *нормализованный* фрагмент, в котором разные словоформы приведены к одной. Например, при нормализации буквы верхнего регистра приводятся к нижнему, а из слов обычно убираются окончания (в частности, *s* или *es* в английском). Благодаря этому можно находить разные формы одного слова, не вводя вручную все возможные варианты. Кроме того, на данном шаге обычно исключаются *стоп-слова*, то есть слова, настолько распространённые, что искать их нет смысла. (Другими словами, фрагменты представляют собой просто подстроки текста документа, а лексемы — это слова, имеющие ценность для индексации и поиска.) Для выполнения этого шага в Postgres Pro используются *словари*. Набор существующих стандартных словарей при необходимости можно расширять, создавая свои собственные.

Хранение документов в форме, подготовленной для поиска. Например, каждый документ может быть представлен в виде отсортированного массива нормализованных лексем. Помимо лексем часто желательно хранить информацию об их положении для *ранжирования по близости*, чтобы документ, в котором слова запроса расположены «плотнее», получал более высокий ранг, чем документ с разбросанными словами.

Словари позволяют управлять нормализацией фрагментов с большой гибкостью. Создавая словари, можно:

- Определять стоп-слова, которые не будут индексироваться.

- Сопоставлять синонимы с одним словом, используя Ispell.
- Сопоставлять словосочетания с одним словом, используя тезаурус.
- Сопоставлять различные склонения слова с канонической формой, используя словарь Ispell.
- Сопоставлять различные склонения слова с канонической формой, используя стеммер Snowball.

Для хранения подготовленных документов в PostgreSQL предназначен тип данных `tsvector`, а для представления обработанных запросов — тип `tsquery` (Раздел 8.11). С этими типами данных работают целый ряд функций и операторов (Раздел 9.13), и наиболее важный из них — оператор соответствия `@@`, с которым мы познакомимся в Подразделе 12.1.2. Для ускорения полнотекстового поиска могут применяться индексы (Раздел 12.9).

12.1.1. Что такое документ?

Документ — это единица обработки в системе полнотекстового поиска; например, журнальная статья или почтовое сообщение. Система поиска текста должна уметь разбирать документы и сохранять связи лексем (ключевых слов) с содержащим их документом. Впоследствии эти связи могут использоваться для поиска документов с заданными ключевыми словами.

В контексте поиска в Postgres Pro документ — это обычно содержимое текстового поля в строке таблицы или, возможно, сочетание (объединение) таких полей, которые могут храниться в разных таблицах или формироваться динамически. Другими словами, документ для индексации может создаваться из нескольких частей и не храниться где-либо как единое целое. Например:

```
SELECT title || ' ' || author || ' ' || abstract || ' ' || body AS document
FROM messages
WHERE mid = 12;
```

```
SELECT m.title || ' ' || m.author || ' ' || m.abstract || ' ' || d.body AS document
FROM messages m, docs d
WHERE m.mid = d.did AND m.mid = 12;
```

Примечание

На самом деле в этих примерах запросов следует использовать функцию `coalesce`, чтобы значение `NULL` в каком-либо одном атрибуте не привело к тому, что результирующим документом окажется `NULL`.

Документы также можно хранить в обычных текстовых файлах в файловой системе. В этом случае база данных может быть просто хранилищем полнотекстового индекса и исполнителем запросов, а найденные документы будут загружаться из файловой системы по некоторым уникальным идентификаторам. Однако для загрузки внешних файлов требуются права суперпользователя или поддержка специальных функций, так что это обычно менее удобно, чем хранить все данные внутри БД Postgres Pro. Кроме того, когда всё хранится в базе данных, это упрощает доступ к метаданным документов при индексации и выводе результатов.

Для нужд текстового поиска каждый документ должен быть сведён к специальному формату `tsvector`. Поиск и ранжирование выполняется исключительно с этим представлением документа — исходный текст потребуется извлечь, только когда документ будет отобран для вывода пользователю. Поэтому мы часто подразумеваем под `tsvector` документ, тогда как этот тип, конечно, содержит только компактное представление всего документа.

12.1.2. Простое соответствие текста

Полнотекстовый поиск в Postgres Pro реализован на базе оператора соответствия `@@`, который возвращает `true`, если `tsvector` (документ) соответствует `tsquery` (запросу). Для этого оператора не важно, какой тип записан первым:

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::tsvector @@ 'cat & rat'::tsquery;
```



```
?column?
```

```
-----
t
```

```
SELECT 'fat & cow'::tsquery @@ 'a fat cat sat on a mat and ate a fat rat'::tsvector;
?column?
```

```
-----
f
```

Как можно догадаться из этого примера, `tsquery` — это не просто текст, как и `tsvector`. Значение типа `tsquery` содержит искомые слова, это должны быть уже нормализованные лексемы, возможно объединённые в выражение операторами И, ИЛИ, НЕ и ПРЕДШЕСТВУЕТ. (Подробнее синтаксис описан в [Подразделе 8.11.2.](#)) Вы можете воспользоваться функциями `to_tsquery`, `plainto_tsquery` и `phraseto_tsquery`, которые могут преобразовать заданный пользователем текст в значение `tsquery`, прежде всего нормализуя слова в этом тексте. Функция `to_tsvector` подобным образом может разобрать и нормализовать текстовое содержимое документа. Так что запрос с поиском соответствия на практике выглядит скорее так:

```
SELECT to_tsvector('fat cats ate fat rats') @@ to_tsquery('fat & rat');
?column?
```

```
-----
t
```

Заметьте, что соответствие не будет обнаружено, если запрос записан как

```
SELECT 'fat cats ate fat rats'::tsvector @@ to_tsquery('fat & rat');
?column?
```

```
-----
f
```

так как слово `rats` не будет нормализовано. Элементами `tsvector` являются лексемы, предположительно уже нормализованные, так что `rats` считается не соответствующим `rat`.

Оператор `@@` также может принимать типы `text`, позволяя опустить явные преобразования текстовых строк в типы `tsvector` и `tsquery` в простых случаях. Всего есть четыре варианта этого оператора:

```
tsvector @@ tsquery
tsquery  @@ tsvector
text     @@ tsquery
text     @@ text
```

Первые два мы уже видели раньше. Форма `text@@tsquery` равнозначна выражению `to_tsvector(x) @@ y`, а форма `text@@text` — выражению `to_tsvector(x) @@ plainto_tsquery(y)`.

В значении `tsquery` оператор `&` (И) указывает, что оба его операнда должны присутствовать в документе, чтобы он удовлетворял запросу. Подобным образом, оператор `|` (ИЛИ) указывает, что в документе должен присутствовать минимум один из его операндов, тогда как оператор `!` (НЕ) указывает, что его операнд *не* должен присутствовать, чтобы условие удовлетворялось. Например, запросу `fat & ! rat` соответствуют документы, содержащие `fat` и не содержащие `rat`.

Фразовый поиск возможен с использованием оператора `<->` (ПРЕДШЕСТВУЕТ) типа `tsquery`, который находит соответствие, только если его операнды расположены рядом и в заданном порядке. Например:

```
SELECT to_tsvector('fatal error') @@ to_tsquery('fatal <-> error');
?column?
```

```
-----
t
```

```
SELECT to_tsvector('error is not fatal') @@ to_tsquery('fatal <-> error');
```

```
?column?
```

```
f
```

Более общая версия оператора ПРЕДШЕСТВУЕТ имеет вид $\langle N \rangle$, где N — целое число, выражающее разность между позициями найденных лексем. Запись $\langle 1 \rangle$ равнозначна $\langle - \rangle$, тогда как $\langle 2 \rangle$ допускает существование ровно одной лексемы между этими лексемами и т. д. Функция `phraseto_tsquery` задействует этот оператор для конструирования `tsquery`, который может содержать многословную фразу, включающую в себя стоп-слова. Например:

```
SELECT phraseto_tsquery('cats ate rats');
       phraseto_tsquery
-----
'cat' <-> 'ate' <-> 'rat'
```

```
SELECT phraseto_tsquery('the cats ate the rats');
       phraseto_tsquery
-----
'cat' <-> 'ate' <2> 'rat'
```

Особый случай, который иногда бывает полезен, представляет собой запись $\langle 0 \rangle$, требующая, чтобы обоим лексемам соответствовало одно слово.

Сочетанием операторов `tsquery` можно управлять, применяя скобки. Без скобок операторы имеют следующие приоритеты, в порядке возрастания: `|`, `&`, $\langle - \rangle$ и самый приоритетный — `!`.

Стоит отметить, что операторы И/ИЛИ/НЕ имеют несколько другое значение, когда они применяются в аргументах оператора ПРЕДШЕСТВУЕТ, так как в этом случае имеет значение точная позиция совпадения. Например, обычному `!x` соответствуют только документы, не содержащие `x` нигде. Но условию `!x <-> y` соответствует `y`, если оно не следует непосредственно за `x`; при вхождении `x` в любом другом месте документа он не будет исключаться из рассмотрения. Другой пример: для условия `x & y` обычно требуется, чтобы и `x`, и `y` встречались в каком-то месте документа, но для выполнения условия `(x & y) <-> z` требуется, чтобы `x` и `y` располагались в одном месте, непосредственно перед `z`. Таким образом, этот запрос отличается от `x <-> z & y <-> z`, которому удовлетворяют документы, содержащие две отдельные последовательности `x z` и `y z`. (Этот конкретный запрос в таком виде, как он записан, не имеет смысла, так как `x` и `y` не могут находиться в одном месте; но в более сложных ситуациях, например, с шаблонами поиска по маске, запросы этого вида могут быть полезны.)

12.1.3. Конфигурации

До этого мы рассматривали очень простые примеры поиска текста. Как было упомянуто выше, весь функционал текстового поиска позволяет делать гораздо больше: пропускать определённые слова (стоп-слова), обрабатывать синонимы и выполнять сложный анализ слов, например, выделять фрагменты не только по пробелам. Все эти функции управляются *конфигурациями текстового поиска*. В Postgres Pro есть набор предопределённых конфигураций для многих языков, но вы также можете создавать собственные конфигурации. (Все доступные конфигурации можно посмотреть с помощью команды `\df` в `psql`.)

Подходящая конфигурация для данной среды выбирается во время установки и записывается в параметре `default_text_search_config` в `postgresql.conf`. Если вы используете для всего кластера одну конфигурацию текстового поиска, вам будет достаточно этого параметра в `postgresql.conf`. Если же требуется использовать в кластере разные конфигурации, но для каждой базы данных одну определённую, её можно задать командой `ALTER DATABASE ... SET`. В противном случае конфигурацию можно выбрать в рамках сеанса, определив параметр `default_text_search_config`.

У каждой функции текстового поиска, зависящей от конфигурации, есть необязательный аргумент `regconfig`, в котором можно явно указать конфигурацию для данной функции. Значение `default_text_search_config` используется, только когда этот аргумент опущен.

Для упрощения создания конфигураций текстового поиска они строятся из более простых объектов. В Postgres Pro есть четыре типа таких объектов:

- *Анализаторы текстового поиска* разделяют документ на фрагменты и классифицируют их (например, как слова или числа).
- *Словари текстового поиска* приводят фрагменты к нормализованной форме и отбрасывают стоп-слова.
- *Шаблоны текстового поиска* предоставляют функции, образующие реализацию словарей. (При создании словаря просто задаётся шаблон и набор параметров для него.)
- *Конфигурации текстового поиска* выбирают анализатор и набор словарей, который будет использоваться для нормализации фрагментов, выданных анализатором.

Анализаторы и шаблоны текстового поиска строятся из низкоуровневых функций на языке C; чтобы создать их, нужно программировать на C, а подключить их к базе данных может только суперпользователь. (В подкаталоге `contrib/` инсталляции Postgres Pro можно найти примеры дополнительных анализаторов и шаблонов.) Так как словари и конфигурации представляют собой просто наборы параметров, связывающие анализаторы и шаблоны, их можно создавать, не имея административных прав. Далее в этой главе будут приведены примеры их создания.

12.2. Таблицы и индексы

В предыдущем разделе приводились примеры, которые показывали, как можно выполнить сопоставление с простыми текстовыми константами. В этом разделе показывается, как находить текст в таблице, возможно с применением индексов.

12.2.1. Поиск в таблице

Полнотекстовый поиск можно выполнить, не применяя индекс. Следующий простой запрос выводит заголовок (`title`) каждой строки, содержащей слово `friend` в поле `body`:

```
SELECT title
FROM pgweb
WHERE to_tsvector('english', body) @@ to_tsquery('english', 'friend');
```

При этом также будут найдены связанные слова, такие как `friends` и `friendly`, так как все они сводятся к одной нормализованной лексеме.

В показанном выше примере для разбора и нормализации строки явно выбирается конфигурация `english`. Хотя параметры, задающие конфигурацию, можно опустить:

```
SELECT title
FROM pgweb
WHERE to_tsvector(body) @@ to_tsquery('friend');
```

Такой запрос будет использовать конфигурацию, заданную в параметре [default_text_search_config](#).

В следующем более сложном примере выбираются десять документов, изменённых последними, со словами `create` и `table` в полях `title` или `body`:

```
SELECT title
FROM pgweb
WHERE to_tsvector(title || ' ' || body) @@ to_tsquery('create & table')
ORDER BY last_mod_date DESC
LIMIT 10;
```

Чтобы найти строки, содержащие `NULL` в одном из полей, нужно воспользоваться функцией `coalesce`, но здесь мы опустили её вызовы для краткости.

Хотя такие запросы будут работать и без индекса, для большинства приложений скорость будет неприемлемой; этот подход рекомендуется только для нерегулярного поиска и динамического содержимого. Для практического применения полнотекстового поиска обычно создаются индексы.

12.2.2. Создание индексов

Для ускорения текстового поиска мы можем создать индекс GIN (см. [Раздел 12.9](#)):

```
CREATE INDEX pgweb_idx ON pgweb USING GIN (to_tsvector('english', body));
```

Заметьте, что здесь используется функция `to_tsvector` с двумя аргументами. В выражениях, определяющих индексы, можно использовать только функции, в которых явно задаётся имя конфигурации текстового поиска (см. [Раздел 11.7](#)). Это объясняется тем, что содержимое индекса не должно зависеть от значения параметра `default_text_search_config`. В противном случае содержимое индекса может быть неактуальным, если разные его элементы `tsvector` будут создаваться с разными конфигурациями текстового поиска и нельзя будет понять, какую именно использовать. Выгрузить и восстановить такой индекс будет невозможно.

Так как при создании индекса использовалась версия `to_tsvector` с двумя аргументами, этот индекс будет использоваться только в запросах, где `to_tsvector` вызывается с двумя аргументами и во втором передаётся имя той же конфигурации. То есть, `WHERE to_tsvector('english', body) @@ 'a & b'` сможет использовать этот индекс, а `WHERE to_tsvector(body) @@ 'a & b'` — нет. Это гарантирует, что индекс будет использоваться только с той конфигурацией, с которой создавались его элементы.

Индекс можно создать более сложным образом, определив для него имя конфигурации в другом столбце таблицы, например:

```
CREATE INDEX pgweb_idx ON pgweb USING GIN (to_tsvector(config_name, body));
```

где `config_name` — имя столбца в таблице `pgweb`. Так можно сохранить имя конфигурации, связанной с элементом индекса, и, таким образом, иметь в одном индексе элементы с разными конфигурациями. Это может быть полезно, например, когда в коллекции документов хранятся документы на разных языках. И в этом случае в запросах должен использоваться тот же индекс (с таким же образом задаваемой конфигурацией), например, так: `WHERE to_tsvector(config_name, body) @@ 'a & b'`.

Индексы могут создаваться даже по объединению столбцов:

```
CREATE INDEX pgweb_idx ON pgweb USING GIN (to_tsvector('english', title || ' ' || body));
```

Ещё один вариант — создать отдельный столбец `tsvector`, в котором сохранить результат `to_tsvector`. Чтобы этот столбец автоматически синхронизировался с исходными данными, он создаётся как хранимый генерируемый столбец. Следующий пример показывает, как можно подготовить для индексации объединённое содержимое столбцов `title` и `body`, применив функцию `coalesce` для получения желаемого результата, даже когда один из столбцов `NULL`:

```
ALTER TABLE pgweb
  ADD COLUMN textsearchable_index_col tsvector
      GENERATED ALWAYS AS (to_tsvector('english', coalesce(title, '') || ' ' || coalesce(body, ''))) STORED;
```

Затем мы создаём индекс GIN для ускорения поиска:

```
CREATE INDEX textsearch_idx ON pgweb USING GIN (textsearchable_index_col);
```

Теперь мы можем быстро выполнять полнотекстовый поиск:

```
SELECT title
FROM pgweb
WHERE textsearchable_index_col @@ to_tsquery('create & table')
ORDER BY last_mod_date DESC
LIMIT 10;
```

Хранение вычисленного выражения индекса в отдельном столбце даёт ряд преимуществ. Во-первых, для использования индекса в запросах не нужно явно указывать имя конфигурации текстового поиска. Как показано в вышеприведённом примере, в этом случае запрос может зависеть от

`default_text_search_config`. Во-вторых, поиск выполняется быстрее, так как для проверки соответствия данных индексу не нужно повторно выполнять `to_tsvector`. (Это актуально больше для индексов GiST, чем для GIN; см. [Раздел 12.9](#).) С другой стороны, схему с индексом по выражению проще реализовать и она позволяет сэкономить место на диске, так как представление `tsvector` не хранится явно.

12.3. Управление текстовым поиском

Для реализации полнотекстового поиска необходимы функции, позволяющие создать `tsvector` из документа и `tsquery` из запроса пользователя. Кроме того, результаты нужно выдавать в удобном порядке, так что нам потребуется функция, оценивающая релевантность документа для данного запроса. Важно также иметь возможность выводить найденный текст подходящим образом. В Postgres Pro есть все необходимые для этого функции.

12.3.1. Разбор документов

Для преобразования документа в тип `tsvector` Postgres Pro предоставляет функцию `to_tsvector`.

```
to_tsvector([конфигурация regconfig,] документ text) returns tsvector
```

`to_tsvector` разбирает текстовый документ на фрагменты, сводит фрагменты к лексемам и возвращает значение `tsvector`, в котором перечисляются лексемы и их позиции в документе. При обработке документа используется указанная конфигурация текстового поиска или конфигурация по умолчанию. Простой пример:

```
SELECT to_tsvector('english', 'a fat cat sat on a mat - it ate a fat rats');
           to_tsvector
-----
'ate':9 'cat':3 'fat':2,11 'mat':7 'rat':12 'sat':4
```

В этом примере мы видим, что результирующий `tsvector` не содержит слова `a`, `on` и `it`, слово `rats` превратилось в `rat`, а знак препинания «-» был проигнорирован.

Функция `to_tsvector` внутри вызывает анализатор, который разбивает текст документа на фрагменты и классифицирует их. Для каждого фрагмента она проверяет список словарей ([Раздел 12.6](#)), определяемый типом фрагмента. Первый же словарь, *распознавший* фрагмент, выдаёт одну или несколько представляющих его лексем. Например, `rats` превращается в `rat`, так как один из словарей понимает, что слово `rats` — это слово `rat` во множественном числе. Некоторое слова распознаются как *stop-слова* ([Подраздел 12.6.1](#)) и игнорируются как слова, фигурирующие в тексте настолько часто, что искать их бессмысленно. В нашем примере это `a`, `on` и `it`. Если фрагмент не воспринимается ни одним словарём из списка, он так же игнорируется. В данном примере это происходит со знаком препинания `-`, так как с таким типом фрагмента (символы-разделители) не связан никакой словарь и значит такие фрагменты никогда не будут индексироваться. Выбор анализатора, словарей и индексируемых типов фрагментов определяется конфигурацией текстового поиска ([Раздел 12.7](#)). В одной базе данных можно использовать разные конфигурации, в том числе, предопределённые конфигурации для разных языков. В нашем примере мы использовали конфигурацию по умолчанию для английского языка — `english`.

Для назначения элементам `tsvector` разных *весов* используется функция `setweight`. Вес элемента задаётся буквой A, B, C или D. Обычно это применяется для обозначения важности слов в разных частях документа, например в заголовке или в теле документа. Затем эта информация может использоваться при ранжировании результатов поиска.

Так как `to_tsvector(NULL)` вернёт `NULL`, мы советуем использовать `coalesce` везде, где соответствующее поле может быть `NULL`. Создавать `tsvector` из структурированного документа рекомендуется так:

```
UPDATE tt SET ti =
    setweight(to_tsvector(coalesce(title, '')), 'A') ||
    setweight(to_tsvector(coalesce(keyword, '')), 'B') ||
```

```
setweight(to_tsvector(coalesce(abstract,'')), 'C') ||
setweight(to_tsvector(coalesce(body,'')), 'D');
```

Здесь мы использовали `setweight` для пометки происхождения каждой лексемы в сформированных значениях `tsvector` и объединили помеченные значения с помощью оператора конкатенации типов `tsvector` `||`. (Подробнее эти операции рассматриваются в [Подразделе 12.4.1.](#))

12.3.2. Разбор запросов

Postgres Pro предоставляет функции `to_tsquery`, `plainto_tsquery`, `phraseto_tsquery` и `websearch_to_tsquery` для приведения запроса к типу `tsquery`. Функция `to_tsquery` даёт больше возможностей, чем `plainto_tsquery` или `phraseto_tsquery`, но более строга к входным данным. Функция `websearch_to_tsquery` представляет собой упрощённую версию `to_tsquery` с альтернативным синтаксисом, подобным тому, что принят в поисковых системах в Интернете.

```
to_tsquery([конфигурация regconfig,] текст_запроса text) returns tsquery
```

`to_tsquery` создаёт значение `tsquery` из текста запроса, который может состоять из простых фрагментов, разделённых логическими операторами `tsquery`: `&` (И), `|` (ИЛИ), `!` (НЕ) и `<->` (ПРЕДШЕСТВУЕТ), возможно, сгруппированных скобками. Другими словами, входное значение для `to_tsquery` должно уже соответствовать общим правилам для значений `tsquery`, описанным в [Подразделе 8.11.2.](#) Различие их состоит в том, что во вводимом в `tsquery` значении фрагменты воспринимались буквально, тогда как `to_tsquery` нормализует фрагменты, приводя их к лексемам, используя явно указанную или подразумеваемую конфигурацию, и отбрасывая стоп-слова. Например:

```
SELECT to_tsquery('english', 'The & Fat & Rats');
       to_tsquery
-----
'fat' & 'rat'
```

Как и при вводе значения `tsquery`, для каждой лексемы можно задать вес(а), чтобы при поиске можно было выбрать из `tsvector` только лексемы с заданными весами. Например:

```
SELECT to_tsquery('english', 'Fat | Rats:AB');
       to_tsquery
-----
'fat' | 'rat':AB
```

К лексеме также можно добавить `*`, определив таким образом условие поиска по префиксу:

```
SELECT to_tsquery('supern:*A & star:A*B');
       to_tsquery
-----
'supern':*A & 'star':*AB
```

Такая лексема будет соответствовать любому слову в `tsvector`, начинающемуся с данной подстроки.

`to_tsquery` может также принимать фразы в апострофах. Это полезно в основном когда конфигурация включает тезаурус, который может обрабатывать такие фразы. В показанном ниже примере предполагается, что тезаурус содержит правило `supernovae stars : sn`:

```
SELECT to_tsquery('supernovae stars' & !crab');
       to_tsquery
-----
'sn' & !'crab'
```

Если убрать эти апострофы, `to_tsquery` не примет фрагменты, не разделённые операторами И, ИЛИ и ПРЕДШЕСТВУЕТ, и выдаст синтаксическую ошибку.

```
plainto_tsquery([конфигурация regconfig,] текст_запроса text) returns tsquery
```

`plainto_tsquery` преобразует неформатированный *текст_запроса* в значение `tsquery`. Текст разбирается и нормализуется подобно тому, как это делает `to_tsvector`, а затем между оставшимися словами вставляются операторы & (И) типа `tsquery`.

Пример:

```
SELECT plainto_tsquery('english', 'The Fat Rats');
plainto_tsquery
-----
'fat' & 'rat'
```

Заметьте, что `plainto_tsquery` не распознает во входной строке операторы `tsquery`, метки весов или обозначения префиксов:

```
SELECT plainto_tsquery('english', 'The Fat & Rats:C');
plainto_tsquery
-----
'fat' & 'rat' & 'c'
```

В данном случае все знаки пунктуации были отброшены.

`phraseto_tsquery([конфигурация regconfig,] текст_запроса text)` returns `tsquery`

`phraseto_tsquery` ведёт себя подобно `plainto_tsquery`, за исключением того, что она вставляет между оставшимися словами оператор `<->` (ПРЕДШЕСТВУЕТ) вместо оператора & (И). Кроме того, стоп-слова не просто отбрасываются, а подсчитываются, и вместо операторов `<->` используются операторы `<N>` с подсчитанным числом. Эта функция полезна при поиске точных последовательностей лексем, так как операторы ПРЕДШЕСТВУЕТ проверяют не только наличие всех лексем, но и их порядок.

Пример:

```
SELECT phraseto_tsquery('english', 'The Fat Rats');
phraseto_tsquery
-----
'fat' <-> 'rat'
```

Как и `plainto_tsquery`, функция `phraseto_tsquery` не распознает во входной строке операторы типа `tsquery`, метки весов или обозначения префиксов:

```
SELECT phraseto_tsquery('english', 'The Fat & Rats:C');
phraseto_tsquery
-----
'fat' <-> 'rat' <-> 'c'
```

`websearch_to_tsquery([конфигурация regconfig,] текст_запроса text)` returns `tsquery`

Функция `websearch_to_tsquery` создаёт значение `tsquery` из *текста_запроса*, используя альтернативный синтаксис, в котором запрос задаётся просто неформатированным текстом. В отличие от `plainto_tsquery` и `phraseto_tsquery`, она также принимает определённые операторы. Более того, эта функция никогда не выдаёт синтаксические ошибки, что позволяет осуществлять поиск по произвольному заданному пользователем запросу. Она поддерживает следующий синтаксис:

- текст не в кавычках: текст, не заключённый в кавычки, который будет преобразован в слова, разделяемые операторами &, как его восприняла бы функция `plainto_tsquery`.
- "текст в кавычках": текст, заключённый в кавычки, будет преобразован в слова, разделяемые операторами `<->`, как его восприняла бы функция `phraseto_tsquery`.
- OR: слово «or» будет преобразовано в оператор `|`.
- -: знак минуса будет преобразован в оператор `!`.

Другая пунктуация игнорируется. Поэтому функция `websearch_to_tsquery`, как и `plainto_tsquery` с `phraseto_tsquery`, не распознаёт во входной строке операторы `tsquery`, метки весов или обозначения префиксов.

Примеры:

```

SELECT websearch_to_tsquery('english', 'The fat rats');
websearch_to_tsquery
-----
'fat' & 'rat'
(1 row)

SELECT websearch_to_tsquery('english', '"supernovae stars" -crab');
websearch_to_tsquery
-----
'supernova' <-> 'star' & !'crab'
(1 row)

SELECT websearch_to_tsquery('english', '"sad cat" or "fat rat"');
websearch_to_tsquery
-----
'sad' <-> 'cat' | 'fat' <-> 'rat'
(1 row)

SELECT websearch_to_tsquery('english', 'signal -"segmentation fault"');
websearch_to_tsquery
-----
'signal' & !( 'segment' <-> 'fault' )
(1 row)

SELECT websearch_to_tsquery('english', '""')( dummy \ query <->');
websearch_to_tsquery
-----
'dummi' & 'queri'
(1 row)

```

12.3.3. Ранжирование результатов поиска

Ранжирование документов можно представить как попытку оценить, насколько они релевантны заданному запросу и отсортировать их так, чтобы наиболее релевантные выводились первыми. В PostgreSQL Pro встроены две функции ранжирования, принимающие во внимание лексическую, позиционную и структурную информацию; то есть, они учитывают, насколько часто и насколько близко встречаются в документе ключевые слова и какова важность содержащей их части документа. Однако само понятие релевантности довольно размытое и во многом определяется приложением. Приложения могут использовать для ранжирования и другую информацию, например, время изменения документа. Встроенные функции ранжирования можно рассматривать лишь как примеры реализации. Для своих конкретных задач вы можете разработать собственные функции ранжирования и/или учесть при обработке их результатов дополнительные факторы.

Ниже описаны две встроенные функции ранжирования:

```
ts_rank([веса float4[],] вектор tsvector, query tsquery [, нормализация integer]) returns float4
```

Ранжирует векторы по частоте найденных лексем.

```
ts_rank_cd([веса float4[],] вектор tsvector, query tsquery [, нормализация integer]) returns float4
```

Эта функция вычисляет *плотность покрытия* для данного вектора документа и запроса, используя метод, разработанный Кларком, Кормаком и Тадхоуп и описанный в статье «Relevance Ranking for One to Three Term Queries» в журнале «Information Processing and Management» в 1999 г. Плотность покрытия вычисляется подобно рангу `ts_rank`, но в расчёт берётся ещё и близость соответствующих лексем друг к другу.

Для вычисления результата этой функции требуется информация о позиции лексем. Поэтому она игнорирует «очищенные» от этой информации лексемы в `tsvector`. Если во входных данных нет неочищенных лексем, результат будет равен нулю. (За дополнительными сведениями о функции `strip` и позиционной информации в данных `tsvector` обратитесь к [Подразделу 12.4.1.](#))

Для обеих этих функций аргумент *веса* позволяет придать больший или меньший вес словам, в зависимости от их меток. В передаваемом массиве весов определяется, насколько весома каждая категория слов, в следующем порядке:

```
{вес D, вес C, вес B, вес A}
```

Если этот аргумент опускается, подразумеваются следующие значения:

```
{0.1, 0.2, 0.4, 1.0}
```

Обычно весами выделяются слова из особых областей документа, например из заголовка или краткого введения, с тем, чтобы эти слова считались более и менее значимыми, чем слова в основном тексте документа.

Так как вероятность найти ключевые слова увеличивается с размером документа, при ранжировании имеет смысл учитывать его, чтобы, например, документ с сотней слов, содержащий пять вхождений искомых слов, считался более релевантным, чем документ с тысячей слов и теми же пятью вхождениями. Обе функции ранжирования принимают целочисленный параметр *нормализации*, определяющий, как ранг документа будет зависеть от его размера. Этот параметр представляет собой битовую маску и управляет несколькими режимами: вы можете включить сразу несколько режимов, объединив значения оператором `|` (например так: `2|4`).

- 0 (по умолчанию): длина документа не учитывается
- 1: ранг документа делится на $1 + \log$ длины документа
- 2: ранг документа делится на его длину
- 4: ранг документа делится на среднее гармоническое расстояние между блоками (это реализовано только в `ts_rank_cd`)
- 8: ранг документа делится на число уникальных слов в документе
- 16: ранг документа делится на $1 + \log$ числа уникальных слов в документе
- 32: ранг делится своё же значение + 1

Если включены несколько флагов, соответствующие операции выполняются в показанном порядке.

Важно заметить, что функции ранжирования не используют никакую внешнюю информацию, так что добиться нормализации до 1% или 100% невозможно, хотя иногда это желательно. Применяв параметр 32 (`rank/(rank+1)`), можно свести все ранги к диапазону 0..1, но это изменение будет лишь косметическим, на порядке сортировки результатов это не отразится.

В данном примере выбираются десять найденных документов с максимальным рангом:

```
SELECT title, ts_rank_cd(textsearch, query) AS rank
FROM apod, to_tsquery('neutrino|(dark & matter)') query
WHERE query @@ textsearch
ORDER BY rank DESC
LIMIT 10;
```

title	rank
Neutrinos in the Sun	3.1
The Sudbury Neutrino Detector	2.4
A MACHO View of Galactic Dark Matter	2.01317
Hot Gas and Dark Matter	1.91171
The Virgo Cluster: Hot Plasma and Dark Matter	1.90953
Rafting for Solar Neutrinos	1.9
NGC 4650A: Strange Galaxy and Dark Matter	1.85774
Hot Gas and Dark Matter	1.6123
Ice Fishing for Cosmic Neutrinos	1.6

Weak Lensing Distorts the Universe | 0.818218

Тот же пример с нормализованным рангом:

```
SELECT title, ts_rank_cd(textsearch, query, 32 /* rank/(rank+1) */ ) AS rank
FROM apod, to_tsquery('neutrino|(dark & matter)') query
WHERE query @@ textsearch
ORDER BY rank DESC
LIMIT 10;
```

title	rank
Neutrinos in the Sun	0.756097569485493
The Sudbury Neutrino Detector	0.705882361190954
A MACHO View of Galactic Dark Matter	0.668123210574724
Hot Gas and Dark Matter	0.65655958650282
The Virgo Cluster: Hot Plasma and Dark Matter	0.656301290640973
Rafting for Solar Neutrinos	0.655172410958162
NGC 4650A: Strange Galaxy and Dark Matter	0.650072921219637
Hot Gas and Dark Matter	0.617195790024749
Ice Fishing for Cosmic Neutrinos	0.615384618911517
Weak Lensing Distorts the Universe	0.450010798361481

Ранжирование может быть довольно дорогостоящей операцией, так как для вычисления ранга необходимо прочитать `tsvector` каждого подходящего документа и это займёт значительное время, если придётся обращаться к диску. К сожалению, избежать этого вряд ли возможно, так как на практике по многим запросам выдаётся большое количество результатов.

12.3.4. Выделение результатов

Представляя результаты поиска, в идеале нужно выделять часть документа и показывать, как он связан с запросом. Обычно поисковые системы показывают фрагменты документа с отмеченными искомыми словами. В Postgres Pro для реализации этой возможности представлена функция `ts_headline`.

```
ts_headline([конфигурация regconfig,] документ text, query tsquery [, параметры text])
returns text
```

`ts_headline` принимает документ вместе с запросом и возвращает выдержку из документа, в которой выделяются слова из запроса. Применяемую для разбора документа конфигурацию можно указать в параметре `config`; если этот параметр опущен, применяется конфигурация `default_text_search_config`.

Если в параметрах передаётся строка `options`, она должна состоять из списка разделённых запятыми пар `параметр=значение`. Параметры могут быть следующими:

- `MaxWords`, `MinWords` (целочисленные): эти числа определяют нижний и верхний предел размера выдержки. Значения по умолчанию: 35 и 15, соответственно.
- `ShortWord` (целочисленный): слова такой длины или короче в начале и конце выдержки будут отбрасываться, за исключением искомых слов. Значение по умолчанию, равное 3, исключает распространённые английские артикли.
- `HighlightAll` (логический): при значении `true` выдержкой будет весь документ, и три предыдущие параметра игнорируются. Значение по умолчанию: `false`.
- `MaxFragments` (целочисленный): максимальное число выводимых текстовых фрагментов. Значение по умолчанию, равное нулю, выбирает режим создания выдержек без фрагментов. При положительном значении выбирается режим с фрагментами (см. ниже).
- `StartSel`, `StopSel`: строки, которые будут разграничивать слова запроса в документе, выделяя их среди остальных. Значения по умолчанию «» и «» подходят для отображения в формате HTML (однако обратите внимание на предупреждение ниже).
- `FragmentDelimiter` (строка): в случае, когда фрагментов несколько, они будут разделяться указанной строкой. Значение по умолчанию: « ... ».

Предупреждение: защита от межсайтового скриптинга (XSS)

Безопасность вывода функции `ts_headline` для прямого включения в веб-страницы не гарантируется. Если для `HighlightAll` установлено значение `false` (по умолчанию), некоторые простые XML-теги удаляются из документа, но это не гарантирует удаление всей HTML-разметки. Таким образом это не даёт эффективную защиту от таких атак, как межсайтовый скриптинг (XSS), при работе с недоверенными вводными данными. Для защиты от таких атак из передаваемого на вход документа необходимо удалять всю HTML-разметку или применять средства очистки HTML для вывода.

Имена этих параметров распознаются без учёта регистра. Значения, содержащие пробелы или запятые, должны заключаться в двойные кавычки.

В режиме формирования выдержек без фрагментов `ts_headline` находит вхождения слов заданного запроса и возвращает одно из вхождений, предпочитая те, что содержат как можно больше слов из запроса в пределах допустимого размера выдержки. В режиме с фрагментами `ts_headline` находит вхождения слов запроса и разделяет эти вхождения на «фрагменты», состоящие не более чем из `MaxWords` слов, предпочитая те, что содержат больше искомых слов, а затем может «растянуть» фрагменты, добавив в них соседние слова. Второй режим полезнее, когда слова запроса находятся не рядом, а разбросаны по документу, или когда желательно увидеть сразу несколько вхождений. В случаях, когда соответствие запросу найти не удаётся, в обоих режимах возвращаются первые `MinWords` слов из документа.

Пример использования:

```
SELECT ts_headline('english',
  'The most common type of search
is to find all documents containing given query terms
and return them in order of their similarity to the
query.',
  to_tsquery('english', 'query & similarity'));
      ts_headline
-----
containing given <b>query</b> terms          +
and return them in order of their <b>similarity</b> to the+
<b>query</b>.
```

```
SELECT ts_headline('english',
  'Search terms may occur
many times in a document,
requiring ranking of the search matches to decide which
occurrences to display in the result.',
  to_tsquery('english', 'search & term'),
  'MaxFragments=10, MaxWords=7, MinWords=3, StartSel=<<, StopSel=>>');
      ts_headline
-----
<<Search>> <<terms>> may occur          +
many times ... ranking of the <<search>> matches to decide
```

Функция `ts_headline` работает с оригинальным документом, а не его сжатым представлением `tsvector`, так что она может быть медленной и использовать её следует осмотрительно.

12.4. Дополнительные возможности

В этом разделе описываются дополнительные функции и операторы, которые могут быть полезны при поиске текста.

12.4.1. Обработка документов

В [Подразделе 12.3.1](#) показывалось, как обычные текстовые документы можно преобразовать в значения `tsvector`. Postgres Pro предлагает также набор функций и операторов для обработки документов, уже представленных в формате `tsvector`.

```
tsvector || tsvector
```

Оператор конкатенации значений `tsvector` возвращает вектор, объединяющий лексемы и позиционную информацию двух векторов, переданных ему в аргументах. В полученном результате сохраняются позиции и метки весов. При этом позиции в векторе справа сдвигаются на максимальное значение позиции в векторе слева, что почти равносильно применению `to_tsvector` к результату конкатенации двух исходных строк документов. (Почти, потому что стоп-слова, исключаемые в конце левого аргумента, при конкатенации исходных строк влияют на позиции лексем в правой части, а при конкатенации `tsvector` — нет.)

Преимущество же конкатенации документов в векторной форме по сравнению с конкатенацией текста до вызова `to_tsvector` заключается в том, что так можно разбирать разные части документа, применяя разные конфигурации. И так как функция `setweight` помечает все лексемы данного вектора одинаково, разбирать текст и выполнять `setweight` нужно до объединения разных частей документа с подразумеваемым разным весом.

```
setweight(вектор tsvector, вес "char") returns tsvector
```

`setweight` возвращает копию входного вектора, помечая в ней каждую позицию заданным *весом*, меткой A, B, C или D. (Метка D по умолчанию назначается всем векторам, так что при выводе она опускается.) Эти метки сохраняются при конкатенации векторов, что позволяет придавать разные веса словам из разных частей документа и, как следствие, ранжировать их по-разному.

Заметьте, что веса назначаются *позициям*, а не *лексемам*. Если входной вектор очищен от позиционной информации, `setweight` не делает ничего.

```
length(вектор tsvector) returns integer
```

Возвращает число лексем, сохранённых в векторе.

```
strip(вектор tsvector) returns tsvector
```

Возвращает вектор с теми же лексемами, что и в данном, но без информации о позиции и весе. Очищенный вектор обычно оказывается намного меньше исходного, но при этом и менее полезным. С очищенными векторами хуже работает ранжирование, а также оператор `<->` (ПРЕДШЕСТВУЕТ) типа `tsquery` никогда не найдёт соответствие в них, так как не сможет определить расстояние между вхождениями лексем.

Полный список связанных с `tsvector` функций приведён в [Таблице 9.43](#).

12.4.2. Обработка запросов

В [Подразделе 12.3.2](#) показывалось, как обычные текстовые запросы можно преобразовывать в значения `tsquery`. Postgres Pro предлагает также набор функций и операторов для обработки запросов, уже представленных в формате `tsquery`.

```
tsquery && tsquery
```

Возвращает логическое произведение (AND) двух данных запросов.

```
tsquery || tsquery
```

Возвращает логическое объединение (OR) двух данных запросов.

```
!! tsquery
```

Возвращает логическое отрицание (NOT) данного запроса.

```
tsquery <-> tsquery
```

Возвращает запрос, который ищет соответствие первому данному запросу, за которым следует соответствие второму данному запросу, с применением оператора <-> (ПРЕДШЕСТВУЕТ) типа `tsquery`. Например:

```
SELECT to_tsquery('fat') <-> to_tsquery('cat | rat');
       ?column?
-----
'fat' <-> ( 'cat' | 'rat' )
```

```
tsquery_phrase(запрос1 tsquery, запрос2 tsquery [, расстояние integer ]) returns tsquery
```

Возвращает запрос, который ищет соответствие первому данному запросу, за которым следует соответствие второму данному запросу (точное число лексем между ними задаётся параметром *расстояние*), с применением оператора <N> типа `tsquery`. Например:

```
SELECT tsquery_phrase(to_tsquery('fat'), to_tsquery('cat'), 10);
       tsquery_phrase
-----
'fat' <10> 'cat'
```

```
numnode(query tsquery) returns integer
```

Возвращает число узлов (лексем и операторов) в значении `tsquery`. Эта функция помогает определить, имеет ли смысл *запрос* (тогда её результат > 0) или он содержит только стоп-слова (тогда она возвращает 0). Примеры:

```
SELECT numnode(plainto_tsquery('the any'));
ЗAMEЧАНИЕ:  запрос поиска текста игнорируется, так как содержит только стоп-слова
или не содержит лексем
numnode
-----
0

SELECT numnode('foo & bar'::tsquery);
numnode
-----
3
```

```
querytree(query tsquery) returns text
```

Возвращает часть `tsquery`, которую можно использовать для поиска по индексу. Эта функция помогает выявить неиндексируемые запросы, к примеру такие, которые содержат только стоп-слова или условия отрицания. Например:

```
SELECT querytree(to_tsquery('defined'));
       querytree
-----
'defin'

SELECT querytree(to_tsquery('!defined'));
       querytree
-----
T
```

12.4.2.1. Перезапись запросов

Семейство запросов `ts_rewrite` ищет в данном `tsquery` вхождения целевого подзапроса и заменяет каждое вхождение указанной подстановкой. По сути эта операция похожа на замену подстроки в строке, только рассчитана на работу с `tsquery`. Сочетание целевого подзапроса с подстановкой можно считать *правилом перезаписи запроса*. Набор таких правил перезаписи может быть очень полезен при поиске. Например, вы можете улучшить результаты, добавив синонимы (например,

big apple, nyc и gotham для new york) или сузить область поиска, чтобы нацелить пользователя на некоторую область. Это в некотором смысле пересекается с функциональностью тезаурусов (Подраздел 12.6.4). Однако при таком подходе вы можете изменять правила перезаписи «на лету», тогда как при обновлении тезауруса необходима переиндексация.

`ts_rewrite (query tsquery, цель tsquery, замена tsquery) returns tsquery`

Эта форма `ts_rewrite` просто применяет одно правило перезаписи: *цель* заменяется *подстановкой* везде, где она находится в *запросе*. Например:

```
SELECT ts_rewrite('a & b'::tsquery, 'a'::tsquery, 'c'::tsquery);
ts_rewrite
-----
'b' & 'c'
```

`ts_rewrite (query tsquery, выборка text) returns tsquery`

Эта форма `ts_rewrite` принимает начальный *запрос* и SQL-команду *select*, которая задаётся текстовой строкой. Команда *select* должна выдавать два столбца типа `tsquery`. Для каждой строки результата *select* вхождения первого столбца (цели) заменяются значениями второго столбца (подстановкой) в тексте *запроса*. Например:

```
CREATE TABLE aliases (t tsquery PRIMARY KEY, s tsquery);
INSERT INTO aliases VALUES('a', 'c');

SELECT ts_rewrite('a & b'::tsquery, 'SELECT t,s FROM aliases');
ts_rewrite
-----
'b' & 'c'
```

Заметьте, что когда таким способом применяются несколько правил перезаписи, порядок их применения может иметь значение, поэтому в исходном запросе следует добавить `ORDER BY` по какому-либо ключу.

Давайте рассмотрим практический пример на тему астрономии. Мы развернём запрос `supernovae`, используя правила перезаписи в таблице:

```
CREATE TABLE aliases (t tsquery primary key, s tsquery);
INSERT INTO aliases VALUES(to_tsquery('supernovae'), to_tsquery('supernovae|sn'));

SELECT ts_rewrite(to_tsquery('supernovae & crab'), 'SELECT * FROM aliases');
ts_rewrite
-----
'crab' & ( 'supernova' | 'sn' )
```

Мы можем скорректировать правила перезаписи, просто изменив таблицу:

```
UPDATE aliases
SET s = to_tsquery('supernovae|sn & !nebulae')
WHERE t = to_tsquery('supernovae');

SELECT ts_rewrite(to_tsquery('supernovae & crab'), 'SELECT * FROM aliases');
ts_rewrite
-----
'crab' & ( 'supernova' | 'sn' & '!nebula' )
```

Перезапись может быть медленной, когда задано много правил перезаписи, так как соответствия будут проверяться для каждого правила. Чтобы отфильтровать явно неподходящие правила, можно использовать проверки включения для типа `tsquery`. В следующем примере выбираются только те правила, которые могут соответствовать исходному запросу:

```
SELECT ts_rewrite('a & b'::tsquery,
```

```

ts_rewrite
-----
's' & 'c'

```

12.4.3. Триггеры для автоматического обновления

Примечание

Описанный в этом разделе подход считается устаревшим в связи с появлением возможности использовать хранимые генерируемые столбцы, как описано в [Подразделе 12.2.2](#).

Когда представление документа в формате `tsvector` хранится в отдельном столбце, необходимо создать триггер, который будет обновлять его содержимое при изменении столбцов, из которых составляется исходный документ. Для этого можно использовать две встроенные триггерные функции или написать свои собственные.

```

tsvector_update_trigger(столбец_tsvector, имя_конфигурации, столбец_текста [, ...])
tsvector_update_trigger_column(столбец_tsvector, столбец_конфигурации, столбец_текста
[, ...])

```

Эти триггерные функции автоматически вычисляют значение для столбца `tsvector` из одного или нескольких текстовых столбцов с параметрами, указанными в команде `CREATE TRIGGER`. Пример их использования:

```

CREATE TABLE messages (
    title      text,
    body       text,
    tsv        tsvector
);

CREATE TRIGGER tsvectorupdate BEFORE INSERT OR UPDATE
ON messages FOR EACH ROW EXECUTE FUNCTION
tsvector_update_trigger(tsv, 'pg_catalog.english', title, body);

INSERT INTO messages VALUES('title here', 'the body text is here');

SELECT * FROM messages;
   title      |          body          |          tsv
-----+-----+-----
title here | the body text is here | 'bodi':4 'text':5 'titl':1

SELECT title, body FROM messages WHERE tsv @@ to_tsquery('title & body');
   title      |          body
-----+-----
title here | the body text is here

```

С таким триггером любое изменение в полях `title` или `body` будет автоматически отражаться в содержимом `tsv`, так что приложению не придётся заниматься этим.

Первым аргументом этих функций должно быть имя столбца `tsvector`, содержимое которого будет обновляться. Ещё один аргумент — конфигурация текстового поиска, которая будет использоваться для преобразования. Для `tsvector_update_trigger` имя конфигурации передаётся просто как второй аргумент триггера. Это имя должно быть определено полностью, чтобы поведение триггера не менялось при изменениях в пути поиска (`search_path`). Для `tsvector_update_trigger_column` во втором аргументе триггера передаётся имя другого столбца таблицы, который должен иметь тип `regconfig`. Это позволяет использовать разные конфигурации для разных строк. В оставшихся аргументах передаются имена текстовых столбцов (типа `text`, `varchar` или `char`). Их содержимое

будет включено в документ в заданном порядке. При этом значения NULL будут пропущены (а другие столбцы будут индексироваться).

Ограничение этих встроенных триггеров заключается в том, что они обрабатывают все столбцы одинаково. Чтобы столбцы обрабатывались по-разному, например для текста заголовка задавался не тот же вес, что для тела документа, потребуется разработать свой триггер. К примеру, так это можно сделать на языке PL/pgSQL:

```
CREATE FUNCTION messages_trigger() RETURNS trigger AS $$
begin
    new.tsv :=
        setweight(to_tsvector('pg_catalog.english', coalesce(new.title, '')), 'A') ||
        setweight(to_tsvector('pg_catalog.english', coalesce(new.body, '')), 'D');
    return new;
end
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tsvectorupdate BEFORE INSERT OR UPDATE
ON messages FOR EACH ROW EXECUTE FUNCTION messages_trigger();
```

Помните, что, создавая значения `tsvector` в триггерах, важно явно указывать имя конфигурации, чтобы содержимое столбца не зависело от изменений `default_text_search_config`. В противном случае могут возникнуть проблемы, например результаты поиска изменятся после выгрузки и восстановления данных.

12.4.4. Сбор статистики по документу

Функция `ts_stat` может быть полезна для проверки конфигурации и нахождения возможных стоп-слов.

```
ts_stat(sql_запрос text, [веса text,]
        OUT слово text, OUT число_док integer,
        OUT число_вхожд integer) returns setof record
```

Здесь `sql_запрос` — текстовая строка, содержащая SQL-запрос, который должен возвращать один столбец `tsvector`. Функция `ts_stat` выполняет запрос и возвращает статистику по каждой отдельной лексеме (слову), содержащейся в данных `tsvector`. Её результат представляется в столбцах

- `слово text` — значение лексемы
- `число_док integer` — число документов (значений `tsvector`), в которых встретилось слово
- `число_вхожд integer` — общее число вхождений слова

Если передаётся параметр `weights`, то подсчитываются только вхождения с указанными в нём весами.

Например, найти десять наиболее часто используемых слов в коллекции документов можно так:

```
SELECT * FROM ts_stat('SELECT vector FROM apod')
ORDER BY nentry DESC, ndoc DESC, word
LIMIT 10;
```

Следующий запрос возвращает тоже десять слов, но при выборе их учитываются только вхождения с весами A или B:

```
SELECT * FROM ts_stat('SELECT vector FROM apod', 'ab')
ORDER BY nentry DESC, ndoc DESC, word
LIMIT 10;
```

12.5. Анализаторы

Задача анализаторов текста — разделить текст документа на *фрагменты* и присвоить каждому из них тип из набора, определённого в самом анализаторе. Заметьте, что анализаторы не меняют

текст — они просто выдают позиции предполагаемых слов. Вследствие такой ограниченности их функций, собственные специфические анализаторы бывают нужны гораздо реже, чем собственные словари. В настоящее время в Postgres Pro есть только один встроенный анализатор, который может быть полезен для широкого круга приложений.

Этот встроенный анализатор называется `pg_catalog.default`. Он распознаёт 23 типа фрагментов, перечисленные в [Таблице 12.1](#).

Таблица 12.1. Типы фрагментов, выделяемых стандартным анализатором

Псевдоним	Описание	Пример
<code>asciiword</code>	Слово только из букв ASCII	<code>elephant</code>
<code>word</code>	Слово из любых букв	<code>mañana</code>
<code>numword</code>	Слово из букв и цифр	<code>beta1</code>
<code>asciihword</code>	Слово только из букв ASCII с дефисами	<code>up-to-date</code>
<code>hword</code>	Слово из любых букв с дефисами	<code>lógico-matemática</code>
<code>numhword</code>	Слово из букв и цифр с дефисами	<code>postgresql-beta1</code>
<code>hword_asciipart</code>	Часть слова с дефисами, только из букв ASCII	<code>postgresql</code> в словосочетании <code>postgresql-beta1</code>
<code>hword_part</code>	Часть слова с дефисами, из любых букв	<code>lógico</code> или <code>matemática</code> в словосочетании <code>lógico-matemática</code>
<code>hword_numpart</code>	Часть слова с дефисами, из букв и цифр	<code>beta1</code> в словосочетании <code>postgresql-beta1</code>
<code>email</code>	Адрес электронной почты	<code>foo@example.com</code>
<code>protocol</code>	Префикс протокола	<code>http://</code>
<code>url</code>	URL	<code>example.com/stuff/index.html</code>
<code>host</code>	Имя узла	<code>example.com</code>
<code>url_path</code>	Путь в адресе URL	<code>/stuff/index.html</code> , как часть URL
<code>file</code>	Путь или имя файла	<code>/usr/local/foo.txt</code> , если не является частью URL
<code>sfloat</code>	Научная запись числа	<code>-1.234e56</code>
<code>float</code>	Десятичная запись числа	<code>-1.234</code>
<code>int</code>	Целое со знаком	<code>-1234</code>
<code>uint</code>	Целое без знака	<code>1234</code>
<code>version</code>	Номер версии	<code>8.3.0</code>
<code>tag</code>	Тег XML	<code></code>
<code>entity</code>	Сущность XML	<code>&amp;</code>
<code>blank</code>	Символы-разделители	(любые пробельные символы или знаки препинания, не попавшие в другие категории)

Примечание

Понятие «буквы» анализатор определяет исходя из локали, заданной для базы данных, в частности параметра `lc_ctype`. Слова, содержащие только буквы из ASCII (латинские буквы),

распознаются как фрагменты отдельного типа, так как иногда бывает полезно выделить их. Для многих европейских языков типы фрагментов `word` и `asciiword` можно воспринимать как синонимы.

`email` принимает не все символы, которые считаются допустимыми по стандарту [RFC 5322](#). В частности, имя почтового ящика помимо алфавитно-цифровых символов может содержать только точку, минус и подчёркивание.

`tag` поддерживает не все действительные имена тегов, определённые в [Рекомендации W3C no XML](#). А именно поддерживаются только имена тегов, которые начинаются с буквы из ASCII, знака подчёркивания или двоеточия и содержат только буквы, цифры, дефисы, знаки подчёркивания и двоеточия. `tag` также включает комментарии XML, начинающиеся с `<!--` и заканчивающиеся `-->`, и объявления XML (однако обратите внимание, что сюда входит всё, что начинается с `<?x` и заканчивается `>`).

Анализатор может выделить в одном тексте несколько перекрывающихся фрагментов. Например, слово с дефисом будет выдано как целое составное слово и по частям:

```
SELECT alias, description, token FROM ts_debug('foo-bar-beta1');
```

alias	description	token
numhword	Hyphenated word, letters and digits	foo-bar-beta1
hword_asciipart	Hyphenated word part, all ASCII	foo
blank	Space symbols	-
hword_asciipart	Hyphenated word part, all ASCII	bar
blank	Space symbols	-
hword_numpart	Hyphenated word part, letters and digits	beta1

Это поведение считается желательным, так как это позволяет находить при последующем поиске и всё слово целиком, и его части. Ещё один показательный пример:

```
SELECT alias, description, token FROM ts_debug('http://example.com/stuff/index.html');
```

alias	description	token
protocol	Protocol head	http://
url	URL	example.com/stuff/index.html
host	Host	example.com
url_path	URL path	/stuff/index.html

12.6. Словари

Словари полнотекстового поиска предназначены для исключения *стоп-слов* (слов, которые не должны учитываться при поиске) и *нормализации* слов, чтобы разные словоформы считались совпадающими. Успешно нормализованное слово называется *лексемой*. Нормализация и исключение стоп-слов не только улучшает качество поиска, но и уменьшает размер представления документа в формате `tsvector`, и, как следствие, увеличивает быстродействие. Нормализация не всегда имеет лингвистический смысл, обычно она зависит от требований приложения.

Несколько примеров нормализации:

- Лингвистическая нормализация — словари `Ispell` пытаются свести слова на входе к нормализованной форме, а стеммеры убирают окончания слов
- Адреса URL могут быть канонизированы, чтобы например следующие адреса считались одинаковыми:
 - `http://www.pgsql.ru/db/mw/index.html`
 - `http://www.pgsql.ru/db/mw/`
 - `http://www.pgsql.ru/db/./db/mw/index.html`
- Названия цветов могут быть заменены их шестнадцатеричными значениями, например `red`, `green`, `blue`, `magenta` → `FF0000`, `00FF00`, `0000FF`, `FF00FF`

- При индексировании чисел можно отбросить цифры в дробной части для сокращения множества всевозможных чисел, чтобы например 3.14159265359, 3.1415926 и 3.14 стали одинаковыми после нормализации, при которой после точки останутся только две цифры.

Словарь — это программа, которая принимает на вход фрагмент и возвращает:

- массив лексем, если входной фрагмент известен в словаре (заметьте, один фрагмент может породить несколько лексем)
- одну лексему с установленным флагом `TSL_FILTER` для замены исходного фрагмента новым, чтобы следующие словари работали с новым вариантом (словарь, который делает это, называется *фильтрующим словарём*)
- пустой массив, если словарь воспринимает этот фрагмент, но считает его стоп-словом
- `NULL`, если словарь не воспринимает полученный фрагмент

В Postgres Pro встроены стандартные словари для многих языков. Есть также несколько предопределённых шаблонов, на основании которых можно создавать новые словари с изменёнными параметрами. Все эти шаблоны описаны ниже. Если же ни один из них не подходит, можно создать и свои собственные шаблоны. Соответствующие примеры можно найти в каталоге `contrib/` инсталляции Postgres Pro.

Конфигурация текстового поиска связывает анализатор с набором словарей, которые будут обрабатывать выделенные им фрагменты. Для каждого типа фрагментов, выданных анализатором, в конфигурации задаётся отдельный список словарей. Найденный анализатором фрагмент проходит через все словари по порядку, пока какой-либо словарь не увидит в нём знакомое для него слово. Если он окажется стоп-словом или его не распознает ни один словарь, этот фрагмент не будет учитываться при индексации и поиске. Обычно результат определяет первый же словарь, который возвращает не `NULL`, и остальные словари уже не проверяются; однако фильтрующий словарь может заменить полученное слово другим, которое и будет передано следующим словарям.

Общее правило настройки списка словарей заключается в том, чтобы поставить наиболее частные и специфические словари в начале, затем перечислить более общие и закончить самым общим словарём, например стеммером Snowball или словарём `simple`, который распознаёт всё. Например, для поиска по теме астрономии (конфигурация `astro_en`) тип фрагментов `asciiword` (слово из букв ASCII) можно связать со словарём синонимов астрономических терминов, затем с обычным английским словарём и наконец со стеммером английских окончаний Snowball:

```
ALTER TEXT SEARCH CONFIGURATION astro_en
ADD MAPPING FOR asciiword WITH astrosyn, english_ispell, english_stem;
```

Фильтрующий словарь можно включить в любом месте списка, кроме конца, где он будет бесполезен. Фильтрующие словари бывают полезны для частичной нормализации слов и упрощения задачи следующих словарей. Например, фильтрующий словарь может удалить из текста диакритические знаки, как это делает модуль [unaccent](#).

12.6.1. Стоп-слова

Стоп-словами называются слова, которые встречаются очень часто, практически в каждом документе, и поэтому не имеют различительной ценности. Таким образом, при полнотекстовом поиске их можно игнорировать. Например, в каждом английском тексте содержатся артикли `a` и `the`, так что хранить их в индексе бессмысленно. Однако стоп-слова влияют на позиции лексем в значении `tsvector`, от чего, в свою очередь, зависит ранжирование:

```
SELECT to_tsvector('english', 'in the list of stop words');
       to_tsvector
-----
'list':3 'stop':5 'word':6
```

В результате отсутствуют позиции 1,2,4, потому что фрагменты в этих позициях оказались стоп-словами. Ранги, вычисленные для документов со стоп-словами и без них, могут значительно различаться:

```
SELECT ts_rank_cd (to_tsvector('english', 'in the list of stop words'),
to_tsquery('list & stop'));
```

```
ts_rank_cd
-----
0.05
```

```
SELECT ts_rank_cd (to_tsvector('english', 'list stop words'), to_tsquery('list &
stop'));
ts_rank_cd
-----
0.1
```

Как именно обрабатывать стоп-слова, определяет сам словарь. Например, словари `ispell` сначала нормализуют слова, а затем просматривают список стоп-слов, тогда как стеммеры `Snowball` просматривают свой список стоп-слов в первую очередь. Это различие в поведении объясняется стремлением уменьшить шум.

12.6.2. Простой словарь

Работа шаблона словарей `simple` сводится к преобразованию входного фрагмента в нижний регистр и проверки результата по файлу со списком стоп-слов. Если это слово находится в файле, словарь возвращает пустой массив и фрагмент исключается из дальнейшего рассмотрения. В противном случае словарь возвращает в качестве нормализованной лексемы слово в нижнем регистре. Этот словарь можно настроить и так, чтобы все слова, кроме стоп-слов, считались неопознанными и передавались следующему словарю в списке.

Определить словарь на основе шаблона `simple` можно так:

```
CREATE TEXT SEARCH DICTIONARY public.simple_dict (
    TEMPLATE = pg_catalog.simple,
    STOPWORDS = english
);
```

Здесь `english` — базовое имя файла со стоп-словами. Полным именем файла будет `$SHAREDIR/tsearch_data/english.stop`, где `$SHAREDIR` указывает на каталог с общими данными Postgres Pro, часто это `/usr/local/share/postgresql` (точно узнать его можно с помощью команды `pg_config --sharedir`). Этот текстовый файл должен содержать просто список слов, по одному слову в строке. Пустые строки и окружающие пробелы игнорируются, все символы переводятся в нижний регистр и на этом обработка файла заканчивается.

Теперь мы можем проверить наш словарь:

```
SELECT ts_lexize('public.simple_dict', 'Yes');
ts_lexize
-----
{yes}

SELECT ts_lexize('public.simple_dict', 'The');
ts_lexize
-----
{}
```

Мы также можем настроить словарь так, чтобы он возвращал `NULL` вместо слова в нижнем регистре, если оно не находится в файле стоп-слов. Для этого нужно присвоить параметру `Accept` значение `false`. Продолжая наш пример:

```
ALTER TEXT SEARCH DICTIONARY public.simple_dict ( Accept = false );

SELECT ts_lexize('public.simple_dict', 'Yes');
ts_lexize
-----
```

```
SELECT ts_lexize('public.simple_dict', 'The');
       ts_lexize
-----
      {}
```

Со значением `Accept = true` (по умолчанию) словарь `simple` имеет смысл включать только в конце списка словарей, так как он никогда не передаст фрагмент следующему словарю. И напротив, `Accept = false` имеет смысл, только если за ним следует ещё минимум один словарь.

Внимание

Большинство словарей работают с дополнительными файлами, например, файлами стоп-слов. Содержимое этих файлов *должно* иметь кодировку UTF-8. Если база данных работает в другой кодировке, они будут переведены в неё, когда сервер будет загружать их.

Внимание

Обычно в рамках одного сеанса дополнительный файл словаря загружается только один раз, при первом использовании. Если же вы измените его и захотите, чтобы существующие сеансы работали с новым содержимым, выполните для этого словаря команду `ALTER TEXT SEARCH DICTIONARY`. Это обновление словаря может быть «фиктивным», фактически не меняющим значения никаких параметров.

12.6.3. Словарь синонимов

Этот шаблон словарей используется для создания словарей, заменяющих слова синонимами. Словосочетания такие словари не поддерживают (используйте для этого тезаурус ([Подраздел 12.6.4](#))). Словарь синонимов может помочь в преодолении лингвистических проблем, например, не дать стеммеру английского уменьшить слово «Paris» до «pari». Для этого достаточно поместить в словарь синонимов строку `Paris paris` и поставить этот словарь перед словарём `english_stem`. Например:

```
SELECT * FROM ts_debug('english', 'Paris');
  alias      | description      | token | dictionaries | dictionary | lexemes
-----+-----+-----+-----+-----+-----
asciword | Word, all ASCII | Paris | {english_stem} | english_stem | {pari}
```

```
CREATE TEXT SEARCH DICTIONARY my_synonym (
  TEMPLATE = synonym,
  SYNONYMS = my_synonyms
);
```

```
ALTER TEXT SEARCH CONFIGURATION english
  ALTER MAPPING FOR asciword
  WITH my_synonym, english_stem;
```

```
SELECT * FROM ts_debug('english', 'Paris');
  alias      | description      | token | dictionaries | dictionary | lexemes
-----+-----+-----+-----+-----+-----
asciword | Word, all ASCII | Paris | {my_synonym,english_stem} | my_synonym | {paris}
```

Шаблон `synonym` принимает единственный параметр, `SYNONYMS`, в котором задаётся базовое имя его файла конфигурации — в данном примере это `my_synonyms`. Полным именем файла будет `$SHAREDIR/tsearch_data/my_synonyms.syn` (где `$SHAREDIR` указывает на каталог общих данных Postgres Pro). Содержимое этого файла должны составлять строки с двумя словами в каждой (пер-

вое — заменяемое слово, а второе — его синоним), разделёнными пробелами. Пустые строки и окружающие пробелы при разборе этого файла игнорируются.

Шаблон `synonym` также принимает необязательный параметр `CaseSensitive`, который по умолчанию имеет значение `false`. Когда `CaseSensitive` равен `false`, слова в файле синонимов переводятся в нижний регистр, вместе с проверяемыми фрагментами. Если же он не равен `true`, регистр слов в файле и проверяемых фрагментов не меняются, они сравниваются «как есть».

В конце синонима в этом файле можно добавить звёздочку (*), тогда этот синоним будет рассматриваться как префикс. Эта звёздочка будет игнорироваться в `to_tsvector()`, но `to_tsquery()` изменит результат, добавив в него маркер сопоставления префикса (см. [Подраздел 12.3.2](#)). Например, предположим, что файл `$SHAREDIR/tsearch_data/synonym_sample.syn` имеет следующее содержание:

```
postgres      pgsql
postgresql    pgsql
postgre pgsql
google        googl
indices index*
```

С ним мы получим такие результаты:

```
mydb=# CREATE TEXT SEARCH DICTIONARY syn (template=synonym, synonyms='synonym_sample');
mydb=# SELECT ts_lexize('syn', 'indices');
      ts_lexize
-----
 {index}
(1 row)
```

```
mydb=# CREATE TEXT SEARCH CONFIGURATION tst (copy=simple);
mydb=# ALTER TEXT SEARCH CONFIGURATION tst ALTER MAPPING FOR asciiword WITH syn;
mydb=# SELECT to_tsvector('tst', 'indices');
      to_tsvector
-----
 'index':1
(1 row)
```

```
mydb=# SELECT to_tsquery('tst', 'indices');
      to_tsquery
-----
 'index':*
(1 row)
```

```
mydb=# SELECT 'indexes are very useful'::tsvector;
      tsvector
-----
 'are' 'indexes' 'useful' 'very'
(1 row)
```

```
mydb=# SELECT 'indexes are very useful'::tsvector @@ to_tsquery('tst', 'indices');
      ?column?
-----
 t
(1 row)
```

12.6.4. Тезаурус

Тезаурус (или сокращённо TZ) содержит набор слов и информацию о связях слов и словосочетаний, то есть более широкие понятия (Broader Terms, BT), более узкие понятия (Narrow Terms, NT), предпочитаемые названия, исключаемые названия, связанные понятия и т. д.

В основном тезаурус заменяет исключаемые слова и словосочетания предпочитаемыми и может также сохранить исходные слова для индексации. Текущая реализация тезауруса в Postgres Pro представляет собой расширение словаря синонимов с поддержкой *фраз*. Конфигурация тезауруса определяется файлом следующего формата:

```
# это комментарий
образец слов(a) : индексируемые слова
другой образец слов(a) : другие индексируемые слова
...
```

Здесь двоеточие (:) служит разделителем между исходной фразой и её заменой.

Прежде чем проверять соответствие фраз, тезаурус нормализует файл конфигурации, используя *внутренний словарь* (который указывается в конфигурации словаря-тезауруса). Этот внутренний словарь для тезауруса может быть только одним. Если он не сможет распознать какое-либо слово, произойдёт ошибка. В этом случае необходимо либо исключить это слово, либо добавить его во внутренний словарь. Также можно добавить звёздочку (*) перед индексируемыми словами, чтобы они не проверялись по внутреннему словарю, но все слова-образцы *должны* быть известны внутреннему словарю.

Если входному фрагменту соответствуют несколько фраз в этом списке, тезаурус выберет самое длинное определение, а если таких окажется несколько, самое последнее из них.

Выделить во фразе какие-то стоп-слова нельзя; вместо этого можно вставить ? в том месте, где может оказаться стоп-слово. Например, в предположении, что a и the — стоп-слова по внутреннему словарю:

```
? one ? two : ssws
```

соответствует входным строкам a one the two и the one a two, так что обе эти строки будут заменены на ssws.

Как и обычный словарь, тезаурус должен привязываться к лексемам определённых типов. Так как тезаурус может распознавать фразы, он должен запоминать своё состояние и взаимодействовать с анализатором. Учитывая свои привязки, он может либо обрабатывать следующий фрагмент, либо прекратить накопление фразы. Поэтому настройка тезаурусов в системе требует особого внимания. Например, если привязать тезаурус только к типу фрагментов `asciiword`, тогда определение в тезаурусе `one 7` не будет работать, так как этот тезаурус не связан с типом `uint`.

Внимание

Тезаурусы используются при индексации, поэтому при любом изменении параметров или содержимого тезауруса *необходима* переиндексация. Для большинства других типов словарей при небольших изменениях, таких как удаление и добавление стоп-слов, переиндексация не требуется.

12.6.4.1. Конфигурация тезауруса

Для создания нового словаря-тезауруса используется шаблон `thesaurus`. Например:

```
CREATE TEXT SEARCH DICTIONARY thesaurus_simple (
    TEMPLATE = thesaurus,
    DictFile = mythesaurus,
    Dictionary = pg_catalog.english_stem
);
```

Здесь:

- `thesaurus_simple` — имя нового словаря
- `mythesaurus` — базовое имя файла конфигурации тезауруса. (Полным путём к файлу будет `$$SHAREDIR/tsearch_data/mythesaurus.ths`, где `$$SHAREDIR` указывает на каталог общих данных PostgreSQL.)

- `pg_catalog.english_stem` — внутренний словарь (в данном случае это стеммер Snowball для английского) для нормализации тезауруса. Заметьте, что внутренний словарь имеет собственную конфигурацию (например, список стоп-слов), но здесь она не рассматривается.

Теперь тезаурус `thesaurus_simple` можно связать с желаемыми типами фрагментов в конфигурации, например так:

```
ALTER TEXT SEARCH CONFIGURATION english
  ALTER MAPPING FOR asciiword, asciihword, hword_asciipart
  WITH thesaurus_simple;
```

12.6.4.2. Пример тезауруса

Давайте рассмотрим простой астрономический тезаурус `thesaurus_astro`, содержащий несколько астрономических терминов:

```
supernovae stars : sn
crab nebulae : crab
```

Ниже мы создадим словарь и привяжем некоторые типы фрагментов к астрономическому тезаурусу и английскому стеммеру:

```
CREATE TEXT SEARCH DICTIONARY thesaurus_astro (
  TEMPLATE = thesaurus,
  DictFile = thesaurus_astro,
  Dictionary = english_stem
);

ALTER TEXT SEARCH CONFIGURATION russian
  ALTER MAPPING FOR asciiword, asciihword, hword_asciipart
  WITH thesaurus_astro, english_stem;
```

Теперь можно проверить, как он работает. Функция `ts_lexize` не очень полезна для проверки тезауруса, так как она обрабатывает входную строку как один фрагмент. Вместо неё мы можем использовать функции `plainto_tsquery` и `to_tsvector`, которые разбивают входную строку на несколько фрагментов:

```
SELECT plainto_tsquery('supernova star');
plainto_tsquery
-----
'sn'

SELECT to_tsvector('supernova star');
to_tsvector
-----
'sn':1
```

В принципе так же можно использовать `to_tsquery`, если заключить аргумент в кавычки:

```
SELECT to_tsquery(' 'supernova star');
to_tsquery
-----
'sn'
```

Заметьте, что `supernova star` совпадает с `supernovae stars` в `thesaurus_astro`, так как мы подключили стеммер `english_stem` в определении тезауруса. Этот стеммер удалил конечные буквы `e` и `s`.

Чтобы проиндексировать исходную фразу вместе с заменой, её нужно просто добавить в правую часть соответствующего определения:

```
supernovae stars : sn supernovae stars

SELECT plainto_tsquery('supernova star');
plainto_tsquery
```

'sn' & 'supernova' & 'star'

12.6.5. Словарь Ispell

Шаблон словарей Ispell поддерживает *морфологические словари*, которые могут сводить множество разных лингвистических форм слова к одной лексеме. Например, английский словарь Ispell может связать вместе все склонения и спряжения ключевого слова bank: banking, banked, banks, banks', bank's и т. п.

Стандартный дистрибутив Postgres Pro не включает файлы конфигурации Ispell. Загрузить словари для множества языков можно со страницы [Ispell](#). Кроме того, поддерживаются и другие современные форматы словарей: [MySpell](#) (OO < 2.0.1) и [Hunspell](#) (OO >= 2.0.2). Большой набор соответствующих словарей можно найти на странице [OpenOffice Wiki](#).

Чтобы создать словарь Ispell, выполните следующие действия:

- загрузите файлы конфигурации словаря. Пакет с дополнительным словарём OpenOffice имеет расширение .oxt. Из него необходимо извлечь файлы .aff и .dic, и сменить их расширения на .affix и .dict, соответственно. Для некоторых файлов словарей также необходимо преобразовать символы в кодировку UTF-8 с помощью, например, таких команд (для норвежского языка):

```
iconv -f ISO_8859-1 -t UTF-8 -o nn_no.affix nn_NO.aff
iconv -f ISO_8859-1 -t UTF-8 -o nn_no.dict nn_NO.dic
```

- скопируйте файлы в каталог \$SHAREDIR/tsearch_data
- загрузите эти файлы в Postgres Pro следующей командой:

```
CREATE TEXT SEARCH DICTIONARY english_hunspell (
    TEMPLATE = ispell,
    DictFile = en_us,
    AffFile = en_us,
    Stopwords = english);
```

Здесь параметры DictFile, AffFile и StopWords определяют базовые имена файлов словаря, аффиксов и стоп-слов. Файл стоп-слов должен иметь тот же формат, что рассматривался выше в описании словаря simple. Формат других файлов здесь не рассматривается, но его можно узнать по вышеуказанным веб-адресам.

Словари Ispell обычно воспринимают ограниченный набор слов, так что за ними следует подключить другой, более общий словарь, например, Snowball, который принимает всё.

Файл .affix для Ispell имеет такую структуру:

```
prefixes
flag *A:
    .          > RE      # As in enter > reenter
suffixes
flag T:
    E          > ST      # As in late > latest
    [^AEIOU]Y > -Y,IEST # As in dirty > dirtiest
    [AEIOU]Y  > EST     # As in gray > grayest
    [^EY]     > EST     # As in small > smallest
```

А файл .dict — такую:

```
lapse/ADGRS
lard/DGRS
large/PRTY
lark/MRS
```

Формат файла .dict следующий:

```
basic_form/affix_class_name
```

В файле `.affix` каждый флаг аффиксов описывается в следующем формате:

```
условие > [-отсекаемые_буквы,] добавляемый_аффикс
```

Здесь условие записывается в формате, подобном формату регулярных выражений. В нём возможно описать группы [...] и [^...]. Например, запись [AEIOU]Y означает, что последняя буква слова — "y", а предпоследней может быть "a", "e", "i", "o" или "u". Запись [^EY] означает, что последняя буква не "e" и не "y".

Словари Ispell поддерживают разделение составных слов, что бывает полезно. Заметьте, что для этого в файле аффиксов нужно пометить специальным оператором `compoundwords controlled` слова, которые могут участвовать в составных образованиях:

```
compoundwords controlled z
```

Вот как это работает для норвежского языка:

```
SELECT ts_lexize('norwegian_ispell', 'overbuljongterningpakkmasterassistent');
       {over,buljong,terning,pakk,mester,assistent}
SELECT ts_lexize('norwegian_ispell', 'sjokoladefabrikk');
       {sjokoladefabrikk,sjokolade,fabrikk}
```

Формат MySpell представляет собой подмножество формата Hunspell. Файл `.affix` словаря Hunspell имеет следующую структуру:

```
PFX A Y 1
PFX A  0      re      .
SFX T N 4
SFX T  0      st      e
SFX T  y      iest     [^aeiou]y
SFX T  0      est      [aeiou]y
SFX T  0      est      [^ey]
```

Первая строка класса аффиксов — заголовок. Поля правил аффиксов указываются после заголовка:

- имя параметра (PFX или SFX)
- флаг (имя класса аффиксов)
- отсекаемые символы в начале (в префиксе) или в конце (в суффиксе) слова
- добавляемый аффикс
- условие в формате, подобном регулярным выражениям.

Файл `.dict` подобен файлу `.dict` словаря Ispell:

```
larder/M
lardy/RT
large/RSPMYT
largehearted
```

Примечание

Словарь MySpell не поддерживает составные слова. С другой стороны, Hunspell поддерживает множество операций с ними, но в настоящее время Postgres Pro использует только самые простые из этого множества.

12.6.6. Словарь Snowball

Шаблон словарей Snowball основан на проекте Мартина Потера, изобретателя популярного алгоритма стемминга для английского языка. Сейчас Snowball предлагает алгоритмы и для многих

других языков (за подробностями обратитесь на [caïm Snowball](#)). Каждый алгоритм знает, как для данного языка свести распространённые словоформы к начальной форме. Для словаря Snowball задаётся обязательный параметр `language`, определяющий, какой именно стеммер использовать, и может задаваться параметр `stopword`, указывающий файл со списком исключаемых слов. (Стандартные списки стоп-слов Postgres Pro используется также в и проекте Snowball.) Например, встроенное определение выглядит так

```
CREATE TEXT SEARCH DICTIONARY english_stem (
    TEMPLATE = snowball,
    Language = english,
    StopWords = english
);
```

Формат файла стоп-слов не отличается от рассмотренного ранее.

Словарь Snowball распознаёт любые фрагменты, даже если он не может упростить слова, так что он должен быть самым последним в списке словарей. Помещать его перед другими словарями нет смысла, так как после него никакой фрагмент не будет передан следующему словарю.

12.7. Пример конфигурации

Конфигурация текстового поиска определяет всё, что необходимо для преобразования документа в формат `tsvector`: анализатор, который будет разбивать текст на фрагменты, и словари, которые будут преобразовывать фрагменты в лексемы. При каждом вызове `to_tsvector` или `to_tsquery` обязательно используется конфигурация текстового поиска. В конфигурации сервера есть параметр [default_text_search_config](#), задающий имя конфигурации текстового поиска по умолчанию, которая будет использоваться, когда при вызове функций поиска соответствующий аргумент не определён. Этот параметр можно задать в `postgresql.conf` или установить в рамках отдельного сеанса с помощью команды `SET`.

В системе есть несколько встроенных конфигураций текстового поиска и вы можете легко дополнить их своими. Для удобства управления объектами текстового поиска в PostgreSQL реализованы соответствующие SQL-команды и специальные команды в `psql`, выводящие информацию об этих объектах ([Раздел 12.10](#)).

В качестве примера использования этих команд мы создадим конфигурацию `pg`, взяв за основу встроенную конфигурацию `english`:

```
CREATE TEXT SEARCH CONFIGURATION public.pg ( COPY = pg_catalog.english );
```

Мы будем использовать список синонимов, связанных с Postgres Pro, в файле `$SHAREDIR/tsearch_data/pg_dict.syn`. Этот файл содержит строки:

```
postgres      pg
pgsql         pg
postgresql    pg
```

Мы определим словарь синонимов следующим образом:

```
CREATE TEXT SEARCH DICTIONARY pg_dict (
    TEMPLATE = synonym,
    SYNONYMS = pg_dict
);
```

Затем мы зарегистрируем словарь Ispell `english_ispell`, у которого есть собственные файлы конфигурации:

```
CREATE TEXT SEARCH DICTIONARY english_ispell (
    TEMPLATE = ispell,
    DictFile = english,
    AffFile = english,
    StopWords = english
);
```

Теперь мы можем настроить сопоставления для слов в конфигурации pg:

```
ALTER TEXT SEARCH CONFIGURATION pg
    ALTER MAPPING FOR asciiword, asciihword, hword_asciipart,
                        word, hword, hword_part
    WITH pg_dict, english_ispell, english_stem;
```

Мы решили не индексировать и не учитывать при поиске некоторые типы фрагментов, которые не обрабатываются встроенной конфигурацией:

```
ALTER TEXT SEARCH CONFIGURATION pg
    DROP MAPPING FOR email, url, url_path, sfloat, float;
```

Теперь мы можем протестировать нашу конфигурацию:

```
SELECT * FROM ts_debug('public.pg', '
Postgres Pro, the highly scalable, SQL compliant, open source
object-relational database management system, is now undergoing
beta testing of the next version of our software.
');
```

И наконец мы выбираем в текущем сеансе эту конфигурацию, созданную в схеме public:

```
=> \dF
      List of text search configurations
 Schema | Name | Description
-----+-----+-----
 public | pg   |

SET default_text_search_config = 'public.pg';
SET

SHOW default_text_search_config;
 default_text_search_config
-----
 public.pg
```

12.8. Тестирование и отладка текстового поиска

Поведение нестандартной конфигурации текстового поиска по мере её усложнения может стать непонятным. В этом разделе описаны функции, полезные для тестирования объектов текстового поиска. Вы можете тестировать конфигурацию как целиком, так и по частям, отлаживая анализаторы и словари по отдельности.

12.8.1. Тестирование конфигурации

Созданную конфигурацию текстового поиска можно легко протестировать с помощью функции `ts_debug`.

```
ts_debug([конфигурация regconfig,] документ text,
        OUT псевдоним text,
        OUT описание text,
        OUT фрагмент text,
        OUT словари regdictionary[],
        OUT словарь regdictionary,
        OUT лексемы text[])
returns setof record
```

`ts_debug` выводит информацию обо всех фрагментах данного документа, которые были выданы анализатором и обработаны настроенными словарями. Она использует конфигурацию, указанную в аргументе `config`, или `default_text_search_config`, если этот аргумент опущен.

`ts_debug` возвращает по одной строке для каждого фрагмента, найденного в тексте анализатором. Эта строка содержит следующие столбцы:

- *синоним* `text` — краткое имя типа фрагмента
- *описание* `text` — описание типа фрагмента
- *фрагмент* `text` — текст фрагмента
- *словари* `regdictionary[]` — словари, назначенные в конфигурации для фрагментов такого типа
- *словарь* `regdictionary` — словарь, распознавший этот фрагмент, или `NULL`, если подходящего словаря не нашлось
- *лексемы* `text[]` — лексемы, выданные словарём, распознавшим фрагмент, или `NULL`, если подходящий словарь не нашёлся; может быть также пустым массивом (`{}`), если фрагмент распознан как стоп-слово

Простой пример:

```
SELECT * FROM ts_debug('english', 'a fat cat sat on a mat - it ate a fat rats');
```

alias	description	token	dictionaries	dictionary	lexemes
asciword	Word, all ASCII	a	{english_stem}	english_stem	{}
blank	Space symbols		{}		
asciword	Word, all ASCII	fat	{english_stem}	english_stem	{fat}
blank	Space symbols		{}		
asciword	Word, all ASCII	cat	{english_stem}	english_stem	{cat}
blank	Space symbols		{}		
asciword	Word, all ASCII	sat	{english_stem}	english_stem	{sat}
blank	Space symbols		{}		
asciword	Word, all ASCII	on	{english_stem}	english_stem	{}
blank	Space symbols		{}		
asciword	Word, all ASCII	a	{english_stem}	english_stem	{}
blank	Space symbols		{}		
asciword	Word, all ASCII	mat	{english_stem}	english_stem	{mat}
blank	Space symbols		{}		
blank	Space symbols	-	{}		
asciword	Word, all ASCII	it	{english_stem}	english_stem	{}
blank	Space symbols		{}		
asciword	Word, all ASCII	ate	{english_stem}	english_stem	{ate}
blank	Space symbols		{}		
asciword	Word, all ASCII	a	{english_stem}	english_stem	{}
blank	Space symbols		{}		
asciword	Word, all ASCII	fat	{english_stem}	english_stem	{fat}
blank	Space symbols		{}		
asciword	Word, all ASCII	rats	{english_stem}	english_stem	{rat}

Для более полной демонстрации мы сначала создадим конфигурацию `public.english` и словарь `Ispell` для английского языка:

```
CREATE TEXT SEARCH CONFIGURATION public.english ( COPY = pg_catalog.english );
```

```
CREATE TEXT SEARCH DICTIONARY english_ispell (
    TEMPLATE = ispell,
    DictFile = english,
    AffFile = english,
    StopWords = english
);
```

```
ALTER TEXT SEARCH CONFIGURATION public.english
    ALTER MAPPING FOR asciword WITH english_ispell, english_stem;
```

```
SELECT * FROM ts_debug('public.english', 'The Brightest supernovaes');
```

alias	description	token	dictionaries
dictionary	lexemes		
asciword	Word, all ASCII	The	{english_ispell,english_stem}
english_ispell	{}		
blank	Space symbols		{}
asciword	Word, all ASCII	Brightest	{english_ispell,english_stem}
english_ispell	{bright}		
blank	Space symbols		{}
asciword	Word, all ASCII	supernovaes	{english_ispell,english_stem}
english_stem	{supernova}		

В этом примере слово `Brightest` было воспринято анализатором как фрагмент ASCII word (синоним `asciword`). Для этого типа фрагментов список словарей включает `english_ispell` и `english_stem`. Данное слово было распознано словарём `english_ispell`, который свёл его к `bright`. Слово `supernovaes` оказалось незнакомо словарю `english_ispell`, так что оно было передано следующему словарю, который его благополучно распознал (на самом деле `english_stem` — это стеммер `Snowball`, который распознаёт всё, поэтому он включён в список словарей последним).

Слово `The` было распознано словарём `english_ispell` как стоп-слово (см. [Подраздел 12.6.1](#)) и поэтому не будет индексироваться. Пробелы тоже отбрасываются, так как в данной конфигурации для них нет словарей.

Вы можете уменьшить ширину вывода, явно перечислив только те столбцы, которые вы хотите видеть:

```
SELECT alias, token, dictionary, lexemes
FROM ts_debug('public.english', 'The Brightest supernovaes');
alias | token | dictionary | lexemes
-----+-----+-----+-----
asciword | The | english_ispell | {}
blank | | | 
asciword | Brightest | english_ispell | {bright}
blank | | | 
asciword | supernovaes | english_stem | {supernova}
```

12.8.2. Тестирование анализатора

Следующие функции позволяют непосредственно протестировать анализатор текстового поиска.

```
ts_parse(имя_анализатора text, документ text,
OUT код_фрагмента integer, OUT фрагмент text) returns setof record
ts_parse(oid_анализатора oid, документ text,
OUT код_фрагмента integer, OUT фрагмент text) returns setof record
```

`ts_parse` разбирает данный документ и возвращает набор записей, по одной для каждого извлечённого фрагмента. Каждая запись содержит код_фрагмента, код назначенного типа фрагмента, и фрагмент, собственно текст фрагмента. Например:

```
SELECT * FROM ts_parse('default', '123 - a number');
tokid | token
-----+-----
22 | 123
12 | 
12 | -
1 | a
12 |
```

1 | number

```
ts_token_type(имя_анализатора text, OUT код_фрагмента integer,
              OUT псевдоним text, OUT описание text) returns setof record
ts_token_type(oid_анализатора oid, OUT код_фрагмента integer,
              OUT псевдоним text, OUT описание text) returns setof record
```

`ts_token_type` возвращает таблицу, описывающую все типы фрагментов, которые может распознать анализатор. Для каждого типа в этой таблице указывается целочисленный `tokid` (идентификатор), который анализатор использует для пометки фрагмента этого типа, `alias` (псевдоним), с которым этот тип фигурирует в командах конфигурации, и `description` (краткое описание). Например:

```
SELECT * FROM ts_token_type('default');
 tokid |      alias      | description
-----+-----+-----
    1 | asciiword       | Word, all ASCII
    2 | word            | Word, all letters
    3 | numword         | Word, letters and digits
    4 | email           | Email address
    5 | url             | URL
    6 | host            | Host
    7 | sfloat          | Scientific notation
    8 | version         | Version number
    9 | hword_numpart   | Hyphenated word part, letters and digits
   10 | hword_part      | Hyphenated word part, all letters
   11 | hword_asciipart | Hyphenated word part, all ASCII
   12 | blank           | Space symbols
   13 | tag             | XML tag
   14 | protocol        | Protocol head
   15 | numhword        | Hyphenated word, letters and digits
   16 | asciihword      | Hyphenated word, all ASCII
   17 | hword           | Hyphenated word, all letters
   18 | url_path        | URL path
   19 | file            | File or path name
   20 | float           | Decimal notation
   21 | int             | Signed integer
   22 | uint            | Unsigned integer
   23 | entity          | XML entity
```

12.8.3. Тестирование словаря

Для тестирования словаря предназначена функция `ts_lexize`.

```
ts_lexize(словарь regdictionary, фрагмент text) returns text[]

ts_lexize возвращает массив лексем, если входной фрагмент известен словарю, либо пустой массив, если этот фрагмент считается в словаре стоп-словом, либо NULL, если он не был распознан.
```

Примеры:

```
SELECT ts_lexize('english_stem', 'stars');
 ts_lexize
-----
 {star}

SELECT ts_lexize('english_stem', 'a');
 ts_lexize
-----
 {}
```

Примечание

Функция `ts_lexize` принимает одиночный *фрагмент*, а не просто текст. Вот пример возможного заблуждения:

```
SELECT ts_lexize('thesaurus_astro', 'supernovae stars') is null;
?column?
-----
t
```

Хотя фраза `supernovae stars` есть в тезаурусе `thesaurus_astro`, `ts_lexize` не работает, так как она не разбирает входной текст, а воспринимает его как один фрагмент. Поэтому для проверки тезаурусов следует использовать функции `plainto_tsquery` и `to_tsvector`, например:

```
SELECT plainto_tsquery('supernovae stars');
plainto_tsquery
-----
'sn'
```

12.9. Типы индексов, предпочитаемые для текстового поиска

Для ускорения полнотекстового поиска можно использовать индексы [GIN](#), [RUM](#) и [GiST](#). Заметьте, что эти индексы не требуются для поиска, но если по какому-то столбцу поиск выполняется регулярно, обычно желательно создать индекс.

Чтобы создать такой индекс, выполните одну из следующих команд:

```
CREATE INDEX имя ON таблица USING GIN (столбец);
```

Создаёт индекс на базе GIN (Generalized Inverted Index, Обобщённый Инвертированный Индекс). *Столбец* должен иметь тип `tsvector`.

```
CREATE INDEX имя ON таблица USING RUM (столбец);
```

Создаёт индекс на базе RUM. *Столбец* должен иметь тип `tsvector`.

```
CREATE INDEX имя ON таблица USING GIST (столбец [ { DEFAULT | tsvector_ops } (siglen = число) ] );
```

Создаёт индекс на базе GiST (Generalized Search Tree, Обобщённое дерево поиска). Здесь *столбец* может иметь тип `tsvector` или `tsquery`. Необязательный числовой параметр `siglen` определяет длину сигнатуры в байтах (подробнее об этом ниже).

Более предпочтительными для текстового поиска являются индексы GIN и основанные на методе доступа GIN индексы RUM. Будучи инвертированными индексами, они содержат записи для всех отдельных слов (лексем) с компактным списком мест их вхождений. При поиске нескольких слов можно найти первое, а затем воспользоваться индексом и исключить строки, в которых дополнительные слова отсутствуют. Индексы GIN хранят только слова (лексеммы) из значений `tsvector` и теряют информацию об их весах. Таким образом для выполнения запроса с весами потребуется перепроверить строки в таблице.

Индекс GiST допускает *неточности*, то есть он допускает ложные попадания и поэтому их нужно исключать дополнительно, сверяя результат с фактическими данными таблицы. (Postgres Pro делает это автоматически.) Индексы GiST являются неточными, так как все документы в них представляются сигнатурой фиксированной длины. Длина сигнатуры в байтах определяется значением необязательного целочисленного параметра `siglen`. По умолчанию (когда параметр `siglen` отсутствует) равно 124 байтам, а максимальная длина сигнатуры равна 2024 байтам. Сигнатура формируется в результате представления присутствия каждого слова как одного бита в строке из `n`-бит, а

затем логического объединения этих битовых строк. Если двум словам будет соответствовать одна битовая позиция, попадание оказывается ложным. Если для всех слов оказались установлены соответствующие биты (в случае фактического или ложного попадания), для проверки правильности предположения о совпадении слов необходимо прочитать строку таблицы. При увеличении размера сигнатуры поиск работает точнее (сканируется меньшая область в индексе и меньше страниц кучи), но сам индекс становится больше.

Индекс GiST может быть покрывающим, то есть использовать функциональность `INCLUDE`. В качестве дополнительных в него могут включаться столбцы, для типа данных которых не определён класс операторов GiST. Атрибуты, включаемые в индекс, сохраняются в нём без сжатия.

Неточность индекса приводит к снижению производительности из-за дополнительных обращений к записям таблицы, для которых предположение о совпадении оказывается ложным. Так как произвольный доступ к таблице обычно не бывает быстрым, это ограничивает применимость индексов GiST. Вероятность ложных попаданий зависит от ряда факторов, например от количества уникальных слов, так что его рекомендуется сокращать, применяя словари.

Заметьте, что построение индекса GIN часто можно ускорить, увеличив [maintenance_work_mem](#), тогда как время построения индекса GiST не зависит от этого параметра.

Правильно используя индексы GIN, RUM и GiST и разделяя большие коллекции документов на секции, можно реализовать очень быстрый поиск с возможностью обновления «на лету». Секционировать данные можно как на уровне базы, с использованием наследования таблиц, так и распределив документы по разным серверам и затем собирая результаты внешнего поиска, например, средствами доступа к [сторонним данным](#). Последний вариант возможен благодаря тому, что функции ранжирования используют только локальную информацию.

12.10. Поддержка psql

Информацию об объектах конфигурации текстового поиска можно получить в psql с помощью следующего набора команд:

```
\dF{d,p,t}[+] [ШАБЛОН]
```

Необязательный `+` в этих командах включает более подробный вывод.

В необязательном параметре *ШАБЛОН* может указываться имя объекта текстового поиска (возможно, дополненное схемой). Если *ШАБЛОН* не указан, выводится информация обо всех видимых объектах. *ШАБЛОН* может содержать регулярное выражение с *разными* масками для схемы и объекта. Это иллюстрируют следующие примеры:

```
=> \dF *fulltext*
      List of text search configurations
 Schema | Name           | Description
-----+-----+-----
 public | fulltext_cfg   |

=> \dF *.fulltext*
      List of text search configurations
 Schema | Name           | Description
-----+-----+-----
 fulltext | fulltext_cfg   |
 public   | fulltext_cfg   |
```

Возможны следующие команды:

```
\dF[+] [ШАБЛОН]
```

Список конфигураций текстового поиска (добавьте `+` для дополнительных сведений).

```
=> \dF russian
      List of text search configurations
```

Schema	Name	Description
pg_catalog	russian	configuration for russian language

=> \dF+ russian

Text search configuration "pg_catalog.russian"

Parser: "pg_catalog.default"

Token	Dictionaries
asciihword	english_stem
asciiword	english_stem
email	simple
file	simple
float	simple
host	simple
hword	russian_stem
hword_asciipart	english_stem
hword_numpart	simple
hword_part	russian_stem
int	simple
numhword	simple
numword	simple
sfloat	simple
uint	simple
url	simple
url_path	simple
version	simple
word	russian_stem

\dFd[+] [ШАБЛОН]

Список словарей текстового поиска (добавьте + для дополнительных сведений).

=> \dFd

List of text search dictionaries			
Schema	Name	Description	
pg_catalog	arabic_stem	snowball stemmer	for arabic language
pg_catalog	armenian_stem	snowball stemmer	for armenian language
pg_catalog	basque_stem	snowball stemmer	for basque language
pg_catalog	catalan_stem	snowball stemmer	for catalan language
pg_catalog	danish_stem	snowball stemmer	for danish language
pg_catalog	dutch_stem	snowball stemmer	for dutch language
pg_catalog	english_stem	snowball stemmer	for english language
pg_catalog	finnish_stem	snowball stemmer	for finnish language
pg_catalog	french_stem	snowball stemmer	for french language
pg_catalog	german_stem	snowball stemmer	for german language
pg_catalog	greek_stem	snowball stemmer	for greek language
pg_catalog	hindi_stem	snowball stemmer	for hindi language
pg_catalog	hungarian_stem	snowball stemmer	for hungarian language
pg_catalog	indonesian_stem	snowball stemmer	for indonesian language
pg_catalog	irish_stem	snowball stemmer	for irish language
pg_catalog	italian_stem	snowball stemmer	for italian language
pg_catalog	lithuanian_stem	snowball stemmer	for lithuanian language
pg_catalog	nepali_stem	snowball stemmer	for nepali language
pg_catalog	norwegian_stem	snowball stemmer	for norwegian language
pg_catalog	portuguese_stem	snowball stemmer	for portuguese language
pg_catalog	romanian_stem	snowball stemmer	for romanian language

pg_catalog	russian_stem	snowball stemmer for russian language
pg_catalog	serbian_stem	snowball stemmer for serbian language
pg_catalog	simple	simple dictionary: just lower case and check for stopword
pg_catalog	spanish_stem	snowball stemmer for spanish language
pg_catalog	swedish_stem	snowball stemmer for swedish language
pg_catalog	tamil_stem	snowball stemmer for tamil language
pg_catalog	turkish_stem	snowball stemmer for turkish language
pg_catalog	yiddish_stem	snowball stemmer for yiddish language

\dFp[+] [ШАБЛОН]

Список анализаторов текстового поиска (добавьте + для дополнительных сведений).

=> \dFp

List of text search parsers		
Schema	Name	Description
pg_catalog	default	default word parser

=> \dFp+

Text search parser "pg_catalog.default"		
Method	Function	Description
Start parse	prsd_start	
Get next token	prsd_nexttoken	
End parse	prsd_end	
Get headline	prsd_headline	
Get token types	prsd_lextype	

Token types for parser "pg_catalog.default"	
Token name	Description
asciihword	Hyphenated word, all ASCII
asciiword	Word, all ASCII
blank	Space symbols
email	Email address
entity	XML entity
file	File or path name
float	Decimal notation
host	Host
hword	Hyphenated word, all letters
hword_asciipart	Hyphenated word part, all ASCII
hword_numpart	Hyphenated word part, letters and digits
hword_part	Hyphenated word part, all letters
int	Signed integer
numhword	Hyphenated word, letters and digits
numword	Word, letters and digits
protocol	Protocol head
sfloat	Scientific notation
tag	XML tag
uint	Unsigned integer
url	URL
url_path	URL path
version	Version number
word	Word, all letters

(23 rows)

\dFt[+] [ШАБЛОН]

Список шаблонов текстового поиска (добавьте + для дополнительных сведений).

=> \dFt

List of text search templates		
Schema	Name	Description
pg_catalog	ispell	ispell dictionary
pg_catalog	simple	simple dictionary: just lower case and check for stopword
pg_catalog	snowball	snowball stemmer
pg_catalog	synonym	synonym dictionary: replace word by its synonym
pg_catalog	thesaurus	thesaurus dictionary: phrase by phrase substitution

12.11. Ограничения

Текущая реализация текстового поиска в Postgres Pro имеет следующие ограничения:

- Длина лексемы не может превышать 2 килобайта
- Длина значения `tsvector` (лексемы и их позиции) не может превышать 1 мегабайт
- Число лексем должно быть меньше 2^{64}
- Значения позиций в `tsvector` должны быть от 0 до 16383
- Расстояние в операторе `<N>` (ПРЕДШЕСТВУЕТ) типа `tsquery` не может быть больше 16384
- Не больше 256 позиций для одной лексемы
- Число узлов (лексемы + операторы) в значении `tsquery` должно быть меньше 32768

Для сравнения, документация PostgreSQL 8.1 содержала 335 420 слов, из них 10 441 уникальных, а наиболее часто употребляющееся в ней слово «postgresql» встречается 6 127 раз в 655 документах.

Другой пример — архивы списков рассылки PostgreSQL содержали 910 989 уникальных слов в 57 491 343 лексемах в 461 020 сообщениях.

Глава 13. Управление конкурентным доступом

В этой главе описывается поведение СУБД Postgres Pro в ситуациях, когда два или более сеансов пытаются одновременно обратиться к одним и тем же данным. В таких ситуациях важно, чтобы все сеансы могли эффективно работать с данными, и при этом сохранялась целостность данных. Обсуждаемые в этой главе темы заслуживают внимания всех разработчиков баз данных.

13.1. Введение

Postgres Pro предоставляет разработчикам богатый набор средств для управления конкурентным доступом к данным. Внутри он поддерживает целостность данных, реализуя модель MVCC (Multiversion Concurrency Control, Многоверсионное управление конкурентным доступом). Это означает, что каждый SQL-оператор видит снимок данных (*версию базы данных*) на определённый момент времени, вне зависимости от текущего состояния данных. Это защищает операторы от несогласованности данных, возможной, если другие конкурирующие транзакции внесут изменения в те же строки данных, и обеспечивает тем самым *изоляцию транзакций* для каждого сеанса баз данных. MVCC, отходя от методик блокирования, принятых в традиционных СУБД, снижает уровень конфликтов блокировок и таким образом обеспечивает более высокую производительность в многопользовательской среде.

Основное преимущество использования модели MVCC по сравнению с блокированием заключается в том, что блокировки MVCC, полученные для чтения данных, не конфликтуют с блокировками, полученными для записи, и поэтому чтение никогда не мешает записи, а запись чтению. Postgres Pro гарантирует это даже для самого строгого уровня изоляции транзакций, используя инновационный уровень изоляции SSI (*Serializable Snapshot Isolation, Сериализуемая изоляция снимков*).

Для приложений, которым в принципе не нужна полная изоляция транзакций и которые предпочитают явно определять точки конфликтов, в Postgres Pro также есть средства блокировки на уровне таблиц и строк. Однако при правильном использовании MVCC обычно обеспечивает лучшую производительность, чем блокировки. Кроме этого, приложения могут использовать рекомендательные блокировки, не привязанные к какой-либо одной транзакции.

13.2. Изоляция транзакций

Стандарт SQL определяет четыре уровня изоляции транзакций. Наиболее строгий из них — сериализуемый, определяется одним абзацем, говорящем, что при параллельном выполнении несколько сериализуемых транзакций должны гарантированно выдавать такой же результат, как если бы они запускались по очереди в некотором порядке. Остальные три уровня определяются через описания особых явлений, которые возможны при взаимодействии параллельных транзакций, но не допускаются на определённом уровне. Как отмечается в стандарте, из определения сериализуемого уровня вытекает, что на этом уровне ни одно из этих явлений не возможно. (В самом деле — если эффект транзакций должен быть тем же, что и при их выполнении по очереди, как можно было бы увидеть особые явления, связанные с другими транзакциями?)

Стандарт описывает следующие особые условия, недопустимые для различных уровней изоляции: «грязное» чтение

Транзакция читает данные, записанные параллельной незавершённой транзакцией.

неповторяемое чтение

Транзакция повторно читает те же данные, что и раньше, и обнаруживает, что они были изменены другой транзакцией (которая завершилась после первого чтения).

фантомное чтение

Транзакция повторно выполняет запрос, возвращающий набор строк для некоторого условия, и обнаруживает, что набор строк, удовлетворяющих условию, изменился из-за транзакции, завершившейся за это время.

аномалия сериализации

Результат успешной фиксации группы транзакций оказывается несогласованным при всевозможных вариантах исполнения этих транзакций по очереди.

Уровни изоляции транзакций, описанные в стандарте SQL и реализованные в Postgres Pro, описываются в [Таблице 13.1](#).

Таблица 13.1. Уровни изоляции транзакций

Уровень изоляции	«Грязное» чтение	Неповторяемое чтение	Фантомное чтение	Аномалия сериализации
Read uncommitted (Чтение незафиксированных данных)	Допускается, но не в PG	Возможно	Возможно	Возможно
Read committed (Чтение зафиксированных данных)	Невозможно	Возможно	Возможно	Возможно
Repeatable read (Повторяемое чтение)	Невозможно	Невозможно	Допускается, но не в PG	Возможно
Serializable (Сериализуемость)	Невозможно	Невозможно	Невозможно	Невозможно

В Postgres Pro вы можете запросить любой из четырёх уровней изоляции транзакций, однако внутри реализованы только три различных уровня, то есть режим Read Uncommitted в Postgres Pro действует как Read Committed. Причина этого в том, что только так можно сопоставить стандартные уровни изоляции с реализованной в Postgres Pro архитектурой многоверсионного управления конкурентным доступом.

В этой таблице также показано, что реализация Repeatable Read в Postgres Pro не допускает фантомного чтения. Это соответствует стандарту SQL, поскольку он устанавливает, каких аномалий *не* должно быть на определённых уровнях изоляции, но допускает и более строгие ограничения. Поведение имеющихся уровней изоляции подробно описывается в следующих подразделах.

Для выбора нужного уровня изоляции транзакций используется команда [SET TRANSACTION](#).

Важно

Поведение некоторых функций и типов данных Postgres Pro в транзакциях подчиняется особым правилам. В частности, изменения последовательностей (и следовательно, счётчика в столбце, объявленному как `serial`) немедленно видны во всех остальных транзакциях и не откатываются назад, если выполнившая их транзакция прерывается. См. [Раздел 9.17](#) и [Подраздел 8.1.4](#).

13.2.1. Уровень изоляции Read Committed

Read Committed — уровень изоляции транзакции, выбираемый в Postgres Pro по умолчанию. В транзакции, работающей на этом уровне, запрос `SELECT` (без предложения `FOR UPDATE/SHARE`) видит только те данные, которые были зафиксированы до начала запроса; он никогда не увидит незафиксированных данных или изменений, внесённых в процессе выполнения запроса параллельными транзакциями. По сути запрос `SELECT` видит снимок базы данных в момент начала выполнения запроса. Однако `SELECT` видит результаты изменений, внесённых ранее в этой же транзакции, даже если они ещё не зафиксированы. Также заметьте, что два последовательных оператора `SELECT`

могут видеть разные данные даже в рамках одной транзакции, если какие-то другие транзакции зафиксируют изменения после запуска первого `SELECT`, но до запуска второго.

Команды `UPDATE`, `DELETE`, `SELECT FOR UPDATE` и `SELECT FOR SHARE` ведут себя подобно `SELECT` при поиске целевых строк: они найдут только те целевые строки, которые были зафиксированы на момент начала команды. Однако к моменту, когда они будут найдены, эти целевые строки могут быть уже изменены (а также удалены или заблокированы) другой параллельной транзакцией. В этом случае запланированное изменение будет отложено до фиксирования или отката первой изменяющей данные транзакции (если она ещё выполняется). Если первая изменяющая транзакция откатывается, её результат отбрасывается и вторая изменяющая транзакция может продолжить изменение изначально полученной строки. Если первая транзакция зафиксировалась, но в результате удалила эту строку, вторая будет игнорировать её, а в противном случае попытается выполнить свою операцию с изменённой версией строки. Условие поиска в команде (предложение `WHERE`) вычисляется повторно для выяснения, соответствует ли по-прежнему этому условию изменённая версия строки. Если да, вторая изменяющая транзакция продолжат свою работу с изменённой версией строки. Применительно к командам `SELECT FOR UPDATE` и `SELECT FOR SHARE` это означает, что изменённая версия строки блокируется и возвращается клиенту.

Похожим образом ведёт себя `INSERT` с предложением `ON CONFLICT DO UPDATE`. В режиме `Read Committed` каждая строка, предлагаемая для добавления, будет либо вставлена, либо изменена. Если не возникнет несвязанных ошибок, гарантируется один из этих двух исходов. Если конфликт будет вызван другой транзакцией, результат которой ещё не видим для `INSERT`, предложение `UPDATE` подействует на эту строку, даже несмотря на то, что эта команда обычным образом может не видеть *никакую* версию этой строки.

При выполнении `INSERT` с предложением `ON CONFLICT DO NOTHING` строка может не добавиться в результате действия другой транзакции, эффект которой не виден в снимке команды `INSERT`. Это опять же имеет место только в режиме `Read Committed`.

`MERGE` позволяет пользователю указать различные комбинации подкоманд `INSERT`, `UPDATE` и `DELETE`. Команда `MERGE` с подкомандами `INSERT` и `UPDATE` работает аналогично `INSERT` с `ON CONFLICT DO UPDATE`, но не гарантирует, что какая-либо из команд `INSERT` или `UPDATE` будет выполнена. Если `MERGE` пытается выполнить `UPDATE` или `DELETE`, и строка одновременно изменяется, но условие соединения по-прежнему выполняется для текущего целевого и текущего исходного кортежа, то `MERGE` будет работать так же, как команды `UPDATE` или `DELETE`, и выполнит свои действия с изменённой версией строки. Однако поскольку в `MERGE` могут указываться несколько действий, и они могут быть условными, условия для всех действий заново вычисляются для изменённой версии строки, начиная с первого действия, даже если действие, которое первоначально оказалось подходящим, идёт в списке действий позже. С другой стороны, если строка одновременно изменяется или удаляется так, что условие соединения не выполняется, `MERGE` будет вычислять следующие условия действий `NOT MATCHED` и выполнит действие с первым подходящим условием. Если `MERGE` пытается выполнить `INSERT` при наличии уникального индекса, и одновременно добавляется повторяющаяся строка, возникает ошибка нарушения уникальности; `MERGE` не пытается избежать таких ошибок, заново вычисляя условия `MATCHED`.

Вследствие описанных выше правил, изменяющая команда может увидеть несогласованное состояние: она может видеть результаты параллельных команд, изменяющих те же строки, что пытается изменить она, но при этом она не видит результаты этих команд в других строках таблиц. Из-за этого поведения уровень `Read Committed` не подходит для команд со сложными условиями поиска; однако он вполне пригоден для простых случаев. Например, рассмотрим изменение баланса счёта в таких транзакциях:

```
BEGIN;  
UPDATE accounts SET balance = balance + 100.00 WHERE acctnum = 12345;  
UPDATE accounts SET balance = balance - 100.00 WHERE acctnum = 7534;  
COMMIT;
```

Если две такие транзакции пытаются параллельно изменить баланс счёта 12345, мы, естественно, хотим, чтобы вторая транзакция работала с изменённой версией строки счёта. Так как каждая

команда влияет только на определённую строку, если она будет видеть изменённую версию строки, это не приведёт к проблемам несогласованности.

В более сложных ситуациях уровень Read Committed может приводить к нежелательным результатам. Например, рассмотрим команду DELETE, работающую со строками, которые параллельно добавляет и удаляет из множества, определённого её условием, другая команда. Например, предположим, что website — таблица из двух строк, в которых website.hits равны 9 и 10:

```
BEGIN;  
UPDATE website SET hits = hits + 1;  
-- выполняется параллельно: DELETE FROM website WHERE hits = 10;  
COMMIT;
```

Команда DELETE не сделает ничего, даже несмотря на то, что строка с website.hits = 10 была в таблице и до, и после выполнения UPDATE. Это происходит потому, что строка со значением 9 до изменения пропускается, а когда команда UPDATE завершается и DELETE получает освободившуюся блокировку, строка с 10 теперь содержит 11, а это значение уже не соответствует условию.

Так как в режиме Read Committed каждая команда начинается с нового снимка состояния, который включает результаты всех транзакций, зафиксированных к этому моменту, последующие команды в одной транзакции будут в любом случае видеть эффекты всех параллельных зафиксированных транзакций. Вопрос здесь состоит в том, видит ли одна команда абсолютно согласованное состояние базы данных.

Частичная изоляция транзакций, обеспечиваемая в режиме Read Committed, приемлема для множества приложений. Этот режим быстр и прост в использовании, однако он подходит не для всех случаев. Приложениям, выполняющим сложные запросы и изменения, могут потребоваться более строго согласованное представление данных, чем то, что даёт Read Committed.

13.2.2. Уровень изоляции Repeatable Read

В режиме *Repeatable Read* видны только те данные, которые были зафиксированы до начала транзакции, но не видны незафиксированные данные и изменения, произведённые другими транзакциями в процессе выполнения данной транзакции. (Однако запрос будет видеть эффекты предыдущих изменений в своей транзакции, несмотря на то, что они не зафиксированы.) Это самое строгое требование, которое стандарт SQL вводит для этого уровня изоляции, и при его выполнении предотвращаются все явления, описанные в [Таблице 13.1](#), за исключением аномалий сериализации. Как было сказано выше, это не противоречит стандарту, так как он определяет только *минимальную* защиту, которая должна обеспечиваться на каждом уровне изоляции.

Этот уровень отличается от Read Committed тем, что запрос в транзакции данного уровня видит снимок данных на момент начала первого оператора в *транзакции* (не считая команд управления транзакциями), а не начала текущего оператора. Таким образом, последовательные команды SELECT в *одной* транзакции видят одни и те же данные; они не видят изменений, внесённых и зафиксированных другими транзакциями после начала их текущей транзакции.

Приложения, использующие этот уровень, должны быть готовы повторить транзакции в случае сбоев сериализации.

Команды UPDATE, DELETE, MERGE, SELECT FOR UPDATE и SELECT FOR SHARE ведут себя подобно SELECT при поиске целевых строк: они найдут только те целевые строки, которые были зафиксированы на момент начала транзакции. Однако к моменту, когда они будут найдены, эти целевые строки могут быть уже изменены (а также удалены или заблокированы) другой параллельной транзакцией. В этом случае транзакция в режиме Repeatable Read будет ожидать фиксирования или отката первой изменяющей данные транзакции (если она ещё выполняется). Если первая изменяющая транзакция откатывается, её результат отбрасывается и текущая транзакция может продолжить изменение изначально полученной строки. Если же первая транзакция зафиксировалась и в результате изменила или удалила эту строку, а не просто заблокировала её, произойдёт откат текущей транзакции с сообщением

ОШИБКА: не удалось сериализовать доступ из-за параллельного изменения

так как транзакция уровня Repeatable Read не может изменять или блокировать строки, изменённые другими транзакциями с момента её начала.

Когда приложение получает это сообщение об ошибке, оно должна прервать текущую транзакцию и попытаться повторить её с самого начала. Во второй раз транзакция увидит внесённое до этого изменение как часть начального снимка базы данных, так что новая версия строки вполне может использоваться в качестве отправной точки для изменения в повторной транзакции.

Заметьте, что потребность в повторении транзакции может возникнуть, только если эта транзакция изменяет данные; в транзакциях, которые только читают данные, конфликтов сериализации не бывает.

Режим Repeatable Read строго гарантирует, что каждая транзакция видит полностью стабильное представление базы данных. Однако это представление не обязательно будет согласовано с некоторым последовательным выполнением транзакций одного уровня. Например, даже транзакция, которая только читает данные, в этом режиме может видеть строку, показывающую, что некоторое задание завершено, но не видеть одну из строк логических частей задания, так как эта транзакция может прочитать более раннюю версию строки задания, чем ту, для которой параллельно добавлялась очередная логическая часть. Строго исполнить бизнес-правила в транзакциях, работающих на этом уровне изоляции, скорее всего не удастся без явных блокировок конфликтующих транзакций.

Для реализации уровня изоляции Repeatable Read применяется подход, который называется в академической литературе по базам данных и в других СУБД *Изоляция снимков* (Snapshot Isolation). По сравнению с системами, использующими традиционный метод блокировок, затрудняющий параллельное выполнение, при этом подходе наблюдается другое поведение и другая производительность. В некоторых СУБД могут существовать даже два отдельных уровня Repeatable Read и Snapshot Isolation с различным поведением. Допускаемые особые условия, представляющие отличия двух этих подходов, не были формализованы разработчиками теории БД до развития стандарта SQL и их рассмотрение выходит за рамки данного руководства. В полном объёме эта тема освещается в [berenson95](#).

Примечание

До версии 9.1 в PostgreSQL при запросе режима Serializable поведение системы в точности соответствовало вышеописанному. Таким образом, чтобы сейчас получить старое поведение Serializable, нужно запрашивать режим Repeatable Read.

13.2.3. Уровень изоляции Serializable

Уровень *Serializable* обеспечивает самую строгую изоляцию транзакций. На этом уровне моделируется последовательное выполнение всех зафиксированных транзакций, как если бы транзакции выполнялись одна за другой, последовательно, а не параллельно. Однако, как и на уровне Repeatable Read, на этом уровне приложения должны быть готовы повторять транзакции из-за сбоев сериализации. Фактически этот режим изоляции работает так же, как и Repeatable Read, только он дополнительно отслеживает условия, при которых результат параллельно выполняемых сериализуемых транзакций может не согласовываться с результатом этих же транзакций, выполняемых по очереди. Это отслеживание не приносит дополнительных препятствий для выполнения, кроме тех, что присущи режиму Repeatable Read, но тем не менее создаёт некоторую добавочную нагрузку, а при выявлении исключительных условий регистрируется *аномалия сериализации* и происходит *сбой сериализации*.

Например, рассмотрим таблицу `mytab`, изначально содержащую:

class	value
1	10
1	20

```
2 | 100
2 | 200
```

Предположим, что сериализуемая транзакция А вычисляет:

```
SELECT SUM(value) FROM mytab WHERE class = 1;
```

а затем вставляет результат (30) в поле `value` в новую строку со значением `class = 2`. В это же время сериализуемая транзакция В вычисляет:

```
SELECT SUM(value) FROM mytab WHERE class = 2;
```

получает результат 300 и вставляет его в новую строку со значением `class = 1`. Затем обе транзакции пытаются зафиксироваться. Если бы одна из этих транзакций работала в режиме `Repeatable Read`, зафиксироваться могли бы обе; но так как полученный результат не соответствовал бы последовательному порядку, в режиме `Serializable` будет зафиксирована только одна транзакция, а вторая закончится откатом с сообщением:

ОШИБКА: не удалось сериализовать доступ из-за зависимостей чтения/записи между транзакциями

Это объясняется тем, что при выполнении А перед В транзакция В вычислила бы сумму 330, а не 300, а при выполнении в обратном порядке А вычислила бы другую сумму.

Рассчитывая, что сериализуемые транзакции предотвратят аномалии, важно понимать, что любые данные, полученные из постоянной таблицы пользователя, не должны считаться действительными, пока транзакция, прочитавшая их, не будет успешно зафиксирована. Это верно даже для транзакций, не модифицирующих данные, за исключением случая, когда данные считываются в *откладываемой* транзакции такого типа. В этом случае данные могут считаться действительными, так как такая транзакция ждёт, пока не сможет получить снимок, гарантированно предотвращающий подобные проблемы. Во всех остальных случаях приложения не должны полагаться на результаты чтения данных в транзакции, которая не была зафиксирована; в случае ошибки и отката приложения должны повторять транзакцию, пока она не будет завершена успешно.

Для полной гарантии сериализуемости в Postgres Pro применяются *предикатные блокировки*, то есть блокировки, позволяющие определить, когда запись могла бы повлиять на результат предыдущего чтения параллельной транзакции, если бы эта запись выполнялась сначала. В Postgres Pro эти блокировки не приводят к фактическим блокировкам данных и, следовательно, никоим образом не могут повлечь взаимоблокировки транзакций. Они помогают выявить и отметить зависимости между параллельными транзакциями уровня `Serializable`, которые в определённых сочетаниях могут приводить к аномалиям сериализации. Транзакции `Read Committed` или `Repeatable Read` для обеспечения целостности данных, напротив, должны либо блокировать таблицы целиком, что мешает пользователям обращаться к этим таблицам, либо применять `SELECT FOR UPDATE` или `SELECT FOR SHARE`, что не только заблокирует другие транзакции, но и создаст дополнительную нагрузку на диск.

Предикатные блокировки в Postgres Pro, как и в большинстве других СУБД, устанавливаются для данных, фактически используемых в транзакции. Они отображаются в системном представлении `pg_locks` со значением `mode` равным `SIReadLock`. Какие именно блокировки будут затребованы при выполнении запроса, зависит от плана запроса, при этом детализированные блокировки (например, блокировки строк) могут объединяться в более общие (например, в блокировки страниц) в процессе транзакции для экономии памяти, расходуемой для отслеживания блокировок. Транзакция `READ ONLY` может даже освободить свои блокировки `SIRead` до завершения, если обнаруживается, что конфликты, которые могли бы привести к аномалии сериализации, исключены. На самом деле для транзакций `READ ONLY` этот факт чаще всего устанавливается в самом начале, так что они обходятся без предикатных блокировок. Если же вы явно запросите транзакцию `SERIALIZABLE READ ONLY DEFERRABLE`, она будет заблокирована до тех пор, пока не сможет установить этот факт. (Это *единственный* случай, когда транзакции уровня `Serializable` блокируются, а транзакции `Repeatable Read` — нет.) С другой стороны, блокировки `SIRead` часто должны сохраняться и после фиксирования транзакции, пока не будут завершены другие, наложившиеся на неё транзакции.

При правильном использовании сериализуемые транзакции могут значительно упростить разработку приложений. Гарантия того, что любое сочетание успешно зафиксированных параллельных

сериализуемых транзакций даст тот же результат, что и последовательность этих транзакций, выполненных по очереди, означает, что если вы уверены, что единственная транзакция определённого содержания работает правильно, когда она запускается отдельно, вы можете быть уверены, что она будет работать так же правильно в любом сочетании сериализуемых транзакций, вне зависимости от того, что они делают, либо же она не будет зафиксирована успешно. При этом важно, чтобы в среде, где применяется этот подход, была реализована общая обработка сбоев сериализации (которые можно определить по значению `SQLSTATE '40001'`), так как заведомо определить, какие именно транзакции могут стать жертвами зависимостей чтения/записи и не будут зафиксированы для предотвращения аномалий сериализации, обычно очень сложно. Отслеживание зависимостей чтения-записи неизбежно создаёт дополнительную нагрузку, как и перезапуск транзакций, не зафиксированных из-за сбоев сериализации, но если на другую чашу весов положить нагрузку и блокирование, связанные с применением явных блокировок и `SELECT FOR UPDATE` или `SELECT FOR SHARE`, использовать сериализуемые транзакции в ряде случаев окажется выгоднее.

Тогда как уровень изоляции транзакций `Serializable` в `Postgres Pro` позволяет фиксировать параллельные транзакции, только если есть уверенность, что тот же результат будет получен при последовательном их выполнении, он не всегда предотвращает ошибки, которые не возникли бы при действительно последовательном выполнении. В частности, можно столкнуться с нарушениями ограничений уникальности, вызванными наложением сериализуемых транзакций, даже после явной проверки отсутствия ключа перед добавлением его. Этого можно избежать, если все сериализуемые транзакции, добавляющие потенциально конфликтующие ключи, будут предварительно явно проверять, можно ли вставить ключ. Например, приложение, добавляющее новый ключ, может запрашивать его у пользователя и затем проверять, существует ли он, сначала пытаясь найти его, либо генерировать новый ключ, выбирая максимальное существующее значение и увеличивая его на один. Если некоторые сериализуемые транзакции добавляют новые ключи сразу, не следуя этому протоколу, возможны нарушения ограничений уникальности, даже когда они не наблюдались бы при последовательном выполнении этих транзакций.

Применяя сериализуемые транзакции для управления конкурентным доступом, примите к сведению следующие рекомендации:

- Объявляйте транзакции как `READ ONLY`, если это отражает их суть.
- Управляйте числом активных подключений, при необходимости используя пул соединений. Это всегда полезно для увеличения производительности, но особенно важно это в загруженной системе с сериализуемыми транзакциями.
- Закрывайте в одну транзакцию не больше команд, чем необходимо для обеспечения целостности.
- Не оставляйте соединения «простаивающими в транзакции» дольше, чем необходимо. Для автоматического отключения затянувшихся транзакций можно применить параметр конфигурации `idle_in_transaction_session_timeout`.
- Исключите явные блокировки, `SELECT FOR UPDATE` и `SELECT FOR SHARE` там, где они не нужны благодаря защите, автоматически предоставляемой сериализуемыми транзакциями.
- Когда система вынуждена объединять предикатные блокировки уровня страницы в одну предикатную блокировку уровня таблицы из-за нехватки памяти, может возрасти частота сбоев сериализации. Избежать этого можно, увеличив параметр `max_pred_locks_per_transaction`, `max_pred_locks_per_relation` и/или `max_pred_locks_per_page`.
- Последовательное сканирование всегда влечёт за собой предикатную блокировку на уровне таблицы. Это приводит к увеличению сбоев сериализации. В таких ситуациях бывает полезно склонить систему к использованию индексов, уменьшая `random_page_cost` и/или увеличивая `cpu_tuple_cost`. Однако тут важно сопоставить выигрыш от уменьшения числа откатов и перезапусков транзакций с проигрышем от возможного менее эффективного выполнения запросов.

Для реализации уровня изоляции `Serializable` применяется подход, который называется в академической литературе по базам данных *Изоляция снимков* (Snapshot Isolation), с дополнительными проверками на предмет аномалий сериализации. По сравнению с другими системами, использую-

щими традиционный метод блокировок, при этом подходе наблюдается другое поведение и другая производительность. Подробнее это освещается в [ports12](#).

13.3. Явные блокировки

Для управления параллельным доступом к данным в таблицах Postgres Pro предоставляет несколько режимов явных блокировок. Эти режимы могут применяться для блокировки данных со стороны приложения в ситуациях, когда MVCC не даёт желаемый результат. Кроме того, большинство команд Postgres Pro автоматически получают блокировки соответствующих режимов, защищающие от удаления или изменения задействованных таблиц, несовместимого с характером выполняемой команды. (Например, `TRUNCATE` не может безопасно выполняться одновременно с другими операциями с этой таблицей, так что во избежание конфликта эта команда получает блокировку `ACCESS EXCLUSIVE` для данной таблицы.)

Список текущих активных блокировок на сервере можно получить, прочитав системное представление `pg_locks`. За дополнительными сведениями о наблюдении за состоянием менеджера блокировок обратитесь к [Главе 26](#).

13.3.1. Блокировки на уровне таблицы

В приведённом ниже списке перечислены имеющиеся режимы блокировок и контексты, где их автоматически применяет Postgres Pro. Вы можете также явно запросить любую из этих блокировок с помощью команды `LOCK`. Помните, что все эти режимы работают на уровне таблицы, даже если имя режима содержит слово «row»; такие имена сложились исторически. В некоторой степени эти имена отражают типичное применение каждого режима блокировки, но смысл у всех один. Единственное, что действительно отличает один режим блокировки от другого, это набор режимов, с которыми конфликтует каждый из них (см. [Таблицу 13.2](#)). Две транзакции не могут одновременно владеть блокировками конфликтующих режимов для одной и той же таблицы. (Однако учтите, что транзакция никогда не конфликтует с собой. Например, она может запросить блокировку `ACCESS EXCLUSIVE`, а затем `ACCESS SHARE` для той же таблицы.) При этом разные транзакции свободно могут одновременно владеть блокировками неконфликтующих режимов. Заметьте, что некоторые режимы блокировки конфликтуют сами с собой (например, блокировкой `ACCESS EXCLUSIVE` в один момент времени может владеть только одна транзакция), а некоторые — нет (например, блокировку `ACCESS SHARE` могут получить сразу несколько транзакций).

Режимы блокировок на уровне таблицы

`ACCESS SHARE (AccessShareLock)`

Конфликтует только с режимом блокировки `ACCESS EXCLUSIVE`.

Команда `SELECT` получает такую блокировку для таблиц, на которые она ссылается. Вообще говоря, блокировку в этом режиме получает любой запрос, который только *читает* таблицу, но не меняет её данные.

`ROW SHARE (RowShareLock)`

Конфликтует с режимами блокировки `EXCLUSIVE` и `ACCESS EXCLUSIVE`.

Команда `SELECT` устанавливает блокировку в этом режиме для всех таблиц, для которых задано указание `FOR UPDATE`, `FOR NO KEY UPDATE`, `FOR SHARE` или `FOR KEY SHARE` (в дополнение к блокировкам `ACCESS SHARE` для любых других таблиц, задействованных без каких-либо явных указаний блокировки `FOR ...`).

`ROW EXCLUSIVE (RowExclusiveLock)`

Конфликтует с режимами блокировки `SHARE`, `SHARE ROW EXCLUSIVE`, `EXCLUSIVE` и `ACCESS EXCLUSIVE`.

Команды `UPDATE`, `DELETE`, `INSERT` и `MERGE` получают такую блокировку для целевой таблицы (в дополнение к блокировкам `ACCESS SHARE` для всех других задействованных таблиц). Вообще говоря, блокировку в этом режиме получает любая команда, которая *изменяет данные* в таблице.

SHARE UPDATE EXCLUSIVE (ShareUpdateExclusiveLock)

Конфликтует с режимами блокировки `SHARE UPDATE EXCLUSIVE`, `SHARE`, `SHARE ROW EXCLUSIVE`, `EXCLUSIVE` и `ACCESS EXCLUSIVE`. Этот режим защищает таблицу от параллельного изменения схемы и запуска процесса `VACUUM`.

Запрашивается командами `VACUUM` (без `FULL`), `ANALYZE`, `CREATE INDEX CONCURRENTLY`, `CREATE STATISTICS`, `COMMENT ON`, `REINDEX CONCURRENTLY` и некоторыми формами `ALTER INDEX` и `ALTER TABLE` (за подробностями обратитесь к описанию этих команд).

SHARE (ShareLock)

Конфликтует с режимами блокировки `ROW EXCLUSIVE`, `SHARE UPDATE EXCLUSIVE`, `SHARE ROW EXCLUSIVE`, `EXCLUSIVE` и `ACCESS EXCLUSIVE`. Этот режим защищает таблицу от параллельного изменения данных.

Запрашивается командой `CREATE INDEX` (без параметра `CONCURRENTLY`).

SHARE ROW EXCLUSIVE (ShareRowExclusiveLock)

Конфликтует с режимами блокировки `ROW EXCLUSIVE`, `SHARE UPDATE EXCLUSIVE`, `SHARE`, `SHARE ROW EXCLUSIVE`, `EXCLUSIVE` и `ACCESS EXCLUSIVE`. Этот режим защищает таблицу от параллельных изменений данных и при этом он является самоисключающим, так что такую блокировку может получить только один сеанс.

Запрашивается командой `CREATE TRIGGER` и некоторыми формами `ALTER TABLE`.

EXCLUSIVE (ExclusiveLock)

Конфликтует с режимами блокировки `ROW SHARE`, `ROW EXCLUSIVE`, `SHARE UPDATE EXCLUSIVE`, `SHARE`, `SHARE ROW EXCLUSIVE`, `EXCLUSIVE` и `ACCESS EXCLUSIVE`. Этот режим совместим только с блокировкой `ACCESS SHARE`, то есть параллельно с транзакцией, получившей блокировку в этом режиме, допускается только чтение таблицы.

Запрашивается командой `REFRESH MATERIALIZED VIEW CONCURRENTLY`.

ACCESS EXCLUSIVE (AccessExclusiveLock)

Конфликтует с режимами блокировок `ACCESS SHARE`, `ROW SHARE`, `ROW EXCLUSIVE`, `SHARE UPDATE EXCLUSIVE`, `SHARE`, `SHARE ROW EXCLUSIVE`, `EXCLUSIVE` и `ACCESS EXCLUSIVE`. Этот режим гарантирует, что кроме транзакции, получившей эту блокировку, никакая другая транзакция не может обращаться к таблице каким-либо способом.

Запрашивается командами `DROP TABLE`, `TRUNCATE`, `REINDEX`, `CLUSTER`, `VACUUM FULL` и `REFRESH MATERIALIZED VIEW` (без `CONCURRENTLY`). Блокировку на этом уровне запрашивают также многие виды `ALTER INDEX` и `ALTER TABLE`. В этом режиме по умолчанию запрашивают блокировку и операторы `LOCK TABLE`, если явно не выбран другой режим.

Подсказка

Только блокировка `ACCESS EXCLUSIVE` блокирует оператор `SELECT` (без `FOR UPDATE/SHARE`).

Кроме того, для поддержки системы 1С:Предприятие реализованы ещё два режима блокировок. Эти режимы не конфликтуют ни с какими режимами, описанными выше. Их можно использовать, но лучше вместо них применять **рекомендательные блокировки**, которые предоставляют ту же функциональность.

APPLICATION SHARE

Конфликтует только с режимом блокировки `APPLICATION EXCLUSIVE`.

APPLICATION EXCLUSIVE

Конфликтует с режимами блокировок APPLICATION SHARE и APPLICATION EXCLUSIVE.

Полученная транзакцией блокировка обычно сохраняется до конца транзакции. Но если блокировка получена после установки точки сохранения, она освобождается немедленно в случае отката к этой точке. Это согласуется с принципом действия ROLLBACK — эта команда отменяет эффекты всех команд после точки сохранения. То же справедливо и для блокировок, полученных в блоке исключений PL/pgSQL: при выходе из блока с ошибкой такие блокировки освобождаются.

Таблица 13.2. Конфликтующие режимы блокировки

Запрашиваемый режим блокировки	Существующий режим блокировки									
	ACCESS SHARE	ROW SHARE	ROW EXCL.	SHARE UPDATE EXCL.	SHARE	SHARE ROW EXCL.	EXCL.	ACCESS EXCL.	APP. SHARE	APP. EXCL.
ACCESS SHARE								X		
ROW SHARE							X	X		
ROW EXCL.					X	X	X	X		
SHARE UPDATE EXCL.				X	X	X	X	X		
SHARE			X	X		X	X	X		
SHARE ROW EXCL.			X	X	X	X	X	X		
EXCL.		X	X	X	X	X	X	X		
ACCESS EXCL.	X	X	X	X	X	X	X	X		
APP. SHARE										X
APP. EXCL.									X	X

13.3.2. Блокировки на уровне строк

Помимо блокировок на уровне таблицы, существуют блокировки на уровне строк, перечисленные ниже с контекстами, где Postgres Pro применяет их по умолчанию. Полный перечень конфликтов блокировок на уровне строк приведён в [Таблице 13.3](#). Заметьте, что одна транзакция может владеть несколькими конфликтующими блокировками одной строки, даже в разных подтранзакциях; но две разных транзакции никогда не получают конфликтующие блокировки одной и той же строки. Блокировки на уровне строк блокируют только запись в определённые строки, но никак не влияют на выборку. Снимаются такие блокировки, как и блокировки на уровне таблицы, в конце транзакции или при откате к точке сохранения.

Режимы блокировки на уровне строк

FOR UPDATE

В режиме FOR UPDATE строки, выданные оператором SELECT, блокируются как для изменения. При этом они защищаются от блокировки, изменения и удаления другими транзакциями до завершения текущей. То есть другие транзакции, пытающиеся выполнить UPDATE, DELETE, SELECT

FOR UPDATE, SELECT FOR NO KEY UPDATE, SELECT FOR SHARE или SELECT FOR KEY SHARE с этими строками, будут заблокированы до завершения текущей транзакции; и наоборот, команда SELECT FOR UPDATE будет ожидать окончания параллельной транзакции, в которой выполнялась одна из этих команд с той же строкой, а затем установит блокировку и вернёт изменённую строку (или не вернёт, если она была удалена). Однако в транзакции REPEATABLE READ или SERIALIZABLE возникнет ошибка, если блокируемая строка изменилась с момента начала транзакции. Подробнее это обсуждается в [Разделе 13.4](#).

Режим блокировки FOR UPDATE также запрашивается на уровне строки любой командой DELETE и командой UPDATE, изменяющей значения определённых столбцов. В настоящее время блокировка с UPDATE касается столбцов, по которым создан уникальный индекс, применимый в качестве внешнего ключа (так что на частичные индексы и индексы выражений это не распространяется), но в будущем это может поменяться.

FOR NO KEY UPDATE

Действует подобно FOR UPDATE, но запрашиваемая в этом режиме блокировка слабее: она не будет блокировать команды SELECT FOR KEY SHARE, пытающиеся получить блокировку тех же строк. Этот режим блокировки также запрашивается любой командой UPDATE, которая не требует блокировки FOR UPDATE.

FOR SHARE

Действует подобно FOR NO KEY UPDATE, за исключением того, что для каждой из полученных строк запрашивается разделяемая, а не исключительная блокировка. Разделяемая блокировка не позволяет другим транзакциям выполнять с этими строками UPDATE, DELETE, SELECT FOR UPDATE или SELECT FOR NO KEY UPDATE, но допускает SELECT FOR SHARE и SELECT FOR KEY SHARE.

FOR KEY SHARE

Действует подобно FOR SHARE, но устанавливает более слабую блокировку: блокируется SELECT FOR UPDATE, но не SELECT FOR NO KEY UPDATE. Блокировка разделяемого ключа не позволяет другим транзакциям выполнять команды DELETE и UPDATE, только если они меняют значение ключа (но не другие UPDATE), и при этом допускает выполнение команд SELECT FOR NO KEY UPDATE, SELECT FOR SHARE и SELECT FOR KEY SHARE.

Postgres Pro не держит информацию об изменённых строках в памяти, так что никаких ограничений на число блокируемых строк нет. Однако блокировка строки может повлечь запись на диск, например, если SELECT FOR UPDATE изменяет выбранные строки, чтобы заблокировать их, при этом происходит запись на диск.

Таблица 13.3. Конфликтующие блокировки на уровне строк

Запрашиваемый режим блокировки	Текущий режим блокировки			
	FOR KEY SHARE	FOR SHARE	FOR NO KEY UPDATE	FOR UPDATE
FOR KEY SHARE				X
FOR SHARE			X	X
FOR NO KEY UPDATE		X	X	X
FOR UPDATE	X	X	X	X

13.3.3. Блокировки на уровне страниц

В дополнение к блокировкам на уровне таблицы и строк, для управления доступом к страницам таблиц в общих буферах используются блокировки на уровне страниц, исключительные и разделяемые. Эти блокировки освобождаются немедленно после выборки или изменения строк. Разработчикам приложений обычно можно не задумываться о блокировках страниц, здесь они упоминаются только для полноты картины.

13.3.4. Взаимоблокировки

Частое применение явных блокировок может увеличить вероятность *взаимоблокировок*, то есть ситуаций, когда две (или более) транзакций держат блокировки так, что взаимно блокируют друг друга. Например, если транзакция 1 получает исключительную блокировку таблицы А, а затем пытается получить исключительную блокировку таблицы В, которую до этого получила транзакция 2, в данный момент требующая исключительную блокировку таблицы А, ни одна из транзакций не сможет продолжить работу. Postgres Pro автоматически выявляет такие ситуации и разрешает их, прерывая одну из сцепившихся транзакций и тем самым позволяя другой (другим) продолжить работу. (Какая именно транзакция будет прервана, обычно сложно предсказать, так что рассчитывать на определённое поведение не следует.)

Заметьте, что взаимоблокировки могут вызываться и блокировками на уровне строк (таким образом, они возможны, даже если не применяются явные блокировки). Рассмотрим случай, когда две параллельных транзакции изменяют таблицу. Первая транзакция выполняет:

```
UPDATE accounts SET balance = balance + 100.00 WHERE acctnum = 11111;
```

При этом она получает блокировку строки с указанным номером счёта. Затем вторая транзакция выполняет:

```
UPDATE accounts SET balance = balance + 100.00 WHERE acctnum = 22222;  
UPDATE accounts SET balance = balance - 100.00 WHERE acctnum = 11111;
```

Первый оператор UPDATE успешно получает блокировку указанной строки и изменяет данные в ней. Однако второй оператор UPDATE обнаруживает, что строка, которую он пытается изменить, уже заблокирована, так что он ждёт завершения транзакции, получившей блокировку. Таким образом, вторая транзакция сможет продолжиться только после завершения первой. Теперь первая транзакция выполняет:

```
UPDATE accounts SET balance = balance - 100.00 WHERE acctnum = 22222;
```

Первая транзакция пытается получить блокировку заданной строки, но ей это не удаётся: эта блокировка уже принадлежит второй транзакции. Поэтому первой транзакции остаётся только ждать завершения второй. В результате первая транзакция блокируется второй, а вторая — первой: происходит взаимоблокировка. Postgres Pro выявляет эту ситуацию и прерывает одну из транзакций.

Обычно лучший способ предотвращения взаимоблокировок — добиться, чтобы все приложения, обращающиеся к базе данных, запрашивали блокировки нескольких объектов единообразно. В данном примере, если бы обе транзакции изменяли строки в одном порядке, взаимоблокировка бы не произошла. Блокировки в транзакции следует упорядочивать так, чтобы первой для какого-либо объекта запрашивалась наиболее ограничивающая из тех, которые для него потребуются. Если заранее обеспечить такой порядок нельзя, взаимоблокировки можно обработать по факту, повторяя прерванные транзакции.

Если ситуация взаимоблокировки не будет выявлена, транзакция, ожидающая блокировки на уровне таблицы или строки, будет ждать её освобождения неограниченное время. Это означает, что приложения не должны оставлять транзакции открытыми долгое время (например, ожидая ввода пользователя).

13.3.5. Рекомендательные блокировки

Postgres Pro также имеет средства создания блокировок, смысл которых определяют сами приложения. Такие блокировки называются *рекомендательными*, так как система не форсирует их использование — правильно их использовать должно само приложение. Рекомендательные блокировки бывают полезны для реализаций стратегий блокирования, плохо вписывающихся в модель MVCC. Например, рекомендательные блокировки часто применяются для исполнения стратегии пессимистичной блокировки, типичной для систем управления данными «плоский файл». Хотя для этого можно использовать и дополнительные флаги в таблицах, рекомендательные блокировки работают быстрее, не приводят к раздуванию таблиц и автоматически ликвидируются сервером в конце сеанса.

В Postgres Pro есть два варианта получить рекомендательные блокировки: на уровне сеанса и на уровне транзакции. Рекомендательная блокировка, полученная на уровне сеанса, удерживается, пока она не будет явно освобождена, или до конца сеанса. В отличие от стандартных рекомендательные блокировки уровня сеанса нарушают логику транзакций — блокировка, полученная в транзакции, даже если произойдёт откат этой транзакции, будет сохраняться в сеансе; аналогично, освобождение блокировки остаётся в силе, даже если транзакция, в которой оно было выполнено, позже прерывается. Вызывающий процесс может запросить блокировку несколько раз; при этом каждому запросу блокировки должен соответствовать запрос освобождения, чтобы она была действительно освобождена. Рекомендательные блокировки на уровне транзакций, напротив, во многом похожи на обычные блокировки: они автоматически освобождаются в конце транзакций и не требуют явного освобождения. Для кратковременного применения блокировок это поведение часто более уместно, чем поведение рекомендательных блокировок на уровне сеанса. Запросы рекомендательных блокировок одного идентификатора на уровне сеанса и на уровне транзакции будут блокировать друг друга вполне предсказуемым образом. Если сеанс уже владеет данной рекомендательной блокировкой, дополнительные запросы её в том же сеансе будут всегда успешны, даже если её ожидают другие сеансы. Это утверждение справедливо вне зависимости от того, на каком уровне (сеанса или транзакции) установлены или запрашиваются новые блокировки.

Как и остальные блокировки в Postgres Pro, все рекомендательные блокировки, связанные с любыми сеансами, можно просмотреть в системном представлении `pg_locks`.

И рекомендательные, и обычные блокировки сохраняются в области общей памяти, размер которой определяется параметрами конфигурации `max_locks_per_transaction` и `max_connections`. Важно, чтобы этой памяти было достаточно, так как в противном случае сервер не сможет выдать никакую блокировку. Таким образом, число рекомендательных блокировок, которые может выдать сервер, ограничивается обычно десятками или сотнями тысяч в зависимости от конфигурации сервера.

В определённых случаях при использовании рекомендательных блокировок, особенно в запросах с явными указаниями `ORDER BY` и `LIMIT`, важно учитывать, что получаемые блокировки могут зависеть от порядка вычисления SQL-выражений. Например:

```
SELECT pg_advisory_lock(id) FROM foo WHERE id = 12345; -- ok
SELECT pg_advisory_lock(id) FROM foo WHERE id > 12345 LIMIT 100; -- опасно!
SELECT pg_advisory_lock(q.id) FROM
(
  SELECT id FROM foo WHERE id > 12345 LIMIT 100
) q; -- ok
```

В этом примере второй вариант опасен, так как `LIMIT` не обязательно будет применяться перед вызовом функции блокировки. В результате приложение может получить блокировки, на которые оно не рассчитывает и которые оно не сможет освободить (до завершения сеанса). С точки зрения приложения такие блокировки окажутся в подвешенном состоянии, хотя они и будут отображаться в `pg_locks`.

Функции, предназначенные для работы с рекомендательными блокировками, описаны в [Подразделе 9.27.10](#).

13.4. Проверки целостности данных на уровне приложения

Используя транзакции `Read Committed`, очень сложно обеспечить целостность данных с точки зрения бизнес-логики, так как представление данных смещается с каждым оператором и даже один оператор может не ограничиваться своим снимком состояния в случае конфликта записи.

Хотя транзакция `Repeatable Read` получает стабильное представление данных в процессе выполнения, с использованием снимков MVCC для проверки целостности данных всё же связаны тонкие моменты, включая так называемые *конфликты чтения/записи*. Если одна транзакция записывает данные, а другая в это же время пытается их прочитать (до или после записи), она не может

увидеть результат работы первой. В таком случае создаётся впечатление, что читающая транзакция выполняется первой вне зависимости от того, какая из них была начата или зафиксирована раньше. Если этим всё и ограничивается, нет никаких проблем, но если читающая транзакция также пишет данные, которые читает параллельная транзакция, получается, что теперь эта транзакция будет исполняться, как будто она запущена перед другими вышеупомянутыми. Если же транзакция, которая должна исполняться как последняя, на самом деле зафиксирована первой, в графе упорядоченных транзакций легко может возникнуть цикл. И когда он возникает, проверки целостности не будут работать правильно без дополнительных мер.

Как было сказано в [Подразделе 13.2.3](#), сериализуемые транзакции представляют собой те же транзакции Repeatable Read, но дополненные неблокирующим механизмом отслеживания опасных условий конфликтов чтения/записи. Когда выявляется условие, приводящее к циклу в порядке транзакций, одна из этих транзакций откатывается и этот цикл таким образом разрывается.

13.4.1. Обеспечение согласованности в сериализуемых транзакциях

Если для всех операций чтения и записи, нуждающихся в согласованном представлении данных, используются транзакции уровня изоляции Serializable, это обеспечивает необходимую согласованность без дополнительных усилий. Приложения из других окружений, применяющие сериализуемые транзакции для обеспечения целостности, в Postgres Pro в этом смысле будут «просто работать».

Применение этого подхода избавляет программистов приложений от лишних сложностей, если приложение использует инфраструктуру, которая автоматически повторяет транзакции в случае отката из-за сбоев сериализации. Возможно, `serializable` стоит даже установить в качестве уровня изоляции по умолчанию (`default_transaction_isolation`). Также имеет смысл принять меры для предотвращения использования других уровней изоляции, непреднамеренного или с целью обойти проверки целостности, например проверять уровень изоляции в триггерах.

Рекомендации по увеличению быстродействия приведены в [Подразделе 13.2.3](#).

Предупреждение

Защита целостности с применением сериализуемых транзакций пока ещё не поддерживается в режиме горячего резерва ([Раздел 25.4](#)). Поэтому там, где применяется горячий резерв, следует использовать уровень Repeatable Read и явные блокировки на главном сервере.

13.4.2. Применение явных блокировок для обеспечения согласованности

Когда возможны несериализуемые операции записи, для обеспечения целостности строк и защиты от одновременных изменений, следует использовать `SELECT FOR UPDATE`, `SELECT FOR SHARE` или соответствующий оператор `LOCK TABLE`. (`SELECT FOR UPDATE` и `SELECT FOR SHARE` защищают от параллельных изменений только возвращаемые строки, тогда как `LOCK TABLE` блокирует всю таблицу.) Это следует учитывать, перенося в Postgres Pro приложения из других СУБД.

Мигрируя в Postgres Pro из других СУБД также следует учитывать, что команда `SELECT FOR UPDATE` сама по себе не гарантирует, что параллельная транзакция не изменит или не удалит выбранную строку. Для получения такой гарантии в Postgres Pro нужно именно изменить эту строку, даже если никакие значения в ней менять не требуется. `SELECT FOR UPDATE` временно блокирует другие транзакции, не давая им получить ту же блокировку или выполнить команды `UPDATE` или `DELETE`, которые бы повлияли на заблокированную строку, но как только транзакция, владеющая этой блокировкой, фиксируется или откатывается, заблокированная транзакция сможет выполнить конфликтующую операцию, если только для данной строки действительно не был выполнен `UPDATE`, пока транзакция владела блокировкой.

Реализация глобальной целостности с использованием несериализуемых транзакций MVCC требует более вдумчивого подхода. Например, банковскому приложению может потребоваться проверить, равняется ли сумма всех расходов в одной таблице сумме приходов в другой, при том, что обе таблицы активно изменяются. Просто сравнивать результаты двух успешных последовательных команд `SELECT sum(...)` в режиме `Read Committed` нельзя, так как вторая команда может захватить результаты транзакций, пропущенных первой. Подсчитывая суммы в одной транзакции `Repeatable Read`, можно получить точную картину только для транзакций, которые были зафиксированы до начала данной, но при этом может возникнуть законный вопрос — будет ли этот результат актуален тогда, когда он будет выдан. Если транзакция `Repeatable Read` сама вносит какие-то изменения, прежде чем проверять равенство сумм, полезность этой проверки становится ещё более сомнительной, так как при проверке будут учитываться некоторые, но не все изменения, произошедшие после начала транзакции. В таких случаях предусмотрительный разработчик может заблокировать все таблицы, задействованные в проверке, чтобы получить картину действительности, не вызывающую сомнений. Для этого применяется блокировка `SHARE` (или более строгая), которая гарантирует, что в заблокированной таблице не будет незафиксированных изменений, за исключением тех, что внесла текущая транзакция.

Также заметьте, что, применяя явные блокировки для предотвращения параллельных операций записи, следует использовать либо режим `Read Committed`, либо в режиме `Repeatable Read` обязательно получать блокировки прежде, чем выполнять запросы. Блокировка, получаемая транзакцией `Repeatable Read`, гарантирует, что никакая другая транзакция, изменяющая таблицу, не выполняется, но если снимок состояния, полученный транзакцией, предшествует блокировке, он может не включать на данный момент уже зафиксированные изменения. Снимок состояния в транзакции `Repeatable Read` создаётся фактически на момент начала первой команды выборки или изменения данных (`SELECT`, `INSERT`, `UPDATE`, `DELETE` или `MERGE`), так что получить явные блокировки можно до того, как он будет сформирован.

13.5. Обработка сбоев сериализации

На уровнях изоляции `Repeatable Read` и `Serializable` могут выдаваться ошибки с целью предотвращения аномалий сериализации. Как отмечалось ранее, приложения, использующие эти уровни, должны быть готовы повторить транзакции, завершившиеся сбоем из-за ошибок сериализации. Текст такого сообщения об ошибке может варьироваться в зависимости от конкретных обстоятельств, но код `SQLSTATE` всегда будет `40001` (`serialization_failure`).

Также может иметь смысл повторять запросы в случае ошибок взаимоблокировки. Такие ошибки имеют код `SQLSTATE` `40P01` (`deadlock_detected`).

В некоторых случаях также целесообразно повторять запросы после ошибок уникальности ключа, которые имеют код `SQLSTATE` `23505` (`unique_violation`), и ошибок нарушения исключения, которые имеют код `SQLSTATE` `23P01` (`exclusion_violation`). Например, если приложение выбирает новое значение для столбца первичного ключа, предварительно выяснив, какие значения уже есть, оно может столкнуться с ошибкой уникальности ключа, потому что другой экземпляр этого же приложения одновременно выбрал то же значение. По сути, это ошибка сериализации, но сервер не обнаруживает её как таковую, потому что он не может «видеть» связь между добавляемым значением и предыдущим чтением. Есть также некоторые особые случаи, когда сервер выдаёт ошибку уникального ключа или нарушения исключения, даже если у него в принципе достаточно информации, чтобы определить, что основной причиной является проблема сериализации. Тогда как запросы, вызвавшие ошибки `serialization_failure`, рекомендуется просто повторять, это не распространяется на запросы, завершающиеся ошибками с другими кодами, поскольку такие ошибки могут свидетельствовать о постоянных проблемах, а не о временных сбоях.

Повторять транзакцию важно целиком, включая всю логику, которая решает, какой SQL выдавать и/или какие значения использовать. `Postgres Pro` не предлагает средства автоматического повторения запросов, так как не может гарантировать их правильность.

Повторение транзакции не гарантирует, что такая транзакция будет завершена успешно; может потребоваться несколько повторных попыток. Когда уровень конкуренции очень высок, для успешного выполнения транзакции может потребоваться много попыток. В случаях, когда конфликт свя-

зан с подготовленной транзакцией, разрешить конфликт может быть невозможно, пока подготовленная транзакция не будет зафиксирована или отменена.

13.6. Ограничения

Некоторые команды DDL, в настоящее время это `TRUNCATE` и формы `ALTER TABLE`, перезаписывающие таблицу, не являются безопасными с точки зрения MVCC. Это значит, что после фиксации усечения или перезаписи таблица окажется пустой для всех параллельных транзакций, если они работают со снимком, полученным перед фиксацией такой команды DDL. Это может проявиться только в транзакции, которая не обращалась к таблице до момента начала команды DDL — любая транзакция, которая обращалась к ней раньше, получила бы как минимум блокировку `ACCESS SHARE`, которая заблокировала бы эту команду DDL до завершения транзакции. Поэтому такие команды не приводят ни к каким видимым несоответствиям с содержимым таблицы при последовательных запросах к целевой таблице, хотя возможно видимое несоответствие между содержимым целевой таблицы и другими таблицами в базе данных.

Поддержка уровня изоляции `Serializable` ещё не реализована для целевых серверов горячего резерва (они описываются в [Разделе 25.4](#)). На данный момент самый строгий уровень изоляции, поддерживаемый в режиме горячего резерва, это `Repeatable Read`. Хотя и тогда, когда главный сервер выполняет запись в транзакциях `Serializable`, все резервные серверы в итоге достигают согласованного состояния, но транзакция `Repeatable Read` на резервном сервере иногда может увидеть промежуточное состояние, не соответствующее результату последовательного выполнения транзакций на главном сервере.

Внутренние обращения к системным каталогам осуществляются за рамками уровня изоляции текущей транзакции. Это означает, что создаваемые объекты базы данных, например таблицы, оказываются видимыми для параллельных транзакций уровня `Repeatable Read` и `Serializable`, несмотря на то, что строки в них не видны. С другой стороны, запросы, обращающиеся к системным каталогам напрямую, не будут видеть строки, представляющие недавно созданные объекты базы данных, если эти запросы используют более высокие уровни изоляции.

13.7. Блокировки и индексы

Хотя Postgres Pro обеспечивает неблокирующий доступ на чтение/запись к данным таблиц, для индексов в настоящий момент это поддерживается не в полной мере. Postgres Pro управляет доступом к различным типам индексов следующим образом:

Индексы типа B-дерево, GiST и SP-GiST

Для управления чтением/записью используются кратковременные блокировки на уровне страницы, исключительные и разделяемые. Блокировки освобождаются сразу после извлечения или добавления строки индекса. Эти типы индексов обеспечивают максимальное распараллеливание операций, не допуская взаимоблокировок.

Хеш-индексы

Для управления чтением/записью используются блокировки на уровне групп хеша. Блокировки освобождаются после обработки всей группы. Такие блокировки с точки зрения распараллеливания лучше, чем блокировки на уровне индекса, но не исключают взаимоблокировок, так как они сохраняются дольше, чем выполняется одна операция с индексом.

Индексы GIN

Для управления чтением/записью используются кратковременные блокировки на уровне страницы, исключительные и разделяемые. Блокировки освобождаются сразу после извлечения или добавления строки индекса. Но заметьте, что добавление значения в поле с GIN-индексом обычно влечёт добавление нескольких ключей индекса, так что GIN может проделывать целый ряд операций для одного значения.

В настоящее время в многопоточной среде наиболее производительны индексы-B-дерева; и так как они более функциональны, чем хеш-индексы, их рекомендуется использовать в такой среде

для приложений, когда нужно индексировать скалярные данные. Если же нужно индексировать не скалярные данные, B-деревья не подходят, и вместо них следует использовать индексы GiST, SP-GiST или GIN.

Глава 14. Оптимизация производительности

Быстродействие запросов зависит от многих факторов. На некоторые из них могут воздействовать пользователи, а другие являются фундаментальными особенностями системы. В этой главе приводятся полезные советы, которые помогут понять их и оптимизировать производительность Postgres Pro.

14.1. Использование EXPLAIN

Выполняя любой полученный запрос, Postgres Pro разрабатывает для него *план запроса*. Выбор правильного плана, соответствующего структуре запроса и характеристикам данным, крайне важен для хорошей производительности, поэтому в системе работает сложный *планировщик*, задача которого — подобрать хороший план. Узнать, какой план был выбран для какого-либо запроса, можно с помощью команды `EXPLAIN`. Понимание плана — это искусство, и чтобы овладеть им, нужен определённый опыт, но этот раздел расскажет о самых простых вещах.

Приведённые ниже примеры показаны на тестовой базе данных, которая создаётся для выявления регрессий в исходных кодах PostgreSQL текущей версии. Для неё предварительно выполняется `VACUUM ANALYZE`. Вы должны получить похожие результаты, если возьмёте ту же базу данных и проделаете следующие действия, но примерная стоимость и ожидаемое число строк у вас может немного отличаться из-за того, что статистика команды `ANALYZE` рассчитывается по случайной выборке, а оценки стоимости зависят от конкретной платформы.

В этих примерах используется текстовый формат вывода `EXPLAIN`, принятый по умолчанию, как более компактный и удобный для восприятия человеком. Если вывод `EXPLAIN` нужно передать какой-либо программе для дальнейшего анализа, лучше использовать один из машинно-ориентированных форматов (XML, JSON или YAML).

14.1.1. Азы EXPLAIN

Структура плана запроса представляет собой дерево *узлов плана*. Узлы на нижнем уровне дерева — это узлы сканирования, которые возвращают необработанные данные таблицы. Разным типам доступа к таблице соответствуют разные узлы: последовательное сканирование, сканирование индекса и сканирование битовой карты. Источниками строк могут быть не только таблицы, но и например, предложения `VALUES` и функции, возвращающие множества `FROM`, и они представляются отдельными типами узлов сканирования. Если запрос требует объединения, агрегатных вычислений, сортировки или других операций с исходными строками, над узлами сканирования появляются узлы, обозначающие эти операции. И так как обычно операции могут выполняться разными способами, на этом уровне тоже могут быть узлы разных типов. В выводе команды `EXPLAIN` для каждого узла в дереве плана отводится одна строка, где показывается базовый тип узла плюс оценка стоимости выполнения данного узла, которую сделал для него планировщик. Если для узла выводятся дополнительные свойства, в вывод могут добавляться дополнительные строки, с отступом от основной информации узла. В самой первой строке (основной строке самого верхнего узла) выводится общая стоимость выполнения для всего плана; именно это значение планировщик старается минимизировать.

Взгляните на следующий простейший пример, просто иллюстрирующий формат вывода:

```
EXPLAIN SELECT * FROM tenk1;
```

```
QUERY PLAN
```

```
-----  
Seq Scan on tenk1  (cost=0.00..458.00 rows=10000 width=244)
```

Этот запрос не содержит предложения `WHERE`, поэтому он должен просканировать все строки таблицы, так что планировщик выбрал план простого последовательного сканирования. Числа, перечисленные в скобках (слева направо), имеют следующий смысл:

- Приблизительная стоимость запуска. Это время, которое проходит, прежде чем начнётся этап вывода данных, например для сортирующего узла это время сортировки.
- Приблизительная общая стоимость. Она вычисляется в предположении, что узел плана выполняется до конца, то есть возвращает все доступные строки. На практике родительский узел может досрочно прекратить чтение строк дочернего (см. приведённый ниже пример с `LIMIT`).
- Ожидаемое число строк, которое должен вывести этот узел плана. При этом так же предполагается, что узел выполняется до конца.
- Ожидаемый средний размер строк, выводимых этим узлом плана (в байтах).

Стоимость может измеряться в произвольных единицах, определяемых параметрами планировщика (см. [Подраздел 18.7.2](#)). Традиционно единицей стоимости считается операция чтения страницы с диска; то есть `seq_page_cost` обычно равен 1.0, а другие параметры задаются относительно него. Примеры в этом разделе выполняются со стандартными параметрами стоимости.

Важно понимать, что стоимость узла верхнего уровня включает стоимость всех его потомков. Также важно осознавать, что эта стоимость отражает только те факторы, которые учитывает планировщик. В частности, она не зависит от времени, необходимого для передачи результирующих строк клиенту, хотя оно может составлять значительную часть общего времени выполнения запроса. Тем не менее планировщик игнорирует эту величину, так как он всё равно не сможет изменить её, выбрав другой план. (Мы верим в то, что любой правильный план запроса выдаёт один и тот же набор строк.)

Значение `rows` здесь имеет особенность — оно выражает не число строк, обработанных или просканированных узлом плана, а число строк, выданных этим узлом. Часто оно окажется меньше числа просканированных строк в результате применённой к узлу фильтрации по условиям `WHERE`. В идеале, на верхнем уровне это значение будет приблизительно равно числу строк, которое фактически возвращает, изменяет или удаляет запрос.

Возвращаясь к нашему примеру:

```
EXPLAIN SELECT * FROM tenk1;
```

QUERY PLAN

```
-----
Seq Scan on tenk1  (cost=0.00..458.00 rows=10000 width=244)
```

Эти числа получаются очень просто. Выполните:

```
SELECT relpages, reltuples FROM pg_class WHERE relname = 'tenk1';
```

и вы увидите, что `tenk1` содержит 358 страниц диска и 10000 строк. Общая стоимость вычисляется как $(\text{число_чтений_диска} * \text{seq_page_cost}) + (\text{число_просканированных_строк} * \text{cpu_tuple_cost})$. По умолчанию, `seq_page_cost` равно 1.0, а `cpu_tuple_cost` — 0.01, так что приблизительная стоимость запроса равна $(358 * 1.0) + (10000 * 0.01) = 458$.

Теперь давайте изменим запрос, добавив в него предложение `WHERE`:

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 7000;
```

QUERY PLAN

```
-----
Seq Scan on tenk1  (cost=0.00..483.00 rows=7001 width=244)
  Filter: (unique1 < 7000)
```

Заметьте, что в выводе `EXPLAIN` показано, что условие `WHERE` применено как «фильтр» к узлу плана `Seq Scan` (Последовательное сканирование). Это означает, что узел плана проверяет это условие для каждого просканированного им узла и выводит только те строки, которые удовлетворяют ему.

Предложение `WHERE` повлияло на оценку числа выходных строк. Однако при сканировании потребуется прочитать все 10000 строк, поэтому общая стоимость не уменьшилась. На деле она даже немного увеличилась (на $10000 * \text{cpu_operator_cost}$, если быть точными), отражая дополнительное время, которое потребуется процессору на проверку условия `WHERE`.

Фактическое число строк результата этого запроса будет равно 7000, но значение `rows` даёт только приблизительное значение. Если вы попытаетесь повторить этот эксперимент, вы можете получить немного другую оценку; более того, она может меняться после каждой команды `ANALYZE`, так как `ANALYZE` получает статистику по случайной выборке таблицы.

Теперь давайте сделаем ограничение более избирательным:

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100;
```

QUERY PLAN

```
-----
Bitmap Heap Scan on tenk1  (cost=5.07..229.20 rows=101 width=244)
  Recheck Cond: (unique1 < 100)
    -> Bitmap Index Scan on tenk1_unique1  (cost=0.00..5.04 rows=101 width=0)
```

В данном случае планировщик решил использовать план из двух этапов: сначала дочерний узел плана просматривает индекс и находит в нём адреса строк, соответствующих условию индекса, а затем верхний узел собственно выбирает эти строки из таблицы. Выбирать строки по отдельности гораздо дороже, чем просто читать их последовательно, но так как читать придётся не все страницы таблицы, это всё равно будет дешевле, чем сканировать всю таблицу. (Использование двух уровней плана объясняется тем, что верхний узел сортирует адреса строк, выбранных из индекса, в физическом порядке, прежде чем читать, чтобы снизить стоимость отдельных чтений. Слово «bitmap» (битовая карта) в имени узла обозначает механизм, выполняющий сортировку.)

Теперь давайте добавим ещё одно условие в предложение `WHERE`:

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND stringu1 = 'xxx';
```

QUERY PLAN

```
-----
Bitmap Heap Scan on tenk1  (cost=5.04..229.43 rows=1 width=244)
  Recheck Cond: (unique1 < 100)
  Filter: (stringu1 = 'xxx'::name)
    -> Bitmap Index Scan on tenk1_unique1  (cost=0.00..5.04 rows=101 width=0)
      Index Cond: (unique1 < 100)
```

Добавленное условие `stringu1 = 'xxx'` уменьшает оценку числа результирующих строк, но не стоимость запроса, так как просматриваться будет тот же набор строк, что и раньше. Заметьте, что условие на `stringu1` не добавляется в качестве условия индекса, так как индекс построен только по столбцу `unique1`. Вместо этого оно применяется как фильтр к строкам, полученным по индексу. В результате стоимость даже немного увеличилась, отражая добавление этой проверки.

В некоторых случаях планировщик предпочтёт «простой» план сканирования индекса:

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 = 42;
```

QUERY PLAN

```
-----
Index Scan using tenk1_unique1 on tenk1  (cost=0.29..8.30 rows=1 width=244)
  Index Cond: (unique1 = 42)
```

В плане такого типа строки таблицы выбираются в порядке индекса, в результате чего чтение их обходится дороже, но так как их немного, дополнительно сортировать положения строк не стоит. Вы часто будете встречать этот тип плана в запросах, которые выбирают всего одну строку. Также он часто задействуется там, где условие `ORDER BY` соответствует порядку индекса, так как в этих случаях для выполнения `ORDER BY` не требуется дополнительный шаг сортировки. В этом примере

добавленная конструкция `ORDER BY unique1` будет использовать тот же план, потому что индекс уже неявно обеспечивает нужный порядок.

Планировщик может обработать конструкцию `ORDER BY` несколькими способами. Предыдущий пример показывает, что нужный порядок может быть получен неявным образом. Также планировщик может задействовать явную операцию `Sort`:

```
EXPLAIN SELECT * FROM tenk1 ORDER BY unique1;
               QUERY PLAN
-----
Sort  (cost=1109.39..1134.39 rows=10000 width=244)
  Sort Key: unique1
    -> Seq Scan on tenk1  (cost=0.00..445.00 rows=10000 width=244)
```

Если подплан гарантирует сортировку по префиксу заданных ключей сортировки, планировщик может применить операцию `Incremental sort` (инкрементальную сортировку):

```
EXPLAIN SELECT * FROM tenk1 ORDER BY four, ten LIMIT 100;
               QUERY PLAN
-----
Limit  (cost=521.06..538.05 rows=100 width=244)
  -> Incremental Sort  (cost=521.06..2220.95 rows=10000 width=244)
      Sort Key: four, ten
      Presorted Key: four
      -> Index Scan using index_tenk1_on_four on tenk1  (cost=0.29..1510.08
rows=10000 width=244)
```

С инкрементальной сортировкой, в отличие от обычной, кортежи могут выдаваться до завершения сортировки всего результата, это в частности позволяет оптимизировать запросы с `LIMIT`. Кроме того, для инкрементальной сортировки может потребоваться меньше памяти, вследствие чего уменьшается вероятность вытеснения сортируемых данных на диск, но с другой стороны, требуется разделять результирующее множество на несколько частей, что влечёт дополнительные накладные расходы.

Если в таблице есть отдельные индексы по разным столбцам, фигурирующим в `WHERE`, планировщик может выбрать сочетание этих индексов (с `AND` и `OR`):

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000;
               QUERY PLAN
-----
Bitmap Heap Scan on tenk1  (cost=25.08..60.21 rows=10 width=244)
  Recheck Cond: ((unique1 < 100) AND (unique2 > 9000))
    -> BitmapAnd  (cost=25.08..25.08 rows=10 width=0)
        -> Bitmap Index Scan on tenk1_unique1  (cost=0.00..5.04 rows=101 width=0)
            Index Cond: (unique1 < 100)
        -> Bitmap Index Scan on tenk1_unique2  (cost=0.00..19.78 rows=999 width=0)
            Index Cond: (unique2 > 9000)
```

Но для этого потребуется обойти оба индекса, так что это не обязательно будет выгоднее, чем просто просмотреть один индекс, а второе условие обработать как фильтр. Измените диапазон и вы увидите, как это повлияет на план.

Следующий пример иллюстрирует эффекты `LIMIT`:

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000 LIMIT 2;
               QUERY PLAN
-----
Limit  (cost=0.29..14.48 rows=2 width=244)
  -> Index Scan using tenk1_unique2 on tenk1  (cost=0.29..71.27 rows=10 width=244)
```

```
Index Cond: (unique2 > 9000)
Filter: (unique1 < 100)
```

Это тот же запрос, что и раньше, но добавили мы в него `LIMIT`, чтобы возвращались не все строки, и планировщик решает выполнять запрос по-другому. Заметьте, что общая стоимость и число строк для узла `Index Scan` рассчитываются в предположении, что он будет выполняться полностью. Однако узел `Limit` должен остановиться, получив только пятую часть всех строк, так что его стоимость будет составлять одну пятую от вычисленной ранее, и это и будет итоговой оценкой стоимости запроса. С другой стороны, планировщик мог бы просто добавить в предыдущий план узел `Limit`, но это не избавило бы от затрат на запуск сканирования битовой карты, а значит, общая стоимость была бы выше 25 единиц.

Давайте попробуем соединить две таблицы по столбцам, которые мы уже использовали:

```
EXPLAIN SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 10 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
-----
Nested Loop  (cost=4.65..118.62 rows=10 width=488)
-> Bitmap Heap Scan on tenk1 t1  (cost=4.36..39.47 rows=10 width=244)
    Recheck Cond: (unique1 < 10)
-> Bitmap Index Scan on tenk1_unique1  (cost=0.00..4.36 rows=10 width=0)
    Index Cond: (unique1 < 10)
-> Index Scan using tenk2_unique2 on tenk2 t2  (cost=0.29..7.91 rows=1 width=244)
    Index Cond: (unique2 = t1.unique2)
```

В этом плане появляется узел соединения с вложенным циклом, на вход которому поступают данные от двух его потомков, узлов сканирования. Эту структуру плана отражает отступ основных строк его узлов. Первый, или «внешний», потомок соединения — узел сканирования битовой карты, похожий на те, что мы видели раньше. Его стоимость и число строк те же, что мы получили бы для запроса `SELECT ... WHERE unique1 < 10`, так как к этому узлу добавлено предложение `WHERE unique1 < 10`. Условие `t1.unique2 = t2.unique2` ещё не учитывается, поэтому оно не влияет на число строк узла внешнего сканирования. Узел соединения с вложенным циклом будет выполнять узел «внутреннего» потомка для каждой строки, полученной из внешнего потомка. Значения столбцов из текущей внешней строки могут использоваться во внутреннем сканировании (в данном случае это значение `t1.unique2`), поэтому мы получаем план и стоимость примерно такие, как и раньше для простого запроса `SELECT ... WHERE t2.unique2 = константа`. (На самом деле оценочная стоимость немного меньше, в предположении, что при неоднократном сканировании индекса по `t2` положительную роль сыграет кеширование.) В результате стоимость узла цикла складывается из стоимости внешнего сканирования, цены внутреннего сканирования, умноженной на число строк (здесь $10 * 7.91$), и небольшой наценки за обработку соединения.

В этом примере число выходных строк соединения равно произведению чисел строк двух узлов сканирования, но это не всегда будет так, потому что в дополнительных условиях `WHERE` могут упоминаться обе таблицы, так что применить их можно будет только в точке соединения, а не в одном из узлов сканирования. Например:

```
EXPLAIN SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 10 AND t2.unique2 < 10 AND t1.hundred < t2.hundred;
```

QUERY PLAN

```
-----
Nested Loop  (cost=4.65..49.46 rows=33 width=488)
  Join Filter: (t1.hundred < t2.hundred)
-> Bitmap Heap Scan on tenk1 t1  (cost=4.36..39.47 rows=10 width=244)
    Recheck Cond: (unique1 < 10)
```

```

-> Bitmap Index Scan on tenk1_unique1 (cost=0.00..4.36 rows=10 width=0)
    Index Cond: (unique1 < 10)
-> Materialize (cost=0.29..8.51 rows=10 width=244)
    -> Index Scan using tenk2_unique2 on tenk2 t2 (cost=0.29..8.46 rows=10
width=244)
        Index Cond: (unique2 < 10)

```

Условие `t1.hundred < t2.hundred` не может быть проверено в индексе `tenk2_unique2`, поэтому оно применяется в узле соединения. Это уменьшает оценку числа выходных строк, тогда как число строк в узлах сканирования не меняется.

Заметьте, что здесь планировщик решил «материализовать» внутреннее отношение соединения, поместив поверх него узел плана `Materialize` (Материализовать). Это значит, что сканирование индекса `t2` будет выполняться только единожды, при том, что узлу вложенного цикла соединения потребуется прочитать данные десять раз, по числу строк во внешнем соединении. Узел `Materialize` сохраняет считанные данные в памяти, чтобы затем выдать их из памяти на следующих проходах.

Выполняя внешние соединения, вы можете встретить узлы плана с присоединёнными условиями, как обычными «Filter», так и «Join Filter» (Фильтр соединения). Условия `Join Filter` формируются из предложения `ON` для внешнего соединения, так что если строка не удовлетворяет условию `Join Filter`, она всё же выдаётся как строка, дополненная значениями `NULL`. Обычное же условие `Filter` применяется после правил внешнего соединения и поэтому полностью исключает строки. Во внутреннем соединении оба этих фильтра работают одинаково.

Если немного изменить избирательность запроса, мы можем получить совсем другой план соединения:

```

EXPLAIN SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2;

```

QUERY PLAN

```

-----
Hash Join (cost=230.47..713.98 rows=101 width=488)
  Hash Cond: (t2.unique2 = t1.unique2)
  -> Seq Scan on tenk2 t2 (cost=0.00..445.00 rows=10000 width=244)
  -> Hash (cost=229.20..229.20 rows=101 width=244)
      -> Bitmap Heap Scan on tenk1 t1 (cost=5.07..229.20 rows=101 width=244)
          Recheck Cond: (unique1 < 100)
          -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101
width=0)
              Index Cond: (unique1 < 100)

```

Здесь планировщик выбирает соединение по хешу, при котором строки одной таблицы записываются в хеш-таблицу в памяти, после чего сканируется другая таблица и для каждой её строки проверяется соответствие по хеш-таблице. Обратите внимание, что и здесь отступы отражают структуру плана: результат сканирования битовой карты по `tenk1` подаётся на вход узлу `Hash`, который конструирует хеш-таблицу. Затем она передаётся узлу `Hash Join`, который читает строки из узла внешнего потомка и проверяет их по этой хеш-таблице.

Ещё один возможный тип соединения — соединение слиянием:

```

EXPLAIN SELECT *
FROM tenk1 t1, onek t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2;

```

QUERY PLAN

```

-----

```

```
Merge Join (cost=198.11..268.19 rows=10 width=488)
  Merge Cond: (t1.unique2 = t2.unique2)
    -> Index Scan using tenk1_unique2 on tenk1 t1 (cost=0.29..656.28 rows=101
width=244)
      Filter: (unique1 < 100)
    -> Sort (cost=197.83..200.33 rows=1000 width=244)
      Sort Key: t2.unique2
    -> Seq Scan on onek t2 (cost=0.00..148.00 rows=1000 width=244)
```

Соединение слиянием требует, чтобы входные данные для него были отсортированы по ключам соединения. В этом плане данные `tenk1` сортируются после сканирования индекса, при котором все строки просматриваются в правильном порядке, но таблицу `onek` выгоднее оказывается последовательно просканировать и отсортировать, так как в этой таблице нужно обработать гораздо больше строк. (Последовательное сканирование и сортировка часто бывает быстрее сканирования индекса, когда нужно отсортировать много строк, так как при сканировании по индексу обращения к диску не упорядочены.)

Один из способов посмотреть различные планы — принудить планировщик не считать выбранную им стратегию самой выгодной, используя флаги, описанные в [Подразделе 18.7.1](#). (Это полезный, хотя и грубый инструмент. См. также [Раздел 14.3](#).) Например, если мы убеждены, что последовательное сканирование и сортировка — не лучший способ обработать таблицу `onek` в предыдущем примере, мы можем попробовать

```
SET enable_sort = off;
```

```
EXPLAIN SELECT *
FROM tenk1 t1, onek t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
-----
Merge Join (cost=0.56..292.65 rows=10 width=488)
  Merge Cond: (t1.unique2 = t2.unique2)
    -> Index Scan using tenk1_unique2 on tenk1 t1 (cost=0.29..656.28 rows=101
width=244)
      Filter: (unique1 < 100)
    -> Index Scan using onek_unique2 on onek t2 (cost=0.28..224.79 rows=1000
width=244)
```

Видно, что планировщик считает сортировку `onek` со сканированием индекса примерно на 12% дороже, чем последовательное сканирование и сортировку. Конечно, может возникнуть вопрос — а правильно ли это? Мы можем ответить на него, используя описанную ниже команду `EXPLAIN ANALYZE`.

14.1.2. EXPLAIN ANALYZE

Точность оценок планировщика можно проверить, используя команду `EXPLAIN` с параметром `ANALYZE`. С этим параметром `EXPLAIN` на самом деле выполняет запрос, а затем выводит фактическое число строк и время выполнения, накопленное в каждом узле плана, вместе с теми же оценками, что выдаёт обычная команда `EXPLAIN`. Например, мы можем получить примерно такой результат:

```
EXPLAIN ANALYZE SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 10 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
Nested Loop (cost=4.65..118.62 rows=10 width=488) (actual time=0.128..0.377 rows=10
loops=1)
-> Bitmap Heap Scan on tenk1 t1 (cost=4.36..39.47 rows=10 width=244) (actual
time=0.057..0.121 rows=10 loops=1)
    Recheck Cond: (unique1 < 10)
-> Bitmap Index Scan on tenk1_unique1 (cost=0.00..4.36 rows=10 width=0)
(actual time=0.024..0.024 rows=10 loops=1)
    Index Cond: (unique1 < 10)
-> Index Scan using tenk2_unique2 on tenk2 t2 (cost=0.29..7.91 rows=1 width=244)
(actual time=0.021..0.022 rows=1 loops=10)
    Index Cond: (unique2 = t1.unique2)
Planning time: 0.181 ms
Execution time: 0.501 ms
```

Заметьте, что значения «actual time» (фактическое время) приводятся в миллисекундах, тогда как оценки cost (стоимость) выражаются в произвольных единицах, так что они вряд ли совпадут. Обычно важнее определить, насколько приблизительная оценка числа строк близка к действительности. В этом примере они в точности совпали, но на практике так бывает редко.

В некоторых планах запросов некоторый внутренний узел может выполняться неоднократно. Например, внутреннее сканирование индекса будет выполняться для каждой внешней строки во вложенном цикле верхнего уровня. В таких случаях значение loops (циклы) показывает, сколько всего раз выполнялся этот узел, а фактическое время и число строк вычисляется как среднее по всем итерациям. Это делается для того, чтобы полученные значения можно было сравнить с выводимыми приблизительными оценками. Чтобы получить общее время, затраченное на выполнение узла, время одной итерации нужно умножить на значение loops. В показанном выше примере мы потратили в общей сложности 0.220 мс на сканирование индекса в tenk2.

В ряде случаев EXPLAIN ANALYZE выводит дополнительную статистику по выполнению, включающую не только время выполнения узлов и число строк. Для узлов Sort и Hash, например выводится следующая информация:

```
EXPLAIN ANALYZE SELECT *
FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2 ORDER BY t1.fivethous;
```

QUERY PLAN

```
-----
Sort (cost=717.34..717.59 rows=101 width=488) (actual time=7.761..7.774 rows=100
loops=1)
  Sort Key: t1.fivethous
  Sort Method: quicksort  Memory: 77kB
-> Hash Join (cost=230.47..713.98 rows=101 width=488) (actual time=0.711..7.427
rows=100 loops=1)
  Hash Cond: (t2.unique2 = t1.unique2)
-> Seq Scan on tenk2 t2 (cost=0.00..445.00 rows=10000 width=244) (actual
time=0.007..2.583 rows=10000 loops=1)
-> Hash (cost=229.20..229.20 rows=101 width=244) (actual time=0.659..0.659
rows=100 loops=1)
  Buckets: 1024  Batches: 1  Memory Usage: 28kB
-> Bitmap Heap Scan on tenk1 t1 (cost=5.07..229.20 rows=101 width=244)
(actual time=0.080..0.526 rows=100 loops=1)
  Recheck Cond: (unique1 < 100)
-> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101
width=0) (actual time=0.049..0.049 rows=100 loops=1)
  Index Cond: (unique1 < 100)
Planning time: 0.194 ms
Execution time: 8.008 ms
```

Для узла Sort показывается использованный метод и место сортировки (в памяти или на диске), а также задействованный объём памяти. Для узла Hash выводится число групп и пакетов хеша, а также максимальный объём, который заняла в памяти хеш-таблица. (Если число пакетов больше одного, часть хеш-таблицы будет выгружаться на диск и занимать какое-то пространство, но его объём здесь не показывается.)

Другая полезная дополнительная информация — число строк, удалённых условием фильтра:

```
EXPLAIN ANALYZE SELECT * FROM tenk1 WHERE ten < 7;
```

QUERY PLAN

```
-----
Seq Scan on tenk1  (cost=0.00..483.00 rows=7000 width=244) (actual time=0.016..5.107
rows=7000 loops=1)
  Filter: (ten < 7)
  Rows Removed by Filter: 3000
Planning time: 0.083 ms
Execution time: 5.905 ms
```

Эти значения могут быть особенно ценны для условий фильтра, применённых к узлам соединения. Строка «Rows Removed» выводится, только когда условие фильтра отбрасывает минимум одну просканированную строку или потенциальную пару соединения, если это узел соединения.

Похожую ситуацию можно наблюдать при сканировании «неточного» индекса. Например, рассмотрим этот план поиска многоугольников, содержащих указанную точку:

```
EXPLAIN ANALYZE SELECT * FROM polygon_tbl WHERE f1 @> polygon '(0.5,2.0)';
```

QUERY PLAN

```
-----
Seq Scan on polygon_tbl  (cost=0.00..1.05 rows=1 width=32) (actual time=0.044..0.044
rows=0 loops=1)
  Filter: (f1 @> '((0.5,2))'::polygon)
  Rows Removed by Filter: 4
Planning time: 0.040 ms
Execution time: 0.083 ms
```

Планировщик полагает (и вполне справедливо), что таблица слишком мала для сканирования по индексу, поэтому он выбирает последовательное сканирование, при котором все строки отбрасываются условием фильтра. Но если мы принудим его выбрать сканирование по индексу, мы получим:

```
SET enable_seqscan TO off;
```

```
EXPLAIN ANALYZE SELECT * FROM polygon_tbl WHERE f1 @> polygon '(0.5,2.0)';
```

QUERY PLAN

```
-----
Index Scan using gpolygonind on polygon_tbl  (cost=0.13..8.15 rows=1 width=32) (actual
time=0.062..0.062 rows=0 loops=1)
  Index Cond: (f1 @> '((0.5,2))'::polygon)
  Rows Removed by Index Recheck: 1
Planning time: 0.034 ms
Execution time: 0.144 ms
```

Здесь мы видим, что индекс вернул одну потенциально подходящую строку, но затем она была отброшена при перепроверке условия индекса. Это объясняется тем, что индекс GiST является «неточным» для проверок включений многоугольников: фактически он возвращает строки с многоугольниками, перекрывающими точку по координатам, а затем для этих строк нужно выполнять точную проверку.

EXPLAIN принимает параметр BUFFERS (который также можно применять с ANALYZE), включающий ещё более подробную статистику выполнения запроса:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000;
```

QUERY PLAN

```
-----
Bitmap Heap Scan on tenk1  (cost=25.08..60.21 rows=10 width=244) (actual
time=0.323..0.342 rows=10 loops=1)
  Recheck Cond: ((unique1 < 100) AND (unique2 > 9000))
  Buffers: shared hit=15
  -> BitmapAnd  (cost=25.08..25.08 rows=10 width=0) (actual time=0.309..0.309 rows=0
loops=1)
    Buffers: shared hit=7
    -> Bitmap Index Scan on tenk1_unique1  (cost=0.00..5.04 rows=101 width=0)
(actual time=0.043..0.043 rows=100 loops=1)
      Index Cond: (unique1 < 100)
      Buffers: shared hit=2
    -> Bitmap Index Scan on tenk1_unique2  (cost=0.00..19.78 rows=999 width=0)
(actual time=0.227..0.227 rows=999 loops=1)
      Index Cond: (unique2 > 9000)
      Buffers: shared hit=5
Planning time: 0.088 ms
Execution time: 0.423 ms
```

Значения, которые выводятся с параметром BUFFERS, помогают понять, на какие части запроса приходится большинство операций ввода-вывода.

Не забывайте, что EXPLAIN ANALYZE действительно выполняет запрос, хотя его результаты могут не показываться, а заменяться выводом команды EXPLAIN. Поэтому при таком анализе возможны побочные эффекты. Если вы хотите проанализировать запрос, изменяющий данные, но при этом сохранить прежние данные таблицы, вы можете откатить транзакцию после запроса:

```
BEGIN;
```

```
EXPLAIN ANALYZE UPDATE tenk1 SET hundred = hundred + 1 WHERE unique1 < 100;
```

QUERY PLAN

```
-----
Update on tenk1  (cost=5.08..230.08 rows=0 width=0) (actual time=3.791..3.792 rows=0
loops=1)
  -> Bitmap Heap Scan on tenk1  (cost=5.08..230.08 rows=102 width=10) (actual
time=0.069..0.513 rows=100 loops=1)
    Recheck Cond: (unique1 < 100)
    Heap Blocks: exact=90
    -> Bitmap Index Scan on tenk1_unique1  (cost=0.00..5.05 rows=102 width=0)
(actual time=0.036..0.037 rows=300 loops=1)
      Index Cond: (unique1 < 100)
Planning Time: 0.113 ms
Execution Time: 3.850 ms
```

```
ROLLBACK;
```

Как показано в этом примере, когда выполняется команда INSERT, UPDATE, DELETE или MERGE, собственно изменение данных в таблице происходит в узле верхнего уровня Insert, Update, Delete или Merge. Узлы плана более низких уровней выполняют работу по нахождению старых строк и/или вычислению новых данных. Поэтому вверху мы видим тот же тип сканирования битовой карты, что и раньше, только теперь его вывод подаётся узлу Update, который сохраняет изменённые строки.

Стоит отметить, что узел, изменяющий данные, может выполняться значительное время (в данном случае это составляет львиную часть всего времени), но планировщик не учитывает эту работу в оценке общей стоимости. Это связано с тем, что эта работа будет одинаковой при любом правильном плане запроса, и поэтому на выбор плана она не влияет.

Когда команда UPDATE, DELETE или MERGE имеет дело с иерархией наследования, вывод может быть таким:

```
EXPLAIN UPDATE parent SET f2 = f2 + 1 WHERE f1 = 101;
                                QUERY PLAN
-----
Update on parent  (cost=0.00..24.59 rows=0 width=0)
  Update on parent parent_1
  Update on child1 parent_2
  Update on child2 parent_3
  Update on child3 parent_4
-> Result  (cost=0.00..24.59 rows=4 width=14)
    -> Append  (cost=0.00..24.54 rows=4 width=14)
      -> Seq Scan on parent parent_1  (cost=0.00..0.00 rows=1 width=14)
          Filter: (f1 = 101)
      -> Index Scan using child1_pkey on child1 parent_2  (cost=0.15..8.17
rows=1 width=14)
          Index Cond: (f1 = 101)
      -> Index Scan using child2_pkey on child2 parent_3  (cost=0.15..8.17
rows=1 width=14)
          Index Cond: (f1 = 101)
      -> Index Scan using child3_pkey on child3 parent_4  (cost=0.15..8.17
rows=1 width=14)
          Index Cond: (f1 = 101)
```

В этом примере узлу Update помимо изначально упомянутой в запросе родительской таблицы нужно обработать ещё три дочерние таблицы. Поэтому формируются четыре плана сканирования, по одному для каждой таблицы. Ясности ради для узла Update добавляется примечание, показывающее, какие именно таблицы будут изменяться, в том же порядке, в каком они идут в соответствующих внутренних планах.

Под заголовком Planning time (Время планирования) команда EXPLAIN ANALYZE выводит время, затраченное на построение плана запроса из разобранного запроса и его оптимизацию. Время собственно разбора или перезаписи запроса в него не включается.

Значение Execution time (Время выполнения), выводимое командой EXPLAIN ANALYZE, включает продолжительность запуска и остановки исполнителя запроса, а также время выполнения всех сработавших триггеров, но не включает время разбора, перезаписи и планирования запроса. Время, потраченное на выполнение триггеров BEFORE (если такие имеются) включается во время соответствующих узлов Insert, Update или Delete node; но время выполнения триггеров AFTER не учитывается, так как триггеры AFTER срабатывают после выполнения всего плана. Общее время, проведённое в каждом триггере (BEFORE или AFTER), также выводится отдельно. Заметьте, что триггеры отложенных ограничений выполняются только в конце транзакции, так что время их выполнения EXPLAIN ANALYZE не учитывает.

14.1.3. Ограничения

Время выполнения, измеренное командой EXPLAIN ANALYZE, может значительно отличаться от времени выполнения того же запроса в обычном режиме. Тому есть две основных причины. Во-первых, так как при анализе никакие строки результата не передаются клиенту, время ввода-вывода и передачи по сети не учитывается. Во-вторых, может быть существенной дополнительной нагрузка, связанная с функциями измерений EXPLAIN ANALYZE, особенно в системах, где вызов `gettimeofday()` выполняется медленно. Для измерения этой нагрузки вы можете воспользоваться утилитой [pg_test_timing](#).

Результаты EXPLAIN не следует распространять на ситуации, значительно отличающиеся от тех, в которых вы проводите тестирование. В частности, не следует полагать, что выводы, полученные для игрушечной таблицы, будут применимы и для настоящих больших таблиц. Оценки стоимости нелинейны и планировщик может выбирать разные планы в зависимости от размера таблицы. Например, в крайнем случае вся таблица может уместиться в одну страницу диска, и тогда вы почти наверняка получите план последовательного сканирования, независимо от того, есть у неё и индексы или нет. Планировщик понимает, что для обработки таблицы ему в любом случае потребуется прочитать одну страницу, так что нет никакого смысла обращаться к ещё одной странице за индексом. (Мы наблюдали это в показанном выше примере с polygon_tbl.)

Бывает, что фактическое и приближённо оценённое значения не совпадают, но в этом нет ничего плохого. Например, это возможно, когда выполнение плана узла прекращается преждевременно из-за указания LIMIT или подобного эффекта. Например, для запроса с LIMIT, который мы пробовали раньше:

```
EXPLAIN ANALYZE SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000 LIMIT 2;
```

QUERY PLAN

```
-----
Limit  (cost=0.29..14.71 rows=2 width=244) (actual time=0.177..0.249 rows=2 loops=1)
->  Index Scan using tenk1_unique2 on tenk1  (cost=0.29..72.42 rows=10 width=244)
    (actual time=0.174..0.244 rows=2 loops=1)
        Index Cond: (unique2 > 9000)
        Filter: (unique1 < 100)
        Rows Removed by Filter: 287
Planning time: 0.096 ms
Execution time: 0.336 ms
```

Оценки стоимости и числа строк для узла Index Scan показываются в предположении, что этот узел будет выполняться до конца. Но в действительности узел Limit прекратил запрашивать строки, как только получил первые две, так что фактическое число строк равно 2 и время выполнения запроса будет меньше, чем рассчитал планировщик. Но это не ошибка, а просто следствие того, что оценённые и фактические значения выводятся по-разному.

Соединения слиянием также имеют свои особенности, которые могут ввести в заблуждение. Соединение слиянием прекратит читать один источник данных, если второй будет прочитан до конца, а следующее значение ключа в первом больше последнего значения во втором. В этом случае пар строк больше не будет, так что сканировать первый источник дальше нет смысла. В результате будут прочитаны не все строки одного потомка и вы получите тот же эффект, что и с LIMIT. Кроме того, если внешний (первый) потомок содержит строки с повторяющимися значениями ключа, внутренний (второй) потомок сдвинется назад и повторно выдаст строки для этого значения ключа. EXPLAIN ANALYZE считает эти повторяющиеся строки, как если бы это действительно были дополнительные строки внутреннего источника. Когда во внешнем узле много таких повторений ключей, фактическое число строк, подсчитанное для внутреннего узла, может значительно превышать число строк в соответствующей таблице.

Для узлов BitmapAnd (Логическое произведение битовых карт) и BitmapOr (Логическое сложение битовых карт) фактическое число строк всегда равно 0 из-за ограничений реализации.

Обычно EXPLAIN выводит подробности для каждого узла плана, сгенерированного планировщиком. Однако бывают ситуации, когда исполнитель может определить, что некоторые узлы не нужно выполнять, так как они не могут выдать никакие строки, с учётом значений параметров, ставших известными уже после планирования. (В настоящее время это может произойти только с дочерними узлами Append или MergeAppend, сканирующими секционированную таблицу.) В таких ситуациях эти узлы плана не попадают в вывод EXPLAIN, а в плане появляется запись Subplans Removed: N (Подпланов удалено: N).

14.2. Статистика, используемая планировщиком

14.2.1. Статистика по одному столбцу

Как было показано в предыдущем разделе, планировщик запросов должен оценить число строк, возвращаемых запросов, чтобы сделать правильный выбор в отношении плана запроса. В этом разделе кратко описывается статистика, которую использует система для этих оценок.

В частности, статистика включает общее число записей в каждой таблице и индексе, а также число дисковых блоков, которые они занимают. Эта информация содержится в таблице `pg_class`, в столбцах `reltuples` и `relpages`. Получить её можно, например так:

```
SELECT relname, relkind, reltuples, relpages
FROM pg_class
WHERE relname LIKE 'tenk1%';
```

relname	relkind	reltuples	relpages
tenk1	r	10000	358
tenk1_hundred	i	10000	30
tenk1_thous_tenthous	i	10000	30
tenk1_unique1	i	10000	30
tenk1_unique2	i	10000	30

(5 rows)

Здесь мы видим, что `tenk1` содержит 10000 строк данных и столько же строк в индексах, но объём индексов гораздо меньше таблицы (что неудивительно).

Для большей эффективности `reltuples` и `relpages` не пересчитываются «на лету», так что они обычно содержат несколько устаревшие значения. Их обновляют команды `VACUUM`, `ANALYZE` и несколько команд DDL, такие как `CREATE INDEX`. `VACUUM` и `ANALYZE` могут не сканировать всю таблицу (и обычно так и делают), а только вычислить приращение `reltuples` по части таблицы, так что результат остаётся приблизительным. В любом случае планировщик пересчитывает значения, полученные из `pg_class`, в пропорции к текущему физическому размеру таблицы и таким образом уточняет приближение.

Большинство запросов возвращают не все строки таблицы, а только немногие из них, ограниченные условиями `WHERE`. Поэтому планировщику нужно оценить *избирательность* условий `WHERE`, то есть определить, какой процент строк будет соответствовать каждому условию в предложении `WHERE`. Нужная для этого информация хранится в системном каталоге `pg_statistic`. Значения в `pg_statistic` обновляются командами `ANALYZE` и `VACUUM ANALYZE` и никогда не бывают точными, даже сразу после обновления.

Для исследования статистики лучше обращаться не непосредственно к таблице `pg_statistic`, а к представлению `pg_stats`, предназначенному для облегчения восприятия этой информации. Кроме того, представление `pg_stats` доступно для чтения всем, тогда как `pg_statistic` — только суперпользователям. (Это сделано для того, чтобы непривилегированные пользователи не могли ничего узнать о содержимом таблиц других людей из статистики. Представление `pg_stats` устроено так, что оно показывает строки только для тех таблиц, которые может читать данный пользователь.) Например, мы можем выполнить:

```
SELECT attname, inherited, n_distinct,
       array_to_string(most_common_vals, E'\n') as most_common_vals
FROM pg_stats
WHERE tablename = 'road';
```

attname	inherited	n_distinct	most_common_vals
name	f	-0.363388	I- 580 Ramp+
			I- 880 Ramp+
			Sp Railroad +

			I- 580	+
			I- 680	Ramp
name	t	-0.284859	I- 880	Ramp+
			I- 580	Ramp+
			I- 680	Ramp+
			I- 580	+
(2 rows)			State Hwy 13	Ramp

Заметьте, что для одного столбца показывается две строки: одна соответствует полной иерархии наследования, построенной для таблицы `road (inherited=t)`, и другая относится непосредственно к таблице `road (inherited=f)`.

Объём информации, сохраняемой в `pg_statistic` командой `ANALYZE`, в частности максимальное число записей в массивах `most_common_vals` (самые популярные значения) и `histogram_bounds` (границы гистограмм) для каждого столбца, можно ограничить на уровне столбцов с помощью команды `ALTER TABLE SET STATISTICS` или глобально, установив параметр конфигурации `default_statistics_target`. В настоящее время ограничение по умолчанию равно 100 записям. Увеличивая этот предел, можно повысить точность оценок планировщика, особенно для столбцов с нерегулярным распределением данных, ценой роста размера `pg_statistic` и незначительного увеличения времени расчёта этих оценок. И напротив, для столбцов с простым распределением данных может быть достаточно меньшего предела.

Подробнее использование статистики планировщиком описывается в [Главе 71](#).

14.2.2. Расширенная статистика

Часто наблюдается картина, когда медленное выполнение запросов объясняется плохим выбором плана из-за того, что несколько столбцов, фигурирующих в условиях запроса, оказываются связанными. Обычно планировщик полагает, что несколько условий не зависят друг от друга, а это предположение оказывается неверным, когда значения этих столбцов коррелируют. Обычная статистика, которая по природе своей строится по отдельным столбцам, не может выявить корреляции между столбцами. Однако Postgres Pro имеет возможность вычислять *многовариантную статистику*, которая может собирать необходимую для этого информацию.

Так как число возможных комбинаций столбцов очень велико, автоматически вычислять многовариантную статистику непрактично. Вместо этого можно создать *объекты расширенной статистики*, чаще называемые просто *объектами статистики*, чтобы сервер собирал статистику по некоторым наборам столбцов, представляющим интерес.

Объекты статистики создаются командой `CREATE STATISTICS`. При создании такого объекта просто добавляется запись в каталоге, выражающая востребованность этой статистики. Собственно сбор данных выполняется процедурой `ANALYZE` (запускаемой вручную или автоматически в фоновом процессе). Изучить собранные значения можно в каталоге `pg_statistic_ext_data`.

Команда `ANALYZE` вычисляет расширенную статистику по той же выборке строк таблицы, которая используется и для вычисления обычной статистики по отдельным столбцам. Так как размер выборки увеличивается с увеличением целевого ограничения статистики для таблицы или любых её столбцов (как описано в предыдущем разделе), при большем целевом ограничении обычно получается более точная расширенная статистика, но и времени на её вычисление требуется больше.

В следующих подразделах описываются виды расширенной статистики, поддерживаемые в настоящее время.

14.2.2.1. Функциональные зависимости

Простейший вид расширенной статистики отслеживает *функциональные зависимости* (это понятие используется в определении нормальных форм баз данных). Мы называем столбец `b` функционально зависимым от столбца `a`, если знания значения `a` достаточно для определения значения `b`, то есть не существует двух строк с одинаковыми значениями `a`, но разными значениями `b`. В пол-

ностью нормализованной базы данных функциональные зависимости должны существовать только в первичных ключах и суперключах. Однако на практике многие наборы данных не нормализуются полностью по разным причинам; например, денормализация часто производится намеренно по соображениям производительности.

Существование функциональных зависимостей напрямую влияет на точность оценок в определённых запросах. Если запрос содержит условия как по независимым, так и по зависимым столбцам, условия по зависимым столбцам дополнительно не сокращают размер результата. Однако без знания о функциональной зависимости планировщик запросов будет полагать, что все условия независимы, и недооценит размер результата.

Для информирования планировщика о функциональных зависимостях команда `ANALYZE` может собирать показатели зависимостей между столбцами. Оценить степень зависимости между всеми наборами столбцов обошлось бы непозволительно дорого, поэтому сбор данных ограничивается только теми группами столбцов, которые фигурируют вместе в объекте статистики, определённом со свойством `dependencies`. Во избежание ненужных издержек при выполнении `ANALYZE` и последующем планировании запросов статистику с `dependencies` рекомендуется создавать только для групп сильно коррелирующих столбцов.

Взгляните на пример сбора статистики функциональной зависимости:

```
CREATE STATISTICS stts (dependencies) ON city, zip FROM zipcodes;
```

```
ANALYZE zipcodes;
```

```
SELECT stxname, stxkeys, stxddependencies
FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid)
WHERE stxname = 'stts';
 stxname | stxkeys | stxddependencies
-----+-----+-----
 stts    | 1 5     | {"1 => 5": 1.000000, "5 => 1": 0.423130}
(1 row)
```

В показанном случае столбец 1 (код zip) полностью определяет столбец 5 (city), так что коэффициент равен 1.0, тогда как город (столбец city) определяет код ZIP только в 42% всех случаев, что означает, что многие города (58%) представлены несколькими кодами ZIP.

При вычислении избирательности запроса, в котором задействованы функционально зависимые столбцы, планировщик корректирует оценки избирательности по условиям, используя коэффициенты зависимостей, чтобы не допустить недооценки размера результата.

14.2.2.1.1. Ограничения функциональных зависимостей

Функциональные зависимости в настоящее время применяются только при рассмотрении простых условий с равенствами, сравнивающих значения столбцов с константами, и условиями `IN` с константами. Они не используются для улучшения оценок при проверке равенства двух столбцов или сравнении столбца с выражением, а также в условиях с диапазоном, условиях `LIKE` или любых других видах условий.

Рассматривая функциональные зависимости, планировщик предполагает, что условия по задействованным столбцам совместимы и таким образом избыточны. Если условия несовместимы, правильной оценкой должен быть ноль строк, но эта возможность не рассматривается. Например, с таким запросом

```
SELECT * FROM zipcodes WHERE city = 'San Francisco' AND zip = '94105';
```

планировщик отбросит условие с `city`, так как оно не влияет на избирательность, что верно. Однако он сделает то же предположение и в таком случае:

```
SELECT * FROM zipcodes WHERE city = 'San Francisco' AND zip = '90210';
```

хотя на самом деле этому запросу будет удовлетворять ноль строк. Но статистика функциональной зависимости не даёт достаточно информации, чтобы прийти к такому заключению.

Во многих практических ситуациях это предположение обычно удовлетворяется; например, графический интерфейс приложения для последующего формирования запроса может не допускать выбор несовместимого сочетания города и кода ZIP. Но когда это не так, статистика функциональной зависимости может не подойти.

14.2.2.2. Многовариантное число различных значений

Статистика по одному столбцу содержит число различных значений в каждом отдельном столбце. Оценки числа различных значений в сочетании нескольких столбцов (например, в `GROUP BY a, b`) часто оказываются ошибочными, когда планировщик имеет статистические данные только по отдельным столбцам, что приводит к выбору плохих планов.

Для улучшения таких оценок операция `ANALYZE` может собирать статистику по различным значениям для группы столбцов. Как и ранее, это непрактично делать для каждой возможной группы столбцов, так что данные собираются только по тем группам столбцов, которые указаны в определении объекта статистики, создаваемого со свойством `ndistinct`. Данные будут собираться по всем возможным сочетаниям из двух или нескольких столбцов из перечисленных в определении.

В продолжение предыдущего примера, количества различных значений в таблице ZIP-кодов могут выглядеть так:

```
CREATE STATISTICS stts2 (ndistinct) ON city, state, zip FROM zipcodes;

ANALYZE zipcodes;

SELECT stxkeys AS k, stxdndistinct AS nd
FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid)
WHERE stxname = 'stts2';
-[ RECORD 1 ]-----
k      | 1 2 5
nd | {"1, 2": 33178, "1, 5": 33178, "2, 5": 27435, "1, 2, 5": 33178}
(1 row)
```

Как видно, есть три комбинации столбцов, имеющих 33178 различных значений: код ZIP и штат; код ZIP и город; код ZIP, город и штат (то, что все эти числа равны, ожидаемый факт, так как сам по себе код ZIP в этой таблице уникален). С другой стороны, сочетание города и штата даёт только 27435 различных значений.

Объект статистики `ndistinct` рекомендуется создавать только для тех сочетаний столбцов, которые действительно используются при группировке, и только когда неправильная оценка числа групп может привести к выбору плохих планов. В противном случае усилия, потраченные на выполнение `ANALYZE`, будут напрасными.

14.2.2.3. Многовариантные списки MCV

Ещё один тип статистики, сохраняемой для каждого столбца, представляют списки частых значений. Такие списки позволяют получать очень точную оценку для отдельных столбцов, но для запросов, содержащих условия с несколькими столбцами, полученная по ним оценка может быть значительно искажена.

Для улучшения таких оценок операция `ANALYZE` может собирать списки MCV по комбинациям столбцов. Подобно статистике функциональных зависимостей и различных значений, такую статистику непрактично собирать для каждой возможной группировки столбцов. В данном случае это ещё более актуально, так как списки MCV (в отличие от двух упомянутых статистик) содержат распространённые значения столбцов. Поэтому данные для них собираются только по тем группам столбцов, которые фигурируют в объекте статистики, определённом с указанием `mcv`.

В продолжение предыдущего примера, список MCV для таблицы ZIP-кодов может выглядеть следующим образом (в отличие от более простых типов статистики, для его анализа требуется применить функцию):

```
CREATE STATISTICS stts3 (mcv) ON city, state FROM zipcodes;
```

```
ANALYZE zipcodes;
```

```
SELECT m.* FROM pg_statistic_ext join pg_statistic_ext_data on (oid = stxoid),
       pg_mcv_list_items(stxdmccv) m WHERE stxname = 'stts3';
```

index	values	nulls	frequency	base_frequency
0	{Washington, DC}	{f,f}	0.003467	2.7e-05
1	{Apo, AE}	{f,f}	0.003067	1.9e-05
2	{Houston, TX}	{f,f}	0.002167	0.000133
3	{El Paso, TX}	{f,f}	0.002	0.000113
4	{New York, NY}	{f,f}	0.001967	0.000114
5	{Atlanta, GA}	{f,f}	0.001633	3.3e-05
6	{Sacramento, CA}	{f,f}	0.001433	7.8e-05
7	{Miami, FL}	{f,f}	0.0014	6e-05
8	{Dallas, TX}	{f,f}	0.001367	8.8e-05
9	{Chicago, IL}	{f,f}	0.001333	5.1e-05

...
(99 rows)

Выводимая информация показывает, что наиболее распространённую комбинацию города и штата образует Washington и DC, с частотой около 0.35% (в объёме выборки). Базовая частота этой комбинации (вычисленная из частот значений в отдельных столбцах) составляет всего 0.0027%, то есть эта оценка оказывается заниженной на два порядка.

Объекты статистики MCV рекомендуется создавать только для тех сочетаний столбцов, которые действительно используются в условиях вместе, и только когда неправильная оценка числа групп может привести к выбору плохих планов. В противном случае усилия, потраченные на выполнение ANALYZE и планирование, будут напрасными.

14.3. Управление планировщиком с помощью явных предложений JOIN

Поведением планировщика в некоторой степени можно управлять, используя явный синтаксис JOIN. Понять, когда и почему это бывает нужно, поможет небольшое введение.

В простом запросе с соединением, например таком:

```
SELECT * FROM a, b, c WHERE a.id = b.id AND b.ref = c.id;
```

планировщик может соединять данные таблицы в любом порядке. Например, он может разработать план, в котором сначала А соединяется с В по условию WHERE a.id = b.id, а затем С соединяется с получившейся таблицей по другому условию WHERE. Либо он может соединить В с С, а затем с А результатом соединения. Он также может соединить сначала А с С, а затем результат с В — но это будет не эффективно, так как ему придётся сформировать полное декартово произведение А и С из-за отсутствия в предложении WHERE условия, подходящего для оптимизации соединения. (В Postgres Pro исполнитель запросов может соединять только по две таблицы, поэтому для получения результата нужно выбрать один из этих способов.) При этом важно понимать, что все эти разные способы соединения дают одинаковые по смыслу результаты, но стоимость их может различаться многократно. Поэтому планировщик должен изучить их все и найти самый эффективный способ выполнения запроса.

Когда запрос включает только две или три таблицы, возможны всего несколько вариантов их соединения. Но их число растёт экспоненциально с увеличением числа задействованных таблиц. Если число таблиц больше десяти, уже практически невозможно выполнить полный перебор всех вариантов, и даже для шести или семи таблиц планирование может занять недопустимо много времени. Когда таблиц слишком много, планировщик Postgres Pro переключается с полного поиска на алгоритм *генетического* вероятностного поиска в ограниченном числе вариантов. (Порог

для этого переключения задаётся параметром выполнения [geqo_threshold](#).) Генетический поиск выполняется быстрее, но не гарантирует, что найденный план будет наилучшим.

Когда запрос включает внешние соединения, планировщик имеет меньше степеней свободы, чем с обычными (внутренними) соединениями. Например, рассмотрим запрос:

```
SELECT * FROM a LEFT JOIN (b JOIN c ON (b.ref = c.id)) ON (a.id = b.id);
```

Хотя ограничения в этом запросе очень похожи на показанные в предыдущем примере, смысл его отличается, так как результирующая строка должна выдаваться для каждой строки А, даже если для неё не находится соответствия в соединении В и С. Таким образом, здесь планировщик не может выбирать порядок соединения: он должен соединить В с С, а затем соединить А с результатом. Соответственно, и план этого запроса построится быстрее, чем предыдущего. В других случаях планировщик сможет определить, что можно безопасно выбрать один из нескольких способов соединения. Например, для запроса:

```
SELECT * FROM a LEFT JOIN b ON (a.bid = b.id) LEFT JOIN c ON (a.cid = c.id);
```

можно соединить А либо с В, либо с С. В настоящее время только `FULL JOIN` полностью ограничивает порядок соединения. На практике в большинстве запросов с `LEFT JOIN` и `RIGHT JOIN` порядком можно управлять в некоторой степени.

Синтаксис явного внутреннего соединения (`INNER JOIN`, `CROSS JOIN` или лаконичный `JOIN`) по смыслу равнозначен перечислению отношений в предложении `FROM`, так что он никак не ограничивает порядок соединений.

Хотя большинство видов `JOIN` не полностью ограничивают порядок соединения, в Postgres Pro можно принудить планировщик обрабатывать все предложения `JOIN` как ограничивающие этот порядок. Например, следующие три запроса логически равнозначны:

```
SELECT * FROM a, b, c WHERE a.id = b.id AND b.ref = c.id;
SELECT * FROM a CROSS JOIN b CROSS JOIN c WHERE a.id = b.id AND b.ref = c.id;
SELECT * FROM a JOIN (b JOIN c ON (b.ref = c.id)) ON (a.id = b.id);
```

Но если мы укажем планировщику соблюдать порядок `JOIN`, на планирование второго и третьего уйдёт меньше времени. Когда речь идёт только о трёх таблицах, выигрыш будет незначительным, но для множества таблиц это может быть очень эффективно.

Чтобы планировщик соблюдал порядок внутреннего соединения, выраженный явно предложениями `JOIN`, нужно присвоить параметру выполнения [join_collapse_limit](#) значение 1. (Другие допустимые значения обсуждаются ниже.)

Чтобы сократить время поиска, необязательно полностью ограничивать порядок соединений, в `JOIN` можно соединять элементы как в обычном списке `FROM`. Например, рассмотрите следующий запрос:

```
SELECT * FROM a CROSS JOIN b, c, d, e WHERE ...;
```

Если `join_collapse_limit = 1`, планировщик будет вынужден соединить А с В раньше, чем результат с другими таблицами, но в дальнейшем выборе вариантов он не ограничен. В данном примере число возможных вариантов соединения уменьшается в 5 раз.

Упрощать для планировщика задачу перебора вариантов таким способом — это полезный приём, помогающий не только выбрать сократить время планирования, но и подтолкнуть планировщик к хорошему плану. Если планировщик по умолчанию выбирает неудачный порядок соединения, вы можете заставить его выбрать лучший, применив синтаксис `JOIN`, конечно если вы сами его знаете. Эффект подобной оптимизации рекомендуется подтверждать экспериментально.

На время планирования влияет и другой, тесно связанный фактор — решение о включении подзапросов в родительский запрос. Пример такого запроса:

```
SELECT *
FROM x, y,
    (SELECT * FROM a, b, c WHERE something) AS ss
```

```
WHERE somethingelse;
```

Такая же ситуация может возникнуть с представлением, содержащим соединение; вместо ссылки на это представление будет вставлено его выражение `SELECT` и в результате получится запрос, похожий на показанный выше. Обычно планировщик старается включить подзапрос в родительский запрос и получить таким образом:

```
SELECT * FROM x, y, a, b, c WHERE something AND somethingelse;
```

Часто это позволяет построить лучший план, чем при планировании подзапросов по отдельности. (Например, внешние условия `WHERE` могут быть таковы, что при соединении сначала `X` с `A` будет исключено множество строк `A`, а значит формировать логический результат подзапроса полностью не потребуется.) Но в то же время тем самым мы увеличиваем время планирования; две задачи соединения трёх элементов мы заменяем одной с пятью элементами. Так как число вариантов увеличивается экспоненциально, сложность задачи увеличивается многократно. Планировщик пытается избежать проблем поиска с огромным числом вариантов, рассматривая подзапросы отдельно, если в предложении `FROM` родительского запроса оказывается больше чем `from_collapse_limit` элементов. Изменяя этот параметр выполнения, можно подобрать оптимальное соотношение времени планирования и качества плана.

Параметры `from_collapse_limit` и `join_collapse_limit` называются похоже, потому что они делают практически одно и то же: первый параметр определяет, когда планировщик будет «сносить» в предложение `FROM` подзапросы, а второй — явные соединения. Обычно `join_collapse_limit` устанавливается равным `from_collapse_limit` (чтобы явные соединения и подзапросы обрабатывались одинаково) или 1 (если требуется управлять порядком соединений). Но вы можете задать другие значения, чтобы добиться оптимального соотношения времени планирования и времени выполнения запросов.

14.4. Наполнение базы данных

Довольно часто в начале или в процессе использования базы данных возникает необходимость загрузить в неё большой объём данных. В этом разделе приведены рекомендации, которые помогут сделать это максимально эффективно.

14.4.1. Отключите автофиксацию транзакций

Выполняя серию команд `INSERT`, выключите автофиксацию транзакций и зафиксируйте транзакцию только один раз в самом конце. (В обычном SQL это означает, что нужно выполнить `BEGIN` до, и `COMMIT` после этой серии. Некоторые клиентские библиотеки могут делать это автоматически, в таких случаях нужно убедиться, что это так.) Если вы будете фиксировать каждое добавление по отдельности, Postgres Pro придётся проделать много действий для каждой добавляемой строки. Выполнять все операции в одной транзакции хорошо ещё и потому, что в случае ошибки добавления одной из строк произойдёт откат к исходному состоянию и вы не окажетесь в сложной ситуации с частично загруженными данными.

14.4.2. Используйте `COPY`

Используйте `COPY`, чтобы загрузить все строки одной командой вместо серии `INSERT`. Команда `COPY` оптимизирована для загрузки большого количества строк; хотя она не так гибка, как `INSERT`, но при загрузке больших объёмов данных она влечёт гораздо меньше накладных расходов. Так как `COPY` — это одна команда, применяя её, нет необходимости отключать автофиксацию транзакций.

В случаях, когда `COPY` не подходит, может быть полезно создать подготовленный оператор `INSERT` с помощью `PREPARE`, а затем выполнять `EXECUTE` столько раз, сколько потребуется. Это позволит избежать накладных расходов, связанных с разбором и анализом каждой команды `INSERT`. В разных интерфейсах это может выглядеть по-разному; за подробностями обратитесь к описанию «подготовленных операторов» в документации конкретного интерфейса.

Заметьте, что с помощью `COPY` большое количество строк практически всегда загружается быстрее, чем с помощью `INSERT`, даже если используется `PREPARE` и серия операций добавления заключена в одну транзакцию.

`COPY` работает быстрее всего, если она выполняется в одной транзакции с командами `CREATE TABLE` или `TRUNCATE`. В таких случаях записывать WAL не нужно, так как в случае ошибки файлы, содержащие загружаемые данные, будут всё равно удалены. Однако это замечание справедливо, только когда параметр `wal_level` равен `minimal`, так как в противном случае все команды должны записывать свои изменения в WAL.

14.4.3. Удалите индексы

Если вы загружаете данные в только что созданную таблицу, быстрее всего будет загрузить данные с помощью `COPY`, а затем создать все необходимые для неё индексы. На создание индекса для уже существующих данных уйдёт меньше времени, чем на последовательное его обновление при добавлении каждой строки.

Если вы добавляете данные в существующую таблицу, может иметь смысл удалить индексы, загрузить таблицу, а затем пересоздать индексы. Конечно, при этом надо учитывать, что временное отсутствие индексов может отрицательно повлиять на скорость работы других пользователей. Кроме того, следует дважды подумать, прежде чем удалять уникальные индексы, так как без них соответствующие проверки ключей не будут выполняться.

14.4.4. Удалите ограничения внешних ключей

Как и с индексами, проверки, связанные с ограничениями внешних ключей, выгоднее выполнять «массово», а не для каждой строки в отдельности. Поэтому может быть полезно удалить ограничения внешних ключей, загрузить данные, а затем восстановить прежние ограничения. И в этом случае тоже приходится выбирать между скоростью загрузки данных и риском допустить ошибки в отсутствие ограничений.

Более того, когда вы загружаете данные в таблицу с существующими ограничениями внешнего ключа, для каждой новой строки добавляется запись в очередь событий триггера (так как именно срабатывающий триггер проверяет такие ограничения для строки). При загрузке многих миллионов строк очередь событий триггера может занять всю доступную память, что приведёт к недопустимой нагрузке на файл подкачки или даже к сбою команды. Таким образом, загружая большие объёмы данных, может быть не просто желательно, а *необходимо* удалять, а затем восстанавливать внешние ключи. Если же временное отключение этого ограничения неприемлемо, единственным возможным решением может быть разделение всей операции загрузки на меньшие транзакции.

14.4.5. Увеличьте `maintenance_work_mem`

Ускорить загрузку больших объёмов данных можно, увеличив параметр конфигурации `maintenance_work_mem` на время загрузки. Это приведёт к увеличению быстродействия `CREATE INDEX` и `ALTER TABLE ADD FOREIGN KEY`. На скорость самой команды `COPY` это не повлияет, так что этот совет будет полезен, только если вы применяете какой-либо из двух вышеописанных приёмов.

14.4.6. Увеличьте `max_wal_size`

Также массовую загрузку данных можно ускорить, изменив на время загрузки параметр конфигурации `max_wal_size`. Загружая большие объёмы данных, PostgreSQL Pro вынужден увеличивать частоту контрольных точек по сравнению с обычной (которая задаётся параметром `checkpoint_timeout`), а значит и чаще сбрасывать «грязные» страницы на диск. Временно увеличив `max_wal_size`, можно уменьшить частоту контрольных точек и связанных с ними операций ввода-вывода.

14.4.7. Отключите архивацию WAL и потоковую репликацию

Для загрузки больших объёмов данных в среде, где используется архивация WAL или потоковая репликация, быстрее будет сделать копию базы данных после загрузки данных, чем обрабатывать множество операций изменений в WAL. Чтобы отключить передачу изменений через WAL в процессе загрузки, отключите архивацию и потоковую репликацию, назначьте параметру `wal_level` значение `minimal`, `archive_mode` — `off`, а `max_wal_senders` — `0`. Но имейте в виду, что изменённые параметры вступят в силу только после перезапуска сервера, а все ранее созданные резервные

копии не будут пригодны для резервного сервера и восстановления архива, что может привести к потере данных.

Это не только поможет сэкономить время архивации и передачи WAL, но и непосредственно ускорит некоторые команды, потому что они не записывают в WAL ничего, если в `wal_level` установлен уровень `minimal` и текущая подтранзакция (или транзакция верхнего уровня) создала и опустошила таблицу или индекс, куда затем вносятся изменения. (Они могут гарантировать безопасность данных при сбое, не записывая их в WAL, а только выполнив `fsync` в конце, что будет гораздо дешевле.)

14.4.8. Выполните в конце ANALYZE

Всякий раз, когда распределение данных в таблице значительно меняется, настоятельно рекомендуется выполнять `ANALYZE`. Эта рекомендация касается и загрузки в таблицу большого объёма данных. Выполнив `ANALYZE` (или `VACUUM ANALYZE`), вы тем самым обновите статистику по данной таблице для планировщика. Когда планировщик не имеет статистики или она не соответствует действительности, он не сможет правильно планировать запросы, что приведёт к снижению быстродействия при работе с соответствующими таблицами. Заметьте, что если включён демон автоочистки, он может запускать `ANALYZE` автоматически; подробнее об этом можно узнать в [Подразделе 23.1.3](#) и [Подразделе 23.1.6](#).

14.4.9. Несколько замечаний относительно pg_dump

В скриптах загрузки данных, которые генерирует `pg_dump`, автоматически учитываются некоторые, но не все из этих рекомендаций. Чтобы загрузить данные, которые выгрузил `pg_dump`, максимально быстро, вам нужно будет выполнить некоторые дополнительные действия вручную. (Заметьте, что эти замечания относятся только к *восстановлению* данных, но не к *выгрузке* их. Следующие рекомендации применимы вне зависимости от того, загружается ли архивный файл `pg_dump` в `psql` или в `pg_restore`.)

По умолчанию `pg_dump` использует команду `COPY` и когда она выгружает полностью схему и данные, в сгенерированном скрипте она сначала предусмотрительно загружает данные, а потом создаёт индексы и внешние ключи. Так что в этом случае часть рекомендаций выполняется автоматически. Вам остаётся учесть только следующие:

- Установите подходящие (то есть превышающие обычные) значения для `maintenance_work_mem` и `max_wal_size`.
- Если вы используете архивацию WAL или потоковую репликацию, по возможности отключите их на время восстановления. Для этого перед загрузкой данных, присвойте параметру `archive_mode` значение `off`, `wal_level` — `minimal`, а `max_wal_senders` — `0`. Закончив восстановление, верните их обычные значения и сделайте свежую базовую резервную копию.
- Поэкспериментируйте с режимами параллельного копирования и восстановления команд `pg_dump` и `pg_restore`, и выберите оптимальное число параллельных заданий. Параллельное копирование и восстановление данных, управляемое параметром `-j`, должно дать значительный выигрыш в скорости по сравнению с последовательным режимом.
- Если это возможно в вашей ситуации, восстановите все данные в рамках одной транзакции. Для этого передайте параметр `-1` или `--single-transaction` команде `psql` или `pg_restore`. Но учтите, что в этом режиме даже незначительная ошибка приведёт к откату всех изменений и часы восстановления будут потрачены зря. В зависимости от того, насколько взаимосвязаны данные, предпочтительнее может быть вычистить их вручную. Команды `COPY` будут работать максимально быстро, когда они выполняются в одной транзакции и архивация WAL выключена.
- Если на сервере баз данных установлено несколько процессоров, полезным может оказаться параметр `--jobs` команды `pg_restore`. С его помощью можно выполнить загрузку данных и создание индексов параллельно.
- После загрузки данных запустите `ANALYZE`.

При выгрузке данных без схемы тоже используется команда `COPY`, но индексы, как обычно и внешние ключи, при этом не удаляются и не пересоздаются.¹ Поэтому, загружая только данные, вы сами должны решить, нужно ли для ускорения загрузки удалять и пересоздавать индексы и внешние ключи. При этом будет так же полезно увеличить параметр `max_wal_size`, но не `maintenance_work_mem`; его стоит менять, только если вы впоследствии пересоздаёте индексы и внешние ключи вручную. И не забудьте выполнить `ANALYZE` после; подробнее об этом можно узнать в [Подразделе 23.1.3](#) и [Подразделе 23.1.6](#).

14.5. Оптимизация, угрожающая стабильности

Стабильность — это свойство базы данных, гарантирующее, что результат зафиксированных транзакций будет сохранён даже в случае сбоя сервера или отключения питания. Однако обеспечивается стабильность за счёт значительной дополнительной нагрузки. Поэтому, если вы можете отказаться от такой гарантии, Postgres Pro можно ускорить ещё больше, применив следующие методы оптимизации. Кроме явно описанных исключений, даже с такими изменениями конфигурации при сбое программного ядра СУБД гарантия стабильности сохраняется; риск потери или разрушения данных возможен только в случае внезапного сбоя операционной системы.

- Поместите каталог данных кластера БД в файловую систему, размещённую в памяти (т. е. в RAM-диск). Так вы исключите всю активность ввода-вывода, связанную с базой данных, если только размер базы данных не превышает объём свободной памяти (возможно, с учётом файла подкачки).
- Выключите `fsync`; сбрасывать данные на диск не нужно.
- Выключите `synchronous_commit`; нет необходимости принудительно записывать WAL на диск при фиксации каждой транзакции. Но учтите, это может привести к потере транзакций (хотя данные останутся согласованными) в случае сбоя *базы данных*.
- Выключите `full_page_writes`; защита от частичной записи страниц не нужна.
- Увеличьте `max_wal_size` и `checkpoint_timeout`; это уменьшит частоту контрольных точек, хотя объём `pg_wal` при этом вырастет.
- Создавайте [нежурналируемые таблицы](#) для оптимизации записи в WAL (но учтите, что такие таблицы не защищены от сбоя).

¹Вы можете отключить внешние ключи, используя параметр `--disable-triggers` — но при этом нужно понимать, что тем самым вы не просто отложите, а полностью выключите соответствующие проверки, что позволит вставить недопустимые данные.

Глава 15. Параллельный запрос

Postgres Pro может вырабатывать такие планы запросов, которые будут задействовать несколько CPU, чтобы получить ответ на запросы быстрее. Эта возможность называется распараллеливанием запросов. Для многих запросов параллельное выполнение не даёт никакого выигрыша, либо из-за ограничений текущей реализации, либо из-за принципиальной невозможности построить параллельный план, который был бы быстрее последовательного. Однако для запросов, в которых это может быть полезно, распараллеливание часто даёт очень значительное ускорение. Многие такие запросы могут выполняться в параллельном режиме как минимум вдвое быстрее, а некоторые — быстрее в четыре и даже более раз. Обычно наибольший выигрыш можно получить с запросами, обрабатывающими большой объём данных, но возвращающими пользователю всего несколько строк. В этой главе достаточно подробно рассказывается, как работают параллельные запросы и в каких ситуациях их можно использовать, чтобы пользователи, желающие применять их, понимали, чего ожидать.

15.1. Как работают параллельно выполняемые запросы

Когда оптимизатор определяет, что параллельное выполнение будет наилучшей стратегией для конкретного запроса, он создаёт план запроса, включающий узел *Gather* (Сбор) или *Gather Merge* (Сбор со слиянием). Взгляните на простой пример:

```
EXPLAIN SELECT * FROM pgbench_accounts WHERE filler LIKE '%x%';
                                QUERY PLAN
-----
Gather  (cost=1000.00..217018.43 rows=1 width=97)
  Workers Planned: 2
    -> Parallel Seq Scan on pgbench_accounts  (cost=0.00..216018.33 rows=1 width=97)
        Filter: (filler ~~ '%x% '::text)
(4 rows)
```

Во всех случаях узел *Gather* или *Gather Merge* будет иметь ровно один дочерний план, представляющий часть общего плана, выполняемую в параллельном режиме. Если узел *Gather* или *Gather Merge* располагается на самом вершху дерева плана, в параллельном режиме будет выполняться весь запрос. Если он находится где-то в другом месте плана, параллельно будет выполняться только часть плана ниже него. В приведённом выше примере запрос обращается только к одной таблице, так что помимо узла *Gather* есть только ещё один узел плана; и так как этот узел является потомком узла *Gather*, он будет выполняться в параллельном режиме.

Используя **EXPLAIN**, вы можете узнать количество исполнителей, выбранное планировщиком для данного запроса. Когда при выполнении запроса достигается узел *Gather*, процесс, обслуживающий сеанс пользователя, запрашивает **фоновые рабочие процессы** в этом количестве. Количество исполнителей, которое может попытаться задействовать планировщик, ограничивается значением **max_parallel_workers_per_gather**. Общее число фоновых рабочих процессов, которые могут существовать одновременно, ограничивается параметрами **max_worker_processes** и **max_parallel_workers**. Таким образом, вполне возможно, что параллельный запрос будет выполняться меньшим числом рабочих процессов, чем планировалось, либо вообще без дополнительных рабочих процессов. Оптимальность плана может зависеть от числа доступных рабочих процессов, так что их нехватка может повлечь значительное снижение производительности. Если это наблюдается часто, имеет смысл увеличить **max_worker_processes** и **max_parallel_workers**, чтобы одновременно могло работать больше процессов, либо наоборот уменьшить **max_parallel_workers_per_gather**, чтобы планировщик запрашивал их в меньшем количестве.

Каждый фоновый рабочий процесс, успешно запущенный для данного параллельного запроса, будет выполнять параллельную часть плана. Ведущий процесс также будет выполнять эту часть плана, но он несёт дополнительную ответственность: он должен также прочитать все кортежи, выданные рабочими процессами. Когда параллельная часть плана выдаёт лишь небольшое количество

кортежей, ведущий часто ведёт себя просто как один из рабочих процессов, ускоряя выполнение запроса. И напротив, когда параллельная часть плана выдаёт множество кортежей, ведущий может быть почти всё время занят чтением кортежей, выдаваемых другими рабочими процессами, и выполнять другие шаги обработки, связанные с узлами плана выше узла `Gather` или `Gather Merge`. В таких случаях ведущий процесс может вносить лишь минимальный вклад в выполнение параллельной части плана.

Когда над параллельной частью плана оказывается узел `Gather Merge`, а не `Gather`, это означает, что все процессы, выполняющие части параллельного плана, выдают кортежи в отсортированном порядке, и что ведущий процесс выполняет слияние с сохранением порядка. Узел же `Gather`, напротив, получает кортежи от подчинённых процессов в произвольном удобном ему порядке, нарушая порядок сортировки, который мог существовать.

15.2. Когда может применяться распараллеливание запросов?

Планировщик запросов может отказаться от построения параллельных планов запросов в любом случае под влиянием нескольких параметров. Чтобы он строил параллельные планы запросов при каких-бы то ни было условиях, описанные далее параметры необходимо настроить указанным образом.

- `max_parallel_workers_per_gather` должен иметь значение, большее нуля. Это особый вариант более общего ограничения на суммарное число используемых рабочих процессов, задаваемого параметром `max_parallel_workers_per_gather`.

В дополнение к этому, система должна работать не в однопользовательском режиме. Так как в этом режиме вся СУБД работает как один процесс, фоновые рабочие процессы в нём недоступны.

Даже если принципиально возможно построить параллельные планы выполнения, планировщик не будет строить такой план для определённого запроса, если имеет место одно из следующих обстоятельств:

- Запрос выполняет запись данных или блокирует строки в базе данных. Если запрос содержит операцию, изменяющую данные либо на верхнем уровне, либо внутри СТЕ, для такого запроса не будут строиться параллельные планы. Исключение составляют следующие команды, которые создают новую таблицу, наполняют её и при этом могут использовать параллельный план для команды `SELECT`, выполняемой внутри запроса:
 - `CREATE TABLE ... AS`
 - `SELECT INTO`
 - `CREATE MATERIALIZED VIEW`
 - `REFRESH MATERIALIZED VIEW`
- Запрос может быть приостановлен в процессе выполнения. В ситуациях, когда система решает, что может иметь место частичное или дополнительное выполнение, план параллельного выполнения не строится. Например, курсор, созданный предложением `DECLARE CURSOR`, никогда не будет использовать параллельный план. Подобным образом, цикл PL/pgSQL вида `FOR x IN query LOOP .. END LOOP` никогда не будет использовать параллельный план, так как система параллельных запросов не сможет определить, может ли безопасно выполняться код внутри цикла во время параллельного выполнения запроса.
- В запросе используются функции, помеченные как `PARALLEL UNSAFE`. Большинство системных функций безопасны для параллельного выполнения (`PARALLEL SAFE`), но пользовательские функции по умолчанию помечаются как небезопасные (`PARALLEL UNSAFE`). Эта характеристика функции рассматривается в [Разделе 15.4](#).
- Запрос работает внутри другого запроса, уже параллельного. Например, если функция, вызываемая в параллельном запросе, сама выполняет SQL-запрос, последний запрос никогда не

будет выполняться параллельно. Это ограничение текущей реализации, но убирать его вряд ли следует, так как это может привести к использованию одним запросом чрезмерного количества процессов.

Даже когда для определённого запроса построен параллельный план, при некоторых условиях этот план нельзя будет выполнить в параллельном режиме. В этих случаях ведущий процесс выполнит часть плана ниже узла `Gather` самостоятельно, как если бы узла `Gather` вовсе не было. Это произойдёт только при выполнении одного из следующих условий:

- Невозможно получить ни одного фонового рабочего процесса из-за ограничения общего числа этих процессов значением [max_worker_processes](#).
- Невозможно получить ни одного фонового рабочего процесса из-за ограничения общего числа таких процессов для параллельного выполнения значением [max_parallel_workers](#).
- Клиент передаёт сообщение `Execute` с ненулевым количеством выбираемых кортежей. За подробностями обратитесь к описанию [расширенного протокола запросов](#). Так как `libpq` в настоящее время не позволяет передавать такие сообщения, это возможно только с клиентом, задействующим не `libpq`. Если это происходит часто, имеет смысл установить в [max_parallel_workers_per_gather](#) 0 в сеансах, для которых это актуально, чтобы система не пыталась строить планы, которые могут быть неэффективны при последовательном выполнении.

15.3. Параллельные планы

Так как каждый рабочий процесс выполняет параллельную часть плана до конца, нельзя просто взять обычный план запроса и запустить его в нескольких исполнителях. В этом случае все исполнители выдавали бы полные копии выходного набора результатов, так что запрос выполнится не быстрее, чем обычно, а его результаты могут быть некорректными. Вместо этого параллельной частью плана должно быть то, что для оптимизатора представляется как *частичный план*; то есть такой план, при выполнении которого в отдельном процессе будет получено только подмножество выходных строк, а каждая требующаяся строка результата будет гарантированно выдана ровно одним из сотрудничающих процессов. Вообще говоря, это означает, что сканирование нижележащей таблицы запроса должно проводиться с учётом распараллеливания.

15.3.1. Параллельные сканирования

В настоящее время поддерживаются следующие виды сканирований таблицы, рассчитанные на параллельное выполнение.

- При *параллельном последовательном сканировании* блоки таблицы будут разделены на диапазоны, распределяемые между взаимодействующими процессами. Каждый рабочий процесс сначала завершает сканирование полученного диапазона блоков, а затем запрашивает следующий диапазон.
- При *параллельном сканировании кучи по битовой карте* один процесс выбирается на роль ведущего. Этот процесс производит сканирование одного или нескольких индексов и строит битовую карту, показывающую, какие блоки таблицы нужно посетить. Затем эти блоки разделяются между взаимодействующими процессами как при параллельном последовательном сканировании. Другими словами, сканирование кучи выполняется в параллельном режиме, а сканирование нижележащего индекса — нет.
- При *параллельном сканировании по индексу или параллельном сканировании только индекса* взаимодействующие процессы читают данные из индекса по очереди. В настоящее время параллельное сканирование индекса поддерживается только для индексов В-деревьев. Каждый процесс будет выбирать один блок индекса с тем, чтобы просканировать и вернуть все кортежи, на которые он ссылается; другие процессы могут в то же время возвращать кортежи для другого блока индекса. Результаты параллельного сканирования В-дерева каждый рабочий процесс возвращает в отсортированном порядке.

В будущем может появиться поддержка параллельного выполнения и для других вариантов сканирования, например, сканирования индексов, отличных от В-дерева.

15.3.2. Параллельные соединения

Как и в непараллельном плане, целевая таблица может соединяться с одной или несколькими другими таблицами с использованием вложенных циклов, соединения по хешу или соединения слиянием. Внутренней стороной соединения может быть любой вид непараллельного плана, который в остальном поддерживается планировщиком, при условии, что он безопасен для выполнения в параллельном исполнителе. Внутренней стороной может быть и параллельный план, в зависимости от типа соединения.

- В *соединении с вложенным циклом* внутренняя сторона всегда непараллельная. Хотя она выполняется полностью, это эффективно, если с внутренней стороны производится сканирование индекса, так как внешние кортежи, а значит и циклы, находящие значения в индексе, разделяются по параллельным процессам.
- При *соединении слиянием* с внутренней стороны всегда будет непараллельный план и, таким образом, он будет выполняться полностью. Это может быть неэффективно, особенно если потребуется произвести сортировку, так как работа и конечные данные будут повторяться в каждом параллельном процессе.
- При *соединении по хешу* (непараллельном, без префикса «parallel») внутреннее соединение выполняется полностью в каждом параллельном процессе, и в результате они строят одинаковые копии хеш-таблицы. Это может быть неэффективно при большой хеш-таблице или дорогом плане. В *параллельном соединении по хешу* с внутренней стороны выполняется *параллельное хеширование*, при котором работа по построению общей хеш-таблицы разделяется между параллельными процессами.

15.3.3. Параллельное агрегирование

Postgres Pro поддерживает параллельное агрегирование, выполняя агрегирование в два этапа. Сначала каждый процесс, задействованный в параллельной части запроса, выполняет шаг агрегирования, выдавая частичный результат для каждой известной ему группы. В плане это отражает узел `Partial Aggregate`. Затем эти промежуточные результаты передаются ведущему через узел `Gather` или `Gather Merge`. И наконец, ведущий заново агрегирует результаты всех рабочих процессов, чтобы получить окончательный результат. Это отражает в плане узел `Finalize Aggregate`.

Так как узел `Finalize Aggregate` выполняется в ведущем процессе, запросы, выдающие достаточно большое количество групп по отношению к числу входных строк, будут расцениваться планировщиком как менее предпочтительные. Например, в худшем случае количество групп, выявленных узлом `Finalize Aggregate`, может равняться числу входных строк, обработанных всеми рабочими процессами на этапе `Partial Aggregate`. Очевидно, что в такой ситуации использование параллельного агрегирования не даст никакого выигрыша производительности. Планировщик запросов учитывает это в процессе планирования, так что выбор параллельного агрегирования в подобных случаях очень маловероятен.

Параллельное агрегирование поддерживается не во всех случаях. Чтобы оно поддерживалось, агрегатная функция должна быть **безопасной** для распараллеливания и должна иметь комбинирующую функцию. Если переходное состояние агрегатной функции имеет тип `internal`, она должна также иметь функции сериализации и десериализации. За подробностями обратитесь к [CREATE AGGREGATE](#). Параллельное агрегирование не поддерживается, если вызов агрегатной функции содержит предложение `DISTINCT` или `ORDER BY`. Также оно не поддерживается для сортирующих агрегатов или когда запрос включает предложение `GROUPING SETS`. Оно может использоваться только когда все соединения, задействованные в запросе, также входят в параллельную часть плана.

15.3.4. Параллельное присоединение

Когда требуется объединить строки из различных источников в единый набор результатов, в Postgres Pro используются узлы плана `Append` или `MergeAppend`. Это обычно происходит при реализации `UNION ALL` или при сканировании секционированной таблицы. Данные узлы могут применяться как в параллельных, так и в обычных планах. Однако в параллельных планах планировщик может заменить их на узел `Parallel Append`.

Если в параллельном плане используется узел `Append`, все задействованные процессы выполняют очередной дочерний план совместно, пока он не будет завершён, и лишь затем, примерно в одно время, переходят к выполнению следующего дочернего плана. Когда же применяется `Parallel Append`, исполнитель старается равномерно распределить между задействованными процессами все дочерние планы, чтобы они выполнялись параллельно. Это позволяет избежать конкуренции и не тратить ресурсы на запуск дочернего плана для тех процессов, которые не будут его выполнять.

Кроме того, в отличие от обычного узла `Append`, использование которого внутри параллельного плана допускается только для частичных дочерних планов, узел `Parallel Append` может обрабатывать как частичные, так и не частичные дочерние планы. Для сканирования не частичного плана будет использоваться только один процесс, поскольку его многократное сканирование приведёт лишь к дублированию результатов. Таким образом, для планов, объединяющих несколько наборов результатов, можно достичь параллельного выполнения на высоком уровне, даже когда эффективные частичные планы отсутствуют. Например, рассмотрим запрос к секционированной таблице, который может быть эффективно реализован только с помощью индекса, не поддерживающего параллельное сканирование. Планировщик может выбрать узел `Parallel Append` для параллельного объединения нескольких обычных планов `Index Scan`; в этом случае каждое сканирование индекса будет выполняться до полного завершения одним процессом, но при этом разные сканирования будут осуществляться параллельно.

Отключить данную функциональность можно с помощью [enable_parallel_append](#).

15.3.5. Советы по параллельным планам

Если для запроса ожидается параллельный план, но такой план не строится, можно попытаться уменьшить [parallel_setup_cost](#) или [parallel_tuple_cost](#). Разумеется, этот план может оказаться медленнее последовательного плана, предпочитаемого планировщиком, но не всегда. Если вы не получаете параллельный план даже с очень маленькими значениями этих параметров (например, сбросив оба их в ноль), может быть какая-то веская причина тому, что планировщик запросов не может построить параллельный план для вашего запроса. За информацией о возможных причинах обратитесь к [Разделу 15.2](#) и [Разделу 15.4](#).

Когда выполняется параллельный план, вы можете применить `EXPLAIN (ANALYZE, VERBOSE)`, чтобы просмотреть статистику по каждому узлу плана в разрезе рабочих процессов. Это может помочь определить, равномерно ли распределяется работа между всеми узлами плана, и на более общем уровне понимать характеристики производительности плана.

15.4. Безопасность распараллеливания

Планировщик классифицирует операции, вовлечённые в выполнение запроса, как либо *безопасные для распараллеливания*, либо *ограниченно распараллеливаемые*, либо *небезопасные для распараллеливания*. Безопасной для распараллеливания операцией считается такая, которая не мешает параллельному выполнению запроса. Ограниченно распараллеливаемой операцией считается такая, которая не может выполняться в параллельном рабочем процессе, но может выполняться в ведущем процессе, когда запрос выполняется параллельно. Таким образом, ограниченно параллельные операции никогда не могут оказаться ниже узла `Gather` или `Gather Merge`, но могут встречаться в других местах плана, содержащего такой узел. Небезопасные для распараллеливания операции не могут выполняться в параллельных запросах, даже в ведущем процессе. Когда запрос содержит что-либо небезопасное для распараллеливания, параллельное выполнение для такого запроса полностью исключается.

Ограниченно распараллеливаемыми всегда считаются следующие операции:

- Сканирование общих табличных выражений (СТЕ).
- Сканирование временных таблиц.
- Сканирование сторонних таблиц, если только обёртка сторонних данных не предоставляет функцию `IsForeignScanParallelSafe`, которая допускает распараллеливание.
- Узлы плана, к которым присоединён узел `InitPlan`.

- Узлы плана, которые ссылаются на связанный SubPlan.

15.4.1. Пометки параллельности для функций и агрегатов

Планировщик не может автоматически определить, является ли пользовательская обычная или агрегатная функция безопасной для распараллеливания, так как это потребовало бы предсказания действия каждой операции, которую могла бы выполнять функция. В общем случае это равнозначно решению проблемы остановки, а значит, невозможно. Даже для простых функций, где это в принципе возможно, мы не пытаемся это делать, так как это будет слишком дорогой и потенциально неточной процедурой. Вместо этого, все определяемые пользователем функции полагаются небезопасными для распараллеливания, если явно не отмечено обратное. Когда используется `CREATE FUNCTION` или `ALTER FUNCTION`, функции можно назначить отметку `PARALLEL SAFE`, `PARALLEL RESTRICTED` или `PARALLEL UNSAFE`, отражающую её характер. В команде `CREATE AGGREGATE` для параметра `PARALLEL` можно задать `SAFE`, `RESTRICTED` или `UNSAFE` в виде соответствующего значения.

Обычные и агрегатные функции должны помечаться небезопасными для распараллеливания (`PARALLEL UNSAFE`), если они пишут в базу данных, обращаются к последовательностям, изменяют состояние транзакции, даже временно (как, например, функция PL/pgSQL, устанавливающая блок `EXCEPTION` для перехвата ошибок), либо производят постоянные изменения параметров. Подобным образом, функции должны помечаться как ограниченно распараллеливаемые (`PARALLEL RESTRICTED`), если они обращаются к временным таблицам, состоянию клиентского подключения, курсорам, подготовленным операторам или разнообразному локальному состоянию обслуживающего процесса, которое система не может синхронизировать между рабочими процессами. Например, по этой причине ограниченно параллельными являются функции `setseed` и `random`.

В целом, если функция помечена как безопасная, когда на самом деле она небезопасна или ограниченно безопасна, или если она помечена как ограниченно безопасная, когда на самом деле она небезопасная, такая функция может выдавать ошибки или возвращать неправильные ответы при использовании в параллельном запросе. Функции на языке C могут теоретически проявлять полностью неопределённое появление при некорректной метке, так как система никаким образом не может защитить себя от произвольного кода C, но чаще всего результат будет не хуже, чем с любой другой функцией. В случае сомнений, вероятно, лучше всего будет помечать функции как небезопасные (`UNSAFE`).

Если функция, выполняемая в параллельном рабочем процессе, затребует блокировки, которыми не владеет ведущий, например, обращаясь к таблице, не упомянутой в запросе, эти блокировки будут освобождены по завершении процесса, а не в конце транзакции. Если вы разрабатываете функцию с таким поведением, и эта особенность выполнения оказывается критичной, пометьте такую функцию как `PARALLEL RESTRICTED`, чтобы она выполнялась только в ведущем процессе.

Заметьте, что планировщик запросов не рассматривает возможность отложенного выполнения ограниченно распараллеливаемых обычных или агрегатных функций, задействованных в запросе, для получения лучшего плана. Поэтому, например, если предложение `WHERE`, применяемое к конкретной таблице, является ограниченно параллельным, планировщик запросов исключит возможность сканирования этой таблицы в параллельной части плана. В некоторых случаях возможно (и, вероятно, более эффективно) включить сканирование этой таблицы в параллельную часть запроса и отложить вычисление предложения `WHERE`, чтобы оно происходило над узлом `Gather`, но планировщик этого не делает.